
C/C++ cheatsheet Documentation

Release 0.1.0

crazyguitar

Sep 02, 2026

CONTENTS

1	Plugin	3
---	--------	---

C and C++ continue to evolve with new standards that bring modern features while maintaining the performance and control that make these languages essential for systems programming. This cheatsheet focuses on collecting useful code snippets and practical examples, covering everything from language fundamentals to advanced topics like coroutines, CUDA programming, and lock-free data structures. Examples are organized by topic with source code available in the repository for building and experimentation.

Whether you're refreshing your memory on pointer arithmetic, exploring modern C++23 features, debugging a tricky memory issue with Valgrind, or figuring out how to set up a CMake project, you'll find practical examples here. The goal is simple: provide clear, working code that you can understand and adapt for your own projects. Contributions and suggestions are always welcome on [GitHub](#).

PLUGIN

cppcheatsheet is available as a [Claude Code](#) plugin. Once installed, Claude automatically uses the cheat sheets to answer C/C++ questions.

```
# Step 1: Add the marketplace
claude plugin marketplace add crazyguitar/cppcheatsheet

# Step 2: Install the plugin
claude plugin install cppcheatsheet@cppcheatsheet
```

For local testing and manual installation, see the main [README](#).

1.1 C/C++ Interview Cheatsheet

This page is a curated, question-indexed map into the rest of the cheatsheet. Each entry below is a question you are likely to see in a C or C++ interview, followed by a link that jumps directly to the section of the notes that answers it. It is intentionally a navigation layer — the actual explanations, code, and caveats live in the linked sections.

Use it two ways:

- **Drilling a topic:** pick a group (e.g. *Templates & Generics*) and walk every question in it.
- **Quick review before an interview:** read the questions, and for any you cannot confidently answer in one or two sentences, click through.

1.1.1 Memory & Resource Management

- What is RAII, and why is it the cornerstone of modern C++ resource management? → [cpp/cpp_raii: RAII Wrapper](#)
- How does RAII interact with exception safety (basic / strong / nothrow guarantees)? → [cpp/cpp_raii: Exception Safety Guarantees](#)
- What happens when a constructor throws after partial resource acquisition? → [cpp/cpp_raii: Constructor Failure and Exception Safety](#)
- Compare `std::lock_guard`, `std::unique_lock`, and `std::scoped_lock`. → [cpp/cpp_raii: Lock-Based RAII](#)
- When does `std::unique_ptr` make sense vs. `std::shared_ptr`? → [cpp/cpp_smartpointers: std::unique_ptr: Exclusive Ownership](#)
- What problem does `std::weak_ptr` solve? → [cpp/cpp_smartpointers: std::weak_ptr: Non-Ownning Observer](#)
- Why prefer `std::make_unique` / `std::make_shared` over raw `new`? → [cpp/cpp_smartpointers: std::make_unique and std::make_shared](#)

- What are common smart-pointer pitfalls (e.g. cycles, double ownership)? → *cpp/cpp_smartpointers: Common Pitfalls*
- Why does memory alignment matter, and how is struct padding computed? → *c/c_memory: Structure Alignment and Padding*

1.1.2 Move Semantics & Value Categories

- What are the special member functions, and when are they implicitly defined or deleted? → *cpp/cpp_move: Special Member Functions*
- Explain the Rule of Zero, Three, and Five. → *cpp/cpp_move: Rule of Zero · Three · Five*
- What are lvalues, rvalues, xvalues, prvalues? → *cpp/cpp_move: Value Categories*
- How does `std::forward` achieve perfect forwarding, and why is it needed? → *cpp/cpp_move: Perfect Forwarding*
- What state is a moved-from object left in, and what can you safely do with it? → *cpp/cpp_move: Moved-From State*
- Why should move constructors typically be `noexcept`? → *cpp/cpp_move: Conditional noexcept*
- `emplace_back` vs. `push_back` — when is there a real difference? → *cpp/cpp_move: Emplace vs Insert*

1.1.3 Templates & Generics

- How does template specialization differ from overloading? → *cpp/cpp_template: Template Specialization*
- What are variadic templates and fold expressions? → *cpp/cpp_template: Variadic Templates and Parameter Packs · Fold Expressions*
- What is SFINAE, and how do C++20 concepts replace most of it? → *cpp/cpp_template: SFINAE and Type Constraints · Defining Concepts*
- What is CRTP and when would you use it? → *cpp/cpp_template: CRTP*
- How does a `requires` clause constrain a template, and how does it compose with `&&/ ||`? → *cpp/cpp_requires: Basic Requires Clause · Conjunction · Disjunction*
- `requires` vs. `if constexpr` — when to reach for which? → *cpp/cpp_requires: Requires vs if constexpr*
- What are reference collapsing rules, and why do they matter for forwarding references? → *cpp/cpp_basic: Reference Collapsing Rules*
- How does template type deduction differ between by-value, by-ref, and forwarding parameters? → *cpp/cpp_basic: Template Type Deduction*

1.1.4 Casting & Type Conversions

- Why prefer `static_cast`, `dynamic_cast`, `const_cast`, and `reinterpret_cast` over C-style casts? → *cpp/cpp_casting: C-Style Cast (Legacy Casting)*
- When is `dynamic_cast` safe, and when does it return null or throw? → *cpp/cpp_casting: dynamic_cast*
- What does `const_cast` actually guarantee, and when is it undefined? → *cpp/cpp_casting: const_cast*

1.1.5 Polymorphism & Inheritance

- How does dynamic dispatch work — what are the vtable and vptr? → *cpp/cpp_polymorphism: The vtable and vptr*

- When does a base class need a virtual destructor, and what happens without one? → *cpp/cpp_polymorphism: Virtual Destructors*
- Why should overriding functions be marked `override`, and what does `final` enable? → *cpp/cpp_polymorphism: override and final*
- What happens if you call a virtual function from a constructor or destructor? → *cpp/cpp_polymorphism: Construction and Destruction Order*
- What is object slicing, and how do you avoid it? → *cpp/cpp_polymorphism: The Slicing Problem*
- What is the diamond problem, and how does virtual inheritance solve it? → *cpp/cpp_polymorphism: The Diamond Problem and Virtual Inheritance*
- When should you reach for virtual functions vs. CRTP? → *cpp/cpp_polymorphism: Virtual vs. CRTP — When to Pick Which*
- What are the three flavors of polymorphism in modern C++, and how does trait-style (type-erased) polymorphism differ from inheritance-based dispatch? → *cpp/cpp_type_erasure: Why Type Erasure*
- Walk through the Concept/Model pattern used to build a type-erased wrapper like `std::function`. → *cpp/cpp_type_erasure: The Concept / Model Pattern*
- What is the small buffer optimization, and why does `std::function` use it? → *cpp/cpp_type_erasure: Small Buffer Optimization (SBO)*
- Why can `std::function` not store a lambda that captures `std::unique_ptr`, and what fixes it in C++23? → *cpp/cpp_type_erasure: std::move_only_function (C++23)*
- What is the cost of a virtual call, and when can the compiler devirtualize it? → *cpp/cpp_polymorphism: Static vs. Dynamic Dispatch*

1.1.6 Compile-Time Programming

- `const` vs. `constexpr` vs. `constexpr` vs. `constinit` — pick the right one. → *cpp/cpp_constexpr: constexpr Functions · constexpr · constinit*
- How does `if constexpr` eliminate branches at compile time? → *cpp/cpp_constexpr: constexpr if (C++17)*
- How has `constexpr` expanded across C++14, 17, 20, and 23? → *cpp/cpp_constexpr: constexpr Evolution by Standard*

1.1.7 STL Containers & Iterators

- Compare `std::vector`, `std::deque`, and `std::list` — memory layout, iterator invalidation, complexity. → *cpp/cpp_container: std::vector · std::deque · std::list*
- `std::map` vs. `std::unordered_map` — trade-offs and when to use each. → *cpp/cpp_container: Ordered vs Unordered Containers*
- What are the iterator invalidation rules for the common containers? → *cpp/cpp_iterator: Iterator Invalidation*
- What are iterator categories, and why do algorithms care? → *cpp/cpp_iterator: Iterator Categories*
- Walk through `std::sort`, `std::stable_sort`, `std::partial_sort`, and `std::nth_element` — when would you pick each? → *cpp/cpp_algorithm: Sorting*
- Difference between `std::find` and `std::binary_search`? → *cpp/cpp_algorithm: Searching*
- What are C++20 ranges and views? Are views lazy? → *cpp/cpp_iterator: C++20 Ranges Overview*

1.1.8 Strings

- `std::string` vs. `std::string_view` — when is `string_view` dangerous? → *cpp/cpp_string: std::string_view*
- Why is naïve `+=` concatenation in a loop a performance trap? → *cpp/cpp_string: String Concatenation Performance*

1.1.9 Modern C++ Features

- What does each lambda capture mode mean: `[]`, `[=]`, `[&]`, `[this]`, init captures? → *cpp/cpp_lambda: Lambda Syntax Overview · Init Capture*
- Why is a captureless lambda convertible to a function pointer? → *cpp/cpp_lambda: Captureless Lambdas and Function Pointers*
- What is a generic / template lambda? → *cpp/cpp_lambda: Generic Lambdas (C++14) · Template Lambdas*
- What are C++20 coroutines, and what does `co_await` actually do? → *cpp/cpp_coroutine: Coroutine Basics · Promise Type · Awaiter*
- Why would you write a generator with coroutines instead of a class? → *cpp/cpp_coroutine: Generator*
- What are C++20 modules, and what problem do they solve? → *cpp/cpp_modules: Module Basics*
- `auto` vs. `decltype` vs. `decltype(auto)`? → *cpp/cpp_basic: Auto Type Deduction · decltype(auto)*
- What does uniform / brace initialization change about narrowing conversions? → *cpp/cpp_basic: Uniform Initialization*

1.1.10 C Language Essentials

- When does an array *not* decay to a pointer, and what are the signed/unsigned gotchas around pointer arithmetic? → *cpp/cpp_basic: Pointer Arithmetic and Negative Indices*
- What does `restrict` promise to the compiler? → *c/c_basic: Restrict Qualifier*
- Designated initializers and compound literals — how are they different? → *c/c_basic: Designated Initializers · Compound Literals*
- What are flexible array members? → *c/c_basic: Flexible Array Members*
- Why is `_Generic` useful, and how is it different from C++ overloading? → *c/c_basic: _Generic Type Selection (C11)*
- What new things did C23 bring (`nullptr`, `constexpr`, `typeof`, attributes, bit-precise ints)? → *c/c_basic: typeof and auto (C23) · nullptr · constexpr*
- Why are macros dangerous, and what safer alternatives exist? → *c/c_macro: Variadic Macros*

1.1.11 Concurrency & OS

- How do you create threads in modern C++? → *os/os_thread: Creating Threads*
- Mutex, condition variable, and read-write lock — when is each appropriate? → *os/os_thread: Mutex Synchronization · Condition Variables · Read-Write Locks*
- `std::future` / `std::promise` / `std::async` — what are the launch policies? → *os/os_thread: C++ Futures and Async · Launch Policies*
- What is thread-local storage? → *os/os_thread: Thread-Local Storage*
- Process vs. thread — what is shared and what is not? → *os/os_process: Fork and Wait · Fork and Exec*

1.1.12 Debugging & Tools

- How do you find memory leaks and invalid accesses with Valgrind? → [debug/valgrind: Memcheck: Memory Errors · Leak Types](#)
- What do AddressSanitizer, ThreadSanitizer, and UBSan each catch? → [debug/sanitizers: AddressSanitizer \(ASan\) · ThreadSanitizer · UndefinedBehaviorSanitizer](#)
- How do you inspect a core dump with gdb after a segfault? → [debug/gdb: Debugging Crashes](#)
- What are breakpoints vs. watchpoints in gdb? → [debug/gdb: Breakpoints · Watchpoints](#)
- Minimal modern CMake project — what are the must-have commands? → [cpp/cpp_cmake: Minimal Project · Modern C++ Standard Setting](#)

1.1.13 See Also

If a question above is not covered, the top-level indices are the best next stop:

- [Modern C Programming](#) — C language reference
- [Modern C++ Programming](#) — Modern C++ reference
- [System Programming](#) — OS and systems programming
- [Debugging & Profiling](#) — Debugging and profiling tools

1.2 Modern C Programming

Despite decades of evolution in programming languages, C remains the lingua franca of systems software, embedded development, and performance-critical applications. Its direct access to memory and hardware, combined with portability across computing platforms, makes it indispensable for operating systems, compilers, and firmware development.

Here you will find coverage of language syntax and semantics, preprocessor directives and macro techniques for code generation, GNU compiler extensions that enhance expressiveness, build automation with Make for managing complex compilation workflows, and x86 assembly language fundamentals for low-level optimization and hardware interaction.

1.2.1 C Basics

Table of Contents

- [C Basics](#)
 - [Introduction](#)
 - [Comma Operator](#)
 - [Static Variables for State Persistence](#)
 - [Machine Endianness Detection](#)
 - [String Splitting](#)
 - [Callback Functions](#)
 - [Duff's Device](#)
 - [Compound Literals](#)
 - [Designated Initializers](#)

- *Flexible Array Members*
- *Variable-Length Arrays (VLA)*
- *Restrict Qualifier*
- *_Static_assert (C11)*
- *_Generic Type Selection (C11)*
- *Anonymous Structs and Unions (C11)*
- *Atomics (C11)*
- *_Alignas and _Alignof (C11)*
- *_Noreturn Functions (C11)*
- *typeof and auto (C23)*
- *nullptr (C23)*
- *constexpr (C23)*
- *Attributes (C23)*
- *Bit-Precise Integers (C23)*

Introduction

C continues to evolve with new standards (C11, C17, C23) that add modern features while maintaining backward compatibility. C11 introduced atomics, threads, and type-generic expressions. C23 adds `typeof`, `auto` type inference, `nullptr`, and improved `constexpr`. This reference covers both classic C patterns and modern features essential for systems programming, embedded development, and high-performance applications.

Comma Operator

Source

[src/c/comma-operator](#)

The comma operator evaluates its left operand, discards the result, then evaluates and returns its right operand. This binary operator has the lowest precedence of all C operators, which often leads to confusion when combined with assignment. Understanding the distinction between comma as an operator versus comma as a separator (in function arguments or variable declarations) is crucial for reading and writing correct C code.

```
#include <stdio.h>

int main(void) {
    int a = 1, b = 2, c = 3; // comma as separator
    int i;

    i = (a, b, c);          // comma operator: evaluates a, b, returns c
    printf("i = (a, b, c) => %d\n", i); // prints 3

    i = (a + 5, a + b);      // evaluates a+5 (discarded), returns a+b
    printf("i = (a + 5, a + b) => %d\n", i); // prints 3

    i = a + 5, a + b;       // equivalent to (i = a + 5), a + b
```

(continues on next page)

(continued from previous page)

```
printf("i = a + 5, a + b => %d\n", i); // prints 6
}
```

```
$ ./comma-operator
i = (a, b, c) => 3
i = (a + 5, a + b) => 3
i = a + 5, a + b => 6
```

Static Variables for State Persistence

Source

[src/c/static-closure](#)

Static local variables retain their values between function calls, providing a simple mechanism for maintaining state without global variables. Unlike automatic (local) variables that are created and destroyed with each function invocation, static variables are initialized once and persist for the program's lifetime. This pattern is useful for counters, caches, and implementing closure-like behavior in C.

```
#include <stdio.h>

void counter(void) {
    static int count = 0; // initialized once, persists across calls
    int local = 0;        // reinitialized each call

    count++;
    local++;
    printf("static count = %d, local = %d\n", count, local);
}

int main(void) {
    for (int i = 0; i < 5; i++) {
        counter();
    }
}
```

```
$ ./static-closure
static count = 1, local = 1
static count = 2, local = 1
static count = 3, local = 1
static count = 4, local = 1
static count = 5, local = 1
```

Machine Endianness Detection

Source

[src/c/endianness-check](#)

Endianness determines how multi-byte values are stored in memory. Little-endian systems store the least significant byte at the lowest address (x86, ARM), while big-endian systems store the most significant byte first (network byte order, some RISC processors). Detecting endianness at runtime is essential when working with binary file formats, network protocols, or cross-platform data exchange.

```
#include <stdio.h>
#include <stdint.h>

int is_little_endian(void) {
    uint16_t val = 0x0001;
    return *(uint8_t *)&val == 0x01;
}

int main(void) {
    if (is_little_endian()) {
        printf("Little Endian Machine\n");
    } else {
        printf("Big Endian Machine\n");
    }
}
```

```
$ ./endian-check
Little Endian Machine
```

String Splitting

Source

[src/c/split-string](#)

Splitting strings by a delimiter is a common operation that C handles through `strtok()`, which modifies the original string by inserting null terminators. For reentrant code or when the original string must be preserved, use `strtok_r()` or implement custom parsing. This example demonstrates dynamic memory allocation for storing the resulting tokens.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char **split(char *str, char delim) {
    int count = 1;
    for (char *p = str; *p; p++) {
        if (*p == delim) count++;
    }

    char **result = calloc(count + 1, sizeof(char *));
    char delims[2] = {delim, '\0'};
    char *token = strtok(str, delims);
    int i = 0;

    while (token) {
        result[i++] = strdup(token);
        token = strtok(NULL, delims);
    }
    return result;
}

int main(void) {
    char str[] = "hello,world,test";
    char **tokens = split(str, ',');
```

(continues on next page)

(continued from previous page)

```

for (char **p = tokens; *p; p++) {
    printf("%s\n", *p);
    free(*p);
}
free(tokens);
}

```

```

$ ./split-string
hello
world
test

```

Callback Functions

Source

[src/c/callback](#)

Callbacks enable flexible, decoupled code by passing function pointers as arguments. The called function invokes the callback at appropriate points, allowing the caller to customize behavior without modifying the callee. This pattern is fundamental to event-driven programming, asynchronous operations, and plugin architectures. In C, callbacks are implemented using function pointers with matching signatures.

```

#include <stdio.h>

typedef void (*callback_fn)(int result);

void on_complete(int result) {
    if (result == 0) {
        printf("Operation succeeded\n");
    } else {
        printf("Operation failed with code %d\n", result);
    }
}

void do_work(int should_fail, callback_fn cb) {
    int result = should_fail ? -1 : 0;
    cb(result);
}

int main(void) {
    do_work(0, on_complete); // success
    do_work(1, on_complete); // failure
}

```

```

$ ./callback
Operation succeeded
Operation failed with code -1

```

Duff's Device

Source

src/c/duffs-device

Duff's device is a loop unrolling technique that exploits C's switch statement fall-through behavior. By interleaving a switch with a do-while loop, it handles both the unrolled iterations and the remainder in a single construct. While modern compilers often perform better automatic optimizations, Duff's device remains an interesting example of C's flexibility and is occasionally useful in embedded systems or when compiler optimizations are limited.

```
#include <stdio.h>

void copy_bytes(char *to, const char *from, int count) {
    int n = (count + 7) / 8;
    switch (count % 8) {
        case 0: do { *to++ = *from++;
        case 7:      *to++ = *from++;
        case 6:      *to++ = *from++;
        case 5:      *to++ = *from++;
        case 4:      *to++ = *from++;
        case 3:      *to++ = *from++;
        case 2:      *to++ = *from++;
        case 1:      *to++ = *from++;
        } while (--n > 0);
    }
}

int main(void) {
    char src[] = "Hello, Duff!";
    char dst[20] = {0};
    copy_bytes(dst, src, sizeof(src));
    printf("Copied: %s\n", dst);
}
```

```
$ ./duffs-device
Copied: Hello, Duff!
```

Compound Literals

Compound literals create unnamed objects with automatic storage duration, useful for passing temporary arrays or structs to functions without declaring separate variables. The syntax (type){initializer} creates an object of the specified type. Compound literals are lvalues, so their address can be taken, but they only persist until the end of the enclosing block.

```
#include <stdio.h>

struct point { int x, y; };

void print_point(struct point *p) {
    printf("(%d, %d)\n", p->x, p->y);
}

int main(void) {
    // Pass temporary struct without declaring variable
```

(continues on next page)

(continued from previous page)

```

print_point(&(struct point){10, 20});

// Temporary array
int *arr = (int[]){1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
}

```

```

$ ./compound-literal
(10, 20)
1 2 3 4 5

```

Designated Initializers

Designated initializers allow initializing specific members of arrays or structs by name or index, leaving others zero-initialized. This improves code clarity and maintainability, especially for large structs where only a few fields need non-default values. Array designators use `[index] = value` syntax, while struct designators use `.member = value`.

```

#include <stdio.h>

struct config {
    int timeout;
    int retries;
    int verbose;
    char name[32];
};

int main(void) {
    // Initialize specific struct members
    struct config cfg = {
        .timeout = 30,
        .name = "default",
        // retries and verbose are zero-initialized
    };

    // Initialize specific array indices
    int arr[10] = {
        [0] = 100,
        [5] = 500,
        [9] = 900,
    };

    printf("timeout=%d, retries=%d\n", cfg.timeout, cfg.retries);
    printf("arr[0]=%d, arr[5]=%d, arr[9]=%d\n", arr[0], arr[5], arr[9]);
}

```

```

$ ./designated-init
timeout=30, retries=0
arr[0]=100, arr[5]=500, arr[9]=900

```

Flexible Array Members

Flexible array members (C99) allow structs to have a variable-length array as their last member. The struct is allocated with extra space for the array elements. This pattern is common in network protocols and file formats where a header contains a count followed by variable data. The flexible array member has no size in the struct definition and must be allocated dynamically.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct message {
    int length;
    char data[]; // flexible array member
};

struct message *create_message(const char *text) {
    int len = strlen(text);
    struct message *msg = malloc(sizeof(struct message) + len + 1);
    msg->length = len;
    strcpy(msg->data, text);
    return msg;
}

int main(void) {
    struct message *msg = create_message("Hello, flexible array!");
    printf("Length: %d, Data: %s\n", msg->length, msg->data);
    free(msg);
}
```

```
$ ./flexible-array
Length: 22, Data: Hello, flexible array!
```

Variable-Length Arrays (VLA)

Variable-length arrays (C99) allow array sizes to be determined at runtime. Unlike dynamically allocated arrays, VLAs have automatic storage duration and are allocated on the stack. While convenient, VLAs can cause stack overflow for large sizes and are optional in C11. Use them cautiously for small, bounded sizes; prefer `malloc()` for large or unbounded allocations.

```
#include <stdio.h>

void print_matrix(int rows, int cols, int matrix[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%3d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main(void) {
    int n = 3;
    int vla[n][n]; // VLA with runtime size
}
```

(continues on next page)

(continued from previous page)

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        vla[i][j] = i * n + j;
    }
}

print_matrix(n, n, vla);
}

```

```

$ ./vla
0  1  2
3  4  5
6  7  8

```

Restrict Qualifier

The `restrict` qualifier (C99) tells the compiler that a pointer is the only way to access the pointed-to object, enabling aggressive optimizations. Without `restrict`, the compiler must assume pointers might alias (point to overlapping memory), preventing certain optimizations. Use `restrict` when you can guarantee no aliasing, particularly in performance-critical functions like `memcpy()`.

```

#include <stdio.h>

// Without restrict, compiler assumes a and b might overlap
void add_arrays(int *a, int *b, int *result, int n) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
}

// With restrict, compiler can optimize more aggressively
void add_arrays_fast(int *restrict a, int *restrict b,
                    int *restrict result, int n) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
}

int main(void) {
    int a[] = {1, 2, 3};
    int b[] = {4, 5, 6};
    int result[3];

    add_arrays_fast(a, b, result, 3);
    printf("%d %d %d\n", result[0], result[1], result[2]);
}

```

```

$ ./restrict
5 7 9

```

`_Static_assert` (C11)

Source

`src/c/static-assert`

Static assertions verify conditions at compile time, catching errors before runtime. Unlike `assert()`, `_Static_assert` has zero runtime cost and fails during compilation if the condition is false. Use it to validate type sizes, struct layouts, and configuration assumptions. C23 allows omitting the message string.

```
#include <stdio.h>
#include <stdint.h>

_Static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
_Static_assert(sizeof(void *) == 8, "64-bit pointers required");

struct packet {
    uint32_t header;
    uint32_t payload;
};
_Static_assert(sizeof(struct packet) == 8, "packet must be 8 bytes");

int main(void) {
    printf("All static assertions passed\n");
}
```

```
$ gcc -std=c11 -o static-assert main.c && ./static-assert
All static assertions passed
```

`_Generic` Type Selection (C11)

Source

`src/c/generic`

`_Generic` enables type-generic macros by selecting expressions based on the type of the controlling expression. This provides a form of function overloading in C, commonly used for type-safe math functions and print formatters. The selection happens at compile time with no runtime overhead.

```
#include <stdio.h>
#include <math.h>

#define abs_val(x) _Generic((x), \
    int: abs, \
    long: labs, \
    float: fabsf, \
    double: fabs \
)(x)

#define type_name(x) _Generic((x), \
    int: "int", \
    float: "float", \
    double: "double", \
    char *: "char *", \
    default: "unknown" \
)
```

(continues on next page)

(continued from previous page)

```
int main(void) {
    printf("abs(-5) = %d\n", abs_val(-5));
    printf("abs(-3.14) = %f\n", abs_val(-3.14));

    int i; float f; char *s;
    printf("i is %s, f is %s, s is %s\n",
          type_name(i), type_name(f), type_name(s));
}
```

```
$ gcc -std=c11 -o generic main.c -lm && ./generic
abs(-5) = 5
abs(-3.14) = 3.140000
i is int, f is float, s is char *
```

Anonymous Structs and Unions (C11)

Source

[src/c/anon-struct](#)

Anonymous structs and unions allow nested members to be accessed directly without an intermediate name. This simplifies code when wrapping related fields or creating variant types. The inner struct/union members become direct members of the enclosing struct.

```
#include <stdio.h>

struct vector3 {
    union {
        struct { float x, y, z; };    // anonymous struct
        float components[3];         // array access
    };                                // anonymous union
};

int main(void) {
    struct vector3 v = { .x = 1.0f, .y = 2.0f, .z = 3.0f };

    // Access via named members
    printf("x=%.1f, y=%.1f, z=%.1f\n", v.x, v.y, v.z);

    // Access via array
    for (int i = 0; i < 3; i++) {
        printf("components[%d] = %.1f\n", i, v.components[i]);
    }
}
```

```
$ gcc -std=c11 -o anon-struct main.c && ./anon-struct
x=1.0, y=2.0, z=3.0
components[0] = 1.0
components[1] = 2.0
components[2] = 3.0
```

Atomics (C11)

Source

src/c/atomics

The `<stdatomic.h>` header provides atomic types and operations for lock-free concurrent programming. Atomic operations are indivisible—no other thread can observe a partial update. Use atomics for simple shared counters and flags; for complex data structures, prefer mutexes.

```

#include <stdio.h>
#include <stdatomic.h>
#include <pthread.h>

atomic_int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        atomic_fetch_add(&counter, 1);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter: %d\n", atomic_load(&counter)); // always 200000
}

```

```

$ gcc -std=c11 -pthread -o atomics main.c && ./atomics
Counter: 200000

```

_Alignas and _Alignof (C11)

Source

src/c/alignas

`_Alignas` specifies alignment requirements for variables, useful for SIMD operations, cache optimization, and hardware interfaces. `_Alignof` queries the alignment requirement of a type. The `<stdalign.h>` header provides `alignas` and `alignof` as convenient aliases.

```

#include <stdio.h>
#include <stdalign.h>
#include <stdint.h>

struct aligned_data {
    alignas(64) char cache_line[64]; // aligned to cache line
    alignas(16) float simd_data[4]; // aligned for SIMD
};

int main(void) {
    printf("int alignment: %zu\n", alignof(int));
}

```

(continues on next page)

(continued from previous page)

```
printf("double alignment: %zu\n", alignof(double));
printf("cache_line alignment: %zu\n", alignof(struct aligned_data));

alignas(32) int aligned_int = 42;
printf("aligned_int address: %p\n", (void *)&aligned_int);
}
```

```
$ gcc -std=c11 -o alignas main.c && ./alignas
int alignment: 4
double alignment: 8
cache_line alignment: 64
aligned_int address: 0x7fff5fbff760
```

`_Noreturn` Functions (C11)

Source

`src/c/noreturn`

The `_Noreturn` specifier indicates a function never returns to its caller, either because it exits the program, loops forever, or always throws. This enables compiler optimizations and suppresses “missing return” warnings. C23 deprecates `_Noreturn` in favor of the `[[noreturn]]` attribute.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdnoreturn.h>

_noreturn void fatal_error(const char *msg) {
    fprintf(stderr, "Fatal: %s\n", msg);
    exit(1);
}

int main(void) {
    int *ptr = NULL;
    if (!ptr) {
        fatal_error("null pointer");
    }
    // Compiler knows this is unreachable
    return 0;
}
```

```
$ gcc -std=c11 -o noreturn main.c && ./noreturn
Fatal: null pointer
```

`typeof` and `auto` (C23)

Source

`src/c/typeof`

C23 adds `typeof` to deduce types from expressions and `auto` for type inference in variable declarations. These features reduce repetition and make code more maintainable, especially with complex types. `typeof` was previously a GNU extension; C23 standardizes it.

```
#include <stdio.h>

#define max(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})

int main(void) {
    // auto type inference (C23)
    auto x = 42;           // int
    auto pi = 3.14159;      // double
    auto msg = "hello";    // const char *

    printf("max(3, 7) = %d\n", max(3, 7));
    printf("max(3.5, 2.1) = %f\n", max(3.5, 2.1));

    // typeof for matching types
    int arr[] = {1, 2, 3, 4, 5};
    typeof(arr[0]) sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += arr[i];
    }
    printf("sum = %d\n", sum);
}
```

```
$ gcc -std=c2x -o typeof main.c && ./typeof
max(3, 7) = 7
max(3.5, 2.1) = 3.500000
sum = 15
```

nullptr (C23)

Source

`src/c/nullptr-test`

C23 introduces `nullptr` as a proper null pointer constant with its own type `nullptr_t`. Unlike `NULL` (which is typically `(void *)0` or `0`), `nullptr` is unambiguous and type-safe. It cannot be implicitly converted to integer types, preventing common bugs.

```
#include <stdio.h>
#include <stddef.h>

void process_int(int n) {
    printf("int: %d\n", n);
}

void process_ptr(int *p) {
    printf("ptr: %p\n", (void *)p);
}

int main(void) {
    int *p = nullptr; // clear intent: null pointer
```

(continues on next page)

(continued from previous page)

```

if (p == nullptr) {
    printf("p is null\n");
}

// nullptr prevents accidental integer conversion
// process_int(nullptr); // compile error in C23
process_ptr(nullptr);    // OK
}

```

```

$ gcc -std=c2x -o nullptr main.c && ./nullptr
p is null
ptr: (nil)

```

constexpr (C23)

Source

[src/c/constexpr-c](#)

C23 extends `constexpr` to allow compile-time constant expressions for variables. Unlike `const`, which merely prevents modification, `constexpr` guarantees the value is computed at compile time and can be used in contexts requiring constant expressions like array sizes.

```

#include <stdio.h>

constexpr int BUFFER_SIZE = 1024;
constexpr int DOUBLED = BUFFER_SIZE * 2;

int main(void) {
    constexpr int local_const = 42;

    // Can use constexpr values for array sizes
    char buffer[BUFFER_SIZE];
    int arr[local_const];

    printf("BUFFER_SIZE = %d\n", BUFFER_SIZE);
    printf("DOUBLED = %d\n", DOUBLED);
    printf("sizeof(buffer) = %zu\n", sizeof(buffer));
}

```

```

$ gcc -std=c2x -o constexpr main.c && ./constexpr
BUFFER_SIZE = 1024
DOUBLED = 2048
sizeof(buffer) = 1024

```

Attributes (C23)

Source

[src/c/attributes](#)

C23 standardizes attributes using the `[[attribute]]` syntax, replacing compiler-specific extensions. Common attributes include `[[nodiscard]]` for return values that shouldn't be ignored, `[[maybe_unused]]` to suppress warnings, and `[[deprecated]]` to mark obsolete APIs.

```
#include <stdio.h>
#include <stdlib.h>

[[nodiscard]] int *allocate(size_t n) {
    return malloc(n * sizeof(int));
}

[[deprecated("use new_function instead")]]
void old_function(void) {
    printf("old function\n");
}

void callback([[maybe_unused]] int unused_param) {
    printf("callback called\n");
}

int main(void) {
    int *p = allocate(10); // OK: result used
    // allocate(10);      // warning: nodiscard value ignored

    callback(42);
    free(p);
}
```

```
$ gcc -std=c2x -o attributes main.c && ./attributes
callback called
```

Bit-Precise Integers (C23)

C23 introduces `_BitInt(N)` for integers with exactly *N* bits, useful for cryptography, hardware interfaces, and memory-constrained systems. Unlike standard integer types whose sizes vary by platform, bit-precise integers have guaranteed widths.

```
#include <stdio.h>

int main(void) {
    _BitInt(24) rgb = 0xFF8040; // exactly 24 bits
    _BitInt(128) big = 0;       // 128-bit integer

    unsigned _BitInt(12) small = 4095; // 12-bit unsigned

    printf("rgb = 0x%X\n", (unsigned)rgb);
    printf("small max = %u\n", (unsigned)small);
}
```

```
$ gcc -std=c2x -o bitint main.c && ./bitint
rgb = 0xFF8040
small max = 4095
```

1.2.2 Memory

Table of Contents

- *Memory*
 - *Introduction*
 - *Why Memory Alignment Matters*
 - *Different Alignment Sizes*
 - *Align Up Technique*
 - *Structure Alignment and Padding*
 - *Benchmark*

Introduction

Memory alignment refers to placing data at memory addresses that are multiples of the data's size or a specified boundary. Modern CPUs are optimized for aligned memory access, and misaligned access can cause performance penalties or even hardware exceptions on some architectures.

Why Memory Alignment Matters

CPUs access memory in fixed-size chunks (typically 4 or 8 bytes). When data crosses these boundaries, the CPU must perform multiple memory operations:

1. **Hardware efficiency:** CPUs fetch data in cache-line-sized blocks (typically 64 bytes). Aligned data ensures single-fetch access.
2. **Atomic operations:** Many atomic instructions require aligned operands. Misaligned atomics may not be atomic at all.
3. **SIMD instructions:** Vector operations (SSE, AVX) require or strongly prefer aligned data (16, 32, or 64-byte boundaries).
4. **Cache utilization:** Aligned structures pack efficiently into cache lines, reducing cache misses and false sharing in multithreaded code.

Different Alignment Sizes

Source

`src/c/memory-alignment`

Alignment follows a hierarchical relationship: higher alignment guarantees all lower alignments, but not vice versa. An address aligned to 64 bytes is automatically aligned to 32, 16, 8, 4, 2, and 1 bytes because any number divisible by 64 is also divisible by all its factors. However, an 8-byte aligned address (like `0x18 = 24`) may not be 16-byte aligned since 24 is not divisible by 16.

- 64-byte aligned → also 32, 16, 8, 4, 2, 1-byte aligned
- 16-byte aligned → also 8, 4, 2, 1-byte aligned
- 8-byte aligned → NOT necessarily 16-byte aligned

```
// Address 0x40 (64): 64%64=0 ✓, 64%16=0 ✓, 64%8=0 ✓
// Address 0x18 (24): 24%16=8 , 24%8=0 ✓
```

Different alignment requirements serve different purposes:

- **8-byte alignment:** Natural alignment for 64-bit types such as `double` and pointers on 64-bit systems. This ensures the CPU can fetch the entire value in a single memory access without crossing a word boundary.
- **16-byte alignment:** Required for SSE (128-bit) SIMD operations on x86. This is also the standard stack alignment on x86-64 as specified by the System V ABI, ensuring function calls maintain proper alignment for vector operations.
- **32-byte alignment:** Required for AVX (256-bit) SIMD operations. AVX instructions operating on misaligned data may incur performance penalties or require separate load instructions.
- **64-byte alignment:** Matches the cache line size on most modern x86 and ARM processors. Aligning data structures to cache line boundaries prevents false sharing in multithreaded code, where different threads modifying adjacent data would otherwise invalidate each other's cache lines.
- **Page alignment (4096 bytes typical):** Required for memory-mapped I/O, `mmap()`, and DMA buffers. The operating system manages memory in pages, so operations like mapping files or allocating huge pages require page-aligned addresses. Query the system page size with `sysconf(_SC_PAGESIZE)`.

```
#include <stdalign.h>
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    // Natural alignment
    printf("alignof(char)    = %zu\n", alignof(char));    // 1
    printf("alignof(int)     = %zu\n", alignof(int));     // 4
    printf("alignof(double) = %zu\n", alignof(double));  // 8

    // System page size
    const size_t page_size = sysconf(_SC_PAGESIZE);
    printf("page size       = %zu\n", page_size);        // 4096 (typical)

    // Custom alignment
    alignas(16) int simd_ready = 42;
    alignas(64) char cache_line[64];

    printf("&simd_ready %% 16 = %zu\n",
           (uintptr_t)&simd_ready % 16); // 0
    printf("&cache_line %% 64 = %zu\n",
           (uintptr_t)&cache_line % 64); // 0
}
```

```
$ ./memory-alignment
alignof(char)    = 1
alignof(int)     = 4
alignof(double) = 8
page size       = 4096
&simd_ready % 16 = 0
&cache_line % 64 = 0
```

Align Up Technique

A common pattern is aligning a pointer or size up to the next alignment boundary. The bitwise formula $(ptr + align - 1) \& \sim(align - 1)$ rounds up to the nearest multiple of `align` (which must be a power of 2). This works because $\sim(align - 1)$ creates a mask that clears the lower bits, and adding `align - 1` first ensures we round up rather than down.

```
// Align pointer up to boundary (align must be power of 2)
constexpr void* align_up(void* ptr, size_t align) noexcept {
    return (void*)((uintptr_t)ptr + align - 1) & ~(align - 1);
}

// Align size up to boundary
constexpr size_t align_up(size_t size, size_t align) noexcept {
    return (size + align - 1) & ~(align - 1);
}

// Example usage for buffer allocation
void* raw = malloc(size + align - 1);    // allocate extra for alignment
void* aligned = align_up(raw, align);    // get aligned pointer within buffer

// Page-aligned allocation
const size_t page_size = sysconf(_SC_PAGESIZE);
const size_t alloc_size = align_up(size, page_size);

/*
Buffer Allocation with Alignment
=====

1. Allocate extra space:

    raw = malloc(size + align - 1)
    |
    v
+-----+
| extra padding (up to align-1) |      usable size      |
+-----+
^
raw (may be unaligned)

2. Align pointer up:

    aligned = align_up(raw, align)
    |
    v
+-----+
| wasted |      aligned buffer      |
+-----+
^
    aligned (guaranteed aligned to 'align' boundary)
*/
```

Structure Alignment and Padding

Source

src/c/struct-alignment

Compilers automatically insert padding bytes between struct members to ensure each member is naturally aligned. This is necessary because accessing misaligned data can be slower or cause hardware faults on some architectures. The struct's total size is also padded to a multiple of its largest member's alignment, ensuring arrays of structs maintain proper alignment.

```
#include <stddef>
#include <stdio>

struct Bad {
    char a;           // 1 byte
                       // 3 bytes padding (to align int)
    int b;            // 4 bytes
    char c;           // 1 byte
                       // 3 bytes padding (to align struct size to 4)
};                  // Total: 12 bytes

struct Good {
    int b;           // 4 bytes
    char a;          // 1 byte
    char c;          // 1 byte
                       // 2 bytes padding
};                  // Total: 8 bytes

struct Packed {
    char a;
    int b;
    char c;
} __attribute__((packed)); // Total: 6 bytes (no padding, may be slower)

int main() {
    printf("sizeof(Bad)    = %zu\n", sizeof(Bad));    // 12
    printf("sizeof(Good)   = %zu\n", sizeof(Good));    // 8
    printf("sizeof(Packed) = %zu\n", sizeof(Packed));  // 6

    printf("offsetof(Bad, a) = %zu\n", offsetof(Bad, a)); // 0
    printf("offsetof(Bad, b) = %zu\n", offsetof(Bad, b)); // 4
    printf("offsetof(Bad, c) = %zu\n", offsetof(Bad, c)); // 8
}
```

```
$ ./struct-alignment
sizeof(Bad)    = 12
sizeof(Good)   = 8
sizeof(Packed) = 6
offsetof(Bad, a) = 0
offsetof(Bad, b) = 4
offsetof(Bad, c) = 8
```

To minimize padding, order struct members from largest to smallest alignment. Use `__attribute__((packed))` only when space is critical and you accept the performance penalty of misaligned access.

Benchmark

Source

`src/c/memory-alignment`

This benchmark measures memory access throughput by summing 1M integers at various byte offsets from a 64-byte aligned base address. By starting the array at offsets 0, 1, 2, 4, 8, 16, and 32 bytes, we can observe how different alignment boundaries affect performance on modern hardware.

```
$ ./memory-alignment-bench
Memory Alignment Performance Benchmark
Array size: 1048576 integers
Iterations: 100
```

Offset	Time (ms)	Aligned to
0	0.45	64-byte
1	0.37	unaligned
2	0.36	unaligned
4	0.35	4-byte
8	0.34	8-byte
16	0.37	16-byte
32	0.40	32-byte

On modern CPUs (x86-64, Apple Silicon), the benchmark shows negligible difference between aligned and unaligned access for scalar code. The hardware handles misalignment transparently. However, alignment remains critical for SIMD operations, atomics, and portability to stricter architectures.

1.2.3 Preprocessor & GNU Extensions

Table of Contents

- *Preprocessor & GNU Extensions*
 - *Introduction*
 - *Predefined Macros*
 - *Conditional Compilation*
 - *Variadic Macros*
 - *Array Size Macro*
 - *Statement Expressions (GNU)*
 - *typeof Operator (GNU)*
 - *__attribute__ Specifiers (GNU)*
 - *Constructor and Destructor*
 - *Compiler Builtins*
 - *Overflow Checking Builtins*
 - *Binary Constants (GNU)*
 - *Nested Functions (GNU)*

- *Zero-Length Arrays (GNU)*
- *Case Ranges (GNU)*

Introduction

The C preprocessor transforms source code before compilation, enabling conditional compilation, macro expansion, and file inclusion. Beyond standard preprocessor features, GCC provides powerful extensions like statement expressions, typeof, and attribute specifiers that enable safer and more expressive code. While these extensions reduce portability, they're widely supported by Clang and essential for systems programming, especially in the Linux kernel.

Predefined Macros

Source

`src/c/predefined-macros`

The C preprocessor automatically defines several macros that provide compile-time information about the source file, compilation environment, and current location in the code. These predefined macros are invaluable for debugging, logging, and generating diagnostic messages that include file names, line numbers, and function names. They enable creating informative error messages without hardcoding location information, and support conditional compilation based on compiler features and standards compliance.

```
#include <stdio.h>

int main(void) {
    printf("File: %s\n", __FILE__);
    printf("Date: %s\n", __DATE__);
    printf("Time: %s\n", __TIME__);
    printf("Line: %d\n", __LINE__);
    printf("Func: %s\n", __func__);
}
```

```
$ ./predefined
File: main.c
Date: Jan 06 2026
Time: 10:30:00
Line: 7
Func: main
```

Common predefined macros:

<code>__FILE__</code>	Current source file name
<code>__LINE__</code>	Current line number
<code>__func__</code>	Current function name (C99)
<code>__DATE__</code>	Compilation date "MMM DD YYYY"
<code>__TIME__</code>	Compilation time "HH:MM:SS"
<code>__STDC__</code>	1 if compiler conforms to ISO C
<code>__GNUC__</code>	GCC major version number

Conditional Compilation

Conditional compilation directives enable including or excluding code based on compile-time conditions, making it possible to maintain a single codebase that adapts to different platforms, configurations, and build types. The `#ifdef` and `#if defined()` directives test whether macros are defined, while `#if` evaluates constant expressions. This mechanism is essential for platform-specific code, feature flags, debug builds, and header guards. The `-D` compiler flag defines macros from the command line, enabling build-time configuration without modifying source files.

```
#include <stdio.h>

int main(void) {
#ifdef DEBUG
    printf("Debug build\n");
#else
    printf("Release build\n");
#endif

#if defined(__linux__)
    printf("Linux\n");
#elif defined(__APPLE__)
    printf("macOS\n");
#endif
}
```

```
$ gcc -o test main.c && ./test
Release build
$ gcc -DDEBUG -o test main.c && ./test
Debug build
```

Variadic Macros

Source

`src/c/variadic-macros`

Variadic macros accept a variable number of arguments using the ellipsis (...) syntax, with arguments accessible through the special `__VA_ARGS__` identifier. This enables creating flexible logging, debugging, and wrapper macros that mirror the interface of functions like `printf`. The GNU extension `##__VA_ARGS__` provides a crucial enhancement: it removes the preceding comma when no variadic arguments are provided, allowing macros to work correctly with both zero and multiple optional arguments. This pattern is ubiquitous in logging frameworks and debug utilities.

```
#include <stdio.h>

#define LOG(fmt, ...) \
    printf("[LOG] " fmt "\n", ##__VA_ARGS__)

#define DEBUG_LOG(fmt, ...) \
    printf("[%s:%d] " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)

int main(void) {
    LOG("Starting");
    LOG("Value: %d", 42);
    DEBUG_LOG("x = %d, y = %d", 10, 20);
}
```

```
$ ./variadic
[LOG] Starting
[LOG] Value: 42
[main.c:12] x = 10, y = 20
```

Array Size Macro

Source

`src/c/arraysize`

Calculating array size at compile time is a fundamental C idiom that divides the total array size by the size of a single element. This technique only works for actual arrays with known size at compile time—it fails silently for pointers, which is a common source of bugs. The macro approach is safer than manually tracking array sizes and automatically updates when elements are added or removed. Modern C++ provides `std::size()` for this purpose, but in C, this macro pattern remains the standard solution.

```
#include <stdio.h>

#define ARRAY_SIZE(arr) (sizeof(arr) / sizeof((arr)[0]))

int main(void) {
    int nums[] = {1, 2, 3, 4, 5};
    char *strs[] = {"hello", "world", NULL};

    printf("nums size: %zu\n", ARRAY_SIZE(nums));
    printf("strs size: %zu\n", ARRAY_SIZE(strs));
}
```

```
$ ./arraysize
nums size: 5
strs size: 3
```

Statement Expressions (GNU)

Source

`src/c/stmt-expr`

Statement expressions are a GNU extension that allows compound statements (blocks) to appear within expressions, enclosed in `{ { }`. The value of the last statement becomes the value of the entire expression. This enables macros to declare local variables, execute multiple statements, and return a computed value—all while evaluating arguments only once. This solves the classic macro problem where `MAX(a++, b++)` would increment arguments multiple times. Statement expressions are heavily used in the Linux kernel and other systems code for creating safe, type-generic macros.

```
#include <stdio.h>

#define max(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})

#define swap(a, b) ({ \
```

(continues on next page)

(continued from previous page)

```

    typeof(a) _tmp = a; \
    a = b;              \
    b = _tmp;           \
})

int main(void) {
    int x = 10, y = 20;
    printf("max(%d, %d) = %d\n", x, y, max(x, y));

    swap(x, y);
    printf("after swap: x=%d, y=%d\n", x, y);
}

```

```

$ ./stmt-expr
max(10, 20) = 20
after swap: x=20, y=10

```

typeof Operator (GNU)

Source

src/c/typeof-gnu

The `typeof` operator is a GNU extension that returns the type of an expression at compile time, enabling the creation of type-safe generic macros. Unlike C++ templates, `typeof` works purely at the preprocessor/compiler level, allowing macros to automatically adapt to any data type passed to them. This eliminates the need for type-specific macro variants and prevents subtle bugs from type mismatches. C23 standardizes this feature, making it portable across compilers.

```

#include <stdio.h>

#define SWAP(a, b) do { \
    typeof(a) _tmp = (a); \
    (a) = (b);           \
    (b) = _tmp;          \
} while (0)

#define MIN(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a < _b ? _a : _b; \
})

int main(void) {
    int i = 5, j = 10;
    double x = 3.14, y = 2.71;

    SWAP(i, j);
    printf("i=%d, j=%d\n", i, j);
    printf("min(%.2f, %.2f) = %.2f\n", x, y, MIN(x, y));
}

```

```

$ ./typeof
i=10, j=5

```

(continues on next page)

(continued from previous page)

```
min(3.14, 2.71) = 2.71
```

__attribute__ Specifiers (GNU)

Source

src/c/attribute

GCC attributes provide compiler hints for optimization, diagnostics, and code generation through the `__attribute__` syntax. These annotations help the compiler generate better code, catch bugs at compile time, and control low-level details like struct layout and function calling conventions. Common use cases include `packed` for removing struct padding in network protocols, `unused` to suppress warnings for intentionally unused variables, `format` for printf-style argument checking, and `noreturn` for functions that never return. While non-portable, most attributes are supported by Clang and are essential for systems programming.

```
#include <stdio.h>
#include <stdint.h>

struct __attribute__((packed)) packet {
    uint8_t type;
    uint32_t data; // no padding
};

__attribute__((unused))
static void helper(void) { }

__attribute__((format(printf, 1, 2)))
void log_msg(const char *fmt, ...) {
    va_list args;
    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);
}

__attribute__((noreturn))
void die(const char *msg) {
    fprintf(stderr, "Fatal: %s\n", msg);
    exit(1);
}

int main(void) {
    printf("packed struct size: %zu\n", sizeof(struct packet));
}
```

```
$ ./attribute
packed struct size: 5
```

Common attributes:

<code>__attribute__((packed))</code>	Remove struct padding
<code>__attribute__((aligned(n)))</code>	Align to n bytes
<code>__attribute__((unused))</code>	Suppress unused warnings
<code>__attribute__((deprecated))</code>	Mark as deprecated

(continues on next page)

(continued from previous page)

<code>__attribute__((noreturn))</code>	Function never returns
<code>__attribute__((format(printf,m,n)))</code>	Check printf format
<code>__attribute__((constructor))</code>	Run before main()
<code>__attribute__((destructor))</code>	Run after main()

Constructor and Destructor

Source

`src/c/ctor-dtor`

Functions marked with `__attribute__((constructor))` execute automatically before `main()` begins, while `__attribute__((destructor))` functions run after `main()` returns or `exit()` is called. This mechanism enables automatic initialization and cleanup without explicit calls, similar to C++ static object constructors. Common use cases include initializing global state, registering signal handlers, setting up logging systems, and releasing resources. Multiple constructors/destructors can specify priority values to control execution order.

```
#include <stdio.h>

__attribute__((constructor))
void init(void) {
    printf("Initializing...\n");
}

__attribute__((destructor))
void cleanup(void) {
    printf("Cleaning up...\n");
}

int main(void) {
    printf("In main()\n");
    return 0;
}
```

```
$ ./ctor-dtor
Initializing...
In main()
Cleaning up...
```

Compiler Builtins

Source

`src/c/builtins`

GCC provides builtin functions that map directly to efficient machine instructions or provide compiler-specific functionality not available in standard C. Bit manipulation builtins like `__builtin_popcount` and `__builtin_clz` compile to single CPU instructions on modern processors, offering significant performance gains over manual implementations. Branch prediction hints via `__builtin_expect` help the compiler optimize hot paths, while `__builtin_unreachable` enables dead code elimination. These builtins are widely supported by Clang and are commonly used in performance-critical code and operating system kernels.

```
#include <stdio.h>

int main(void) {
```

(continues on next page)

(continued from previous page)

```

unsigned int x = 0b10110000;

printf("popcount(0x%x) = %d\n", x, __builtin_popcount(x));
printf("clz(0x%x) = %d\n", x, __builtin_clz(x));
printf("ctz(0x%x) = %d\n", x, __builtin_ctz(x));
printf("ffs(0x%x) = %d\n", x, __builtin_ffs(x));

// Branch prediction hints
int likely_true = 1;
if (__builtin_expect(likely_true, 1)) {
    printf("Expected path\n");
}
}

```

```

$ ./builtins
popcount(0xb0) = 4
clz(0xb0) = 24
ctz(0xb0) = 4
ffs(0xb0) = 5
Expected path

```

Common builtins:

<code>__builtin_popcount(x)</code>	Count set bits
<code>__builtin_clz(x)</code>	Count leading zeros
<code>__builtin_ctz(x)</code>	Count trailing zeros
<code>__builtin_ffs(x)</code>	Find first set bit (1-indexed)
<code>__builtin_expect(x, v)</code>	Branch prediction hint
<code>__builtin_unreachable()</code>	Mark unreachable code
<code>__builtin_prefetch(p)</code>	Prefetch memory

Overflow Checking Builtins

Source

[src/c/overflow-check](#)

Integer overflow in C is undefined behavior for signed types and wraps silently for unsigned types, making it a common source of security vulnerabilities. GCC's overflow checking builtins perform arithmetic operations and return a boolean indicating whether overflow occurred, enabling safe arithmetic without undefined behavior. These functions are essential for security-critical code handling untrusted input, memory allocation size calculations, and any arithmetic that could exceed type limits. The result is stored via pointer, allowing both the overflow check and the computed value to be used.

```

#include <stdio.h>
#include <stdint.h>

int main(void) {
    int a = INT32_MAX, b = 1, result;

    if (__builtin_add_overflow(a, b, &result)) {
        printf("Overflow detected!\n");
    } else {

```

(continues on next page)

(continued from previous page)

```

    printf("Result: %d\n", result);
}

// Safe multiplication
size_t count = 1000000, size = 10000, total;
if (__builtin_mul_overflow(count, size, &total)) {
    printf("Allocation would overflow\n");
}
}

```

```

$ ./overflow
Overflow detected!
Allocation would overflow

```

Binary Constants (GNU)

Source

`src/c/binary-const`

Binary literals with the `0b` or `0B` prefix allow expressing integer constants in base-2 notation, making bit patterns immediately readable without mental conversion from hexadecimal. This is particularly valuable when working with hardware registers, protocol flags, bitmasks, and any code where individual bit positions have specific meanings. Originally a GNU extension, binary constants are now standardized in C23, ensuring portability across modern compilers while maintaining backward compatibility with GCC and Clang.

```

#include <stdio.h>

#define BIT(n) (1U << (n))

int main(void) {
    int flags = 0b10110100;
    int mask  = 0b00001111;

    printf("flags = 0x%02x\n", flags);
    printf("flags & mask = 0x%02x\n", flags & mask);
    printf("BIT(3) = 0x%02x\n", BIT(3));
}

```

```

$ ./binary
flags = 0xb4
flags & mask = 0x04
BIT(3) = 0x08

```

Nested Functions (GNU)

Source

`src/c/nested-func`

GCC allows defining functions inside other functions, creating nested functions with access to the enclosing function's local variables through lexical scoping. This provides closure-like behavior in C, where the inner function captures and can modify variables from its enclosing scope. Nested functions are useful for creating helper functions that need access to local state without passing many parameters or using global variables. However, taking the address of a nested function requires executable stack trampolines, which may conflict with security hardening measures on some systems.

```
#include <stdio.h>

int main(void) {
    int multiplier = 10;

    int multiply(int x) {
        return x * multiplier; // accesses outer variable
    }

    printf("multiply(5) = %d\n", multiply(5));

    multiplier = 20;
    printf("multiply(5) = %d\n", multiply(5));
}
```

```
$ ./nested
multiply(5) = 50
multiply(5) = 100
```

Zero-Length Arrays (GNU)

Source

src/c/flex-array

Zero-length arrays (`char data[0]`) at the end of structs are a GNU extension that predates C99 flexible array members. They enable variable-length structures where the array size is determined at runtime through dynamic allocation. This pattern is common in network protocols, file formats, and any data structure where a header is followed by variable-length payload. C99 standardized this concept as flexible array members (`char data[]`), which should be preferred in new code for portability. Both approaches require careful memory management to allocate sufficient space for the actual data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct old_style {
    int length;
    char data[0]; // GNU zero-length array
};

struct new_style {
    int length;
    char data[]; // C99 flexible array member (preferred)
};

int main(void) {
    const char *msg = "Hello";
    int len = strlen(msg);

    struct new_style *s = malloc(sizeof(*s) + len + 1);
    s->length = len;
    strcpy(s->data, msg);
}
```

(continues on next page)

(continued from previous page)

```
printf("length=%d, data=%s\n", s->length, s->data);
free(s);
}
```

```
$ ./flex-array
length=5, data=Hello
```

Case Ranges (GNU)

Source

`src/c/case-range`

GCC allows ranges in case labels using `...` syntax, reducing repetitive code.

```
#include <stdio.h>

const char *grade(int score) {
    switch (score) {
        case 90 ... 100: return "A";
        case 80 ... 89:  return "B";
        case 70 ... 79:  return "C";
        case 60 ... 69:  return "D";
        case 0 ... 59:   return "F";
        default:         return "Invalid";
    }
}

int main(void) {
    printf("95 -> %s\n", grade(95));
    printf("73 -> %s\n", grade(73));
    printf("45 -> %s\n", grade(45));
}
```

```
$ ./case-range
95 -> A
73 -> C
45 -> F
```

1.2.4 Makefile

Table of Contents

- *Makefile*
 - *Introduction*
 - *Automatic Variables*
 - *using \$(warning text) check make rules (for debug)*
 - *String Functions*

- *using \$(sort list) sort list and remove duplicates*
- *Single Dollar Sign and Double Dollar Sign*
- *Build Executable Files Respectively*
- *using \$(eval) predefine variables*
- *Build Subdir and Link Together*
- *Build Shared Library*
- *Build Shared and Static Library*
- *Build Recursively*
- *Replace Current Shell*
- *One Line Condition*
- *Using Define to Control CFLAGS*

Introduction

GNU Make is the standard build automation tool for C and C++ projects, using Makefiles to define compilation rules and dependencies. Make tracks file modification times to rebuild only what has changed, dramatically speeding up incremental builds. This reference covers essential Make features from automatic variables and string functions to building shared libraries and recursive project structures. Understanding Make is fundamental for systems programming and contributes to faster development cycles.

Automatic Variables

Automatic variables are set by Make for each rule and provide access to target and prerequisite names without hard-coding. These variables enable writing generic pattern rules that work across different files. The most common are \$@ for the target, \$< for the first prerequisite, and \$^ for all prerequisites with duplicates removed.

automatic variables	descriptions
\$@	The file name of the target
\$<	The name of the first prerequisite
\$^	The names of all the prerequisites
\$+	prerequisites listed more than once are duplicated in the order

Makefile

```
.PHONY: all

all: hello world

hello world: foo foo foo bar bar
    @echo "== target: $@"
    @echo $<
    @echo $^
    @echo $+

foo:
    @echo "Hello foo"
```

(continues on next page)

(continued from previous page)

```
bar:
    @echo "Hello Bar"
```

output

```
Hello foo
Hello Bar
== target: hello ==
foo
foo bar
foo foo foo bar bar
== target: world ==
foo
foo bar
foo foo foo bar bar
```

using \$(warning text) check make rules (for debug)

The \$(warning text) function prints a message during Makefile parsing without stopping execution, making it invaluable for debugging complex Makefiles. By placing warnings at strategic points, you can trace variable expansion order, understand when rules are evaluated, and identify why variables have unexpected values. Unlike \$(error), warnings allow the build to continue.

```
$(warning Top level warning)

FOO := $(warning FOO variable)foo
BAR  = $(warning BAR variable)bar

$(warning target)target: $(warning prerequisite list)Makefile $(BAR)
    $(warning tagrget script)
    @ls
$(BAR):
```

output

```
Makefile:1: Top level warning
Makefile:3: FOO variable
Makefile:6: target
Makefile:6: prerequisite list
Makefile:6: BAR variable
Makefile:9: BAR variable
Makefile:7: tagrget script
Makefile
```

String Functions

Make provides powerful string manipulation functions for transforming file lists, extracting components, and pattern matching. These functions operate on whitespace-separated words and are essential for deriving object file lists from sources, filtering files by pattern, and performing substitutions. Common functions include `subst`, `patsubst`, `filter`, `filter-out`, and word extraction functions.

Makefile

```
SRC      = hello_foo.c hello_bar.c foo_world.c bar_world.c

SUBST    = $(subst .c,,$(SRC))

SRCST    = $(SRC:.c=.o)
PATSRCT  = $(SRC:%.c=%.o)
PATSUBST = $(patsubst %.c, %.o, $(SRC))

.PHONY: all

all: sub filter findstring words word wordlist

sub:
    @echo "== sub example =="
    @echo "SUBST: " $(SUBST)
    @echo "SRCST: " $(SRCST)
    @echo "PATSRCT: " $(PATSRCT)
    @echo "PATSUBST: " $(PATSUBST)
    @echo ""

filter:
    @echo "== filter example =="
    @echo "filter: " $(filter hello_%, $(SRC))
    @echo "filter-out: $(filter-out hello_%, $(SRC))"
    @echo ""

findstring:
    @echo "== findstring example =="
    @echo "Res: " $(findstring hello, hello world)
    @echo "Res: " $(findstring hello, ker)
    @echo "Res: " $(findstring world, worl)
    @echo ""

words:
    @echo "== words example =="
    @echo "num of words: "$(words $(SRC))
    @echo ""

word:
    @echo "== word example =="
    @echo "1st word: " $(word 1,$(SRC))
    @echo "2nd word: " $(word 2,$(SRC))
    @echo "3th word: " $(word 3,$(SRC))
    @echo ""

wordlist:
    @echo "== wordlist example =="
    @echo "[1:3]:$(wordlist 1,3,$(SRC))"
    @echo ""
```

output

```

$ make
== sub example ==
SUBST: hello_foo hello_bar foo_world bar_world
SRCST: hello_foo.o hello_bar.o foo_world.o bar_world.o
PATSRCT: hello_foo.o hello_bar.o foo_world.o bar_world.o
PATSUBST: hello_foo.o hello_bar.o foo_world.o bar_world.o

== filter example ==
filter: hello_foo.c hello_bar.c
filter-out: foo_world.c bar_world.c

== findstring example ==
Res: hello
Res:
Res:

== words example ==
num of words: 4

== word example ==
1st word: hello_foo.c
2nd word: hello_bar.c
3th word: foo_world.c

== wordlist example ==
[1:3]:hello_foo.c hello_bar.c foo_world.c

```

using \$(sort list) sort list and remove duplicates

The \$(sort list) function sorts words lexicographically and removes duplicates in a single operation. This is particularly useful for normalizing file lists, ensuring consistent ordering in builds, and eliminating redundant entries that might cause duplicate symbol errors during linking.

Makefile

```

SRC = foo.c bar.c ker.c foo.h bar.h ker.h

.PHONY: all

all:
    @echo $(suffix $(SRC))
    @echo $(sort $(suffix $(SRC)))

```

output

```

$ make
.c .c .c .h .h .h
.c .h

```

Single Dollar Sign and Double Dollar Sign

Understanding dollar sign escaping is crucial when mixing Make variables with shell commands. A single `$` references Make variables and is expanded before the shell sees the command. A double `$$` escapes to a single `$` for the shell, allowing access to shell variables and loop counters within recipes. This distinction is a common source of confusion in Makefiles.

dollar sign	descriptions
<code>\$</code>	reference a make variable using <code>\$</code>
<code>\$\$</code>	reference a shell variable using <code>\$\$</code>

Makefile

```
LIST = one two three

.PHONY: all single_dollar double_dollar

all: single_dollar double_dollar

double_dollar:
    @echo "=== double dollar sign example ==="
    @for i in $(LIST); do \
        echo $$i; \
    done

single_dollar:
    @echo "=== single dollar sign example ==="
    @for i in $(LIST); do \
        echo $i; \
    done
```

output

```
$ make
=== single dollar sign example ===

=== double dollar sign example ===
one
two
three
```

Build Executable Files Respectively

This pattern demonstrates building multiple executables from individual source files using implicit rules and wildcard functions. The `$(wildcard)` function finds all matching files, and substitution patterns derive object and executable names. This approach scales automatically as new source files are added to the directory.

directory layout

```
.
|-- Makefile
|-- bar.c
```

(continues on next page)

(continued from previous page)

```
|-- bar.h
|-- foo.c
`-- foo.h
```

Makefile

```
# CFLAGS: Extra flags to give to the C compiler
CFLAGS += -Werror -Wall -O2 -g
SRC      = $(wildcard *.c)
OBJ      = $(SRC:.c=.o)
EXE      = $(subst .c,, $(SRC))

.PHONY: all clean

all: $(OBJ) $(EXE)

clean:
    rm -rf *.o *.so *.a *.la $(EXE)
```

output

```
$ make
cc -Werror -Wall -O2 -g    -c -o foo.o foo.c
cc -Werror -Wall -O2 -g    -c -o bar.o bar.c
cc  foo.o    -o foo
cc  bar.o    -o bar
```

using \$(eval) predefine variables

The \$(eval) function parses its argument as Makefile syntax, enabling dynamic rule and variable generation. Combined with \$(call) and \$(foreach), it allows creating per-target variables and rules programmatically. Without eval, the generated text is treated as literal output rather than Makefile constructs, causing syntax errors.

without \$(eval)

```
SRC = $(wildcard *.c)
EXE = $(subst .c,, $(SRC))

define PROGRAM_template
$1_SHARED = lib$(strip $1).so
endef

.PHONY: all

$(foreach exe, $(EXE), $(call PROGRAM_template, $(exe)))

all:
    @echo $(foo_SHARED)
    @echo $(bar_SHARED)
```

output

```
$ make
Makefile:11: *** missing separator.  Stop.
```

with \$(eval)

```
CFLAGS += -Wall -g -O2 -I./include
SRC = $(wildcard *.c)
EXE = $(subst .c,, $(SRC))

define PROGRAM_template
$1_SHARED = lib$(strip $1).so
endef

.PHONY: all

$(foreach exe, $(EXE), $(eval $(call PROGRAM_template, $(exe))))

all:
    @echo $(foo_SHARED)
    @echo $(bar_SHARED)
```

output

```
$ make
libfoo.so
libbar.so
```

Build Subdir and Link Together

This pattern compiles source files from a subdirectory and links them into a single executable. The `-I` flag adds include paths, and pattern rules handle the compilation. This structure separates headers, sources, and build artifacts while maintaining a clean project layout.

directory layout

```
.
|-- Makefile
|-- include
|   `-- foo.h
`-- src
    |-- foo.c
    `-- main.c
```

Makefile

```
CFLAGS += -Wall -g -O2 -I./include
SRC     = $(wildcard src/*.c)
OBJ     = $(SRC:.c=.o)
EXE     = main

.PHONY: all clean

all: $(OBJ) $(EXE)

$(EXE): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^
```

(continues on next page)

(continued from previous page)

```
%o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -rf *.o *.so *.a *.la $(EXE) src/*.o src/*.so src/*.a
```

output

```
$ make
cc -Wall -g -O2 -I./include -c src/foo.c -o src/foo.o
cc -Wall -g -O2 -I./include -c src/main.c -o src/main.o
cc -o main src/foo.o src/main.o
```

Build Shared Library

Shared libraries (.so files) allow code reuse across multiple programs and reduce memory usage through shared memory mapping. Building them requires position-independent code (-fPIC) and the -shared linker flag. The -Wl, -soname option embeds the library's canonical name for runtime linking, enabling version management through symbolic links.

directory layout

```
.
|-- Makefile
|-- include
|   |-- common.h
|-- src
|   |-- bar.c
|   |-- foo.c
```

Makefile

```
SONAME    = libfoobar.so.1
SHARED    = src/libfoobar.so.1.0.0
SRC       = $(wildcard src/*.c)
OBJ       = $(SRC:.c=.o)

CFLAGS    += -Wall -Werror -fPIC -O2 -g -I./include
LDFLAGS   += -shared -Wl,-soname,$(SONAME)

.PHONY: all clean

all: $(SHARED) $(OBJ)

$(SHARED): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $^ -o $@

clean:
    rm -rf src/*.o src/*.so.* src/*.a src/*.la
```

output

```
$ make
cc -Wall -Werror -fPIC -O2 -g -I./include -c src/foo.c -o src/foo.o
cc -Wall -Werror -fPIC -O2 -g -I./include -c src/bar.c -o src/bar.o
cc -shared -Wl,-soname,libfoobar.so.1 -o src/libfoobar.so.1.0.0 src/foo.o src/bar.o
```

Build Shared and Static Library

This example builds both static (.a) and shared (.so) libraries from the same source files. Static libraries are archives of object files linked directly into executables, while shared libraries are loaded at runtime. The symbolic links (libfoo.so → libfoo.so.1 → libfoo.so.1.0.0) follow the standard versioning convention for ABI compatibility.

directory layout

```
.
|-- Makefile
|-- include
|   |-- bar.h
|   `-- foo.h
`-- src
    |-- Makefile
    |-- bar.c
    `-- foo.c
```

Makefile

```
SUBDIR = src

.PHONY: all clean $(SUBDIR)

all: $(SUBDIR)

clean: $(SUBDIR)

$(SUBDIR):
    make -C $@ $(MAKECMDGOALS)
```

src/Makefile

```
SRC      = $(wildcard *.c)
OBJ      = $(SRC:.c=.o)
LIB      = libfoobar

STATIC   = $(LIB).a
SHARED   = $(LIB).so.1.0.0
SONAME    = $(LIB).so.1
SOFILE    = $(LIB).so

CFLAGS   += -Wall -Werror -g -O2 -fPIC -I../include
LDFLAGS   += -shared -Wl,-soname,$(SONAME)

.PHONY: all clean

all: $(STATIC) $(SHARED) $(SONAME) $(SOFILE)
```

(continues on next page)

(continued from previous page)

```
$(SOFIELD): $(SHARED)
    ln -sf $(SHARED) $(SOFIELD)

$(SONAME): $(SHARED)
    ln -sf $(SHARED) $(SONAME)

$(SHARED): $(STATIC)
    $(CC) $(LDFLAGS) -o $@ $<

$(STATIC): $(OBJ)
    $(AR) $(ARFLAGS) $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm -rf *.o *.a *.so *.so.*
```

output

```
$ make
make -C src
make[1]: Entering directory '/root/test/src'
cc -Wall -Werror -g -O2 -fPIC -I../include -c -o foo.o foo.c
cc -Wall -Werror -g -O2 -fPIC -I../include -c -o bar.o bar.c
ar rv libfoobar.a foo.o bar.o
ar: creating libfoobar.a
a - foo.o
a - bar.o
cc -shared -Wl,-soname,libfoobar.so.1 -o libfoobar.so.1.0.0 libfoobar.a
ln -sf libfoobar.so.1.0.0 libfoobar.so.1
ln -sf libfoobar.so.1.0.0 libfoobar.so
make[1]: Leaving directory '/root/test/src'
```

Build Recursively

Recursive Make invokes Make in subdirectories, enabling modular project structures where each component has its own Makefile. The `$(MAKE)` variable ensures proper propagation of flags and job server connections for parallel builds. The `$(MAKECMDGOALS)` variable passes the original target (e.g., `clean`) to subdirectory Makefiles.

directory layout

```
.
|-- Makefile
|-- include
|   |-- common.h
|-- src
|   |-- Makefile
|   |-- bar.c
|   |-- foo.c
|-- test
|   |-- Makefile
|   |-- test.c
```

Makefile

```
SUBDIR = src test

.PHONY: all clean $(SUBDIR)

all: $(SUBDIR)

clean: $(SUBDIR)

$(SUBDIR):
    $(MAKE) -C $@ $(MAKECMDGOALS)
```

src/Makefile

```
SONAME    = libfoobar.so.1
SHARED    = libfoobar.so.1.0.0
SOFILE    = libfoobar.so

CFLAGS    += -Wall -g -O2 -Werror -fPIC -I../include
LDFLAGS    += -shared -Wl,-soname,$(SONAME)

SRC        = $(wildcard *.c)
OBJ        = $(SRC:.c=.o)

.PHONY: all clean

all: $(SHARED) $(OBJ)

$(SHARED): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^
    ln -sf $(SHARED) $(SONAME)
    ln -sf $(SHARED) $(SOFILE)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -rf *.o *.so.* *.a *.so
```

test/Makefile

```
CFLAGS    += -Wall -Werror -g -I../include
LDFLAGS    += -Wall -L../src -lfoobar

SRC        = $(wildcard *.c)
OBJ        = $(SRC:.c=.o)
EXE        = test_main

.PHONY: all clean

all: $(OBJ) $(EXE)

$(EXE): $(OBJ)
```

(continues on next page)

(continued from previous page)

```

$(CC) -o $@ $^ $(LDFLAGS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -rf *.so *.o *.a $(EXE)

```

output

```

$ make
make -C src
make[1]: Entering directory '/root/proj/src'
cc -Wall -g -O2 -Werror -fPIC -I../include -c foo.c -o foo.o
cc -Wall -g -O2 -Werror -fPIC -I../include -c bar.c -o bar.o
cc -shared -Wl,-soname,libfoobar.so.1 -o libfoobar.so.1.0.0 foo.o bar.o
ln -sf libfoobar.so.1.0.0 libfoobar.so.1
ln -sf libfoobar.so.1.0.0 libfoobar.so
make[1]: Leaving directory '/root/proj/src'
make -C test
make[1]: Entering directory '/root/proj/test'
cc -Wall -Werror -g -I../include -c test.c -o test.o
cc -o test_main test.o -Wall -L../src -lfoobar
make[1]: Leaving directory '/root/proj/test'
$ tree .
.
|-- Makefile
|-- include
|   `-- common.h
|-- src
|   |-- Makefile
|   |-- bar.c
|   |-- bar.o
|   |-- foo.c
|   |-- foo.o
|   |-- libfoobar.so -> libfoobar.so.1.0.0
|   |-- libfoobar.so.1 -> libfoobar.so.1.0.0
|   `-- libfoobar.so.1.0.0
`-- test
    |-- Makefile
    |-- test.c
    |-- test.o
    `-- test_main

3 directories, 14 files

```

Replace Current Shell

Make normally uses `/bin/sh` to execute recipe commands, but the `SHELL` variable can be overridden to use any interpreter. This enables writing recipes in Python, Perl, or other languages directly in the Makefile, though it reduces portability and should be used sparingly.

```
OLD_SHELL := $(SHELL)
SHELL = /usr/bin/python

.PHONY: all

all:
    @import os; print os.uname()[0]
```

output

```
$ make
Linux
```

One Line Condition

The `$(if condition,then-part,else-part)` function provides inline conditional logic for variable assignments and function calls. The condition is true if it expands to any non-empty string. This is useful for setting defaults, conditional flags, and adapting behavior based on variable values without verbose `ifdef` blocks.

syntax: `$(if cond, then part, else part)`

Makefile

```
VAR =
IS_EMPTY = $(if $(VAR), $(info not empty), $(info empty))

.PHONY: all

all:
    @echo $(IS_EMPTY)
```

output

```
$ make
empty

$ make VAR=true
not empty
```

Using Define to Control CFLAGS

The `ifdef` directive tests whether a variable is defined, enabling conditional compilation flags. This pattern allows enabling debug symbols, sanitizers, or optimization levels from the command line without modifying the Makefile. Variables can be set via `make VAR=value` or environment variables, providing flexible build configuration.

Makefile

```
CFLAGS += -Wall -Werror -g -O2
SRC      = $(wildcard *.c)
OBJ      = $(SRC:.c=.o)
EXE      = $(subst .c,, $(SRC))

ifdef DEBUG
CFLAGS += -DDEBUG
endif
```

(continues on next page)

(continued from previous page)

```
.PHONY: all clean

all: $(OBJ) $(EXE)

clean:
    rm -rf $(OBJ) $(EXE)
```

output

```
$ make
cc -Wall -Werror -g -O2 -c -o foo.o foo.c
cc  foo.o -o foo
$ make DEBUG=1
cc -Wall -Werror -g -O2 -DDEBUG -c -o foo.o foo.c
cc  foo.o -o foo
```

1.2.5 X86 Assembly

Table of Contents

- *X86 Assembly*
 - *Introduction*
 - *Registers*
 - *System Calls*
 - *Hello World*
 - *Loops*
 - *Procedures*
 - *Command Line Arguments*
 - *GCC Inline Assembly*
 - *CPUID*
 - *Syscall Wrappers*
 - *Calling Convention Summary*
 - *Reference*

Introduction

Assembly language provides direct control over CPU instructions and is essential for understanding how compilers generate code, writing performance-critical routines, and interfacing with hardware at the lowest level. While high-level languages abstract away machine details, assembly exposes the true nature of computation: registers, memory addressing, instruction encoding, and the hardware/software interface. This reference covers both standalone assembly programs using Linux syscalls and GCC inline assembly for embedding assembly in C code, with examples for x86_64 and ARM64 architectures. Understanding assembly is fundamental for systems programming, reverse engineering, security research, compiler development, and performance optimization where every cycle counts.

Registers

Registers are the fastest storage locations in a CPU, used for arithmetic operations, function arguments, return values, and temporary data. The x86_64 architecture provides 16 general-purpose 64-bit registers, each accessible in smaller widths (32-bit, 16-bit, 8-bit) for backward compatibility and efficient storage. The System V AMD64 ABI defines conventions for which registers hold function arguments, return values, and which must be preserved across function calls (callee-saved vs caller-saved).

x86_64 general-purpose registers and their conventional uses:

Register	64-bit	32-bit	16-bit	8-bit	Purpose
RAX	rax	eax	ax	al	Return value, accumulator
RBX	rbx	ebx	bx	bl	Callee-saved
RCX	rcx	ecx	cx	cl	4th argument, counter
RDX	rdx	edx	dx	dl	3rd argument, I/O
RSI	rsi	esi	si	sil	2nd argument, source index
RDI	rdi	edi	di	dil	1st argument, dest index
RBP	rbp	ebp	bp	bpl	Base pointer (callee-saved)
RSP	rsp	esp	sp	spl	Stack pointer
R8-R15	r8-r15	r8d-r15d	r8w-r15w	r8b-r15b	5th-6th args, callee-saved (r12-r15)

ARM64 registers:

Register	64-bit	32-bit	Purpose
X0-X7	x0-x7	w0-w7	Arguments and return values
X8	x8	w8	Indirect result, syscall number
X9-X15	x9-x15	w9-w15	Temporary (caller-saved)
X16-X17	x16-x17	w16-w17	Intra-procedure scratch
X18	x18	w18	Platform register
X19-X28	x19-x28	w19-w28	Callee-saved
X29	x29/fp	w29	Frame pointer
X30	x30/lr	w30	Link register (return address)
SP	sp	wsp	Stack pointer

System Calls

Source

`src/asm/exit`

System calls are the fundamental interface between user-space programs and the operating system kernel. When a program needs to perform privileged operations like file I/O, memory allocation, or process management, it must request these services from the kernel via syscalls. On x86_64 Linux, the `syscall` instruction invokes 64-bit system calls with the syscall number in `rax` and up to six arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. The return value appears in `rax`, with negative values indicating errors (the negated `errno`). The legacy `int $0x80` instruction invokes 32-bit syscalls using different syscall numbers and the older convention of arguments in `ebx`, `ecx`, `edx`, `esi`, `edi`, `ebp`.

x86_64 `exit` (64-bit syscall):

```
# gcc -nostdlib -no-pie -o exit64 exit64.s

.global _start
.section .text

_start:
```

(continues on next page)

(continued from previous page)

```

mov $60, %rax    # syscall: exit (64-bit)
mov $42, %rdi    # exit code
syscall

```

x86 exit (32-bit syscall via int 0x80):

```

# gcc -nostdlib -no-pie -m32 -o exit32 exit32.s

.global _start
.section .text

_start:
    mov $1, %eax    # syscall: exit (32-bit)
    mov $42, %ebx   # exit code
    int $0x80

```

ARM64 exit:

```

# gcc -nostdlib -o exit exit.s

.global _start
.section .text

_start:
    mov x8, #93     // syscall: exit
    mov x0, #42     // exit code
    svc #0

```

Hello World

Source

src/asm/hello

The classic “Hello World” program demonstrates essential assembly concepts: making system calls, working with string data, and program termination. This example uses the `write` syscall (number 1 on x86_64) to output text to stdout (file descriptor 1), then exits cleanly. The `lea` (load effective address) instruction with RIP-relative addressing (`message(%rip)`) loads the address of the message string in a position-independent way, which is required for modern executables. Data is placed in the `.rodata` section for read-only constants, keeping code and data properly separated.

```

# gcc -nostdlib -no-pie -o hello hello.s

.global _start
.section .text

_start:
    # write(1, message, 13)
    mov $1, %rax    # syscall: write
    mov $1, %rdi    # fd: stdout
    lea message(%rip), %rsi # buf: message (RIP-relative)
    mov $13, %rdx   # count
    syscall

    # exit(0)

```

(continues on next page)

(continued from previous page)

```
    mov $60, %rax          # syscall: exit
    xor %rdi, %rdi          # status: 0
    syscall

.section .rodata
message:
    .ascii "Hello World\n"
```

```
$ ./hello
Hello World
```

Loops

Source

src/asm/loop

Loop constructs in assembly use conditional jumps based on CPU flags set by comparison or arithmetic instructions. The `cmp` instruction subtracts operands and sets flags without storing the result, while `dec` decrements and sets the zero flag when the result is zero. Jump instructions like `jnz` (jump if not zero), `je` (jump if equal), and `j1` (jump if less) branch based on these flags. When making syscalls inside loops, use callee-saved registers (`r12-r15`, `rbx`, `rbp`) for loop counters since syscalls may clobber caller-saved registers. Local labels starting with `.` (like `.loop`) are scoped to the current function, preventing name collisions.

```
.global _start
.section .text

_start:
    mov $5, %r12            # counter = 5

.loop:
    # write(1, message, 13)
    mov $1, %rax
    mov $1, %rdi
    lea message(%rip), %rsi
    mov $13, %rdx
    syscall

    dec %r12                # counter--
    jnz .loop               # if (counter != 0) goto loop

    # exit(0)
    mov $60, %rax
    xor %rdi, %rdi
    syscall

.section .rodata
message:
    .ascii "Hello World\n"
```

Procedures

Source

src/asm/procedure

Functions in assembly follow the System V AMD64 ABI calling convention, which defines how arguments are passed, values returned, and registers preserved. The `call` instruction pushes the return address (address of the next instruction) onto the stack and jumps to the target function. The `ret` instruction pops this return address and jumps back to the caller. Stack frames are established by the standard prologue: push `rbp` to save the caller's base pointer, then set `rbp` to `rsp` to create a fixed reference point for local variables and parameters. The epilogue reverses this by popping `rbp` before returning. Functions must preserve callee-saved registers (`rbx`, `rbp`, `r12-r15`) and ensure the stack is 16-byte aligned before any `call` instruction.

```
.global _start
.section .text

_start:
    call print_hello
    call print_hello

    mov $60, %rax
    xor %rdi, %rdi
    syscall

print_hello:
    push %rbp                # save base pointer
    mov %rsp, %rbp          # set up stack frame

    mov $1, %rax
    mov $1, %rdi
    lea message(%rip), %rsi
    mov $13, %rdx
    syscall

    pop %rbp                # restore base pointer
    ret

.section .rodata
message:
    .ascii "Hello World\n"
```

Command Line Arguments

Source

src/asm/args

When a Linux program starts at `_start`, the kernel has already set up the stack with command-line arguments and environment variables. The stack layout places `argc` (argument count) at `(%rsp)`, followed by the `argv` array of pointers starting at `8(%rsp)`, terminated by a NULL pointer, then the `envp` array of environment variable pointers. Each `argv[i]` points to a null-terminated C string. This example demonstrates iterating through all arguments, computing string lengths with a custom `strlen` function, and printing each argument followed by a newline. The `cmpb` instruction compares bytes, and indexed addressing `(%rdi, %rax)` accesses `string[index]`.

```
.global _start
.section .text
```

(continues on next page)

(continued from previous page)

```

_start:
    mov (%rsp), %r12      # r12 = argc
    lea 8(%rsp), %r13     # r13 = &argv[0]

.print_args:
    cmp $0, %r12
    je .done

    mov (%r13), %rdi      # current argv
    call strlen           # length in %rax

    mov %rax, %rdx        # count
    mov $1, %rax          # write
    mov $1, %rdi          # stdout
    mov (%r13), %rsi      # buf
    syscall

    # print newline
    mov $1, %rax
    mov $1, %rdi
    lea newline(%rip), %rsi
    mov $1, %rdx
    syscall

    add $8, %r13          # next argv
    dec %r12
    jmp .print_args

.done:
    mov $60, %rax
    xor %rdi, %rdi
    syscall

strlen:
    xor %rax, %rax
.strlen_loop:
    cmpb $0, (%rdi, %rax)
    je .strlen_done
    inc %rax
    jmp .strlen_loop
.strlen_done:
    ret

.section .rodata
newline: .ascii "\n"

```

GCC Inline Assembly

Source

src/asm/inline-asm

GCC inline assembly embeds assembly instructions directly in C code, providing access to CPU features, special

instructions, and hardware interfaces not exposed through standard C. The extended asm syntax uses a template string with operand placeholders (%0, %1, etc.) and specifies output operands, input operands, and clobbered registers. This information allows the compiler to allocate registers, schedule instructions, and optimize around the assembly block. The `volatile` keyword prevents the compiler from removing or reordering the assembly, essential for operations with side effects like memory barriers or I/O. Inline assembly is commonly used for atomic operations, SIMD intrinsics, reading performance counters, and implementing lock-free data structures.

Basic syntax:

```
__asm__ [volatile] (
    "assembly template"
    : outputs      // "=constraint"(variable)
    : inputs       // "constraint"(expression)
    : clobbers     // registers modified
);
```

Constraint letters:

```
=  Output operand (write-only)
+  Output operand (read-write)
r  General register
a  rax/eax/ax/al
b  rbx/ebx/bx/bl
c  rcx/ecx/cx/cl
d  rdx/edx/dx/di
m  Memory operand
i  Immediate integer
```

Read timestamp counter:

```
static inline uint64_t rdtsc(void) {
    uint32_t lo, hi;
    __asm__ volatile ("rdtsc" : "=a"(lo), "=d"(hi));
    return ((uint64_t)hi << 32) | lo;
}
```

Memory barrier:

```
// x86_64
static inline void mfence(void) {
    __asm__ volatile ("mfence" ::: "memory");
}

// ARM64
static inline void dmb(void) {
    __asm__ volatile ("dmb ish" ::: "memory");
}
```

Byte swap:

```
static inline uint32_t bswap32(uint32_t x) {
    __asm__ ("bswapl %0" : "=r"(x));
    return x;
}
```

CPUID

Source

[src/asm/cpuid](#)

The `cpuid` instruction is the standard mechanism for querying x86 CPU identification and feature information at runtime. It takes a “leaf” number in `eax` (and optionally a “subleaf” in `ecx`) and returns information in all four registers (`eax`, `ebx`, `ecx`, `edx`). Leaf 0 returns the maximum supported leaf and the vendor identification string (“GenuineIntel” or “AuthenticAMD”). Leaf 1 returns processor version information and feature flags indicating support for SSE, AVX, AES-NI, and other extensions. Extended leaves (0x80000002-0x80000004) return the processor brand string. Runtime feature detection using CPUID enables writing code that adapts to available CPU capabilities, using optimized paths when advanced instructions are available.

```
typedef struct { uint32_t eax, ebx, ecx, edx; } cpuid_t;

static inline cpuid_t cpuid(uint32_t leaf, uint32_t subleaf) {
    cpuid_t r;
    __asm__ volatile (
        "cpuid\n"
        : "=a"(r.eax), "=b"(r.ebx), "=c"(r.ecx), "=d"(r.edx)
        : "a"(leaf), "c"(subleaf)
    );
    return r;
}

void get_vendor(char *vendor) {
    cpuid_t id = cpuid(0, 0);
    memcpy(vendor + 0, &id.ebx, 4);
    memcpy(vendor + 4, &id.edx, 4);
    memcpy(vendor + 8, &id.ecx, 4);
    vendor[12] = '\\0';
}
```

Feature detection:

```
cpuid_t f = cpuid(1, 0);
int has_sse42 = (f.ecx >> 20) & 1;
int has_avx   = (f.ecx >> 28) & 1;
int has_aesni = (f.ecx >> 25) & 1;

cpuid_t e = cpuid(7, 0);
int has_avx2 = (e.ebx >> 5) & 1;
```

Syscall Wrappers

Source

[src/asm/syscall-wrapper](#)

Inline assembly enables creating lightweight syscall wrappers that invoke system calls directly without going through the C library. This is useful for minimal runtime environments, static binaries without `libc`, or when you need precise control over the syscall interface. The wrappers must follow the kernel’s syscall ABI: syscall number in `rax`, arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, and return value in `rax`. The `syscall` instruction clobbers `rcx` (saved return address) and `r11` (saved RFLAGS), which must be declared in the clobber list. The “memory” clobber tells the compiler that the syscall may read or write arbitrary memory. useful for minimal runtime environments or when `libc` is unavailable. The `syscall` instruction clobbers `rcx` and `r11` in addition to the return value in `rax`.

```

static inline long syscall3(long n, long a1, long a2, long a3) {
    long ret;
    __asm__ volatile (
        "syscall"
        : "=a"(ret)
        : "a"(n), "D"(a1), "S"(a2), "d"(a3)
        : "rcx", "r11", "memory"
    );
    return ret;
}

// Usage
#define SYS_write 1
syscall3(SYS_write, 1, (long)"Hello\n", 6);

```

Calling Convention Summary

Calling conventions define the contract between caller and callee for passing arguments, returning values, and preserving registers. The System V AMD64 ABI, used by Linux, macOS, and BSD, passes the first six integer/pointer arguments in registers for efficiency, with additional arguments on the stack. Floating-point arguments use XMM registers. Caller-saved registers may be overwritten by any function call, so the caller must save them if needed. Callee-saved registers must be preserved by functions that use them, typically by pushing them in the prologue and popping in the epilogue. The stack must be 16-byte aligned before any `call` instruction to ensure proper alignment for SSE instructions.

System V AMD64 ABI (Linux, macOS, BSD):

Arguments:	rdi, rsi, rdx, rcx, r8, r9 (then stack)
Return:	rax (and rdx for 128-bit)
Caller-saved:	rax, rcx, rdx, rsi, rdi, r8-r11
Callee-saved:	rbx, rbp, r12-r15
Stack:	16-byte aligned before call

Syscall convention (Linux x86_64):

Syscall #:	rax
Arguments:	rdi, rsi, rdx, r10, r8, r9
Return:	rax (negative = -errno)
Clobbered:	rcx, r11

Reference

- [Linux System Call Table](#)
- [System V AMD64 ABI](#)
- [GCC Inline Assembly HOWTO](#)
- [ARM64 Instruction Set](#)

1.3 Modern C++ Programming

The C++ language has undergone a remarkable transformation since C++11, evolving into a powerful, expressive language that supports multiple programming paradigms while maintaining backward compatibility and zero-cost ab-

stractions. Each subsequent standard—C++14, C++17, C++20, and C++23—has introduced features that simplify code, improve safety, and enable new programming patterns.

The following chapters explore resource management through RAII and smart pointers, generic programming with templates and concepts, functional programming with lambda expressions, compile-time computation with constexpr, asynchronous programming with coroutines, and data manipulation using STL containers and algorithms.

1.3.1 C++ Basics

Table of Contents

- *C++ Basics*
 - *std::print (C++23)*
 - *C Linkage and Name Mangling*
 - *Uniform Initialization (Brace Initialization)*
 - *Pointer Arithmetic and Negative Indices*
 - *Template Type Deduction*
 - *Auto Type Deduction*
 - *decltype(auto) Type Deduction*
 - *Reference Collapsing Rules*
 - *Perfect Forwarding with std::forward*
 - *Bit Manipulation with std::bitset*
 - *Safe Address-of with std::addressof*

std::print (C++23)

Source

src/basic/print

C++23 introduces `std::print` and `std::println` from the `<print>` header, bringing Python-style formatted output to C++. This is the modern way to print in C++, replacing the traditional `std::cout` approach.

```
#include <print>

int main() {
    std::print("Hello, World!");    // No newline
    std::println("Hello, World!"); // With newline
}
```

The old way using `std::cout`:

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
}
```


C Linkage and Name Mangling

Source

`src/basic/c-linkage`

In C++, the compiler applies *name mangling* to function symbols to support function overloading. Name mangling encodes function signatures (parameter types, namespaces, and template arguments) into the symbol name, allowing multiple functions with the same name but different signatures to coexist. When interfacing with C libraries or exposing functions to C code, `extern "C"` disables name mangling to ensure binary compatibility across different compilers and languages.

With `extern "C"` (C linkage):

Using `nm` to inspect the symbol table, we can observe that the `fib` function retains its original name without mangling. The symbol `_fib` follows C naming conventions, making it accessible from C code or other languages that use C-compatible calling conventions:

```
#include <iostream>

#ifdef __cplusplus
extern "C" {
#endif

int fib(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; ++i) {
        int tmp = b;
        b = a + b;
        a = tmp;
    }
    return a;
}

#ifdef __cplusplus
}
#endif

int main(int argc, char *argv[]) {
    std::cout << fib(10) << "\n"; // Output: 55
}
```

```
$ g++ -std=c++17 -Wall -Werror -O3 a.cc
$ nm -g a.out | grep fib
00000000100003a58 T _fib      # C-style symbol (no mangling)
```

Without `extern "C"` (C++ linkage):

Without the `extern "C"` wrapper, the compiler mangles the function name to encode type information for overload resolution. The mangled symbol `__Z3fibi` follows the Itanium C++ ABI naming scheme: `_Z` indicates a mangled name, `3fib` represents the function name with its length prefix, and `i` denotes an `int` parameter. This encoding enables the linker to distinguish between overloaded functions:

```
#include <iostream>

int fib(int n) {
    int a = 0, b = 1;
```

(continues on next page)

(continued from previous page)

```

for (int i = 0; i < n; ++i) {
    int tmp = b;
    b = a + b;
    a = tmp;
}
return a;
}

int main(int argc, char *argv[]) {
    std::cout << fib(10) << "\n";
}

```

```

$ nm -g a.out | grep fib
00000000100003a58 T __Z3fibi      # Mangled symbol: _Z3fibi

```

Platform-specific macros:

On BSD and macOS systems, `<sys/cdefs.h>` provides `__BEGIN_DECLS` and `__END_DECLS` as portable alternatives to `extern "C"`. These macros expand to the appropriate linkage specification when compiled as C++ and to nothing when compiled as C, simplifying header files shared between both languages:

```

#include <iostream>
#include <sys/cdefs.h>

__BEGIN_DECLS
int fib(int n) { /* ... */ }
__END_DECLS

```

Uniform Initialization (Brace Initialization)**Source**

[src/basic/uniform-initialization](#)

Introduced in C++11, *uniform initialization* (also known as *brace initialization*) provides a consistent syntax for initializing objects of any type. This syntax works for primitives, aggregates, containers, and user-defined types. However, the compiler prioritizes `std::initializer_list` constructors when applicable, which can lead to surprising behavior if not understood properly.

Initializer list takes precedence:

When both a direct constructor and an `std::initializer_list` constructor are viable, the compiler strongly prefers the initializer list version. In this example, `Widget{10, 5.0}` invokes the initializer list constructor even though `Widget(int, double)` appears to be a better match. The values `10` and `5.0` are implicitly converted to `long double` and passed as a two-element initializer list. This behavior is mandated by the C++ standard to ensure consistent semantics for brace initialization:

```

#include <iostream>
#include <initializer_list>

class Widget {
public:
    Widget(int a, double b) { std::cout << "Direct constructor\n"; }

    Widget(std::initializer_list<long double> il) { std::cout << "Initializer list,

```

(continues on next page)

(continued from previous page)

```

↪constructor\n"; }
};

int main(int argc, char *argv[]) {
    Widget w{10, 5.0}; // Output: Initializer list constructor
}

```

Narrowing conversions are prohibited:

Brace initialization prevents implicit narrowing conversions, providing an additional layer of type safety compared to parentheses initialization. The following code produces a compilation error because `int` and `double` cannot implicitly narrow to `bool` without potential data loss. This protection helps catch bugs at compile time that might otherwise cause subtle runtime errors:

```

#include <initializer_list>

class Widget {
public:
    Widget(int a, double b) {}
    Widget(std::initializer_list<bool> il) {}
};

int main(int argc, char *argv[]) {
    Widget w{10, 5.0}; // Compilation error: narrowing conversion
}

```

Fallback to direct constructor:

When no valid conversion exists for the initializer list element type, the compiler falls back to the matching direct constructor. Here, `int` and `double` cannot convert to `std::string` (there is no implicit conversion path), so the compiler bypasses the initializer list constructor entirely and selects the direct constructor instead:

```

#include <iostream>
#include <initializer_list>
#include <string>

class Widget {
public:
    Widget(int a, double b) { std::cout << "Direct constructor\n"; }

    Widget(std::initializer_list<std::string> il) { std::cout << "Initializer list_
↪constructor\n"; }
};

int main(int argc, char *argv[]) {
    Widget w{10, 5.0}; // Output: Direct constructor
}

```

Pointer Arithmetic and Negative Indices**Source**

[src/basic/negative-array-index](#)

In C++, the subscript operator `[]` is defined as pointer arithmetic: `arr[i]` is equivalent to `*(arr + i)`. This equivalence is fundamental to how arrays work in C and C++, and it allows negative indices when the pointer references

an element beyond the array's beginning. While this flexibility is powerful, it requires careful bounds management to avoid undefined behavior.

In this example, `ptr` points to `arr[1]`, the second element of the array. Using `ptr[-1]` computes `*(ptr - 1)`, which accesses `arr[0]`. This technique is commonly used in algorithms that need to look backward from a current position, such as insertion sort or string parsing:

```
#include <iostream>

int main(int argc, char *argv[]) {
    int arr[] = {1, 2, 3};
    int *ptr = &arr[1]; // Points to second element

    std::cout << ptr[-1] << "\n"; // Output: 1 (arr[0])
    std::cout << ptr[0] << "\n";  // Output: 2 (arr[1])
    std::cout << ptr[1] << "\n";  // Output: 3 (arr[2])
}
```

Template Type Deduction

Source

[src/basic/reference-collapsing](#)

Understanding how template parameters deduce types is essential for writing generic C++ code. The deduction rules differ based on the parameter declaration, and mastering these rules is crucial for effective use of templates, `auto`, and perfect forwarding. The following three examples demonstrate how the same arguments produce different deduced types depending on whether the parameter is an lvalue reference, universal reference, or value.

Lvalue reference parameters (T&):

When the parameter is an lvalue reference, the deduced type `T` preserves const-qualifiers from the argument, but the reference itself is not part of `T`. This means passing a `const int` deduces `T` as `const int`, and the parameter type becomes `const int&`. This behavior ensures that const-correctness is maintained through template instantiation:

```
template <typename T>
void f(T& param) noexcept {}

int main(int argc, char *argv[]) {
    int x = 123;
    const int cx = x;
    const int& rx = x;

    f(x);    // T = int,      param: int&
    f(cx);   // T = const int, param: const int&
    f(rx);   // T = const int, param: const int&
}
```

Universal reference parameters (T&&):

Universal references (also called forwarding references) behave differently for lvalues and rvalues, making them the foundation of perfect forwarding. When passed an lvalue, `T` deduces to an lvalue reference type (e.g., `int&`), and reference collapsing produces an lvalue reference parameter. When passed an rvalue, `T` deduces to a non-reference type, and the parameter becomes an rvalue reference. This dual behavior allows a single function template to accept both lvalues and rvalues:

```

template <typename T>
void f(T&& param) noexcept {}

int main(int argc, char *argv[]) {
    int x = 123;
    const int cx = x;
    const int& rx = x;

    f(x);    // x is lvalue: T = int&,      param: int&
    f(cx);   // cx is lvalue: T = const int&, param: const int&
    f(rx);   // rx is lvalue: T = const int&, param: const int&
    f(12);   // 12 is rvalue: T = int,      param: int&&
}

```

Value parameters (T):

When the parameter is passed by value, the argument is copied, and both references and top-level const-qualifiers are stripped from the deduced type. This occurs because the function receives an independent copy that can be modified without affecting the original. The decay behavior mirrors what happens when you assign to a new variable of type auto:

```

template <typename T>
void f(T param) noexcept {}

int main(int argc, char *argv[]) {
    int x = 123;
    const int cx = x;
    const int& rx = x;

    f(x);    // T = int, param: int (copy)
    f(cx);   // T = int, param: int (const dropped)
    f(rx);   // T = int, param: int (reference and const dropped)
    f(12);   // T = int, param: int
}

```

Auto Type Deduction

The auto keyword follows the same deduction rules as template value parameters, with one notable exception: braced initializers. This example demonstrates how auto deduces types for different initializers, including the special behavior of auto&& as a universal reference. Understanding these rules is essential for writing modern C++ code that leverages type inference effectively:

```

int main() {
    auto x = 123;           // int
    const auto cx = x;      // const int
    const auto& rx = x;     // const int&

    auto&& urx = x;         // int& (x is lvalue)
    auto&& urcx = cx;       // const int& (cx is lvalue)
    auto&& urrx = rx;       // const int& (rx is lvalue)
    auto&& urrv = 12;       // int&& (12 is rvalue)
}

```

decltype(auto) Type Deduction

Unlike `auto`, `decltype(auto)` preserves the exact type including references and cv-qualifiers. This distinction is critical when the original type information must be retained, such as when forwarding return values from wrapped functions. While `auto` applies template argument deduction rules (which strip references), `decltype(auto)` applies `decltype` semantics to the initializer expression.

The first example contrasts `decltype(auto)` with `auto`. Note how `auto` strips the reference and `const` from `crx`, producing a plain `int`, while `decltype(auto)` preserves the full `const int&` type. Similarly, `decltype(auto)` preserves rvalue references, which `auto` would decay to a value type:

```
#include <type_traits>

int main(int argc, char *argv[]) {
    int x = 0;
    const int cx = x;
    const int& crx = x;
    int&& z = 0;

    // decltype(auto) preserves cv-qualifiers and references
    decltype(auto) y1 = crx;
    static_assert(std::is_same_v<const int&, decltype(y1)>);

    // auto strips cv-qualifiers and references
    auto y2 = crx;
    static_assert(std::is_same_v<int, decltype(y2)>);

    // decltype(auto) preserves rvalue references
    decltype(auto) z1 = std::move(z);
    static_assert(std::is_same_v<int&&, decltype(z1)>);
}
```

Application in return type deduction:

This behavior is particularly useful for generic functions that must preserve the exact return type of an expression. In this example, `foo` uses `auto` return type deduction, which strips the reference and returns by value. In contrast, `bar` uses `decltype(auto)`, preserving the `const int&` return type. This distinction matters for performance (avoiding copies) and semantics (maintaining reference semantics):

```
#include <type_traits>

auto foo(const int& x) {
    return x; // Returns int (reference stripped)
}

decltype(auto) bar(const int& x) {
    return x; // Returns const int& (reference preserved)
}

int main(int argc, char *argv[]) {
    static_assert(std::is_same_v<int, decltype(foo(1))>);
    static_assert(std::is_same_v<const int&, decltype(bar(1))>);
}
```

Reference Collapsing Rules

When references to references occur during template instantiation or type aliasing, the compiler applies *reference collapsing* rules. These rules are fundamental to understanding how universal references and `std::forward` work. In C++, you cannot directly declare a reference to a reference, but such types can arise indirectly through template instantiation or type aliases:

```
T& &    -> T&
T& &&   -> T&
T&& &   -> T&
T&& &&  -> T&&
```

The rule can be summarized as: **lvalue reference always wins**. Only when both references are rvalue references does the result remain an rvalue reference. This mechanism enables perfect forwarding by allowing a single function template to handle both lvalues and rvalues correctly. When an lvalue is passed to a universal reference parameter, T deduces to an lvalue reference, and reference collapsing ensures the parameter type is also an lvalue reference.

Perfect Forwarding with `std::forward`

Source

[src/basic/perfect-forwarding](#)

Perfect forwarding preserves the value category (lvalue/rvalue) of function arguments when passing them to other functions. This technique is essential for writing wrapper functions, factory functions, and generic code that must not alter the semantics of the forwarded arguments. Perfect forwarding combines universal references with `std::forward` and relies on the reference collapsing rules described above.

Implementation of `std::forward`:

The standard library's `std::forward` uses `static_cast` combined with reference collapsing to conditionally cast to an rvalue reference. The first overload handles lvalue arguments: when T is an lvalue reference type, T&& collapses to an lvalue reference. The second overload handles rvalue arguments and includes a static assertion to prevent forwarding an rvalue as an lvalue:

```
#include <type_traits>

template <typename T>
T&& forward(std::remove_reference_t<T>& t) noexcept {
    return static_cast<T&&>(t);
}

template <typename T>
T&& forward(std::remove_reference_t<T>&& t) noexcept {
    static_assert(!std::is_lvalue_reference_v<T>);
    return static_cast<T&&>(t);
}
```

Wrapper function:

This example demonstrates a wrapper function that forwards arguments to a callable while preserving their value category. When d (an lvalue) is passed, T deduces to `Data&`, and `std::forward` returns an lvalue reference. When `std::move(t)` is passed, T deduces to `Data`, and `std::forward` returns an rvalue reference:

```
#include <iostream>
#include <utility>

template <typename T, typename Func>
```

(continues on next page)

(continued from previous page)

```

void wrapper(T&& arg, Func fn) {
    fn(std::forward<T>(arg));
}

struct Data { int x, y, result; };

int main() {
    Data d{1, 2, 0};
    wrapper(d, [](Data& d) { d.result = d.x + d.y; });
    std::cout << d.result << "\n"; // 3

    Data t{5, 6, 0};
    wrapper(std::move(t), [](Data&& d) { d.result = d.x * d.y; });
    std::cout << t.result << "\n"; // 30
}

```

Decorator pattern:

A timing decorator that wraps any callable and measures execution time while perfectly forwarding all arguments to the wrapped function:

```

#include <iostream>
#include <utility>
#include <chrono>

template <typename Func, typename ...Args>
auto timed(Func &&f, Args&&... args) {
    auto start = std::chrono::system_clock::now();
    auto ret = f(std::forward<Args>(args)...);
    std::chrono::duration<double> d = std::chrono::system_clock::now() - start;
    std::cout << "Time: " << d.count() << "s\n";
    return ret;
}

long fib(long n) { return n < 2 ? n : fib(n-1) + fib(n-2); }

int main() { timed(fib, 35); }

```

Factory pattern:

A generic factory function that constructs objects by forwarding arguments to constructors, preserving value categories for efficient object creation:

```

#include <memory>
#include <utility>

template <typename T, typename ...Args>
std::unique_ptr<T> make(Args&&... args) {
    return std::make_unique<T>(std::forward<Args>(args)...);
}

struct Widget { int x; double y; };

int main() { auto w = make<Widget>(42, 3.14); }

```


See [Move Semantics](#) for the related move semantics topics: special member functions, Rule of Zero/Three/Five, value categories, move-only types, and common move semantics pitfalls.

Bit Manipulation with `std::bitset`

Source

`src/basic/bit-manipulation`

The `std::bitset` class template provides a fixed-size sequence of bits with convenient member functions for bit manipulation. Unlike raw integer bit operations, `std::bitset` offers type safety, clear semantics, and bounds checking in debug builds. The template parameter specifies the number of bits, which is fixed at compile time.

This example creates a 4-bit `bitset` initialized to 8 (binary 1000). The `count()` member function returns the number of set bits (population count), which is useful for algorithms that need to count flags or compute Hamming weights. The equality operator allows direct comparison with integer values, automatically handling the conversion:

```
#include <bitset>
#include <iostream>

int main(int argc, char *argv[]) {
    std::bitset<4> bits{0b1000}; // Binary: 1000

    std::cout << bits.count() << "\n"; // Output: 1 (popcount)
    std::cout << (bits == 8) << "\n"; // Output: 1 (true)
}
```

Safe Address-of with `std::addressof`

Source

`src/basic/addressof`

C++ permits overloading `operator&`, which can interfere with obtaining an object's actual memory address. While overloading this operator is rare and generally discouraged, it does occur in some legacy codebases and smart pointer implementations. The `std::addressof` function template bypasses any overloaded `operator&` to return the true address, making it essential for generic code that must work with arbitrary types.

In this example, the overloaded `operator&` could potentially return an arbitrary pointer (perhaps for proxy object patterns or debugging). Using `std::addressof` guarantees we obtain the object's actual memory location regardless of any operator overloading. This is particularly important in allocator implementations, container internals, and other low-level generic code:

```
#include <iostream>
#include <memory>

struct Widget {
    int value;
};

const Widget* operator&(const Widget& w) {
    // Custom behavior - could return anything
    return std::addressof(w); // Safe: returns actual address
}

int main(int argc, char *argv[]) {
    Widget w;
    std::cout << &w << "\n"; // Uses overloaded operator&
}
```

(continues on next page)

(continued from previous page)

```
std::cout << std::addressof(w) << "\n"; // Always returns true address
}
```

Note

Always use `std::addressof` in generic code where the type may have an overloaded operator`&`. The standard library containers and algorithms use `std::addressof` internally for this reason.

1.3.2 Resource Management

Table of Contents

- *Resource Management*
 - *Initializer Lists*
 - *RAII Wrapper*
 - *Constructor Failure and Exception Safety*
 - *Exception Safety Guarantees*
 - *Lock-Based RAII*
 - *Scope Guards*
 - *Two-Phase Initialization: Why RAII Avoids It*
 - *See Also*

Resource Acquisition Is Initialization (RAII) is a fundamental C++ programming idiom that ties resource management to object lifetime. When an object is constructed, it acquires resources; when it is destroyed, it releases them. This pattern eliminates resource leaks and ensures exception safety by leveraging C++'s deterministic destruction guarantees.

RAII underpins the standard library: `std::unique_ptr` owns a heap allocation, `std::lock_guard` owns a mutex lock, `std::fstream` owns a file handle. Each releases its resource automatically when the wrapping object goes out of scope, even if an exception propagates through the scope. For the copy/move machinery that makes RAII types composable with containers and generic code, see [Move Semantics](#).

Note that [Rust](#) enforces ownership at compile time, offering strong safety guarantees. Interestingly, Rust's ownership model closely resembles C++'s RAII and move semantics (available since C++11). For example, Rust's ownership transfer is analogous to `std::move`, and its `Drop` trait mirrors C++ destructors.

Initializer Lists

Source

[src/raii/initializer-list](#)

`std::initializer_list` provides a lightweight proxy for accessing an array of objects. It enables uniform initialization syntax and is commonly used for constructors that accept a variable number of homogeneous arguments.

The initializer list does not own its elements; it merely provides access to a temporary array created by the compiler. This makes it efficient for passing initialization data:

```

#include <initializer_list>
#include <iostream>

template <typename T>
T sum(std::initializer_list<T> values) {
    T result = 0;
    for (const auto &v : values) {
        result += v;
    }
    return result;
}

int main(int argc, char *argv[]) {
    std::cout << sum({1, 2, 3, 4, 5}) << "\n";           // Output: 15
    std::cout << sum({1.5, 2.5, 3.0}) << "\n";           // Output: 7.0
}

```

RAII Wrapper

Source

[src/raii/raii-wrapper](#)

The RAII wrapper pattern encapsulates resource management in a class, ensuring resources are properly acquired in the constructor and released in the destructor. This pattern is the foundation of smart pointers, lock guards, and file handles in the C++ standard library.

This example demonstrates a simple RAII wrapper for a file handle:

```

#include <cstdio>
#include <stdexcept>
#include <utility>

class File {
public:
    explicit File(const char *path, const char *mode) : handle_(std::fopen(path, mode)) {
        if (!handle_) {
            throw std::runtime_error("Failed to open file");
        }
    }

    ~File() {
        if (handle_) {
            std::fclose(handle_);
        }
    }

    // Non-copyable
    File(const File &) = delete;
    File &operator=(const File &) = delete;

    // Movable
    File(File &&other) noexcept : handle_(std::exchange(other.handle_, nullptr)) {}
    File &operator=(File &&other) noexcept {
        if (this != &other) {

```

(continues on next page)

(continued from previous page)

```

    if (handle_) std::fclose(handle_);
    handle_ = std::exchange(other.handle_, nullptr);
}
return *this;
}

std::FILE *get() const { return handle_; }

private:
    std::FILE *handle_;
};

int main(int argc, char *argv[]) {
    try {
        File f("/tmp/test.txt", "w");
        std::fputs("RAII in action\n", f.get());
    } catch (const std::exception &e) {
        // File automatically closed even if exception thrown
    }
    // File automatically closed when 'f' goes out of scope
}

```

Note

Modern C++ provides `std::unique_ptr` with custom deleters and `std::fstream` for most resource management needs. Use custom RAII wrappers only when standard library alternatives are insufficient.

Constructor Failure and Exception Safety**Source**

[src/raii/constructor-failure](#)

When a constructor throws, the object being constructed **never existed** from the language's perspective: its destructor does **not** run. However, destructors for any fully-constructed subobjects—base classes and data members—**do** run, in reverse order of construction. This asymmetry is the single most important rule for handling failed resource acquisition in C++ constructors. It has two concrete consequences:

1. A raw-pointer member that has been `new`'d but not yet handed off to an owning wrapper leaks when the constructor throws.
2. A smart-pointer or container member that owns its resource cleans up automatically, because the *member's* destructor runs even though the *enclosing object's* destructor does not.

Leaky: raw pointer acquired, then throw:

```

class LeakyResource {
public:
    LeakyResource() {
        first_ = new Handle("first");
        throw std::runtime_error("second resource failed");
        // Never reached. ~LeakyResource does not run. first_ leaks.
    }
    ~LeakyResource() { delete first_; }
}

```

(continues on next page)

(continued from previous page)

```
private:
    Handle *first_ = nullptr;
};
```

Safe: `unique_ptr` member cleans up on throw:

```
class SafeResource {
public:
    SafeResource() : first_(std::make_unique<Handle>("first")) {
        throw std::runtime_error("second resource failed");
        // first_ is a fully-constructed member; its destructor runs.
    }
private:
    std::unique_ptr<Handle> first_;
};
```

The rule generalises: acquire every resource through an owning member (`std::unique_ptr`, `std::vector`, `std::string`, a custom RAII wrapper). Never leave a raw `new` exposed between acquisition and ownership transfer.

function-try-block:

A *function-try-block* catches exceptions thrown from the member-initializer-list. The catch handler cannot suppress the exception—if the handler does not rethrow explicitly, the exception is rethrown implicitly at the end of the block. By the time the handler runs, members have already been destroyed:

```
class Reported {
public:
    Reported() try : handle_(std::make_unique<Handle>("reported")) {
        throw std::runtime_error("post-init failure");
    } catch (const std::exception &) {
        // handle_ is already destroyed; exception rethrows implicitly.
    }
private:
    std::unique_ptr<Handle> handle_;
};
```

Exception Safety Guarantees

Source

[src/raii/exception-safety](#)

When an operation throws, the calling code needs to know what state the affected object is left in. C++ code is categorised into four escalating guarantees (defined by Abrahams):

Guarantee	Meaning
No-throw	The operation never throws. Declared <code>noexcept</code> . Required for move operations used by <code>std::vector</code> reallocation, for swap, and for destructors.
Strong	The operation either succeeds or has no effect (commit-or-rollback). Achieved by the copy-and-swap idiom: do the work on a copy, then swap — the swap is <code>nothrow</code> , so the commit cannot fail.
Basic	Invariants are preserved and no resources leak, but the object's observable state may be partially updated.
None	Avoid. Broken invariants are possible.

Copy-and-swap for the strong guarantee:

```

class Counter {
public:
    void swap(Counter &other) noexcept {
        std::swap(value_, other.value_);
        std::swap(log_, other.log_);
    }

    // Strong guarantee: copy constructs a temporary (may throw and die
    // unused, leaving *this unchanged), then nothrow-swaps it in.
    Counter &operator=(Counter other) noexcept {
        swap(other);
        return *this;
    }
private:
    int value_ = 0;
    std::vector<int> log_;
};

```

Strong vs basic in practice:

```

void Counter::add(int delta, bool fail) {
    log_.push_back(delta);           // happens first
    if (fail) throw std::runtime_error("");
    value_ += delta;
}

```

After `add(5, true)` throws, `value_` is unchanged but `log_` has grown — the object is still valid and usable (basic guarantee), but the operation did not roll back (no strong guarantee). Reorder the work so the throwing step runs first, before any visible mutation, to upgrade basic to strong.

Lock-Based RAI**Source**

`src/raii/lock-guard`

The standard library ships three RAI wrappers for mutex locking. Each acquires the lock in its constructor and releases it in its destructor, guaranteeing that the mutex is released even if the protected scope exits via exception.

Wrapper	Since	Use when
<code>std::lock_guard</code>	C++11	Simple scoped locking. No manual unlock or deferred acquisition.
<code>std::unique_lock</code>	C++11	Deferred/timed acquisition, manual unlock, transferable ownership, required by <code>std::condition_variable::wait</code> .
<code>std::scoped_lock</code>	C++17	Locking two or more mutexes at once with built-in deadlock avoidance. Prefer over <code>std::lock(m1, m2) + lock_guard</code> .

Scoped mutual exclusion:

```

std::mutex m;
int counter = 0;
auto worker = [&] {
    for (int i = 0; i < 10000; ++i) {

```

(continues on next page)

(continued from previous page)

```
std::lock_guard<std::mutex> guard(m);
++counter;
}
};
```

Deferred acquisition with unique_lock:

```
std::unique_lock<std::mutex> lock(m, std::defer_lock);
// ... some work that does not need the lock ...
lock.lock();
// ... critical section ...
lock.unlock(); // release early; destructor no longer unlocks
```

Multiple mutexes without deadlock:

```
std::mutex m1, m2;
{
    std::scoped_lock lock(m1, m2); // acquires both safely
    // critical section
}
```

`std::scoped_lock` uses a deadlock-avoidance algorithm (equivalent to `std::lock`) when given multiple mutexes, so two threads locking `(m1, m2)` and `(m2, m1)` cannot deadlock.

Scope Guards**Source**

[src/raii/scope-guard](#)

Writing a dedicated RAII wrapper class is overkill when the cleanup is a single ad-hoc statement: restore a flag, decrement a counter, roll back a partially-completed operation. A *scope guard* runs an arbitrary callable when it leaves scope. It generalises RAII to any cleanup action, without requiring a new class per resource.

C++26 standardises `std::scope_exit` / `std::scope_fail` / `std::scope_success` (formerly Library Fundamentals TS v3). Until those are available, a small template is enough:

```
template <typename F>
class ScopeGuard {
public:
    explicit ScopeGuard(F f) : f_(std::move(f)) {}
    ~ScopeGuard() { if (active_) f_(); }
    void dismiss() noexcept { active_ = false; }
    ScopeGuard(const ScopeGuard &) = delete;
    ScopeGuard &operator=(const ScopeGuard &) = delete;
private:
    F f_;
    bool active_ = true;
};

template <typename F>
ScopeGuard<F> make_scope_guard(F f) { return ScopeGuard<F>(std::move(f)); }
```

Rollback on early return:

```
void transfer(Account &from, Account &to, int amount) {
    from.debit(amount);
    auto rollback = make_scope_guard([&] { from.credit(amount); });
    to.credit(amount);    // may throw - rollback runs
    rollback.dismiss();   // commit reached; skip the rollback
}
```

std::unique_ptr with custom deleter as a scope guard:

A `std::unique_ptr` with a custom deleter is a ready-made scope guard — the “pointer” can be any sentinel and the deleter is the cleanup:

```
auto deleter = [](void *) { /* cleanup */ };
std::unique_ptr<void, decltype(deleter)> guard(
    reinterpret_cast<void *>(uintptr_t{1}), deleter);
```

This pattern is especially useful for wrapping C APIs with free-function cleanup (e.g., `std::unique_ptr<FILE, decltype(&std::fclose)>`). See *Smart Pointers* for more on custom deleters.

Two-Phase Initialization: Why RAII Avoids It

Source

`src/raii/two-phase-init`

Some older C++ code — and code translated from languages without exceptions — uses *two-phase initialization*: a default constructor that does almost nothing, followed by a separate `init()` call that does the real acquisition and returns a success flag. RAII replaces this pattern.

The two-phase object has a zombie state between construction and `init()` where it exists but is not usable. Every method must check an `initialized_` flag, callers must remember to call `init()`, and there is no type-system guarantee that a constructed object is a usable object:

```
class TwoPhase {
public:
    TwoPhase() = default;           // produces a zombie
    bool init(int v) {              // error signalled by bool - easy to ignore
        if (v < 0) return false;
        value_ = v;
        initialized_ = true;
        return true;
    }
    int value() const {
        if (!initialized_) throw std::logic_error("not initialized");
        return value_;
    }
private:
    int value_ = 0;
    bool initialized_ = false;
};
```

The RAII equivalent makes construction all-or-nothing. If acquisition fails, the constructor throws and the object never existed; if it succeeds, every method can assume valid state:

```
class Raii {
public:
```

(continues on next page)

(continued from previous page)

```

explicit Raii(int v) : value_(v) {
    if (v < 0) throw std::invalid_argument("v must be non-negative");
}
int value() const noexcept { return value_; } // no flag check
private:
    int value_;
};

```

Why RAII wins:

- No invalid intermediate state — the type system enforces “constructed usable”.
- Errors cannot be silently ignored; an exception is hard to miss.
- Member functions do not carry an `if (!initialized_)` tax.
- Composes cleanly with containers, smart pointers, and generic code, which assume constructed objects are usable.

The historical argument against RAII was exception-handling overhead, but modern compilers make the nothrow path essentially free. Use RAII.

See Also

- *Move Semantics* — special member functions, Rule of Zero/Three/Five, value categories, move-only types, perfect forwarding, and common move semantics pitfalls.
- *Smart Pointers* — `std::unique_ptr` and `std::shared_ptr`, the standard-library RAII wrappers for heap allocations.
- *Return Value Optimization* — return value optimization and copy elision, which interact with RAII-managed return values.

1.3.3 Move Semantics**Table of Contents**

- *Move Semantics*
 - *Special Member Functions*
 - *Rule of Zero*
 - *Rule of Three*
 - *Rule of Five*
 - *Value Categories*
 - *Move-Only Types*
 - *Perfect Forwarding*
 - *Move and Destructor*
 - *Using `std::exchange`*
 - *Conditional `noexcept`*
 - *Self-Move Assignment*

- *Moved-From State*
- *Return Value Optimization and Move Semantics*
- *Emplace vs Insert*
- *Common Move Semantics Pitfalls*
- *Preventing Object Slicing*

Move semantics (introduced in C++11) enable efficient transfer of resources between objects without deep copying. Where copy operations duplicate state, move operations *steal* it—leaving the source in a valid but unspecified state. This distinction transforms what used to be expensive $O(n)$ deep copies into $O(1)$ pointer swaps, making value types practical to return from functions and store in containers.

Understanding move semantics requires four interrelated concepts: the six **special member functions** that the compiler generates or you write by hand, the **value categories** (lvalue, rvalue, and their refinements) that determine which of those functions are called, the **Rule of Zero/Three/Five** for writing correct resource-owning classes, and **perfect forwarding** (`std::forward`) for template code that must preserve value categories across function calls. This chapter covers all four, along with common pitfalls.

Special Member Functions

Source

`src/raii/constructors`

C++ provides six special member functions that control object lifecycle: default constructor, destructor, copy constructor, copy assignment operator, move constructor, and move assignment operator. Understanding when each is called is essential for implementing correct resource management.

The following example demonstrates all special member functions and the scenarios that trigger each one. Note how copy operations create independent copies while move operations transfer ownership, leaving the source in a valid but unspecified state:

```
#include <iostream>
#include <utility>

class Resource {
public:
    // Constructor: acquires resource
    Resource(int x) : x_(x) {}

    // Default constructor
    Resource() = default;

    // Copy constructor: creates independent copy
    Resource(const Resource &other) : Resource(other.x_) {
        std::cout << "copy constructor\n";
    }

    // Copy assignment: replaces with copy
    Resource &operator=(const Resource &other) {
        std::cout << "copy assignment\n";
        x_ = other.x_;
        return *this;
    }
}
```

(continues on next page)

(continued from previous page)

```

// Move constructor: transfers ownership
Resource(Resource &&other) noexcept : x_(std::move(other.x_)) {
    std::cout << "move constructor\n";
    other.x_ = 0;
}

// Move assignment: transfers ownership
Resource &operator=(Resource &&other) noexcept {
    std::cout << "move assignment\n";
    x_ = std::move(other.x_);
    return *this;
}

private:
    int x_ = 0;
};

int main(int argc, char *argv[]) {
    Resource r1;                // default constructor
    Resource r2(1);              // constructor
    Resource r3 = Resource(2);    // constructor (copy elision)
    Resource r4(r2);              // copy constructor
    Resource r5(std::move(Resource(2))); // move constructor
    Resource r6 = r1;              // copy constructor
    Resource r7 = std::move(r3);    // move constructor

    Resource r8, r9;
    r8 = r2;                      // copy assignment
    r9 = std::move(r4);            // move assignment
    r9 = Resource(2);              // move assignment
}

```

Rule of Zero

Source

`src/raii/rule-of-zero`

The **Rule of Zero** states that classes should not define custom destructors, copy/move constructors, or copy/move assignment operators. Instead, they should rely on member objects that manage their own resources (like `std::string`, `std::vector`, or smart pointers). This approach minimizes boilerplate code and reduces the risk of resource management bugs.

When all members handle their own cleanup, the compiler-generated special member functions work correctly by default:

```

#include <iostream>
#include <string>

class Document {
public:
    Document(const std::string &content) : content_(content) {}

```

(continues on next page)

(continued from previous page)

```

// No destructor, copy/move constructors, or assignment operators needed.
// The compiler-generated versions correctly handle std::string member.

friend std::ostream &operator<<(std::ostream &os, const Document &doc);

private:
    std::string content_;
};

std::ostream &operator<<(std::ostream &os, const Document &doc) {
    return os << doc.content_;
}

int main(int argc, char *argv[]) {
    Document doc("Rule of Zero");
    std::cout << doc << "\n";
}

```

Rule of Three

Source

[src/raii/rule-of-three](#)

The **Rule of Three** (C++98) states that if a class requires a user-defined destructor, copy constructor, or copy assignment operator, it almost certainly requires all three. This rule applies when a class manages resources that are not automatically handled by its members (e.g., raw pointers, file handles, network connections).

What happens without move semantics? When you define any of the Big Three, the compiler will NOT generate move constructor or move assignment operator. Instead, move operations fall back to copy operations. This is safe but inefficient—moving an object performs a deep copy instead of transferring ownership. For classes managing expensive resources (large buffers, file handles), this overhead matters. See [Rule of Five](#) for the efficient C++11 solution.

Failing to implement all three leads to resource leaks or double-free bugs. In this example, the class manages a dynamically allocated character array:

```

#include <cstring>
#include <iostream>
#include <memory>
#include <string>

class Buffer {
public:
    Buffer(const char *data, size_t size) : data_(new char[size]), size_(size) {
        std::memcpy(data_, data, size);
    }

    // 1. User-defined destructor: releases resource
    ~Buffer() { delete[] data_; }

    // 2. User-defined copy constructor: deep copy
    Buffer(const Buffer &other) : Buffer(other.data_, other.size_) {}

    // 3. User-defined copy assignment: exception-safe deep copy

```

(continues on next page)

(continued from previous page)

```

Buffer &operator=(const Buffer &other) {
    if (this == std::addressof(other)) {
        return *this;
    }
    char *new_data = new char[other.size_]; // Allocate first
    std::memcpy(new_data, other.data_, other.size_);
    delete[] data_; // Then release old resource
    data_ = new_data;
    size_ = other.size_;
    return *this;
}

friend std::ostream &operator<<(std::ostream &os, const Buffer &buf);

private:
    char *data_;
    size_t size_;
};

std::ostream &operator<<(std::ostream &os, const Buffer &buf) {
    return os << buf.data_;
}

int main(int argc, char *argv[]) {
    std::string s = "Rule of Three";
    Buffer buf(s.c_str(), s.size() + 1);
    std::cout << buf << "\n";
}

```

Rule of Five

Source

[src/raii/rule-of-five](#)

The **Rule of Five** (C++11) extends the Rule of Three to include move semantics. If a class defines any of the five special member functions (destructor, copy constructor, copy assignment, move constructor, move assignment), it should explicitly define or delete all five.

Move operations enable efficient transfer of resources without copying—ownership is transferred in $O(1)$ instead of $O(n)$ deep copy. The `std::exchange` utility is particularly useful for implementing move constructors, as it atomically replaces a value and returns the old one:

```

#include <cstring>
#include <iostream>
#include <memory>
#include <string>
#include <utility>

class Buffer {
public:
    Buffer(const char *data, size_t size) : data_(new char[size]), size_(size) {
        std::memcpy(data_, data, size);
    }
}

```

(continues on next page)

(continued from previous page)

```

// 1. Destructor
~Buffer() { delete[] data_; }

// 2. Copy constructor
Buffer(const Buffer &other) : Buffer(other.data_, other.size_) {}

// 3. Move constructor: transfers ownership using std::exchange
Buffer(Buffer &&other) noexcept
    : data_(std::exchange(other.data_, nullptr)),
      size_(std::exchange(other.size_, 0)) {}

// 4. Copy assignment: copy-and-swap idiom
Buffer &operator=(const Buffer &other) {
    return *this = Buffer(other);
}

// 5. Move assignment: swap resources
Buffer &operator=(Buffer &&other) noexcept {
    std::swap(data_, other.data_);
    std::swap(size_, other.size_);
    return *this;
}

friend std::ostream &operator<<(std::ostream &os, const Buffer &buf);

private:
    char *data_ = nullptr;
    size_t size_ = 0;
};

std::ostream &operator<<(std::ostream &os, const Buffer &buf) {
    return os << buf.data_;
}

int main(int argc, char *argv[]) {
    std::string s = "Rule of Five";
    Buffer buf(s.c_str(), s.size() + 1);
    std::cout << buf << "\n";
}

```

When to use which rule:

Rule	When to Use	Example
Rule of Zero	No custom destructor or special member functions needed	<code>std::string</code> , <code>std::vector</code> , smart pointers as members
Rule of Three	Custom destructor required (pre-C++11)	Raw pointers, C file handles
Rule of Five	Custom destructor required (C++11+)	Same as Rule of Three, with move support

Value Categories

Source

[src/raii/value-categories](#)

Understanding value categories is essential for mastering move semantics. C++11 introduced a refined taxonomy that determines when move operations are invoked versus copy operations. Every expression in C++ has both a type and a value category. The value category determines whether the expression can be moved from, whether it has identity (can take its address), and which overloaded function will be selected during overload resolution. This classification is fundamental to understanding why `std::move` works and when move constructors are called.

The five value categories:

Category	Has Identity	Description
lvalue	Yes	Has a name or address; can appear on left side of assignment
xvalue	Yes	“eXpiring value”; about to be moved from (e.g., <code>std::move(x)</code>)
prvalue	No	“Pure rvalue”; temporary with no name (e.g., <code>42</code> , <code>Foo()</code>)
glvalue	Yes	Generalized lvalue (lvalue or xvalue)
rvalue	•	Can be moved from (xvalue or prvalue)

What `std::move` actually does:

`std::move` is an unconditional cast to rvalue reference—it does not move anything by itself. The actual move happens when a move constructor or assignment is invoked. See [Common Move Semantics Pitfalls](#) for details and examples.

Move-Only Types

Source

[src/raii/move-only](#)

Some types represent unique ownership of a resource and should never be copied—only moved. Examples include `std::unique_ptr`, `std::thread`, `std::fstream`, and custom file handles. These types enforce exclusive ownership semantics: at any point in time, exactly one object owns the resource. Copying would violate this invariant by creating two owners, leading to double-free bugs or resource conflicts. Implement move-only types by explicitly deleting copy operations while providing move operations:

```
#include <cstdio>
#include <utility>

class UniqueFile {
public:
    explicit UniqueFile(const char *path)
        : handle_(std::fopen(path, "w")) {}

    ~UniqueFile() {
        if (handle_) std::fclose(handle_);
    }

    // Delete copy operations
    UniqueFile(const UniqueFile &) = delete;
    UniqueFile &operator=(const UniqueFile &) = delete;

    // Provide move operations
```

(continues on next page)

(continued from previous page)

```

UniqueFile(UniqueFile &&other) noexcept
    : handle_(std::exchange(other.handle_, nullptr)) {}

UniqueFile &operator=(UniqueFile &&other) noexcept {
    if (this != &other) {
        if (handle_) std::fclose(handle_);
        handle_ = std::exchange(other.handle_, nullptr);
    }
    return *this;
}

std::FILE *get() const { return handle_; }

private:
    std::FILE *handle_ = nullptr;
};

int main() {
    UniqueFile f1("/tmp/test.txt");
    // UniqueFile f2 = f1;           // Error: copy deleted
    UniqueFile f2 = std::move(f1);  // OK: move
}

```

Perfect Forwarding

Source

[src/raii/forwarding](#)

Perfect forwarding preserves the value category (lvalue or rvalue) of arguments when passing them through template functions. Without perfect forwarding, a named parameter inside a function is always an lvalue (because it has a name), even if the original argument was an rvalue. This causes unnecessary copies when the forwarded argument should have been moved. Perfect forwarding is essential for implementing generic wrapper functions, factory functions like `std::make_unique`, and decorator patterns that need to forward arguments to other functions without losing efficiency or changing semantics.

Universal references (forwarding references):

In template contexts, `T&&` is a *universal reference* (also called *forwarding reference* in C++17 terminology), not an rvalue reference. This special deduction rule only applies when `T` is a deduced template parameter. Universal references bind to both lvalues and rvalues through reference collapsing rules, making them the foundation of perfect forwarding:

```

template <typename T>
void wrapper(T&& arg); // Universal reference, not rvalue reference

int x = 42;
wrapper(x);           // T deduced as int&, T&& becomes int& (lvalue)
wrapper(42);          // T deduced as int, T&& becomes int&& (rvalue)

```

Reference collapsing rules:

When references to references are formed (through template instantiation or typedef), they collapse according to these rules. The key insight is that any reference involving an lvalue reference collapses to an lvalue reference:


```
T& &    → T&
T& &&   → T&
T&& &   → T&
T&& &&  → T&&
```

See *C++ Basics* for a fuller treatment of template type deduction that produces these collapsing cases.

Implementation of `std::forward`:

`std::forward<T>` is a conditional cast: it casts to rvalue reference only if `T` is not an lvalue reference type. This preserves the original value category of the argument through the forwarding chain. Unlike `std::move` which always casts to rvalue, `std::forward` examines the template parameter to decide. The standard library's `std::forward` uses `static_cast` combined with reference collapsing. The first overload handles lvalue arguments: when `T` is an lvalue reference type, `T&&` collapses to an lvalue reference. The second overload handles rvalue arguments and includes a static assertion to prevent forwarding an rvalue as an lvalue:

```
#include <type_traits>

template <typename T>
T&& forward(std::remove_reference_t<T>& t) noexcept {
    return static_cast<T&&>(t);
}

template <typename T>
T&& forward(std::remove_reference_t<T>&& t) noexcept {
    static_assert(!std::is_lvalue_reference_v<T>);
    return static_cast<T&&>(t);
}
```

Using `std::forward`:

```
#include <iostream>
#include <utility>

void process(int &x) { std::cout << "lvalue: " << x << "\n"; }
void process(int &&x) { std::cout << "rvalue: " << x << "\n"; }

template <typename T>
void wrapper(T &&arg) {
    process(std::forward<T>(arg)); // Preserves value category
}

int main() {
    int x = 42;
    wrapper(x);           // Calls process(int&)
    wrapper(123);         // Calls process(int&&)
}
```

Wrapper function:

This example demonstrates a wrapper function that forwards arguments to a callable while preserving their value category. When `d` (an lvalue) is passed, `T` deduces to `Data&`, and `std::forward` returns an lvalue reference. When `std::move(t)` is passed, `T` deduces to `Data`, and `std::forward` returns an rvalue reference:

```
#include <iostream>
#include <utility>

template <typename T, typename Func>
void wrapper(T&& arg, Func fn) {
    fn(std::forward<T>(arg));
}

struct Data { int x, y, result; };

int main() {
    Data d{1, 2, 0};
    wrapper(d, [](Data& d) { d.result = d.x + d.y; });
    std::cout << d.result << "\n"; // 3

    wrapper(Data{5, 6, 0}, [](Data&& d) {
        std::cout << d.x * d.y << "\n"; // 30
    });
}
```

Decorator pattern:

A timing decorator that wraps any callable and measures execution time while perfectly forwarding all arguments to the wrapped function:

```
#include <iostream>
#include <utility>
#include <chrono>

template <typename Func, typename ...Args>
auto timed(Func &&f, Args&&... args) {
    auto start = std::chrono::system_clock::now();
    auto ret = f(std::forward<Args>(args)...);
    std::chrono::duration<double> d = std::chrono::system_clock::now() - start;
    std::cout << "Time: " << d.count() << "s\n";
    return ret;
}

long fib(long n) { return n < 2 ? n : fib(n-1) + fib(n-2); }

int main() { timed(fib, 35); }
```

Factory pattern:

A generic factory function that constructs objects by forwarding arguments to constructors, preserving value categories for efficient object creation:

```
#include <memory>
#include <utility>

template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

(continues on next page)

(continued from previous page)

```

struct Widget {
    Widget(int x, double y) : x_(x), y_(y) {}
    int x_;
    double y_;
};

int main() {
    auto w = make_unique<Widget>(42, 3.14); // Perfect forwarding
}

```

Move and Destructor

Source

[src/raii/move-leak](#)

A critical detail: moving from an object does **not** call its destructor. The moved-from object remains alive until it goes out of scope. If the move constructor fails to properly reset the source object's state, the destructor may attempt to release resources that were already transferred, causing double-free bugs—or worse, the moved-from object may still hold resources that leak.

Problematic move constructor (causes resource leak):

```

#include <cstdio>
#include <utility>

class File {
public:
    explicit File(const char *path) : handle_(std::fopen(path, "w")) {}

    ~File() {
        if (handle_) std::fclose(handle_);
    }

    // Bug: does not reset other.handle_ to nullptr
    File(File &&other) noexcept : handle_(other.handle_) {
        // other.handle_ still points to the file!
    }

private:
    std::FILE *handle_;
};

int main() {
    File f1("/tmp/test.txt");
    File f2(std::move(f1));
    // When f1's destructor runs, it closes the file
    // When f2's destructor runs, it closes the same file again -> double-free!
}

```

Correct move constructor:

Always reset the moved-from object to a valid empty state. Using `std::exchange` makes this pattern concise and less error-prone:

```
// Correct: reset source to nullptr
File(File &&other) noexcept : handle_(std::exchange(other.handle_, nullptr)) {}
```

The moved-from object's destructor will still run, but with `handle_` set to `nullptr`, the conditional check if (`handle_`) prevents any invalid operation.

Using `std::exchange`

Source

`src/raii/move-exchange`

`std::exchange` (C++14) atomically replaces a value and returns the old one, making it ideal for implementing move operations. It eliminates the risk of forgetting to reset the source object and produces cleaner, more readable code.

Complete example with `std::exchange`:

```
#include <cstdio>
#include <utility>

class File {
public:
    explicit File(const char *path) : handle_(std::fopen(path, "w")) {}

    ~File() {
        if (handle_) std::fclose(handle_);
    }

    // Move constructor using std::exchange
    File(File &&other) noexcept
        : handle_(std::exchange(other.handle_, nullptr)) {}

    // Move assignment using std::exchange
    File &operator=(File &&other) noexcept {
        if (this != &other) {
            if (handle_) std::fclose(handle_); // Release current resource
            handle_ = std::exchange(other.handle_, nullptr);
        }
        return *this;
    }

    // Non-copyable
    File(const File &) = delete;
    File &operator=(const File &) = delete;

private:
    std::FILE *handle_ = nullptr;
};
```

Alternative: swap-based move assignment:

Another common pattern uses `std::swap`, which is exception-safe and handles self-assignment automatically:

```
File &operator=(File &&other) noexcept {
    std::swap(handle_, other.handle_);
    return *this;
}
```

(continues on next page)

(continued from previous page)

```
// other's destructor will close our old handle
}
```

Conditional noexcept

Source

src/raii/conditional-noexcept

Move operations should be `noexcept` whenever possible. This is critical for performance because the standard library containers make optimization decisions based on the `noexcept` specification. Specifically, `std::vector` uses move operations during reallocation only if they are `noexcept`; otherwise, it falls back to copying for exception safety. The reason is that if a move operation throws mid-reallocation, the vector would be left in an inconsistent state with some elements moved and others not. By requiring `noexcept`, the vector can safely use moves knowing they won't throw.

Why noexcept matters:

```
#include <vector>

struct Widget {
    Widget(Widget &&) noexcept { /* ... */ } // std::vector will move
};

struct Gadget {
    Gadget(Gadget &&) { /* ... */ } // std::vector will copy (not noexcept)
};
```

Conditional noexcept:

When a class contains members, the move operation's exception specification should depend on the members' move operations. A wrapper class should be `noexcept` if and only if its contained type's move is `noexcept`. Use `noexcept(noexcept(...))` or type traits to conditionally propagate exception specifications. This ensures that your wrapper types work efficiently with standard containers when possible:

```
#include <type_traits>
#include <utility>

template <typename T>
class Wrapper {
public:
    Wrapper(Wrapper &&other) noexcept(std::is_nothrow_move_constructible_v<T>)
        : data_(std::move(other.data_)) {}

private:
    T data_;
};
```

Checking move operation properties:

The `<type_traits>` header provides compile-time checks for move operation properties. Use these to verify your types meet the requirements for efficient container operations:

```
#include <string>
#include <type_traits>
```

(continues on next page)

(continued from previous page)

```
static_assert(std::is_nothrow_move_constructible_v<std::string>);
static_assert(std::is_nothrow_move_assignable_v<std::string>);
```

Self-Move Assignment

Source

src/raii/self-move

Self-move assignment (`x = std::move(x)`) is rare in practice but can occur through aliasing, particularly when working with pointers or references. While the standard library guarantees that self-move leaves objects in a valid but unspecified state, user-defined types must explicitly handle this case to avoid undefined behavior. A naive implementation that deletes resources before checking for self-assignment will corrupt the object.

Problematic implementation:

```
class Buffer {
public:
    Buffer &operator=(Buffer &&other) noexcept {
        delete[] data_;
        data_ = std::exchange(other.data_, nullptr); // Bug if this == &other
        return *this;
    }
private:
    char *data_;
};

Buffer b;
b = std::move(b); // Undefined behavior: data_ deleted then set to nullptr
```

Correct implementations:

Option 1: Explicit self-check:

The straightforward approach adds a self-assignment check before modifying state:

```
Buffer &operator=(Buffer &&other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = std::exchange(other.data_, nullptr);
    }
    return *this;
}
```

Option 2: Swap-based (handles self-move automatically):

The swap idiom handles self-move naturally because swapping an object with itself is a no-op. This approach is also exception-safe and often results in cleaner code:

```
Buffer &operator=(Buffer &&other) noexcept {
    std::swap(data_, other.data_);
    return *this;
    // other's destructor cleans up our old data
}
```

The swap-based approach is preferred because it handles self-move naturally without explicit checks and is exception-safe.

Moved-From State

Source

src/raii/moved-from-state

After an object is moved from, it must remain in a *valid but unspecified state*. This is a crucial invariant that allows moved-from objects to be safely destroyed or reassigned. The “valid” part means the object’s class invariants are maintained enough for the destructor to run correctly. The “unspecified” part means you cannot rely on any particular value—the object might be empty, might retain its old value, or might be in some other consistent state. This guarantee applies to standard library types and should be maintained by user-defined types.

What operations are valid on moved-from objects:

1. The object’s destructor can be safely called
2. The object can be assigned a new value
3. Other operations have unspecified behavior (don’t rely on them)

Standard library guarantees:

```
#include <string>
#include <vector>

std::string s1 = "hello";
std::string s2 = std::move(s1);

// Valid operations on s1:
s1.~basic_string(); // OK: destructor
s1 = "world";       // OK: assignment

// Unspecified behavior (avoid):
// s1.size()         // Unspecified result
// s1[0]             // Undefined behavior if empty
```

Implementing moved-from state:

For user-defined types, ensure moved-from objects are in a valid state that can be safely destroyed. The simplest approach is to reset all resource handles to null or zero, which the destructor already handles:

```
class Resource {
public:
    Resource(Resource &&other) noexcept
        : ptr_(std::exchange(other.ptr_, nullptr)),
          size_(std::exchange(other.size_, 0)) {
        // other is now in valid empty state
    }

    ~Resource() {
        delete[] ptr_; // Safe even if ptr_ is nullptr
    }

private:
    int *ptr_ = nullptr;
    size_t size_ = 0;
};
```

See *Common Move Semantics Pitfalls* for examples of incorrect usage patterns.

Return Value Optimization and Move Semantics

Source

`src/rvo/no-move-return`

When returning objects from functions, the compiler applies optimizations that can eliminate both copy and move operations entirely. Understanding the interaction between RVO, NRVO, and move semantics is crucial for writing efficient code. In many cases, the compiler constructs the return value directly in the caller's memory, bypassing both copy and move constructors. This optimization is so important that C++17 made it mandatory for certain cases (guaranteed copy elision).

Guaranteed copy elision (C++17):

When returning a prvalue (temporary), C++17 guarantees copy elision—no copy or move constructor is called. The object is constructed directly in the caller's storage:

```
Widget make_widget() {  
    return Widget(42); // Guaranteed elision: constructed directly in caller  
}
```

Named Return Value Optimization (NRVO):

When returning a named local variable, NRVO is an optional optimization that most compilers implement. However, it cannot apply in all cases (e.g., multiple return paths with different variables). If NRVO doesn't apply, the compiler uses the move constructor if available, otherwise falls back to copy:

```
Widget make_widget(bool flag) {  
    Widget w1(1);  
    Widget w2(2);  
    return flag ? w1 : w2; // NRVO cannot apply (multiple return paths)  
                          // Falls back to move constructor  
}
```

Don't use `std::move` on return:

A common mistake is using `std::move` when returning local variables. This is a pessimization because it prevents RVO/NRVO and forces a move operation when elision would have eliminated the operation entirely. The compiler already treats returned local variables as rvalues when move is needed:

```
// Bad: prevents RVO  
Widget make_widget() {  
    Widget w(42);  
    return std::move(w); // Pessimization!  
}  
  
// Good: allows RVO  
Widget make_widget() {  
    Widget w(42);  
    return w; // RVO or move (compiler decides)  
}
```

For comprehensive coverage of RVO, NRVO, and copy elision, see [Return Value Optimization](#).

Emplace vs Insert

Source

src/raii/emplace

Container operations like `emplace_back` and `emplace` construct elements in-place, avoiding temporary objects and unnecessary moves/copies. This is particularly beneficial for types that are expensive to construct.

`push_back` vs `emplace_back`:

```
#include <vector>
#include <string>

std::vector<std::string> v;

// push_back: creates temporary, then moves it into vector
v.push_back(std::string("hello")); // 1 construction + 1 move

// emplace_back: constructs directly in vector's storage
v.emplace_back("hello");           // 1 construction only
```

When `push_back` copies vs moves:

`push_back` copies when given an lvalue, moves when given an rvalue:

```
std::vector<std::string> v;
std::string s = "hello";

v.push_back(s);           // lvalue: COPIES s into vector
v.push_back(std::move(s)); // rvalue: MOVES s into vector
v.push_back("world");     // rvalue (temporary): MOVES into vector
```

Expensive types benefit most:

For types with costly constructors, avoiding the temporary + move overhead matters:

```
std::vector<std::vector<int>> vv;

// emplace_back: constructs 1000-element vector directly in vv
vv.emplace_back(1000, 0);

// push_back equivalent would be:
// vv.push_back(std::vector<int>(1000, 0)); // construct + move
```

Map `emplace` avoids pair construction:

```
std::map<std::string, std::string> m;

// emplace: constructs pair<string,string> in-place
m.emplace("key", "value");

// insert equivalent would be:
// m.insert(std::make_pair("key", "value")); // construct pair + move
```

Non-copyable types:

For move-only types, `emplace` constructs in-place without requiring a copy:

```

struct NonCopyable {
    int value;
    explicit NonCopyable(int v) : value(v) {}
    NonCopyable(const NonCopyable&) = delete;
    NonCopyable(NonCopyable&&) noexcept = default;
};

std::vector<NonCopyable> v;
v.emplace_back(42); // OK: constructs in-place

```

Caveat: emplace can hide bugs:

emplace uses perfect forwarding, which can lead to unexpected implicit conversions that push_back would catch:

```

std::vector<std::vector<int>> vv;

vv.push_back(10); // Error: no conversion from int to vector<int>
vv.emplace_back(10); // Compiles! Creates vector<int>(10) - probably not intended

```

Common Move Semantics Pitfalls**Source**

[src/raii/move-pitfalls](#)

Move semantics introduce several subtle pitfalls that can lead to bugs or performance issues. This section catalogs the most common mistakes developers make when working with move operations.

1. std::move doesn't move:

The most common misconception is that std::move performs a move. It doesn't—it's just an unconditional cast to rvalue reference. The actual move happens when a move constructor or move assignment is invoked with the result:

```

std::string s1 = "hello";
std::move(s1); // Does nothing!
std::string s2 = std::move(s1); // Now s1 is moved

```

2. Moving from const objects copies:

Move operations require modifying the source object, so they cannot work with const objects. When you try to move from a const object, overload resolution selects the copy constructor instead, silently performing a copy:

```

const std::string s1 = "hello";
std::string s2 = std::move(s1); // Calls copy constructor, not move

```

3. Accidentally using moved-from objects:

After moving from an object, it's in a valid but unspecified state. Using it (other than destruction or assignment) leads to unspecified or undefined behavior:

```

std::vector<int> v1 = {1, 2, 3};
std::vector<int> v2 = std::move(v1);

for (int x : v1) { /* ... */ } // Bug: v1 is in unspecified state

v1 = {4, 5, 6}; // OK: assign new value first
for (int x : v1) { /* ... */ } // Now safe

```

4. Forgetting noexcept on move operations:

Without `noexcept`, standard containers fall back to copying during reallocation for exception safety. This can cause significant performance degradation:

```

struct Widget {
    Widget(Widget &&) { /* ... */ } // Missing noexcept
};

std::vector<Widget> v;
v.push_back(Widget());
// Vector reallocation will copy instead of move!

```

5. Not resetting moved-from state:

Failing to reset the source object's state in move operations leads to double-free bugs when both objects' destructors run:

```

class Resource {
public:
    Resource(Resource &&other) noexcept : ptr_(other.ptr_) {
        // Bug: forgot to set other.ptr_ = nullptr
    }
    ~Resource() { delete ptr_; } // Double-free!
private:
    int *ptr_;
};

```

6. Self-move without protection:

Self-move through aliasing (e.g., `x = std::move(*ptr)` where `ptr == &x`) can corrupt objects if not handled. Use explicit checks or the swap idiom:

```

Buffer &operator=(Buffer &&other) noexcept {
    delete[] data_;
    data_ = other.data_; // Bug if this == &other
    return *this;
}

```

Preventing Object Slicing

Source

`src/raii/polymorphic`

Polymorphic base classes should suppress public copy and move operations to prevent *object slicing*. Slicing occurs when a derived class object is copied into a base class object, losing the derived class's data and behavior.

Problematic code (allows slicing):

```

#include <iostream>
#include <string>

class Animal {
public:
    virtual std::string speak() { return "..."; }
};

```

(continues on next page)

(continued from previous page)

```

class Dog : public Animal {
public:
    std::string speak() override { return "woof"; }
};

void process(Animal &a) {
    auto copy = a; // Slicing! Only Animal part is copied
    std::cout << copy.speak() << "\n"; // Prints "..." not "woof"
}

int main(int argc, char *argv[]) {
    Dog dog;
    process(dog); // Output: "..." (sliced!)
}

```

Correct approach (prevents slicing):

By deleting copy operations in the base class, the compiler prevents accidental slicing at compile time:

```

#include <iostream>
#include <string>

class Animal {
public:
    Animal() = default;
    Animal(const Animal &) = delete;
    Animal &operator=(const Animal &) = delete;
    virtual ~Animal() = default;

    virtual std::string speak() { return "..."; }
};

class Dog : public Animal {
public:
    std::string speak() override { return "woof"; }
};

void process(Animal &a) {
    // auto copy = a; // Compile error: copy constructor deleted
    std::cout << a.speak() << "\n";
}

int main(int argc, char *argv[]) {
    Dog dog;
    process(dog); // Output: "woof"
}

```

1.3.4 String

Table of Contents

- *String*
 - *Character to String Conversion*
 - *C String to std::string*
 - *Substring Extraction*
 - *String Splitting*
 - *String Joining*
 - *Case Conversion*
 - *String Concatenation Performance*
 - *String Literals*
 - *std::string_view*

Character to String Conversion

Source

src/string/char-to-string

Converting a single character to a `std::string` is a common operation in C++. The language provides several approaches, each with different semantics and use cases. Understanding these options helps write clearer, more intentional code.

Using the fill constructor:

The `std::string` constructor `string(size_t n, char c)` creates a string containing `n` copies of character `c`. This is the most explicit approach when you need to create a string from a single character, as it clearly communicates the intent:

```
#include <string>

int main(int argc, char *argv[]) {
    std::string s(1, 'a'); // Creates "a"
}
```

Using the append operator:

The `+=` operator appends a character to an existing string. This approach is useful when building strings incrementally or when you already have a string object that needs modification:

```
#include <string>

int main(int argc, char *argv[]) {
    std::string s;
    s += 'a'; // s is now "a"
}
```

Using the assignment operator:

Direct assignment replaces the string's contents with a single character. This is concise but less explicit about the intent compared to the constructor approach:

```
#include <string>

int main(int argc, char *argv[]) {
    std::string s;
    s = 'a'; // s is now "a"
}
```

C String to std::string

Source

[src/string/cstr-to-string](#)

Converting C-style strings (null-terminated character arrays) to `std::string` is straightforward due to implicit conversion. The `std::string` class has a constructor that accepts a `const char*`, which handles the conversion automatically. This makes interoperability between C and C++ code seamless:

```
#include <string>

int main(int argc, char *argv[]) {
    char cstr[] = "hello cstr";
    std::string s = cstr; // Implicit conversion
}
```

Substring Extraction

Source

[src/string/substr](#)

The `substr(pos, len)` member function extracts a portion of a string starting at position `pos` with length `len`. If `len` is omitted or exceeds the remaining characters, it extracts until the end of the string:

```
#include <iostream>
#include <string>

int main(int argc, char *argv[]) {
    std::string s = "Hello World";

    std::cout << s.substr(0, 5) << "\n"; // Output: Hello
    std::cout << s.substr(6) << "\n";   // Output: World
    std::cout << s.substr(6, 3) << "\n"; // Output: Wor
}
```

String Splitting

Source

[src/string/split](#)

Splitting strings by a delimiter is a fundamental text processing operation. Unlike some languages, C++ does not provide a built-in split function in the standard library. However, several approaches exist using standard library components.

Using find and substr:

This approach manually searches for the delimiter and extracts substrings. It modifies a copy of the input string by erasing processed portions. While verbose, this method provides full control over the splitting logic:

```

#include <iostream>
#include <string>
#include <vector>

std::vector<std::string> split(const std::string &str, char delim) {
    std::string s = str;
    std::vector<std::string> out;
    size_t pos = 0;

    while ((pos = s.find(delim)) != std::string::npos) {
        out.emplace_back(s.substr(0, pos));
        s.erase(0, pos + 1);
    }
    out.emplace_back(s);
    return out;
}

int main(int argc, char *argv[]) {
    auto v = split("abc,def,ghi", ',');
    for (const auto &c : v) {
        std::cout << c << "\n";
    }
}

```

Using std::getline:

The `std::getline` function can read from a string stream using a custom delimiter. This approach is concise and leverages the standard library's stream facilities. It handles edge cases like empty tokens correctly:

```

#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main(int argc, char *argv[]) {
    std::string in = "abc,def,ghi";
    std::vector<std::string> out;
    std::string token;
    std::istringstream stream(in);

    while (std::getline(stream, token, ',')) {
        out.emplace_back(token);
    }

    for (const auto &c : out) {
        std::cout << c << "\n";
    }
}

```

String Joining

Source

src/string/join

Joining strings with a delimiter is the inverse of splitting. C++ does not provide a built-in join function, but it can be implemented using iterators and `std::ostringstream` or simple string concatenation.

Using `ostringstream`:

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

std::string join(const std::vector<std::string> &v, char delim) {
    std::ostringstream oss;
    for (size_t i = 0; i < v.size(); ++i) {
        if (i > 0) oss << delim;
        oss << v[i];
    }
    return oss.str();
}

int main(int argc, char *argv[]) {
    std::vector<std::string> v = {"abc", "def", "ghi"};
    std::cout << join(v, ',') << "\n"; // Output: abc,def,ghi
}
```

Case Conversion

Source

src/string/case-conversion

Converting strings to uppercase or lowercase requires transforming each character individually. The `std::transform` algorithm combined with `::toupper` or `::tolower` from `<cctype>` provides an efficient, in-place solution. Note that these functions operate on individual characters and assume ASCII encoding:

```
#include <algorithm>
#include <iostream>
#include <string>

int main(int argc, char *argv[]) {
    std::string s = "Hello World";

    // Convert to uppercase
    std::transform(s.begin(), s.end(), s.begin(), ::toupper);
    std::cout << s << "\n"; // Output: HELLO WORLD

    // Convert to lowercase
    std::transform(s.begin(), s.end(), s.begin(), ::tolower);
    std::cout << s << "\n"; // Output: hello world
}
```


String Concatenation Performance

Source

src/string/concat-perf

String concatenation performance varies significantly depending on where characters are added. Appending to the end of a string is an amortized $O(1)$ operation due to `std::string`'s internal buffer management. However, prepending requires shifting all existing characters, making it $O(n)$ per operation.

This example demonstrates the performance difference. Appending 100,000 characters completes nearly instantly, while prepending the same number takes significantly longer. Even with `reserve()`, prepending remains slow because the data must still be shifted:

```
#include <chrono>
#include <iostream>
#include <string>

constexpr int total = 100000;

template <typename F>
void profile(const char *label, F &&func) {
    const auto start = std::chrono::steady_clock::now();
    func();
    const auto end = std::chrono::steady_clock::now();
    const auto ms =
        std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << label << ": " << ms.count() << " ms\n";
}

int main(int argc, char *argv[]) {
    profile("Append", [] {
        std::string s;
        for (int i = 0; i < total; ++i) {
            s += 'a';
        }
    });

    profile("Prepend", [] {
        std::string s;
        for (int i = 0; i < total; ++i) {
            s = std::string(1, 'a') + s;
        }
    });

    profile("Prepend with reserve", [] {
        std::string s;
        s.reserve(total + 1);
        for (int i = 0; i < total; ++i) {
            s = std::string(1, 'a') + s;
        }
    });
}
```

String Literals

Source

[src/string/literals](#)

C++14 and C++17 introduced user-defined literals for string types, providing a more expressive way to create strings. The `s` suffix creates a `std::string`, while `sv` creates a `std::string_view`. These literals are defined in the `std::literals` namespace and help avoid ambiguity between C strings and C++ string types:

```
#include <iostream>
#include <string>
#include <string_view>

int main(int argc, char *argv[]) {
    using namespace std::literals;

    auto s1 = "c string";           // const char*
    auto s2 = "std::string"s;       // std::string
    auto s3 = "std::string_view"sv; // std::string_view

    std::cout << s1 << "\n";
    std::cout << s2 << "\n";
    std::cout << s3 << "\n";
}
```

std::string_view

Source

[src/string/string-view](#)

Introduced in C++17, `std::string_view` provides a non-owning reference to a character sequence. It avoids copying when you only need to read string data, making it ideal for function parameters that don't need ownership. However, care must be taken to ensure the underlying data outlives the view.

Accepting string_view parameters:

Functions that only read string data should prefer `std::string_view` parameters. This accepts both `std::string` and C strings without copying:

```
#include <iostream>
#include <string_view>

void print(std::string_view s) {
    std::cout << s << "\n";
}

int main(int argc, char *argv[]) {
    const std::string s = "foo";
    print(s);           // Works: string converts to string_view
    print("bar");       // Works: C string converts to string_view
}
```

Converting string_view to string:

A `std::string_view` cannot implicitly convert to `std::string` because this would involve a potentially expensive copy. Explicit conversion is required to make the cost visible:

```

#include <iostream>
#include <string>
#include <string_view>

void process(std::string s) {
    std::cout << s << "\n";
}

int main(int argc, char *argv[]) {
    std::string_view sv = "foo";
    // process(sv); // Error: no implicit conversion
    process(std::string(sv)); // OK: explicit conversion
}

```

Null-termination caveat:

Unlike `std::string`, a `std::string_view` is not guaranteed to be null-terminated. This is important when interfacing with C APIs that expect null-terminated strings. The `data()` member function returns a pointer to the underlying character array, but there may be no null terminator:

```

#include <cstring>
#include <iostream>
#include <string_view>

int main(int argc, char *argv[]) {
    char array[3] = {'B', 'a', 'r'}; // No null terminator
    std::string_view sv(array, sizeof(array));

    // Safe: use size() to limit iteration
    for (size_t i = 0; i < sv.size(); ++i) {
        std::cout << sv[i];
    }
    std::cout << "\n";

    // Dangerous: strlen expects null-terminated string
    // std::cout << std::strlen(sv.data()) << "\n"; // Undefined behavior
}

```

Note

When passing `std::string_view` to C APIs, first convert to `std::string` to ensure null-termination, or verify the view was created from a null-terminated source.

1.3.5 Container**Table of Contents**

- *Container*
 - *std::vector*

- *std::deque*
- *std::list*
- *std::set*
- *std::unordered_set*
- *std::map*
- *std::unordered_map*
- *Ordered vs Unordered Containers*
- *std::priority_queue*
- *Container Performance Comparison*

std::vector

Source

src/container/vector

`std::vector` is a dynamic array that provides fast random access and efficient insertion/removal at the end. It is the most commonly used container in C++ due to its cache-friendly memory layout and versatility. Elements are stored in contiguous memory, which means pointer arithmetic works and the underlying array can be passed to C APIs via `data()`.

The vector manages its own memory and automatically grows when needed. When capacity is exceeded, it allocates a new larger buffer (typically 1.5x or 2x the current capacity), copies/moves all elements, and deallocates the old buffer. This reallocation invalidates all iterators and references to elements.

Pros:

- $O(1)$ random access by index
- $O(1)$ amortized `push_back()` and `pop_back()`
- Cache-friendly contiguous memory layout
- Compatible with C APIs via `data()`
- Most memory-efficient for storing elements (no per-element overhead)

Cons:

- $O(n)$ insertion/removal at front or middle (elements must be shifted)
- Reallocation invalidates all iterators and references
- Growing may cause expensive copy/move of all elements
- `push_front()` not available (would be $O(n)$)

Initialization:

```
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v1;           // Empty vector
    std::vector<int> v2(5);        // 5 elements, value-initialized (0)
    std::vector<int> v3(5, 10);    // 5 elements, all 10
    std::vector<int> v4{1, 2, 3, 4, 5}; // Initializer list
```

(continues on next page)

(continued from previous page)

```
std::vector<int> v5(v4);           // Copy constructor
std::vector<int> v6(v4.begin(), v4.end()); // Range constructor
}
```

Element access:

```
#include <iostream>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5};

    std::cout << v[0] << "\n";      // No bounds checking
    std::cout << v.at(0) << "\n";    // Throws std::out_of_range if invalid
    std::cout << v.front() << "\n";  // First element
    std::cout << v.back() << "\n";   // Last element
    std::cout << v.data()[0] << "\n"; // Raw pointer access
}
```

Modifiers:

```
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v;

    v.push_back(1);           // Add to end
    v.emplace_back(2);        // Construct in place at end
    v.pop_back();              // Remove from end
    v.insert(v.begin(), 0);    // Insert at position
    v.erase(v.begin());        // Remove at position
    v.clear();                  // Remove all elements
}
```

Capacity management:

Understanding capacity vs size is crucial for performance. `size()` is the number of elements, while `capacity()` is the allocated storage. Use `reserve()` to pre-allocate memory when you know the approximate size to avoid repeated reallocations:

```
#include <iostream>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v;

    v.reserve(100); // Pre-allocate memory for 100 elements
    std::cout << v.capacity() << "\n"; // 100
    std::cout << v.size() << "\n";     // 0

    v.resize(50);    // Change size to 50 (value-initialized)
    std::cout << v.size() << "\n";     // 50
}
```

(continues on next page)

(continued from previous page)

```

v.shrink_to_fit(); // Request to reduce capacity to fit size
}

```

std::deque

Source

`src/container/deque`

`std::deque` (double-ended queue) provides fast insertion and removal at both ends. Unlike `std::vector`, it is implemented as a sequence of fixed-size arrays (chunks), with a central map keeping track of the chunks. This design allows $O(1)$ operations at both front and back without moving existing elements.

The deque provides random access like vector, but with slightly more overhead due to the indirection through the chunk map. Elements are not stored in fully contiguous memory, so `data()` is not available and cache performance may be slightly worse than vector for sequential access.

Pros:

- $O(1)$ `push_front()`, `pop_front()`, `push_back()`, `pop_back()`
- $O(1)$ random access by index (slightly slower than vector)
- No reallocation of existing elements when growing
- Iterators to non-erased elements remain valid after insertions at ends

Cons:

- Higher memory overhead than vector (chunk management)
- Not contiguous; no `data()` member function
- Slightly slower random access due to indirection
- $O(n)$ insertion/removal in the middle

```

#include <deque>
#include <iostream>

int main(int argc, char *argv[]) {
    std::deque<int> d{2, 3, 4};

    d.push_front(1); // Add to front: {1, 2, 3, 4}
    d.push_back(5);  // Add to back:  {1, 2, 3, 4, 5}
    d.pop_front();   // Remove front: {2, 3, 4, 5}
    d.pop_back();    // Remove back:  {2, 3, 4}

    std::cout << d.front() << "\n"; // 2
    std::cout << d.back() << "\n";  // 4
    std::cout << d[1] << "\n";      // 3 (random access)
}

```

std::list

Source

`src/container/list`

`std::list` is a doubly-linked list where each element is stored in a separate node containing the data and pointers to the previous and next nodes. This structure allows $O(1)$ insertion and removal at any position given an iterator, but sacrifices random access and cache locality.

The list is the only standard sequence container that provides `splice()`, which can transfer elements between lists in $O(1)$ time without copying or moving the actual elements. It also provides `merge()`, `sort()`, and `unique()` as member functions optimized for linked list operations.

Pros:

- $O(1)$ insertion/removal anywhere given an iterator
- Iterators and references never invalidated (except for erased elements)
- $O(1)$ `splice()` to transfer elements between lists
- Stable addresses; elements never move in memory

Cons:

- No random access; must traverse from beginning or end
- $O(n)$ to access element by index
- Poor cache locality; nodes scattered in memory
- High memory overhead (two pointers per element)
- Slower iteration than vector due to pointer chasing

```
#include <iostream>
#include <list>

int main(int argc, char *argv[]) {
    std::list<int> l{1, 2, 3};

    l.push_front(0);    // {0, 1, 2, 3}
    l.push_back(4);     // {0, 1, 2, 3, 4}

    auto it = l.begin();
    std::advance(it, 2);
    l.insert(it, 99);    // {0, 1, 99, 2, 3, 4}

    l.remove(99);        // Remove all elements equal to 99
    l.reverse();         // Reverse the list in  $O(n)$ 
    l.sort();            // Sort the list in  $O(n \log n)$ 

    // Splice: transfer elements from another list
    std::list<int> other{10, 20};
    l.splice(l.end(), other); // other is now empty

    for (const auto &x : l) {
        std::cout << x << " ";
    }
    std::cout << "\n";
}
```

std::set

Source

src/container/set

std::set is an ordered associative container that stores unique elements. It is typically implemented as a self-balancing binary search tree (red-black tree), which guarantees $O(\log n)$ time complexity for insertion, deletion, and lookup operations regardless of the input order.

The container automatically maintains elements in sorted order according to the comparison function (default is `std::less<T>`). This makes it ideal for scenarios where you need to maintain a sorted collection with fast membership testing, or when you need to perform range-based queries like finding all elements between two values.

Pros:

- Elements are always sorted, enabling efficient range queries with `lower_bound()` and `upper_bound()`
- Guaranteed $O(\log n)$ worst-case performance for all operations
- No duplicates allowed, automatically enforced
- Iterators remain valid after insertions (except for erased elements)

Cons:

- $O(\log n)$ operations are slower than $O(1)$ hash-based containers for simple lookups
- Higher memory overhead due to tree node pointers (typically 3 pointers per element)
- Poor cache locality since nodes are scattered in memory
- Requires elements to be comparable with `<` operator

```
#include <iostream>
#include <set>

int main(int argc, char *argv[]) {
    std::set<int> s{3, 1, 4, 1, 5}; // Duplicates ignored: {1, 3, 4, 5}

    s.insert(2);                  // {1, 2, 3, 4, 5}
    s.erase(3);                   // {1, 2, 4, 5}

    if (s.find(2) != s.end()) {
        std::cout << "Found 2\n";
    }

    if (s.contains(4)) {           // C++20
        std::cout << "Contains 4\n";
    }

    // Range query: find elements in [2, 5)
    auto lo = s.lower_bound(2);
    auto hi = s.lower_bound(5);
    for (auto it = lo; it != hi; ++it) {
        std::cout << *it << " "; // Output: 2 4
    }
    std::cout << "\n";
}
```


std::unordered_set**Source**`src/container/unordered-set`

`std::unordered_set` is a hash table-based container that stores unique elements. It uses a hash function to map elements to buckets, providing $O(1)$ average-case time complexity for insertion, deletion, and lookup operations.

The container does not maintain any particular order of elements. The actual order depends on the hash function and the internal bucket structure, which may change when the container is rehashed (typically when the load factor exceeds a threshold). This makes it unsuitable for scenarios requiring ordered iteration.

Pros:

- $O(1)$ average-case operations make it significantly faster than `std::set` for large datasets
- Excellent performance when hash function distributes elements evenly
- Simple membership testing with `count()` or `contains()` (C++20)

Cons:

- Elements are not sorted; iteration order is unspecified and may change
- $O(n)$ worst-case when many elements hash to the same bucket (hash collisions)
- Requires elements to be hashable (`std::hash<T>` must be defined) and equality comparable
- Higher memory usage due to hash table overhead (bucket array, load factor management)
- Iterators may be invalidated by insertions that trigger rehashing

```
#include <iostream>
#include <unordered_set>

int main(int argc, char *argv[]) {
    std::unordered_set<int> s{3, 1, 4, 1, 5};

    s.insert(2);
    s.erase(3);

    if (s.find(2) != s.end()) {
        std::cout << "Found 2\n";
    }

    // Check load factor and bucket count
    std::cout << "Load factor: " << s.load_factor() << "\n";
    std::cout << "Bucket count: " << s.bucket_count() << "\n";

    // Iteration order is unspecified
    for (const auto &x : s) {
        std::cout << x << " ";
    }
    std::cout << "\n";
}
```

std::map

Source

src/container/map

std::map is an ordered associative container that stores key-value pairs with unique keys. Like std::set, it is typically implemented as a red-black tree, providing $O(\log n)$ operations. The elements are sorted by keys according to the comparison function.

The operator[] provides convenient access but has a subtle behavior: if the key doesn't exist, it inserts a default-constructed value. Use find() or at() when you don't want to accidentally insert elements. The at() method throws std::out_of_range if the key is not found.

Pros:

- Keys are always sorted, enabling efficient range queries and ordered iteration
- Guaranteed $O(\log n)$ worst-case performance
- lower_bound() and upper_bound() enable efficient range-based access
- Stable iterators (not invalidated by insertions)

Cons:

- $O(\log n)$ operations are slower than hash-based containers for simple lookups
- operator[] silently inserts default values for missing keys
- Higher memory overhead due to tree structure
- Poor cache locality compared to contiguous containers

```
#include <iostream>
#include <map>
#include <string>

int main(int argc, char *argv[]) {
    std::map<std::string, int> m{{"apple", 1}, {"banana", 2}};

    m["cherry"] = 3;                // Insert or update
    m.insert({"date", 4});           // Insert only if key doesn't exist
    m.erase("banana");

    // Safe access without inserting
    if (auto it = m.find("apple"); it != m.end()) {
        std::cout << "apple: " << it->second << "\n";
    }

    // at() throws if key not found
    try {
        std::cout << m.at("grape") << "\n";
    } catch (const std::out_of_range &e) {
        std::cout << "Key not found\n";
    }

    for (const auto &[key, value] : m) { // Structured binding (C++17)
        std::cout << key << ": " << value << "\n"; // Sorted by key
    }
}
```

std::unordered_map

Source

src/container/unordered-map

std::unordered_map is a hash table-based container that stores key-value pairs with unique keys. It provides O(1) average-case operations, making it the fastest associative container for most use cases where ordering is not required.

The container is widely used for caching, counting occurrences, and building lookup tables. For custom key types, you must provide both a hash function (specializing std::hash<T>) and an equality operator.

Pros:

- O(1) average-case operations; typically the fastest associative container
- Excellent for frequency counting, caching, and lookup tables
- try_emplace() (C++17) avoids unnecessary constructions

Cons:

- Keys are not sorted; iteration order is unspecified
- O(n) worst-case with poor hash functions or adversarial input
- Requires hashable keys (std::hash<T>) and equality comparable
- No range queries or ordered iteration
- Rehashing can invalidate iterators and cause performance spikes

```
#include <iostream>
#include <string>
#include <unordered_map>

int main(int argc, char *argv[]) {
    std::unordered_map<std::string, int> m{{"apple", 1}, {"banana", 2}};

    m["cherry"] = 3;
    m.insert({"date", 4});
    m.erase("banana");

    // Counting pattern
    std::unordered_map<char, int> freq;
    for (char c : std::string("hello")) {
        freq[c]++;
    }
    std::cout << "l appears " << freq['l'] << " times\n";

    // Safe access
    if (auto it = m.find("apple"); it != m.end()) {
        std::cout << "apple: " << it->second << "\n";
    }

    // Iteration order is unspecified
    for (const auto &[key, value] : m) {
        std::cout << key << ": " << value << "\n";
    }
}
```

Ordered vs Unordered Containers

Feature	set/map	unordered_set/map
Implementation	Red-black tree	Hash table
Lookup	$O(\log n)$	$O(1)$ average, $O(n)$ worst
Insert/Delete	$O(\log n)$	$O(1)$ average, $O(n)$ worst
Ordered iteration	Yes	No
Key requirement	Comparable ($<$)	Hashable + equality ($==$)
Memory overhead	Tree pointers	Hash buckets

When to use ordered (set/map):

- Need sorted iteration or range queries
- Keys are not easily hashable
- Predictable worst-case performance required

When to use unordered (unordered_set/map):

- Performance is critical and dataset is large
- Don't need sorted order
- Keys have good hash distribution

std::priority_queue

Source

[src/container/priority-queue](#)

`std::priority_queue` is a container adapter that provides constant time lookup of the largest (by default) element, with $O(\log n)$ insertion and extraction. It is implemented as a binary heap, typically using `std::vector` as the underlying container.

Unlike other containers, `std::priority_queue` only provides access to the top element. You cannot iterate over elements or access them by index. The heap property ensures the largest element (according to the comparator) is always at the top, but the remaining elements are not fully sorted.

Pros:

- $O(1)$ access to the maximum (or minimum with custom comparator) element
- $O(\log n)$ insertion and removal of top element
- Efficient for scenarios requiring repeated access to extreme values
- Useful for algorithms like Dijkstra's shortest path, merge k sorted lists

Cons:

- No iteration or random access to elements
- Cannot modify elements other than removing the top
- No `clear()` member function (must pop all elements or reassign)
- Underlying container is not directly accessible

Default max-heap:

```

#include <iostream>
#include <queue>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> data{1, 5, 2, 1, 3};
    std::priority_queue<int> pq;

    for (const auto &x : data) {
        pq.push(x);
    }

    while (!pq.empty()) {
        std::cout << pq.top() << " "; // Output: 5 3 2 1 1
        pq.pop();
    }
    std::cout << "\n";
}

```

Min-heap using std::greater:

To create a min-heap, use std::greater<T> as the comparator:

```

#include <functional>
#include <iostream>
#include <queue>
#include <vector>

int main(int argc, char *argv[]) {
    std::priority_queue<int, std::vector<int>, std::greater<int>> pq;

    for (int x : {1, 5, 2, 1, 3}) {
        pq.push(x);
    }

    while (!pq.empty()) {
        std::cout << pq.top() << " "; // Output: 1 1 2 3 5
        pq.pop();
    }
    std::cout << "\n";
}

```

Custom comparator with lambda:

```

#include <iostream>
#include <queue>
#include <vector>

int main(int argc, char *argv[]) {
    auto cmp = [](int a, int b) { return a > b; }; // Min-heap
    std::priority_queue<int, std::vector<int>, decltype(cmp)> pq(cmp);

    for (int x : {1, 5, 2, 1, 3}) {
        pq.push(x);
    }
}

```

(continues on next page)

(continued from previous page)

```

}

while (!pq.empty()) {
    std::cout << pq.top() << " "; // Output: 1 1 2 3 5
    pq.pop();
}
std::cout << "\n";
}

```

Merging sorted lists:

Priority queues are useful for merging multiple sorted sequences:

```

#include <iostream>
#include <queue>
#include <vector>

int main(int argc, char *argv[]) {
    std::priority_queue<int> pq;
    std::vector<int> list1{9, 7, 8};
    std::vector<int> list2{0, 5, 3};

    for (const auto &x : list1) pq.push(x);
    for (const auto &x : list2) pq.push(x);

    while (!pq.empty()) {
        std::cout << pq.top() << " "; // Output: 9 8 7 5 3 0
        pq.pop();
    }
    std::cout << "\n";
}

```

Container Performance Comparison**Source**

[src/container/profile](#)

Different containers have different performance characteristics. The following table summarizes the time complexity of common operations:

Operation	vector	deque	list
push_back	O(1)*	O(1)	O(1)
push_front	O(n)	O(1)	O(1)
pop_back	O(1)	O(1)	O(1)
pop_front	O(n)	O(1)	O(1)
Random access	O(1)	O(1)	O(n)

* Amortized constant time

Note

See `src/container/profile` for benchmark code comparing these operations across different container types.

1.3.6 Iterator

Table of Contents

- *Iterator*
 - *Iterator Categories*
 - *Iterator Operations*
 - *lower_bound and upper_bound*
 - *Iterator Invalidation*
 - *Custom Iterator*
 - *C++20 Ranges Overview*
 - *views::iota*
 - *views::filter*
 - *views::transform*
 - *views::take and views::drop*
 - *views::reverse*
 - *views::split and views::join*
 - *views::enumerate (C++23)*
 - *views::zip (C++23)*
 - *Composing Views*
 - *Converting Views to Containers*
 - *Range Algorithms*

Iterator Categories

Source

`src/iterator/basics`

Iterators are the glue between containers and algorithms in C++. They provide a uniform way to traverse and access elements regardless of the underlying container type. This abstraction allows the same algorithm (like `std::sort` or `std::find`) to work with vectors, lists, arrays, and even custom containers.

The standard library defines **five iterator categories**, each building upon the capabilities of the previous one. Understanding these categories is essential for writing generic code and choosing the right algorithm for your container.

Table 1: Iterator Categories Summary

Category	Capabilities	Example Containers
Input	Read-only, single-pass, forward only (++, *, ==)	istream_iterator
Output	Write-only, single-pass, forward only (++, *=)	ostream_iterator, back_inserter
Forward	Read/write, multi-pass, forward only (++)	forward_list, unordered_set
Bidirectional	Forward + backward (++, --)	list, set, map
Random Access	Bidirectional + arithmetic (+, -, [])	vector, array, deque

Note

Iterator categories form a hierarchy: Random Access > Bidirectional > Forward > Input/Output. An algorithm requiring a Forward iterator will work with Bidirectional or Random Access iterators, but not with Input or Output iterators.

Input Iterator

Input iterators support **read-only**, **single-pass**, **forward-only** traversal. They can only be dereferenced to read values (not write), and once you advance past an element, **you cannot go back to it**. This makes them suitable for reading from streams where data is consumed as it's read. The classic example is `std::istream_iterator`, which reads from an input stream:

```
#include <iostream>
#include <iterator>
#include <sstream>

int main(int argc, char *argv[]) {
    std::istringstream iss("1 2 3");
    std::istream_iterator<int> it(iss), end;

    while (it != end) {
        std::cout << *it << " "; // Read each value once
        ++it;
    }
    // Output: 1 2 3
}
```

Warning

Input iterators are **single-pass**: once you call `++it`, the previous position is gone forever. Copying an input iterator and advancing one copy **may invalidate the other copy**.

Output Iterator

Output iterators are the **write-only** counterpart to input iterators. They support single-pass, forward-only traversal, but can only be dereferenced to write values (**not read**). You cannot read back what you wrote or revisit previous positions. The typical example is `std::ostream_iterator`, which writes to an output stream. Output iterators are commonly used as destinations for algorithms like `std::copy`:


```
#include <iostream>
#include <iterator>
#include <vector>

int main(int argc, char *argv[]) {
    std::ostream_iterator<int> out(std::cout, " ");

    *out = 1; ++out;
    *out = 2; ++out;
    *out = 3; ++out;
    // Output: 1 2 3
}
```

Forward Iterator

Forward iterators combine the capabilities of input and output iterators: they support **both reading and writing**, and they allow **multi-pass traversal**. This means you can iterate through a range multiple times, and the iterators remain valid. However, they **only move forward** (no `--` operator). `std::forward_list` provides forward iterators, as its singly-linked structure doesn't support backward traversal:

```
#include <forward_list>
#include <iostream>

int main(int argc, char *argv[]) {
    std::forward_list<int> fl{1, 2, 3};

    // Can traverse multiple times
    for (auto it = fl.begin(); it != fl.end(); ++it) {
        *it *= 2; // Read and write
    }
    for (auto it = fl.begin(); it != fl.end(); ++it) {
        std::cout << *it << " "; // 2 4 6
    }
}
```

Bidirectional Iterator

Bidirectional iterators extend forward iterators by adding the ability to **move backward** using the decrement operator (`--`). This enables algorithms that need to traverse in both directions, such as `std::reverse` or backward searches. `std::list` and `std::set` provide bidirectional iterators. The doubly-linked structure of `std::list` naturally supports movement in both directions:

```
#include <iostream>
#include <list>

int main(int argc, char *argv[]) {
    std::list<int> l{1, 2, 3, 4, 5};

    auto it = l.end();
    while (it != l.begin()) {
        --it; // Can move backward
        std::cout << *it << " "; // 5 4 3 2 1
    }
}
```

Random Access Iterator

Random access iterators are the **most powerful category**, adding **constant-time arithmetic operations** to bidirectional iterators. You can jump forward or backward by any number of positions using `+`, `-`, `+=`, `-=`, and access elements at arbitrary offsets using subscript `[]`. You can also compute the **distance between two iterators in $O(1)$ time**. `std::vector`, `std::array`, and `std::deque` provide random access iterators. This is what enables efficient algorithms like `std::sort` (which **requires random access** for $O(n \log n)$ performance):

```
#include <iostream>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5};

    auto it = v.begin();
    it += 3;           // Jump forward by 3
    std::cout << *it << "\n"; // 4

    it -= 2;           // Jump backward by 2
    std::cout << *it << "\n"; // 2

    std::cout << it[2] << "\n"; // Subscript: 4
    std::cout << (v.end() - v.begin()) << "\n"; // Distance: 5
}
```

Note

`std::sort` **requires random access iterators**. You cannot use `std::sort` on `std::list`; use `list.sort()` member function instead.

Iterator Operations

The `<iterator>` header provides utility functions that work across different iterator categories. These functions abstract away the differences between iterator types, allowing you to write **generic code** that works with any iterator:

- `std::advance(it, n)`: Moves iterator `it` by `n` positions (**modifies in place**)
- `std::distance(first, last)`: Returns the number of elements between two iterators
- `std::next(it, n=1)`: Returns a **new iterator** `n` positions after `it`
- `std::prev(it, n=1)`: Returns a **new iterator** `n` positions before `it`

These functions **automatically use the most efficient implementation** based on the iterator category. For random access iterators, `std::advance` uses `+=` ($O(1)$), while for other iterators it uses repeated `++` ($O(n)$).

```
#include <iostream>
#include <iterator>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5};

    // std::advance - move iterator by n positions
    auto it = v.begin();
```

(continues on next page)

(continued from previous page)

```

std::advance(it, 2);
std::cout << *it << "\n"; // 3

// std::distance - count elements between iterators
auto dist = std::distance(v.begin(), it);
std::cout << dist << "\n"; // 2

// std::next / std::prev - return new iterator
auto next_it = std::next(it); // Points to 4
auto prev_it = std::prev(it); // Points to 2
std::cout << *next_it << " " << *prev_it << "\n"; // 4 2
}

```

 Tip

Prefer `std::next(it)` over `it + 1` in generic code. `std::next` works with **any forward iterator**, while `+` **only works with random access iterators**.

lower_bound and upper_bound

Source

`src/iterator/bounds`

Binary search algorithms `std::lower_bound` and `std::upper_bound` are essential for working with **sorted ranges**. They use binary search to find elements in **$O(\log n)$** time for random access iterators. Understanding the subtle difference between them is crucial:

- `lower_bound(first, last, val)`: Returns iterator to **first element $\geq$ val**
- `upper_bound(first, last, val)`: Returns iterator to **first element $>$ val**

Together they define the range of elements equal to `val`: `[lower_bound, upper_bound)`. If the value doesn't exist, both return the **position where it would be inserted** to maintain sorted order. This makes them ideal for insertion and deletion in sorted containers.

```

#include <algorithm>
#include <iostream>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5, 7, 10};

    auto x = 5;
    auto lo = std::lower_bound(v.begin(), v.end(), x);
    auto hi = std::upper_bound(v.begin(), v.end(), x);

    std::cout << "lower_bound(5): " << *lo << "\n"; // 5 ( $\geq$  5)
    std::cout << "upper_bound(5): " << *hi << "\n"; // 7 ( $>$  5)

    // Insert into sorted container
    auto pos = std::upper_bound(v.begin(), v.end(), 6);
    v.insert(pos, 6); // v is now {1, 2, 3, 4, 5, 6, 7, 10}
}

```

(continues on next page)

(continued from previous page)

```
// Erase from sorted container
pos = std::lower_bound(v.begin(), v.end(), 4);
v.erase(pos);      // v is now {1, 2, 3, 5, 6, 7, 10}
}
```

Iterator Invalidation

Source

src/iterator/invalidation

Understanding when iterators become invalid is **crucial for avoiding undefined behavior**. Different containers have different invalidation rules:

std::vector:

- `push_back()`: Invalidates **all iterators if reallocation occurs**
- `insert()`: Invalidates iterators **at and after** the insertion point
- `erase()`: Invalidates iterators **at and after** the erased element

std::deque:

- Insertion/erasure at ends: Only invalidates iterators to the affected end
- Insertion/erasure in middle: **Invalidates all iterators**

std::list:

- **Only iterators to erased elements are invalidated**
- Other iterators remain valid after any operation

```
#include <iostream>
#include <list>
#include <vector>

int main(int argc, char *argv[]) {
    // Vector: iterator invalidated after erase
    std::vector<int> v{1, 2, 3, 4, 5};
    auto vit = v.begin() + 2;
    v.erase(vit);
    // vit is now invalid! Use returned iterator instead:
    // vit = v.erase(vit);

    // List: only erased iterator is invalidated
    std::list<int> l{1, 2, 3, 4, 5};
    auto lit = l.begin();
    auto next = std::next(lit);
    l.erase(lit);
    // next is still valid
    std::cout << *next << "\n"; // 2
}
```



Warning

Using an invalidated iterator is **undefined behavior**. Always use the iterator returned by `erase()` to continue iteration: `it = container.erase(it);`

💡 Tip

When iterating and erasing, use the **erase-remove idiom** or iterate carefully:

```
// Safe pattern for erasing while iterating
for (auto it = v.begin(); it != v.end(); ) {
    if (should_erase(*it)) {
        it = v.erase(it); // erase returns next valid iterator
    } else {
        ++it;
    }
}
```

Custom Iterator

Source

[src/iterator/custom](#)

Creating a custom iterator requires implementing the iterator interface. For a forward iterator, you need: `operator*`, `operator->`, `operator++`, `operator==`, and `operator!=`. C++20 simplifies this with default comparisons.

📌 Note

Custom iterators must define five type aliases (`iterator_category`, `value_type`, `difference_type`, `pointer`, `reference`) to work with standard algorithms and `std::iterator_traits`.

```
#include <iostream>
#include <iterator>
#include <memory>

template <typename T>
class Array {
public:
    class iterator {
    public:
        using iterator_category = std::forward_iterator_tag;
        using value_type = T;
        using difference_type = std::ptrdiff_t;
        using pointer = T *;
        using reference = T &;

        iterator(T *ptr) : ptr_{ptr} {}
        reference operator*() { return *ptr_; }
        pointer operator->() { return ptr_; }
        iterator &operator++() { ++ptr_; return *this; }
        iterator operator++(int) { auto tmp = *this; ++ptr_; return tmp; }
        bool operator==(const iterator &rhs) const { return ptr_ == rhs.ptr_; }
    };
};
```

(continues on next page)

(continued from previous page)

```

    bool operator!=(const iterator &rhs) const { return ptr_ != rhs.ptr_; }

private:
    T *ptr_;
};

Array(size_t size) : size_(size), data_{std::make_unique<T[]>(size)} {}
size_t size() const { return size_; }
T &operator[](size_t i) { return data_[i]; }
iterator begin() { return iterator(data_.get()); }
iterator end() { return iterator(data_.get() + size_); }

private:
    size_t size_;
    std::unique_ptr<T[]> data_;
};

int main(int argc, char *argv[]) {
    Array<int> arr(3);
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    for (auto &x : arr) {
        std::cout << x << " ";
    }
    std::cout << "\n";
}

```

C++20 Ranges Overview

C++20 introduced the Ranges library (`<ranges>`) which provides a modern, composable way to work with sequences. Ranges are an abstraction over iterator pairs, and views are lazy, non-owning ranges that can be composed using the pipe operator (`|`).

Key concepts:

- **Range:** Anything with `begin()` and `end()` (containers, arrays, views)
- **View:** A lightweight, non-owning range with $O(1)$ copy/move
- **Range Adaptor:** A function that takes a range and returns a view
- **Pipe Operator:** Composes adaptors: `range | adaptor1 | adaptor2`

Note

Views are **lazy**: no computation happens until you iterate. This means you can chain multiple views **without creating intermediate containers**, and only the elements you actually access are processed.

Warning

Views **do not own their data**. The underlying range must outlive the view. Returning a view to a local container is **undefined behavior**.

views::iota

Source

src/iterator/iota

`std::views::iota` generates a sequence of **incrementing values**. It can be bounded or **unbounded (infinite)**. This is useful for generating index sequences or replacing traditional for loops.

```
#include <iostream>
#include <ranges>

int main(int argc, char *argv[]) {
    // Bounded: [1, 6)
    for (auto i : std::views::iota(1, 6)) {
        std::cout << i << " "; // 1 2 3 4 5
    }
    std::cout << "\n";

    // Unbounded with take
    for (auto i : std::views::iota(1) | std::views::take(5)) {
        std::cout << i << " "; // 1 2 3 4 5
    }
    std::cout << "\n";
}
```

💡 Tip

Use `std::views::iota(0, n)` instead of `for (int i = 0; i < n; ++i)` when you need a range-based approach or want to **compose with other views**.

views::filter

Source

src/iterator/filter

`std::views::filter` creates a view containing only elements that satisfy a predicate. The predicate is evaluated lazily during iteration.

```
#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    auto evens = v | std::views::filter([](int x) { return x % 2 == 0; });

    for (auto x : evens) {
        std::cout << x << " "; // 2 4 6 8 10
    }
}
```

(continues on next page)

(continued from previous page)

```

}
std::cout << "\n";
}

```

views::transform

Source

src/iterator/transform

`std::views::transform` applies a function to each element, creating a view of the transformed values. Similar to `map` in functional programming.

```

#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5};

    auto squares = v | std::views::transform([](int x) { return x * x; });

    for (auto x : squares) {
        std::cout << x << " "; // 1 4 9 16 25
    }
    std::cout << "\n";
}

```

views::take and views::drop

Source

src/iterator/take-drop

`std::views::take(n)` returns the first `n` elements, while `std::views::drop(n)` skips the first `n` elements. Both are useful for slicing ranges.

```

#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // First 3 elements
    for (auto x : v | std::views::take(3)) {
        std::cout << x << " "; // 1 2 3
    }
    std::cout << "\n";

    // Skip first 3 elements
    for (auto x : v | std::views::drop(3)) {
        std::cout << x << " "; // 4 5 6 7 8 9 10
    }
    std::cout << "\n";
}

```

(continues on next page)

(continued from previous page)

```
// Slice [3, 6): drop 3, take 3
for (auto x : v | std::views::drop(3) | std::views::take(3)) {
    std::cout << x << " "; // 4 5 6
}
std::cout << "\n";
}
```

views::reverse

Source

src/iterator/reverse

std::views::reverse creates a view that iterates in reverse order. It requires a bidirectional range.

```
#include <iostream>
#include <ranges>
#include <string>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5};

    for (auto x : v | std::views::reverse) {
        std::cout << x << " "; // 5 4 3 2 1
    }
    std::cout << "\n";

    // Reverse a string
    std::string s = "hello";
    for (auto c : s | std::views::reverse) {
        std::cout << c; // olleh
    }
    std::cout << "\n";
}
```

views::split and views::join

Source

src/iterator/split-join

std::views::split divides a range by a delimiter, while std::views::join flattens a range of ranges into a single range.

```
#include <iostream>
#include <ranges>
#include <string>
#include <string_view>
#include <vector>

int main(int argc, char *argv[]) {
    // Split string by delimiter
    std::string s = "one,two,three";
```

(continues on next page)

(continued from previous page)

```

for (auto word : s | std::views::split(', ')) {
    std::cout << std::string_view(word) << "\n";
}

// Join nested ranges
std::vector<std::string> words = {"hello", "world"};
for (auto c : words | std::views::join) {
    std::cout << c; // helloworld
}
std::cout << "\n";
}

```

views::enumerate (C++23)**Source**

src/iterator/enumerate

`std::views::enumerate` pairs each element with its index. This is available in C++23. For C++20, you can use `std::views::iota` with `std::views::zip` or a simple counter.

```

#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<std::string> v{"apple", "banana", "cherry"};

    #if __cplusplus >= 202302L
        // C++23: views::enumerate
        for (auto [i, val] : v | std::views::enumerate) {
            std::cout << i << ": " << val << "\n";
        }
    #else
        // C++20 workaround
        for (size_t i = 0; auto &val : v) {
            std::cout << i++ << ": " << val << "\n";
        }
    #endif
}

```

views::zip (C++23)**Source**

src/iterator/zip

`std::views::zip` combines multiple ranges into a range of tuples. Available in C++23.

```

#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> a{1, 2, 3};

```

(continues on next page)

(continued from previous page)

```

std::vector<std::string> b{"one", "two", "three"};

#if __cplusplus >= 202302L
// C++23: views::zip
for (auto [x, y] : std::views::zip(a, b)) {
    std::cout << x << " -> " << y << "\n";
}
#else
// C++20 workaround
for (size_t i = 0; i < a.size(); ++i) {
    std::cout << a[i] << " -> " << b[i] << "\n";
}
#endif
}

```

Composing Views

Source

src/iterator/compose

The power of ranges comes from composing multiple views. Views are lazy, so no intermediate containers are created. The pipeline is evaluated element by element during iteration.

```

#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Filter evens, square them, take first 3
    auto result = v
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x * x; })
        | std::views::take(3);

    for (auto x : result) {
        std::cout << x << " "; // 4 16 36
    }
    std::cout << "\n";
}

```

Converting Views to Containers

Source

src/iterator/to-container

Views are lazy and non-owning. To materialize results into a container, use `std::ranges::to` (C++23) or manual construction.

```

#include <iostream>
#include <ranges>
#include <vector>

```

(continues on next page)

(continued from previous page)

```

int main(int argc, char *argv[]) {
    auto view = std::views::iota(1, 6)
        | std::views::transform([](int x) { return x * x; });

    #if __cplusplus >= 202302L
        // C++23: ranges::to
        auto v = view | std::ranges::to<std::vector>();
    #else
        // C++20 workaround
        std::vector<int> v(view.begin(), view.end());
    #endif

    for (auto x : v) {
        std::cout << x << " "; // 1 4 9 16 25
    }
    std::cout << "\n";
}

```

Range Algorithms

Source

src/iterator/algorithms

C++20 also provides range-based versions of standard algorithms in the `std::ranges` namespace. These accept ranges directly instead of iterator pairs.

```

#include <algorithm>
#include <iostream>
#include <ranges>
#include <vector>

int main(int argc, char *argv[]) {
    std::vector<int> v{3, 1, 4, 1, 5, 9, 2, 6};

    // Range-based sort
    std::ranges::sort(v);

    // Range-based find
    auto it = std::ranges::find(v, 5);
    if (it != v.end()) {
        std::cout << "Found: " << *it << "\n";
    }

    // Range-based all_of
    bool all_positive = std::ranges::all_of(v, [](int x) { return x > 0; });
    std::cout << "All positive: " << all_positive << "\n";

    // Range-based count_if
    auto count = std::ranges::count_if(v, [](int x) { return x % 2 == 0; });
    std::cout << "Even count: " << count << "\n";
}

```

1.3.7 Template

Table of Contents

- *Template*
 - *Explicit Template Instantiation*
 - *Template Specialization*
 - *Class Template Inheritance*
 - *Variadic Templates and Parameter Packs*
 - *Fold Expressions (C++17)*
 - *SFINAE and Type Constraints*
 - *CRTP (Curiously Recurring Template Pattern)*
 - *Template Template Parameters*
 - *Accessing Protected Members in Derived Templates*
 - *Compile-Time Loop Unrolling*

C++ templates enable generic programming by allowing functions and classes to operate with different data types without rewriting code. Templates are a cornerstone of modern C++ development, powering the Standard Template Library (STL) and enabling type-safe, high-performance abstractions. Understanding templates is essential for writing reusable, efficient C++ code.

Explicit Template Instantiation

Source

`src/template/instantiate`

Explicit template instantiation forces the compiler to generate code for a specific template specialization at a particular point in the source. This technique is particularly valuable in large codebases where the same template is used across multiple translation units. By explicitly instantiating templates in a single source file, you can significantly reduce compile times and control where template code is generated. This approach is commonly used in library development to reduce binary size and improve build performance.

Key benefits: Reduced compilation time, controlled symbol generation, smaller binary size when templates are used across many files.

```
#include <iostream>

struct A {};
struct B {};

template <typename T, typename U>
struct Foo {
    Foo(T t, U u) : t_(t), u_(u) {}
    T t_;
    U u_;
};

template <typename F, typename T, typename U>
```

(continues on next page)

(continued from previous page)

```

struct Bar {
    Bar(T t, U u) : f_(t, u) {}
    F f_;
};

// explicit instantiation
template class Foo<A, B>;

int main() {
    Bar<Foo<A, B>, A, B>(A(), B());
}

```

Template Specialization

Source

[src/template/specialization](#)

Template specialization allows customizing template behavior for specific types, enabling optimized or alternative implementations for particular type combinations. This is one of the most powerful features of C++ templates, allowing generic code to have type-specific optimizations without sacrificing the generic interface.

Partial specialization fixes some template parameters while leaving others generic. This is useful when you want specialized behavior for a category of types (e.g., all pointers, all containers with a specific allocator).

Full specialization provides a complete implementation for specific types, replacing the primary template entirely for those types. This enables highly optimized implementations for known types.

```

#include <iostream>

template <typename T, typename U>
class Base {
    T m_a;
    U m_b;
public:
    Base(T a, U b) : m_a(a), m_b(b) {}
    T foo() { return m_a; }
    U bar() { return m_b; }
};

// partial specialization - second parameter fixed to int
template<typename T>
class Base<T, int> {
    T m_a;
    int m_b;
public:
    Base(T a, int b) : m_a(a), m_b(b) {}
    T foo() { return m_a; }
    int bar() { return m_b; }
};

// full specialization - both parameters fixed
template<>
class Base<double, double> {

```

(continues on next page)

(continued from previous page)

```

double d_a;
double d_b;
public:
    Base(double a, double b) : d_a(a), d_b(b) {}
    double foo() { return d_a; }
    double bar() { return d_b; }
};

int main() {
    Base<float, int> foo(3.33, 1);           // uses partial specialization
    Base<double, double> bar(55.66, 95.27); // uses full specialization
    std::cout << foo.foo() << "\n";
    std::cout << bar.bar() << "\n";
}

```

Class Template Inheritance

Source

src/template/class-template

Class templates can serve as base classes for both regular classes and other templates, enabling powerful code reuse patterns. This flexibility allows you to build inheritance hierarchies that combine generic and concrete types.

A non-template class can inherit from a specific template instantiation, which is useful when you need a concrete class with a fixed type but want to reuse template-based functionality. A template class can also inherit from a template base, preserving generic behavior through the entire inheritance hierarchy.

Important: When inheriting from a template base class, dependent names (names that depend on template parameters) require special handling due to two-phase name lookup rules.

```

#include <iostream>

template <typename T>
class Area {
protected:
    T w, h;
public:
    Area(T a, T b) : w(a), h(b) {}
    T get() { return w * h; }
};

// non-template inherits from specific instantiation
class Rectangle : public Area<int> {
public:
    Rectangle(int a, int b) : Area<int>(a, b) {}
};

// template inherits from template base
template <typename T>
class GenericRectangle : public Area<T> {
public:
    GenericRectangle(T a, T b) : Area<T>(a, b) {}
};

```

(continues on next page)

(continued from previous page)

```
int main() {
    Rectangle r(2, 5);
    GenericRectangle<double> g(2.5, 3.0);
    std::cout << r.get() << "\n";
    std::cout << g.get() << "\n";
}
```

Variadic Templates and Parameter Packs

Source

[src/template/variadic](#)

Variadic templates accept an arbitrary number of template arguments using parameter packs (. . .). This feature, introduced in C++11, revolutionized generic programming by enabling functions and classes that work with any number of arguments of any types. Variadic templates are the foundation of many modern C++ idioms including `std::tuple`, `std::make_unique`, and perfect forwarding utilities.

Parameter pack expansion is the mechanism by which the compiler generates code for each element in the pack. Before C++17, this required recursive function calls. C++17 introduced fold expressions for more concise and often more efficient pack expansion.

Recursive expansion (C++11):

The classic approach uses function overloading with a base case (single argument) and a recursive case that peels off one argument at a time:

```
#include <iostream>

template <typename T>
T sum(T x) { return x; }

template <typename T, typename ...Args>
T sum(T x, Args ...args) { return x + sum(args...); }

int main() {
    std::cout << sum(1, 2, 3, 4, 5) << "\n"; // 15
}
```

Fold expression (C++17):

Fold expressions provide a more elegant and often more efficient way to expand parameter packs using binary operators:

```
#include <iostream>

template <typename ...Args>
auto sum(Args ...args) { return (args + ...); }

int main() {
    std::cout << sum(1, 2, 3, 4, 5) << "\n"; // 15
}
```

Variadic class constructor:

Variadic templates combined with perfect forwarding enable constructors that accept any number of arguments and forward them efficiently. This pattern is used extensively in the standard library for in-place construction:


```

#include <iostream>
#include <utility>
#include <vector>

template <typename T>
class Vector {
    std::vector<T> v;
public:
    template<typename ...Args>
    Vector(Args&&... args) {
        (v.emplace_back(std::forward<Args>(args)), ...);
    }

    auto begin() { return v.begin(); }
    auto end() { return v.end(); }
};

int main() {
    Vector<int> v{1, 2, 3};
    for (auto x : v) std::cout << x << "\n";
}

```

Fold Expressions (C++17)

Source

[src/template/fold](#)

Fold expressions reduce parameter packs using binary operators, providing a concise syntax for operations that would otherwise require recursive templates. They significantly simplify variadic template code and can generate more efficient code by avoiding recursive function calls.

Syntax forms:

- (args op ...) - unary right fold: a op (b op (c op d))
- (... op args) - unary left fold: ((a op b) op c) op d
- (init op ... op args) - binary right fold with initial value
- (args op ... op init) - binary left fold with initial value

Fold expressions work with any binary operator including the comma operator for side effects and << for stream output. This makes them extremely versatile for a wide range of use cases.

```

#include <iostream>
#include <vector>

int main() {
    // unary right fold: (args op ...)
    auto sum = [](auto ...args) { return (args + ...); };
    std::cout << sum(1, 2, 3, 4, 5) << "\n"; // 15

    // fold with comma operator for side effects
    std::vector<int> v;
    [&v](auto ...args) { (v.emplace_back(args), ...); }(1, 2, 3);
}

```

(continues on next page)

(continued from previous page)

```
// fold with stream operator
[](auto ...args) {
    (std::cout << ... << args) << "\n";
}(1, 2, 3, 4, 5); // 12345

// fold with transformation
auto double_sum = [](auto &&f, auto ...args) {
    return (... + f(args));
};
std::cout << double_sum([](auto x) { return x * 2; }, 1, 2, 3) << "\n"; // 12
}
```

SFINAE and Type Constraints

Source

src/template/sfinae

SFINAE (Substitution Failure Is Not An Error) is a fundamental C++ template mechanism that enables conditional template instantiation based on type properties. When the compiler attempts to substitute template arguments and the substitution results in an invalid type, instead of producing a compilation error, the compiler simply removes that template from the overload set.

This technique is fundamental for writing type-safe generic code that only accepts types meeting certain requirements. Combined with type traits from `<type_traits>`, SFINAE enables powerful compile-time type checking and conditional compilation.

Using `std::enable_if` with type traits restricts which types can instantiate a template. This is the classic SFINAE approach, though C++20 concepts provide a more readable alternative for new code.

```
#include <iostream>
#include <string>
#include <type_traits>

template<typename S,
        typename = std::enable_if_t<
            std::is_same_v<std::string, std::decay_t<S>>
        >
>
void Foo(S s) {
    std::cout << s << "\n";
}

int main() {
    std::string s1 = "Foo";
    const std::string s2 = "Bar";
    Foo(s1);
    Foo(s2);
    // Foo(123);    // compile error
    // Foo("Baz");  // compile error
}
```

Template specialization approach:

An alternative to SFINAE is providing explicit specializations for supported types, leaving the primary template empty or deleted. This approach is simpler to understand but less flexible than SFINAE for complex type constraints:

```

#include <iostream>
#include <string>

template<typename S>
void Foo(S s) {}

template <>
void Foo<int>(int s) {
    std::cout << "integer: " << s << "\n";
}

template<>
void Foo<std::string>(std::string s) {
    std::cout << "string: " << s << "\n";
}

int main() {
    Foo(123);
    Foo(std::string("hello"));
}

```

CRTP (Curiously Recurring Template Pattern)

Source

[src/template/crtp](#)

CRTP is a powerful C++ idiom where a class derives from a template instantiated with itself as the argument. This enables static polymorphism, allowing base class methods to call derived class implementations without the runtime overhead of virtual functions. CRTP is widely used for mixin classes, compile-time polymorphism, and implementing the “template method” design pattern with zero runtime cost.

Understanding the trade-offs between virtual functions and CRTP is crucial for making informed design decisions in performance-sensitive code.

Virtual function (dynamic polymorphism):

Traditional runtime polymorphism uses virtual functions and vtable lookup. Each object with virtual functions contains a hidden pointer to a vtable, and each virtual call requires an indirect function call through this table. This approach supports runtime type decisions and is essential when the concrete type is not known until runtime, but it incurs overhead from indirect function calls and prevents the compiler from inlining:

```

#include <iostream>
#include <memory>

class Base {
public:
    virtual void impl() { std::cout << "Base\n"; }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void impl() override { std::cout << "Derived\n"; }
};

```

(continues on next page)

(continued from previous page)

```
int main() {
    std::unique_ptr<Base> p = std::make_unique<Derived>();
    p->impl(); // runtime dispatch via vtable
}
```

CRTP (static polymorphism):

CRTP resolves the derived type at compile time through template instantiation. The base class uses `static_cast` to access the derived class, which is safe because the template parameter guarantees the relationship. No vtable is needed, enabling the compiler to inline the derived implementation and eliminating all virtual call overhead:

```
#include <iostream>

template <typename D>
class Base {
public:
    void interface() { static_cast<D*>(this)->impl(); }
    void impl() { std::cout << "Base\n"; }
};

class Derived : public Base<Derived> {
public:
    void impl() { std::cout << "Derived\n"; }
};

int main() {
    Derived d;
    d.interface(); // compile-time dispatch, no vtable
}
```

Comparison:

Aspect	Virtual Functions	CRTP
Dispatch	Runtime (vtable)	Compile-time (static)
Performance	Indirect call overhead	Inlinable, zero overhead
Flexibility	Runtime type switching	Types fixed at compile time
Memory	vtable pointer per object	No extra memory
Binary size	Smaller (shared vtable)	Larger (template instantiation)

When to use each: Use virtual functions when you need runtime polymorphism (e.g., plugin systems, heterogeneous containers). Use CRTP when types are known at compile time and performance is critical (e.g., numeric libraries, game engines, embedded systems).

Template Template Parameters**Source**

[src/template/template-template](#)

Template template parameters allow templates to accept other templates as arguments, rather than just types or values. This enables writing highly generic functions and classes that work with any container type or any template following a specific signature, providing maximum flexibility for generic algorithms.

This feature is essential for writing container-agnostic algorithms and for implementing policy-based design patterns where behaviors are injected via template parameters.

```

#include <vector>
#include <deque>
#include <iostream>

template <template<class, class> class V, class T, class A>
void pop(V<T, A> &v) {
    std::cout << v.back() << "\n";
    v.pop_back();
}

int main() {
    std::vector<int> v{1, 2, 3};
    std::deque<int> q{4, 5, 6};
    pop(v); // 3
    pop(q); // 6
}

```

Accessing Protected Members in Derived Templates

Source

[src/template/protected-access](#)

When a template class inherits from another template, protected members of the base class are not directly visible in the derived class. This behavior is due to C++'s two-phase name lookup rules: names that depend on template parameters are not looked up until the template is instantiated.

This is a common source of confusion for developers new to template programming. The compiler cannot know at parse time whether `p_` refers to a base class member or something else, so it requires explicit qualification.

Two solutions exist: using a `using` declaration to bring names into the current scope, or qualifying access through the `this` pointer to make the name dependent:

Using declaration:

```

#include <iostream>

template <typename T>
class A {
protected:
    T p_;
public:
    A(T p) : p_(p) {}
};

template <typename T>
class B : A<T> {
    using A<T>::p_; // bring into scope
public:
    B(T p) : A<T>(p) {}
    void print() { std::cout << p_ << "\n"; }
};

int main() {
    B<int> b(42);
}

```

(continues on next page)

(continued from previous page)

```
b.print();
}
```

Via this pointer:

```
template <typename T>
class B : A<T> {
public:
    B(T p) : A<T>(p) {}
    void print() { std::cout << this->p_ << "\n"; }
};
```

Compile-Time Loop Unrolling

Source

[src/template/loop-unroll](#)

Template metaprogramming enables compile-time loop unrolling through recursive template instantiation. The compiler generates fully unrolled code at compile time, completely eliminating loop overhead including the loop counter, condition check, and branch instruction. This technique is particularly valuable for performance-critical code with known iteration counts, such as matrix operations, SIMD processing, and embedded systems programming.

The trade-off is increased binary size due to code duplication, but for small, hot loops, the performance benefit often outweighs this cost. Modern compilers may unroll loops automatically, but template-based unrolling gives you explicit control.

```
#include <iostream>
#include <utility>

template <size_t N>
struct Loop {
    template <typename F>
    static void run(F &&f) {
        Loop<N-1>::run(std::forward<F>(f));
        f(N-1);
    }
};

template <>
struct Loop<0> {
    template <typename F>
    static void run(F&&) {}
};

int main() {
    size_t sum = 0;
    Loop<5>::run([&](auto i) { sum += i; });
    std::cout << sum << "\n"; // 10 (0+1+2+3+4)
}
```

1.3.8 Casting

Table of Contents

- *Casting*
 - *C-Style Cast (Legacy Casting)*
 - *const_cast: Removing or Adding const*
 - *reinterpret_cast: Low-Level Bit Reinterpretation*
 - *static_cast: Compile-Time Type Conversion*
 - *dynamic_cast: Runtime Type-Safe Downcasting*

C-Style Cast (Legacy Casting)

Source

src/casting/c-style

C-style casts use the syntax `(type)expression` inherited from C. While concise, they are dangerous because the compiler tries `const_cast`, `static_cast`, and `reinterpret_cast` in sequence until one succeeds. This makes it hard to know which conversion actually happens. The lack of visual distinction also makes C-style casts difficult to search for in large codebases. Modern C++ code should prefer explicit C++ casts for clarity, safety, and maintainability.

```
#include <cmath>
#include <iostream>

int main(int argc, char *argv[]) {
    double x = M_PI;
    int xx = (int)x; // Truncates: 3.14159... -> 3
    std::cout << xx << "\n";

    long z = LONG_MAX;
    int zz = (int)z; // Dangerous: overflow, undefined behavior
    std::cout << zz << "\n";
}
```

Warning

C-style casts bypass type safety checks and can silently perform dangerous conversions. Prefer `static_cast` for safe conversions and avoid C-style casts in modern C++ code.

const_cast: Removing or Adding const

Source

src/casting/const-cast

The `const_cast` operator adds or removes `const` (or `volatile`) qualifiers from a pointer or reference. This is the only C++ cast that can modify cv-qualifiers. The primary use case is interfacing with legacy C APIs or older C++ code that lacks proper const-correctness. Modifying a truly const object through `const_cast` results in undefined behavior, so it should only be used when you are certain the underlying object was not originally declared as const.

```
#include <iostream>

void legacy_api(int &x) {
    const_cast<int> &>(x) = 0; // Remove const to modify
}

int main(int argc, char *argv[]) {
    int x = 123;
    legacy_api(x);
    std::cout << x << "\n"; // Output: 0
}
```

Note

Using `const_cast` to modify an object that was originally declared `const` is undefined behavior. Only use it when you know the underlying object is non-const.

reinterpret_cast: Low-Level Bit Reinterpretation

Source

[src/casting/reinterpret-cast](#)

The `reinterpret_cast` operator performs low-level reinterpretation of bit patterns between unrelated types. It is primarily used for converting between pointer types, pointer-to-integer conversions, and accessing raw memory representations. Common use cases include serialization, memory-mapped I/O, and interfacing with hardware. This cast provides no type safety and the results are implementation-defined, so it should be used sparingly and only in systems programming contexts where such low-level access is necessary.

```
#include <iostream>

struct Point {
    int x;
    int y;
};

int main(int argc, char *argv[]) {
    Point p{1, 2};

    // View struct as raw bytes
    char *buf = reinterpret_cast<char*>(&p);
    for (size_t i = 0; i < sizeof(Point); ++i) {
        std::cout << static_cast<int>(buf[i]) << " ";
    }
    // Output (little-endian): 1 0 0 0 2 0 0 0
}
```

static_cast: Compile-Time Type Conversion

Source

[src/casting/static-cast](#)

The `static_cast` operator performs well-defined conversions between related types at compile time. It handles numeric conversions (int to double), enum-to-int conversions, void pointer conversions, and pointer upcasts in class hi-

erarchies. It can also invoke explicit constructors and conversion operators. Unlike `dynamic_cast`, it performs no runtime type checking, so downcasting with `static_cast` is unsafe unless you are certain of the object's actual type. When the conversion is valid, `static_cast` generates efficient code with no runtime overhead.

```
#include <iostream>
#include <memory>

struct Base {
    virtual ~Base() = default;
};

struct Derived : public Base {
    int value = 42;
};

int main(int argc, char *argv[]) {
    auto d = std::make_unique<Derived>();

    // Safe upcast: Derived* -> Base*
    Base *b = static_cast<Base *>(d.get());

    // Unsafe downcast: only safe if b actually points to Derived
    auto d2 = static_cast<Derived *>(b);
    std::cout << d2->value << "\n"; // Output: 42
}
```

Warning

Using `static_cast` for downcasting is unsafe. If the object is not actually of the target type, the behavior is undefined. Use `dynamic_cast` when runtime type checking is needed.

dynamic_cast: Runtime Type-Safe Downcasting

Source

[src/casting/dynamic-cast](#)

The `dynamic_cast` operator performs runtime type checking for safe downcasting in polymorphic class hierarchies. It requires the source type to have at least one virtual function (typically the destructor) because it relies on RTTI (Run-Time Type Information) stored in the virtual table. For pointer casts, it returns `nullptr` on failure; for reference casts, it throws `std::bad_cast`. This is the safest way to navigate class hierarchies when the actual type is uncertain at compile time, such as when processing heterogeneous collections of objects.

```
#include <iostream>
#include <memory>

struct Base {
    virtual ~Base() = default;
};

struct Derived : public Base {
    void greet() { std::cout << "Hello from Derived\n"; }
};
```

(continues on next page)

(continued from previous page)

```

int main(int argc, char *argv[]) {
    auto base = std::make_unique<Base>();
    auto derived = std::make_unique<Derived>();

    // Failed downcast: base doesn't point to Derived
    auto d1 = dynamic_cast<Derived*>(base.get());
    std::cout << "base -> Derived: " << (d1 ? "success" : "failed") << "\n";

    // Successful upcast: Derived* -> Base*
    auto b2 = dynamic_cast<Base*>(derived.get());
    std::cout << "Derived -> Base: " << (b2 ? "success" : "failed") << "\n";
}

```

Note

`dynamic_cast` has runtime overhead due to RTTI (Run-Time Type Information). In performance-critical code where types are known, prefer `static_cast`.

1.3.9 Polymorphism & Inheritance

Table of Contents

- *Polymorphism & Inheritance*
 - *Virtual Functions*
 - *override and final*
 - *Pure Virtual Functions and Abstract Classes*
 - *Virtual Destructors*
 - *The vtable and vptr*
 - *Static vs. Dynamic Dispatch*
 - *Construction and Destruction Order*
 - *The Slicing Problem*
 - *Multiple Inheritance*
 - *The Diamond Problem and Virtual Inheritance*
 - *RTTI and dynamic_cast*
 - *Pure Interfaces and the Non-Virtual Interface Idiom*
 - *Virtual vs. CRTP — When to Pick Which*
 - *Trait-Style Polymorphism (Type Erasure)*
 - *Common Pitfalls*

Polymorphism lets a single interface drive different concrete behaviors. Modern C++ supports three distinct flavors:

- **Subtype polymorphism** — inheritance and virtual functions, resolved at run time through a per-object pointer

to a table of function pointers (the vtable). Concrete types share a common base class.

- **Static (parametric) polymorphism** — templates, overloading, CRTP, and concepts, resolved at compile time. Zero indirection, full inlining.
- **Trait-style polymorphism** — type erasure. Unrelated concrete types are stored behind a value-semantic wrapper that calls into a hidden vtable. No shared base class is required; the contract is purely behavioral. This is the C++ analog of Rust's `dyn Trait`.

This chapter focuses on the first flavor, the vtable mechanics that implement it, and the pitfalls of class hierarchies — then revisits CRTP and type erasure at the end so you can pick the right tool.

For the static counterpart — CRTP, concepts, and template-based dispatch — see [Template](#) and [Concepts](#). For Rust's trait model, see [Traits and Generics](#). For the safe runtime-checked downcast, see [Casting](#).

Virtual Functions

Source

[src/polymorphism/virtual-functions](#)

A member function declared `virtual` in a base class can be overridden in a derived class. When called through a pointer or reference to the base, the *dynamic* type of the object decides which override runs. Without `virtual`, the call is bound to the *static* type at compile time.

```
#include <iostream>

struct Animal {
    virtual void speak() const { std::cout << "... " << "\n"; }
    void name() const { std::cout << "Animal" << "\n"; } // non-virtual
};

struct Dog : Animal {
    void speak() const override { std::cout << "Woof" << "\n"; }
    void name() const { std::cout << "Dog" << "\n"; }
};

int main() {
    Dog d;
    Animal &a = d;
    a.speak(); // "Woof" - dynamic dispatch through vtable
    a.name();  // "Animal" - static dispatch, base version called
}
```

The non-virtual `name` is *hidden* in `Dog`, not overridden. The choice of which to call is made by the compiler from the static type, so `a.name()` calls `Animal::name` even though `a` actually refers to a `Dog`.

override and final

Source

[src/polymorphism/override-final](#)

`override` (C++11) tells the compiler that a function is intended to override a base virtual. If the signature does not actually match a base virtual, the program fails to compile. This catches typos, forgotten `const` qualifiers, and signature drift after a refactor.

`final` prevents further overriding (when applied to a function) or further derivation (when applied to a class). It also enables a useful optimization: the compiler can devirtualize calls to a `final` override.

```
struct Base {  
    virtual void f() const;  
    virtual void g(int);  
};  
  
struct Derived : Base {  
    void f() const override;           // OK  
    // void f() override;           // ERROR: missing const  
    void g(int) override final;       // cannot be overridden further  
};  
  
struct Sealed final : Derived {       // cannot be derived from  
    void f() const override;  
};
```

Always write `override` on overriding functions. It is not redundant — it is a compile-time contract that the override still tracks the base.

Pure Virtual Functions and Abstract Classes

Source

`src/polymorphism/pure-virtual`

A function with `= 0` is *pure virtual*; the class that declares it is *abstract* and cannot be instantiated. Derived classes must override every pure virtual function before they become concrete.

```
struct Shape {  
    virtual double area() const = 0;  
    virtual ~Shape() = default;       // see Virtual Destructors  
};  
  
struct Circle : Shape {  
    explicit Circle(double r) : r_(r) {}  
    double area() const override { return 3.14159 * r_ * r_; }  
private:  
    double r_;  
};  
  
// Shape s;           // ERROR: cannot instantiate abstract class  
Circle c(1.0);        // OK  
Shape *p = &c;        // OK: pointer to abstract base
```

A pure virtual function can still have a definition; derived classes that explicitly call `Base::fn()` will then run that definition. This is occasionally useful for shared default behavior in an otherwise abstract interface.

Virtual Destructors

Source

`src/polymorphism/virtual-destructor`

If a class is meant to be a polymorphic base — that is, deleted through a base pointer — its destructor **must** be virtual. Without it, deleting through the base pointer invokes only the base destructor, leaking the derived part of the object. This is undefined behavior.

```

struct Base {
    ~Base() { /* not virtual */ }
};

struct Derived : Base {
    std::vector<int> data_{1, 2, 3};
    ~Derived() { /* destructor never runs */ }
};

Base *p = new Derived;
delete p;    // UB: ~Derived not called, data_ leaks

```

Two safe defaults:

- If a class has any virtual function, give it a `virtual ~Base() = default;`.
- If a class is **not** intended for polymorphic deletion, mark it `final` or document that fact, so callers do not silently rely on a non-virtual destructor.

The vtable and vptr

Most compilers implement dynamic dispatch with a *vtable*: a per-class array of function pointers, one slot per virtual function. Each polymorphic object stores a hidden *vptr* pointing to its class's vtable. A virtual call becomes two indirections: load the vptr, index into the vtable, jump to the target.

```

struct B {
    virtual void f();           // vtable slot 0
    virtual void g();           // vtable slot 1
    int x;
};

struct D : B {
    void f() override;         // overrides slot 0
    void h();                   // not virtual
    int y;
};

// Memory layout (typical Itanium ABI):
//
//   B object:           D object:
//   +-----+          +-----+
//   | vptr |  ----> | vptr |  ----> D's vtable: [&D::f, &B::g, ...]
//   |  x  |          |  x  |
//   +-----+          |  y  |
//                   +-----+

```

Consequences of this layout:

- Every polymorphic object pays one pointer (typically 8 bytes on 64-bit) of storage for the vptr.
- A virtual call is one extra load and an indirect branch compared to a direct call. The cost is small but real, and it can defeat inlining.
- The vptr is set by each constructor as the object's type changes during construction (see Construction and Destruction Order).

The exact layout is ABI-defined. The Itanium ABI used by GCC, Clang, and the Apple toolchain places the vptr at offset 0 and shares vtables between translation units; MSVC's ABI differs in the details (and in how it handles multiple inheritance) but follows the same general scheme.

You can inspect the layout with `-fdump-lang-class` (GCC) or `-Xclang -fdump-record-layouts` (Clang).

Static vs. Dynamic Dispatch

A non-virtual call is bound at compile time and can be inlined. A virtual call requires the dynamic type, so it can only be devirtualized when the compiler *proves* the dynamic type — for example, when the object's most-derived type is visible at the call site, or when the function is declared `final`.

```
struct I { virtual int f() const = 0; };
struct A final : I { int f() const override { return 1; } };

int callI(const I &i) { return i.f(); } // virtual call
int callA(const A &a) { return a.f(); } // devirtualized: A is final
int callD()          { A a; return a.f(); } // direct call, inlinable
```

Rules of thumb:

- Calls through a base pointer or reference are virtual.
- Calls on a known concrete object are direct.
- `final` on the override (or the class) lets the compiler devirtualize.
- Inside a constructor or destructor, virtual calls dispatch to the *currently-constructed* type, not the most-derived type (see below).

Construction and Destruction Order

Source

`src/polymorphism/ctor-dtor-dispatch`

During construction, the object's vptr is set to each base subobject's vtable in turn before its own constructor body runs. So a virtual call from inside a base constructor dispatches to the base version, not the derived override — the derived part of the object does not exist yet. Destruction is the mirror image.

```
struct B {
    B() { f(); } // calls B::f, not D::f
    virtual void f() { std::puts("B::f"); }
    virtual ~B() { f(); } // calls B::f
};

struct D : B {
    void f() override { std::puts("D::f"); }
};

D d; // prints "B::f" twice (ctor and dtor)
```

This is a common interview question and a real source of bugs. Never call virtual functions from a constructor or destructor expecting derived behavior. If you need polymorphic initialization, use a two-step create factory or a separate `init` call after construction.

The Slicing Problem

Source

src/polymorphism/slicing

Assigning a derived object to a base *value* copies only the base subobject — the derived part is “sliced off”. The result is a base object that has lost its dynamic type, and any virtual call on it will run the base version.

```
struct Animal { virtual void speak() const { std::puts("..."); } };
struct Dog : Animal { void speak() const override { std::puts("Woof"); } };

void by_value(Animal a)      { a.speak(); } // sliced -> "..."
void by_reference(const Animal &a) { a.speak(); } // polymorphic -> "Woof"

int main() {
    Dog d;
    by_value(d);
    by_reference(d);
}
```

To preserve polymorphism, pass by reference (`const Animal &`), pointer (`const Animal *`), or a smart pointer (`std::unique_ptr<Animal>`). Containers of polymorphic objects must hold pointers, not values: `std::vector<Animal>` slices on every `push_back`; `std::vector<std::unique_ptr<Animal>>` does not.

Multiple Inheritance

Source

src/polymorphism/multiple-inheritance

A class may have more than one direct base. Each non-virtual base contributes its own subobject and (if polymorphic) its own vptr. Casting between base pointers may adjust the pointer value to land on the right subobject.

```
struct Drawable { virtual void draw() const = 0; virtual ~Drawable() = default; };
struct Serializable { virtual void save() const = 0; virtual ~Serializable() = default; };
↔;

struct Widget : Drawable, Serializable {
    void draw() const override;
    void save() const override;
};

Widget w;
Drawable *d = &w; // points at Drawable subobject
Serializable *s = &w; // points at Serializable subobject (offset!)
// static_cast<void*>(d) != static_cast<void*>(s) in general
```

Multiple inheritance is fine for *interface inheritance* (abstract bases with no state). It gets complicated quickly when two bases share a common ancestor — the diamond problem.

The Diamond Problem and Virtual Inheritance

Source

src/polymorphism/diamond

When a class indirectly inherits from the same base through two paths, the default is to embed two copies of that base. Member access becomes ambiguous, and the two copies can diverge.

```
struct A { int x; };
struct B : A {};
struct C : A {};
struct D : B, C {};    // two A subobjects

D d;
// d.x;                // ERROR: ambiguous
d.B::x = 1;
d.C::x = 2;            // independent of d.B::x
```

virtual inheritance collapses the duplicates into a single shared subobject:

```
struct A { int x; };
struct B : virtual A {};
struct C : virtual A {};
struct D : B, C {};    // one A subobject

D d;
d.x = 1;               // unambiguous
```

Virtual inheritance has costs: an extra indirection (the *vbase pointer*) to locate the shared subobject, and the most-derived class becomes responsible for initializing the virtual base. Reach for it only when you genuinely need shared state across diamond paths; prefer composition or interface-only multiple inheritance otherwise.

RTTI and `dynamic_cast`

Run-Time Type Information (RTTI) lets you query the dynamic type of a polymorphic object. `typeid` returns a `std::type_info`; `dynamic_cast` performs a checked downcast.

```
#include <typeinfo>

struct Base { virtual ~Base() = default; };
struct Derived : Base { void special(); };

void handle(Base *b) {
    if (auto *d = dynamic_cast<Derived *>(b)) { // null on failure
        d->special();
    }
    try {
        Derived &dr = dynamic_cast<Derived &>(*b); // throws std::bad_cast
        dr.special();
    } catch (const std::bad_cast &) {
    }
}
```

`dynamic_cast` requires a polymorphic source type (at least one virtual function in the static type). It is not free: it walks the type hierarchy at run time. If you find yourself reaching for it often, the design probably wants another virtual function instead.

For the cast hierarchy (`static_cast`, `const_cast`, `reinterpret_cast`) see [Casting](#).

Pure Interfaces and the Non-Virtual Interface Idiom

Source

[src/polymorphism/nvi](#)

A common pattern is to expose a non-virtual public API that calls into private virtual hooks. The base class fixes pre/post conditions; derived classes only customize behavior.

```
struct Logger {
    void log(std::string_view msg) {           // public, non-virtual
        prefix();
        write(msg);                           // protected/private virtual
        flush();
    }
    virtual ~Logger() = default;
private:
    void prefix() { /* ... */ }
    virtual void write(std::string_view) = 0;
    void flush() { /* ... */ }
};
```

This *Non-Virtual Interface* (NVI) idiom keeps the contract in one place and makes the override surface explicit.

Virtual vs. CRTP — When to Pick Which

Source

[src/template/crtp](#)

If you do not need heterogeneous containers or runtime dispatch, the *Curiously Recurring Template Pattern* gives polymorphism without a vtable and inlines cleanly:

```
template <typename Derived>
struct Shape {
    double area() const {
        return static_cast<const Derived &>(*this).area_impl();
    }
};

struct Circle : Shape<Circle> {
    double area_impl() const { return 3.14159 * r_ * r_; }
    double r_;
};
```

Choose dynamic polymorphism when:

- You need to store mixed concrete types in one collection.
- The set of derived types is not known at the call site.
- The runtime cost of an indirect call is negligible compared to the work inside the function.

Choose static polymorphism (CRTP, concepts, templates) when:

- All types are known at compile time.
- The function is small and benefits from inlining.
- You want zero-overhead abstractions.

See [Template](#) for CRTP details and [Concepts](#) for the modern concepts-based alternative.

Trait-Style Polymorphism (Type Erasure)

The third flavor of polymorphism — the one Rust makes idiomatic with `dyn Trait` — is *type erasure*. Unrelated concrete types are stored behind a value-semantic wrapper that calls into a hidden vtable; no shared base class is required.

```
std::vector<AnyDrawable> shapes;
shapes.emplace_back(Circle{1.0}); // no inheritance from anything
shapes.emplace_back(Square{2.0});
```

This pattern is large enough that it has its own chapter: see *Type Erasure* for the Concept/Model recipe, small-buffer optimization, `std::function`/`std::any`/`std::move_only_function`, and the comparison with Rust trait objects.

Common Pitfalls

- **Non-virtual destructor in a polymorphic base.** Deleting through a base pointer is undefined behavior. Always make the destructor `virtual` or `protected` (the latter prevents base-pointer deletion entirely).
- **Calling virtual functions from constructors or destructors.** Dispatch uses the type currently under construction, not the most-derived type.
- **Slicing on copy.** Storing or passing polymorphic objects by value silently drops their derived state.
- **Forgetting override.** Without it, a signature change in the base silently turns an override into a new function.
- **Default argument values.** Defaults are picked from the *static* type of the call, not the dynamic type. `base->f()` uses `Base`'s defaults even if the override declares different ones — confusing and a frequent source of bugs. Avoid changing defaults in overrides.
- **Calling `dynamic_cast` in a hot loop.** It walks the type hierarchy. Cache the result, or refactor to add a virtual function.
- **Virtual inheritance “just to be safe”.** It adds storage and indirection. Use it only for genuine diamonds with shared state.

1.3.10 Type Erasure

Table of Contents

- *Type Erasure*
 - *Why Type Erasure*
 - *The Concept / Model Pattern*
 - *Small Buffer Optimization (SBO)*
 - *std::function*
 - *std::any*
 - *std::move_only_function (C++23)*
 - *When to Reach for Type Erasure*
 - *Performance Notes*
 - *Common Pitfalls*
 - *See Also*

Type erasure is the third flavor of polymorphism in modern C++ — alongside inheritance-based dispatch (see *Polymorphism & Inheritance*) and template-based static dispatch (see *Template, Concepts*). It lets a *single non-templated value type* hold any concrete object that satisfies a behavioral contract, without forcing those objects to inherit from a common base class.

The standard library uses this technique extensively: `std::function`, `std::any`, `std::move_only_function` (C++23), and the polymorphic allocators in `<memory_resource>` are all type-erased wrappers. The same technique lets you build your own value-semantic interfaces.

In Rust, the equivalent is `Box<dyn Trait>`: an opaque pointer to any implementor of a trait, with a hidden vtable. C++ reaches the same end with a hand-rolled wrapper because there is no language-level construct for it. See *Traits and Generics* for the Rust side.

Why Type Erasure

Inheritance-based polymorphism couples behavior to type identity:

```
struct Drawable {                                // contract ...
    virtual std::string draw() const = 0;
    virtual ~Drawable() = default;
};
struct Circle : Drawable { /* ... */ };         // ... forces inheritance.
struct Square : Drawable { /* ... */ };
```

This is fine for closed sets you control. It breaks down when:

- A third-party library exports `Circle` and you cannot change it.
- You want value semantics — copy, move, store-in-vector — without `unique_ptr<Drawable>` plumbing on every call site.
- Two unrelated codebases each want to plug into the same interface.

Type erasure removes the inheritance requirement: any type with the right shape (`draw()` returning `std::string`) can be wrapped without modification.

The Concept / Model Pattern

Source

`src/type-erasure/any-drawable`

The standard recipe: a *Concept* abstract class declares the operations, a *Model* template implements them for any concrete type, and the public wrapper holds a `unique_ptr<Concept>`.

```
class AnyDrawable {
    // 1. Internal interface - never seen by callers.
    struct Concept {
        virtual ~Concept() = default;
        virtual std::string do_draw() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };
    // 2. Templated bridge from any T to the Concept interface.
    template <typename T>
    struct Model : Concept {
        T value;
        explicit Model(T v) : value(std::move(v)) {}
        std::string do_draw() const override { return value.draw(); }
        std::unique_ptr<Concept> clone() const override {
```

(continues on next page)

(continued from previous page)

```

    return std::make_unique<Model>(*this);
}
};
std::unique_ptr<Concept> p_;
public:
// 3. Type-erasing constructor.
template <typename T>
AnyDrawable(T x) : p_(std::make_unique<Model<T>>(std::move(x))) {}
// 4. Value semantics: deep copy via clone().
AnyDrawable(const AnyDrawable &o) : p_(o.p_->clone()) {}
AnyDrawable(AnyDrawable &&) noexcept = default;
// 5. Public API forwards to the hidden vtable.
std::string draw() const { return p_->do_draw(); }
};

```

Now any type with `.draw()` -> `std::string` works:

```

struct Circle { double r; std::string draw() const { return "circle"; } };
struct Square { double s; std::string draw() const { return "square"; } };

std::vector<AnyDrawable> shapes;
shapes.emplace_back(Circle{1.0});
shapes.emplace_back(Square{2.0});
for (const auto &s : shapes) std::cout << s.draw() << "\n";

```

Notice that `Circle` and `Square` know nothing about `AnyDrawable`; they just have to provide `draw()`.

Small Buffer Optimization (SBO)

Source

`src/type-erasure/sbo`

The naive wrapper above always heap-allocates. For small, frequently-copied types (lambdas, function pointers, integer-shaped state) the allocation dominates the call cost. The fix is a *small buffer optimization*: reserve a fixed-size aligned buffer in the wrapper, store small types in-place, fall back to the heap for everything else.

```

class AnyDrawable {
    static constexpr std::size_t kBuf = 32;

    struct VTable {
        std::string (*draw)(const void *);
        void (*copy)(void *dst, const void *src);
        void (*move)(void *dst, void *src) noexcept;
        void (*destroy)(void *) noexcept;
        bool inline_storage;
    };

    // One VTable per stored T, picked by `vtable_for<T>()` based on whether
    // T fits in `kBuf` and is nothrow-move-constructible.

    alignas(std::max_align_t) std::byte storage_[kBuf];
    const VTable *vt_ = nullptr;
    // ... constructors / destructor dispatch through `vt_`.
};

```

Trade-offs:

- **Pro:** small types dispatch through a single indirection with no allocator traffic; cache locality improves dramatically.
- **Con:** `sizeof(AnyDrawable)` grows by `kBuf`; the wrapper is no longer a thin pointer.
- **Con:** SBO interacts badly with type-changing assignment if you do not destroy the old value in place before constructing the new one.

The standard library's `std::function` implementations all use SBO; the buffer size is implementation-defined (libc++ historically uses a few pointers' worth, MSVC slightly more). Do not rely on a particular cutoff.

std::function

Source

[src/type-erasure/std-function](#)

`std::function<R(Args...)>` is the canonical type-erased *callable* wrapper. It accepts function pointers, lambdas (with or without captures), member function pointers, and any object with a matching `operator()`.

```
std::function<int(int, int)> f;
f = [](int a, int b) { return a + b; };
f = std::plus<int>{};
```

Three things to remember about `std::function`:

- **Copyable target requirement.** The stored callable must be CopyConstructible. This rules out lambdas that capture `std::unique_ptr` by value. Workarounds: capture a `shared_ptr`, or use `std::move_only_function` (C++23, see below).
- **It is not free.** A virtual-style indirection per call, plus possibly an allocation if the callable does not fit in the SBO. On a hot path this matters; `std::function_ref` (proposed) or a hand-rolled wrapper does not.
- **Type erasure of the call signature, not the callable type.** Two `std::function<int(int)>` values can hold completely different underlying types — that is the whole point — but they are not comparable for equality.

std::any

Source

[src/type-erasure/std-any](#)

`std::any` (C++17) erases the *type* of a value but not any behavior. You can store anything copyable, then `any_cast` it back to the original type to do anything useful.

```
std::any a = 42;
int x = std::any_cast<int>(a);           // OK
std::string s = std::any_cast<std::string>(a); // throws std::bad_any_cast
int *p = std::any_cast<int>(&a);         // pointer form: nullptr on mismatch
```

Use `std::any` for opaque payloads (configuration values, attached properties, plugin interop) where the consumer knows the type at the retrieval point. If you want to *call* methods polymorphically without knowing the underlying type, you want a Concept/Model wrapper instead — `std::any` cannot do that.

std::move_only_function (C++23)

Source

src/type-erasure/move-only-function

std::move_only_function lifts the CopyConstructible requirement so move-only callables fit:

```
std::move_only_function<int()> f =  
    [p = std::make_unique<int>(42)] { return *p; }; // OK  
auto g = std::move(f);                             // copy: ill-formed
```

It is move-constructible and move-assignable but not copyable. Use it for one-shot tasks, channel sends, and anywhere the captured state is itself move-only (most heap-owning resources). Where std::function would force you to shared_ptr the captured state, std::move_only_function lets you keep the original ownership semantics.

When to Reach for Type Erasure

Choose type erasure when:

- You need polymorphic *value* semantics — copies, containers, pass-by-value — without unique_ptr<Base> plumbing.
- The set of concrete types is open or owned by other people.
- You want the call site to take a single concrete type rather than a template parameter.

Choose virtual functions (see [Polymorphism & Inheritance](#)) when:

- The hierarchy is closed and you control all derived types.
- Inheritance-based modeling is already idiomatic in the codebase.

Choose CRTP / concepts (see [Template](#), [Concepts](#)) when:

- All types are known at compile time.
- You want zero-overhead dispatch and full inlining.
- The interface is small and the ABI surface is your own translation unit.

Performance Notes

- Each type-erased call costs **one indirect call** (vtable lookup) — same order as a virtual call.
- Without SBO, construction and copy include a heap allocation. SBO removes that for small targets.
- The compiler usually cannot inline through the vtable. Hot inner loops should not call through AnyDrawable or std::function; pull the call out of the loop or use a static interface.
- Stack frames in profilers may show a generic Model<T>::do_draw with T mangled; debug symbols help.

Common Pitfalls

- **Forgetting clone() on the Concept.** Without it the wrapper is move-only — fine if intentional, surprising if not.
- **Slicing on construction.** AnyDrawable(T x) takes T by value, which copies. To avoid the copy, write AnyDrawable(T &&x) and forward.
- **Lifetime bugs in non-owning erasers.** A function_ref-style wrapper that stores a raw pointer to its target dangles if the target dies first. Owning wrappers (std::function, the example above) avoid this; refs are an explicit performance trade.

- **Hidden allocation in tight loops.** A `std::function` re-assigned in a loop may allocate on each assignment if the new callable does not fit in the SBO. Reuse the wrapper, or move it.
- **Mixing copy-only and move-only erasers.** `std::function` cannot hold a move-only lambda. Either capture by `shared_ptr` or switch to `std::move_only_function`.

See Also

- *Polymorphism & Inheritance* — virtual functions, the inheritance-based flavor
- *Template* — CRTP and template-based static dispatch
- *Concepts* — modern constraints replacing SFINAE
- *Traits and Generics* — Rust trait objects (`dyn Trait`), the language-level analog

1.3.11 Constexpr

Table of Contents

- *Constexpr*
 - *constexpr Functions*
 - *constexpr vs Template Metaprogramming*
 - *constexpr Variables*
 - *constexpr if (C++17)*
 - *if consteval: Detecting Constant Evaluation (C++23)*
 - *constexpr: Immediate Functions (C++20)*
 - *constexpr: Constant Initialization (C++20)*
 - *constexpr Evolution by Standard*

The `constexpr` keyword, introduced in C++11, enables compile-time evaluation of expressions and functions. By shifting computation from runtime to compile time, `constexpr` improves performance, enables stronger type checking, and allows values to be used in contexts requiring compile-time constants (such as array sizes and template arguments). C++14, C++17, and C++20 progressively relaxed restrictions, making `constexpr` increasingly powerful. Related keywords `constexpr` (C++20) and `constexpr` (C++20) provide finer control over when evaluation and initialization must occur.

constexpr Functions

Source

[src/constexpr/constexpr-function](#)

The `constexpr` specifier declares that a function can be evaluated at compile time when called with constant expressions. This enables the compiler to compute results during compilation, eliminating runtime overhead entirely. When called with non-constant arguments, the function executes at runtime like a normal function. This dual behavior makes `constexpr` functions versatile for both compile-time and runtime use.

```
#include <chrono>
#include <iostream>
```

(continues on next page)

(continued from previous page)

```
constexpr long fib(long n) {
    return (n < 2) ? n : fib(n - 1) + fib(n - 2);
}

int main() {
    // Runtime evaluation
    auto start = std::chrono::system_clock::now();
    long r1 = fib(40);
    std::chrono::duration<double> d1 = std::chrono::system_clock::now() - start;
    std::cout << "Runtime: " << d1.count() << "s\n";

    // Compile-time evaluation
    start = std::chrono::system_clock::now();
    constexpr long r2 = fib(40);
    std::chrono::duration<double> d2 = std::chrono::system_clock::now() - start;
    std::cout << "Compile-time: " << d2.count() << "s\n";
}
```

```
$ g++ -std=c++17 -O3 a.cpp && ./a.out
Runtime: 0.268229s
Compile-time: 8e-06s
```

constexpr vs Template Metaprogramming

Source

[src/constexpr/constexpr-vs-tmp](#)

Before C++11, compile-time computation required template metaprogramming (TMP), which uses recursive template instantiation. While powerful, TMP is verbose and difficult to read. The `constexpr` keyword provides a cleaner alternative that looks like regular code. Both approaches achieve compile-time evaluation, but `constexpr` is more maintainable and easier to debug.

```
#include <iostream>

// Template metaprogramming approach (pre-C++11 style)
template <long N>
struct Fib {
    static constexpr long value = Fib<N - 1>::value + Fib<N - 2>::value;
};

template <>
struct Fib<0> {
    static constexpr long value = 0;
};

template <>
struct Fib<1> {
    static constexpr long value = 1;
};

// constexpr approach (modern C++)
constexpr long fib(long n) {
```

(continues on next page)

(continued from previous page)

```

    return (n < 2) ? n : fib(n - 1) + fib(n - 2);
}

int main() {
    constexpr long r1 = Fib<40>::value; // TMP
    constexpr long r2 = fib(40);        // constexpr
    std::cout << r1 << " " << r2 << "\n"; // Both: 102334155
}

```

constexpr Variables

Source

[src/constexpr/constexpr-variable](#)

A `constexpr` variable must be initialized with a constant expression and its value is fixed at compile time. Unlike `const`, which only promises the variable won't be modified after initialization, `constexpr` guarantees the value is known at compile time. This makes `constexpr` variables usable in contexts requiring compile-time constants, such as array sizes and template arguments.

```

#include <array>
#include <iostream>

constexpr int square(int x) { return x * x; }

int main() {
    constexpr int size = square(4); // Computed at compile time: 16
    std::array<int, size> arr{};      // Array size must be compile-time constant
    std::cout << arr.size() << "\n"; // Output: 16

    const int runtime_val = size; // const: won't change, but could be runtime
    constexpr int compile_val = size; // constexpr: guaranteed compile-time
}

```

constexpr if (C++17)

Source

[src/constexpr/constexpr-if](#)

C++17 introduced `if constexpr`, which evaluates conditions at compile time and discards the untaken branch entirely. This is essential for template metaprogramming because it prevents compilation errors in branches that would be invalid for certain template instantiations. Unlike regular `if`, both branches of a normal `if` must be valid code even if one is never executed.

```

#include <iostream>
#include <type_traits>

template <typename T>
auto get_value(T t) {
    if constexpr (std::is_pointer_v<T>) {
        return *t; // Only compiled when T is a pointer
    } else {
        return t; // Only compiled when T is not a pointer
    }
}

```

(continues on next page)

(continued from previous page)

```

}

int main() {
    int x = 42;
    int *p = &x;

    std::cout << get_value(x) << "\n"; // Output: 42
    std::cout << get_value(p) << "\n"; // Output: 42
}

```

if constexpr: Detecting Constant Evaluation (C++23)

Source

[src/constexpr/if-constexpr](#)

C++23 introduced `if constexpr` to detect whether code is executing during constant evaluation. This allows a function to choose different implementations for compile-time versus runtime execution. Unlike `if constexpr`, which evaluates a compile-time boolean condition, `if constexpr` checks the evaluation context itself. This is useful when compile-time evaluation requires a different algorithm (e.g., a safer but slower approach) than runtime execution.

```

#include <iostream>

constexpr int compute(int x) {
    if constexpr {
        // Compile-time path: safe but potentially slower
        return x * x;
    } else {
        // Runtime path: can use optimized or platform-specific code
        return x * x; // In practice, might call intrinsics here
    }
}

int main() {
    constexpr int a = compute(5); // Uses compile-time path
    int b = compute(5);           // Uses runtime path
    std::cout << a << " " << b << "\n"; // Output: 25 25
}

```

Negated form (if !constexpr):

```

constexpr int optimized(int x) {
    if !constexpr {
        // Runtime-only optimizations
        return x * x;
    }
    // Compile-time fallback
    return x * x;
}

```

constexpr: Immediate Functions (C++20)

Source

src/constexpr/constexpr

C++20 introduced `constexpr` to declare *immediate functions* that must be evaluated at compile time. Unlike `constexpr`, which allows runtime evaluation when arguments are not constant, `constexpr` functions produce a compilation error if called with non-constant arguments. This guarantees that the function never generates runtime code, making it useful for compile-time-only utilities like string hashing or lookup table generation.

```
#include <iostream>

constexpr int square(int x) { return x * x; }

int main() {
    constexpr int a = square(4); // OK: compile-time evaluation
    std::cout << a << "\n";      // Output: 16

    // int b = 5;
    // int c = square(b); // Error: b is not a constant expression
}
```

Comparison of `constexpr` vs `constexpr`:

```
constexpr int ce_square(int x) { return x * x; }
constexpr int cv_square(int x) { return x * x; }

int main() {
    int runtime_val = 5;

    int a = ce_square(runtime_val); // OK: runtime evaluation
    // int b = cv_square(runtime_val); // Error: must be compile-time

    constexpr int c = ce_square(5); // OK: compile-time
    constexpr int d = cv_square(5); // OK: compile-time
}
```

constexpr: Constant Initialization (C++20)

Source

src/constexpr/constexpr

C++20 also introduced `constexpr` to ensure a variable is initialized at compile time, avoiding the *static initialization order fiasco*. Unlike `constexpr`, a `constexpr` variable is not `const` and can be modified at runtime. It only guarantees that initialization happens at compile time, preventing issues where static variables in different translation units depend on each other's initialization order.

```
#include <iostream>

constexpr int compute() { return 42; }

constexpr int global = compute(); // Initialized at compile time

int main() {
    std::cout << global << "\n"; // Output: 42
}
```

(continues on next page)

(continued from previous page)

```
global = 100;           // OK: constexpr doesn't mean const
std::cout << global << "\n"; // Output: 100
}
```

constexpr Evolution by Standard

C++11:

- Introduced `constexpr` for simple functions (single return statement)
- `constexpr` variables must be initialized with constant expressions

C++14:

- `constexpr` functions can have multiple statements, loops, and local variables
- `constexpr` member functions are no longer implicitly `const`

C++17:

- `if constexpr` for compile-time conditional compilation
- `constexpr` lambdas

C++20:

- `constexpr` for immediate functions (must evaluate at compile time)
- `constexpr` for compile-time initialization of non-`const` variables
- `constexpr` virtual functions
- `constexpr` dynamic allocation (`new/delete`) within constant expressions
- `constexpr` `std::vector` and `std::string`

C++23:

- `if constexpr` for detecting constant evaluation context
- Relaxed `constexpr` restrictions on non-literal types

1.3.12 Lambda

Table of Contents

- *Lambda*
 - *Lambda Syntax Overview*
 - *Callable Objects vs Lambdas*
 - *Captureless Lambdas and Function Pointers*
 - *Mutable Lambdas*
 - *Immediately Invoked Lambda Expression (IILE)*
 - *Lambda as Callback*
 - *Recursive Lambdas*
 - *Init Capture (C++14)*

- *Generic Lambdas (C++14)*
- *constexpr Lambdas (C++17)*
- *Template Lambdas (C++20)*
- *Default-Constructible Lambdas (C++20)*
- *Lambdas in Unevaluated Contexts (C++20)*
- *Pack Capture (C++20)*
- *Deducing this (C++23)*
- *Lambda Attributes (C++23)*
- *Lambda Evolution by Standard*

Lambda expressions, introduced in C++11, provide a concise way to create anonymous function objects inline. They are essential for modern C++ programming, enabling functional programming patterns, simplifying callbacks, and working seamlessly with STL algorithms. Lambdas eliminate the boilerplate of writing separate functor classes while offering the same performance characteristics through compiler-generated closure types.

Lambda Syntax Overview

Source

[src/lambda/syntax](#)

A lambda expression consists of a capture clause, parameter list, optional specifiers, and a body. The capture clause `[]` determines which variables from the enclosing scope are accessible inside the lambda. The compiler generates a unique closure type for each lambda, with `operator()` containing the lambda body.

```
// [capture](parameters) specifiers -> return_type { body }

#include <iostream>

int main() {
    int x = 10;

    auto by_value = [x]() { return x * 2; };           // Capture x by value
    auto by_ref = [&x]() { x *= 2; };                 // Capture x by reference
    auto all_val = [=]() { return x; };               // Capture all by value
    auto all_ref = [&]() { x = 0; };                   // Capture all by reference

    std::cout << by_value() << "\n"; // Output: 20
}
```

Callable Objects vs Lambdas

Source

[src/lambda/callable](#)

Before lambdas, callable objects (functors) required defining a class with `operator()`. Lambdas provide the same functionality with less boilerplate. The compiler transforms each lambda into an anonymous class where captures become member variables initialized via constructor, and the lambda body becomes `operator()`.

```
int x = 10;
auto f = [x](int y) { return x + y; };
```

The compiler generates something equivalent to:

```
class __lambda_1 {
    int x; // Captured variable as member
public:
    __lambda_1(int x) : x(x) {} // Constructor initializes capture
    int operator()(int y) const { return x + y; } // Lambda body
};

__lambda_1 f(x); // Instantiate with captured value
```

Capture by reference ([&x]) stores a reference member instead. This is why lambdas with captures cannot convert to function pointers—they require storage for those members.

Functor vs Lambda comparison:

```
#include <functional>
#include <iostream>

// Functor approach (pre-C++11)
class Fib {
public:
    long operator()(long n) const {
        return (n < 2) ? n : operator()(n - 1) + operator()(n - 2);
    }
};

int main() {
    Fib fib;
    std::cout << fib(10) << "\n"; // Output: 55

    // Lambda equivalent (requires std::function for recursion)
    std::function<long(long)> fib_lambda = [&](long n) {
        return (n < 2) ? n : fib_lambda(n - 1) + fib_lambda(n - 2);
    };
    std::cout << fib_lambda(10) << "\n"; // Output: 55
}
```

Captureless Lambdas and Function Pointers

Source

[src/lambda/captureless](#)

Lambdas without captures can implicitly convert to function pointers, making them compatible with C APIs and legacy code expecting function pointers. This conversion is only possible when the lambda captures nothing, as captured variables require storage that function pointers cannot provide.

```
#include <iostream>

int main() {
    // Captureless lambda converts to function pointer
```

(continues on next page)

(continued from previous page)

```

long (*fib)(long) = [](long n) {
    long a = 0, b = 1;
    for (long i = 0; i < n; ++i) {
        long tmp = b;
        b = a + b;
        a = tmp;
    }
    return a;
};

std::cout << fib(10) << "\n"; // Output: 55
}

```

Mutable Lambdas

Source

[src/lambda/mutable](#)

By default, lambdas capturing by value have a `const` `operator()`, preventing modification of captured copies. The `mutable` keyword removes this `const` qualification, allowing the lambda to modify its captured state across calls.

```

#include <iostream>

int main() {
    int counter = 0;

    auto increment = [counter]() mutable {
        return ++counter; // Modifies the lambda's copy
    };

    std::cout << increment() << "\n"; // Output: 1
    std::cout << increment() << "\n"; // Output: 2
    std::cout << counter << "\n";    // Output: 0 (original unchanged)
}

```

Immediately Invoked Lambda Expression (IILE)

Source

[src/lambda/iile](#)

Lambdas can be invoked immediately after definition, useful for complex initialization of const variables or breaking out of nested loops. This pattern is called IILE (Immediately Invoked Lambda Expression).

```

#include <iostream>

int main() {
    // Complex const initialization
    const int value = []() {
        int result = 0;
        for (int i = 1; i <= 10; ++i) result += i;
        return result;
    }();
}

```

(continues on next page)

(continued from previous page)

```

std::cout << value << "\n"; // Output: 55

// Breaking nested loops
[&]() {
    for (int i = 0; i < 5; ++i) {
        for (int j = 0; j < 5; ++j) {
            if (i + j == 5) return; // Breaks both loops
            std::cout << i + j << " ";
        }
    }
}();
std::cout << "\n";
}

```

Lambda as Callback

Source

`src/lambda/callback`

Lambdas are ideal for callbacks in algorithms and asynchronous operations. Use templates for zero-overhead callbacks, `std::function` when type erasure is needed, or function pointers for C API compatibility (captureless only).

```

#include <functional>
#include <iostream>

// Template: zero overhead, accepts any callable
template <typename F>
void process(int n, F callback) {
    for (int i = 0; i < n; ++i) callback(i);
}

// std::function: type erasure, slight overhead
void process2(int n, std::function<void(int)> callback) {
    for (int i = 0; i < n; ++i) callback(i);
}

int main() {
    process(5, [](int x) { std::cout << x << " "; });
    std::cout << "\n";

    int sum = 0;
    process2(5, [&sum](int x) { sum += x; });
    std::cout << sum << "\n"; // Output: 10
}

```

Recursive Lambdas

Source

`src/lambda/recursive`

Lambdas cannot directly reference themselves. Two common solutions exist: using `std::function` (simpler but slower due to type erasure) or passing the lambda to itself (faster, approaching normal function performance).


```

#include <functional>
#include <iostream>

int main() {
    // Method 1: std::function (slower)
    std::function<long(long)> fib1 = [&](long n) {
        return n < 2 ? n : fib1(n - 1) + fib1(n - 2);
    };

    // Method 2: Self-passing (faster)
    auto fib2 = [](auto&& self, long n) -> long {
        return n < 2 ? n : self(self, n - 1) + self(self, n - 2);
    };

    std::cout << fib1(20) << "\n";          // Output: 6765
    std::cout << fib2(fib2, 20) << "\n";    // Output: 6765
}

```

Init Capture (C++14)

Source

[src/lambda/init-capture](#)

C++14 introduced *init capture* (also called generalized lambda capture), allowing you to create new variables in the capture clause with arbitrary initializers. This enables move-capturing, renaming variables, and capturing expressions. Init capture is essential for capturing move-only types like `std::unique_ptr`.

```

#include <iostream>
#include <memory>
#include <utility>

int main() {
    auto ptr = std::make_unique<int>(42);

    // Move-capture: transfers ownership into lambda
    auto f = [p = std::move(ptr)]() { std::cout << *p << "\n"; };
    f(); // Output: 42

    // ptr is now nullptr

    // Capture with expression
    int x = 10;
    auto g = [y = x * 2]() { return y; };
    std::cout << g() << "\n"; // Output: 20
}

```

Generic Lambdas (C++14)

Source

[src/lambda/generic](#)

C++14 introduced generic lambdas with `auto` parameters, creating a template `operator()` under the hood. This allows a single lambda to work with multiple types without explicit template syntax. Combined with fold expressions (C++17), generic lambdas enable powerful variadic operations.

```
#include <iostream>
#include <utility>

int main() {
    // Generic lambda with auto parameters
    auto sum = [](auto&&... args) {
        return (std::forward<decltype(args)>(args) + ...); // Fold expression
    };

    std::cout << sum(1, 2, 3, 4, 5) << "\n"; // Output: 15
    std::cout << sum(1.5, 2.5, 3.0) << "\n"; // Output: 7.0
}
```

constexpr Lambdas (C++17)

Source

[src/lambda/constexpr-lambda](#)

Since C++17, lambdas are implicitly `constexpr` when possible, allowing compile-time evaluation. You can also explicitly mark a lambda `constexpr` or `constexpr` (C++20) to enforce compile-time evaluation.

```
#include <iostream>

int main() {
    auto fib = [](long n) {
        long a = 0, b = 1;
        for (long i = 0; i < n; ++i) {
            long tmp = b;
            b = a + b;
            a = tmp;
        }
        return a;
    };

    static_assert(fib(10) == 55); // Compile-time evaluation
    std::cout << fib(10) << "\n";
}
```

Template Lambdas (C++20)

Source

[src/lambda/template-lambda](#)

C++20 allows explicit template parameter lists in lambdas, providing more control than generic lambdas with `auto`. This enables template constraints, accessing type information directly, and cleaner syntax when forwarding arguments. Template lambdas eliminate the need for `decltype` workarounds.

```
#include <iostream>
#include <utility>
#include <concepts>

int main() {
    // Template lambda with explicit parameters
    auto sum = []<typename... Args>(Args&&... args) {
```

(continues on next page)

(continued from previous page)

```

    return (std::forward<Args>(args) + ...);
};

std::cout << sum(1, 2, 3, 4, 5) << "\n"; // Output: 15

// With concepts constraint
auto add = [<typename T>(T a, T b) requires std::integral<T> {
    return a + b;
};

std::cout << add(10, 20) << "\n"; // Output: 30
}

```

Default-Constructible Lambdas (C++20)

Source

src/lambda/default-constructible

In C++20, stateless (captureless) lambdas are default-constructible and assignable. This allows using lambda types directly as template arguments without passing an instance, simplifying code for containers and algorithms that require comparators or hash functions.

```

#include <iostream>
#include <map>
#include <set>

int main() {
    // C++20: Lambda type as template argument, no instance needed
    auto cmp = [](int a, int b) { return a > b; };

    std::set<int, decltype(cmp)> s1; // C++20: default constructs comparator
    s1.insert({3, 1, 4, 1, 5});

    for (int x : s1) std::cout << x << " "; // Output: 5 4 3 1
    std::cout << "\n";
}

```

Lambdas in Unevaluated Contexts (C++20)

Source

src/lambda/unevaluated

C++20 permits lambdas in unevaluated contexts like `sizeof`, `decltype`, and template arguments. Combined with default construction, this enables declaring containers with lambda comparators without creating lambda instances.

```

#include <iostream>
#include <map>
#include <string>

int main() {
    // Lambda in unevaluated context (decltype)
    std::map<int, std::string, decltype([](int a, int b) { return a > b; })> m;
}

```

(continues on next page)

(continued from previous page)

```

m[3] = "three";
m[1] = "one";
m[2] = "two";

for (const auto& [k, v] : m) {
    std::cout << k << ": " << v << "\n";
}
// Output: 3: three, 2: two, 1: one (descending order)
}

```

Pack Capture (C++20)

Source

[src/lambda/pack-capture](#)

C++20 allows capturing parameter packs with `init-capture`, enabling perfect forwarding of variadic arguments into lambdas for deferred execution.

```

#include <iostream>
#include <tuple>
#include <utility>

template <typename... Args>
auto make_delayed_call(Args&&... args) {
    return [...args = std::forward<Args>(args)]() {
        return (args + ...);
    };
}

int main() {
    auto delayed = make_delayed_call(1, 2, 3, 4, 5);
    std::cout << delayed() << "\n"; // Output: 15
}

```

Deducing this (C++23)

Source

[src/lambda/deducing-this](#)

C++23 introduces *deducing this*, allowing lambdas to take an explicit object parameter. This enables recursive lambdas without `std::function` or self-passing, and provides access to the lambda's own type for advanced patterns.

```

#include <iostream>

int main() {
    // C++23: Recursive lambda with deducing this
    auto fib = [](this auto&& self, long n) -> long {
        return n < 2 ? n : self(n - 1) + self(n - 2);
    };

    std::cout << fib(20) << "\n"; // Output: 6765
}

```

Lambda Attributes (C++23)

C++23 allows attributes on the lambda's `operator()`, enabling optimizations and compiler hints like `[[nodiscard]]`, `[[likely]]`, and `[[deprecated]]`.

```
#include <iostream>

int main() {
    auto compute = []() [[nodiscard]] { return 42; };

    // compute(); // Warning: ignoring return value with [[nodiscard]]
    int x = compute(); // OK
    std::cout << x << "\n";
}
```

Lambda Evolution by Standard

C++11:

- Basic lambda syntax with capture, parameters, and body
- Capture by value [=] and by reference [&]

C++14:

- Generic lambdas with auto parameters
- Init capture (generalized lambda capture)
- Return type deduction for all lambdas

C++17:

- `constexpr` lambdas (implicit when possible)
- Capture of `*this` by value

C++20:

- Template parameter lists `[]<typename T>(T x) {}`
- Default-constructible and assignable stateless lambdas
- Lambdas in unevaluated contexts
- Pack capture with init-capture
- Lambdas with concepts constraints

C++23:

- Deducing `this` for explicit object parameters
- Attributes on lambda `operator()`
- Simplified syntax improvements

1.3.13 Concepts

Table of Contents

- *Concepts*
 - *Defining Concepts*
 - *Standard Library Concepts*
 - * *Common Standard Concepts*
 - *Requires Expressions*
 - *Concept Syntax Variations*
 - *Concept Subsumption*
 - *Practical Example: Serializable*
 - *See Also*

Concepts, introduced in C++20, provide a way to specify constraints on template parameters directly in the language. They replace verbose SFINAE techniques and `std::enable_if` with readable, composable constraints that produce clear compiler error messages. A concept is essentially a named compile-time boolean predicate that documents and enforces requirements on template arguments. This guide covers defining custom concepts, using standard library concepts, writing requires expressions, and understanding how concept subsumption affects overload resolution.

Defining Concepts

Source

`src/concepts/defining`

A concept is defined using the `concept` keyword followed by a `requires` expression or any compile-time boolean expression. You can build concepts from type traits like `std::is_arithmetic_v`, from `requires` expressions that check for valid operations, or by combining existing concepts with logical operators (`&&`, `|`). Once defined, a concept can be used to constrain template parameters, providing clear documentation of requirements and producing readable error messages when constraints are not satisfied.

```
#include <concepts>
#include <type_traits>

// Simple concept using type traits
template <typename T>
concept Numeric = std::is_arithmetic_v<T>;

// Concept using requires expression
template <typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>;
};

// Compound concept
template <typename T>
concept Number = Numeric<T> && Addable<T>;

template <Number T>
T add(T a, T b) {
    return a + b;
}
```

Standard Library Concepts

Source

src/concepts/stdlib

The `<concepts>` header provides a rich set of predefined concepts organized into categories: core language concepts (`same_as`, `derived_from`, `convertible_to`), comparison concepts (`equality_comparable`, `totally_ordered`), object concepts (`movable`, `copyable`, `regular`), and callable concepts (`invocable`, `predicate`). These standard concepts cover common constraints and serve as building blocks for defining more specific custom concepts.

```
#include <concepts>

// Core language concepts
static_assert(std::same_as<int, int>);
static_assert(std::derived_from<std::string, std::string>);
static_assert(std::convertible_to<int, double>);

// Comparison concepts
static_assert(std::equality_comparable<int>);
static_assert(std::totally_ordered<double>);

// Object concepts
static_assert(std::movable<std::string>);
static_assert(std::copyable<int>);
static_assert(std::regular<int>);

// Callable concepts
template <std::invocable<int> F>
void call_with_42(F&& f) {
    f(42);
}
```

Common Standard Concepts

Concept	Description
<code>std::same_as<T, U></code>	T and U are the same type
<code>std::derived_from</code>	Derived inherits from Base
<code>std::convertible_to</code>	T is implicitly convertible to U
<code>std::integral</code>	T is an integral type
<code>std::floating_point</code>	T is a floating-point type
<code>std::invocable<F, Args></code>	F can be called with Args
<code>std::predicate<F, Args></code>	F returns bool when called with Args

Requires Expressions

Source

src/concepts/requires-expr

Requires expressions provide a powerful way to check compile-time properties of types. They support four kinds of requirements: simple requirements check that an expression is valid, type requirements verify that a nested type exists, compound requirements constrain both validity and return type of an expression, and nested requirements allow embedding additional boolean constraints. This makes requires expressions ideal for defining concepts that check interface conformance, like verifying a type has specific member functions with expected signatures.

```
template <typename T>
concept Container = requires(T c) {
    // Simple requirement - expression must be valid
    c.begin();
    c.end();
    c.size();

    // Type requirement - nested type must exist
    typename T::value_type;
    typename T::iterator;

    // Compound requirement - expression with return type constraint
    { c.size() } -> std::convertible_to<std::size_t>;
    { *c.begin() } -> std::same_as<typename T::value_type&>;

    // Nested requirement
    requires std::same_as<decltype(c.begin()), typename T::iterator>;
};
```

Concept Syntax Variations

Source

[src/concepts/syntax](#)

C++20 provides four equivalent ways to apply concept constraints to templates. The requires clause after the template parameter list is the most explicit form. The trailing requires clause places constraints after the function signature. Constrained template parameters embed the concept directly in the parameter declaration. Finally, abbreviated function templates use auto with a concept for the most concise syntax. All four forms are semantically equivalent, so choose based on readability and team conventions.

```
#include <concepts>

// 1. Requires clause after template
template <typename T>
    requires std::integral<T>
T square1(T x) {
    return x * x;
}

// 2. Trailing requires clause
template <typename T>
T square2(T x) requires std::integral<T> {
    return x * x;
}

// 3. Constrained template parameter
template <std::integral T>
T square3(T x) {
    return x * x;
}

// 4. Abbreviated function template (auto)
auto square4(std::integral auto x) {
```

(continues on next page)

(continued from previous page)

```

return x * x;
}

```

Concept Subsumption

Source

`src/concepts/subsumption`

Concept subsumption determines overload resolution when multiple constrained functions match a call. A concept A subsumes concept B if A's constraints logically imply B's constraints. When both overloads are viable, the compiler selects the more constrained one. In the example below, `Dog` subsumes `Animal` because `Dog` requires everything `Animal` requires plus additional constraints. This enables writing generic fallback functions while providing specialized implementations for more specific types.

```

#include <concepts>
#include <iostream>

template <typename T>
concept Animal = requires(T t) { t.speak(); };

template <typename T>
concept Dog = Animal<T> && requires(T t) { t.bark(); };

void greet(Animal auto& a) {
    std::cout << "Hello, animal\n";
}

void greet(Dog auto& d) {
    std::cout << "Hello, dog\n";
    d.bark();
}

struct Cat {
    void speak() { std::cout << "Meow\n"; }
};

struct Beagle {
    void speak() { std::cout << "Woof\n"; }
    void bark() { std::cout << "Bark!\n"; }
};

int main() {
    Cat c;
    Beagle b;
    greet(c); // "Hello, animal"
    greet(b); // "Hello, dog" - Dog subsumes Animal
}

```

Practical Example: Serializable

This example demonstrates a practical `Serializable` concept that constrains types to those supporting stream insertion and extraction operators. The concept verifies that a type can be written to an `ostream` and read from an `istream`, enabling generic serialization functions that work with any conforming type including built-in types, `std::string`, and user-defined types with overloaded stream operators.

```
#include <concepts>
#include <sstream>
#include <string>

template <typename T>
concept Serializable = requires(T t, std::ostream& os, std::istream& is) {
    { os << t } -> std::same_as<std::ostream&>;
    { is >> t } -> std::same_as<std::istream&>;
};

template <Serializable T>
std::string to_string(const T& value) {
    std::ostringstream oss;
    oss << value;
    return oss.str();
}

template <Serializable T>
T from_string(const std::string& s) {
    std::istringstream iss(s);
    T value;
    iss >> value;
    return value;
}
```

See Also

- [Requires](#) - Detailed coverage of requires clauses and expressions
- [Template](#) - Template fundamentals

1.3.14 Requires

Table of Contents

- [Requires](#)
 - [Basic Requires Clause](#)
 - [Nested Requires Expression](#)
 - [Conjunction \(&&\)](#)
 - [Disjunction \(||\)](#)
 - [Requires vs if constexpr](#)
 - [Requires with Boolean Flags](#)

C++20 introduced the `requires` keyword as part of the concepts feature, providing a cleaner and more expressive way to constrain template parameters. Unlike traditional SFINAE techniques that rely on template substitution failures, `requires` clauses produce readable compiler error messages and make template constraints explicit in the function signature. This chapter covers basic `requires` clauses, nested `requires` expressions, logical combinations with conjunction and disjunction, and practical patterns for compile-time interface selection.

Basic Requires Clause

Source

`src/require/basic`

The `requires` clause appears after the template parameter list and before the function return type. It accepts any constant expression that evaluates to a boolean, including standard library concepts like `std::integral`, `std::floating_point`, or `std::copyable`. When the constraint evaluates to false, the template is excluded from overload resolution entirely, resulting in clear “no matching function” errors rather than cryptic substitution failure messages.

```
#include <concepts>

template <typename T>
    requires std::integral<T>
T add(T a, T b) {
    return a + b;
}

int main() {
    add(1, 2);           // OK: int is integral
    add(10L, 20L);       // OK: long is integral
    // add(1.0, 2.0);    // Error: double is not integral
}
```

Nested Requires Expression

Source

`src/require/nested`

A *requires expression* allows defining ad-hoc constraints inline without declaring a separate named concept. The syntax `requires requires(T x) { ... }` may look redundant, but the first `requires` introduces the clause while the second begins the expression body. Inside the body, you can check that expressions compile, verify return types using `-> concept` syntax, and test for noexcept guarantees. This approach is useful for one-off constraints that don’t warrant a reusable concept definition.

```
#include <concepts>

// Ad-hoc constraint: T must support + and * returning T
template <typename T>
    requires requires(T x) {
        { x + x } -> std::convertible_to<T>;
        { x * x } -> std::convertible_to<T>;
    }
T square_sum(T a, T b) {
    return (a + b) * (a + b);
}

int main() {
```

(continues on next page)

(continued from previous page)

```

square_sum(2, 3);      // OK: int supports + and *
square_sum(1.5, 2.5); // OK: double supports + and *
// square_sum(std::string("a"), std::string("b")); // Error: string has no *
}

```

Conjunction (&&)

Source

src/require/conjunction

Multiple constraints can be combined using the logical AND operator `&&` to require that all conditions are satisfied simultaneously. The compiler evaluates constraints from left to right and short-circuits on the first failure, which can improve compile times when earlier constraints are cheaper to check. Conjunction is commonly used to layer additional requirements on top of broad concepts, such as requiring both `std::integral` and `std::signed_integral` to exclude unsigned types.

```

#include <concepts>

// Both constraints must be satisfied
template <typename T>
requires std::integral<T> && std::signed_integral<T>
T abs_val(T x) {
    return x < 0 ? -x : x;
}

int main() {
    abs_val(-5);    // OK: int is signed integral
    abs_val(-100L); // OK: long is signed integral
    // abs_val(5u); // Error: unsigned int is not signed
}

```

Disjunction (||)

Source

src/require/disjunction

Constraints can be combined using the logical OR operator `||` to accept types that satisfy at least one of the specified conditions. This enables writing generic functions that work across multiple unrelated type categories without code duplication. For example, a numeric utility function might accept both integral and floating-point types by combining `std::integral<T> || std::floating_point<T>`, covering all built-in arithmetic types with a single template.

```

#include <concepts>

// Either constraint can be satisfied
template <typename T>
requires std::integral<T> || std::floating_point<T>
T twice(T x) {
    return x + x;
}

int main() {
    twice(5);    // OK: int is integral
    twice(2.5); // OK: double is floating_point
}

```

(continues on next page)

(continued from previous page)

```
// twice(std::string("x")); // Error: string is neither
}
```

Requires vs if constexpr

Source

src/require/constexpr-like

Both `if constexpr` (C++17) and `requires` (C++20) enable compile-time branching based on type properties, but they use different mechanisms. `if constexpr` keeps all code paths in a single function body and discards branches that don't apply, while `requires` uses overload resolution to select between entirely separate function definitions. The `requires` approach can be cleaner when the implementations differ significantly, and it allows each overload to have different return types or exception specifications.

```
#include <type_traits>

// Using if constexpr
template <typename T>
auto get_value_constexpr(T t) {
    if constexpr (std::is_pointer_v<T>) {
        return *t; // dereference pointer
    } else {
        return t; // return value directly
    }
}

// Using requires (overload resolution)
template <typename T>
requires std::is_pointer_v<T>
auto get_value_requires(T t) {
    return *t; // selected when T is pointer
}

template <typename T>
requires (!std::is_pointer_v<T>)
auto get_value_requires(T t) {
    return t; // selected when T is not pointer
}

int main() {
    int x = 42;
    int* p = &x;
    get_value_constexpr(x); // returns value
    get_value_constexpr(p); // dereferences pointer
    get_value_requires(x); // selects non-pointer overload
    get_value_requires(p); // selects pointer overload
}
```

Requires with Boolean Flags

Source

src/require/bool-flag

A powerful design pattern combines `requires` with compile-time boolean constants defined in traits or policy classes. By checking flags like `Backend::kSupportFoo`, you can conditionally enable or disable class methods based on backend capabilities. This technique is widely used in high-performance libraries where different backends (CPU, GPU, network transports) support different operations. Methods guarded by `requires` simply don't exist when the flag is false, preventing accidental use and enabling clean API design without runtime overhead.

```
// Backend traits with compile-time feature flags
struct BackendA {
    static constexpr bool kSupportFoo = true;
    static constexpr bool kSupportBar = true;
};

struct BackendB {
    static constexpr bool kSupportFoo = false;
    static constexpr bool kSupportBar = false;
};

template <typename Backend>
class Channel {
public:
    // Only available when Backend::kSupportFoo is true
    int Foo(int x)
        requires Backend::kSupportFoo
    {
        return x;
    }

    // Only available when Backend::kSupportBar is true
    int Bar()
        requires Backend::kSupportBar
    {
        return 42;
    }

    // Always available
    int Baz(int x) { return x * 2; }
};

int main() {
    Channel<BackendA> a;
    a.Foo(10); // OK: kSupportFoo is true
    a.Bar();   // OK: kSupportBar is true
    a.Baz(5);  // OK: always available

    Channel<BackendB> b;
    // b.Foo(10); // Error: kSupportFoo is false
    // b.Bar();   // Error: kSupportBar is false
    b.Baz(5);    // OK: always available
}
```

1.3.15 Smart Pointers

Table of Contents

- *Smart Pointers*
 - *std::unique_ptr: Exclusive Ownership*
 - *std::shared_ptr: Shared Ownership*
 - *std::weak_ptr: Non-Owning Observer*
 - *std::make_unique and std::make_shared*
 - *Custom Deleters*
 - *enable_shared_from_this*
 - *Aliasing Constructor (shared_ptr)*
 - *std::make_unique_for_overwrite (C++20)*
 - *std::shared_ptr Array Support (C++17/C++20)*
 - *Atomic Operations on shared_ptr (C++20)*
 - *out_ptr and inout_ptr (C++23)*
 - *Common Pitfalls*
 - *Smart Pointer Comparison*

Smart pointers, introduced in C++11, provide automatic memory management through RAII (Resource Acquisition Is Initialization). They ensure that dynamically allocated memory is properly deallocated when no longer needed, preventing memory leaks and dangling pointers. Unlike raw pointers, smart pointers automatically manage the lifetime of the objects they point to, calling the appropriate destructor or deleter when the pointer goes out of scope. The three main smart pointer types are `unique_ptr` for exclusive ownership, `shared_ptr` for shared ownership with reference counting, and `weak_ptr` for non-owning references that don't prevent deallocation.

std::unique_ptr: Exclusive Ownership

Source

`src/smartptr/unique-ptr`

`std::unique_ptr` represents exclusive ownership of a dynamically allocated object. Only one `unique_ptr` can own a given object at a time, enforcing a clear ownership model that prevents accidental sharing. When the `unique_ptr` is destroyed, goes out of scope, or is reset, the owned object is automatically deleted. Because ownership is exclusive, `unique_ptr` cannot be copied—only moved—which transfers ownership from one pointer to another. This makes `unique_ptr` ideal for factory functions, RAII wrappers, and any situation where a single owner is responsible for an object's lifetime. It has zero overhead compared to raw pointers when using the default deleter.

```
#include <memory>
#include <iostream>

int main() {
    auto ptr = std::make_unique<int>(42);
    std::cout << *ptr << "\n"; // Output: 42

    // Transfer ownership
```

(continues on next page)

(continued from previous page)

```

auto ptr2 = std::move(ptr);
// ptr is now nullptr

// Array support
auto arr = std::make_unique<int[]>(5);
arr[0] = 10;
}

```

std::shared_ptr: Shared Ownership

Source

src/smartptr/shared-ptr

std::shared_ptr allows multiple pointers to share ownership of an object through reference counting. Each shared_ptr maintains a pointer to a control block that tracks how many shared_ptr instances own the object. When a shared_ptr is copied, the reference count increments; when one is destroyed or reset, the count decrements. The managed object is deleted only when the last owning shared_ptr is destroyed (when the count reaches zero). This makes shared_ptr suitable for scenarios where multiple parts of your code need access to the same object and the lifetime cannot be determined statically. However, the reference counting adds overhead, so prefer unique_ptr when exclusive ownership is sufficient.

```

#include <memory>
#include <iostream>

int main() {
    auto ptr1 = std::make_shared<int>(42);
    std::cout << "Count: " << ptr1.use_count() << "\n"; // 1

    {
        auto ptr2 = ptr1; // Share ownership
        std::cout << "Count: " << ptr1.use_count() << "\n"; // 2
    } // ptr2 destroyed, count decremented

    std::cout << "Count: " << ptr1.use_count() << "\n"; // 1
}

```

std::weak_ptr: Non-Owning Observer

Source

src/smartptr/weak-ptr

std::weak_ptr holds a non-owning reference to an object managed by shared_ptr. Unlike shared_ptr, it doesn't contribute to the reference count, so it doesn't prevent the object from being deleted. This makes weak_ptr essential for breaking circular references that would otherwise cause memory leaks. It's also useful for implementing caches, observer patterns, and any situation where you need to check if an object still exists without extending its lifetime. To access the object, call lock() which returns a shared_ptr—if the object still exists, you get a valid shared_ptr that temporarily extends the object's lifetime; if it's been deleted, you get an empty shared_ptr. The expired() method provides a quick check without creating a shared_ptr.

```

#include <memory>
#include <iostream>

int main() {

```

(continues on next page)

(continued from previous page)

```

std::weak_ptr<int> weak;

{
    auto shared = std::make_shared<int>(42);
    weak = shared;

    if (auto locked = weak.lock()) {
        std::cout << *locked << "\n"; // 42
    }
} // shared destroyed

std::cout << "Expired: " << weak.expired() << "\n"; // 1 (true)
}

```

std::make_unique and std::make_shared

Source

src/smartptr/make-functions

Always prefer `std::make_unique` (C++14) and `std::make_shared` over direct `new` expressions. These factory functions provide several important benefits. First, they are exception-safe: if an exception is thrown during argument evaluation, no memory is leaked. Second, `make_shared` is more efficient because it performs a single memory allocation for both the object and the control block, improving cache locality and reducing allocation overhead. Third, they eliminate the redundancy of writing the type twice. Fourth, they prevent potential memory leaks in complex expressions where the evaluation order of function arguments is unspecified (before C++17).

```

#include <memory>

struct Widget { int x, y; };

int main() {
    // Preferred: exception-safe, efficient
    auto u = std::make_unique<Widget>();
    auto s = std::make_shared<Widget>();

    // Avoid: potential leak if exception thrown between allocations
    // std::unique_ptr<Widget> u2(new Widget());
}

```

Exception safety issue:

```

void process(std::unique_ptr<A> a, std::unique_ptr<B> b);

// DANGEROUS: evaluation order unspecified before C++17
process(std::unique_ptr<A>(new A()), std::unique_ptr<B>(new B()));

// SAFE: make_unique ensures no leak
process(std::make_unique<A>(), std::make_unique<B>());

```

Custom Deleters

Source

[src/smartptr/custom-deleter](#)

Smart pointers can use custom deleters for resources that require special cleanup beyond simple `delete`. This is essential when working with C library resources (file handles, sockets, database connections), memory from custom allocators, or any resource that needs specific cleanup logic. For `unique_ptr`, the deleter type is part of the pointer type, which means different deleters create different types. For `shared_ptr`, the deleter is type-erased and stored in the control block, so different deleters don't affect the pointer type—this provides more flexibility but adds slight overhead.

```
#include <cstdio>
#include <memory>

int main() {
    // unique_ptr with custom deleter (deleter is part of type)
    auto file = std::unique_ptr<FILE, int(*)(FILE*)>(
        fopen("test.txt", "w"), fclose);

    if (file) {
        fprintf(file.get(), "Hello\n");
    } // fclose called automatically

    // shared_ptr with custom deleter (type-erased)
    std::shared_ptr<FILE> file2(fopen("test2.txt", "w"), fclose);
}
```

Lambda as deleter:

```
auto deleter = [](int* p) {
    std::cout << "Deleting " << *p << "\n";
    delete p;
};

std::unique_ptr<int, decltype(deleter)> ptr(new int(42), deleter);
```

enable_shared_from_this

Source

[src/smartptr/enable-shared-from-this](#)

When an object managed by `shared_ptr` needs to obtain a `shared_ptr` to itself (for example, to register itself as a callback or pass itself to another function), inherit from `std::enable_shared_from_this<T>`. This base class provides the `shared_from_this()` method that returns a `shared_ptr` sharing ownership with existing owners. This is necessary because creating a new `shared_ptr` directly from `this` would create a separate ownership group with its own reference count, leading to double deletion when both groups reach zero. The `weak_from_this()` method (C++17) returns a `weak_ptr` for cases where you don't want to extend the lifetime.

```
#include <memory>
#include <iostream>

class Widget : public std::enable_shared_from_this<Widget> {
public:
    std::shared_ptr<Widget> get_shared() {
        return shared_from_this();
    }
}
```

(continues on next page)

(continued from previous page)

```

    }
};

int main() {
    auto w = std::make_shared<Widget>();
    auto w2 = w->get_shared();
    std::cout << "Count: " << w.use_count() << "\n"; // 2
}

```

Aliasing Constructor (shared_ptr)

Source

[src/smartptr/aliasing](#)

The aliasing constructor creates a `shared_ptr` that shares ownership with another `shared_ptr` but points to a different object (typically a member of the owned object). This is useful when you need to pass a pointer to a member while ensuring the parent object stays alive. The aliasing `shared_ptr` increments the reference count of the original owner, so the parent object won't be deleted while the aliasing pointer exists. This pattern is commonly used when interfacing with APIs that expect a `shared_ptr` to a specific type but you have a `shared_ptr` to a containing object.

```

#include <memory>
#include <iostream>

struct Inner { int value = 42; };
struct Outer { Inner inner; };

int main() {
    auto outer = std::make_shared<Outer>();

    // Aliasing: shares ownership with outer, points to inner
    std::shared_ptr<Inner> inner(outer, &outer->inner);

    std::cout << inner->value << "\n"; // 42
    std::cout << outer.use_count() << "\n"; // 2
}

```

std::make_unique_for_overwrite (C++20)

C++20 added `std::make_unique_for_overwrite` which creates a `unique_ptr` with default-initialized (not value-initialized) memory. For scalar types and arrays of scalars, this means the memory is left uninitialized rather than being zeroed. This optimization is useful when you plan to immediately overwrite all the memory anyway, such as when reading data from a file or network into a buffer. Avoiding unnecessary zero-initialization can provide measurable performance improvements for large allocations.

```

#include <memory>

int main() {
    // Value-initialized (zeroed): slower
    auto arr1 = std::make_unique<int[]>(1000);

    // Default-initialized (uninitialized): faster
    auto arr2 = std::make_unique_for_overwrite<int[]>(1000);
}

```

(continues on next page)

(continued from previous page)

```
// Must initialize before reading!
}
```

std::shared_ptr Array Support (C++17/C++20)

C++17 added `shared_ptr<T[]>` support with proper array semantics, including `operator[]` for element access and automatic use of `delete[]` for cleanup. C++20 completed the feature by adding `std::make_shared` overloads for arrays. Before C++17, using `shared_ptr` with arrays required a custom deleter to call `delete[]` instead of `delete`, which was error-prone and verbose.

```
#include <memory>

int main() {
    // C++20: make_shared for arrays
    auto arr = std::make_shared<int[]>(5);
    arr[0] = 10;

    // C++17: shared_ptr with array type
    std::shared_ptr<int[]> arr2(new int[5]);
}
```

Atomic Operations on shared_ptr (C++20)

C++20 introduced `std::atomic<std::shared_ptr<T>>` for thread-safe atomic operations on shared pointers. This provides a proper atomic type that supports all standard atomic operations (load, store, exchange, compare_exchange) with correct memory ordering semantics. Before C++20, thread-safe access to `shared_ptr` required using the free functions `std::atomic_load` and `std::atomic_store`, which were less convenient and had subtle correctness issues. The new atomic specialization is essential for lock-free data structures and concurrent algorithms that need to safely share ownership across threads.

```
#include <atomic>
#include <memory>

std::atomic<std::shared_ptr<int>> atomic_ptr;

void writer() {
    atomic_ptr.store(std::make_shared<int>(42));
}

void reader() {
    if (auto p = atomic_ptr.load()) {
        // Use *p safely
    }
}
```

out_ptr and inout_ptr (C++23)

C++23 added `std::out_ptr` and `std::inout_ptr` adaptors for interfacing smart pointers with C APIs that return pointers through output parameters. These adaptors eliminate the error-prone manual pattern of releasing the smart pointer, calling the C function, and then resetting the smart pointer with the result. `out_ptr` is for functions that only output a pointer (the smart pointer should be empty beforehand), while `inout_ptr` is for functions that may release an existing resource before outputting a new one.

```
#include <memory>

// C API that allocates and returns through output parameter
extern "C" int create_resource(void** out);
extern "C" void destroy_resource(void* p);

int main() {
    std::unique_ptr<void, decltype(&destroy_resource)> ptr(nullptr, destroy_resource);

    // C++23: out_ptr handles release and reset automatically
    create_resource(std::out_ptr(ptr));
}
```

Common Pitfalls

Smart pointers eliminate many memory management bugs, but they introduce their own set of pitfalls that can cause crashes, memory leaks, or undefined behavior. Understanding these common mistakes helps you use smart pointers correctly.

Creating `shared_ptr` from raw pointer multiple times:

This is one of the most dangerous mistakes. When you create a `shared_ptr` from a raw pointer, it creates a new control block with a reference count of 1. If you create another `shared_ptr` from the same raw pointer, it creates a completely separate control block, also with count 1. When both `shared_ptr` instances are destroyed, each thinks it's the last owner and calls `delete`, resulting in a double-free crash. Always create `shared_ptr` using `make_shared` or by copying/moving existing `shared_ptr` instances.

```
int* raw = new int(42);
std::shared_ptr<int> p1(raw);
std::shared_ptr<int> p2(raw); // BUG: double delete!

// Correct approach:
auto p1 = std::make_shared<int>(42);
auto p2 = p1; // Shares ownership correctly
```

Circular references with `shared_ptr`:

When two or more objects hold `shared_ptr` references to each other, they create a cycle that prevents either from being deleted. Each object's reference count never reaches zero because the other object holds a reference. This is a memory leak that persists until the program ends. The solution is to use `weak_ptr` for at least one direction of the relationship, breaking the cycle. Common scenarios include parent-child relationships (parent owns child with `shared_ptr`, child references parent with `weak_ptr`), doubly-linked lists, and graph structures.

```
struct Node {
    std::shared_ptr<Node> next; // Creates cycle, memory leak!
};

auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;
b->next = a; // Cycle: neither a nor b will ever be deleted

// Fix: use weak_ptr for one direction
struct FixedNode {
```

(continues on next page)

(continued from previous page)

```
std::shared_ptr<FixedNode> next;
std::weak_ptr<FixedNode> prev; // Breaks the cycle
};
```

Calling `shared_from_this` before `shared_ptr` exists:

The `shared_from_this()` method only works after at least one `shared_ptr` to the object has been created. Internally, `enable_shared_from_this` stores a `weak_ptr` that is initialized when the first `shared_ptr` is created. If you call `shared_from_this()` in the constructor (before any `shared_ptr` exists) or on a stack-allocated object (which should never be managed by `shared_ptr`), it throws `std::bad_weak_ptr`. Always ensure the object is managed by `shared_ptr` before calling `shared_from_this()`.

```
class Widget : public std::enable_shared_from_this<Widget> {
public:
    Widget() {
        // BUG: no shared_ptr owns this yet
        // auto self = shared_from_this(); // throws bad_weak_ptr
    }

    void init() {
        // OK: called after make_shared creates the shared_ptr
        auto self = shared_from_this();
    }
};

// Correct usage:
auto w = std::make_shared<Widget>();
w->init(); // Now shared_from_this() works
```

Storing `this` in a `shared_ptr`:

Never create a `shared_ptr` directly from `this`. This creates a new ownership group that doesn't know about any existing `shared_ptr` owners, leading to double deletion. Use `enable_shared_from_this` instead.

```
class Bad {
public:
    std::shared_ptr<Bad> get_ptr() {
        return std::shared_ptr<Bad>(this); // BUG: double delete!
    }
};
```

Smart Pointer Comparison

The following table summarizes when to use each smart pointer type. Choose the simplest pointer that meets your ownership requirements—prefer `unique_ptr` for its zero overhead and clear ownership semantics, use `shared_ptr` only when ownership truly needs to be shared, and use `weak_ptr` to observe without owning.

Type	Ownership	Use Case
<code>unique_ptr</code>	Exclusive	Single owner, factory functions, RAII wrappers
<code>shared_ptr</code>	Shared	Multiple owners, shared resources, caches
<code>weak_ptr</code>	None	Breaking cycles, observers, caches with expiration

1.3.16 Return Value Optimization

Table of Contents

- *Return Value Optimization*
 - *Instrumented Class for Testing*
 - *Return Value Optimization (RVO)*
 - *Named Return Value Optimization (NRVO)*
 - *Copy Elision in Function Arguments*
 - *Cases Where RVO Does NOT Apply*
 - *Don't Use std::move on Return*
 - *Runtime Conditional Returns*
 - *Guaranteed Copy Elision (C++17)*
 - *Summary Table*

Return Value Optimization (RVO) and Named Return Value Optimization (NRVO) are compiler optimizations that eliminate unnecessary copy or move operations when returning objects from functions. Instead of constructing an object locally and then copying it to the caller's storage, the compiler constructs the object directly in the caller's memory location. This optimization can significantly improve performance for types with expensive copy operations. C++17 made RVO mandatory (guaranteed copy elision) for certain cases, meaning the optimization is no longer optional but required by the standard. NRVO remains an optional optimization that most compilers implement but is not guaranteed.

Instrumented Class for Testing

Source

`src/rvo/instrumented`

To observe copy elision behavior, use a class that logs all special member function calls. This technique helps verify when copies and moves are elided by the compiler. By examining the output, you can determine whether RVO or NRVO was applied, or if a copy/move operation occurred instead.

```
#include <iostream>

struct Foo {
    Foo() { std::cout << "Constructor\n"; }
    ~Foo() { std::cout << "Destructor\n"; }
    Foo(const Foo&) { std::cout << "Copy Constructor\n"; }
    Foo(Foo&&) { std::cout << "Move Constructor\n"; }
    Foo& operator=(const Foo&) { std::cout << "Copy Assignment\n"; return *this; }
    Foo& operator=(Foo&&) { std::cout << "Move Assignment\n"; return *this; }
};
```

Return Value Optimization (RVO)

Source

`src/rvo/rvo`

RVO applies when returning a temporary (prvalue) directly from a function. The compiler constructs the object directly in the caller's storage location, completely eliminating any copy or move operation. Since C++17, this optimization is

guaranteed by the standard (mandatory copy elision), meaning the copy or move constructor doesn't even need to be accessible or defined. This is particularly useful for factory functions and builder patterns where objects are created and returned frequently.

```
Foo MakeRVO() {  
    return Foo(); // Guaranteed RVO since C++17  
}  
  
int main() {  
    Foo f = MakeRVO(); // Only one Constructor call, no copy/move  
}
```

Output:
Constructor
Destructor

Named Return Value Optimization (NRVO)

Source

[src/rvo/nrvo](#)

NRVO applies when returning a named local variable from a function. The compiler may construct the local variable directly in the caller's return slot, avoiding any copy or move when the function returns. Unlike RVO, NRVO is not guaranteed by the C++ standard and depends on compiler optimization settings. However, most modern compilers (GCC, Clang, MSVC) implement NRVO reliably when optimization is enabled. When NRVO cannot be applied, the compiler will use move semantics if available, falling back to copy only when necessary.

```
Foo MakeNRVO() {  
    Foo foo; // Named local variable  
    return foo; // NRVO may apply  
}  
  
int main() {  
    Foo f = MakeNRVO(); // Typically one Constructor, no copy/move  
}
```

Output (with NRVO):
Constructor
Destructor

Copy Elision in Function Arguments

Source

[src/rvo/copy-elision](#)

When passing a temporary to a function by value, the compiler can construct the argument directly in the parameter's storage location, eliding the copy entirely. This optimization applies when the argument is a prvalue (temporary) and the parameter is passed by value. Combined with move semantics, this makes passing objects by value efficient in many cases, especially for sink parameters that will be stored or consumed by the function.

```
void TakeByValue(Foo foo) {}  
  
int main() {
```

(continues on next page)

(continued from previous page)

```
TakeByValue(Foo()); // Copy elision: no copy/move
}
```

Output:
 Constructor
 Destructor

Cases Where RVO Does NOT Apply

Source

`src/rvo/no-rvo`

RVO and NRVO cannot apply in certain situations where the compiler cannot determine a single return location at compile time, or when the returned object already exists elsewhere. Understanding these cases helps you write more efficient code and avoid unexpected performance costs.

Returning a global or static variable:

Global and static variables have fixed storage locations that cannot be changed. When returning such a variable, the compiler must copy it to the caller's storage because the original must remain intact.

```
const Foo global_foo;

Foo ReturnGlobal() {
    return global_foo; // Must copy, cannot elide
}
```

Returning a function parameter:

Function parameters are allocated in the caller's stack frame before the function is called. The return value location is separate, so the parameter must be moved or copied to the return location.

```
Foo ReturnParam(Foo foo) {
    return foo; // Must move (or copy), cannot elide
}
```

Returning a member of another object:

When returning a member of an object, the member's storage is part of the containing object. The compiler cannot construct it directly in the return location, so a move or copy is required.

```
struct Bar { Foo foo; };

Foo ReturnMember() {
    return Bar().foo; // Must move, cannot elide
}
```

Multiple return paths with different named variables:

When a function has multiple return statements returning different named variables, the compiler cannot determine which variable's storage to use as the return location. NRVO requires a single, unambiguous return variable.

```
Foo NoNRVO(bool flag) {
    Foo x, y;
    return flag ? x : y; // NRVO cannot apply
}
```

Don't Use `std::move` on Return

Source

`src/rvo/no-move-return`

Using `std::move` on a return statement is a common mistake that actually pessimizes performance. It prevents RVO and NRVO because `std::move` converts the expression to an xvalue, which disables copy elision. The compiler already applies implicit move semantics for local variables in return statements when NRVO doesn't apply, so explicit `std::move` is never needed and always harmful for return values.

```
// BAD: Prevents RVO
Foo BadRVO() {
    return std::move(Foo()); // Forces move, prevents RVO
}

// BAD: Prevents NRVO
Foo BadNRVO() {
    Foo foo;
    return std::move(foo); // Forces move, prevents NRVO
}

// GOOD: Let the compiler optimize
Foo GoodRVO() {
    return Foo(); // RVO applies
}

Foo GoodNRVO() {
    Foo foo;
    return foo; // NRVO may apply, implicit move otherwise
}
```

Warning

Never use `std::move` on return statements for local variables. It pessimizes performance by preventing copy elision. The compiler will automatically apply move semantics when appropriate.

Runtime Conditional Returns

Source

`src/rvo/conditional`

When returning different temporaries based on a runtime condition, RVO can still apply because each branch returns a prvalue that can be constructed directly in the return location. The compiler generates code that constructs whichever temporary is selected directly in the caller's storage.

```
Foo ConditionalRVO(bool flag) {
    return flag ? Foo() : Foo(); // RVO applies to both branches
}
```

However, mixing named variables with temporaries or using different named variables in different branches prevents NRVO because the compiler cannot determine a single variable to use as the return location:

```
Foo MixedReturn(bool flag) {
    Foo x;
```

(continues on next page)

(continued from previous page)

```

    return flag ? x : Foo(); // NRVO cannot apply
}

```

Guaranteed Copy Elision (C++17)

C++17 mandates copy elision in specific cases, making RVO guaranteed rather than optional. This is a significant language change because it means the copy or move constructor doesn't even need to exist for these cases to work. This enables returning non-copyable, non-movable types from functions and makes factory functions work with any type.

The guaranteed cases are:

- Returning a prvalue of the same type as the function return type
- Initializing a variable from a prvalue of the same type

```

struct NonMovable {
    NonMovable() = default;
    NonMovable(const NonMovable&) = delete;
    NonMovable(NonMovable&&) = delete;
};

NonMovable Make() {
    return NonMovable(); // OK in C++17: guaranteed copy elision
}

int main() {
    NonMovable n = Make(); // OK: no copy/move needed
}

```

Summary Table

The following table provides a quick reference for when RVO and NRVO apply. Use this to identify patterns that enable copy elision and avoid patterns that prevent it. Remember that RVO (returning temporaries) is guaranteed in C++17, while NRVO (returning named variables) is a compiler optimization that typically applies but is not guaranteed by the standard.

Scenario	RVO/NRVO	Notes
return Foo();	✓ Guaranteed (C++17)	Prvalue return
return local_var;	✓ Likely (NRVO)	Compiler optimization
return std::move(x);	Prevented	Never do this
return global;	No	Must copy
return param;	No	Must move
return obj.member;	No	Must move
return flag ? x : y;	No NRVO	Multiple candidates

1.3.17 Algorithm

Table of Contents

- *Algorithm*

- *Introduction*
- *Complexity Summary*
- *Sorting*
 - * `std::sort`
 - * `std::stable_sort`
 - * `std::partial_sort`
 - * `std::nth_element`
- *Searching*
 - * `std::find`
 - * `std::find_if` and `std::find_if_not`
 - * `std::binary_search`
- *Counting*
 - * `std::count` and `std::count_if`
- *Transforming*
 - * `std::transform`
 - * `std::for_each`
- *Accumulating*
 - * `std::accumulate`
 - * `std::reduce` (C++17)
- *Removing Elements*
 - * *Erase-Remove Idiom*
 - * `std::erase` and `std::erase_if` (C++20)
 - * *Erasing from Associative Containers*
- *Generating*
 - * `std::generate`
 - * `std::iota`
- *Copying*
 - * `std::copy` and `std::copy_if`
- *Min/Max*
 - * `std::min_element` and `std::max_element`
- *Checking Conditions*
 - * `std::all_of`, `std::any_of`, `std::none_of`
- *Unique Elements*
 - * `std::unique`
- *Reversing and Rotating*

- * `std::reverse` and `std::rotate`
- Set Operations
- `std::map` with Custom Comparator
 - * `std::map` with Object as Key
- C++20 Ranges

Introduction

The `<algorithm>` header provides a collection of functions for searching, sorting, counting, and manipulating ranges of elements. These algorithms operate on iterator pairs and are designed to work with any container that provides compatible iterators. This separation of algorithms from containers is a fundamental design principle of the C++ Standard Template Library (STL), enabling code reuse and generic programming.

C++20 introduced the Ranges library which provides a more composable and safer alternative to iterator-based algorithms. For comprehensive coverage of C++20 ranges and views, see [Iterator](#).

Complexity Summary

The following table summarizes the time complexity and iterator requirements for common STL algorithms. Understanding these characteristics helps you choose the right algorithm for your use case and predict performance at scale.

The **Iterator Required** column indicates the minimum iterator category needed. Algorithms work with that category or higher (Random Access > Bidirectional > Forward > Input). See the [Iterator Categories](#) section for details on each category and their capabilities.

Algorithm	Time Complexity	Iterator Required	Notes
<code>sort</code>	$O(n \log n)$	Random Access	Not stable; use <code>stable_sort</code> if order matters
<code>stable_sort</code>	$O(n \log n)$	Random Access	$O(n \log^2 n)$ if no extra memory available
<code>partial_sort</code>	$O(n \log k)$	Random Access	k = number of elements to sort
<code>nth_element</code>	$O(n)$ average	Random Access	$O(n^2)$ worst case
<code>find</code>	$O(n)$	Input	Use <code>binary_search</code> for sorted ranges
<code>find_if</code>	$O(n)$	Input	Short-circuits on first match
<code>binary_search</code>	$O(\log n)$	Forward	Requires sorted range
<code>lower_bound</code>	$O(\log n)$	Forward	$O(n)$ for non-random access iterators
<code>upper_bound</code>	$O(\log n)$	Forward	$O(n)$ for non-random access iterators
<code>count</code>	$O(n)$	Input	Full range traversal
<code>count_if</code>	$O(n)$	Input	Full range traversal
<code>transform</code>	$O(n)$	Input/Output	One function call per element
<code>for_each</code>	$O(n)$	Input	One function call per element
<code>accumulate</code>	$O(n)$	Input	Sequential left-to-right
<code>reduce</code>	$O(n)$	Input	Parallelizable (C++17)
<code>remove</code>	$O(n)$	Forward	Does not resize container
<code>remove_if</code>	$O(n)$	Forward	Does not resize container
<code>unique</code>	$O(n)$	Forward	Removes consecutive duplicates only

continues on next page

Table 2 – continued from previous page

Algorithm	Time Complexity	Iterator Required	Notes
<code>copy</code>	$O(n)$	Input/Output	Destination must have space
<code>copy_if</code>	$O(n)$	Input/Output	One predicate call per element
<code>generate</code>	$O(n)$	Forward	One generator call per element
<code>iota</code>	$O(n)$	Forward	Sequential increment
<code>reverse</code>	$O(n)$	Bidirectional	In-place swap
<code>rotate</code>	$O(n)$	Forward	At most n swaps
<code>min_element</code>	$O(n)$	Forward	$n-1$ comparisons
<code>max_element</code>	$O(n)$	Forward	$n-1$ comparisons
<code>minmax_element</code>	$O(n)$	Forward	$\sim 1.5n$ comparisons (more efficient)
<code>all_of</code>	$O(n)$	Input	Short-circuits on false
<code>any_of</code>	$O(n)$	Input	Short-circuits on true
<code>none_of</code>	$O(n)$	Input	Short-circuits on true
<code>set_union</code>	$O(n + m)$	Input	Requires sorted ranges
<code>set_intersection</code>	$O(n + m)$	Input	Requires sorted ranges
<code>set_difference</code>	$O(n + m)$	Input	Requires sorted ranges

Sorting

`std::sort`

Source

`src/algorithm/sort`

`std::sort` rearranges elements in ascending order by default using an introsort algorithm (a hybrid of quicksort, heapsort, and insertion sort) that guarantees $O(n \log n)$ worst-case complexity. The algorithm requires random access iterators, so it works with `std::vector`, `std::array`, and `std::deque`, but not with `std::list` (use `list.sort()` instead). For custom ordering, provide a comparison function or use `std::greater<>` for descending order. Note that `std::sort` is **not stable**—equal elements may be reordered relative to their original positions.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{3, 1, 4, 1, 5, 9, 2, 6};

    std::sort(v.begin(), v.end()); // ascending
    std::sort(v.begin(), v.end(), std::greater<>{}); // descending

    // custom comparator
    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
}
```

`std::stable_sort`

Source

`src/algorithm/stable-sort`

`std::stable_sort` preserves the relative order of elements that compare equal, which is essential when sorting by multiple criteria sequentially. For example, if you first sort employees by name and then by department, employees within the same department will remain sorted by name. The algorithm uses merge sort and guarantees $O(n \log n)$ complexity when sufficient memory is available, or $O(n \log^2 n)$ otherwise. Use `std::stable_sort` when the original

ordering of equivalent elements carries semantic meaning.

```
#include <algorithm>
#include <vector>

struct Item {
    int priority;
    int order;
};

int main() {
    std::vector<Item> items{{1, 1}, {2, 2}, {1, 3}, {2, 4}};

    // stable_sort preserves original order for equal elements
    std::stable_sort(items.begin(), items.end(),
        [](auto &a, auto &b) { return a.priority < b.priority; });
    // items with same priority keep their original relative order
}
```

std::partial_sort

Source

src/algorithm/partial-sort

`std::partial_sort` sorts only the first *N* elements of a range, leaving the remaining elements in an unspecified order. This is significantly more efficient than a full sort when you only need the top *N* results—it runs in $O(n \log k)$ time where *k* is the number of elements to sort, compared to $O(n \log n)$ for a full sort. Common use cases include finding the top 10 scores, the 5 smallest values, or implementing pagination where only the first page needs to be sorted.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{5, 7, 4, 2, 8, 6, 1, 9, 0, 3};

    // sort only first 3 elements
    std::partial_sort(v.begin(), v.begin() + 3, v.end());
    // v: {0, 1, 2, ...} first 3 are sorted, rest unspecified
}
```

std::nth_element

Source

src/algorithm/nth-element

`std::nth_element` is a selection algorithm that partially sorts a range so that the element at the *n*th position is exactly the element that would be there if the range were fully sorted. All elements before the *n*th are less than or equal to it, and all elements after are greater than or equal. This runs in $O(n)$ average time, making it ideal for finding medians, percentiles, or partitioning data around a pivot value without the overhead of a full sort.

```
#include <algorithm>
#include <vector>

int main() {
```

(continues on next page)

(continued from previous page)

```
std::vector v{5, 6, 4, 3, 2, 6, 7, 9, 3};

// find median (middle element)
auto mid = v.begin() + v.size() / 2;
std::nth_element(v.begin(), mid, v.end());
// *mid is now the median value
}
```

Searching

std::find

Source

src/algorithm/find

`std::find` performs a linear search through a range, returning an iterator to the first element that equals the specified value, or `end()` if not found. The algorithm has $O(n)$ complexity and works with any input iterator, making it universally applicable but potentially slow for large unsorted containers. For sorted containers, prefer `std::binary_search`, `std::lower_bound`, or `std::upper_bound` which offer $O(\log n)$ complexity. For associative containers like `std::set` or `std::map`, use their member `find()` functions which are optimized for their internal structure.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    auto it = std::find(v.begin(), v.end(), 3);
    if (it != v.end()) {
        std::cout << "Found: " << *it << "\n";
    }
}
```

std::find_if and std::find_if_not

Source

src/algorithm/find-if

`std::find_if` searches for the first element satisfying a predicate function, while `std::find_if_not` finds the first element that does **not** satisfy the predicate. These are more flexible than `std::find` because they allow arbitrary search conditions rather than simple equality. Common use cases include finding the first negative number, the first element exceeding a threshold, or the first invalid entry in a dataset. Both algorithms have $O(n)$ complexity.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    auto even = std::find_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
    auto odd = std::find_if_not(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
}
```

(continues on next page)

(continued from previous page)

```
std::cout << "First even: " << *even << "\n"; // 2
std::cout << "First odd: " << *odd << "\n"; // 1
}
```

std::binary_search**Source**[src/algorithm/binary-search](#)

`std::binary_search` checks whether a value exists in a **sorted** range using binary search with $O(\log n)$ complexity. It returns a boolean indicating presence but does not provide the element's position. To get an iterator to the element, use `std::lower_bound` (first element $\geq$ value) or `std::upper_bound` (first element $>$ value). The `std::equal_range` function returns both bounds as a pair, defining the range of elements equal to the search value. These functions are essential for efficient searching and insertion in sorted containers.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    bool found = std::binary_search(v.begin(), v.end(), 3);

    // lower_bound: first element >= value
    auto lb = std::lower_bound(v.begin(), v.end(), 3);

    // upper_bound: first element > value
    auto ub = std::upper_bound(v.begin(), v.end(), 3);

    // equal_range: pair of lower_bound and upper_bound
    auto [lo, hi] = std::equal_range(v.begin(), v.end(), 3);
}
```

Counting**std::count and std::count_if****Source**[src/algorithm/count](#)

`std::count` returns the number of elements in a range that equal a specified value, while `std::count_if` counts elements satisfying a predicate. Both algorithms traverse the entire range with $O(n)$ complexity. These are useful for frequency analysis, validation (checking how many elements meet criteria), and statistical computations. For associative containers, the member `count()` function is more efficient as it leverages the container's internal structure.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{1, 2, 2, 3, 2, 4, 2};
}
```

(continues on next page)

(continued from previous page)

```

auto n = std::count(v.begin(), v.end(), 2);
std::cout << "Count of 2: " << n << "\n"; // 4

auto evens = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
std::cout << "Even count: " << evens << "\n"; // 5
}

```

Transforming

std::transform

Source

src/algorithm/transform

`std::transform` applies a unary or binary function to elements and stores the results in a destination range. It can transform elements in place (when source and destination are the same) or into a separate container. This is the C++ equivalent of functional programming's map operation. Common uses include converting strings to uppercase, scaling numeric values, extracting fields from structures, or combining two ranges element-wise. The algorithm does not resize the destination container, so ensure it has sufficient capacity.

```

#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::string s = "Hello World";

    // transform in place
    std::transform(s.begin(), s.end(), s.begin(), ::toupper);
    std::cout << s << "\n"; // HELLO WORLD

    // transform to another container
    std::vector<int> v{1, 2, 3};
    std::vector<int> squared(v.size());
    std::transform(v.begin(), v.end(), squared.begin(), [](int x) { return x * x; });
}

```

std::for_each

Source

src/algorithm/for-each

`std::for_each` applies a function to each element in a range, primarily for side effects like printing, accumulating values, or modifying elements in place. Unlike `std::transform`, it doesn't write to a destination range. The function object is passed by value and returned after iteration, which can be useful for stateful functors. In modern C++, range-based for loops often replace `std::for_each` for simple iteration, but the algorithm remains useful when you need to pass a callable to another function or work with iterator pairs.

```

#include <algorithm>
#include <iostream>
#include <vector>

```

(continues on next page)

(continued from previous page)

```
int main() {
    std::vector v{1, 2, 3};

    std::for_each(v.begin(), v.end(), [](int &x) { x *= 2; });
    std::for_each(v.begin(), v.end(), [](int x) { std::cout << x << " "; });
    // Output: 2 4 6
}
```

Accumulating

std::accumulate

Source

src/algorithm/accumulate

`std::accumulate` (from `<numeric>`) computes a single value from a range by repeatedly applying a binary operation. By default, it sums elements starting from an initial value, but custom operations enable products, string concatenation, or any associative reduction. The initial value's type determines the result type, so use `0.0` instead of `0` when accumulating floating-point values to avoid truncation. Note that `std::accumulate` processes elements strictly left-to-right, which matters for non-commutative operations.

```
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    int sum = std::accumulate(v.begin(), v.end(), 0);
    std::cout << "Sum: " << sum << "\n"; // 15

    int product = std::accumulate(v.begin(), v.end(), 1, std::multiplies<>{});
    std::cout << "Product: " << product << "\n"; // 120
}
```

std::reduce (C++17)

Source

src/algorithm/reduce

`std::reduce` (from `<numeric>`) is similar to `std::accumulate` but permits out-of-order execution, enabling parallel implementations. The operation must be both associative and commutative for correct results with parallel execution policies. When used with `std::execution::par`, it can significantly speed up reductions on large datasets by utilizing multiple CPU cores. For sequential execution, it behaves identically to `std::accumulate`.

```
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    // reduce allows out-of-order execution (parallelizable)
```

(continues on next page)

(continued from previous page)

```
int sum = std::reduce(v.begin(), v.end(), 0);
std::cout << "Sum: " << sum << "\n";
}
```

Removing Elements

Erase-Remove Idiom

Source

[src/algorithm/erase-remove](#)

The erase-remove idiom is the classic way to remove elements from sequence containers. `std::remove` and `std::remove_if` don't actually erase elements—they move unwanted elements to the end of the range and return an iterator to the new logical end. You must then call the container's `erase()` method to actually remove them. This two-step process exists because algorithms work with iterators and don't know about container memory management. The idiom is efficient, performing the removal in $O(n)$ time with a single pass through the data.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 2, 4, 2, 5};

    // erase-remove idiom
    v.erase(std::remove(v.begin(), v.end(), 2), v.end());
    // v: {1, 3, 4, 5}

    // with predicate
    std::vector v2{1, 2, 3, 4, 5, 6};
    v2.erase(std::remove_if(v2.begin(), v2.end(), [](int x) { return x % 2 == 0; }),
            v2.end());
    // v2: {1, 3, 5}
}
```

`std::erase` and `std::erase_if` (C++20)

Source

[src/algorithm/erase-cpp20](#)

C++20 introduced `std::erase` and `std::erase_if` as free functions that combine the erase-remove idiom into a single call. These functions are overloaded for each standard container type and handle the container-specific removal logic automatically. They return the number of elements removed, which is useful for logging or validation. This is now the preferred way to remove elements in modern C++ code due to its clarity and reduced chance of errors.

```
#include <vector>

int main() {
    std::vector v{1, 2, 3, 2, 4, 2, 5};

    // C++20: direct erase
    std::erase(v, 2);
    // v: {1, 3, 4, 5}
}
```

(continues on next page)

(continued from previous page)

```
std::vector v2{1, 2, 3, 4, 5, 6};
std::erase_if(v2, [](int x) { return x % 2 == 0; });
// v2: {1, 3, 5}
}
```

Erasing from Associative Containers

Source

src/algorithm/erase-map

Associative containers (`std::map`, `std::set`, etc.) require careful iteration when erasing elements because erasing invalidates the iterator to the erased element. The safe pattern is to use the iterator returned by `erase()`, which points to the next element. In C++20, `std::erase_if` works with associative containers too, providing a cleaner solution. For `std::map`, the predicate receives a `std::pair<const Key, Value>&`.

```
#include <map>
#include <string>

int main() {
    std::map<int, std::string> m{{1, "a"}, {2, "b"}, {3, "c"}};

    // pre-C++20: iterate and erase
    for (auto it = m.begin(); it != m.end(); ) {
        if (it->first > 1) {
            it = m.erase(it);
        } else {
            ++it;
        }
    }

    // C++20: std::erase_if works on maps too
    // std::erase_if(m, [](auto& p) { return p.first > 1; });
}
```

Generating

`std::generate`

Source

src/algorithm/generate

`std::generate` fills a range by repeatedly calling a generator function and assigning the returned values to successive elements. The generator can be stateful (using captured variables or member state) to produce sequences like incrementing numbers, random values, or computed series. Unlike `std::fill` which assigns the same value to all elements, `std::generate` can produce different values for each position. This is useful for initializing containers with computed values, generating test data, or creating sequences that depend on external state.

```
#include <algorithm>
#include <random>
#include <vector>

int main() {
    std::random_device rd;
```

(continues on next page)

(continued from previous page)

```

std::mt19937 gen(rd());
std::uniform_int_distribution<> dist(1, 100);

std::vector<int> v(5);
std::generate(v.begin(), v.end(), [&] { return dist(gen); });

// generate sequence: 0, 1, 2, 3, ...
std::vector<int> seq(5);
int n = 0;
std::generate(seq.begin(), seq.end(), [&n] { return n++; });
}

```

std::iota

Source

[src/algorithm/iota](#)

`std::iota` (from `<numeric>`) fills a range with sequentially increasing values starting from a specified initial value. Each element is assigned the previous value plus one (using `operator++`). This is more concise than `std::generate` with a counter for simple incrementing sequences. The name comes from the APL programming language where `iota()` represents an index generator. Common uses include creating index arrays, initializing test data, and generating sequences for algorithms that need position information.

```

#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v(5);
    std::iota(v.begin(), v.end(), 1); // v: {1, 2, 3, 4, 5}

    for (int x : v) std::cout << x << " ";
}

```

Copying

std::copy and std::copy_if

Source

[src/algorithm/copy](#)

`std::copy` copies elements from a source range to a destination, while `std::copy_if` copies only elements satisfying a predicate. The destination must have sufficient space—use `std::back_inserter` to automatically grow the destination container. These algorithms return an iterator past the last copied element, useful for chaining operations or determining how many elements were copied. For overlapping ranges where the destination begins within the source, use `std::copy_backward` instead to avoid overwriting source elements before they're copied.

```

#include <algorithm>
#include <iostream>
#include <iterator>
#include <vector>

int main() {

```

(continues on next page)

(continued from previous page)

```

std::vector v{1, 2, 3, 4, 5};
std::vector<int> dest(5);

std::copy(v.begin(), v.end(), dest.begin());

// copy to output stream
std::copy(v.begin(), v.end(), std::ostream_iterator<int>(std::cout, " "));

// copy_if with predicate
std::vector<int> evens;
std::copy_if(v.begin(), v.end(), std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
}

```

Min/Max

std::min_element and std::max_element

Source

src/algorithm/minmax

std::min_element and std::max_element return iterators to the smallest and largest elements in a range, respectively. std::minmax_element finds both in a single pass, which is more efficient than calling both separately (approximately 1.5n comparisons vs 2n). These algorithms use operator< by default but accept custom comparators. std::clamp (C++17) constrains a value to a range, returning the value if within bounds or the nearest bound otherwise. These functions are essential for range validation, normalization, and finding extrema in datasets.

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{3, 1, 4, 1, 5, 9, 2, 6};

    auto min_it = std::min_element(v.begin(), v.end());
    auto max_it = std::max_element(v.begin(), v.end());
    auto [lo, hi] = std::minmax_element(v.begin(), v.end());

    std::cout << "Min: " << *min_it << ", Max: " << *max_it << "\n";

    // clamp value to range [lo, hi]
    int val = std::clamp(10, 0, 5); // returns 5
}

```

Checking Conditions

std::all_of, std::any_of, std::none_of

Source

src/algorithm/predicates

These algorithms test whether elements in a range satisfy a predicate: std::all_of returns true if **every** element satisfies the predicate, std::any_of returns true if **at least one** element satisfies it, and std::none_of returns true if **no** elements satisfy it. They short-circuit evaluation, stopping as soon as the result is determined. For empty ranges,

`std::all_of` and `std::none_of` return true (vacuous truth), while `std::any_of` returns false. These are useful for validation, precondition checking, and expressing intent more clearly than manual loops.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{2, 4, 6, 8};

    bool all_even = std::all_of(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
    bool any_odd = std::any_of(v.begin(), v.end(), [](int x) { return x % 2 != 0; });
    bool none_neg = std::none_of(v.begin(), v.end(), [](int x) { return x < 0; });

    std::cout << std::boolalpha;
    std::cout << "All even: " << all_even << "\n";    // true
    std::cout << "Any odd: " << any_odd << "\n";      // false
    std::cout << "None negative: " << none_neg << "\n"; // true
}
```

Unique Elements

`std::unique`

Source

`src/algorithm/unique`

`std::unique` removes **consecutive** duplicate elements by moving unique elements to the front and returning an iterator to the new logical end. Like `std::remove`, it doesn't actually erase elements—you must call `erase()` afterward. To remove **all** duplicates (not just consecutive ones), sort the range first. The algorithm compares adjacent elements using `operator==` by default, or a custom predicate for more complex equality definitions. This is commonly used for deduplication after sorting or for collapsing runs of identical values.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{1, 1, 2, 2, 2, 3, 1, 1};

    // remove consecutive duplicates only
    auto last = std::unique(v.begin(), v.end());
    v.erase(last, v.end());
    // v: {1, 2, 3, 1}

    // to remove all duplicates, sort first
    std::vector v2{3, 1, 2, 1, 3, 2};
    std::sort(v2.begin(), v2.end());
    v2.erase(std::unique(v2.begin(), v2.end()), v2.end());
    // v2: {1, 2, 3}
}
```


Reversing and Rotating

`std::reverse` and `std::rotate`

Source

`src/algorithm/reverse-rotate`

`std::reverse` reverses the order of elements in place using bidirectional iterators. `std::rotate` performs a left rotation, moving the element at the specified middle position to the front while maintaining the relative order of other elements. Rotation is useful for implementing circular buffers, shifting elements, or reordering sequences. Both algorithms operate in $O(n)$ time. `std::rotate` returns an iterator to the element that was originally at the beginning, now at its new position after rotation.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector v{1, 2, 3, 4, 5};

    std::reverse(v.begin(), v.end());
    // v: {5, 4, 3, 2, 1}

    std::vector v2{1, 2, 3, 4, 5};
    std::rotate(v2.begin(), v2.begin() + 2, v2.end());
    // v2: {3, 4, 5, 1, 2} - element at position 2 becomes first
}
```

Set Operations

Source

`src/algorithm/set-operations`

Set operations work on **sorted** ranges and compute mathematical set operations: `std::set_union` produces elements in either range, `std::set_intersection` produces elements in both ranges, `std::set_difference` produces elements in the first range but not the second, and `std::set_symmetric_difference` produces elements in either range but not both. All operations preserve sorted order in the output and handle duplicates according to their multiplicity. These algorithms require output iterators and don't modify the input ranges.

```
#include <algorithm>
#include <iterator>
#include <vector>

int main() {
    std::vector a{1, 2, 3, 4, 5};
    std::vector b{3, 4, 5, 6, 7};

    std::vector<int> result;

    // union: {1, 2, 3, 4, 5, 6, 7}
    std::set_union(a.begin(), a.end(), b.begin(), b.end(), std::back_inserter(result));

    result.clear();
    // intersection: {3, 4, 5}
    std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
                          std::back_inserter(result));
}
```

(continues on next page)

(continued from previous page)

```

result.clear();
// difference (in a but not in b): {1, 2}
std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
                   std::back_inserter(result));
}

```

std::map with Custom Comparator

Source

[src/algorithm/map-comparator](#)

std::map maintains keys in sorted order using std::less<Key> by default, which sorts in ascending order. To change the ordering, provide a different comparator as the third template parameter. std::greater<> sorts in descending order. The comparator must define a strict weak ordering (irreflexive, asymmetric, transitive). Using transparent comparators (like std::less<>) enables heterogeneous lookup, allowing searches with types convertible to the key type without creating temporary key objects.

```

#include <iostream>
#include <map>

int main() {
    // ascending (default)
    std::map<int, int, std::less<>> asc{{3, 3}, {1, 1}, {2, 2}};

    // descending
    std::map<int, int, std::greater<>> desc{{3, 3}, {1, 1}, {2, 2}};

    for (auto &[k, v] : asc) std::cout << k << " "; // 1 2 3
    std::cout << "\n";
    for (auto &[k, v] : desc) std::cout << k << " "; // 3 2 1
}

```

std::map with Object as Key

Source

[src/algorithm/map-object-key](#)

To use custom objects as map keys, you must provide a comparison function that defines a strict weak ordering. This can be a lambda, function object, or operator< member/free function. The comparator determines both the ordering and equality (two keys are equal if neither is less than the other). For complex keys, consider implementing operator<=> (C++20) which automatically provides all comparison operators. Alternatively, for unordered containers, implement std::hash and operator== instead.

```

#include <iostream>
#include <map>

struct Point {
    int x, y;
};

int main() {
    // lambda comparator

```

(continues on next page)

(continued from previous page)

```

auto cmp = [](const Point &a, const Point &b) {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
};

std::map<Point, int, decltype(cmp)> m(cmp);
m[{1, 2}] = 1;
m[{3, 4}] = 2;

for (auto &[k, v] : m) {
    std::cout << "(" << k.x << "," << k.y << "): " << v << "\n";
}
}

```

C++20 Ranges

C++20 introduced the Ranges library which provides range-based versions of standard algorithms in the `std::ranges` namespace. These accept ranges directly instead of iterator pairs, reducing boilerplate and potential for errors. For comprehensive coverage of C++20 ranges, views, and range adaptors, see [Iterator](#).

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector v{5, 3, 1, 4, 2};

    // Range-based sort - no need for begin/end
    std::ranges::sort(v);

    // Range-based find
    auto it = std::ranges::find(v, 3);

    // Range-based predicates
    bool all_pos = std::ranges::all_of(v, [](int x) { return x > 0; });
}

```

1.3.18 Coroutine

Table of Contents

- *Coroutine*
 - *Introduction*
 - *Coroutine Basics*
 - *Promise Type*
 - *Awaiter*
 - *Generator*
 - *Asynchronous I/O*

- * *Coroutine*
- * *Future*
- * *IO (Scheduler)*
- * *Stream*

Introduction

C++20 coroutines provide a powerful mechanism for writing asynchronous code that maintains the readability of synchronous code. Unlike traditional functions that execute from start to finish in a single invocation, coroutines can **suspend** their execution at specific points and **resume** later, potentially after some external event occurs (like I/O completion or a timer expiring). This capability enables efficient cooperative multitasking without the complexity of explicit callbacks, state machines, or thread synchronization.

When the compiler encounters a function containing `co_await`, `co_yield`, or `co_return`, it transforms that function into a coroutine. The transformation creates a state machine that tracks the coroutine's progress through its execution points. The compiler also generates a **coroutine frame** (typically heap-allocated) that stores local variables across suspension points, and a **promise object** that controls the coroutine's behavior at each suspension and resumption.

Table 3: Coroutine Keywords

Keyword	Description
<code>co_await</code>	Suspend execution until the awaited operation completes; the awaited expression determines whether suspension actually occurs and what value is produced when resuming
<code>co_yield</code>	Suspend and produce a value to the caller; syntactic sugar for <code>co_await promise.yield_value(expr)</code> ; commonly used in generators
<code>co_return</code>	Complete the coroutine and optionally return a final value; after this, the coroutine cannot be resumed

Coroutine Basics

Source

[src/coroutine/basics](#)

A minimal coroutine requires a return type with a nested `promise_type` that satisfies the coroutine promise requirements. The compiler uses this promise type to manage the coroutine's lifecycle, including how it starts, how it handles return values and exceptions, and what happens when it completes. The examples use the `io` library's `Coro<T>` type which provides a complete promise implementation with scheduling support.

```
#include <io/coro.h>
#include <io/future.h>
#include <io/io.h>

Coro<> hello() {
    // do work
    co_return;
}

int main() {
    auto fut = Future(hello()); // Schedule coroutine
    IO::Get().Run();           // Run event loop
}
```

Promise Type

Source

[src/coroutine/promise-type](#)

The promise type is the central control mechanism for a coroutine. It defines the coroutine's behavior at every significant point in its lifecycle: startup, suspension, resumption, completion, and error handling. The compiler generates code that calls these methods at the appropriate times, allowing you to customize how your coroutine type behaves. Understanding the promise type is essential for building custom coroutine types.

Table 4: Promise Type Methods

Method	Description
<code>get_return_object()</code>	Called immediately after promise construction to create the object returned to the caller; happens before the coroutine body executes
<code>initial_suspend()</code>	Return <code>suspend_always</code> for lazy coroutines that wait for explicit resumption, or <code>suspend_never</code> for eager coroutines that start immediately
<code>final_suspend()</code>	Called after coroutine completes; must be <code>noexcept</code> ; returning <code>suspend_always</code> keeps the frame alive for result retrieval
<code>return_void()</code> <code>return_value(v)</code>	/ Called on <code>co_return</code> or <code>co_return expr</code> ; use <code>return_void()</code> for coroutines that don't produce a value
<code>unhandled_exception()</code>	Called when an exception escapes the coroutine body; typically stores the exception via <code>std::current_exception()</code> for later rethrowing
<code>yield_value(v)</code>	Called on <code>co_yield expr</code> ; returns an awaiter that controls suspension; used primarily in generator patterns

```
#include <io/coro.h>
#include <io/future.h>
#include <io/io.h>

Coro<int> compute() {
    co_return 42;
}

int main() {
    auto fut = Future(compute());
    IO::Get().Run();
    assert(fut.result() == 42);
}
```

Awaiter

Source

[src/coroutine/awaiter](#)

An awaiter is an object that controls what happens at a `co_await` expression. When you write `co_await expr`, the compiler obtains an awaiter from the expression (either directly if it has the right interface, or via operator `co_await`). The awaiter then determines whether the coroutine actually suspends, what happens during suspension, and what value the `co_await` expression produces. The three awaiter methods form a protocol:

- `await_ready()`: Called first; if it returns `true`, the coroutine skips suspension entirely (the result is already available)
- `await_suspend(handle)`: Called only if `await_ready()` returned `false`; receives the coroutine handle and can schedule resumption; can return `void` (always suspend), `bool` (suspend if true), or another handle (symmetric transfer)

- `await_resume()`: Called when the coroutine resumes; its return value becomes the result of the `co_await` expression

```
struct ReturnValue {
    int value;
    bool await_ready() const noexcept { return true; }
    void await_suspend(std::coroutine_handle<>) const noexcept {}
    int await_resume() const noexcept { return value; }
};

Coro<> example() {
    int x = co_await ReturnValue{42}; // x = 42, no suspension
}
```

Generator

Source

`src/coroutine/generator`

A generator is a coroutine that produces a sequence of values on demand using `co_yield`. Each `co_yield` expression suspends the coroutine and makes a value available to the caller. When the caller requests the next value, the coroutine resumes from where it left off. This pattern is ideal for lazy sequences where values are computed only when needed, infinite streams, and memory-efficient iteration over large datasets. Generators are a separate pattern from async coroutines—they don't use the io library's event loop.

```
template <typename T>
class Generator {
public:
    struct promise_type {
        T current_value;
        Generator get_return_object() {
            return Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(T value) {
            current_value = std::move(value);
            return {};
        }
        void return_void() {}
        void unhandled_exception() {}
    };

    struct iterator {
        std::coroutine_handle<promise_type> handle;
        iterator& operator++() { handle.resume(); return *this; }
        T& operator*() { return handle.promise().current_value; }
        bool operator==(std::default_sentinel_t) const { return handle.done(); }
    };

    iterator begin() { handle_.resume(); return {handle_}; }
    std::default_sentinel_t end() { return {}; }

    explicit Generator(std::coroutine_handle<promise_type> h) : handle_(h) {}
```

(continues on next page)

(continued from previous page)

```

~Generator() { if (handle_) handle_.destroy(); }

private:
    std::coroutine_handle<promise_type> handle_;
};

Generator<int> range(int start, int end) {
    for (int i = start; i < end; ++i) co_yield i;
}

int main() {
    for (int x : range(1, 5)) std::cout << x << " "; // 1 2 3 4
}

```

Asynchronous I/O

Source

[src/coroutine/io](#)

Traditional blocking I/O wastes CPU cycles waiting for operations to complete. Thread-per-connection models add overhead from context switching and memory usage. Asynchronous I/O solves this by allowing a single thread to handle thousands of concurrent connections through event-driven programming.

The io library combines C++20 coroutines with OS-level I/O multiplexing (epoll on Linux, kqueue on macOS/BSD) to provide an async framework where coroutines suspend on I/O operations and resume when data is ready. This enables writing sequential-looking code that executes asynchronously without callback hell or explicit state machines.

Core components:

- **Coro<T>**: Coroutine type for async operations
- **Future<C>**: Wrapper that schedules a coroutine and provides `result()` access
- **IO**: Event loop with ready queue, delayed task queue, and I/O selector integration
- **Stream**: Non-blocking socket I/O with async Read/Write operations

Coroutine

Source

[src/coroutine/coro](#)

Coroutines can be chained using `co_await`—when you await another `Coro<T>`, the current coroutine suspends and automatically resumes when the awaited coroutine completes. The inner coroutine stores a pointer to the outer coroutine's handle, and when it finishes, it schedules the outer coroutine to resume. This enables composing complex async operations from simpler building blocks while maintaining readable, sequential-looking code.

```

#include <io/coro.h>
#include <io/future.h>
#include <io/io.h>

Coro<int> square(int x) { co_return x * x; }

Coro<int> sum_of_squares(int a, int b) {
    auto a2 = co_await square(a);
    auto b2 = co_await square(b);
}

```

(continues on next page)

(continued from previous page)

```

    co_return a2 + b2;
}

template <typename C>
decltype(auto) Run(C&& coro) {
    auto fut = Future(std::move(coro));
    IO::Get().Run();
    return std::move(fut).result();
}

int main() {
    assert(Run(sum_of_squares(3, 4)) == 25);
}

```

Future

Source

src/coroutine/future

The Future wrapper automatically schedules a coroutine upon construction and provides access to its result. When you create a Future, it immediately adds the coroutine to the scheduler's ready queue. After calling `IO::Get().Run()`, you can retrieve the return value or any exception via `result()`.

```

#include <io/coro.h>
#include <io/future.h>
#include <io/io.h>

Coro<int> compute() { co_return 42; }

int main() {
    auto fut = Future(compute()); // Schedules coroutine
    IO::Get().Run();              // Run until complete
    assert(fut.result() == 42);   // Get result
}

```

IO (Scheduler)

Source

src/coroutine/scheduler

The IO class provides an event loop that orchestrates coroutine execution. It maintains a ready queue for tasks that can run immediately, a priority queue for delayed tasks (sorted by wake time), and integrates with the platform's I/O selector (epoll/kqueue) for socket readiness events. The event loop processes ready tasks, checks for I/O events, and advances delayed tasks whose timers have expired. The Sleep awaiter uses the delayed task queue to suspend coroutines for a specified duration.

```

#include <io/handle.h>
#include <io/io.h>
#include <io/sleep.h>

struct MyTask : Handle {
    void run() override { /* execute */ }
}

```

(continues on next page)

(continued from previous page)

```

    void stop() override { /* cleanup */ }
};

MyTask task;
IO::Get().Call(task);                // Schedule immediately
IO::Get().Call(std::chrono::milliseconds(100), task); // Delayed
IO::Get().Cancel(task);               // Cancel
IO::Get().Run();                      // Run event loop

// Sleep example
Coro<> delayed_task() {
    co_await Sleep(std::chrono::milliseconds(100));
    // Continues after 100ms
}

```

Stream

Source

[src/coroutine/stream](#)

The Stream class provides non-blocking socket I/O with coroutine-based async operations. When you call `co_await stream.Read()`, the coroutine suspends and registers with the I/O selector; when data arrives, the selector notifies the scheduler which resumes the coroutine. This allows a single thread to handle many concurrent connections efficiently.

```

#include <io/stream.h>
#include <io/client.h>
#include <io/server.h>

Coro<> handle_client(Stream stream) {
    char buf[1024];
    size_t n = co_await stream.Read(buf, sizeof(buf));
    if (n > 0) {
        co_await stream.Write(buf, n); // Echo back
    }
}

Coro<> run_server(int port) {
    auto server = Server("127.0.0.1", port, handle_client);
    server.Start();
    co_await server.Wait();
}

Coro<> run_client(int port) {
    Client client("127.0.0.1", port);
    auto stream = co_await client.Connect();
    co_await stream.Write("hello", 5);
    char buf[256];
    size_t n = co_await stream.Read(buf, sizeof(buf));
    // buf contains "hello"
}

```

The echo-server example demonstrates more advanced patterns including large data transfer (10MB) and concurrent multi-client scenarios. See [src/coroutine/echo-server](#).

1.3.19 Time

Table of Contents

- *Time*
 - *Getting Current Timestamp*
 - *Duration and Time Arithmetic*
 - *Measuring Elapsed Time (Profiling)*
 - *Converting to time_t*
 - *Converting from Timestamp to time_point*
 - *ISO 8601 Date Format*
 - *Formatting with Time Zones*
 - *C++20 Calendar and Time Zones*
 - *Clock Types Comparison*
 - *Duration Rounding: floor, ceil, round (C++17)*
 - *hh_mm_ss: Breaking Down Duration (C++20)*
 - *Timer Utility Class*
 - *C++20 Formatting with std::format*
 - *C++20 Additional Clocks*

The `<chrono>` library, introduced in C++11, provides a type-safe way to work with time durations, clocks, and time points. It eliminates common errors from mixing time units and offers high-resolution timing for performance measurement. C++20 added calendar and time zone support, making `<chrono>` a complete solution for date and time handling.

Getting Current Timestamp

Source

`src/time/timestamp`

The `system_clock` provides wall-clock time that can be converted to calendar time. The `time_since_epoch()` method returns the duration since the Unix epoch (January 1, 1970). Use `duration_cast` to convert between time units.

```
#include <chrono>
#include <iostream>

int main() {
    auto now = std::chrono::system_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
        now.time_since_epoch());
    std::cout << ms.count() << "\n"; // Milliseconds since epoch
}
```

Duration and Time Arithmetic

Source

src/time/duration

Durations represent time spans with compile-time unit safety. The library provides predefined durations (hours, minutes, seconds, milliseconds, microseconds, nanoseconds) and supports arithmetic operations. C++14 added user-defined literals for convenient duration creation.

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;

    auto duration = 2h + 30min + 45s; // 2 hours, 30 minutes, 45 seconds
    auto total_seconds = std::chrono::duration_cast<std::chrono::seconds>(duration);
    std::cout << total_seconds.count() << " seconds\n"; // Output: 9045 seconds

    // Time arithmetic
    auto now = std::chrono::system_clock::now();
    auto future = now + 24h; // 24 hours from now
}
```

Measuring Elapsed Time (Profiling)

Source

src/time/profiling

Use `steady_clock` for measuring elapsed time because it is monotonic and not affected by system clock adjustments. `system_clock` can jump forward or backward due to NTP synchronization or manual changes, making it unsuitable for benchmarking.

```
#include <chrono>
#include <iostream>
#include <thread>

int main() {
    auto start = std::chrono::steady_clock::now();

    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    auto end = std::chrono::steady_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "Elapsed: " << elapsed.count() << " ms\n";
}
```

Converting to `time_t`

Source

src/time/time-t

The `to_time_t` function converts a `time_point` to the C-style `time_t` type, enabling interoperability with legacy C time functions like `gmtime`, `localtime`, and `strftime`.

```
#include <chrono>
#include <ctime>
#include <iomanip>
#include <iostream>

int main() {
    auto now = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(now);
    std::cout << std::put_time(std::gmtime(&t), "%Y-%m-%d %H:%M:%S UTC") << "\n";
}
```

Converting from Timestamp to time_point

Source

`src/time/from-timestamp`

To convert a Unix timestamp (milliseconds or seconds since epoch) back to a `time_point`, construct a duration and use it to initialize the time point.

```
#include <chrono>
#include <ctime>
#include <iomanip>
#include <iostream>

int main() {
    using namespace std::chrono_literals;

    // Convert milliseconds timestamp to time_point
    auto timestamp_ms = 1602207217323ms;
    std::chrono::system_clock::time_point tp(timestamp_ms);

    std::time_t t = std::chrono::system_clock::to_time_t(tp);
    std::cout << std::put_time(std::gmtime(&t), "%FT%TZ") << "\n";
}
```

ISO 8601 Date Format

Source

`src/time/iso8601`

ISO 8601 is the international standard for date and time representation. Use `std::put_time` with format specifiers to produce ISO 8601 compliant strings. The `%F` specifier is equivalent to `%Y-%m-%d` and `%T` is equivalent to `%H:%M:%S`.

```
#include <chrono>
#include <ctime>
#include <iomanip>
#include <iostream>

int main() {
    auto now = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(now);

    // ISO 8601 formats
    std::cout << std::put_time(std::gmtime(&t), "%Y-%m-%dT%H:%M:%SZ") << "\n";
}
```

(continues on next page)

(continued from previous page)

```
std::cout << std::put_time(std::gmtime(&t), "%FT%TZ") << "\n"; // Shorthand
}
```

Formatting with Time Zones

Source

src/time/timezone

Before C++20, time zone handling required platform-specific code or the TZ environment variable. Use `gmtime` for UTC and `localtime` for the system's local time zone.

```
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <iostream>

int main() {
    std::time_t t = std::time(nullptr);

    std::cout << "UTC:   " << std::put_time(std::gmtime(&t), "%F %T") << "\n";
    std::cout << "Local: " << std::put_time(std::localtime(&t), "%F %T %Z") << "\n";
}
```

C++20 Calendar and Time Zones

C++20 introduced comprehensive calendar support with `year`, `month`, `day`, `year_month_day`, and time zone handling via `std::chrono::zoned_time`. This eliminates the need for C-style time functions and environment variables.

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono;

    // Calendar types (C++20)
    auto today = year{2024}/month{3}/day{15};
    std::cout << today << "\n"; // 2024-03-15

    // Time zones (C++20)
    auto now = system_clock::now();
    zoned_time zt{"America/New_York", now};
    std::cout << zt << "\n";
}
```

Clock Types Comparison

C++ provides three clock types for different use cases:

- `system_clock`: Wall-clock time, can be converted to calendar time. May jump due to NTP or manual adjustments.
- `steady_clock`: Monotonic clock, never decreases. Best for measuring elapsed time and timeouts.
- `high_resolution_clock`: Highest available resolution. May be an alias for `system_clock` or `steady_clock` depending on implementation.

```
#include <chrono>
#include <iostream>

int main() {
    // Check clock properties
    std::cout << "system_clock is steady: "
               << std::chrono::system_clock::is_steady << "\n"; // Usually false
    std::cout << "steady_clock is steady: "
               << std::chrono::steady_clock::is_steady << "\n"; // Always true
}
```

Duration Rounding: floor, ceil, round (C++17)

Source

src/time/rounding

C++17 added floor, ceil, and round functions for rounding durations to specific units. Unlike duration_cast which truncates toward zero, these functions provide explicit control over rounding behavior.

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono;
    using namespace std::chrono_literals;

    auto d = 2700ms; // 2.7 seconds

    auto floored = floor<seconds>(d); // 2s (round down)
    auto ceiled = ceil<seconds>(d);   // 3s (round up)
    auto rounded = round<seconds>(d); // 3s (round to nearest)

    std::cout << "floor: " << floored.count() << "s\n";
    std::cout << "ceil:  " << ceiled.count() << "s\n";
    std::cout << "round: " << rounded.count() << "s\n";
}
```

hh_mm_ss: Breaking Down Duration (C++20)

Source

src/time/hh-mm-ss

The hh_mm_ss class template splits a duration into hours, minutes, seconds, and subseconds. This is useful for displaying durations in human-readable format without manual arithmetic.

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono;
    using namespace std::chrono_literals;

    auto d = 3h + 25min + 45s + 123ms;
    hh_mm_ss hms{d};
```

(continues on next page)

(continued from previous page)

```
std::cout << hms.hours().count() << "h "
          << hms.minutes().count() << "m "
          << hms.seconds().count() << "s "
          << hms.subseconds().count() << "ms\n";
// Output: 3h 25m 45s 123ms
}
```

Timer Utility Class

Source

src/time/timer

A reusable RAII-based timer class simplifies profiling by automatically measuring elapsed time. This pattern is commonly used for benchmarking code sections.

```
#include <chrono>
#include <iostream>
#include <string>

class Timer {
public:
    Timer(std::string name) : name_(std::move(name)), start_(std::chrono::steady_
↳clock::now()) {}

    ~Timer() {
        auto end = std::chrono::steady_clock::now();
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end - start_);
        std::cout << name_ << ": " << ms.count() << " ms\n";
    }

private:
    std::string name_;
    std::chrono::steady_clock::time_point start_;
};

int main() {
{
    Timer t("loop");
    volatile int sum = 0;
    for (int i = 0; i < 1000000; ++i) sum += i;
} // Timer prints elapsed time when destroyed
}
```

C++20 Formatting with std::format

C++20 introduced std::format support for chrono types, providing a modern alternative to std::put_time. The format specifiers follow the same pattern as strftime but with cleaner syntax.

```
#include <chrono>
#include <format>
#include <iostream>
```

(continues on next page)

(continued from previous page)

```

int main() {
    using namespace std::chrono;

    auto now = system_clock::now();

    // Format time_point directly (C++20)
    std::cout << std::format("{:%Y-%m-%d %H:%M:%S}", now) << "\n";
    std::cout << std::format("{:%F %T}", now) << "\n";

    // Format duration
    auto d = 3h + 25min + 10s;
    std::cout << std::format("{:%H:%M:%S}", d) << "\n"; // 03:25:10
}

```

C++20 Additional Clocks

C++20 added several new clock types for specialized use cases:

- `utc_clock`: UTC time including leap seconds
- `tai_clock`: International Atomic Time
- `gps_clock`: GPS time
- `file_clock`: File system time (for `std::filesystem`)

```

#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono;

    auto sys_now = system_clock::now();
    auto utc_now = utc_clock::now();

    // Convert between clocks
    auto utc_from_sys = utc_clock::from_sys(sys_now);

    std::cout << "System: " << sys_now << "\n";
    std::cout << "UTC:    " << utc_now << "\n";
}

```

1.3.20 Modules

Table of Contents

- *Modules*
 - *Module Basics*
 - *Export Declarations*
 - *Module Implementation Units*

- *Module Partitions*
- *Importing Standard Library*
- *Global Module Fragment*
- *Building Modules with CMake*
- *Compiler Support*
- *See Also*

C++20 modules replace the traditional preprocessor-based `#include` system with a modern import mechanism. Unlike headers that are textually included and re-parsed for every translation unit, modules are compiled once into a binary format and imported efficiently. This eliminates redundant parsing, prevents macro leakage between files, removes include order dependencies, and significantly improves compilation times for large projects.

Module Basics

Source

`src/modules/basic`

A module consists of a module interface unit (the public API) and optionally module implementation units. The `export` keyword marks declarations visible to importers. Everything not exported remains internal to the module.

```
// math.cppm - module interface unit
export module math; // module declaration

// Exported - visible to importers
export int add(int a, int b) {
    return a + b;
}

export int multiply(int a, int b) {
    return a * b;
}

// Not exported - internal to module
int helper() {
    return 42;
}
```

```
// main.cpp
import math; // import the module

int main() {
    int sum = add(1, 2); // OK: exported
    int prod = multiply(3, 4); // OK: exported
    // helper(); // Error: not exported
}
```

Export Declarations

Multiple ways to export declarations from a module. Use export blocks to group related exports together, or export individual declarations for fine-grained control over the public interface.

```
export module shapes;

// Export individual declaration
export constexpr double pi = 3.14159265359;

// Export a class
export class Circle {
public:
    Circle(double r) : radius(r) {}
    double area() const { return pi * radius * radius; }

private:
    double radius;
};

// Export block - everything inside is exported
export {
    class Rectangle {
    public:
        Rectangle(double w, double h) : width(w), height(h) {}
        double area() const { return width * height; }

    private:
        double width, height;
    };

    double triangle_area(double base, double height) {
        return 0.5 * base * height;
    }
}

// Export namespace
export namespace geometry {
    double circumference(double radius) {
        return 2 * pi * radius;
    }
}
```

Module Implementation Units

Separate interface from implementation by using module implementation units. The implementation unit has access to all module internals but cannot export new declarations. This pattern keeps interfaces clean and reduces recompilation when only implementation changes.

```
// calculator.cppm - interface unit
export module calculator;

export class Calculator {
public:
```

(continues on next page)

(continued from previous page)

```

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
int divide(int a, int b);
};

```

```

// calculator_impl.cpp - implementation unit
module calculator; // no 'export' keyword

int Calculator::add(int a, int b) {
    return a + b;
}

int Calculator::subtract(int a, int b) {
    return a - b;
}

int Calculator::multiply(int a, int b) {
    return a * b;
}

int Calculator::divide(int a, int b) {
    return b != 0 ? a / b : 0;
}

```

Module Partitions

Source

src/modules/partitions

Large modules can be split into partitions for better organization. Partitions are internal to the module and must be explicitly re-exported from the primary module interface if they should be visible to importers.

```

// graphics-shapes.cppm - partition
export module graphics:shapes;

export class Point {
public:
    double x, y;
};

export class Line {
public:
    Point start, end;
};

```

```

// graphics-colors.cppm - partition
export module graphics:colors;

export struct Color {
    unsigned char r, g, b, a;
};

```

(continues on next page)

(continued from previous page)

```
export constexpr Color Red{255, 0, 0, 255};
export constexpr Color Green{0, 255, 0, 255};
export constexpr Color Blue{0, 0, 255, 255};
```

```
// graphics.cppm - primary interface
export module graphics;

export import :shapes; // re-export shapes partition
export import :colors; // re-export colors partition

export class Canvas {
public:
    void draw_line(const Line& line, const Color& color);
};
```

```
// main.cpp
import graphics;

int main() {
    Point p1{0, 0};
    Point p2{100, 100};
    Line line{p1, p2};

    Canvas canvas;
    canvas.draw_line(line, Red);
}
```

Importing Standard Library

C++23 introduces `import std;` to import the entire standard library as a module. For C++20, you can import individual standard headers as header units, though compiler support varies.

```
// C++23: import entire standard library
import std;

int main() {
    std::vector<std::string> names{"Alice", "Bob"};
    for (const auto& name : names) {
        std::cout << name << "\n";
    }
}
```

```
// C++20: import standard headers as header units (compiler-dependent)
import <vector>;
import <string>;
import <iostream>;

int main() {
    std::vector<int> v{1, 2, 3};
    std::cout << v.size() << "\n";
}
```

Global Module Fragment

The global module fragment allows including traditional headers that must be processed by the preprocessor before the module declaration. Use this for headers that rely on macros or cannot be imported as header units.

```
module; // start global module fragment

// Traditional includes processed here
#include <cstdint>
#include <cassert>
#ifdef _WIN32
#include <windows.h>
#endif

export module platform; // module declaration ends global fragment

export size_t get_page_size() {
#ifdef _WIN32
    SYSTEM_INFO si;
    GetSystemInfo(&si);
    return si.dwPageSize;
#else
    return 4096;
#endif
}
```

Building Modules with CMake

CMake 3.28+ supports C++20 modules. Enable with `CXX_SCAN_FOR_MODULES` and use `FILE_SET` to specify module interface files.

```
cmake_minimum_required(VERSION 3.28)
project(myproject CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(math)
target_sources(math
    PUBLIC
    FILE_SET CXX_MODULES FILES
    math.cppm
)

add_executable(main main.cpp)
target_link_libraries(main PRIVATE math)
```

Compiler Support

Module support varies across compilers. Check your compiler version and flags:

Compiler	Min Version	Flags
MSVC	VS 2019 16.10+	/std:c++20 (best support)
GCC	11+	-std=c++20 -fmodules-ts
Clang	16+	-std=c++20 -fmodules

See Also

- *CMake* - CMake build system
- *C++ Basics* - C++ fundamentals

1.3.21 CMake

Table of Contents

- *CMake*
 - *CMake CLI Quick Reference*
 - * *Configure and Build*
 - * *Build Types and Options*
 - * *Inspect and Debug*
 - * *Testing and Packaging*
 - *Project Setup*
 - * *Minimal Project*
 - * *Modern C++ Standard Setting*
 - * *Build Commands*
 - *Targets and Properties*
 - * *Executables and Libraries*
 - * *Include Directories*
 - * *Compile Options and Definitions*
 - * *Linking*
 - * *PUBLIC vs PRIVATE vs INTERFACE*
 - *Dependency Management*
 - * *FetchContent (Recommended)*
 - * *Multiple Dependencies*
 - * *find_package*
 - * *Common find_package Examples*
 - * *pkg-config Fallback*

- *Project Structure*
 - * *Subdirectories*
- *Generator Expressions*
 - * *Common Patterns*
 - * *Cross-Platform Compile Options*
- *Installation and Packaging*
 - * *Basic Installation*
 - * *Export for find_package*
 - * *CPack for Distribution*
- *Testing*
- *Useful Variables*
- *Options and Cache Variables*
- *Presets (CMake 3.19+)*
- *CUDA Support*
 - * *Basic CUDA Project*
 - * *Mixed C++/CUDA*
 - * *CUDA Architectures*
 - * *CUDA Libraries*

CMake has evolved significantly, and modern CMake (3.12+) emphasizes target-based configuration over global variables. This cheatsheet covers contemporary best practices for building C++ projects, managing dependencies, and creating reusable packages.

CMake CLI Quick Reference

Common command-line operations for configuring, building, and managing CMake projects.

Configure and Build

```
# Configure (generate build system)
cmake -B build                # Configure into build/
cmake -B build -S src         # Specify source directory
cmake -B build -G Ninja       # Use Ninja generator
cmake -B build -G "Unix Makefiles" # Use Make generator

# Build
cmake --build build           # Build the project
cmake --build build -j 8      # Parallel build (8 jobs)
cmake --build build --target app # Build specific target
cmake --build build --clean-first # Clean before building

# Install
cmake --install build         # Install to default prefix
cmake --install build --prefix /opt # Install to custom prefix
```

Build Types and Options

The `-D` flag sets CMake cache variables. These persist in `CMakeCache.txt` and control build configuration.

```
# Build types
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake -B build -DCMAKE_BUILD_TYPE=MinSizeRel

# Compiler selection
cmake -B build -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++
cmake -B build -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++

# Install prefix
cmake -B build -DCMAKE_INSTALL_PREFIX=/usr/local
cmake -B build -DCMAKE_INSTALL_PREFIX=$HOME/.local

# Project options (defined by option() in CMakeLists.txt)
cmake -B build -DBUILD_TESTS=ON
cmake -B build -DBUILD_SHARED_LIBS=ON
cmake -B build -DENABLE_FEATURE_X=OFF

# Multiple options
cmake -B build \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_CXX_STANDARD=17 \
    -DBUILD_SHARED_LIBS=ON \
    -DBUILD_TESTS=OFF
```

Inspect and Debug

```
# List available targets
cmake --build build --target help

# List cache variables
cmake -B build -L           # List non-advanced variables
cmake -B build -LA          # List all variables
cmake -B build -LAH         # List all with help strings

# Print variable value
cmake -B build -N -L | grep CMAKE_CXX

# Verbose build output
cmake --build build --verbose
cmake --build build -- VERBOSE=1  # Make-specific
```

Testing and Packaging

```
# Run tests
ctest --test-dir build           # Run all tests
ctest --test-dir build -V        # Verbose output
```

(continues on next page)

(continued from previous page)

```
ctest --test-dir build -j 8      # Parallel testing
ctest --test-dir build -R "regex" # Run matching tests
ctest --test-dir build --rerun-failed

# Create package
cmake --build build --target package
cpack --config build/CPackConfig.cmake
```

Project Setup

Every CMake project starts with basic configuration. Modern CMake recommends setting the C++ standard on targets rather than globally, which provides better control and avoids polluting the global namespace.

Minimal Project

The simplest CMake project requires only a few lines. The `cmake_minimum_required` ensures compatibility, while `project` defines metadata used throughout the build.

```
cmake_minimum_required(VERSION 3.16)
project(MyProject VERSION 1.0.0 LANGUAGES CXX)

add_executable(app main.cpp)
target_compile_features(app PRIVATE cxx_std_17)
```

Modern C++ Standard Setting

Using `target_compile_features` is preferred over global `CMAKE_CXX_STANDARD` because it applies only to specific targets and can require specific language features.

```
# Per-target (preferred)
target_compile_features(app PRIVATE cxx_std_17)
target_compile_features(app PRIVATE cxx_std_20)

# Or require specific features
target_compile_features(app PRIVATE cxx_constexpr cxx_lambdas)

# Global (legacy, avoid)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

Build Commands

CMake separates configuration from building. The `-B` flag specifies the build directory, keeping source and build files separate (out-of-source build).

```
# Configure and build
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)

# Common build types
```

(continues on next page)

(continued from previous page)

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo

# Specify generator
cmake -B build -G Ninja
cmake -B build -G "Unix Makefiles"

# Install
cmake --install build --prefix /usr/local
```

Targets and Properties

Modern CMake is target-centric. Use `target_*` commands instead of global variables like `CMAKE_CXX_FLAGS`. This approach provides better encapsulation and makes dependencies explicit.

Executables and Libraries

CMake supports multiple library types. Use `STATIC` for archives, `SHARED` for dynamic libraries, and `INTERFACE` for header-only libraries.

```
# Executable
add_executable(app main.cpp utils.cpp)

# Static library
add_library(mylib STATIC lib.cpp)

# Shared library
add_library(mylib SHARED lib.cpp)

# Header-only library
add_library(mylib INTERFACE)

# Let CMake decide (BUILD_SHARED_LIBS)
add_library(mylib lib.cpp)
```

Include Directories

The `target_include_directories` command specifies where the compiler should look for header files. Generator expressions like `$<BUILD_INTERFACE>` and `$<INSTALL_INTERFACE>` allow different paths during build and after installation. This is essential for creating relocatable packages that work both in the build tree and when installed to a system location.

```
target_include_directories(mylib
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/src
)
```

Compile Options and Definitions

Compile flags and preprocessor definitions should be set per-target. Generator expressions enable conditional flags based on build type or compiler.

```
# Compile flags
target_compile_options(app PRIVATE
    -Wall -Wextra -Wpedantic
    $<$<CONFIG:Debug>:-g -O0>
    $<$<CONFIG:Release>:-O3>
)

# Preprocessor definitions
target_compile_definitions(app PRIVATE
    APP_VERSION="${PROJECT_VERSION}"
    $<$<CONFIG:Debug>:DEBUG_MODE>
)
```

Linking

The `target_link_libraries` command specifies libraries to link against and automatically handles transitive dependencies. When you link to a library, CMake propagates that library's PUBLIC include directories and compile definitions to your target. Libraries with `::` in their name (like `Threads::Threads`) are imported targets that carry all necessary usage requirements.

```
target_link_libraries(app
    PRIVATE
        mylib
        Threads::Threads
    PUBLIC
        Boost::boost
)
```

PUBLIC vs PRIVATE vs INTERFACE

These keywords control how properties propagate to dependent targets:

- **PRIVATE:** The property applies only to the target itself, not to anything that links to it. Use for implementation details like source files or internal dependencies.
- **PUBLIC:** The property applies to both the target and anything that links to it. Use for headers that consumers need to include.
- **INTERFACE:** The property applies only to targets that link to this one, not to the target itself. Use for header-only libraries.

Understanding these keywords is crucial for creating well-encapsulated libraries that don't leak unnecessary dependencies.

```
# PRIVATE: only for this target
# PUBLIC: for this target and dependents
# INTERFACE: only for dependents (header-only libs)

add_library(mylib STATIC lib.cpp)
target_include_directories(mylib PUBLIC include/) # Consumers need headers
```

(continues on next page)

(continued from previous page)

```
target_link_libraries(mylib PRIVATE internal_dep)  # Implementation detail
target_compile_definitions(mylib INTERFACE USE_MYLIB)  # Define for consumers
```

Dependency Management

Modern CMake provides multiple ways to manage external dependencies. FetchContent is preferred for most cases as it integrates seamlessly with the build.

FetchContent (Recommended)

FetchContent downloads and configures dependencies at configure time, making them available as regular CMake targets. Unlike ExternalProject (which builds at build time), FetchContent integrates dependencies into your build system so they can be used immediately with `target_link_libraries`. This is the recommended approach for most projects as it provides the simplest integration.

```
include(FetchContent)

FetchContent_Declare(
    fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG 10.1.1
)
FetchContent_MakeAvailable(fmt)

target_link_libraries(app PRIVATE fmt::fmt)
```

Multiple Dependencies

Multiple dependencies can be declared together and made available with a single `FetchContent_MakeAvailable` call for efficiency.

```
include(FetchContent)

FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG v1.14.0
)
FetchContent_Declare(
    nlohmann_json
    GIT_REPOSITORY https://github.com/nlohmann/json.git
    GIT_TAG v3.11.3
)

FetchContent_MakeAvailable(googletest nlohmann_json)

target_link_libraries(app PRIVATE nlohmann_json::nlohmann_json)
target_link_libraries(tests PRIVATE GTest::gtest_main)
```

find_package

Use `find_package` for system-installed libraries or when you want to use pre-built binaries instead of building from source. CMake supports two modes:

- **Config mode** (preferred): The library provides its own CMake config files (e.g., `fmtConfig.cmake`). These are typically installed alongside the library.
- **Module mode**: CMake provides a `Find*.cmake` module that knows how to locate the library. Used for libraries that don't provide CMake support.

Config mode is preferred because the library author knows best how to expose their targets.

```
# Config mode (preferred, library provides CMake config)
find_package(fmt REQUIRED CONFIG)
target_link_libraries(app PRIVATE fmt::fmt)

# Module mode (CMake provides Find*.cmake)
find_package(Threads REQUIRED)
find_package(OpenSSL REQUIRED)
target_link_libraries(app PRIVATE
    Threads::Threads
    OpenSSL::SSL
    OpenSSL::Crypto
)

# Optional dependency
find_package(Boost COMPONENTS filesystem)
if(Boost_FOUND)
    target_link_libraries(app PRIVATE Boost::filesystem)
endif()
```

Common find_package Examples

These examples show how to link against commonly used libraries that provide CMake config files or have built-in Find modules.

```
# Boost (header-only and compiled components)
find_package(Boost 1.70 REQUIRED COMPONENTS system filesystem)
target_link_libraries(app PRIVATE Boost::system Boost::filesystem)

# Boost header-only
find_package(Boost REQUIRED)
target_link_libraries(app PRIVATE Boost::boost)

# Protobuf
find_package(Protobuf REQUIRED)
target_link_libraries(app PRIVATE protobuf::libprotobuf)

# CURL
find_package(CURL REQUIRED)
target_link_libraries(app PRIVATE CURL::libcurl)

# OpenSSL
find_package(OpenSSL REQUIRED)
```

(continues on next page)

(continued from previous page)

```
target_link_libraries(app PRIVATE OpenSSL::SSL OpenSSL::Crypto)

# ZLIB
find_package(ZLIB REQUIRED)
target_link_libraries(app PRIVATE ZLIB::ZLIB)
```

pkg-config Fallback

For libraries that don't provide CMake config files, pkg-config can be used as a fallback. The `IMPORTED_TARGET` option creates a proper CMake target.

```
find_package(PkgConfig REQUIRED)
pkg_check_modules(LIBSSH2 REQUIRED IMPORTED_TARGET libssh2)
target_link_libraries(app PRIVATE PkgConfig::LIBSSH2)
```

Project Structure

Organizing larger projects with subdirectories keeps code modular and maintainable. Each subdirectory can have its own `CMakeLists.txt` defining local targets.

Subdirectories

A typical project structure separates source, library, and test code into distinct directories, each with its own `CMakeLists.txt`.

```
project/
├── CMakeLists.txt
├── src/
│   ├── CMakeLists.txt
│   └── main.cpp
├── lib/
│   ├── CMakeLists.txt
│   ├── mylib.cpp
│   └── mylib.h
└── tests/
    ├── CMakeLists.txt
    └── test_mylib.cpp
```

Root `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.16)
project(MyProject VERSION 1.0.0 LANGUAGES CXX)

option(BUILD_TESTS "Build tests" ON)

add_subdirectory(lib)
add_subdirectory(src)

if(BUILD_TESTS)
    enable_testing()
    add_subdirectory(tests)
endif()
```

lib/CMakeLists.txt:

```
add_library(mylib mylib.cpp)
add_library(MyProject::mylib ALIAS mylib)

target_include_directories(mylib
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>
        $<INSTALL_INTERFACE:include>
)
target_compile_features(mylib PUBLIC cxx_std_17)
```

src/CMakeLists.txt:

```
add_executable(app main.cpp)
target_link_libraries(app PRIVATE MyProject::mylib)
```

Generator Expressions

Generator expressions are evaluated during build system generation, enabling conditional configuration based on build type, compiler, platform, and more. They use the syntax `$<...>`.

Common Patterns

Generator expressions enable conditional logic without `if()` statements, making `CMakeLists.txt` cleaner and more portable.

```
# Build type
$<$<CONFIG:Debug>:-g>
$<$<CONFIG:Release>:-O3>
$<IF:$<CONFIG:Debug>,-g,-O3>

# Compiler
$<$<CXX_COMPILER_ID:GNU>:-Wall>
$<$<CXX_COMPILER_ID:Clang>:-Weverything>
$<$<CXX_COMPILER_ID:MSVC>:/W4>

# Platform
$<$<PLATFORM_ID:Linux>:-pthread>
$<$<PLATFORM_ID:Windows>:-DWIN32>

# Build vs install paths
$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
$<INSTALL_INTERFACE:include>
```

Cross-Platform Compile Options

Generator expressions make it easy to set compiler-specific flags without platform-specific `if()` blocks.

```
target_compile_options(app PRIVATE
    $<$<CXX_COMPILER_ID:MSVC>:/W4 /WX>
    $<$<NOT:$<CXX_COMPILER_ID:MSVC>>:-Wall -Wextra -Werror>
)
```

Installation and Packaging

Making your library installable allows other projects to use it via `find_package`. This requires exporting targets and creating config files.

Basic Installation

The `install` command copies built artifacts to the installation prefix (default `/usr/local` on Unix). Including `GNUInstallDirs` provides standard directory variables (`CMAKE_INSTALL_LIBDIR`, `CMAKE_INSTALL_LIBDIR`, etc.) that follow platform conventions, ensuring your project installs correctly on different systems.

```
include(GNUInstallDirs)

install(TARGETS mylib app
  EXPORT MyProjectTargets
  LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
  ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
  RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
  INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
)

install(DIRECTORY include/
  DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
)
```

Export for `find_package`

To allow other CMake projects to use your library with `find_package(MyProject)`, you need to export your targets and create config files. This involves:

1. Exporting targets to a `*Targets.cmake` file
2. Creating a `*Config.cmake` that includes the targets file
3. Optionally creating a `*ConfigVersion.cmake` for version checking

The namespace (e.g., `MyProject::`) prevents target name collisions and signals that the target is imported.

```
install(EXPORT MyProjectTargets
  FILE MyProjectTargets.cmake
  NAMESPACE MyProject::
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)

include(CMakePackageConfigHelpers)

configure_package_config_file(
  cmake/MyProjectConfig.cmake.in
  ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
  INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)

write_basic_package_version_file(
  ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
  VERSION ${PROJECT_VERSION}
  COMPATIBILITY SameMajorVersion
)
```

(continues on next page)

(continued from previous page)

```

)

install(FILES
  ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
  ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)

```

cmake/MyProjectConfig.cmake.in:

```

@PACKAGE_INIT@
include(CMakeFindDependencyMacro)
# find_dependency(Boost REQUIRED) # if needed
include("${CMAKE_CURRENT_LIST_DIR}/MyProjectTargets.cmake")

```

CPack for Distribution

CPack generates distributable packages (ZIP, TGZ, DEB, RPM) from your CMake project.

```

set(CPACK_PACKAGE_NAME ${PROJECT_NAME})
set(CPACK_PACKAGE_VERSION ${PROJECT_VERSION})
set(CPACK_PACKAGE_VENDOR "Your Name")
set(CPACK_GENERATOR "TGZ;ZIP")

# For DEB packages
set(CPACK_DEBIAN_PACKAGE_MAINTAINER "your@email.com")

include(CPack)

```

```
cmake --build build --target package
```

Testing

CTest is CMake's built-in testing framework that integrates with CMake to provide test discovery and execution. For GoogleTest users, the `gtest_discover_tests` function automatically registers all `TEST()` macros as individual CTest tests, enabling parallel execution and better reporting.

```

enable_testing()

add_executable(tests test_main.cpp)
target_link_libraries(tests PRIVATE mylib GTest::gtest_main)

include(GoogleTest)
gtest_discover_tests(tests)

# Or manual test registration
add_test(NAME MyTest COMMAND tests)

```

```

ctest --test-dir build -V
ctest --test-dir build -j$(nproc)

```

Useful Variables

CMake provides many built-in variables for accessing project information, paths, and build configuration. Understanding these variables helps write portable CMakeLists.txt files that work across different environments.

```
# Project info
${PROJECT_NAME}
${PROJECT_VERSION}
${PROJECT_SOURCE_DIR}
${PROJECT_BINARY_DIR}

# Current directory
${CMAKE_CURRENT_SOURCE_DIR}
${CMAKE_CURRENT_BINARY_DIR}
${CMAKE_CURRENT_LIST_DIR}

# Install paths (use with GNUInstallDirs)
${CMAKE_INSTALL_PREFIX}
${CMAKE_INSTALL_BINDIR}
${CMAKE_INSTALL_LIBDIR}
${CMAKE_INSTALL_INCLUDEDIR}

# Build info
${CMAKE_BUILD_TYPE}
${CMAKE_CXX_COMPILER_ID}
${CMAKE_SYSTEM_NAME}
```

Options and Cache Variables

Options allow users to customize the build without modifying CMakeLists.txt. Cache variables persist across CMake invocations (stored in CMakeCache.txt) and can be set via command line with `-D`. This is how users enable/disable features, specify paths, or configure build options.

```
# Boolean option
option(BUILD_SHARED_LIBS "Build shared libraries" ON)
option(BUILD_TESTS "Build tests" OFF)

# Cache variable with type
set(MY_OPTION "default" CACHE STRING "Description")
set(MY_PATH "/usr/local" CACHE PATH "Install path")

# Use in CMake
if(BUILD_TESTS)
    add_subdirectory(tests)
endif()
```

```
cmake -B build -DBUILD_TESTS=ON -DMY_OPTION=custom
```

Presets (CMake 3.19+)

CMakePresets.json provides reproducible, shareable build configurations that can be version-controlled with your project. It eliminates the need to remember complex command-line options and ensures all developers use consistent settings. Presets can define configure options, build settings, and test configurations.

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "debug",
      "binaryDir": "${sourceDir}/build/debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug"
      }
    },
    {
      "name": "release",
      "binaryDir": "${sourceDir}/build/release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    }
  ],
  "buildPresets": [
    {
      "name": "debug",
      "configurePreset": "debug"
    },
    {
      "name": "release",
      "configurePreset": "release"
    }
  ]
}
```

```
cmake --preset debug
cmake --build --preset debug
```

CUDA Support

CMake has native CUDA support since version 3.8, treating CUDA as a first-class language alongside C and C++. This eliminates the need for the legacy FindCUDA module. The CUDAToolkit package (CMake 3.17+) provides imported targets for CUDA libraries like cuBLAS and cuFFT.

Basic CUDA Project

Enable CUDA by adding it to the LANGUAGES list. CMake automatically finds the CUDA compiler (nvcc) and configures .cu file compilation.

```
cmake_minimum_required(VERSION 3.18)
project(cuda_example LANGUAGES CXX CUDA)

find_package(CUDAToolkit REQUIRED)

add_executable(app main.cu)
target_compile_features(app PRIVATE cuda_std_17)
target_link_libraries(app PRIVATE CUDA::cudart)
```

Mixed C++/CUDA

For projects with both `.cpp` and `.cu` files, enable separable compilation to allow device code in different translation units to call each other.

```
add_library(kernels kernels.cu)
set_target_properties(kernels PROPERTIES
    CUDA_SEPARABLE_COMPILATION ON
    CUDA_ARCHITECTURES "70;80;86;90" # Volta, Ampere, Hopper
)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE kernels CUDA::cudart)
```

CUDA Architectures

`CUDA_ARCHITECTURES` specifies GPU compute capabilities to target. Use semicolon-separated values for multiple architectures. Common values:

- 70: Volta (V100)
- 75: Turing (RTX 20xx)
- 80: Ampere (A100)
- 86: Ampere (RTX 30xx)
- 89: Ada Lovelace (RTX 40xx)
- 90: Hopper (H100)

```
# Per-target (preferred)
set_target_properties(app PROPERTIES CUDA_ARCHITECTURES "80;86;90") # Ampere, Hopper

# Global default
set(CMAKE_CUDA_ARCHITECTURES "80;86;90")

# Detect host GPU at configure time (CMake 3.24+)
set(CMAKE_CUDA_ARCHITECTURES native)
```

CUDA Libraries

The `CUDAToolkit` package provides imported targets for NVIDIA libraries.

```
find_package(CUDAToolkit REQUIRED)

target_link_libraries(app PRIVATE
    CUDA::cudart      # Runtime API
    CUDA::cublas      # Dense linear algebra
    CUDA::cufft       # Fast Fourier transforms
    CUDA::curand       # Random number generation
    CUDA::cusolver     # Direct solvers
    CUDA::cusparse     # Sparse linear algebra
    CUDA::nvml         # GPU monitoring and management
)
```

1.4 System Programming

Building software that interacts directly with the operating system requires understanding system calls, POSIX APIs, and the abstractions that Unix-like systems provide. System programming bridges the gap between application logic and kernel services, enabling developers to build efficient servers, daemons, and utilities.

Covered topics include file I/O operations and descriptor management, network communication through BSD sockets supporting TCP/IP and UDP protocols, signal handling for asynchronous events, and concurrent execution using threads and synchronization primitives. Mastery of these concepts is essential for developing robust network services and high-performance applications.

1.4.1 Process

Table of Contents

- *Process*
 - *Introduction*
 - *Fork and Wait*
 - *Fork and Exec*
 - *Waitpid with Options*
 - *Process IDs*
 - *Environment Variables*
 - *System Information*

Introduction

Unix processes are the fundamental unit of execution, each with its own address space, file descriptors, and execution context. The `fork()` system call creates new processes by duplicating the calling process, while the `exec()` family replaces the current process image with a new program. Together, fork and exec form the basis for running programs, building shells, and implementing job control.

Process management also involves waiting for child processes to terminate using `wait()` or `waitpid()`, which prevents zombie processes (terminated children whose exit status hasn't been collected). Understanding process relationships—parent-child hierarchies, process groups, and sessions—is essential for building robust system software, daemons, and interactive applications.

Fork and Wait

Source

`src/process/fork-wait`

The `fork()` system call creates a new process by duplicating the calling process. After fork returns, two processes continue execution from the same point: the parent receives the child's process ID as the return value, while the child receives zero. A negative return value indicates fork failed, typically due to resource limits.

The `wait()` function blocks the parent until any child process terminates, returning the child's PID and storing its exit status. Use the `WEXITSTATUS()` macro to extract the exit code from the status value. Failing to wait for children creates zombie processes—entries in the process table that consume system resources until the parent collects their status or terminates.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        exit(42);
    }
    int status;
    wait(&status);
    printf("Child exited with %d\n", WEXITSTATUS(status));
}
```

```
$ ./fork-wait
Child PID: 12345
Child exited with 42
```

Fork and Exec

Source

src/process/fork-exec

The `exec()` family of functions replaces the current process image with a new program. After a successful `exec` call, the original code is completely replaced—`exec` only returns if an error occurs. The various `exec` variants differ in how they accept arguments and whether they search the `PATH` environment variable.

`execvp()` takes an argument vector (array of strings) and searches `PATH` for the executable, making it convenient for shell-like behavior. `execve()` is the underlying system call that requires the full path and explicit environment. Other variants like `execl()` and `execvp()` accept arguments as variadic parameters rather than an array.

The fork-exec pattern is fundamental to Unix: `fork` creates a child process, which then calls `exec` to run a different program. This separation allows the parent to set up the child's environment (redirecting file descriptors, changing directories, setting resource limits) before the new program starts.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid == 0) {
        char *args[] = {"ls", "-l", NULL};
```

(continues on next page)

(continued from previous page)

```

    execvp("ls", args);
    perror("execvp"); // only reached on error
    exit(1);
}
wait(NULL);
printf("Child finished\n");
}

```

```

$ ./fork-exec
total 16
-rwxr-xr-x  1 user  staff  8432 Jan  6 10:00 fork-exec
Child finished

```

Waitpid with Options

Source

src/process/waitpid

The `waitpid()` function provides more control than `wait()`, allowing you to wait for a specific child process or use non-blocking checks. The first argument specifies which child to wait for: a positive PID waits for that specific child, -1 waits for any child (like `wait()`), 0 waits for any child in the same process group, and a negative value waits for any child in the specified process group.

The `WNOHANG` option makes `waitpid` return immediately if no child has exited, returning 0 instead of blocking. This enables polling patterns where the parent can do other work while periodically checking for child termination. The `WUNTRACED` option also reports children that have been stopped by a signal.

Use the status inspection macros to determine how the child terminated: `WIFEXITED()` checks for normal exit, `WIFSIGNALED()` checks if killed by a signal, and `WIFSTOPPED()` checks if stopped. Extract details with `WEXITSTATUS()`, `WTERMSIG()`, and `WSTOPSIG()` respectively.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(1);
        exit(0);
    }

    int status;
    while (waitpid(pid, &status, WNOHANG) == 0) {
        printf("Child still running...\n");
        usleep(200000);
    }

    if (WIFEXITED(status))
        printf("Exited: %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Killed by signal: %d\n", WTERMSIG(status));
}

```

```
$ ./waitpid
Child still running...
Child still running...
Child still running...
Child still running...
Child still running...
Exited: 0
```

Process IDs

Source

[src/process/pid](#)

Every process has several associated identifiers that define its place in the process hierarchy. `getpid()` returns the process's own ID, while `getppid()` returns the parent's ID. When a parent terminates before its children, those orphaned processes are adopted by `init` (PID 1) or a subreaper process.

Process groups and sessions organize processes for job control. `getpgrp()` returns the process group ID, which groups related processes (like a pipeline) for signal delivery. `getsid()` returns the session ID, which groups process groups under a controlling terminal. These concepts are essential for implementing shells and understanding how signals like `SIGINT` (Ctrl+C) are delivered to foreground process groups.

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());
    printf("PGRP: %d\n", getpgrp());
    printf("SID: %d\n", getsid(0));
}
```

```
$ ./pid
PID: 12345
PPID: 12300
PGRP: 12345
SID: 12300
```

Environment Variables

Source

[src/process/envIRON](#)

Environment variables provide a way to pass configuration to processes without command-line arguments. Each process inherits its parent's environment, which can be modified before calling `exec` to customize the child's environment. Common variables include `PATH` (executable search directories), `HOME` (user's home directory), and `LANG` (locale settings).

`getenv()` retrieves a variable's value, returning `NULL` if not set. `setenv()` sets a variable, with the third argument controlling whether to overwrite an existing value. `unsetenv()` removes a variable from the environment. These functions modify the process's own environment; changes don't affect the parent or other processes.

The global `environ` variable points to the full environment array, useful for iterating over all variables or passing to `execve()`. Environment variables are commonly used for configuration that shouldn't be hardcoded, such as API keys, database URLs, or feature flags.


```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("HOME: %s\n", getenv("HOME"));

    setenv("MY_VAR", "hello", 1);
    printf("MY_VAR: %s\n", getenv("MY_VAR"));

    unsetenv("MY_VAR");
    printf("MY_VAR after unset: %s\n", getenv("MY_VAR"));
}
```

```
$ ./environ
HOME: /home/user
MY_VAR: hello
MY_VAR after unset: (null)
```

System Information

Source

[src/process/sysinfo](#)

The `uname()` system call fills a `struct utsname` with information about the operating system and hardware. Fields include `sysname` (OS name like “Linux” or “Darwin”), `nodename` (network hostname), `release` (kernel version), `version` (build information), and `machine` (hardware architecture like “x86_64” or “arm64”).

`gethostname()` retrieves just the system’s hostname, which may differ from the `nodename` in `utsname` on some systems. These functions are useful for logging, generating platform-specific behavior, or displaying system information to users. The information returned reflects the running kernel, not necessarily the distribution or installed packages.

```
#include <stdio.h>
#include <sys/utsname.h>
#include <unistd.h>

int main(void) {
    struct utsname buf;
    uname(&buf);
    printf("OS:      %s\n", buf.sysname);
    printf("Host:     %s\n", buf.nodename);
    printf("Release: %s\n", buf.release);
    printf("Arch:    %s\n", buf.machine);

    char hostname[256];
    gethostname(hostname, sizeof(hostname));
    printf("Hostname: %s\n", hostname);
}
```

```
$ ./sysinfo
OS:      Linux
Host:     myhost
Release: 5.15.0-generic
Arch:     x86_64
```

(continues on next page)

(continued from previous page)

Hostname: myhost

1.4.2 File

Table of Contents

- *File*
 - *Introduction*
 - *File Size with `lseek`*
 - *File Size with `fstat`*
 - *Copy File Contents*
 - *Read Lines from File*
 - *Read Entire File into Memory*
 - *Check File Types*
 - *Directory Tree Walk*
 - *Pipe to Shell Command with `popen`*
 - *Named Pipe (FIFO)*

Introduction

File I/O is fundamental to Unix programming. The Unix philosophy of “everything is a file” means that the same system calls—`open()`, `read()`, `write()`, `close()`—work for regular files, devices, pipes, and sockets. This unified interface simplifies programming and enables powerful composition of tools.

Unix provides two levels of file I/O: low-level system calls (`open`, `read`, `write`) that work with file descriptors, and the standard C library’s buffered I/O (`fopen`, `fread`, `fwrite`) that provides automatic buffering and portability. Low-level calls offer more control and are necessary for certain operations like file locking and memory mapping, while buffered I/O is more convenient for text processing and sequential access.

File Size with `lseek`

Source

`src/file/lseek-size`

The `lseek()` system call repositions the file offset for an open file descriptor. By seeking to the end of the file (`SEEK_END`) and reading the resulting offset, you can determine the file size. This technique works for regular files but not for pipes, sockets, or some device files where seeking is not supported.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) return 1;
```

(continues on next page)

(continued from previous page)

```

off_t start = lseek(fd, 0, SEEK_SET);
off_t end = lseek(fd, 0, SEEK_END);
printf("File size: %lld bytes\n", (long long)(end - start));

close(fd);
}

```

```

$ echo "Hello" > hello.txt
$ ./lseek-size hello.txt
File size: 6 bytes

```

File Size with fstat

Source

src/file/fstat-size

The `fstat()` system call retrieves metadata about an open file, including its size, permissions, ownership, and timestamps. Unlike `lseek()`, `fstat()` doesn't modify the file offset and provides additional information. The `stat()` variant works with a pathname instead of a file descriptor.

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) return 1;

    struct stat st;
    fstat(fd, &st);
    printf("File size: %lld bytes\n", (long long)st.st_size);

    close(fd);
}

```

```

$ echo "Hello" > hello.txt
$ ./fstat-size hello.txt
File size: 6 bytes

```

Copy File Contents

Source

src/file/copy-file

Copying a file involves reading from the source and writing to the destination in a loop until all data is transferred. The buffer size affects performance: larger buffers reduce system call overhead but use more memory. This example also preserves the source file's permissions using `fstat()` to read them and passing them to `open()` when creating the destination.

```

#include <stdio.h>
#include <fcntl.h>

```

(continues on next page)

(continued from previous page)

```
#include <unistd.h>
#include <sys/stat.h>

#define BUF_SIZE 4096

int main(int argc, char *argv[]) {
    int sfd = open(argv[1], O_RDONLY);
    struct stat st;
    fstat(sfd, &st);

    int dfd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, st.st_mode);

    char buf[BUF_SIZE];
    ssize_t n;
    while ((n = read(sfd, buf, BUF_SIZE)) > 0)
        write(dfd, buf, n);

    close(sfd);
    close(dfd);
}
```

```
$ echo "Hello World" > source.txt
$ ./copy-file source.txt dest.txt
$ diff source.txt dest.txt
```

Read Lines from File

Source

[src/file/read-lines](#)

The `getline()` function reads an entire line from a stream, dynamically allocating or reallocating the buffer as needed. It handles lines of any length and includes the newline character in the result. The caller must free the buffer when done. This is the preferred way to read lines in C as it avoids buffer overflow vulnerabilities present in `gets()` and the complexity of `fgets()` with unknown line lengths.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *f = fopen(argv[1], "r");
    if (!f) return 1;

    char *line = NULL;
    size_t len = 0;
    while (getline(&line, &len, f) != -1)
        printf("%s", line);

    free(line);
    fclose(f);
}
```

Read Entire File into Memory

Source

src/file/read-all

For small to medium files, reading the entire contents into memory simplifies processing. This pattern uses `fseek()` and `ftell()` to determine the file size, allocates a buffer, then reads everything with a single `fread()` call. Be cautious with large files as this approach can exhaust available memory.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *f = fopen(argv[1], "r");
    if (!f) return 1;

    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    rewind(f);

    char *buf = malloc(size + 1);
    fread(buf, 1, size, f);
    buf[size] = '\0';

    printf("%s", buf);

    free(buf);
    fclose(f);
}
```

Check File Types

Source

src/file/file-type

The `stat()` function returns file metadata including the file type encoded in the `st_mode` field. Unix supports several file types: regular files, directories, symbolic links, block devices, character devices, FIFOs (named pipes), and sockets. The `S_IFMT` mask extracts the type bits, and macros like `S_ISREG()` and `S_ISDIR()` provide convenient type checking.

```
#include <stdio.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    struct stat st;
    if (stat(argv[1], &st) < 0) return 1;

    switch (st.st_mode & S_IFMT) {
        case S_IFREG: printf("Regular file\n"); break;
        case S_IFDIR: printf("Directory\n"); break;
        case S_IFLNK: printf("Symbolic link\n"); break;
        case S_IFBLK: printf("Block device\n"); break;
        case S_IFCHR: printf("Character device\n"); break;
        case S_IFIFO: printf("FIFO/pipe\n"); break;
        case S_IFSOCK: printf("Socket\n"); break;
        default:      printf("Unknown\n");
    }
}
```

(continues on next page)

(continued from previous page)

```
}
}
```

```
$ ./file-type /etc/hosts
Regular file
$ ./file-type /usr
Directory
$ ./file-type /dev/null
Character device
```

Directory Tree Walk

Source

[src/file/tree-walk](#)

The `nftw()` function (new file tree walk) recursively traverses a directory tree, calling a user-provided callback for each entry. It handles the complexity of opening directories, reading entries, and managing recursion depth. The `FTW_DEPTH` flag processes directory contents before the directory itself (post-order), useful for operations like recursive deletion. `FTW_PHYS` prevents following symbolic links.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <ftw.h>

int callback(const char *path, const struct stat *sb,
             int type, struct FTW *ftwbuf) {
    printf("%s\n", path);
    return 0;
}

int main(int argc, char *argv[]) {
    nftw(argv[1], callback, 64, FTW_DEPTH | FTW_PHYS);
}
```

```
$ ./tree-walk .
./main.c
./a.out
.
```

Pipe to Shell Command with `popen`

Source

[src/file/popen-cmd](#)

The `popen()` function creates a pipe to a shell command, returning a `FILE*` stream for reading the command's output or writing to its input. This simplifies executing external commands and processing their results without manually managing `fork()`, `exec()`, and `pipe()`. Always use `pclose()` to close the stream and retrieve the command's exit status.

```
#include <stdio.h>
#include <sys/wait.h>
```

(continues on next page)

(continued from previous page)

```
int main(void) {
    FILE *fp = popen("ls -la", "r");
    if (!fp) return 1;

    char buf[256];
    while (fgets(buf, sizeof(buf), fp))
        printf("%s", buf);

    int status = pclose(fp);
    printf("Exit status: %d\n", WEXITSTATUS(status));
}
```

```
$ ./popen-cmd
total 32
drwxr-xr-x  3 user user 4096 Jan  6 10:00 .
-rwxr-xr-x  1 user user 8192 Jan  6 10:00 popen-cmd
...
Exit status: 0
```

Named Pipe (FIFO)

Source

src/file/fifo

Named pipes (FIFOs) are similar to anonymous pipes but exist as filesystem entries, allowing communication between unrelated processes. Create a FIFO with `mkfifo()` or the `mkfifo` command. One process opens it for writing and another for reading. The open blocks until both ends are connected. FIFOs are useful for simple IPC without the complexity of sockets.

```
// writer.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

int main(void) {
    mkfifo("/tmp/myfifo", 0666);
    int fd = open("/tmp/myfifo", O_WRONLY);
    write(fd, "Hello FIFO", 11);
    close(fd);
}
```

```
// reader.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd = open("/tmp/myfifo", O_RDONLY);
    char buf[128];
    read(fd, buf, sizeof(buf));
    printf("Received: %s\n", buf);
}
```

(continues on next page)

(continued from previous page)

```
close(fd);  
unlink("/tmp/myfifo");  
}
```

```
# Terminal 1  
$ ./writer  
  
# Terminal 2  
$ ./reader  
Received: Hello FIFO
```

1.4.3 Socket

Table of Contents

- *Socket*
 - *Introduction*
 - *DNS Lookup with gethostbyname*
 - *Byte Order Conversion*
 - *Basic TCP Server*
 - *Basic UDP Server*
 - *I/O Multiplexing with select*
 - *I/O Multiplexing with poll*
 - *Multithreaded TCP Server*
 - *I/O Multiplexing with epoll/kqueue*
 - *Unix Domain Socket*
 - *Modern DNS with getaddrinfo*
 - *IP Address Conversion*
 - *Pipe Communication*
 - *Bidirectional IPC with socketpair*

Introduction

Socket programming is the foundation of network communication in Unix-like operating systems. The Berkeley sockets API, introduced in BSD 4.2 (1983), has become the de facto standard for network programming across all major platforms including Linux, macOS, Windows, and embedded systems.

At its core, a socket is an endpoint for communication—a file descriptor that represents a connection to another process, either on the same machine or across a network. The API provides a unified interface for multiple protocols (TCP, UDP, Unix domain sockets) and address families (IPv4, IPv6).

DNS Lookup with gethostbyname

Source

src/socket/gethostbyname

The `gethostbyname` function resolves a hostname to one or more IP addresses. While this function is deprecated in favor of the more modern `getaddrinfo`, it remains widely used in legacy codebases and is simpler for basic IPv4-only lookups. The function returns a pointer to a `hostent` structure containing the canonical hostname, alias names, address type, address length, and a null-terminated list of network addresses. Note that `gethostbyname` is not thread-safe; use `getaddrinfo` for concurrent applications.

```
#include <stdio.h>
#include <netdb.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    struct hostent *h = gethostbyname(argv[1]);
    if (!h) return 1;

    printf("Host: %s\n", h->h_name);
    for (struct in_addr **p = (struct in_addr **)h->h_addr_list; *p; p++)
        printf("IP: %s\n", inet_ntoa(**p));
}
```

```
$ ./gethostbyname www.google.com
Host: www.google.com
IP: 142.250.80.100
```

Byte Order Conversion

Source

src/socket/byteorder

Network protocols universally use big-endian (network) byte order for transmitting multi-byte integers, while most modern processors (x86, ARM) use little-endian (host) byte order. The `htons` (host-to-network-short) and `htonl` (host-to-network-long) functions convert 16-bit and 32-bit values from host to network byte order respectively. The inverse functions `ntohs` and `ntohl` convert from network to host byte order. These conversions are essential for writing portable network code that works correctly on both big-endian and little-endian machines. On big-endian systems, these functions are no-ops.

```
#include <stdio.h>
#include <arpa/inet.h>

int main(void) {
    uint16_t port = 8080;
    uint32_t addr = 0x7f000001; // 127.0.0.1

    printf("host port: 0x%x -> network: 0x%x\n", port, htons(port));
    printf("host addr: 0x%x -> network: 0x%x\n", addr, htonl(addr));
}
```

```
$ ./byteorder
host port: 0x1f90 -> network: 0x901f
host addr: 0x7f000001 -> network: 0x1000007f
```

Basic TCP Server

Source

`src/socket/tcp-server`

A TCP server follows the classic socket-bind-listen-accept pattern. First, create a socket with `socket()`, specifying `AF_INET` for IPv4 and `SOCK_STREAM` for TCP. Setting `SO_REUSEADDR` allows the server to restart immediately without waiting for the `TIME_WAIT` state to expire. The `bind()` call associates the socket with a local address and port, while `listen()` marks it as a passive socket ready to accept connections. The `accept()` call blocks until a client connects, returning a new socket descriptor for that specific connection. This simple echo server receives data and sends it back to the client before closing the connection.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(5566),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 10);

    for (;;) {
        int c = accept(s, NULL, NULL);
        char buf[1024] = {0};
        recv(c, buf, sizeof(buf) - 1, 0);
        send(c, buf, strlen(buf), 0);
        close(c);
    }
}
```

```
$ ./tcp-server &
$ echo "Hello" | nc localhost 5566
Hello
```

Basic UDP Server

Source

`src/socket/udp-server`

UDP (User Datagram Protocol) is a connectionless protocol, meaning there is no handshake or persistent connection between client and server. Instead of `accept()`, `recv()`, and `send()`, UDP servers use `recvfrom()` and `sendto()` which include the remote address as a parameter. Each datagram is independent and contains the sender's address, allowing the server to respond without maintaining any connection state. This makes UDP ideal for applications where low latency is more important than guaranteed delivery, such as DNS queries, streaming media, or online gaming. The trade-off is that UDP provides no built-in reliability, ordering, or flow control.

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main(void) {
    int s = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(5566),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(s, (struct sockaddr *)&addr, sizeof(addr));

    for (;;) {
        char buf[1024] = {0};
        struct sockaddr_in client;
        socklen_t len = sizeof(client);
        ssize_t n = recvfrom(s, buf, sizeof(buf), 0,
                             (struct sockaddr *)&client, &len);
        sendto(s, buf, n, 0, (struct sockaddr *)&client, len);
    }
}

```

```

$ ./udp-server &
$ echo "Hello" | nc -u localhost 5566
Hello

```

I/O Multiplexing with select

Source

[src/socket/select-server](#)

The `select` system call enables a single-threaded server to monitor multiple file descriptors simultaneously, waiting until one or more become ready for I/O operations. This is the oldest and most portable I/O multiplexing mechanism, available on virtually all Unix-like systems and Windows. The function takes three sets of file descriptors (read, write, exception) implemented as bitmasks, and blocks until at least one descriptor is ready or a timeout occurs. While `select` has limitations—notably the `FD_SETSIZE` limit (typically 1024) and $O(n)$ scanning overhead—it remains useful for applications with a moderate number of connections and where portability is paramount.

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>

int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));
}

```

(continues on next page)

(continued from previous page)

```

struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port = htons(5566),
    .sin_addr.s_addr = INADDR_ANY
};
bind(s, (struct sockaddr *)&addr, sizeof(addr));
listen(s, 10);

fd_set master, readfds;
FD_ZERO(&master);
FD_SET(s, &master);

for (;;) {
    readfds = master;
    select(FD_SETSIZE, &readfds, NULL, NULL, NULL);

    for (int i = 0; i < FD_SETSIZE; i++) {
        if (!FD_ISSET(i, &readfds)) continue;
        if (i == s) {
            int c = accept(s, NULL, NULL);
            FD_SET(c, &master);
        } else {
            char buf[1024] = {0};
            ssize_t n = read(i, buf, sizeof(buf) - 1);
            if (n <= 0) { close(i); FD_CLR(i, &master); }
            else write(i, buf, n);
        }
    }
}
}

```

```

# Terminal 1
$ ./select-server &

```

```

# Terminal 2
$ nc localhost 5566
Hello
Hello

```

```

# Terminal 3 (concurrent)
$ nc localhost 5566
World
World

```

I/O Multiplexing with poll

Source

src/socket/poll-server

The `poll` system call addresses some limitations of `select` by using an array of `pollfd` structures instead of fixed-size bitmasks. This removes the `FD_SETSIZE` limitation and allows monitoring any number of file descriptors (limited only by system resources). Each `pollfd` structure contains the file descriptor, requested events (`POLLIN` for readable, `POLLOUT` for writable), and returned events filled in by the kernel. While `poll` still requires $O(n)$ scanning of all

descriptors, it provides a cleaner interface and better scalability than `select` for applications with many connections. It is available on all POSIX-compliant systems.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <poll.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define MAX_FDS 1024

int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(5566),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 10);

    struct pollfd fds[MAX_FDS];
    fds[0].fd = s;
    fds[0].events = POLLIN;
    int nfds = 1;

    for (;;) {
        poll(fds, nfds, -1);
        for (int i = 0; i < nfds; i++) {
            if (!(fds[i].revents & POLLIN)) continue;
            if (fds[i].fd == s) {
                fds[nfds].fd = accept(s, NULL, NULL);
                fds[nfds++].events = POLLIN;
            } else {
                char buf[1024] = {0};
                ssize_t n = read(fds[i].fd, buf, sizeof(buf) - 1);
                if (n <= 0) { close(fds[i].fd); fds[i] = fds[--nfds]; }
                else write(fds[i].fd, buf, n);
            }
        }
    }
}
```

```
$ ./poll-server &
$ nc localhost 5566
Hello poll
Hello poll
```

Multithreaded TCP Server

Source

src/socket/pthread-server

For CPU-bound workloads or when simpler programming models are preferred, spawning a dedicated thread for each client connection is a straightforward approach. The main thread continuously accepts new connections and creates a detached thread to handle each one independently. This model allows true parallel execution on multi-core systems and simplifies the code since each thread can use blocking I/O without affecting other clients. However, this approach has scalability limits: each thread consumes memory for its stack (typically 1-8 MB), and context switching overhead increases with many concurrent connections. Thread-per-connection works well for servers with moderate concurrency (hundreds of clients) or long-lived connections.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>

void *handle(void *arg) {
    int c = *(int *)arg;
    char buf[1024];
    ssize_t n;
    while ((n = recv(c, buf, sizeof(buf), 0)) > 0)
        send(c, buf, n, 0);
    close(c);
    return NULL;
}

int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(5566),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 10);

    for (;;) {
        static int c;
        c = accept(s, NULL, NULL);
        pthread_t t;
        pthread_create(&t, NULL, handle, &c);
        pthread_detach(t);
    }
}
```

```
$ ./pthread-server &
$ nc localhost 5566
```

(continues on next page)

(continued from previous page)

```
Hello
Hello
```

I/O Multiplexing with epoll/kqueue

Source

src/socket/epoll-kqueue-server

For high-performance servers handling thousands of concurrent connections, Linux provides `epoll` and BSD/macOS provides `kqueue`. Unlike `select` and `poll` which require scanning all file descriptors on each call, these mechanisms use an event-driven model with $O(1)$ complexity for adding/removing descriptors and returning only the ready ones. With `epoll`, you create an `epoll` instance with `epoll_create1()`, register interest in file descriptors using `epoll_ctl()`, and wait for events with `epoll_wait()`. Similarly, `kqueue` uses `kqueue()` to create the queue and `kevent()` for both registration and waiting. This example uses preprocessor directives to compile on both Linux (`epoll`) and macOS/BSD (`kqueue`), demonstrating portable high-performance I/O multiplexing.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#ifdef __linux__
#include <sys/epoll.h>
#define MAX_EVENTS 64
#elif defined(__APPLE__) || defined(__FreeBSD__)
#include <sys/event.h>
#define MAX_EVENTS 64
#endif

int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(5566),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 10);

#ifdef __linux__
    int eq = epoll_create1(0);
    struct epoll_event ev = {.events = EPOLLIN, .data.fd = s};
    epoll_ctl(eq, EPOLL_CTL_ADD, s, &ev);
    struct epoll_event events[MAX_EVENTS];

    for (;;) {
        int n = epoll_wait(eq, events, MAX_EVENTS, -1);
        for (int i = 0; i < n; i++) {
```

(continues on next page)

(continued from previous page)

```

    int fd = events[i].data.fd;
    if (fd == s) {
        int c = accept(s, NULL, NULL);
        ev.events = EPOLLIN;
        ev.data.fd = c;
        epoll_ctl(eq, EPOLL_CTL_ADD, c, &ev);
    } else {
        char buf[1024] = {0};
        ssize_t len = read(fd, buf, sizeof(buf) - 1);
        if (len <= 0) { close(fd); }
        else write(fd, buf, len);
    }
}
}
}
#elif defined(__APPLE__) || defined(__FreeBSD__)
int kq = kqueue();
struct kevent ev;
EV_SET(&ev, s, EVFILT_READ, EV_ADD, 0, 0, NULL);
kevent(kq, &ev, 1, NULL, 0, NULL);
struct kevent events[MAX_EVENTS];

for (;;) {
    int n = kevent(kq, NULL, 0, events, MAX_EVENTS, NULL);
    for (int i = 0; i < n; i++) {
        int fd = (int)events[i].ident;
        if (fd == s) {
            int c = accept(s, NULL, NULL);
            EV_SET(&ev, c, EVFILT_READ, EV_ADD, 0, 0, NULL);
            kevent(kq, &ev, 1, NULL, 0, NULL);
        } else {
            char buf[1024] = {0};
            ssize_t len = read(fd, buf, sizeof(buf) - 1);
            if (len <= 0) { close(fd); }
            else write(fd, buf, len);
        }
    }
}
}
#endif
}

```

```

# Linux
$ ./epoll-kqueue-server &
$ nc localhost 5566
Hello epoll
Hello epoll

# macOS/BSD
$ ./epoll-kqueue-server &
$ nc localhost 5566
Hello kqueue
Hello kqueue

```


Unix Domain Socket

Source

`src/socket/unix-socket`

Unix domain sockets (also called local sockets or IPC sockets) provide inter-process communication between processes running on the same host. They use the filesystem namespace for addressing—the socket is represented as a special file that must be removed before binding. Unix domain sockets are significantly faster than TCP/IP loopback (localhost) connections because they bypass the entire network stack: no routing, no checksums, no packet fragmentation. They also support passing file descriptors and credentials between processes, enabling powerful IPC patterns. Use `AF_UNIX` instead of `AF_INET` and `sockaddr_un` instead of `sockaddr_in` for addressing.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

int main(void) {
    unlink("/tmp/echo.sock");
    int s = socket(AF_UNIX, SOCK_STREAM, 0);

    struct sockaddr_un addr = {.sun_family = AF_UNIX};
    strncpy(addr.sun_path, "/tmp/echo.sock", sizeof(addr.sun_path) - 1);
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 10);

    for (;;) {
        int c = accept(s, NULL, NULL);
        char buf[1024] = {0};
        ssize_t n = recv(c, buf, sizeof(buf) - 1, 0);
        if (n > 0) send(c, buf, n, 0);
        close(c);
    }
}
```

```
$ ./unix-socket &
$ nc -U /tmp/echo.sock
Hello Unix
Hello Unix
```

Modern DNS with getaddrinfo

Source

`src/socket/getaddrinfo`

The `getaddrinfo` function is the modern, protocol-independent replacement for `gethostbyname`. It handles both IPv4 and IPv6 addresses transparently, resolves service names to port numbers, and is fully thread-safe. The function takes hints specifying the desired address family (`AF_INET`, `AF_INET6`, or `AF_UNSPEC` for both), socket type, and protocol. It returns a linked list of `addrinfo` structures, each containing a ready-to-use `sockaddr` that can be passed directly to `socket()`, `bind()`, or `connect()`. Always call `freeaddrinfo()` to release the allocated memory. This function is the recommended way to write network code that works seamlessly with both IPv4 and IPv6.

```
#include <stdio.h>
#include <string.h>
```

(continues on next page)

(continued from previous page)

```
#include <netdb.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    struct addrinfo hints = {.ai_family = AF_UNSPEC, .ai_socktype = SOCK_STREAM};
    struct addrinfo *res;

    if (getaddrinfo(argv[1], NULL, &hints, &res) != 0) return 1;

    for (struct addrinfo *p = res; p; p = p->ai_next) {
        char ip[INET6_ADDRSTRLEN];
        void *addr = p->ai_family == AF_INET
            ? (void *)&((struct sockaddr_in *)p->ai_addr)->sin_addr
            : (void *)&((struct sockaddr_in6 *)p->ai_addr)->sin6_addr;
        inet_ntop(p->ai_family, addr, ip, sizeof(ip));
        printf("%s: %s\n", p->ai_family == AF_INET ? "IPv4" : "IPv6", ip);
    }
    freeaddrinfo(res);
}
```

```
$ ./getaddrinfo www.google.com
IPv6: 2607:f8b0:4004:800::2004
IPv4: 142.250.80.100
```

IP Address Conversion

Source

[src/socket/inet-conv](#)

The `inet_pton` (presentation to network) and `inet_ntop` (network to presentation) functions convert IP addresses between human-readable text strings and binary network format. Unlike the older `inet_aton` and `inet_ntoa` functions, these support both IPv4 and IPv6 addresses through the address family parameter. The `inet_pton` function returns 1 on success, 0 if the input is not a valid address in the specified family, or -1 on error. The `inet_ntop` function returns a pointer to the destination buffer on success or NULL on error. These functions are essential for parsing IP addresses from configuration files or user input and for displaying addresses in log messages or user interfaces.

```
#include <stdio.h>
#include <arpa/inet.h>

int main(void) {
    // IPv4
    struct in_addr addr4;
    inet_pton(AF_INET, "192.168.1.1", &addr4);
    char str4[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &addr4, str4, sizeof(str4));
    printf("IPv4: %s (0x%x)\n", str4, ntohl(addr4.s_addr));

    // IPv6
    struct in6_addr addr6;
    inet_pton(AF_INET6, "::1", &addr6);
    char str6[INET6_ADDRSTRLEN];
    inet_ntop(AF_INET6, &addr6, str6, sizeof(str6));
}
```

(continues on next page)

(continued from previous page)

```
printf("IPv6: %s\n", str6);
}
```

```
$ ./inet-conv
IPv4: 192.168.1.1 (0xc0a80101)
IPv6: ::1
```

Pipe Communication

Source

[src/socket/pipe](#)

Pipes provide unidirectional inter-process communication. The `pipe()` system call creates a pair of file descriptors: `fd[0]` for reading and `fd[1]` for writing. Data written to the write end can be read from the read end in FIFO order. Pipes are commonly used after `fork()` to establish communication between parent and child processes. The typical pattern is for one process to close the read end and write, while the other closes the write end and reads.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int fd[2];
    pipe(fd);

    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);
        char buf[128];
        read(fd[0], buf, sizeof(buf));
        printf("Child received: %s\n", buf);
        close(fd[0]);
    } else {
        close(fd[0]);
        const char *msg = "Hello from parent";
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);
    }
}
```

```
$ ./pipe
Child received: Hello from parent
```

Bidirectional IPC with socketpair

Source

[src/socket/socketpair](#)

Unlike `pipe()` which is unidirectional, `socketpair()` creates a pair of connected Unix domain sockets that support bidirectional communication. Each end can both read and write, making it ideal for two-way IPC between related processes. This is simpler than creating two pipes for full-duplex communication and provides socket semantics including the ability to pass file descriptors between processes using `SCM_RIGHTS`.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    socketpair(AF_UNIX, SOCK_STREAM, 0, fd);

    pid_t pid = fork();
    if (pid == 0) {
        close(fd[0]);
        char buf[128];
        read(fd[1], buf, sizeof(buf));
        printf("Child got: %s\n", buf);
        write(fd[1], "Hi parent", 10);
        close(fd[1]);
    } else {
        close(fd[1]);
        write(fd[0], "Hi child", 9);
        char buf[128];
        read(fd[0], buf, sizeof(buf));
        printf("Parent got: %s\n", buf);
        close(fd[0]);
        wait(NULL);
    }
}
```

```
$ ./socketpair
Child got: Hi child
Parent got: Hi parent
```

1.4.4 Signal

Table of Contents

- *Signal*
 - *Introduction*
 - *Print Signal Names*
 - *Basic Signal Handler*
 - *Signal Handling with sigaction*
 - *Pthread Signal Handling*
 - *Monitoring Child Processes*
 - *Blocking and Unblocking Signals*

Introduction

Signals are software interrupts that provide a mechanism for handling asynchronous events in Unix-like operating systems. When a signal is delivered to a process, the operating system interrupts the process's normal execution flow and invokes a signal handler function.

Signals can originate from various sources: the kernel (e.g., SIGSEGV for memory access violations), other processes (via `kill()`), terminal input (e.g., Ctrl+C generates SIGINT), or the process itself (via `raise()` or `abort()`). Each signal has a default action—typically terminating the process, ignoring the signal, or stopping/continuing execution—but most signals can be caught and handled by user-defined functions.

Understanding signal handling is essential for writing robust Unix applications, particularly servers and daemons that must handle termination requests gracefully, manage child processes, and respond to timer events.

Print Signal Names

Source

`src/signal/print-signals`

The `sys_siglist` array (or `strsignal()` function) provides human-readable names for signal numbers. This is useful for logging and debugging signal-related issues. Note that `sys_siglist` is deprecated on some systems; prefer `strsignal()` for portable code.

```
#include <stdio.h>
#include <signal.h>

static int signals[] = {
    SIGABRT, SIGALRM, SIGBUS,  SIGCHLD, SIGCONT, SIGFPE,
    SIGHUP,  SIGILL,  SIGINT,  SIGIO,   SIGKILL, SIGPIPE,
    SIGPROF, SIGQUIT, SIGSEGV, SIGSYS,  SIGTERM, SIGTRAP,
    SIGTSTP, SIGTTIN, SIGTTOU, SIGURG,  SIGVTALRM, SIGUSR1,
    SIGUSR2, SIGXCPU, SIGXFSZ
};

int main(void) {
    for (size_t i = 0; i < sizeof(signals)/sizeof(signals[0]); i++)
        printf("Signal[%2d]: %s\n", signals[i], sys_siglist[signals[i]]);
}
```

```
$ ./print-signals
Signal[ 6]: Abort trap
Signal[14]: Alarm clock
Signal[10]: Bus error
Signal[20]: Child exited
...
```

Basic Signal Handler

Source

`src/signal/basic-handler`

The `signal()` function registers a handler for a specific signal. When that signal is delivered, the kernel interrupts the process and calls the handler function with the signal number as its argument. Use `SIG_IGN` to ignore a signal or `SIG_DFL` to restore the default behavior. Note that `signal()` has portability issues across Unix variants; `sigaction()` is preferred for new code.

```
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>

void handler(int signo) {
    printf("[%d] Got signal: %s\n", getpid(), strsignal(signo));
}

int main(void) {
    signal(SIGHUP, handler);
    signal(SIGINT, handler);
    signal(SIGALRM, handler);
    signal(SIGUSR1, SIG_IGN); // ignore SIGUSR1
    while (1) sleep(3);
}
```

```
$ ./basic-handler &
[54652]
$ kill -HUP %1
[54652] Got signal: Hangup
$ kill -INT %1
[54652] Got signal: Interrupt
```

Signal Handling with sigaction

Source

[src/signal/sigaction-handler](#)

The `sigaction()` function provides more control over signal handling than `signal()`. It allows specifying a signal mask to block other signals during handler execution, flags to modify behavior (e.g., `SA_RESTART` to restart interrupted system calls), and access to additional signal information via `SA_SIGINFO`. Always prefer `sigaction()` over `signal()` for reliable, portable signal handling.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int signo) {
    printf("Got signal: %s\n", sys_siglist[signo]);
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, NULL, &sa); // get current
    if (sa.sa_handler != SIG_IGN)
        sigaction(SIGINT, &sa, NULL);

    sigaction(SIGHUP, NULL, &sa);
}
```

(continues on next page)

(continued from previous page)

```

if (sa.sa_handler != SIG_IGN)
    sigaction(SIGHUP, &sa, NULL);

printf("PID: %d\n", getpid());
while (1) sleep(3);
}

```

```

$ ./sigaction-handler &
PID: 57140
$ kill -HUP 57140
Got signal: Hangup
$ kill -INT 57140
Got signal: Interrupt

```

Pthread Signal Handling

Source

[src/signal/pthread-signal](#)

In multithreaded programs, signals can be delivered to any thread that hasn't blocked them. A common pattern is to block signals in all threads except a dedicated signal-handling thread that uses `sigwait()` to synchronously receive signals. This avoids the complexity and restrictions of asynchronous signal handlers in threaded code.

```

#include <stdio.h>
#include <pthread.h>
#include <signal.h>

void *sig_thread(void *arg) {
    sigset_t *set = (sigset_t *)arg;
    int signo;
    for (;;) {
        sigwait(set, &signo);
        printf("Got signal[%d]: %s\n", signo, sys_siglist[signo]);
    }
}

int main(void) {
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGQUIT);
    sigaddset(&set, SIGUSR1);
    pthread_sigmask(SIG_BLOCK, &set, NULL);

    pthread_t t;
    pthread_create(&t, NULL, sig_thread, &set);
    pause();
}

```

```

$ ./pthread-signal &
[1] 21258
$ kill -USR1 %1
Got signal[10]: User defined signal 1

```

(continues on next page)

(continued from previous page)

```
$ kill -QUIT %1
Got signal[3]: Quit
```

Monitoring Child Processes

Source

src/signal/child-monitor

When a child process terminates, the kernel sends SIGCHLD to the parent. By installing a handler for SIGCHLD, the parent can be notified immediately when children exit, rather than blocking in `wait()`. This is essential for servers that spawn worker processes and need to track their lifecycle without blocking the main event loop.

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handler(int signo) {
    printf("[%d] Got signal[%d]: %s\n", getpid(), signo, sys_siglist[signo]);
}

int main(void) {
    signal(SIGCHLD, handler);
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child[%d]\n", getpid());
        sleep(3);
    } else {
        printf("Parent[%d]\n", getpid());
        pause();
    }
}
```

```
$ ./child-monitor
Parent[59113]
Child[59114]
[59113] Got signal[20]: Child exited
```

Blocking and Unblocking Signals

Source

src/signal/signal-mask

The `sigprocmask()` function controls which signals are blocked (deferred) for the calling thread. Blocked signals remain pending until unblocked, at which point they are delivered. This is useful for protecting critical sections from interruption or for temporarily deferring signal handling. Combined with `siglongjmp()`, it enables complex control flow in signal handlers.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <setjmp.h>
```

(continues on next page)

(continued from previous page)

```
static sigjmp_buf jmpbuf;

void handler(int signo) {
    printf("Got signal[%d]: %s\n", signo, sys_siglist[signo]);
    if (signo == SIGUSR1)
        siglongjmp(jmpbuf, 1);
}

int main(void) {
    sigset_t mask;
    sigemptyset(&mask);
    sigaddset(&mask, SIGHUP);

    signal(SIGHUP, handler);
    signal(SIGALRM, handler);
    signal(SIGUSR1, handler);

    if (sigsetjmp(jmpbuf, 1))
        sigprocmask(SIG_UNBLOCK, &mask, NULL); // unblock after jump
    else
        sigprocmask(SIG_BLOCK, &mask, NULL);   // block SIGHUP

    while (1) sleep(3);
}
```

```
$ ./signal-mask &
$ kill -HUP %1      # blocked, no output
$ kill -ALRM %1
Got signal[14]: Alarm clock
$ kill -USR1 %1     # triggers jump, unblocks SIGHUP
Got signal[10]: User defined signal 1
Got signal[1]: Hangup
```

1.4.5 Thread

Table of Contents

- *Thread*
 - *Introduction*
 - *Creating Threads*
 - *Mutex Synchronization*
 - *Condition Variables*
 - *Read-Write Locks*
 - *Thread-Local Storage*
 - *C++ Futures and Async*
 - *Launch Policies*

- *Promise and Future*
- *Creating a Unix Daemon*
- *Using the `daemon()` Function*

Introduction

POSIX threads (pthreads) provide a standardized API for creating and managing threads in C programs. Unlike processes, threads share the same address space, making inter-thread communication efficient through shared variables. However, this shared memory model requires careful synchronization to prevent race conditions, where multiple threads access shared data simultaneously and produce unpredictable results.

The pthread library offers several synchronization primitives: mutexes provide mutual exclusion to protect critical sections, condition variables enable threads to wait for specific conditions, and read-write locks optimize scenarios with frequent reads and infrequent writes. Understanding these primitives is essential for building correct concurrent programs that avoid common pitfalls like deadlocks and data races.

Creating Threads

Source

`src/thread/thread-basic`

The `pthread_create()` function spawns a new thread that begins execution at the specified start routine. The function takes four arguments: a pointer to store the thread identifier, optional thread attributes (NULL for defaults), the function to execute, and an argument to pass to that function. The thread runs concurrently with the calling thread until it returns or calls `pthread_exit()`.

Use `pthread_join()` to wait for a thread to terminate and optionally retrieve its return value. Failing to join or detach threads causes resource leaks. Always check return values from pthread functions, as thread creation can fail due to system resource limits or invalid attributes.

```
#include <stdio.h>
#include <pthread.h>

void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d running\n", id);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("All threads done\n");
}
```

```
$ gcc -pthread -o thread-basic main.c && ./thread-basic
Thread 1 running
Thread 2 running
All threads done
```

Mutex Synchronization

Source

[src/thread/mutex](#)

A mutex (mutual exclusion lock) ensures that only one thread can execute a critical section at a time. When a thread calls `pthread_mutex_lock()`, it either acquires the lock immediately or blocks until the lock becomes available. The thread must call `pthread_mutex_unlock()` to release the lock, allowing other waiting threads to proceed.

Without mutex protection, concurrent access to shared variables causes race conditions. For example, incrementing a counter involves reading the current value, adding one, and writing back—three operations that can interleave unpredictably across threads. Mutexes serialize these operations, ensuring correct results at the cost of some performance overhead.

Initialize static mutexes with `PTHREAD_MUTEX_INITIALIZER`. For dynamically allocated mutexes, use `pthread_mutex_init()` and `pthread_mutex_destroy()`. Always pair lock and unlock calls, and consider using RAII patterns in C++ to prevent forgetting to unlock.

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Counter: %d\n", counter); // always 200000
}
```

```
$ gcc -pthread -o mutex main.c && ./mutex
Counter: 200000
```

Condition Variables

Source

`src/thread/condvar`

Condition variables allow threads to block until a particular condition becomes true, enabling efficient producer-consumer patterns and other synchronization scenarios. Unlike busy-waiting (repeatedly checking a condition in a loop), condition variables put the thread to sleep until another thread signals that the condition may have changed.

The `pthread_cond_wait()` function atomically releases the associated mutex and blocks the calling thread. When the thread is awakened by a signal, it automatically reacquires the mutex before returning. This atomic release-and-wait prevents race conditions between checking the condition and going to sleep.

Always use a while loop to recheck the condition after `pthread_cond_wait()` returns, because spurious wakeups can occur and multiple threads may be competing for the same condition. Use `pthread_cond_signal()` to wake one waiting thread or `pthread_cond_broadcast()` to wake all waiters when the condition affects multiple threads.

```
#include <stdio.h>
#include <pthread.h>

int ready = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    pthread_mutex_lock(&lock);
    ready = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *consumer(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready)
        pthread_cond_wait(&cond, &lock);
    printf("Consumer: data ready\n");
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t prod, cons;
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
}
```

```
$ gcc -pthread -o condvar main.c && ./condvar
Consumer: data ready
```

Read-Write Locks

Source

`src/thread/rwlock`

Read-write locks optimize access patterns where reads are frequent and writes are rare. Multiple threads can hold the read lock simultaneously, allowing concurrent read access to shared data. However, the write lock is exclusive—only one thread can hold it, and no readers are allowed while a writer has the lock.

Use `pthread_rwlock_rdlock()` for read access and `pthread_rwlock_wrlock()` for write access. This distinction improves throughput compared to a regular mutex when most operations are reads, as readers don't block each other. However, read-write locks have higher overhead than mutexes, so they're only beneficial when the read-to-write ratio is high and critical sections are long enough to amortize the locking cost.

Be aware of writer starvation: if readers continuously acquire the lock, writers may wait indefinitely. Some implementations provide fairness policies to prevent this, but behavior varies across platforms.

```
#include <stdio.h>
#include <pthread.h>

int data = 0;
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;

void *reader(void *arg) {
    pthread_rwlock_rdlock(&rwlock);
    printf("Read: %d\n", data);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

void *writer(void *arg) {
    pthread_rwlock_wrlock(&rwlock);
    data = 42;
    printf("Wrote: %d\n", data);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

int main(void) {
    pthread_t r1, r2, w;
    pthread_create(&w, NULL, writer, NULL);
    pthread_create(&r1, NULL, reader, NULL);
    pthread_create(&r2, NULL, reader, NULL);
    pthread_join(w, NULL);
    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
}
```

```
$ gcc -pthread -o rwlock main.c && ./rwlock
Wrote: 42
Read: 42
Read: 42
```

Thread-Local Storage

Source

[src/thread/thread-local](#)

Thread-local storage (TLS) provides each thread with its own instance of a variable, eliminating the need for synchronization when threads need private copies of data. This is useful for per-thread caches, error codes, or context that shouldn't be shared across threads.

The `__thread` keyword (GCC/Clang extension) or `_Thread_local` (C11 standard) declares thread-local variables with static storage duration. Each thread gets its own copy, initialized independently. For more complex scenarios requiring dynamic allocation or destructor callbacks, use `pthread_key_create()` with `pthread_setspecific()` and `pthread_getspecific()`.

Thread-local storage is more efficient than passing context through function arguments when the data is needed deep in the call stack, and it avoids the overhead of mutex locking for thread-private data.

```
#include <stdio.h>
#include <pthread.h>

__thread int tls_var = 0;

void *worker(void *arg) {
    tls_var = *(int *)arg;
    printf("Thread %d: tls_var = %d\n", tls_var, tls_var);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;
    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

```
$ gcc -pthread -o thread-local main.c && ./thread-local
Thread 1: tls_var = 1
Thread 2: tls_var = 2
```

C++ Futures and Async

Source

[src/thread/future-basic](#)

C++11 introduced `std::future` and `std::async` as higher-level abstractions for asynchronous programming. A future represents a value that will be available at some point, allowing you to launch computations and retrieve results later without manually managing threads. The `get()` method blocks until the result is ready, while `wait_for()` allows polling with a timeout to check if the computation has completed.

Unlike raw threads where you must handle synchronization manually, futures automatically propagate exceptions from the async task to the calling thread when you call `get()`. This makes error handling cleaner and prevents silent failures in concurrent code.

```

#include <future>
#include <thread>

int Compute(int x) { return x * x; }

int main() {
    std::future<int> fut = std::async(std::launch::async, Compute, 5);
    int result = fut.get(); // blocks until result ready, returns 25

    // check if ready without blocking
    std::future<int> fut2 = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
        return 42;
    });
    auto status = fut2.wait_for(std::chrono::milliseconds(100));
    if (status == std::future_status::ready) {
        int val = fut2.get(); // 42
    }
}

```

Launch Policies

Source

`src/thread/future-async`

The `std::async` function accepts a launch policy that controls when and how the task executes. `std::launch::async` guarantees the task runs on a new thread immediately, while `std::launch::deferred` delays execution until `get()` or `wait()` is called, running the task on the calling thread. The default policy (`async | deferred`) lets the implementation choose, which can lead to surprising behavior if you expect parallel execution.

For CPU-bound parallel algorithms, use `std::launch::async` explicitly to ensure tasks run concurrently. Deferred execution is useful for lazy evaluation where the result might not be needed, avoiding the overhead of thread creation.

```

#include <future>
#include <numeric>
#include <vector>

int main() {
    // async: runs immediately on new thread
    auto fut1 = std::async(std::launch::async, [] { return 100; });

    // deferred: runs lazily when get() called
    auto fut2 = std::async(std::launch::deferred, [] { return 200; });

    // parallel sum example
    std::vector<int> v(1000);
    std::iota(v.begin(), v.end(), 1); // 1 to 1000

    auto mid = v.begin() + v.size() / 2;
    auto sum1 = std::async(std::launch::async, [&] {
        return std::accumulate(v.begin(), mid, 0);
    });
    auto sum2 = std::async(std::launch::async, [&] {
        return std::accumulate(mid, v.end(), 0);
    });
}

```

(continues on next page)

(continued from previous page)

```
});

int total = sum1.get() + sum2.get(); // 500500
}
```

Promise and Future

Source

src/thread/future-promise

While `std::async` creates both the task and its associated future, `std::promise` gives you explicit control over when and how a future is fulfilled. A promise is a write-end that you use to set a value or exception, while the future is the read-end that consumers use to retrieve the result. This separation is useful when the producer thread is created elsewhere or when you need to fulfill a future from a callback.

Call `set_value()` to provide the result or `set_exception()` to signal an error. The associated future's `get()` will either return the value or rethrow the exception. Each promise can only be fulfilled once; attempting to set a value twice throws `std::future_error`.

```
#include <future>
#include <thread>

int main() {
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();

    std::thread t([&prom] {
        prom.set_value(42); // fulfill the promise
    });

    int result = fut.get(); // blocks until value set, returns 42
    t.join();

    // exception propagation
    std::promise<int> prom2;
    std::future<int> fut2 = prom2.get_future();

    std::thread t2([&prom2] {
        prom2.set_exception(std::make_exception_ptr(std::runtime_error("error")));
    });

    try {
        fut2.get(); // throws std::runtime_error
    } catch (const std::runtime_error& e) {
        // handle error
    }
    t2.join();
}
```


Creating a Unix Daemon

Source

[src/thread/daemon](#)

A daemon is a background process that runs independently of any controlling terminal, typically providing system services like web servers, database engines, or scheduled tasks. Creating a proper daemon requires several steps to fully detach from the parent process and terminal environment.

The traditional daemonization sequence involves: calling `fork()` to create a child process (parent exits, ensuring the child isn't a process group leader), calling `setsid()` to create a new session and become the session leader, optionally forking again to prevent acquiring a controlling terminal, changing the working directory to root to avoid blocking filesystem unmounts, resetting the file creation mask with `umask()`, and closing standard file descriptors to release terminal resources.

Since stdout and stderr are closed, daemons typically use `syslog()` for logging. The syslog facility provides centralized, persistent logging with severity levels and is the standard approach for daemon diagnostics.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <syslog.h>
#include <sys/stat.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) exit(1);
    if (pid > 0) exit(0); // parent exits

    umask(0);
    if (setsid() < 0) exit(1);
    if (chdir("/") < 0) exit(1);

    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);

    while (1) {
        sleep(3);
        syslog(LOG_INFO, "Daemon running");
    }
}
```

```
$ ./daemon
$ ps aux | grep daemon
user 12345 0.0 0.0 daemon
```

Using the `daemon()` Function

Source

[src/thread/daemon-simple](#)

The BSD `daemon()` function provides a convenient wrapper that handles the standard daemonization steps in a single call. The first argument `nochdir` controls whether to change the working directory to root (0 means change, nonzero means stay). The second argument `noclose` controls whether to redirect stdin, stdout, and stderr to `/dev/null` (0 means redirect, nonzero means keep open).

While `daemon()` simplifies the code, it's not part of the POSIX standard and may not be available on all Unix-like systems. For maximum portability across different platforms, implement the daemonization steps manually. Additionally, modern systems often use `systemd` or other init systems that handle daemonization externally, making the traditional double-fork approach less necessary for new services.

```
#include <stdio.h>
#include <unistd.h>
#include <syslog.h>

int main(void) {
    if (daemon(0, 0) < 0) {
        syslog(LOG_ERR, "daemon() failed");
        return 1;
    }

    while (1) {
        sleep(3);
        syslog(LOG_INFO, "Daemon running");
    }
}
```

```
$ ./daemon-simple
$ pgrep daemon-simple
12346
```

1.4.6 Inter-Process Communication

Table of Contents

- *Inter-Process Communication*
 - *Introduction*
 - *Pipes*
 - *FIFOs (Named Pipes)*
 - *POSIX Shared Memory*
 - *POSIX Message Queues*
 - *POSIX Semaphores*
 - *Unix Domain Sockets*
 - *System V IPC (Legacy)*
 - *Comparison Summary*

Introduction

Inter-process communication (IPC) enables processes to exchange data and coordinate their actions. Unix-like systems provide multiple IPC mechanisms, each with different characteristics suited to specific use cases.

The choice of IPC mechanism depends on several factors: whether processes are related (parent-child) or unrelated, the volume of data being exchanged, whether message boundaries matter, and performance requirements. Pipes and

FIFOs provide simple byte streams, shared memory offers the fastest data transfer, message queues preserve message boundaries, and Unix domain sockets provide full-duplex communication with socket semantics.

IPC Mechanism Comparison:

Mechanism	Related?	Persistent?	Bidirectional?	Best For
Pipe	Yes	No	No	Parent-child
FIFO	No	Yes	No	Unrelated procs
Shared Memory	No	Optional	Yes	Large data, fast
Message Queue	No	Yes	Yes	Structured msgs
Semaphore	No	Yes	N/A	Synchronization
Unix Socket	No	Yes	Yes	Full-duplex comm

Pipes

Source

`src/ipc/pipe`

Pipes are the simplest form of IPC, providing a unidirectional byte stream between related processes. The `pipe()` system call creates a pair of file descriptors: one for reading and one for writing. Data written to the write end can be read from the read end, following FIFO (first-in, first-out) order.

Pipes are anonymous—they exist only as long as the processes using them are running and have no filesystem presence. This makes them ideal for parent-child communication where the parent creates the pipe before forking. After the fork, both processes inherit the file descriptors, and each typically closes the end it doesn't need to establish a clear data flow direction.

The kernel buffers pipe data (typically 64KB on Linux), allowing the writer to continue without blocking until the buffer fills. When the buffer is full, the writer blocks until the reader consumes some data. Reading from an empty pipe blocks until data is available or all write ends are closed (returning EOF).

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <cstring>

int main() {
    int pipefd[2]; // [0]=read, [1]=write
    pipe(pipefd);

    if (fork() == 0) {
        // Child: read from pipe
        close(pipefd[1]);
        char buf[100];
        ssize_t n = read(pipefd[0], buf, sizeof(buf) - 1);
        buf[n] = '\0';
        printf("Child received: %s\n", buf);
        close(pipefd[0]);
    } else {
        // Parent: write to pipe
        close(pipefd[0]);
        const char* msg = "Hello from parent!";
        write(pipefd[1], msg, strlen(msg));
        close(pipefd[1]);
    }
}
```

(continues on next page)

(continued from previous page)

```

    wait(NULL);
}
}

```

```

$ ./ipc-pipe
Child received: Hello from parent!

```

FIFOs (Named Pipes)

Source

`src/ipc/fifo`

FIFOs (also called named pipes) extend the pipe concept to unrelated processes by giving the pipe a name in the filesystem. The `mkfifo()` function creates a special file that acts as a rendezvous point—any process that opens this file for reading or writing connects to the same underlying pipe.

Unlike regular pipes, FIFOs persist in the filesystem until explicitly removed with `unlink()`. This allows processes started at different times to communicate. Opening a FIFO for reading blocks until another process opens it for writing (and vice versa), unless `O_NONBLOCK` is specified.

FIFOs maintain the same semantics as pipes: unidirectional data flow, FIFO ordering, and blocking behavior when the buffer is full or empty. For bidirectional communication, you need two FIFOs or should consider Unix domain sockets instead.

```

#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <string.h>

int main() {
    const char* fifo_path = "/tmp/myfifo";
    mkfifo(fifo_path, 0666);

    if (fork() == 0) {
        // Child: write to FIFO
        int fd = open(fifo_path, O_WRONLY);
        const char* msg = "Hello via FIFO!";
        write(fd, msg, strlen(msg));
        close(fd);
    } else {
        // Parent: read from FIFO
        int fd = open(fifo_path, O_RDONLY);
        char buf[100];
        ssize_t n = read(fd, buf, sizeof(buf) - 1);
        buf[n] = '\0';
        printf("Received: %s\n", buf);
        close(fd);
        wait(NULL);
        unlink(fifo_path);
    }
}

```

```
$ ./ipc-fifo
Received: Hello via FIFO!
```

POSIX Shared Memory

Source

src/ipc/posix-shm

Shared memory is the fastest IPC mechanism because data is not copied between processes—both processes access the same physical memory pages. POSIX shared memory uses `shm_open()` to create or open a shared memory object (which appears in `/dev/shm` on Linux), `ftruncate()` to set its size, and `mmap()` to map it into the process's address space.

The key advantage of shared memory is performance: once mapped, accessing shared data is as fast as accessing regular memory. However, this speed comes with responsibility—you must synchronize access using mutexes, semaphores, or atomic operations to prevent data races. The kernel provides no automatic synchronization for shared memory.

Shared memory objects persist until explicitly removed with `shm_unlink()`, even after all processes unmap them. This allows processes to attach and detach at different times. The `MAP_SHARED` flag ensures that modifications are visible to all processes mapping the same object.

```
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <string.h>

int main() {
    const char* shm_name = "/myshm";
    const size_t SIZE = 4096;

    // Create shared memory
    int fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, SIZE);
    void* ptr = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    close(fd);

    if (fork() == 0) {
        // Child: write to shared memory
        sprintf((char*)ptr, "Hello from child process!");
    } else {
        // Parent: wait and read
        wait(NULL);
        printf("Parent read: %s\n", (char*)ptr);
        munmap(ptr, SIZE);
        shm_unlink(shm_name);
    }
}
```

```
$ ./ipc-posix-shm
Parent read: Hello from child process!
```

Shared Memory with Synchronization:

When multiple processes read and write shared memory concurrently, you need synchronization. Process-shared mutexes (initialized with `PTHREAD_PROCESS_SHARED`) can be placed in the shared memory region itself.

```
#include <pthread.h>

struct SharedData {
    pthread_mutex_t mutex;
    int counter;
};

// Initialize (once, by creator)
SharedData* data = (SharedData*)ptr;
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setshared(&attr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&data->mutex, &attr);
data->counter = 0;

// Use (any process)
pthread_mutex_lock(&data->mutex);
data->counter++;
pthread_mutex_unlock(&data->mutex);
```

POSIX Message Queues

Source

`src/ipc/posix-mqueue`

Message queues provide a way to exchange discrete messages between processes. Unlike pipes which provide a byte stream, message queues preserve message boundaries—each `mq_send()` creates a distinct message that is retrieved atomically by a single `mq_receive()`. Messages can also have priorities, with higher-priority messages delivered first.

POSIX message queues are created with `mq_open()`, which takes attributes specifying the maximum number of messages and maximum message size. The queue persists in the kernel until removed with `mq_unlink()`. On Linux, message queues appear as files under `/dev/mqueue`.

Message queues are ideal when you need to send structured data units and want the kernel to handle buffering and ordering. They're commonly used for command/response protocols and work distribution systems. Note that POSIX message queues are not available on macOS—use System V message queues or other mechanisms instead.

```
// Note: POSIX message queues not available on macOS
#include <mqueue.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdio.h>
#include <string.h>

int main() {
    const char* mq_name = "/myqueue";
    struct mq_attr attr = {.mq_maxmsg = 10, .mq_msgsize = 256};

    mqd_t mq = mq_open(mq_name, O_CREAT | O_RDWR, 0666, &attr);

    if (fork() == 0) {
        // Child: send message
```

(continues on next page)

(continued from previous page)

```

const char* msg = "Hello via message queue!";
mq_send(mq, msg, strlen(msg) + 1, 1); // priority = 1
mq_close(mq);
} else {
    // Parent: receive message
    wait(NULL);
    char buf[256];
    unsigned int prio;
    mq_receive(mq, buf, sizeof(buf), &prio);
    printf("Received (prio=%u): %s\n", prio, buf);
    mq_close(mq);
    mq_unlink(mq_name);
}
}

```

Compile with `-lrt` on Linux.

POSIX Semaphores

Source

`src/ipc/posix-semaphore`

Semaphores are synchronization primitives used to coordinate access to shared resources or signal events between processes. A semaphore maintains an integer count: `sem_wait()` decrements the count (blocking if it would go negative), and `sem_post()` increments it (potentially waking a blocked waiter).

POSIX provides two types of semaphores: named semaphores (created with `sem_open()`) for unrelated processes, and unnamed semaphores (initialized with `sem_init()`) typically placed in shared memory. Named semaphores persist in the filesystem (under `/dev/shm` on Linux) until removed with `sem_unlink()`.

Common use cases include mutual exclusion (binary semaphore initialized to 1), signaling between processes (initialized to 0, one process posts, another waits), and resource counting (initialized to the number of available resources).

```

#include <semaphore.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    const char* sem_name = "/mysem";

    // Create named semaphore (initial value = 0)
    sem_t* sem = sem_open(sem_name, O_CREAT, 0666, 0);

    if (fork() == 0) {
        // Child: do work then signal
        printf("Child: doing work...\n");
        sleep(1);
        printf("Child: signaling parent\n");
        sem_post(sem);
        sem_close(sem);
    } else {
        // Parent: wait for child's signal
    }
}

```

(continues on next page)

(continued from previous page)

```

printf("Parent: waiting for signal...\n");
sem_wait(sem);
printf("Parent: received signal!\n");
wait(NULL);
sem_close(sem);
sem_unlink(sem_name);
}
}

```

```

$ ./ipc-posix-semaphore
Parent: waiting for signal...
Child: doing work...
Child: signaling parent
Parent: received signal!

```

Unix Domain Sockets

Source

`src/ipc/unix-socket`

Unix domain sockets provide the full socket API for local IPC, supporting both stream (SOCK_STREAM) and datagram (SOCK_DGRAM) modes. Unlike network sockets, Unix domain sockets use filesystem paths for addressing and never leave the kernel, making them faster than TCP/IP loopback.

Stream-mode Unix sockets work like TCP: a server binds to a path, listens for connections, and accepts clients. Each accepted connection creates a new bidirectional socket for that client. Datagram-mode sockets work like UDP, preserving message boundaries without connection establishment.

A unique feature of Unix domain sockets is the ability to pass file descriptors between processes using `sendmsg()` with `SCM_RIGHTS` ancillary data. This allows one process to open a file and pass the open descriptor to another process, enabling sophisticated privilege separation patterns.

```

#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <cstring>

int main() {
    const char* sock_path = "/tmp/mysock";
    unlink(sock_path);

    if (fork() == 0) {
        // Child: client
        sleep(1); // Wait for server
        int fd = socket(AF_UNIX, SOCK_STREAM, 0);
        struct sockaddr_un addr = {.sun_family = AF_UNIX};
        strcpy(addr.sun_path, sock_path);
        connect(fd, (struct sockaddr*)&addr, sizeof(addr));

        const char* msg = "Hello from client!";
        write(fd, msg, strlen(msg));
    }
}

```

(continues on next page)

(continued from previous page)

```

char buf[100];
ssize_t n = read(fd, buf, sizeof(buf) - 1);
buf[n] = '\0';
printf("Client received: %s\n", buf);
close(fd);
} else {
    // Parent: server
    int srv = socket(AF_UNIX, SOCK_STREAM, 0);
    struct sockaddr_un addr = {.sun_family = AF_UNIX};
    strcpy(addr.sun_path, sock_path);
    bind(srv, (struct sockaddr*)&addr, sizeof(addr));
    listen(srv, 1);

    int client = accept(srv, NULL, NULL);
    char buf[100];
    ssize_t n = read(client, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("Server received: %s\n", buf);

    const char* reply = "Hello from server!";
    write(client, reply, strlen(reply));

    close(client);
    close(srv);
    wait(NULL);
    unlink(sock_path);
}
}

```

```

$ ./ipc-unix-socket
Server received: Hello from client!
Client received: Hello from server!

```

Passing File Descriptors:

Unix domain sockets can transfer open file descriptors between processes using ancillary data. This is useful for privilege separation—a privileged process can open restricted files and pass the descriptors to unprivileged workers.

```

#include <sys/socket.h>

void send_fd(int sock, int fd) {
    char buf[1] = {0};
    struct iovec iov = {.iov_base = buf, .iov_len = 1};
    char cmsg_buf[CMMSG_SPACE(sizeof(int))];
    struct msghdr msg = {
        .msg_iov = &iov, .msg_iovlen = 1,
        .msg_control = cmsg_buf, .msg_controllen = sizeof(cmsg_buf)
    };
    struct cmsghdr* cmsg = CMSG_FIRSTHDR(&msg);
    cmsg->cmsg_level = SOL_SOCKET;
    cmsg->cmsg_type = SCM_RIGHTS;

```

(continues on next page)

(continued from previous page)

```

cmsg->cmsg_len = CMSG_LEN(sizeof(int));
*(int*)CMSG_DATA(cmsg) = fd;
sendmsg(sock, &msg, 0);
}

int recv_fd(int sock) {
    char buf[1];
    struct iovec iov = {.iov_base = buf, .iov_len = 1};
    char cmsg_buf[CMSG_SPACE(sizeof(int))];
    struct msghdr msg = {
        .msg_iov = &iov, .msg_iovlen = 1,
        .msg_control = cmsg_buf, .msg_controllen = sizeof(cmsg_buf)
    };
    recvmsg(sock, &msg, 0);
    struct cmsghdr* cmsg = CMSG_FIRSTHDR(&msg);
    return *(int*)CMSG_DATA(cmsg);
}

```

System V IPC (Legacy)

Source

[src/ipc/sysv-shm](#)

System V IPC is an older API that predates POSIX IPC. It uses integer keys (generated with `ftok()`) instead of filesystem paths to identify IPC objects. While POSIX IPC is generally preferred for new code, System V IPC remains widely used and is available on all Unix-like systems.

System V provides three IPC mechanisms: shared memory (`shmget/shmat`), semaphores (`semget/semop`), and message queues (`msgget/msgsnd/msgrcv`). These objects persist in the kernel until explicitly removed and can be inspected with the `ipcs` command and removed with `ipcrm`.

```

#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <cstring>

int main() {
    key_t key = ftok("/tmp", 'A');
    int shmid = shmget(key, 4096, IPC_CREAT | 0666);
    char* ptr = (char*)shmat(shmid, NULL, 0);

    if (fork() == 0) {
        // Child: write to shared memory
        strcpy(ptr, "Hello from child (System V)!");
        shmdt(ptr);
    } else {
        // Parent: wait and read
        wait(NULL);
        printf("Parent read: %s\n", ptr);
        shmdt(ptr);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

```

(continues on next page)

(continued from previous page)

```
}
}
```

```
$ ./ipc-sysv-shm
Parent read: Hello from child (System V)!
```

System V IPC Commands:

```
# List all IPC objects
$ ipcs

# List shared memory segments
$ ipcs -m

# List semaphores
$ ipcs -s

# List message queues
$ ipcs -q

# Remove shared memory segment by ID
$ ipcrm -m <shmid>

# Remove semaphore by ID
$ ipcrm -s <semid>
```

Comparison Summary

Feature	Pipe/FIFO	Shm	Msg Queue	Unix Socket
Speed	Medium	Fastest	Medium	Medium
Sync needed	No	Yes	No	No
Message boundary	No	N/A	Yes	Datagram only
Bidirectional	No	Yes	Yes	Yes
Pass FDs	No	No	No	Yes

1.5 CUDA Programming

CUDA is NVIDIA's parallel computing platform that enables developers to harness the massive parallelism of GPUs for general-purpose computing. Unlike CPUs optimized for sequential tasks with a few powerful cores, GPUs contain thousands of smaller cores designed for throughput-oriented workloads. This architecture excels at data-parallel problems where the same operation is applied to many elements simultaneously, such as matrix operations, image processing, and scientific simulations.

The CUDA programming model extends C++ with keywords for defining kernel functions that execute on the GPU and APIs for managing memory transfers between host (CPU) and device (GPU). Understanding memory hierarchy is crucial for performance: global memory has high bandwidth but high latency, while shared memory provides fast on-chip storage shared within thread blocks. Effective CUDA programming involves minimizing data transfers, maximizing parallelism, and optimizing memory access patterns.

1.5.1 CUDA Basics

Table of Contents

- *CUDA Basics*
 - *Error Checking Macros*
 - *Device Properties*
 - *Hello World*
 - *Function Qualifiers*
 - *Vector Addition*
 - *Memory Management*
 - *Shared Memory*
 - *Parallel Reduction*
 - *Streams*

CUDA enables massively parallel computation on NVIDIA GPUs. Programs consist of host code (CPU) that launches kernel functions executing on the device (GPU). Threads are organized into blocks, and blocks form a grid. Each thread identifies itself using `blockIdx` and `threadIdx` to map to data elements.

Error Checking Macros

CUDA API calls return error codes that should always be checked. Silent failures lead to hard-to-debug issues where kernels produce wrong results or memory operations fail without notice. These macros wrap API calls with automatic error checking, printing the file, line number, and error description before exiting. The `LAUNCH_KERNEL` macro uses `cudaLaunchKernelEx` which provides more control over launch configuration than the `<<<<>>>` syntax.

```
#define CUDA_CHECK(exp)
do {
    cudaError_t err = (exp);
    if (err != cudaSuccess) {
        fprintf(stderr, "[%s:%d] %s failed: %s\n", __FILE__, __LINE__, #exp,
cudaGetErrorString(err));
        exit(1);
    }
} while (0)

#define LAUNCH_KERNEL(cfg, kernel, ...) CUDA_CHECK(cudaLaunchKernelEx(cfg, kernel, ##__
VA_ARGS__))

// Usage
CUDA_CHECK(cudaMalloc(&d_data, size));
CUDA_CHECK(cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice));
```

(continues on next page)

(continued from previous page)

```

cudaLaunchConfig_t cfg = {};
cfg.gridDim = dim3(blocks, 1, 1);
cfg.blockDim = dim3(threads, 1, 1);
cfg.stream = stream;
LAUNCH_KERNEL(&cfg, my_kernel, d_data, n);

```

Device Properties

Source

`src/cuda/device-properties`

Before launching kernels, query device capabilities using `cudaGetDeviceProperties`. This is essential for tuning kernel parameters like block size and shared memory usage based on the hardware.

```

int deviceCount = 0;
cudaGetDeviceCount(&deviceCount);

cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, 0);

printf("Device: %s\n", prop.name);
printf("Compute capability: %d.%d\n", prop.major, prop.minor);
printf("Total global memory: %.2f GB\n", prop.totalGlobalMem / (1024.0 * 1024.0 * 1024.
↪0));
printf("Shared memory per block: %zu bytes\n", prop.sharedMemPerBlock);
printf("Registers per block: %d\n", prop.regsPerBlock);
printf("Warp size: %d\n", prop.warpSize);
printf("Max threads per block: %d\n", prop.maxThreadsPerBlock);
printf("Max threads dim: (%d, %d, %d)\n", prop.maxThreadsDim[0], prop.maxThreadsDim[1], ↪
↪prop.maxThreadsDim[2]);
printf("Max grid size: (%d, %d, %d)\n", prop.maxGridSize[0], prop.maxGridSize[1], prop.
↪maxGridSize[2]);
printf("Clock rate: %.2f MHz\n", prop.clockRate / 1000.0);
printf("Memory clock rate: %.2f MHz\n", prop.memoryClockRate / 1000.0);
printf("Memory bus width: %d bits\n", prop.memoryBusWidth);
printf("L2 cache size: %d bytes\n", prop.l2CacheSize);
printf("Multiprocessor count: %d\n", prop.multiProcessorCount);
printf("Max threads per multiprocessor: %d\n", prop.maxThreadsPerMultiProcessor);
printf("Unified addressing: %s\n", prop.unifiedAddressing ? "Yes" : "No");
printf("Concurrent kernels: %s\n", prop.concurrentKernels ? "Yes" : "No");
printf("ECC enabled: %s\n", prop.ECCEnabled ? "Yes" : "No");
printf("PCI Domain:Bus:Device: %04x:%02x:%02x\n", prop.pciDomainID, prop.pciBusID, prop.
↪pciDeviceID);

```

Hello World

Source

`src/cuda/hello`

The `__global__` keyword declares a kernel function that runs on the GPU but is callable from host code. The `<<<blocks, threads>>>` execution configuration specifies how many thread blocks to launch and how many threads per block. Each thread computes its unique global index using `blockIdx.x * blockDim.x + threadIdx.x`, which

maps threads to data elements. Without `cudaDeviceSynchronize()` or stream synchronization, the host continues execution immediately after launching the kernel, potentially accessing results before they're ready.

```
__global__ void hello_kernel(int* output) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    output[idx] = idx;
}

// Launch: 2 blocks, 4 threads each = 8 total threads
hello_kernel<<<2, 4>>>(d_output);
cudaDeviceSynchronize(); // Wait for kernel to complete
```

Function Qualifiers

CUDA provides function qualifiers to specify where functions execute and where they can be called from. Understanding these is essential for organizing code between host and device.

```
// __device__ - Device function: called from device, runs on device
__device__ float square(float x) { return x * x; }

// __host__ __device__ - Compiles for both host and device
__host__ __device__ float add(float a, float b) { return a + b; }

// __global__ - Kernel function: called from host, runs on device
__global__ void compute(float* out, float a, float b) {
    out[0] = add(a, b); // Call __host__ __device__ function
    out[1] = square(a); // Call __device__ function
}
```

Vector Addition

Source

`src/cuda/vector-add`

Vector addition demonstrates the most common CUDA pattern: one thread per data element. Each thread independently computes one output value, achieving massive parallelism with thousands of threads running simultaneously. The bounds check `if (i < n)` is critical because the total number of threads (`blocks × threads per block`) is typically rounded up to a multiple of the block size, creating extra threads that would access memory out of bounds. The ceiling division formula `(n + threads - 1) / threads` ensures enough blocks are launched to cover all elements.

```
__global__ void vector_add(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) { // Bounds check is critical
        c[i] = a[i] + b[i];
    }
}

int threads = 256;
int blocks = (n + threads - 1) / threads; // Ceiling division
vector_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
```

Memory Management

Source

[src/cuda/vector-add](#)

CUDA provides several memory allocation methods with different performance characteristics. Choosing the right one depends on access patterns and transfer requirements.

Device Memory - Standard GPU memory. Fastest for GPU computation but requires explicit transfers. Use for data that stays on GPU.

```
float* d_data;
cudaMalloc(&d_data, size);
cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);
cudaFree(d_data);
```

Pinned (Page-Locked) Memory - Host memory that cannot be paged out. Required for cudaMemcpyAsync to truly overlap with computation. ~2x faster transfers than pageable memory, but reduces available system memory.

```
float* h_pinned;
cudaMallocHost(&h_pinned, size);
cudaMemcpyAsync(d_data, h_pinned, size, cudaMemcpyHostToDevice, stream);
cudaFreeHost(h_pinned);
```

Host Registered Memory - Pins existing malloc'd memory. Useful when you can't change allocation code but need async transfers. Has registration overhead, so avoid for short-lived buffers.

```
float* h_data = (float*)malloc(size);
cudaHostRegister(h_data, size, cudaHostRegisterDefault);
cudaHostUnregister(h_data);
free(h_data);
```

Unified Memory - Single pointer accessible from both CPU and GPU. Driver automatically migrates pages on demand. Simplifies code but may have performance overhead from page faults. Best for prototyping or irregular access patterns.

```
float* data;
cudaMallocManaged(&data, size);
kernel<<<blocks, threads>>>(data);    // GPU access
cudaDeviceSynchronize();
data[0] = 1.0f;                       // CPU access
cudaFree(data);
```

Zero-Copy Memory - GPU directly reads/writes host memory over PCIe. No explicit transfers needed, but each access has PCIe latency. Good for data accessed once or when GPU memory is limited. Poor for repeated access.

```
float* h_mapped;
float* d_mapped;
cudaHostAlloc(&h_mapped, size, cudaHostAllocMapped);
cudaHostGetDevicePointer(&d_mapped, h_mapped, 0);
kernel<<<blocks, threads>>>(d_mapped);
cudaFreeHost(h_mapped);
```

Summary (64 MB benchmark, run `cuda-memory-benchmark` to test on your system):

Method	H2D Speed	D2H Speed	Use Case
Pageable (malloc)	~10.6 GB/s	~1.3 GB/s	Simple, non-critical
cudaMallocHost	~13.3 GB/s	~2.0 GB/s	Async transfers
cudaHostRegister	~13.2 GB/s	~2.1 GB/s	Pin existing memory
cudaMallocManaged	~0.8 GB/s	~0.4 GB/s	Prototyping, irregular

Source[src/cuda/memory-benchmark](#)**RAII Pattern** - Ensures proper cleanup:

```
struct CudaBuffer {
    float* h_data;
    float* d_data;
    cudaStream_t stream;

    CudaBuffer(size_t size) {
        cudaStreamCreate(&stream);
        cudaMallocHost(&h_data, size);
        cudaMalloc(&d_data, size);
    }

    ~CudaBuffer() {
        cudaFree(d_data);
        cudaFreeHost(h_data);
        cudaStreamDestroy(stream);
    }
};
```

Shared Memory**Source**[src/cuda/matrix-mul](#)**Source**[src/cuda/shared-memory](#)

Shared memory is a programmer-managed cache located on the GPU chip, providing much lower latency than global memory. All threads within a block share the same shared memory space, making it ideal for data reuse patterns like tiled matrix multiplication or stencil computations. The `__shared__` keyword declares arrays in shared memory, and `__syncthreads()` creates a barrier ensuring all threads complete their operations before proceeding. This synchronization is essential because threads execute in warps of 32, and without barriers, some threads may read data that others haven't written yet.

```
#define TILE_SIZE 16

__global__ void tiled_kernel(float* data, int n) {
    __shared__ float tile[TILE_SIZE]; // Shared within block

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    tile[threadIdx.x] = data[idx]; // Load to shared memory

    __syncthreads(); // Wait for all threads to finish loading
```

(continues on next page)

(continued from previous page)

```
// Now all threads can safely read from tile[]
float result = tile[threadIdx.x] * 2.0f;
data[idx] = result;
}
```

Common pitfalls:

- Missing `__syncthreads()` causes race conditions
- Shared memory is limited (~48KB per block)
- Bank conflicts reduce performance

For complete examples including tiled matrix multiplication and stencil with halo regions, see the source files linked above.

Parallel Reduction

Source

[src/cuda/reduction](#)

Reduction is a fundamental parallel pattern for computing aggregates like sum, min, max, or product across large arrays. The tree-based approach repeatedly halves the number of active threads: in each iteration, thread `i` combines its value with thread `i + stride`. Shared memory holds intermediate results within each block, and `__syncthreads()` ensures all threads complete each level before proceeding to the next. After the kernel, each block has produced one partial result, which can be combined on the CPU or with a second reduction kernel. The block size should be a power of two for this simple implementation.

```
#define BLOCK_SIZE 256

__global__ void reduce_sum(const float* input, float* output, int n) {
    __shared__ float sdata[BLOCK_SIZE];

    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    sdata[tid] = (i < n) ? input[i] : 0.0f;
    __syncthreads();

    // Tree reduction: 128, 64, 32, 16, 8, 4, 2, 1
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}
```

Streams

Source

`src/cuda/streams`

CUDA streams are independent execution queues that enable concurrent operations. Operations within a stream execute in order, but operations in different streams can overlap. This allows hiding memory transfer latency by pipelining: while one chunk of data is being processed by a kernel, the next chunk can be transferred to the GPU, and the previous chunk's results can be copied back to the host. Pinned memory (`cudaMallocHost`) is mandatory for true asynchronous transfers; pageable memory forces synchronous behavior even with `cudaMemcpyAsync`. The optimal number of streams depends on the workload and hardware, but 2-4 streams typically provide good overlap without excessive scheduling overhead.

```
constexpr int num_streams = 4;
cudaStream_t streams[num_streams];

for (int i = 0; i < num_streams; i++) {
    cudaStreamCreate(&streams[i]);
}

// Pipeline: each stream handles a chunk independently
for (int i = 0; i < num_streams; i++) {
    int offset = i * chunk_size;
    cudaMemcpyAsync(d_data + offset, h_data + offset, chunk_bytes, cudaMemcpyHostToDevice,
↳streams[i]);
    kernel<<<blocks, threads, 0, streams[i]>>>(d_data + offset, chunk_size, i);
    cudaMemcpyAsync(h_data + offset, d_data + offset, chunk_bytes, cudaMemcpyDeviceToHost,
↳streams[i]);
}

for (int i = 0; i < num_streams; i++) {
    cudaStreamSynchronize(streams[i]);
    cudaStreamDestroy(streams[i]);
}
```

1.5.2 CUDA C++

Table of Contents

- *CUDA C++*
 - *Atomic Operations*
 - *Barriers*
 - *Semaphores*
 - *Latches*
 - *Memory Order*
 - *Thread Scopes*
 - *Pipeline with Barrier*
 - *Quick Reference*

libc++ is NVIDIA's implementation of the C++ Standard Library for CUDA. It provides thread-safe, GPU-compatible versions of standard library components including atomics, barriers, semaphores, and memory ordering primitives. Unlike raw CUDA intrinsics, libc++ offers portable abstractions that work across different GPU architectures and can even compile for CPU targets, making code more maintainable and easier to reason about.

The library uses C++ memory model semantics (`memory_order_acquire`, `memory_order_release`, etc.) which map to appropriate PTX instructions on the GPU. This allows writing correct concurrent code without manually managing memory fences or understanding PTX details. For detailed coverage of memory visibility, fences, and PTX-level control, see [Memory Visibility](#).

Atomic Operations

Source

`src/cuda/libcuxx-atomic`

Atomics provide thread-safe read-modify-write operations. libc++ atomics support all standard memory orderings and work across thread scopes (block, device, system). Use `cuda::atomic` for device-wide visibility or `cuda::atomic_ref` to wrap existing memory locations.

```
#include <cuda/atomic>

// Device-scope atomic counter
__device__ cuda::atomic<int, cuda::thread_scope_device> counter{0};

__global__ void increment_kernel() {
    counter.fetch_add(1, cuda::memory_order_relaxed);
}

// Atomic with acquire-release for synchronization
__device__ cuda::atomic<int, cuda::thread_scope_device> flag{0};
__device__ int data;

__global__ void producer() {
    data = 42; // Write data
    flag.store(1, cuda::memory_order_release); // Release: data visible before
    ↪ flag
}

__global__ void consumer() {
    while (flag.load(cuda::memory_order_acquire) == 0); // Acquire: see data after flag
    int x = data; // Guaranteed to see 42
}

// Atomic reference to existing memory
__global__ void atomic_ref_kernel(int* arr, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        cuda::atomic_ref<int, cuda::thread_scope_device> ref(arr[idx]);
        ref.fetch_add(1, cuda::memory_order_relaxed);
    }
}
```

Barriers

Source

src/cuda/libcuxx-barrier

Barriers synchronize groups of threads, ensuring all threads reach a point before any proceed. libcu++ provides `cuda::barrier` which supports both arrival/wait patterns and the more efficient `arrive_and_wait`. Barriers can also track completion of async operations like `memcpy_async`.

```
#include <cuda/barrier>
#include <cooperative_groups.h>

namespace cg = cooperative_groups;

__global__ void barrier_kernel() {
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;

    auto block = cg::this_thread_block();
    if (block.thread_rank() == 0) {
        init(&bar, block.size()); // Initialize with thread count
    }
    block.sync();

    // Phase 1: all threads do work
    // ...

    bar.arrive_and_wait(); // Synchronize

    // Phase 2: all threads see Phase 1 results
}

// Split arrive/wait for overlapping work
__global__ void split_barrier_kernel() {
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;
    __shared__ int shared_data[256];

    auto block = cg::this_thread_block();
    if (block.thread_rank() == 0) init(&bar, block.size());
    block.sync();

    // Arrive early, do independent work, then wait
    auto token = bar.arrive(); // Signal arrival
    // ... do work that doesn't need barrier ...
    bar.wait(std::move(token)); // Wait for others
}
```

Semaphores

Source

src/cuda/libcuxx-semaphore

Semaphores control access to limited resources. `counting_semaphore` allows up to N concurrent accesses, while `binary_semaphore` (N=1) acts as a mutex. Use for producer-consumer patterns or limiting concurrent operations. For more detail, see [Memory Visibility](#).

```

#include <cuda/semaphore>

// Limit concurrent access to 4 threads
__device__ cuda::counting_semaphore<cuda::thread_scope_device, 4> sem{4};

__global__ void limited_access_kernel() {
    sem.acquire(); // Block if 4 threads already inside
    // ... critical section with limited concurrency ...
    sem.release();
}

// Binary semaphore as mutex
__device__ cuda::binary_semaphore<cuda::thread_scope_device> mtx{1};

__global__ void mutex_kernel(int* shared_counter) {
    mtx.acquire();
    (*shared_counter)++; // Only one thread at a time
    mtx.release();
}

```

Latches

Source

src/cuda/libcuxx-latch

A latch is a single-use barrier. Threads call `count_down()` to decrement the counter, and `wait()` blocks until the counter reaches zero. Unlike barriers, latches cannot be reused after reaching zero. For more detail, see [Memory Visibility](#).

```

#include <cuda/latch>

__global__ void latch_kernel() {
    __shared__ cuda::latch<cuda::thread_scope_block> lat;

    auto block = cg::this_thread_block();
    if (block.thread_rank() == 0) {
        init(&lat, block.size());
    }
    block.sync();

    // Each thread does work then counts down
    // ... work ...
    lat.count_down();

    // Wait for all threads to finish
    lat.wait();
}

// Arrive and wait in one call
lat.arrive_and_wait(); // count_down() + wait()

```

Memory Order

libc++ supports C++ memory orderings that control visibility of memory operations across threads. Understanding these is crucial for writing correct lock-free code. See *Memory Visibility* for detailed coverage of CUDA memory model.

```
#include <cuda/atomic>

// memory_order_relaxed: No ordering guarantees, fastest
// Use for counters where order doesn't matter
__device__ cuda::atomic<int, cuda::thread_scope_device> counter{0};
counter.fetch_add(1, cuda::memory_order_relaxed);

// memory_order_acquire: Reads after this see writes before matching release
// memory_order_release: Writes before this visible after matching acquire
// Use for producer-consumer synchronization
__device__ cuda::atomic<bool, cuda::thread_scope_device> ready{false};
__device__ int payload;

// Producer                                // Consumer
payload = 42;                               while (!ready.load(cuda::memory_order_acquire));
ready.store(true, memory_order_release);    int x = payload; // Sees 42

// memory_order_acq_rel: Both acquire and release
// Use for read-modify-write that both reads and publishes
counter.fetch_add(1, cuda::memory_order_acq_rel);

// memory_order_seq_cst: Total ordering, slowest but simplest
// Use when unsure or for debugging
counter.fetch_add(1, cuda::memory_order_seq_cst);
```

Thread Scopes

Thread scopes define the visibility domain for synchronization primitives. Narrower scopes can be more efficient but only synchronize within that scope.

```
#include <cuda/atomic>

// thread_scope_thread: Single thread (rarely useful)
cuda::atomic<int, cuda::thread_scope_thread> local_atomic;

// thread_scope_block: Threads in same block (uses shared memory)
__shared__ cuda::atomic<int, cuda::thread_scope_block> block_atomic;

// thread_scope_device: All threads on GPU (uses global memory)
__device__ cuda::atomic<int, cuda::thread_scope_device> device_atomic;

// thread_scope_system: GPU + CPU (for unified memory)
// Slowest but works across host and device
cuda::atomic<int, cuda::thread_scope_system>* system_atomic;
cudaMallocManaged(&system_atomic, sizeof(*system_atomic));
```

Pipeline with Barrier

Source

src/cuda/libcuxx-barrier

Combine barriers with `memcpy_async` for efficient pipelining. The barrier tracks completion of async copies, allowing overlap of memory transfers with computation. See *Pipelines* for more pipeline patterns.

```
#include <cuda/barrier>
#include <cuda/pipeline>
#include <cooperative_groups.h>

__global__ void pipeline_with_barrier(const float* global_in, float* global_out, int n) {
    __shared__ float smem[256];
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;

    auto block = cg::this_thread_block();
    if (block.thread_rank() == 0) init(&bar, block.size());
    block.sync();

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        // Async copy with barrier tracking
        cuda::memcpy_async(&smem[threadIdx.x], &global_in[idx], sizeof(float), bar);
    }

    bar.arrive_and_wait(); // Wait for all copies to complete

    // Process data in shared memory
    if (idx < n) {
        global_out[idx] = smem[threadIdx.x] * 2.0f;
    }
}
```

Quick Reference

```
// Headers
#include <cuda/atomic>           // atomic, atomic_ref
#include <cuda/barrier>          // barrier
#include <cuda/semaphore>        // counting_semaphore, binary_semaphore
#include <cuda/latch>            // latch
#include <cuda/pipeline>         // pipeline, memcpy_async

// Thread scopes (narrower = faster)
cuda::thread_scope_thread       // Single thread
cuda::thread_scope_block        // Same block (shared memory)
cuda::thread_scope_device       // All GPU threads (global memory)
cuda::thread_scope_system       // GPU + CPU (unified memory)

// Memory orders (weaker = faster)
cuda::memory_order_relaxed      // No ordering
cuda::memory_order_acquire      // Reads see prior releases
cuda::memory_order_release      // Writes visible to acquires
cuda::memory_order_acq_rel      // Both acquire and release
```

(continues on next page)

(continued from previous page)

```

cuda::memory_order_seq_cst      // Total ordering

// Atomic operations
atomic.load(order)              // Read
atomic.store(val, order)        // Write
atomic.exchange(val, order)     // Swap, return old
atomic.fetch_add(val, order)    // Add, return old
atomic.fetch_sub(val, order)    // Subtract, return old
atomic.fetch_and(val, order)    // Bitwise AND, return old
atomic.fetch_or(val, order)     // Bitwise OR, return old
atomic.fetch_xor(val, order)    // Bitwise XOR, return old
atomic.compare_exchange_weak(expected, desired, order)
atomic.compare_exchange_strong(expected, desired, order)

// Barrier operations
init(&bar, count)               // Initialize (once per barrier)
bar.arrive()                    // Signal arrival, return token
bar.wait(token)                 // Wait for phase completion
bar.arrive_and_wait()           // arrive() + wait()

// Semaphore operations
sem.acquire()                   // Decrement, block if zero
sem.release()                   // Increment
sem.try_acquire()               // Non-blocking acquire

// Latch operations
init(&lat, count)               // Initialize
lat.count_down()                // Decrement counter
lat.wait()                      // Block until zero
lat.arrive_and_wait()           // count_down() + wait()

```

1.5.3 Thrust

Table of Contents

- *Thrust*
 - *Device and Host Vectors*
 - *Transform*
 - *Reduction*
 - *Sorting*
 - *Custom Functors*
 - *See Also*

Thrust is a parallel algorithms library that provides an STL-like interface for GPU programming. It abstracts away CUDA kernel launches and memory management, allowing developers to write high-level code that automatically executes on the GPU. Thrust supports multiple backends including CUDA, OpenMP, and TBB, making code portable across different parallel architectures. The library includes containers (`device_vector`, `host_vector`), iterators, and

a comprehensive set of parallel algorithms for transformations, reductions, sorting, and scans.

Device and Host Vectors

Source

src/cuda/thrust-basics

Thrust provides `device_vector` for GPU memory and `host_vector` for CPU memory. Data transfers between host and device happen automatically through assignment, making memory management straightforward. Both containers behave like `std::vector` with familiar operations like `push_back`, `resize`, and iterator access.

```
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>

int main() {
    // Create host vector and initialize
    thrust::host_vector<int> h_vec(4);
    h_vec[0] = 10;
    h_vec[1] = 20;
    h_vec[2] = 30;
    h_vec[3] = 40;

    // Transfer to device (automatic copy)
    thrust::device_vector<int> d_vec = h_vec;

    // Modify on device
    d_vec[0] = 100; // Note: slow, triggers kernel launch

    // Transfer back to host
    thrust::host_vector<int> result = d_vec;
}
```

Transform

Source

src/cuda/thrust-transform

`thrust::transform` applies a function to each element, similar to `std::transform`. It supports unary operations on a single range and binary operations combining two ranges. Thrust provides built-in functors like `thrust::negate` and `thrust::plus`, or you can use lambdas with `__host__` `__device__` annotations.

```
#include <thrust/device_vector.h>
#include <thrust/functional.h>
#include <thrust/transform.h>

int main() {
    thrust::device_vector<int> a(4, 10); // [10, 10, 10, 10]
    thrust::device_vector<int> b(4, 5);  // [5, 5, 5, 5]
    thrust::device_vector<int> c(4);

    // Unary transform: negate each element
    thrust::transform(a.begin(), a.end(), c.begin(), thrust::negate<int>());
    // c = [-10, -10, -10, -10]

    // Binary transform: add two vectors
```

(continues on next page)

(continued from previous page)

```

thrust::transform(a.begin(), a.end(), b.begin(), c.begin(), thrust::plus<int>());
// c = [15, 15, 15, 15]

// Custom functor with lambda (CUDA 11+)
thrust::transform(
    a.begin(),
    a.end(),
    c.begin(),
    [] __host__ __device__(int x) { return x * x; }
);
// c = [100, 100, 100, 100]
}

```

Reduction

Source

src/cuda/thrust-reduce

`thrust::reduce` combines all elements into a single value using a binary operation. Common reductions include sum, product, min, and max. For more complex reductions, `thrust::transform_reduce` combines a transformation with reduction in a single pass, avoiding intermediate storage.

```

#include <thrust/device_vector.h>
#include <thrust/functional.h>
#include <thrust/reduce.h>

int main() {
    thrust::device_vector<int> d_vec(4);
    d_vec[0] = 1;
    d_vec[1] = 2;
    d_vec[2] = 3;
    d_vec[3] = 4;

    // Sum all elements (default)
    int sum = thrust::reduce(d_vec.begin(), d_vec.end());
    // sum = 10

    // Sum with initial value
    int sum_plus_100 = thrust::reduce(d_vec.begin(), d_vec.end(), 100);
    // sum_plus_100 = 110

    // Product of all elements
    int product = thrust::reduce(d_vec.begin(), d_vec.end(), 1, thrust::multiplies<int>());
    // product = 24

    // Find maximum
    int max_val = thrust::reduce(d_vec.begin(), d_vec.end(), INT_MIN, thrust::maximum<int>
↪());
    // max_val = 4
}

```

Sorting

Source

src/cuda/thrust-sort

Thrust provides efficient parallel sorting with `thrust::sort` for in-place sorting and `thrust::sort_by_key` for sorting key-value pairs. Custom comparators allow sorting in descending order or by custom criteria.

```
#include <thrust/device_vector.h>
#include <thrust/functional.h>
#include <thrust/sort.h>

int main() {
    thrust::device_vector<int> d_vec(4);
    d_vec[0] = 30;
    d_vec[1] = 10;
    d_vec[2] = 40;
    d_vec[3] = 20;

    // Sort ascending (default)
    thrust::sort(d_vec.begin(), d_vec.end());
    // d_vec = [10, 20, 30, 40]

    // Sort descending
    thrust::sort(d_vec.begin(), d_vec.end(), thrust::greater<int>());
    // d_vec = [40, 30, 20, 10]

    // Sort by key
    thrust::device_vector<int> keys = {3, 1, 4, 2};
    thrust::device_vector<char> vals = {'c', 'a', 'd', 'b'};
    thrust::sort_by_key(keys.begin(), keys.end(), vals.begin());
    // keys = [1, 2, 3, 4], vals = ['a', 'b', 'c', 'd']
}
```

Custom Functors

Source

src/cuda/thrust-custom

For complex operations, define custom functors as structs with `__host__ __device__ operator()`. This approach works across all CUDA versions and allows stateful functors with constructor parameters.

```
#include <thrust/device_vector.h>
#include <thrust/transform.h>

struct saxpy_functor {
    const float a;

    saxpy_functor(float _a) : a(_a) {}

    __host__ __device__ float operator()(float x, float y) const {
        return a * x + y;
    }
};
```

(continues on next page)

(continued from previous page)

```

void saxpy(float a, thrust::device_vector<float>& x, thrust::device_vector<float>& y) {
    thrust::transform(x.begin(), x.end(), y.begin(), y.begin(), saxpy_functor(a));
}

int main() {
    thrust::device_vector<float> x(4, 1.0f);
    thrust::device_vector<float> y(4, 2.0f);

    saxpy(3.0f, x, y);
    // y = [5.0, 5.0, 5.0, 5.0]  (3*1 + 2 = 5)
}

```

See Also

- *CUDA Basics* - CUDA fundamentals
- *CUDA C++* - libc++ for CUDA C++ standard library features

1.5.4 Cooperative Groups

Table of Contents

- *Cooperative Groups*
 - *Introduction*
 - *Quick Reference*
 - * *Creating Groups*
 - * *Group Properties*
 - * *Synchronization*
 - * *Collective Reductions*
 - * *Scan Operations*
 - * *Shuffle Operations*
 - * *Predicates (Vote Operations)*
 - * *Partitioning*
 - * *Thread Indexing*
 - *Legacy Synchronization*
 - *Thread Groups Hierarchy*
 - *Collective Operations*
 - *Tiled Partition for Warp-Level Programming*
 - *Coalesced Threads for Divergent Code*
 - *Comparison: Legacy vs Cooperative Groups*
 - *Low-Level PTX Alternatives*

- * *Memory Fences*
- * *Warp Reduction with PTX Shuffle*
- * *Acquire/Release Semantics*
- * *When to Use Each Approach*

Introduction

Cooperative Groups is a CUDA programming model extension that provides flexible thread synchronization and collective operations. Before Cooperative Groups, synchronization was limited to `__syncthreads()` (all threads in a block) or implicit warp synchronization. Cooperative Groups enables explicit synchronization at any granularity: warps, tiles, thread blocks, or even across multiple blocks.

The key insight is that threads form hierarchical groups, and you can synchronize or perform collective operations on any group. This makes code more composable and easier to reason about, especially for complex algorithms that need different synchronization scopes.

Quick Reference

Source

`src/cuda/coop-groups-cheatsheet`

Creating Groups

Create thread groups at different granularities for synchronization and collective ops.

```
#include <cooperative_groups.h>
#include <cooperative_groups/reduce.h>
namespace cg = cooperative_groups;

cg::thread_block block = cg::this_thread_block();           // All threads in block
cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block); // 32-thread warp
cg::thread_block_tile<4> tile = cg::tiled_partition<4>(warp); // 4-thread tile
cg::coalesced_group active = cg::coalesced_threads();       // Currently active threads
```

Group Properties

Query group size and thread position within the group.

```
block.size();           // Number of threads in group
block.thread_rank();    // Thread index within group (0 to size-1)
warp.meta_group_rank(); // Which warp in block (0, 1, 2, ...)
warp.meta_group_size(); // Number of warps in block
```

Synchronization

Barrier synchronization - all threads must reach before any proceed. For advanced barrier patterns with split arrive/wait, see `cuda::barrier` in *CUDA C++*.

```
block.sync();           // __syncthreads() equivalent
warp.sync();            // __syncwarp() equivalent
tile.sync();            // Sync 4 threads
```

Collective Reductions

Combine values across all threads in a group. All threads receive the result.

```
float sum = cg::reduce(warp, val, cg::plus<float>()); // Sum
float max = cg::reduce(warp, val, cg::greater<float>()); // Max
float min = cg::reduce(warp, val, cg::less<float>()); // Min
int all_and = cg::reduce(warp, bits, cg::bit_and<int>()); // Bitwise AND
int all_or = cg::reduce(warp, bits, cg::bit_or<int>()); // Bitwise OR
int all_xor = cg::reduce(warp, bits, cg::bit_xor<int>()); // Bitwise XOR
```

Scan Operations

Prefix sum - each thread gets cumulative result of all preceding threads.

```
float incl = cg::inclusive_scan(warp, val, cg::plus<float>()); // [a, a+b, a+b+c, ...]
float excl = cg::exclusive_scan(warp, val, cg::plus<float>()); // [0, a, a+b, ...]
```

Shuffle Operations

Direct register-to-register data exchange between threads without shared memory.

```
float broadcasted = warp.shfl(val, 0); // Broadcast from thread 0
float from_src = warp.shfl(val, src_lane); // Get value from src_lane
float shifted = warp.shfl_down(val, 1); // Shift values down by 1
float shifted_up = warp.shfl_up(val, 1); // Shift values up by 1
float swapped = warp.shfl_xor(val, 1); // XOR swap with neighbor
```

Predicates (Vote Operations)

Evaluate conditions across threads and collect results.

```
bool any_true = warp.any(predicate); // True if any thread has predicate
bool all_true = warp.all(predicate); // True if all threads have predicate
unsigned ballot = warp.ballot(predicate); // Bitmask of predicate results
```

Partitioning

Dynamically group threads based on runtime values.

```
cg::coalesced_group same_label = cg::labeled_partition(warp, label);
unsigned match_mask = __match_any_sync(0xffffffff, value); // Threads with same value
```

Thread Indexing

Map between cooperative groups API and traditional CUDA indexing.

```
int global_idx = block.group_index().x * block.group_dim().x + block.thread_index().x;
dim3 block_idx = block.group_index(); // blockIdx equivalent
dim3 block_dim = block.group_dim(); // blockDim equivalent
dim3 thread_idx = block.thread_index(); // threadIdx equivalent
```

Legacy Synchronization

Before Cooperative Groups, CUDA provided these synchronization primitives:

- `__syncthreads()`: Barrier for all threads in a block. All threads must reach this point before any can proceed. Required when threads share data via shared memory.
- `__syncwarp(mask)`: Barrier for threads in a warp specified by the mask. Introduced in CUDA 9.0 to make warp-level synchronization explicit after the Volta architecture changed warp execution semantics.

```
// Legacy synchronization
__shared__ float sdata[256];

sdata[threadIdx.x] = input[idx];
__syncthreads(); // All threads must complete store before any reads

// Warp-level sync (threads 0-31)
if (threadIdx.x < 32) {
    __syncwarp(0xFFFFFFFF); // Full warp mask
}
```

The problem with legacy primitives is they’re not composable. You can’t easily write a function that synchronizes “its” threads without knowing the calling context. Cooperative Groups solves this by making groups first-class objects.

Thread Groups Hierarchy

Source

`src/cuda/coop-groups-basic`

Cooperative Groups provides a hierarchy of thread groups:

- `thread_block`: All threads in the current block (like `__syncthreads()`)
- `tiled_partition<N>`: Static partition of a group into tiles of N threads
- `coalesced_threads()`: Active threads at the current point (dynamic)
- `this_thread`: Single thread (useful for generic code)

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__global__ void hierarchy_example(float* data, int n) {
    // Get the thread block group
    cg::thread_block block = cg::this_thread_block();

    // Partition into 32-thread tiles (warps)
    cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block);

    // Partition warp into 4-thread tiles
    cg::thread_block_tile<4> tile4 = cg::tiled_partition<4>(warp);

    // Synchronize at different levels
    block.sync(); // All threads in block
    warp.sync();  // All threads in this warp
    tile4.sync(); // All 4 threads in this tile

    // Get thread's rank within each group
```

(continues on next page)

(continued from previous page)

```

int block_rank = block.thread_rank(); // 0 to blockDim.x-1
int warp_rank = warp.thread_rank();   // 0 to 31
int tile_rank = tile4.thread_rank();   // 0 to 3
}

```

Collective Operations

Source

src/cuda/coop-groups-reduce

Cooperative Groups provides built-in collective operations that work on any group:

- `reduce(group, val, op)`: Reduce values across group using binary operator
- `shfl(group, val, src)`: Broadcast value from source thread
- `shfl_up/shfl_down`: Shift values within group
- `any/all`: Predicate operations

```

#include <cooperative_groups.h>
#include <cooperative_groups/reduce.h>
namespace cg = cooperative_groups;

__global__ void reduce_example(const float* input, float* output, int n) {
    cg::thread_block block = cg::this_thread_block();
    cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block);

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float val = (idx < n) ? input[idx] : 0.0f;

    // Warp-level reduction - no shared memory needed!
    float warp_sum = cg::reduce(warp, val, cg::plus<float>());

    // First thread in each warp has the result
    if (warp.thread_rank() == 0) {
        atomicAdd(output, warp_sum);
    }
}

```

Tiled Partition for Warp-Level Programming

Source

src/cuda/coop-groups-tile

`tiled_partition` creates fixed-size thread groups, commonly used for warp-level programming. The tile size must be a power of 2 and at most 32.

```

// Helper function for manual warp/tile reduction
template <int TileSize, typename T>
__device__ T tile_reduce_sum(cg::thread_block_tile<TileSize>& tile, T val) {
    for (int offset = tile.size() / 2; offset > 0; offset /= 2) {
        val += tile.shfl_down(val, offset);
    }
    return val; // Only thread 0 has correct result
}

```

(continues on next page)

(continued from previous page)

```

}

__global__ void warp_reduce(const float* input, float* output, int n) {
    cg::thread_block block = cg::this_thread_block();
    cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block);

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float val = (idx < n) ? input[idx] : 0.0f;

    float sum = tile_reduce_sum(warp, val);

    if (warp.thread_rank() == 0) {
        output[blockIdx.x * (blockDim.x / 32) + warp.meta_group_rank()] = sum;
    }
}

```

Coalesced Threads for Divergent Code

Source

src/cuda/coop-groups-coalesced

When threads in a warp take different branches (e.g., if/else), only some threads execute each branch. `coalesced_threads()` creates a group containing only the threads that reached that specific point in the code. This lets you perform collective operations (like reduce) on just those active threads, rather than the entire warp.

```

// Without coalesced_threads: must handle inactive threads manually
if (val > 0) {
    atomicAdd(output, val); // Each thread does atomic - slow!
}

// With coalesced_threads: reduce among active threads first
if (val > 0) {
    cg::coalesced_group active = cg::coalesced_threads(); // Only threads where val > 0
    int sum = cg::reduce(active, val, cg::plus<int>()); // Sum among active only
    if (active.thread_rank() == 0) {
        atomicAdd(output, sum); // Single atomic - fast!
    }
}

```

```

__global__ void coalesced_example(const int* input, int* output, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    int val = input[idx];

    // Only process positive values
    if (val > 0) {
        // Get group of threads that took this branch
        cg::coalesced_group active = cg::coalesced_threads();

        // Reduce only among active threads
        int sum = cg::reduce(active, val, cg::plus<int>());
    }
}

```

(continues on next page)

(continued from previous page)

```

// First active thread writes result
if (active.thread_rank() == 0) {
    atomicAdd(output, sum);
}
}
}

```

Comparison: Legacy vs Cooperative Groups

```

// Legacy reduction (last warp)
__shared__ float sdata[256];
// ... load data ...
__syncthreads();

// Unrolled warp reduction - implicit warp sync (unsafe on Volta+)
if (tid < 32) {
    volatile float* smem = sdata;
    smem[tid] += smem[tid + 32];
    smem[tid] += smem[tid + 16];
    smem[tid] += smem[tid + 8];
    smem[tid] += smem[tid + 4];
    smem[tid] += smem[tid + 2];
    smem[tid] += smem[tid + 1];
}

// Cooperative Groups reduction - explicit, safe, readable
cg::thread_block block = cg::this_thread_block();
cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block);

float val = sdata[tid];
block.sync();

val = cg::reduce(warp, val, cg::plus<float>());

```

Benefits of Cooperative Groups:

- **Explicit synchronization:** No relying on implicit warp behavior
- **Composable:** Functions can sync their own groups without global knowledge
- **Portable:** Works correctly across GPU architectures
- **Readable:** Intent is clear from the code

Low-Level PTX Alternatives

Source

[src/cuda/coop-groups-ptx](#)

For maximum performance, some libraries (like DeepSeek’s DeepEP) use inline PTX assembly instead of cooperative groups. PTX (Parallel Thread Execution) is NVIDIA’s low-level virtual instruction set that sits between CUDA C++ and the actual GPU machine code (SASS). Using PTX directly provides finer control over memory ordering, cache behavior, and instruction selection, but sacrifices portability and readability.

The main reasons to use PTX over cooperative groups:

- **Memory ordering control:** PTX exposes acquire/release semantics for lock-free algorithms, allowing precise control over when writes become visible to other threads or devices.
- **Cache hints:** PTX can specify L1/L2 cache policies (no-allocate, evict-first) to optimize memory access patterns for specific workloads.
- **Compile-time optimization:** Template-based PTX wrappers enable the compiler to fully unroll reduction loops and eliminate branches at compile time.
- **Cross-GPU/CPU synchronization:** System-wide fences are needed for multi-GPU communication or GPU-CPU synchronization in distributed training.

Memory Fences

Memory fences ensure that memory operations before the fence are visible to other threads after the fence. CUDA provides built-in fence functions, while PTX offers more granular control with acquire/release semantics.

Built-in CUDA fence functions:

```
__threadfence_block(); // Visible to threads in same block
__threadfence();       // Visible to all threads on same GPU
__threadfence_system(); // Visible to all threads + CPU (multi-GPU, pinned memory)
```

PTX fence instructions:

```
__device__ void memory_fence_cta() {
    asm volatile("fence.acq_rel.cta;" ::: "memory");
}
__device__ void memory_fence_gpu() {
    asm volatile("fence.acq_rel.gpu;" ::: "memory");
}
__device__ void memory_fence_sys() {
    asm volatile("fence.acq_rel.sys;" ::: "memory");
}
```

Comparison:

CUDA Built-in	PTX Equivalent	Use Case
<code>__threadfence_block</code>	<code>fence.acq_rel.cta</code>	Shared memory sync within block
<code>__threadfence</code>	<code>fence.acq_rel.gpu</code>	Global memory sync across blocks
<code>__threadfence_system</code>	<code>fence.acq_rel.sys</code>	Multi-GPU or GPU-CPU sync

The key difference is that PTX fences support explicit acquire/release semantics, while CUDA built-ins provide sequentially consistent fences. PTX also allows combining fences with specific load/store instructions (e.g., `ld.acquire`, `st.release`) for finer-grained control in lock-free algorithms.

```
// Sequential consistency (CUDA built-in): HEAVY
// Ensures ALL prior memory operations complete before ANY subsequent ones
data[idx] = value;
__threadfence(); // Full barrier - waits for everything
flag = 1;

// Acquire/Release (PTX): LIGHTER
// Only orders specific operations, not everything
```

(continues on next page)

(continued from previous page)

```

data[idx] = value;
st_release_gpu(&flag, 1); // Ensures 'data' write completes before 'flag' write

// Producer-Consumer Example:
//
// Thread 0 (Producer)      Thread 1 (Consumer)
// -----
// data = 42;                while (ld_acquire(&flag) == 0);
// st_release(&flag, 1);      result = data; // Guaranteed to see 42
//      |                      ^
//      +-----+
//      release on 'flag' ensures    acquire on 'flag' ensures
//      'data=42' is visible          'data' read sees the write
//      before 'flag=1'              that happened before 'flag=1'

```

Warp Reduction with PTX Shuffle

Both cooperative groups and PTX use warp shuffle instructions for reduction, but the PTX approach allows template-based compile-time unrolling. The `if constexpr` pattern ensures the compiler generates only the shuffle instructions needed for the specified lane count, with no runtime branches. This is particularly useful when reducing over sub-warp groups (e.g., 4 or 8 threads) where you want to avoid unnecessary shuffle operations.

```

// Cooperative Groups reduction
cg::thread_block_tile<32> warp = cg::tiled_partition<32>(block);
float sum = cg::reduce(warp, val, cg::plus<float>());

// PTX-style reduction with compile-time lane count
template<int kNumLanes, typename T, typename Op>
__device__ T warp_reduce(T value, Op op) {
    if constexpr (kNumLanes >= 32) value = op(value, __shfl_xor_sync(0xffffffff, value,
↪ 16));
    if constexpr (kNumLanes >= 16) value = op(value, __shfl_xor_sync(0xffffffff, value,
↪ 8));
    if constexpr (kNumLanes >= 8) value = op(value, __shfl_xor_sync(0xffffffff, value,
↪ 4));
    if constexpr (kNumLanes >= 4) value = op(value, __shfl_xor_sync(0xffffffff, value,
↪ 2));
    if constexpr (kNumLanes >= 2) value = op(value, __shfl_xor_sync(0xffffffff, value,
↪ 1));
    return value;
}

// Usage with __device__ lambda (requires --extended-lambda)
inline constexpr auto ReduceSum = [] __device__(auto a, auto b) { return a + b; };
float sum = warp_reduce<32>(val, ReduceSum);

```

Acquire/Release Semantics

PTX provides explicit memory ordering semantics for implementing lock-free data structures and synchronization primitives. The acquire/release model ensures that:

- **Release:** All memory writes before a release store are visible to any thread that performs an acquire load of the same location.

- **Acquire:** All memory reads after an acquire load see the writes that happened before the corresponding release store.

This is essential for implementing correct spinlocks, producer-consumer queues, and other synchronization patterns without using expensive full memory barriers.

```
// Store with release semantics (makes prior writes visible)
__device__ void st_release(int* ptr, int val) {
    asm volatile("st.release.gpu.global.s32 [%0], %1;" :: "l"(ptr), "r"(val) : "memory");
}

// Load with acquire semantics (sees prior released writes)
__device__ int ld_acquire(int* ptr) {
    int ret;
    asm volatile("ld.acquire.gpu.global.s32 %0, [%1];" : "=r"(ret) : "l"(ptr));
    return ret;
}

// Spinlock using acquire/release
__device__ void acquire_lock(int* mutex) {
    while (atomicCAS(mutex, 0, 1) != 0); // Simplified; real impl uses PTX
}
__device__ void release_lock(int* mutex) {
    atomicExch(mutex, 0);
}
```

When to Use Each Approach

Feature	Cooperative Groups	PTX Assembly
Readability	High	Low
Portability	Across GPU architectures	May need SM-specific code
Performance	Good	Optimal (hand-tuned)
Memory ordering	Implicit	Explicit control
Use case	Most CUDA code	HPC, ML frameworks

Use Cooperative Groups for most code. Use PTX only when profiling shows it's necessary, such as in communication-heavy distributed training kernels where precise memory ordering and cache control can significantly impact performance.

1.5.5 Memory Visibility

Table of Contents

- *Memory Visibility*
 - *Introduction*
 - *C++ Memory Model*
 - *CUDA Memory Hierarchy*
 - *CUDA Synchronization Primitives*

- *Asynchronous Barriers* (`cuda::barrier`)
- *Semaphores* (`cuda::counting_semaphore`)
- *Latches* (`cuda::latch`)
- *cuda::std::atomic* (`libcu++`)
- *PTX Memory Operations*
- *Common Patterns*
- *Quick Reference*

Introduction

Memory visibility determines when a write by one thread becomes visible to reads by other threads. In parallel programming, this is one of the most subtle and error-prone areas because modern CPUs and GPUs aggressively optimize memory operations. Without proper synchronization, compilers may reorder instructions, hardware may cache values in registers or L1 cache, and writes may be delayed or buffered—leading to bugs that only appear under specific timing conditions or on certain architectures.

The key insight is that **visibility ordering atomicity**:

- **Atomicity**: Operation completes as a single unit (no torn reads/writes)
- **Ordering**: Operations appear in a specific sequence relative to others
- **Visibility**: Writes propagate to other threads/cores/GPUs

C++ Memory Model

C++ provides memory ordering guarantees through `std::atomic` and memory order specifiers. Understanding these concepts is essential for CUDA programming because CUDA's memory model builds on similar principles but adds GPU-specific scopes (block, device, system) to handle the hierarchical nature of GPU memory.

Source

`src/cpp/memory-visibility`

```
#include <atomic>

std::atomic<int> flag{0};
int data = 0;

// Producer thread
data = 42;
flag.store(1, std::memory_order_release); // Prior writes visible

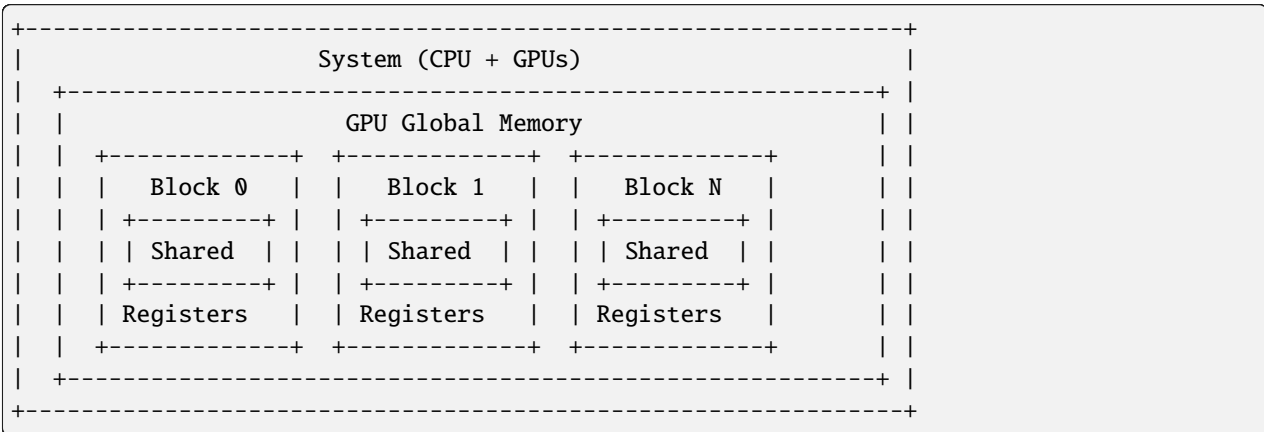
// Consumer thread
while (flag.load(std::memory_order_acquire) == 0); // Sees prior writes
assert(data == 42); // Guaranteed
```

Memory Order Options:

Order	Guarantee
<code>memory_order_relaxed</code>	No ordering, only atomicity
<code>memory_order_acquire</code>	Reads after this see writes before matching release
<code>memory_order_release</code>	Writes before this visible to matching acquire
<code>memory_order_acq_rel</code>	Both acquire and release
<code>memory_order_seq_cst</code>	Total ordering (default, strongest, slowest)

CUDA Memory Hierarchy

CUDA has a hierarchical memory architecture where each level has different visibility characteristics. Writes to shared memory are only visible within the same block, while writes to global memory can be made visible to all GPU threads or even the CPU, depending on the fence used.



CUDA Synchronization Primitives

CUDA provides built-in primitives for synchronization. Barriers like `__syncthreads()` both wait for threads and ensure memory visibility, while fences like `__threadfence()` only order memory operations without waiting. Understanding the difference between volatile, atomics, and fences is critical for correct inter-thread communication.

Source

[src/cuda/memory-visibility-primitives](#)

Barriers vs Fences:

Barriers synchronize thread execution AND memory, while fences only order memory. Use `__syncthreads()` when threads need to wait for each other (e.g., after writing to shared memory). Use `__threadfence()` when you only need to ensure memory writes are visible to other threads without waiting.

Primitive	Waits for Threads	Orders Memory
<code>__syncthreads()</code>	Yes (block)	Yes (block)
<code>__syncwarp(mask)</code>	Yes (warp)	Yes (warp)
<code>__threadfence_block</code>	No	Yes (block)
<code>__threadfence</code>	No	Yes (GPU)
<code>__threadfence_system</code>	No	Yes (system)

```
// Barrier: waits + orders memory
__shared__ int data[256];
```

(continues on next page)

(continued from previous page)

```
data[threadIdx.x] = compute();
__syncthreads(); // All threads reach here, then continue
use(data[...]); // Safe to read any element

// Fence: orders memory only (no wait)
global_data[idx] = value;
__threadfence(); // Ensure write visible before flag
flag = 1; // Other threads may not be here yet
```

Volatile vs Atomic vs Fence:

A common mistake is using `volatile` for synchronization. While `volatile` prevents the compiler from caching values, it provides no hardware guarantees—writes may still be buffered or reordered by the GPU. Atomics guarantee the operation itself is indivisible, but ordering depends on the specific operation. Fences provide explicit ordering without atomicity.

Mechanism	Atomicity	Compiler Fence	Hardware Fence
<code>volatile</code>	No	Yes	No
<code>atomicOp()</code>	Yes	Yes	Varies
<code>__threadfence</code>	No	Yes	Yes

```
// WRONG: volatile doesn't guarantee hardware visibility
volatile int* flag = ...;
*data_ptr = 42;
*flag = 1; // Other SM may see flag=1 but stale data!

// CORRECT: use fence
*data_ptr = 42;
__threadfence();
*flag = 1; // Other SM sees data=42 when flag=1
```

Asynchronous Barriers (cuda::barrier)

`cuda::barrier` extends synchronization beyond `__syncthreads()` by enabling split arrive/wait operations. This allows threads to signal arrival, do other work, then wait—overlapping computation with synchronization. Unlike `__syncthreads()` which blocks immediately, barriers let you decouple the “I’m done” signal from the “wait for others” operation.

Source

[src/cuda/memory-visibility-barrier](#)

Reference: [CUDA Asynchronous Barriers](#)

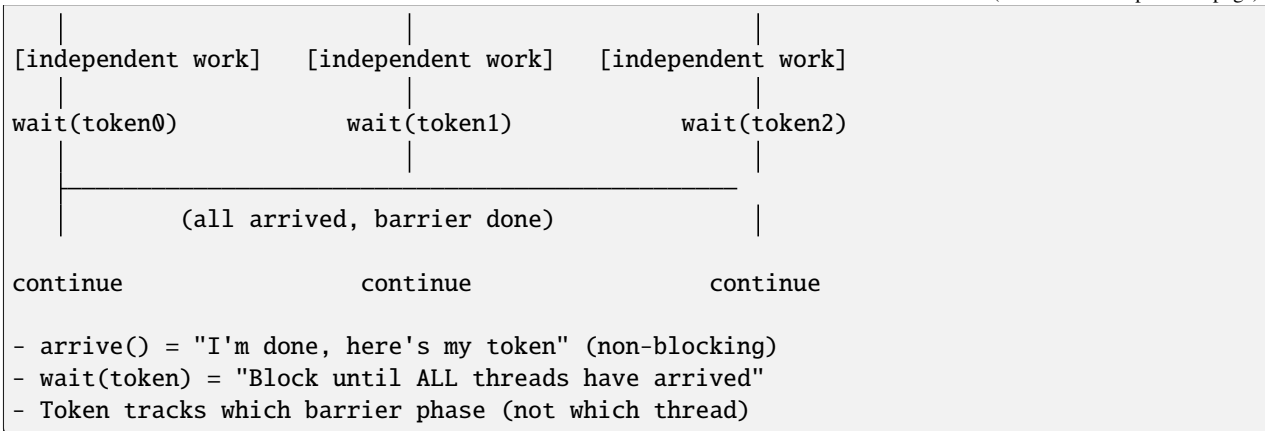
Basic Usage:

A barrier tracks arrivals using tokens. When a thread calls `arrive()`, it gets a token representing the current phase. The thread later calls `wait(token)` to block until all expected arrivals for that phase complete. This split allows independent work between arrive and wait.

Thread 0	Thread 1	Thread 2
arrive() —————	arrive() —————	arrive() —————
token0	token1	token2

(continues on next page)

(continued from previous page)



```

#include <cuda/barrier>
#include <cooperative_groups.h>

__global__ void barrier_example() {
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;
    __shared__ int smem[256];
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0) init(&bar, block.size());
    block.sync();

    smem[threadIdx.x] = compute(); // Write to shared memory
    auto token = bar.arrive();      // Signal "I'm done writing"
    int local = expensive_compute(); // Do independent work while others arrive
    bar.wait(std::move(token));     // Now wait for all arrivals
    use(smem[...], local);         // Safe to read shared memory
}

```

arrive_and_wait vs Split Arrive/Wait:

Pattern	Use Case
bar.arrive_and_wait()	Simple sync, like __syncthreads()
bar.arrive() + work + bar.wait(token)	Overlap computation while waiting
bar.arrive_and_drop()	Thread exits early, reduces expected count

Semaphores (cuda::counting_semaphore)

Semaphores control access to limited resources by maintaining a counter. Threads call `acquire()` to decrement the counter (blocking if zero) and `release()` to increment it. Unlike barriers which synchronize all threads at a point, semaphores limit how many threads can be in a critical section simultaneously.

Source

[src/cuda/libcuxx](#)

counting_semaphore allows up to N concurrent accesses. The template parameter specifies the maximum count. Use for resource pools, rate limiting, or bounded producer-consumer queues.

binary_semaphore (max count = 1) acts as a mutex. Only one thread can hold it at a time. Simpler than a full mutex implementation but provides the same mutual exclusion guarantee.

```

#include <cuda/semaphore>

// Limit to 4 concurrent threads in critical section
__device__ cuda::counting_semaphore<cuda::thread_scope_device, 4> sem{4};

__global__ void limited_concurrency_kernel() {
    sem.acquire(); // Blocks if 4 threads already inside
    // ... critical section (max 4 threads here) ...
    sem.release(); // Allow another thread to enter
}

// Binary semaphore as mutex (only 1 thread at a time)
__device__ cuda::binary_semaphore<cuda::thread_scope_device> mtx{1};

__global__ void mutex_kernel(int* counter) {
    mtx.acquire();
    (*counter)++; // Only one thread executes this at a time
    mtx.release();
}

```

Semaphore vs Barrier vs Atomic:

Primitive	Purpose	Threads Affected
Barrier	All threads sync at a point	All N threads must arrive
Semaphore	Limit concurrent access	Up to N threads proceed
Atomic	Single indivisible operation	One thread at a time per op

Latches (cuda::latch)

A latch is a single-use synchronization primitive. Threads call `count_down()` to decrement an internal counter, and `wait()` blocks until the counter reaches zero. Unlike barriers, latches cannot be reused—once the counter hits zero, the latch is “spent.” This makes latches ideal for one-time initialization or fan-in patterns where multiple threads contribute to a single completion event.

Source

src/cuda/libcuxx

```

#include <cuda/latch>

__global__ void latch_kernel() {
    __shared__ cuda::latch<cuda::thread_scope_block> lat;

    auto block = cooperative_groups::this_thread_block();
    if (block.thread_rank() == 0) {
        init(&lat, block.size()); // Initialize with expected count
    }
    block.sync();

    // Each thread does work then signals completion
    do_work();
    lat.count_down(); // Decrement counter (non-blocking)
}

```

(continues on next page)

(continued from previous page)

```
// Wait for all threads to finish
lat.wait(); // Blocks until counter reaches 0

// All threads proceed together
}

// Combined count_down + wait
lat.arrive_and_wait(); // Equivalent to count_down() followed by wait()
```

Latch vs Barrier:

Aspect	Latch	Barrier
Reusability	Single-use (one-shot)	Reusable (multiple phases)
count_down()	Decrement only	N/A (arrive returns token)
Use case	One-time init, fan-in	Iterative algorithms, phases

cuda::std::atomic (libcu++)

CUDA's libcu++ library provides `cuda::atomic`, which mirrors C++ `std::atomic` but adds explicit thread scope control. This allows you to specify the visibility scope (block, device, or system) and use the same memory order semantics as C++. This is the recommended approach for portable, readable synchronization code.

Source

`src/cuda/memory-visibility-libcu`

Thread Scopes:

```
#include <cuda/atomic>

// Block scope - visible to threads in same block (fastest)
cuda::atomic<int, cuda::thread_scope_block> blk_atomic{0};

// Device scope - visible to all threads on GPU (default)
cuda::atomic<int, cuda::thread_scope_device> gpu_atomic{0};

// System scope - visible to GPU + CPU (slowest)
cuda::atomic<int, cuda::thread_scope_system> sys_atomic{0};
```

Memory Orders (same as C++):

```
// Relaxed - atomicity only, no ordering
val = a.load(cuda::std::memory_order_relaxed);
a.store(1, cuda::std::memory_order_relaxed);
a.fetch_add(1, cuda::std::memory_order_relaxed);

// Acquire - sees writes before matching release
val = a.load(cuda::std::memory_order_acquire);

// Release - prior writes visible to acquire
a.store(1, cuda::std::memory_order_release);

// Acquire-Release - both (for read-modify-write)
```

(continues on next page)

(continued from previous page)

```
old = a.exchange(1, cuda::std::memory_order_acq_rel);

// Sequential consistency - total ordering (default, slowest)
val = a.load(); // seq_cst by default
```

Common Operations:

cuda::atomic	CUDA C-style
a.store(42)	a = 42 (non-atomic)
a.load()	a (non-atomic)
a.exchange(1)	atomicExch(&a, 1)
a.fetch_add(1)	atomicAdd(&a, 1)
a.fetch_sub(1)	atomicSub(&a, 1)
a.fetch_and(mask)	atomicAnd(&a, mask)
a.fetch_or(mask)	atomicOr(&a, mask)
a.fetch_xor(mask)	atomicXor(&a, mask)
a.fetch_min(v)	atomicMin(&a, v)
a.fetch_max(v)	atomicMax(&a, v)
a.compare_exchange_strong(e,d)	atomicCAS(&a, e, d)

Note: CUDA C-style atomics don't support explicit memory ordering—they use device-scope with unspecified ordering. Use `cuda::atomic` when you need specific scopes or memory orders.

PTX Memory Operations

For maximum control over memory ordering, you can use PTX inline assembly. PTX provides explicit acquire/release semantics with configurable scopes (cta, gpu, sys). This is useful when you need finer-grained control than what CUDA built-ins provide, or when implementing lock-free data structures like spinlocks.

Source

`src/cuda/memory-visibility-ptx`

Reference: `DeepEP utils.cuh`

Load/Store with Ordering:

PTX load/store instructions can include ordering semantics directly. An acquire load ensures subsequent operations see writes that happened before the matching release store. This is more efficient than using separate fence instructions.

```
// GPU scope
__device__ int ld_acquire_gpu(const int* ptr) {
    int ret;
    asm volatile("ld.acquire.gpu.global.s32 %0, [%1];" : "=r"(ret) : "l"(ptr));
    return ret;
}

__device__ void st_release_gpu(int* ptr, int val) {
    asm volatile("st.release.gpu.global.s32 [%0], %1;" :: "l"(ptr), "r"(val) : "memory");
}

// System scope (multi-GPU / CPU)
__device__ int ld_acquire_sys(const int* ptr) {
    int ret;
```

(continues on next page)

(continued from previous page)

```

asm volatile("ld.acquire.sys.global.s32 %0, [%1];" : "=r"(ret) : "l"(ptr));
return ret;
}

__device__ void st_release_sys(int* ptr, int val) {
    asm volatile("st.release.sys.global.s32 [%0], %1;" :: "l"(ptr), "r"(val) : "memory");
}

```

Fences:

PTX fences provide standalone memory barriers without being tied to a specific load or store. Use `fence.acq_rel` to ensure all prior writes are visible before subsequent reads. The scope suffix controls visibility range.

```

__device__ void fence_cta() { asm volatile("fence.acq_rel.cta;" ::: "memory"); }
__device__ void fence_gpu() { asm volatile("fence.acq_rel.gpu;" ::: "memory"); }
__device__ void fence_sys() { asm volatile("fence.acq_rel.sys;" ::: "memory"); }

```

PTX Fence vs CUDA Built-in Fence:

CUDA's `__threadfence()` provides sequential consistency—a stronger guarantee that orders all memory operations. PTX `fence.acq_rel` provides acquire-release semantics which is sufficient for most synchronization patterns and may be faster on some architectures. In practice, the difference is often negligible, but PTX gives you explicit control when optimizing hot paths.

Aspect	<code>__threadfence_*</code>	<code>fence.acq_rel.*</code>
Ordering	Sequential consistency	Acquire-release
Strength	Stronger (all ops ordered)	Weaker (only acq/rel)
Performance	Potentially slower	Potentially faster
Use case	Simple, safe default	Fine-tuned performance

PTX Spinlock:

A spinlock protects critical sections where only one thread can execute at a time. Use `atom.cas` with acquire semantics to take the lock (ensuring subsequent reads see prior writes), and `atom.exch` with release semantics to release it (ensuring prior writes are visible to the next lock holder).

```

__device__ void acquire_lock(int* mutex) {
    int ret;
    do {
        asm volatile("atom.acquire.cta.shared::cta.cas.b32 %0, [%1], %2, %3;"
                     : "=r"(ret) : "l"(mutex), "r"(0), "r"(1) : "memory");
    } while (ret != 0);
}

__device__ void release_lock(int* mutex) {
    int ret;
    asm volatile("atom.release.cta.shared::cta.exch.b32 %0, [%1], %2;"
                 : "=r"(ret) : "l"(mutex), "r"(0) : "memory");
}

```

Common Patterns

The producer-consumer pattern is fundamental to GPU programming. One kernel (or thread) produces data and signals completion via a flag, while another waits for the flag and then reads the data. Without proper memory ordering, the consumer may see the flag set but read stale data.

Source

[src/cuda/memory-visibility-patterns](#)

Producer-Consumer (single GPU):

Three approaches with different trade-offs. Method 1 uses CUDA built-ins and is the simplest but requires two fences. Method 2 uses libcu++ atomics which is portable and readable with C++-style syntax. Method 3 uses PTX acquire/release which is the lightest weight because ordering is built into the load/store.

```
// Method 1: __threadfence + atomic (simple, safe)
__global__ void producer() {
    data = 42;
    __threadfence();
    atomicExch(&flag, 1);
}

__global__ void consumer(int* result) {
    while (atomicAdd(&flag, 0) == 0);
    __threadfence();
    *result = data;
}

// Method 2: cuda::atomic (portable, readable)
__device__ cuda::atomic<int, cuda::thread_scope_device> flag{0};
__global__ void producer() {
    data = 42;
    flag.store(1, cuda::std::memory_order_release);
}

__global__ void consumer(int* result) {
    while (flag.load(cuda::std::memory_order_acquire) == 0);
    *result = data;
}

// Method 3: PTX acquire/release (lightest weight)
__global__ void producer() {
    data = 42;
    st_release_gpu(&flag, 1);
}

__global__ void consumer(int* result) {
    while (ld_acquire_gpu(&flag) == 0);
    *result = data;
}
```

Multi-GPU Communication:

For communication between GPUs or between GPU and CPU, use system scope. The data must be in pinned (page-locked) host memory or P2P-enabled device memory. System scope fences are expensive, so minimize their use in performance-critical code.

```
// GPU 0: Write to pinned host memory
*host_data = result;
__threadfence_system(); // Or st_release_sys()
*host_flag = 1;

// GPU 1: Read from pinned host memory
while (*host_flag == 0);
__threadfence_system(); // Or ld_acquire_sys()
use(*host_data);
```

Quick Reference

Use the narrowest scope possible for best performance—block scope is much faster than system scope. Prefer `cuda::atomic` for readability; use PTX only when optimizing hot paths. For higher-level synchronization primitives like semaphores and latches, see [CUDA C++](#).

When to Use What:

Scenario	Recommended Approach
Intra-block sync	<code>__syncthreads()</code> or <code>cuda::atomic<block></code>
Inter-block on same GPU	<code>cuda::atomic<device></code> or PTX acquire/release
Multi-GPU / CPU-GPU	<code>cuda::atomic<system></code> or PTX <code>.sys</code> scope
Critical section (block)	PTX spinlock with acquire/release

1.5.6 CUDA Graph

Table of Contents

- *CUDA Graph*
 - *Why CUDA Graphs*
 - *Stream Capture*
 - *Explicit API*
 - *Graph Update*
 - *Memory Nodes*
 - *Conditional Nodes*
 - *Best Practices*

CUDA Graphs separate the definition of GPU work from its execution. Instead of launching kernels one at a time, a graph captures a sequence of operations (kernels, memory copies, etc.) and their dependencies, then replays the entire graph with a single launch. This eliminates per-launch CPU overhead and enables driver-level optimizations across the whole workflow.

Why CUDA Graphs

Traditional CUDA launches incur CPU overhead for each kernel dispatch. For workloads with many small kernels, this overhead dominates execution time. Graphs amortize this cost by submitting the entire workflow at once. The driver can also optimize memory and scheduling across the graph.

```
// Without graphs: N kernel launches = N times CPU overhead
for (int i = 0; i < N; i++) {
    kernel<<<grid, block, 0, stream>>>(args);
}

// With graphs: 1 launch for the entire workflow
cudaGraphLaunch(instance, stream);
```

Stream Capture

Source

`src/cuda/graph-stream-capture`

Stream capture is the easiest way to create a graph. Existing stream-based code can be wrapped with `cudaStreamBeginCapture` and `cudaStreamEndCapture` without modifying kernel launches. Operations issued to the captured stream are recorded into a graph instead of being executed.

```
cudaGraph_t graph;
cudaStream_t stream;
CUDA_CHECK(cudaStreamCreate(&stream));

// Begin capture
CUDA_CHECK(cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal));

// Record work (not executed yet)
kernelA<<<grid, block, 0, stream>>>(d_data);
kernelB<<<grid, block, 0, stream>>>(d_data);

// End capture and get graph
CUDA_CHECK(cudaStreamEndCapture(stream, &graph));

// Instantiate and launch
cudaGraphExec_t instance;
CUDA_CHECK(cudaGraphInstantiate(&instance, graph, nullptr, nullptr, 0));
CUDA_CHECK(cudaGraphLaunch(instance, stream));
CUDA_CHECK(cudaStreamSynchronize(stream));

// Cleanup
cudaGraphExecDestroy(instance);
cudaGraphDestroy(graph);
cudaStreamDestroy(stream);
```

Capture mode `cudaStreamCaptureModeGlobal` prevents any non-captured stream from launching work while capture is active, catching accidental work outside the graph.

Explicit API

Source

[src/cuda/graph-explicit](#)

Source

[src/cuda/graph-explicit-1d](#)

The explicit API provides fine-grained control over graph topology. Nodes and edges are added manually, which is useful when the dependency structure does not map to a linear stream.

```

cudaGraph_t graph;
CUDA_CHECK(cudaGraphCreate(&graph, 0));

// Add kernel nodes
cudaGraphNode_t nodeA, nodeB, nodeC;
cudaKernelNodeParams paramsA = {};
paramsA.func = (void *)kernelA;
paramsA.gridDim = grid;
paramsA.blockDim = block;
paramsA.kernelParams = argsA;
CUDA_CHECK(cudaGraphAddKernelNode(&nodeA, graph, nullptr, 0, &paramsA));

cudaKernelNodeParams paramsB = {};
paramsB.func = (void *)kernelB;
paramsB.gridDim = grid;
paramsB.blockDim = block;
paramsB.kernelParams = argsB;
// nodeB depends on nodeA
CUDA_CHECK(cudaGraphAddKernelNode(&nodeB, graph, &nodeA, 1, &paramsB));

cudaKernelNodeParams paramsC = {};
paramsC.func = (void *)kernelC;
paramsC.gridDim = grid;
paramsC.blockDim = block;
paramsC.kernelParams = argsC;
// nodeC depends on nodeA (parallel with nodeB)
CUDA_CHECK(cudaGraphAddKernelNode(&nodeC, graph, &nodeA, 1, &paramsC));

// Instantiate and launch
cudaGraphExec_t instance;
CUDA_CHECK(cudaGraphInstantiate(&instance, graph, nullptr, nullptr, 0));

```

This creates a diamond-like dependency: nodeA runs first, then nodeB and nodeC run in parallel.

Graph Update

Source

[src/cuda/graph-update](#)

Updating an existing graph executable avoids the cost of destroying and re-instantiating. Use `cudaGraphExecUpdate` to apply changes from a modified graph to an existing executable. The update succeeds only if the graph topology matches.

```

// Modify the original graph (e.g., change kernel parameters)
cudaGraphExecUpdateResultInfo info;

```

(continues on next page)

(continued from previous page)

```

cudaGraphExecUpdateResult result;
result = cudaGraphExecUpdate(instance, newGraph, &info);
if (result != cudaGraphExecUpdateSuccess) {
    // Topology changed, must re-instantiate
    cudaGraphExecDestroy(instance);
    CUDA_CHECK(cudaGraphInstantiate(&instance, newGraph, nullptr, nullptr, 0));
}

```

For simple parameter changes, `cudaGraphExecKernelNodeSetParams` updates a single kernel node without rebuilding the graph.

```

// Update kernel parameters in-place
paramsA.kernelParams = newArgs;
CUDA_CHECK(cudaGraphExecKernelNodeSetParams(instance, nodeA, &paramsA));

```

Memory Nodes

Source

[src/cuda/graph-memory-nodes](#)

CUDA 11.4+ supports `cudaGraphAddMemAllocNode` and `cudaGraphAddMemFreeNode` to allocate and free memory as part of the graph. This avoids pre-allocating buffers and lets the driver optimize memory reuse across graph launches.

```

cudaMemAllocNodeParams allocParams = {};
allocParams.poolProps.allocType = cudaMemAllocationTypePinned;
allocParams.poolProps.location.type = cudaMemLocationTypeDevice;
allocParams.poolProps.location.id = 0;
allocParams.bytesize = N * sizeof(float);

cudaGraphNode_t allocNode;
CUDA_CHECK(cudaGraphAddMemAllocNode(&allocNode, graph, nullptr, 0, &allocParams));

// The allocated pointer is available after instantiation
float *d_buf = (float *)allocParams.dptr;

```

Conditional Nodes

Source

[src/cuda/graph-conditional](#)

CUDA 12.4+ introduces conditional nodes that enable if/else and loop constructs within a graph. A conditional node evaluates a device-side value to decide which subgraph to execute, removing the need to re-capture or rebuild graphs for data-dependent control flow.

```

cudaGraphConditionalHandle handle;
CUDA_CHECK(cudaGraphConditionalHandleCreate(&handle, graph,
    1, // default value
    cudaGraphCondAssignDefault));

// Create conditional node with IF type
cudaGraphNodeParams cond_params = {};
cond_params.type = cudaGraphNodeTypeConditional;
cond_params.conditional.handle = handle;

```

(continues on next page)

(continued from previous page)

```

cond_params.conditional.type = cudaGraphCondTypeIf;
cond_params.conditional.size = 1;

cudaGraphNode_t condNode;
CUDA_CHECK(cudaGraphAddNode(&condNode, graph, nullptr, 0, &cond_params));

// Populate the body graph (executed when condition is non-zero)
cudaGraph_t bodyGraph = cond_params.conditional.phGraph_out[0];

```

Best Practices

- Prefer stream capture over the explicit API for simpler code
- Reuse graph executables across iterations instead of re-instantiating
- Use `cudaGraphExecUpdate` for parameter changes with the same topology
- Profile with `nsys` to verify launch overhead reduction
- Avoid capturing operations with variable topology (use conditional nodes instead)
- Graph instantiation is expensive; do it once and replay many times

1.5.7 CUDA Quantization

Table of Contents

- *CUDA Quantization*
 - *Glossary*
 - *FP8 E4M3 Format*
 - *Quantization Math*
 - *Static Per-Tensor Quantization*
 - *Dynamic Per-Tensor Quantization*
 - *Dynamic Per-Token Quantization*
 - *Best Practices*

Quantization reduces the precision of tensor values (e.g., FP32/FP16 → FP8) to cut memory bandwidth and storage by 2–4×, with minimal accuracy loss. Modern LLM inference engines like vLLM use FP8 quantization on Hopper (H100) and Ada Lovelace (RTX 4090) GPUs, which have native FP8 tensor core support.

Glossary

Term	Definition
MMA	Matrix Multiply-Accumulate. The fundamental tensor core instruction that computes $D = A \times B + C$ on small matrix tiles in hardware. Evolved across generations: <code>wmma</code> (Volta, warp-level) $\rightarrow$ <code>mma</code> (Ampere, finer PTX control) $\rightarrow$ <code>wgmma</code> (Hopper, warp-group level with TMA).
wgmma	Warp Group MMA. Hopper's tensor core instruction where 4 warps (128 threads) cooperate on larger tiles, fed asynchronously via TMA.
TMA	Tensor Memory Accelerator. Hopper hardware unit for async bulk data movement between global memory and shared memory, bypassing the register file.
W8A8	Weight 8-bit, Activation 8-bit. Both weights and activations quantized to 8-bit (FP8 or INT8). The scheme used in vLLM's FP8 path.
WnA16	Weight n-bit, Activation 16-bit (e.g., W4A16). Weights quantized to n bits, activations stay in FP16/BF16. Weight-only quantization.
AWQ	Activation-aware Weight Quantization. A 4-bit weight-only method that selects which channels to keep at higher precision based on activation importance rather than weight magnitude.
GPTQ	Post-training quantization using approximate second-order (Hessian) information to minimize quantization error layer by layer. Typically 4-bit weights.
Marlin	High-performance 4-bit weight-only GEMM kernel (W4A16) optimized for NVIDIA GPUs, used in vLLM for W4A16 inference. See IST-DASLab/marlin .
SmoothQuant	Technique from Xiao et al., 2023 that migrates quantization difficulty from activations to weights. Activations often have outlier channels (a few columns with very large values) that make them hard to quantize, while weights are usually smooth. SmoothQuant divides activations by a per-channel factor s and multiplies weights by the same s : $Y = (X / s) @ (W * s)$. After smoothing, both sides have similar dynamic ranges and can be quantized to INT8/FP8 with less error. The factor s is calibrated offline from a small dataset. Used in vLLM and TensorRT-LLM as a preprocessing step before W8A8 INT8 quantization. Less critical for FP8 which handles outliers better than INT8.
mm	Matrix Multiply (GEMM). A single matmul kernel.
group_mm	Grouped matrix multiply. Batched GEMMs where each batch element can have different sizes, used in Mixture-of-Experts (MoE) models.
c2x / c3x	CUTLASS 2.x / 3.x kernel architecture. c2x uses classic templates; c3x uses CuTe layout algebra and wgmma (Hopper). c3x is significantly faster on sm_90+.
E4M3 / E5M2	Two FP8 variants. E4M3 (4 exponent, 3 mantissa, range ± 448) for forward pass inference. E5M2 (5 exponent, 2 mantissa, range ± 57344) for gradients during training (wider range, less precision).

FP8 E4M3 Format

FP8 E4M3 (`__nv_fp8_e4m3`) uses 1 sign + 4 exponent + 3 mantissa bits:

```

Bit layout:  S EEEE MMM
Range:       [-448, 448]
Precision:   ~3 significant decimal digits near 1.0
Special:     No Inf (overflow saturates to  $\pm 448$ ), NaN exists

```

Compared to FP16 (range ± 65504 , 10-bit mantissa), FP8 trades precision for 2× memory savings. The key insight is that neural network weights and activations are approximately normally distributed, so most values cluster near zero where FP8 has adequate precision.

Quantization Math

```
Quantize:    fp8_val = clamp(fp32_val / scale, -448, 448)
Dequantize:  fp32_val = fp8_val × scale
```

The scale factor maps the tensor’s dynamic range into FP8’s representable range. Choosing the right scale is the core challenge:

Strategy	Scale Computation	Use Case
Static per-tensor	<code>scale = max(tensor) / 448</code> (offline calibration)	Weights (fixed after training)
Dynamic per-tensor	Same formula, computed at runtime	Activations (when all tokens share similar range)
Dynamic per-token	<code>scale[i] = max(row_i) / 448</code> (one per row)	Activations (vLLM W8A8 default, best accuracy)

Static Per-Tensor Quantization

Source

[src/cuda/quant-fp8-static](#)

Static quantization uses a scale factor that is pre-computed offline during a calibration step and then baked into the model as a constant. This is the standard approach for **weight quantization** in LLM inference: the model weights are fixed after training, so their dynamic range is known ahead of time. During calibration, a representative dataset is run through the model to record the maximum absolute value of each weight tensor, and the scale is set to `max(|weights|) / 448`.

At inference time, the kernel simply multiplies each element by the reciprocal of the scale (`1.0f / scale`), clamps the result to the FP8 representable range `[-448, 448]`, and converts to `__nv_fp8_e4m3`. Using the reciprocal avoids a per-element floating-point division, which is significantly more expensive than multiplication on GPU hardware.

```
// Core quantization operation (per-element)
float inv_scale = 1.0f / (*scale);
float val = input[i] * inv_scale;
val = fmaxf(-FP8_E4M3_MAX, fminf(val, FP8_E4M3_MAX));
out[i] = __nv_fp8_e4m3(val);
```

The clamping step is critical: FP8 E4M3 has no representation for infinity, so values outside `[-448, 448]` would produce NaN without clamping. The `__nv_fp8_e4m3()` constructor performs the actual bit conversion using hardware instructions on sm_89+ GPUs.

This maps directly to vLLM’s `scaled_fp8_quant_kernel_strided`, which uses the same multiply-clamp-convert pattern. Dequantization is the reverse: multiply the FP8 value by the scale to recover an approximate FP32 value. The round-trip error is bounded by the quantization step size, which depends on the magnitude of the original value and the scale factor.

Dynamic Per-Tensor Quantization

Source

[src/cuda/quant-fp8-dynamic](#)

When quantizing **activations** (the intermediate tensors flowing between layers), the value range is not known ahead of time — it changes with every input. Dynamic quantization solves this by computing the scale at runtime from the actual tensor data. This requires a two-pass approach because the scale depends on a global property (the maximum absolute value) that no single thread can compute alone.

Pass 1 — Absmax Reduction: Each thread block scans a portion of the tensor, finding the local maximum absolute value via a strided loop. The per-thread maximums are reduced within the block using shared memory. The block's result is then merged into a global scale value using an atomic operation. Since CUDA has no native `atomicMax` for floats, the standard trick is to reinterpret the float bits as an integer. This works because IEEE 754 positive floats have the same ordering as unsigned integers when viewed as bit patterns:

```
// atomicMax for non-negative floats via int reinterpretation
// Works because IEEE 754 floats sort the same as ints when positive
__device__ float atomicMaxFloat(float* addr, float value) {
    return __int_as_float(
        atomicMax((__int*)addr, __float_as_int(value)));
}
```

The reduction kernel writes `scale = absmax / FP8_E4M3_MAX` to device memory. The scale must be initialized to zero before the kernel launch (via `cudaMemsetAsync`) so that the atomic max starts from a clean baseline.

Pass 2 — Quantize: A second kernel reads the computed scale from device memory and quantizes every element using the same multiply-clamp-convert pattern as static quantization. The two-kernel design avoids the need for a global barrier (which CUDA does not support within a single kernel launch across blocks). The scale lives in device memory between the two passes, so no device-to-host round-trip is needed.

```
Pass 1: absmax reduction → scale = absmax / 448
Pass 2: quantize using the computed scale
```

This two-pass pattern matches vLLM's `segmented_max_reduction_strided` followed by `scaled_fp8_quant_kernel_strided_dynamic`. The overhead of the extra kernel launch is negligible compared to the memory bandwidth cost of scanning the tensor twice, but it is the simplest correct approach without cooperative groups or multi-block synchronization.

Dynamic Per-Token Quantization

Source

`src/cuda/quant-fp8-per-token`

Per-token quantization assigns a **separate scale to each row** of a `[tokens × hidden]` matrix. This is the default activation quantization strategy in vLLM's W8A8 path (`dynamic_per_token_scaled_fp8_quant_kernel_strided`) and provides the best accuracy among the three approaches.

Why per-token matters: In a transformer, different tokens in a batch can have wildly different activation magnitudes. A prompt token early in the sequence might have activations in the range `[-5, 5]`, while a later token might spike to `[-300, 300]`. With a single per-tensor scale, the scale is dominated by the largest token, and smaller tokens use only a tiny fraction of the FP8 range, wasting most of the 3-bit mantissa precision:

```
Token 0 (small activations): scale[0] = 0.02 → full FP8 range utilized
Token 1 (large activations): scale[1] = 0.71 → full FP8 range utilized
```

vs. per-tensor scale = 0.71 → Token 0 uses only 3% of FP8 range (wasted precision)

Single-pass kernel design: Unlike the per-tensor dynamic approach which needs two separate kernels, per-token quantization can be done in a single kernel because each block handles exactly one row. There is no cross-block dependency — each block independently finds its row's absmax, computes the scale, and quantizes. The three phases within each block are separated by `__syncthreads()` barriers:

```
// Phase 1: Strided scan to find row absmax
// Each thread reads elements at stride=blockDim.x, tracking its local max.
// This coalesces memory accesses across threads in the warp.
```

(continues on next page)

(continued from previous page)

```

float local_max = 0.0f;
for (int col = tid; col < hidden_size; col += blockDim.x)
    local_max = fmaxf(local_max, fabsf(row_in[col]));

// Phase 2: Shared memory reduction → thread 0 writes scale[row]
// Classic parallel reduction: each iteration halves the active threads,
// combining pairs of maximums until thread 0 holds the row's global max.
// ... block reduction in shared memory ...
if (tid == 0) scales[row] = fmaxf(smem[0] / FP8_E4M3_MAX, MIN_SCALE);
__syncthreads();

// Phase 3: Quantize with per-row scale
// All threads read the shared row_scale and quantize their elements.
// The same strided access pattern ensures coalesced writes to the output.
float inv_scale = 1.0f / row_scale;
for (int col = tid; col < hidden_size; col += blockDim.x) {
    float val = row_in[col] * inv_scale;
    row_out[col] = __nv_fp8_e4m3(fmaxf(-448, fminf(val, 448)));
}

```

The MIN_SCALE floor (typically $1 / (448 * 512)$) prevents division by zero for all-zero rows. Without it, a row of zeros would produce `scale = 0`, and the subsequent `1.0f / scale` would yield infinity, corrupting the output.

The output scales array has shape `[num_tokens]` and must be carried alongside the quantized tensor for dequantization. During matrix multiplication, the per-token activation scales are combined with per-channel weight scales to produce the final dequantization factor.

Best Practices

- Use per-token scaling for activations — it preserves accuracy across tokens with different magnitude distributions
- Use static per-tensor scaling for weights — they don't change at inference time
- Always clamp before converting to FP8 to avoid NaN (E4M3 has no Inf)
- Use `1.0f / scale` (reciprocal multiply) instead of per-element division
- Initialize the scale to 0 before dynamic reduction kernels
- Guard against all-zero inputs with a minimum scale floor
- FP8 requires sm_89+ (Ada Lovelace) or sm_90+ (Hopper); check with `cudaDeviceGetAttribute(&major, cudaDevAttrComputeCapabilityMajor, dev)`

1.5.8 Pipelines

Table of Contents

- *Pipelines*
 - *Introduction*
 - * *Pipelines vs Async Data Copies*
 - * *Async Copy with Barrier (No Pipeline)*

- *Quick Reference*
 - * *Headers*
 - * *Pipeline Lifecycle*
 - * *Block-Scope Pipeline*
 - * *Partitioned Pipeline*
 - * *Wait Variants*
 - * *Async Copy Without Pipeline*
 - * *Primitives API (Legacy)*
 - * *Common Patterns*
 - * *Gotchas*
- *Double Buffering Pattern*
- *Multi-Stage Pipeline*
- *Warp Entanglement*
- *Early Exit*
- *API Comparison*
- *Real-World Example: Multi-LoRA Serving*

Introduction

Pipelines coordinate multi-buffer producer-consumer patterns to overlap compute with asynchronous data copies. While one buffer is being processed, another is being filled - hiding memory latency.

Without Pipeline:

```
[Copy A][Compute A][Copy B][Compute B][Copy C][Compute C]
```

With Pipeline (2 stages):

```
[Copy A][Copy B  ][Copy C  ]
      [Compute A][Compute B][Compute C]
```

↑ Overlapped execution

Pipelines vs Async Data Copies

Pipelines and async data copies are related but different abstraction levels:

cuda::pipeline (Multi-stage orchestration: acquire/commit/wait/release)
Async Data Copy APIs cuda::memcpy_async, cooperative_groups::memcpy_async
Hardware Instructions LDGSTS (CC 8.0+), TMA (CC 9.0+), STAS (CC 9.0+)

- **Async Data Copies:** The underlying copy mechanism (global → shared memory). Uses LDGSTS (Load Global, Store Shared) instruction - a single PTX op that bypasses registers to copy directly. Completion signaled via `cuda::barrier` or pipeline.
- **Pipelines:** Orchestration layer for multi-stage prefetching. Adds state machine (acquire/commit/wait/release) to enable double/multi-buffering.

When to use what:

- **Single async copy:** Use `cuda::memcpy_async` + `cuda::barrier`
- **Overlapping load/compute:** Use `cuda::pipeline` with `memcpy_async`
- **CC 9.0+ bulk transfers:** Consider TMA (Tensor Memory Accelerator)

Async Copy with Barrier (No Pipeline)

Use this simpler pattern when you need a single async copy without multi-stage orchestration. Common scenarios: loading a tile once before a reduction, or prefetching read-only data at kernel start. The barrier tracks copy completion without the overhead of pipeline state management.

```
#include <cuda/barrier>
#include <cooperative_groups.h>

__global__ void kernel(float* global_data) {
    namespace cg = cooperative_groups;
    __shared__ float smem[256];
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;

    auto block = cg::this_thread_block();
    if (block.thread_rank() == 0) {
        init(&bar, block.size()); // Initialize barrier
    }
    block.sync();

    // Async copy: global → shared
    cuda::memcpy_async(block, smem, global_data, sizeof(smem), bar);

    bar.arrive_and_wait(); // Wait for copy completion

    // Use smem here...
}
```

Quick Reference

Essential APIs and patterns for CUDA pipelines. Use `cuda::pipeline` for multi-stage prefetching, or `cuda::barrier` with `memcpy_async` for simpler single-copy scenarios.

Headers

```
#include <cuda/pipeline> // cuda::pipeline, memcpy_async
#include <cuda/barrier> // cuda::barrier (no pipeline)
#include <cuda/pipeline.h> // __pipeline_* primitives
```

Pipeline Lifecycle

Thread-scope pipeline is the simplest form - each thread manages its own pipeline state independently. Use for per-thread prefetching patterns.

```
// Flow: acquire → copy → commit → wait → use → release
//
//
//   ┌───┐ ┌───┐ ┌───┐
//   │acquire│─│copy│─│commit│ ← Producer
//   └───┘ └───┘ └───┘
//                                     |
//   ┌───┐ ┌───┐ ┌───┐
//   │release│─│use│─│wait│ ← Consumer
//   └───┘ └───┘ └───┘

cuda::pipeline<cuda::thread_scope_thread> pipe = cuda::make_pipeline();

pipe.producer_acquire();           // Acquire stage
cuda::memcpy_async(dst, src, size, pipe); // Submit async copy
pipe.producer_commit();            // Commit stage

pipe.consumer_wait();              // Wait for oldest stage
// ... use data ...
pipe.consumer_release();           // Release stage for reuse
```

Block-Scope Pipeline

Block-scope pipelines synchronize across all threads in a block. Required when threads cooperatively load shared memory tiles.

```
// All threads share pipeline state in shared memory
//
// Thread 0 - - - Thread 0
// Thread 1 — [shared] — Thread 1
// Thread N - pipeline └ Thread N

constexpr int stages = 2;
__shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, stages> shared_state;
auto pipe = cuda::make_pipeline(block, &shared_state);
```

Partitioned Pipeline

Separate producer and consumer threads for asymmetric workloads. Producers handle data loading while consumers process - useful when compute is heavier than copy.

```
// Threads 0-63: producers   Threads 64-127: consumers
//
//   ┌───┐ ┌───┐ ┌───┐
//   │load │─[smem]─│compute│
//   └───┘ └───┘ └───┘

size_t num_producers = block.size() / 2;
auto pipe = cuda::make_pipeline(block, &shared_state, num_producers);
```

(continues on next page)

(continued from previous page)

```
// Or explicit roles
auto role = (threadIdx.x < 64) ? cuda::pipeline_role::producer
                               : cuda::pipeline_role::consumer;
auto pipe = cuda::make_pipeline(block, &shared_state, role);
```

Wait Variants

`wait_prior<N>` keeps N stages in-flight while waiting. Use $N > 0$ to maintain overlap when you need to wait but want to keep prefetching ahead.

```
// wait_prior<0>: wait for ALL stages (same as consumer_wait)
// wait_prior<1>: keep 1 in-flight, wait for rest
// wait_prior<2>: keep 2 in-flight, wait for rest
//
//   Stages: [0][1][2][3]   wait_prior<2>
//             └── kept in-flight
//             └── waited
//
pipe.consumer_wait(); // Wait for oldest (tail) stage
cuda::pipeline_consumer_wait_prior<N>(pipe); // Wait for all except last N stages
```

Async Copy Without Pipeline

Simpler alternative when you only need one async copy without multi-stage orchestration. Use for single-tile loads or read-only data prefetch.

```
// Single async copy with barrier (no pipeline overhead)
//
//   [init barrier] → [async copy] → [arrive_and_wait] → [use data]
//
__shared__ cuda::barrier<cuda::thread_scope_block> bar;
init(&bar, block.size());
cuda::memcpy_async(block, dst, src, bytes, bar);
bar.arrive_and_wait();
```

Primitives API (Legacy)

Lower-level C-style API. Prefer `cuda::pipeline` for new code, but primitives work on older CUDA versions and have less overhead.

```
__pipeline_memcpy_async(dst, src, size); // Submit async copy
__pipeline_commit();                     // Commit current batch
__pipeline_wait_prior(N);                 // Wait for all except last N (0 = wait all)
```

Common Patterns

- Double buffer: `buf[2][TILE], stage = t % 2`
- Multi-stage: `buf[STAGES][TILE], stage = t % STAGES`
- Early exit: `pipe.quit(); return;`

Gotchas

- `__syncwarp()` before commit to avoid warp entanglement
- `pipe.quit()` before early return (block-scope)
- `Stages = memory_latency / compute_time` (typically 2-4)
- LDGSTS (Load Global, Store Shared) requires CC 8.0+, TMA requires CC 9.0+

Double Buffering Pattern

Source

`src/cuda/pipeline`

The most common use case: process one buffer while loading the next. This hides memory latency by overlapping data transfer with computation. The key insight is that global memory loads take hundreds of cycles, but with pipelining, those cycles are spent doing useful compute work instead of waiting.

```
// Without pipeline (sequential):
// [Load T0][Proc T0][Load T1][Proc T1][Load T2][Proc T2]
// ↑ GPU idle during loads
//
// With double buffering:
// Stage 0: [Load T0]           [Load T2]
// Stage 1:           [Load T1]           [Load T3]
// Compute:           [Proc T0][Proc T1][Proc T2][Proc T3]
// ↑ Loads overlap with compute

__global__ void double_buffer(const float* in, float* out, int n) {
    constexpr int TILE = 256;
    __shared__ float buf[2][TILE]; // Double buffer

    cuda::pipeline<cuda::thread_scope_thread> pipe = cuda::make_pipeline();

    int tid = threadIdx.x;
    int num_tiles = (n + TILE - 1) / TILE;

    // Prefetch first tile
    pipe.producer_acquire();
    if (tid < TILE && tid < n)
        cuda::memcpy_async(&buf[0][tid], &in[tid], sizeof(float), pipe);
    pipe.producer_commit();

    for (int t = 0; t < num_tiles; t++) {
        int curr = t % 2;
        int next = (t + 1) % 2;

        // Prefetch next tile (if exists)
        if (t + 1 < num_tiles) {
            pipe.producer_acquire();
            int idx = (t + 1) * TILE + tid;
            if (tid < TILE && idx < n)
                cuda::memcpy_async(&buf[next][tid], &in[idx], sizeof(float), pipe);
            pipe.producer_commit();
        }
    }
}
```

(continues on next page)

(continued from previous page)

```

// Wait for current tile
pipe.consumer_wait();
__syncthreads();

// Process current tile
int idx = t * TILE + tid;
if (tid < TILE && idx < n)
    out[idx] = buf[curr][tid] * 2.0f;

pipe.consumer_release();
__syncthreads();
}
}

```

Multi-Stage Pipeline

More stages = more latency hiding, but more shared memory. Use 2-4 stages typically. More stages help when memory latency is high relative to compute time. The number of stages should match the ratio of memory latency to compute time.

```

// 4-stage pipeline processing 8 tiles:
// Stage 0: [T0]          [T4]
// Stage 1:   [T1]          [T5]
// Stage 2:      [T2]          [T6]
// Stage 3:        [T3]          [T7]
// Compute:          [T0][T1][T2][T3][T4][T5][T6][T7]

template<int STAGES>
__global__ void multi_stage(const float* in, float* out, int n) {
    constexpr int TILE = 128;
    __shared__ float buf[STAGES][TILE];

    cuda::pipeline<cuda::thread_scope_thread> pipe = cuda::make_pipeline();

    int tid = threadIdx.x;
    int num_tiles = (n + TILE - 1) / TILE;

    // Fill pipeline with initial stages
    for (int s = 0; s < STAGES && s < num_tiles; s++) {
        pipe.producer_acquire();
        int idx = s * TILE + tid;
        if (tid < TILE && idx < n)
            cuda::memcpy_async(&buf[s][tid], &in[idx], sizeof(float), pipe);
        pipe.producer_commit();
    }

    // Process tiles
    for (int t = 0; t < num_tiles; t++) {
        int stage = t % STAGES;

        // Prefetch future tile

```

(continues on next page)

(continued from previous page)

```

    if (t + STAGES < num_tiles) {
        pipe.producer_acquire();
        int idx = (t + STAGES) * TILE + tid;
        if (tid < TILE && idx < n)
            cuda::memcpy_async(&buf[stage][tid], &in[idx], sizeof(float), pipe);
        pipe.producer_commit();
    }

    // Wait and process
    pipe.consumer_wait();
    __syncthreads();

    int idx = t * TILE + tid;
    if (tid < TILE && idx < n)
        out[idx] = buf[stage][tid] * 2.0f;

    pipe.consumer_release();
    __syncthreads();
}

```

Warp Entanglement

Pipeline commits are coalesced within a warp. When threads diverge before commit, each diverged thread creates a separate stage, spreading operations across many stages. This causes `consumer_wait()` to wait longer than expected because it waits for the actual (warp-shared) sequence, not the thread's perceived sequence.

Converged warp (good):
All 32 threads commit together → 1 stage created

Diverged warp (bad):
Thread 0 commits alone → stage 0
Thread 1 commits alone → stage 1
...
Thread 31 commits alone → stage 31
`consumer_wait()` waits for ALL 32 stages!

```

// BAD: Divergent commit spreads operations across many stages
if (some_condition) {
    pipe.producer_acquire();
    cuda::memcpy_async(...);
    pipe.producer_commit(); // Each diverged thread commits separately
}

// GOOD: Reconverge before commit
if (some_condition) {
    pipe.producer_acquire();
    cuda::memcpy_async(...);
}
__syncwarp(); // Reconverge
pipe.producer_commit(); // Single coalesced commit

```

Early Exit

When using block-scope pipelines, threads exiting early must call `quit()` to properly drop out of participation. Otherwise, remaining threads may deadlock waiting for the exited thread at synchronization points.

```
if (should_exit) {
    pipe.quit();
    return;
}
```

API Comparison

cuda::pipeline	Primitives
<code>producer_acquire()</code>	(implicit)
<code>memcpy_async(..., pipe)</code>	<code>__pipeline_memcpy_async()</code>
<code>producer_commit()</code>	<code>__pipeline_commit()</code>
<code>consumer_wait()</code>	<code>__pipeline_wait_prior(0)</code>
<code>consumer_release()</code>	(implicit)
<code>wait_prior<N></code>	<code>__pipeline_wait_prior(N)</code>

Real-World Example: Multi-LoRA Serving

vLLM's multi-LoRA implementation uses pipelines to overlap loading LoRA weight tiles with matrix-vector computation. Each request may use a different LoRA adapter, so weights are gathered per-batch and pipelined to hide latency.

Source

Punica BGMV kernel (vLLM integration PR #1804)

Multi-tenant LoRA serving:

Request 1 → LoRA adapter A	→ Batched inference with weight gathering
Request 2 → LoRA adapter B	
Request 3 → LoRA adapter A	

Pipeline hides weight loading latency:

```
Stage 0: [Load W tile 0][Load W tile 2]...
Stage 1:      [Load W tile 1][Load W tile 3]...
Compute:      [GEMV tile 0 ][GEMV tile 1 ]...
```

Simplified pattern from the BGMV kernel:

```
// Double-buffered weight loading for batched LoRA GEMV
template<int feat_in, int feat_out>
__global__ void lora_gemv_pipelined(
    float* Y, const float* X, const float* W,
    const int64_t* lora_indices, int batch_size) {

    constexpr int STAGES = 2;
    constexpr int TILE = 128; // tx * ty * vec_size
    __shared__ float W_shared[STAGES][TILE];
    __shared__ float X_shared[STAGES][TILE];
```

(continues on next page)

(continued from previous page)

```

auto block = cg::this_thread_block();
int batch_idx = blockIdx.y;
int64_t lora_idx = lora_indices[batch_idx];

auto pipe = cuda::make_pipeline();

// Kick off first tile load
pipe.producer_acquire();
cuda::memcpy_async(W_shared[0] + threadIdx.x,
                   W + lora_idx * feat_out * feat_in + threadIdx.x,
                   sizeof(float), pipe);
cuda::memcpy_async(X_shared[0] + threadIdx.x,
                   X + batch_idx * feat_in + threadIdx.x,
                   sizeof(float), pipe);
pipe.producer_commit();

float y = 0.f;
for (int tile = 1; tile < feat_in / TILE; tile++) {
    int copy_stage = tile % STAGES;
    int compute_stage = (tile - 1) % STAGES;

    // Async load next tile
    pipe.producer_acquire();
    cuda::memcpy_async(W_shared[copy_stage] + threadIdx.x,
                       W + lora_idx * feat_out * feat_in + tile * TILE + threadIdx.x,
                       sizeof(float), pipe);
    cuda::memcpy_async(X_shared[copy_stage] + threadIdx.x,
                       X + batch_idx * feat_in + tile * TILE + threadIdx.x,
                       sizeof(float), pipe);
    pipe.producer_commit();

    // Compute on previous tile
    pipe.consumer_wait();
    block.sync();
    y += W_shared[compute_stage][threadIdx.x] * X_shared[compute_stage][threadIdx.x];
    // ... warp reduction ...
    pipe.consumer_release();
    block.sync();
}

// Final tile
pipe.consumer_wait();
// ... final compute and write Y ...
}

```

Key techniques from vLLM's implementation:

- **Batched gather:** Each request indexes into different LoRA weights via `lora_indices`
- **Double buffering:** 2 pipeline stages overlap load/compute
- **Vectorized loads:** Uses `vec_t` for coalesced 8-element transfers
- **Warp reduction:** Partial sums reduced within warps using `__shfl_down_sync`

1.5.9 NCCL

Table of Contents

- *NCCL*
 - *Bootstrap*
 - * *Single-Process (ncclCommInitAll)*
 - * *Multi-Process with MPI (ncclCommInitRank)*
 - *AllReduce*
 - *Broadcast*
 - *Reduce*
 - *AllGather*
 - *ReduceScatter*
 - *Grouping Collectives*
 - *CUDA Graph Capture*
 - *Best Practices*

NCCL (pronounced “Nickel”) is NVIDIA’s library for multi-GPU and multi-node collective communication. It implements optimized primitives like AllReduce, Broadcast, and AllGather that automatically exploit the topology — NVLink, PCIe, InfiniBand — to maximize bandwidth. NCCL is the communication backbone of most distributed deep learning frameworks (PyTorch DDP, Megatron-LM, DeepSpeed).

Bootstrap

Before any collective can run, every participating GPU needs a communicator. NCCL provides two initialization paths depending on whether you run single-process or multi-process.

Single-Process (ncclCommInitAll)

When one process owns all GPUs on a single node, `ncclCommInitAll` creates communicators for every device in one call. No ID exchange is needed.

```
int nDev;
cudaGetDeviceCount(&nDev);
std::vector<ncclComm_t> comms(nDev);
std::vector<int> devs(nDev);
std::iota(devs.begin(), devs.end(), 0); // {0, 1, ..., nDev-1}
ncclCommInitAll(comms.data(), nDev, devs.data());
```

This is the simplest setup and what all the examples below use.

Multi-Process with MPI (ncclCommInitRank)

For multi-node or one-GPU-per-process setups, rank 0 generates a unique ID and broadcasts it to all ranks via MPI (or any out-of-band channel). Each rank then calls `ncclCommInitRank` with the shared ID.

```
#include <mpi.h>
#include <nccl.h>

int rank, nRanks;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &nRanks);

// Rank 0 generates the unique ID
ncclUniqueId id;
if (rank == 0) ncclGetUniqueId(&id);

// Broadcast ID to all ranks (ncclUniqueId is a 128-byte opaque struct)
MPI_Bcast(&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD);

// Each rank selects its GPU and initializes its communicator
cudaSetDevice(rank % numLocalGPUs);
ncclComm_t comm;
ncclCommInitRank(&comm, nRanks, id, rank);
```

The `ncclUniqueId` can be exchanged via any mechanism — MPI, TCP sockets, shared filesystem, etc. MPI is the most common choice because distributed GPU workloads typically already use MPI for process management.

AllReduce

Source

`src/cuda/nccl-allreduce`

AllReduce combines values from all ranks with a reduction operation and distributes the result back to every rank. This is the most common collective in distributed training — used to synchronize gradients across data-parallel workers.

```
// Before: rank0=[1,2,3] rank1=[4,5,6]
// After:  rank0=[5,7,9] rank1=[5,7,9] (element-wise sum)
//
// Supports ncclSum, ncclProd, ncclMax, ncclMin, ncclAvg
// sendbuff and recvbuff can be the same pointer (in-place)

constexpr int count = 1024;
std::vector<float> h_send(count);
std::iota(h_send.begin(), h_send.end(), 0.0f); // [0, 1, 2, ...]
float *d_send, *d_recv;
cudaSetDevice(devId);
cudaMalloc(&d_send, count * sizeof(float));
cudaMalloc(&d_recv, count * sizeof(float));
cudaMemcpy(d_send, h_send.data(), count * sizeof(float), cudaMemcpyHostToDevice);

ncclAllReduce(d_send, d_recv, count, ncclFloat, ncclSum, comm, stream);
cudaStreamSynchronize(stream);
```

Broadcast

Source

`src/cuda/nccl-broadcast`

Broadcast copies data from one root rank to all other ranks. Useful for distributing model weights or configuration data at initialization.

```
// Before:  rank0=[1,2,3]  rank1=[0,0,0]
// After:   rank0=[1,2,3]  rank1=[1,2,3]  (root=0)
//
// On root, sendbuff is the source. On non-root, sendbuff is ignored.

constexpr int count = 1024;
constexpr int root = 0;
std::vector<float> h_buf(count);
std::iota(h_buf.begin(), h_buf.end(), 0.0f); // [0, 1, 2, ...]
float *d_buf;
cudaSetDevice(devId);
cudaMalloc(&d_buf, count * sizeof(float));
if (devId == root) cudaMemcpy(d_buf, h_buf.data(), count * sizeof(float),
    ↪ cudaMemcpyHostToDevice);
ncclBroadcast(d_buf, d_buf, count, ncclFloat, root, comm, stream);
cudaStreamSynchronize(stream);
```

Reduce

Source

src/cuda/nccl-reduce

Reduce combines values from all ranks but stores the result only on the root rank. Other ranks' receive buffers are untouched.

```
// Before:  rank0=[1,2,3]  rank1=[4,5,6]
// After:   rank0=[5,7,9]  rank1=[...]  (root=0, only root gets result)

constexpr int count = 1024;
constexpr int root = 0;
std::vector<float> h_send(count, static_cast<float>(rank + 1));
float *d_send, *d_recv;
cudaSetDevice(devId);
cudaMalloc(&d_send, count * sizeof(float));
cudaMalloc(&d_recv, count * sizeof(float));
cudaMemcpy(d_send, h_send.data(), count * sizeof(float), cudaMemcpyHostToDevice);

ncclReduce(d_send, d_recv, count, ncclFloat, ncclSum, root, comm, stream);
cudaStreamSynchronize(stream);
```

AllGather

Source

src/cuda/nccl-allgather

AllGather concatenates each rank's buffer into a single output on every rank. Each rank contributes sendcount elements and receives sendcount * nRanks elements.

```
// Before:  rank0=[A]      rank1=[B]
// After:   rank0=[A,B]    rank1=[A,B]

constexpr int sendcount = 512;
int recvcnt = sendcount * nRanks;
std::vector<float> h_send(sendcount, static_cast<float>(rank));
```

(continues on next page)

(continued from previous page)

```
float *d_send, *d_recv;
cudaSetDevice(devId);
cudaMalloc(&d_send, sendcount * sizeof(float));
cudaMalloc(&d_recv, recvcnt * sizeof(float)); // nRanks * sendcount
cudaMemcpy(d_send, h_send.data(), sendcount * sizeof(float), cudaMemcpyHostToDevice);

ncclAllGather(d_send, d_recv, sendcount, ncclFloat, comm, stream);
cudaStreamSynchronize(stream);
```

ReduceScatter

Source

[src/cuda/nccl-reducescatter](#)

ReduceScatter reduces across ranks then scatters equal chunks to each rank. It is the inverse of AllGather — each rank ends up with a reduced slice. This is used in ZeRO-style optimizers where each rank owns a shard of the parameters.

```
// Before: rank0=[1,2,3,4] rank1=[5,6,7,8] (2 ranks, 4 elements)
// After:  rank0=[6,8]      rank1=[10,12]   (sum, 2 elements each)

constexpr int totalCount = 1024;
int recvcnt = totalCount / nRanks;
std::vector<float> h_send(totalCount);
std::iota(h_send.begin(), h_send.end(), 0.0f); // [0, 1, 2, ...]
float *d_send, *d_recv;
cudaSetDevice(devId);
cudaMalloc(&d_send, totalCount * sizeof(float));
cudaMalloc(&d_recv, recvcnt * sizeof(float)); // totalCount / nRanks
cudaMemcpy(d_send, h_send.data(), totalCount * sizeof(float), cudaMemcpyHostToDevice);

ncclReduceScatter(d_send, d_recv, recvcnt, ncclFloat, ncclSum, comm, stream);
cudaStreamSynchronize(stream);
```

Grouping Collectives

Source

[src/cuda/nccl-alltoall](#)

When multiple operations need to run as a batch, wrapping them in a group lets NCCL fuse them into fewer kernel launches. This is required when issuing point-to-point calls from a single thread, and recommended for performance in general.

A common use case is building AlltoAll out of grouped Send/Recv. NCCL does not provide a dedicated AlltoAll primitive, but it can be composed:

```
// Naive AlltoAll: each rank sends a chunk to every other rank
//
// Before: rank0=[A0,A1] rank1=[B0,B1] (2 ranks, chunk per peer)
// After:  rank0=[A0,B0] rank1=[A1,B1]

constexpr int chunkCount = 512;
size_t chunkSize = chunkCount * sizeof(float);
```

(continues on next page)

(continued from previous page)

```

// d_send: nRanks chunks laid out contiguously [chunk_for_rank0, chunk_for_rank1, ...]
// d_recv: nRanks chunks to receive into
float *d_send, *d_recv;
cudaMalloc(&d_send, nRanks * chunkSize);
cudaMalloc(&d_recv, nRanks * chunkSize);
cudaMemcpy(d_send, h_send, nRanks * chunkSize, cudaMemcpyHostToDevice);

ncclGroupStart();
for (int peer = 0; peer < nRanks; peer++) {
    ncclSend(d_send + peer * chunkCount, chunkCount, ncclFloat, peer, comm, stream);
    ncclRecv(d_recv + peer * chunkCount, chunkCount, ncclFloat, peer, comm, stream);
}
ncclGroupEnd();
cudaStreamSynchronize(stream);

```

All calls between `ncclGroupStart` and `ncclGroupEnd` are batched into a single operation. Without grouping, each Send/Recv pair would deadlock waiting for its matching peer.

CUDA Graph Capture

NCCL collectives can be captured into CUDA graphs via stream capture, eliminating per-iteration kernel launch overhead. This is particularly effective for training loops where the same collective pattern repeats thousands of times. NCCL >= 2.15.1 and CUDA >= 11.7 are required.

Stream capture only **records** the operations — no kernels are executed during capture. The work is deferred until `cudaGraphLaunch` replays the graph.

The pattern is: warmup the collective first (NCCL needs to establish internal state), then capture into a graph, instantiate, and replay. A small RAII wrapper makes this clean:

```

struct CUDAGraph {
    cudaGraphExec_t exec_ = nullptr;

    template <typename F>
    CUDAGraph(cudaStream_t s, F&& fn) {
        cudaGraph_t g;
        cudaStreamBeginCapture(s, cudaStreamCaptureModeGlobal);
        fn();
        cudaStreamEndCapture(s, &g);
        cudaGraphInstantiate(&exec_, g, nullptr, nullptr, 0);
        cudaGraphDestroy(g);
    }

    ~CUDAGraph() { if (exec_) cudaGraphExecDestroy(exec_); }
    void launch(cudaStream_t s) { cudaGraphLaunch(exec_, s); }
};

```

Usage with a single collective:

```

// Warmup - NCCL initializes internal buffers on first call
ncclAllReduce(d_send, d_recv, count, ncclFloat, ncclSum, comm, stream);
cudaStreamSynchronize(stream);

// Capture and replay

```

(continues on next page)

(continued from previous page)

```

CUDAGraph graph(stream, [&] {
    ncclAllReduce(d_send, d_recv, count, ncclFloat, ncclSum, comm, stream);
});

for (int iter = 0; iter < num_iters; iter++) graph.launch(stream);
cudaStreamSynchronize(stream);

```

Grouped operations (ncclGroupStart/ncclGroupEnd) are also capturable. The entire group becomes a single graph node:

```

CUDAGraph graph(stream, [&] {
    ncclGroupStart();
    for (int peer = 0; peer < nRanks; peer++) {
        ncclSend(d_send + peer * chunk, chunk, ncclFloat, peer, comm, stream);
        ncclRecv(d_recv + peer * chunk, chunk, ncclFloat, peer, comm, stream);
    }
    ncclGroupEnd();
});

```

For multi-GPU single-process setups, each device needs its own graph captured on its own stream. All graphs are then launched and synchronized independently.

Note

When using ncclMemAlloc for user buffer registration (NCCL >= 2.19.1), ncclCommRegister is not required if NCCL kernels are launched from a CUDA graph captured via stream capture — NCCL handles registration automatically during capture.

Best Practices

- Always wrap multi-communicator calls in ncclGroupStart/ncclGroupEnd
- Use in-place operations (sendbuff == recvbuff) to save memory
- Pin host memory with cudaMallocHost for staging buffers
- Match NCCL calls 1:1 across all ranks — mismatched calls deadlock
- Use NCCL_DEBUG=INFO environment variable to diagnose topology and ring selection
- Capture NCCL collectives into CUDA graphs for repeated operations to eliminate launch overhead

1.5.10 GPU-GPU Communication

Table of Contents

- *GPU-GPU Communication*
 - *Introduction*
 - *Quick Reference*
 - * *P2P Direct (Single-Process)*

- * *IPC (Multi-Process)*
- * *P2P Attributes*
- *P2P Direct Example*
- *IPC Example*
- *Virtual Memory Management (VMM)*

Introduction

CUDA provides two mechanisms for GPU-to-GPU communication: IPC (Inter-Process Communication) for multi-process scenarios and P2P Direct for single-process multi-GPU applications. Both use the same hardware path (NVLink or PCIe) but differ in their programming model.

- **P2P Direct:** Single process owns all GPUs. Use `cudaDeviceEnablePeerAccess` to allow direct pointer dereferencing across GPUs. Simpler setup, no handle exchange needed. Best for single applications controlling all GPUs, simple multi-GPU parallelism, and low-latency scenarios.
- **IPC:** Multiple processes (e.g., MPI ranks) each own one GPU. Use `cudaIpcGetMemHandle` to export memory, exchange handles via MPI/sockets, then `cudaIpcOpenMemHandle` to access peer memory. Used by NCCL/NVSHMEM internally. Best for MPI-based distributed training and when fault isolation is needed.

Quick Reference

P2P Direct (Single-Process)

Single process owns all GPUs. Enable peer access to allow direct load/store to peer GPU memory without explicit memcpy. Once enabled, kernels on GPU 0 can dereference pointers allocated on GPU 1 as if they were local. The hardware (NVLink or PCIe) handles the actual data transfer transparently.

```
// Single process with multiple GPUs
//
//
//
//
//
//
//
//
//
//
```

Process

GPU 0
ptr=A

GPU 1
ptr=B

✓ Same address space
✓ Can dereference B from GPU 0

```
//
//
// Check and enable peer access
int canAccess;
cudaDeviceCanAccessPeer(&canAccess, gpu0, gpu1);
if (canAccess) {
    cudaSetDevice(gpu0);
    cudaDeviceEnablePeerAccess(gpu1, 0);
}

// Now GPU 0 kernels can directly access GPU 1 memory
// kernel<<<...>>>(gpu1_ptr); // Direct access, no memcpy needed

// Cleanup
cudaDeviceDisablePeerAccess(gpu1);
```


(continued from previous page)

```
// 2. Export handle for sharing
int fd;
cuMemExportToShareableHandle(&fd, allocHandle, CU_MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR, 0);

// 3. Reserve virtual address range and map
CUdeviceptr ptr;
cuMemAddressReserve(&ptr, size, 0, 0, 0);
cuMemMap(ptr, size, 0, allocHandle, 0);

// 4. Set access permissions
CUmemAccessDesc accessDesc = {};
accessDesc.location.type = CU_MEM_LOCATION_TYPE_DEVICE;
accessDesc.location.id = device;
accessDesc.flags = CU_MEM_ACCESS_FLAGS_PROT_READWRITE;
cuMemSetAccess(ptr, size, &accessDesc, 1);

// Now ptr is usable. Cleanup: cuMemUnmap, cuMemRelease, cuMemAddressFree
```

See [NVIDIA VMM Documentation](#) and `cuda-samples/memMapIPCDrv` for complete examples.

1.5.11 Hardware Topology

Table of Contents

- *Hardware Topology*
 - *Basic Topology Discovery*
 - *NUMA Node Discovery*
 - *PCI Device Enumeration*
 - *GPU-to-CUDA Mapping*
 - *Identifying Device Types*
 - *CPU Core Enumeration*
 - *Topology Traversal*

Understanding hardware topology is critical for high-performance GPU applications. Modern systems have complex NUMA architectures where memory access latency varies based on which CPU socket accesses which memory region. Similarly, GPUs and network devices (like EFA/InfiniBand) are connected through specific PCI bridges, and placing communicating components on the same NUMA node minimizes latency.

The `hwloc` (Hardware Locality) library provides a portable abstraction for discovering and querying hardware topology across different platforms. For comprehensive GPU affinity examples including EFA device discovery and NVLink bandwidth detection, see [Libefaxx](#). The `hwloc` project also provides `lstopo`, a reference implementation for topology visualization.

Basic Topology Discovery

Source

`src/cuda/hwloc-topology`

The hwloc library requires explicit initialization before use. The topology object represents the entire hardware hierarchy from the machine level down to individual cores and I/O devices. After calling `hwloc_topology_init()`, the topology is empty until `hwloc_topology_load()` performs the actual hardware discovery. The depth value indicates how many levels exist in the hierarchy (machine, package, NUMA node, L3 cache, core, etc.). Always destroy the topology when done to free allocated resources.

```
#include <hwloc.h>

hwloc_topology_t topology;
hwloc_topology_init(&topology);
hwloc_topology_load(topology);

int depth = hwloc_topology_get_depth(topology);
printf("Topology depth: %d\n", depth);

hwloc_topology_destroy(topology);
```

NUMA Node Discovery

Non-Uniform Memory Access (NUMA) architectures have multiple memory controllers, each associated with a subset of CPU cores. Accessing local memory (attached to the same controller) is faster than accessing remote memory. For GPU workloads, understanding NUMA topology helps place CPU threads on cores that have the lowest latency path to both the GPU and its associated memory. The `hwloc_get_type_depth()` function returns the level in the hierarchy where NUMA nodes appear, and `hwloc_get_nbobjs_by_depth()` counts how many exist at that level.

```
hwloc_topology_t topology;
hwloc_topology_init(&topology);
hwloc_topology_load(topology);

int numa_depth = hwloc_get_type_depth(topology, HWLOC_OBJ_NUMANODE);
int numa_count = hwloc_get_nbobjs_by_depth(topology, numa_depth);
printf("NUMA nodes: %d\n", numa_count);

for (int i = 0; i < numa_count; i++) {
    hwloc_obj_t numa = hwloc_get_obj_by_depth(topology, numa_depth, i);
    printf("NUMA %d: %lu MB local memory\n", i, numa->attr->numanode.local_memory / (1024 * 1024));
}

hwloc_topology_destroy(topology);
```

PCI Device Enumeration

By default, hwloc only discovers CPU and memory topology. To find GPUs, network adapters, and other I/O devices, you must enable I/O filtering before loading the topology. The `HWLOC_TYPE_FILTER_KEEP_IMPORTANT` filter includes PCI devices that have associated OS devices (like `/dev/nvidia0` or network interfaces), filtering out bridges and other infrastructure. Each PCI device object contains attributes including domain, bus, device, function numbers, and vendor/device IDs that can be used to identify specific hardware.

```
hwloc_topology_t topology;
hwloc_topology_init(&topology);
hwloc_topology_set_io_types_filter(topology, HWLOC_TYPE_FILTER_KEEP_IMPORTANT);
hwloc_topology_load(topology);

hwloc_obj_t obj = nullptr;
while ((obj = hwloc_get_next_pcidev(topology, obj)) != nullptr) {
    printf("PCI %04x:%02x:%02x.%x vendor=%04x device=%04x class=%04x\n",
        obj->attr->pcidev.domain,
        obj->attr->pcidev.bus,
        obj->attr->pcidev.dev,
        obj->attr->pcidev.func,
        obj->attr->pcidev.vendor_id,
        obj->attr->pcidev.device_id,
        obj->attr->pcidev.class_id);
}

hwloc_topology_destroy(topology);
```

GPU-to-CUDA Mapping

CUDA assigns device indices (0, 1, 2, ...) that may not correspond to physical slot order or PCI bus order. To correlate CUDA devices with hwloc topology, use the PCI address from `cudaDeviceProp` (which provides `pciDomainID`, `pciBusID`, and `pciDeviceID`) to find the matching hwloc object. This mapping is essential for determining which NUMA node a GPU belongs to, which CPU cores are local to it, and which network adapters share the same PCI bridge for optimal data placement.

```
#include <cuda_runtime.h>
#include <hwloc.h>

cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, 0);

hwloc_topology_t topology;
hwloc_topology_init(&topology);
hwloc_topology_set_io_types_filter(topology, HWLOC_TYPE_FILTER_KEEP_IMPORTANT);
hwloc_topology_load(topology);

hwloc_obj_t obj = nullptr;
while ((obj = hwloc_get_next_pcidev(topology, obj)) != nullptr) {
    if (obj->attr->pcidev.domain == prop.pciDomainID &&
        obj->attr->pcidev.bus == prop.pciBusID &&
        obj->attr->pcidev.dev == prop.pciDeviceID) {
        printf("Found GPU in hwloc: %04x:%02x:%02x\n",
            obj->attr->pcidev.domain,
            obj->attr->pcidev.bus,
            obj->attr->pcidev.dev);
        break;
    }
}

hwloc_topology_destroy(topology);
```

Identifying Device Types

PCI devices are identified by their class ID and vendor ID. The class ID upper byte indicates the device category: `0x03` for display controllers (GPUs), `0x02` for network controllers, `0x01` for storage. NVIDIA GPUs have vendor ID `0x10de`, AMD GPUs use `0x1002`. Host bridges connect the CPU to PCI hierarchies and are identified by their upstream type not being PCI. These helper functions are useful when traversing topology to categorize devices.

```
constexpr uint16_t NVIDIA_VENDOR_ID = 0x10de;
constexpr uint16_t AMD_VENDOR_ID = 0x1002;

bool is_nvidia_gpu(hwloc_obj_t obj) {
    if (obj->type != HWLOC_OBJ_PCI_DEVICE) return false;
    auto class_id = obj->attr->pcidev.class_id >> 8;
    if (class_id != 0x03) return false; // Display controller class
    return obj->attr->pcidev.vendor_id == NVIDIA_VENDOR_ID;
}

bool is_host_bridge(hwloc_obj_t obj) {
    if (obj->type != HWLOC_OBJ_BRIDGE) return false;
    return obj->attr->bridge.upstream_type != HWLOC_OBJ_BRIDGE_PCI;
}
```

CPU Core Enumeration

Each NUMA node has an associated cpuset bitmap indicating which CPU cores belong to it. For GPU-intensive workloads, pinning application threads to cores on the same NUMA node as the GPU reduces memory access latency and avoids cross-socket traffic. The `hwloc_bitmap_foreach_begin/end` macros iterate over set bits in the cpuset. This information can be used with `pthread_setaffinity_np()` or `sched_setaffinity()` to bind threads to specific cores.

```
hwloc_obj_t numa = hwloc_get_obj_by_type(topology, HWLOC_OBJ_NUMANODE, 0);

// Get cpuset for this NUMA node
hwloc_cpuset_t cpuset = hwloc_bitmap_dup(numa->cpuset);

int core_id;
hwloc_bitmap_foreach_begin(core_id, cpuset) {
    printf("Core %d belongs to NUMA 0\n", core_id);
}
hwloc_bitmap_foreach_end();

hwloc_bitmap_free(cpuset);
```

Topology Traversal

The hwloc topology is a tree structure with multiple child types: regular children (CPU objects), memory children (NUMA nodes), I/O children (PCI devices), and misc children. A complete traversal must visit all child types to build a full picture of the hardware. This recursive approach is useful for building affinity maps that associate GPUs with their local NUMA nodes, CPU cores, and network devices based on their position in the PCI hierarchy.

```
void traverse(hwloc_obj_t obj, int depth) {
    for (int i = 0; i < depth; i++) printf(" ");
    printf("%s %d\n", hwloc_obj_type_string(obj->type), obj->logical_index);
}
```

(continues on next page)

(continued from previous page)

```

// Traverse children
for (hwloc_obj_t child = obj->first_child; child; child = child->next_sibling) {
    traverse(child, depth + 1);
}
// Traverse I/O children (PCI devices)
for (hwloc_obj_t child = obj->io_first_child; child; child = child->next_sibling) {
    traverse(child, depth + 1);
}
}

// Start from root
traverse(hwloc_get_root_obj(topology), 0);

```

1.6 Bash & System Tools

The command line remains the most powerful interface for automating tasks, managing systems, and orchestrating complex workflows. Bash, as the default shell on most Linux distributions and macOS, provides a rich scripting environment that glues together the Unix toolbox into cohesive solutions.

These references cover shell syntax and control flow, regular expressions for text processing, the `find` command for filesystem traversal, date manipulation for logs and scheduling, hardware monitoring with tools like `lspci` and `nvidia-smi`, and `systemd` for modern service management. Together, they form the toolkit for effective system administration and development automation.

1.6.1 Bash

Table of Contents

- *Bash*
 - *Special Parameters*
 - *Set Positional Parameters*
 - *Brace Expansion*
 - *Globs (Pattern Matching)*
 - *Variable Expansion*
 - *String Manipulation*
 - *Pattern Removal*
 - *Pattern Substitution*
 - *Arrays*
 - *Associative Arrays (Dictionary)*
 - *Conditionals*
 - *Arithmetic*
 - *Loops*
 - *Functions*

- *Here Documents & Strings*
- *Read Input*
- *Read File Line by Line*
- *Process Substitution*
- *Command Substitution*
- *Check Command Exists*
- *Error Handling*
- *Argument Parsing*
- *Logging*
- *Regular Expressions*
- *grep Basic vs Extended Regex*
- *Text Processing*
- *Temporary Files*
- *Parallel Execution*
- *Script Template*

This cheatsheet provides a comprehensive reference for Bash shell scripting, covering essential concepts from basic variable expansion to advanced text processing and error handling. Each section includes practical examples that demonstrate real-world usage patterns commonly encountered in automation scripts and system administration tasks.

An interactive demo script is available at [src/bash/bash.sh](#) to help you experiment with the concepts covered in this cheatsheet. The script provides colored output showing both the commands and their results, making it easier to understand how each feature works.

```
./src/bash/bash.sh           # Run all demos
./src/bash/bash.sh strings   # Run specific section (e.g., strings, arrays, regex)
./src/bash/bash.sh --help    # Show all available sections
```

Special Parameters

Bash provides special parameters that give access to script arguments, process information, and command exit status. These read-only variables are automatically set by the shell and are essential for writing flexible, reusable scripts that can adapt to different inputs and execution contexts.

```
$*    # all positional params as "$1$2..."
$@    # all positional params as "$1" "$2" ...
$#    # number of positional params
$$    # current process PID
$?    # exit status of last command
$0    # script name
$!    # PID of last background command
```

Example:

```
foo() {
  echo "All args: $@"
}
```

(continues on next page)

(continued from previous page)

```

echo "Arg count: $#"  

echo "Script: $0"  

}  

foo "a" "b" "c"

```

Set Positional Parameters

The `set` builtin can reassign positional parameters, which is useful for parsing command output or restructuring arguments within a script. This technique allows you to split strings into separate arguments or reset the argument list entirely for subsequent processing.

```

set -- a b c  

echo "$1" "$2" "$3" # a b c

```

Brace Expansion

Brace expansion generates arbitrary strings before any other expansion occurs, making it a powerful tool for creating multiple files, directories, or generating sequences. Unlike globs, brace expansion does not depend on existing files and always produces output regardless of filesystem state.

```

echo foo.{pdf,txt,png}      # foo.pdf foo.txt foo.png  

echo {1..5}                 # 1 2 3 4 5  

echo {a..z}                  # a b c ... z  

echo {01..10}                # 01 02 03 ... 10  

mkdir -p project/{src,bin,doc}

```

Globs (Pattern Matching)

Glob patterns enable filename expansion using wildcards. Unlike regular expressions, globs are expanded by the shell before command execution and match against actual filenames in the filesystem. Understanding the difference between `*` (any string), `?` (single character), and `[]` (character class) is fundamental to shell scripting.

```

ls *.txt                    # match any string  

ls file?.txt                # match single character  

ls file[abc].txt            # match a, b, or c  

ls file[a-z].txt            # match range a-z  

ls file[!0-9].txt           # match non-digit

```

POSIX character classes:

```

[:alnum:] # alphanumeric  

[:alpha:] # alphabetic  

[:digit:] # digits  

[:space:] # whitespace  

[:upper:] # uppercase  

[:lower:] # lowercase

```

Variable Expansion

Parameter expansion provides powerful mechanisms for default values, substring extraction, and variable introspection without spawning external processes. These expansions are evaluated by the shell itself, making them significantly faster than calling utilities like `cut` or `sed` for simple string operations.


```

${var:-default}    # use default if var unset/empty
${var:=default}    # assign default if var unset/empty
${var:+alt}        # use alt if var is set
${var:?error}       # exit with error if var unset/empty

${#var}            # string length
${var:offset}       # substring from offset
${var:offset:len}   # substring with length

${!prefix*}        # names matching prefix
${!prefix@}         # names matching prefix (quoted)

```

String Manipulation

Bash provides built-in string operations for length calculation, substring extraction, and case conversion. These native operations eliminate the need for external tools like `cut`, `tr`, or `awk` in many common scenarios, resulting in faster execution and simpler scripts.

```

foo="hello world"

${#foo}            # 11 (length)
${foo:6}           # world (from offset 6)
${foo:0:5}         # hello (first 5 chars)
${foo: -5}         # world (last 5 chars, note space)

${foo^^}          # HELLO WORLD (uppercase)
${foo,,}          # hello world (lowercase)
${foo^}           # Hello world (capitalize first)

```

Pattern Removal

Pattern removal operators strip matching prefixes or suffixes from variable values. The `#` operator removes from the front while `%` removes from the back. Single characters perform shortest match; doubled characters perform longest (greedy) match. These operators are particularly useful for path manipulation and file extension handling.

```

path="/home/user/file.tar.gz"

${path#*/}         # home/user/file.tar.gz (remove shortest from front)
${path##*/}        # file.tar.gz (remove longest from front)
${path%.*}         # /home/user/file.tar (remove shortest from back)
${path%%.*}        # /home/user/file (remove longest from back)

```

Pattern Substitution

Pattern substitution replaces matched patterns within variable values, providing functionality similar to `sed` but without spawning a subprocess. Single slash replaces the first occurrence; double slash replaces all occurrences. Anchors `#` and `%` restrict matching to the beginning or end of the string.

```

str="foo bar foo"

${str/foo/baz}     # baz bar foo (replace first)
${str//foo/baz}    # baz bar baz (replace all)

```

(continues on next page)

(continued from previous page)

```

${str/#foo/baz}    # baz bar foo (replace if starts with)
${str/%foo/baz}    # foo bar baz (replace if ends with)

```

Arrays

Indexed arrays store ordered lists of values accessible by numeric index. Arrays in Bash are sparse, meaning indices need not be contiguous. They support efficient iteration, element counting, and dynamic growth through append operations.

```

arr=(a b c d)      # declare array
arr+=(e f)          # append elements

echo ${arr[0]}      # first element
echo ${arr[@]}      # all elements
echo ${#arr[@]}     # array length
echo ${!arr[@]}     # all indices

unset arr[1]        # remove element

```

Associative Arrays (Dictionary)

Associative arrays, available in Bash 4.0 and later, provide key-value storage enabling dictionary-like data structures. They must be explicitly declared with `declare -A` before use. Common applications include configuration management, caching computed values, and building lookup tables for efficient data retrieval.

```

declare -A dict
dict=([foo]="FOO" [bar]="BAR")
dict["baz"]="BAZ"

echo ${dict[foo]}   # FOO
echo ${!dict[@]}    # all keys
echo ${dict[@]}     # all values

# check if key exists
[[ -v dict[foo] ]] && echo "exists"

unset dict[foo]     # remove key

```

Conditionals

Conditional expressions test file attributes, compare strings, and evaluate numeric relationships. The `[[ ]]` construct is preferred over `[ ]` for its enhanced features including pattern matching, regex support, and safer handling of empty variables. Always quote variables inside conditions to prevent word splitting issues.

File tests:

```

[[ -e file ]]      # exists
[[ -f file ]]      # regular file
[[ -d dir ]]       # directory
[[ -r file ]]      # readable
[[ -w file ]]      # writable
[[ -x file ]]      # executable

```

(continues on next page)

(continued from previous page)

```
[[ -s file ]]      # size > 0
[[ -L file ]]      # symbolic link
```

String tests:

```
[[ -z "$str" ]]    # empty string
[[ -n "$str" ]]    # non-empty string
[[ "$a" == "$b" ]] # equal
[[ "$a" != "$b" ]] # not equal
[[ "$a" < "$b" ]]  # less than (lexicographic)
[[ "$a" =~ regex ]] # regex match
```

Numeric tests:

```
[[ $a -eq $b ]]    # equal
[[ $a -ne $b ]]    # not equal
[[ $a -lt $b ]]    # less than
[[ $a -le $b ]]    # less or equal
[[ $a -gt $b ]]    # greater than
[[ $a -ge $b ]]    # greater or equal
```

Arithmetic

Bash supports integer arithmetic through `(( ))` for evaluation and `$(())` for expansion. These constructs allow C-style syntax including increment/decrement operators, compound assignment, and comparison. For floating-point calculations, external tools like `bc` or `awk` are required since Bash only handles integers natively.

```
((a = 5 + 3))      # arithmetic assignment
((a++))            # increment
((a += 10))        # add and assign

result=$((5 * 3))  # arithmetic expansion
echo $(2 ** 10)    # 1024 (exponentiation)
```

Loops

Bash provides multiple loop constructs for iteration over sequences, arrays, and conditions. The `for` loop iterates over word lists, while `while` and `until` loops continue based on condition evaluation. The `:` builtin (equivalent to `true`) enables infinite loops commonly used for daemons and interactive menus.

```
# for loop
for i in {1..5}; do echo $i; done

# C-style for
for ((i=0; i<5; i++)); do echo $i; done

# while loop
while [[ $i -lt 5 ]]; do ((i++)); done

# infinite loop
while ;; do sleep 1; done
```

(continues on next page)

(continued from previous page)

```
# iterate array
for item in "${arr[@]}; do echo "$item"; done

# iterate with index
for i in "${!arr[@]}; do echo "$i: ${arr[$i]}"; done
```

Functions

Functions encapsulate reusable logic with local variable scope support. Arguments are accessed via positional parameters (\$1, \$2, etc.), and return values are limited to exit status codes (0-255). For returning strings or complex data, use command substitution to capture function output or modify global variables.

```
myfunc() {
  local var="local"    # local variable
  echo "arg1: $1"
  return 0              # return status
}

myfunc "hello"
echo $?                # check return status
```

Here Documents & Strings

Here documents embed multi-line text directly within scripts, while here strings pass single-line input to commands expecting stdin. Both constructs avoid the need for temporary files or complex echo pipelines. Quoting the delimiter ('EOF') prevents variable expansion within the here document.

```
# here document
cat <<EOF
multi-line
content here
EOF

# here document (no variable expansion)
cat <<'EOF'
$var not expanded
EOF

# here string
bc <<< "1 + 2 * 3"
```

Read Input

The read builtin captures user input with options for custom prompts, timeouts, silent mode for passwords, and character limits. Setting IFS controls field splitting, and the -r flag prevents backslash interpretation for raw input handling.

```
read -p "Enter name: " name
read -s -p "Password: " pass    # silent input
read -t 5 -p "Quick: " ans      # 5 second timeout
read -n 1 -p "Press key: " key  # single character
```

Read File Line by Line

Processing files line-by-line with `while read` preserves whitespace and handles special characters correctly when IFS is properly configured. Setting IFS= before read prevents leading/trailing whitespace trimming, and `-r` prevents backslash escape interpretation.

```
while IFS= read -r line; do
    echo "$line"
done < file.txt

# read with field separator
while IFS=: read -r user _ uid gid _ home shell; do
    echo "user: $home"
done < /etc/passwd
```

Process Substitution

Process substitution `<()` treats command output as a file descriptor, enabling commands that require file arguments to accept dynamic input. This is particularly useful for comparing outputs of two commands or feeding generated data to programs that only accept file inputs.

```
# compare two command outputs
diff <(ls dir1) <(ls dir2)

# feed command output as file
while read line; do echo "$line"; done <<(ls -la)
```

Command Substitution

Command substitution captures command output into variables for further processing. The `$()` syntax is preferred over legacy backticks for improved readability, easier nesting, and consistent quoting behavior. The captured output has trailing newlines stripped automatically.

```
now=$(date +%Y-%m-%d)
files=$(ls *.txt)
count=$(wc -l < file.txt)
```

Check Command Exists

Verifying command availability before execution prevents runtime errors and enables graceful fallbacks in portable scripts. The command `-v` builtin is POSIX-compliant and preferred over `which`, which may behave inconsistently across systems.

```
if command -v git &>/dev/null; then
    echo "git is installed"
fi

# alternative
type -P git &>/dev/null && echo "found"
```

Error Handling

Robust scripts require explicit error handling to fail fast and provide meaningful diagnostics. The `set` builtin configures shell behavior for error detection, while `trap` enables cleanup routines on exit, error, or signal reception. The combination `set -euo pipefail` is considered best practice for production scripts.

```
set -e          # exit on error
set -u          # error on undefined variable
set -o pipefail # pipeline fails if any command fails
set -x          # debug mode (print commands)

# common combination
set -euo pipefail

# trap errors
trap 'echo "Error at line $LINENO"' ERR
trap 'cleanup' EXIT
```

Argument Parsing

Command-line argument parsing enables flexible script interfaces with options and positional parameters. The `getopts` builtin handles POSIX-style short options with automatic error handling, while manual parsing loops provide support for GNU-style long options and more complex argument structures.

```
usage() { echo "Usage: $0 [-h] [-f file] args..."; exit 1; }

while getopts ":hf:" opt; do
  case $opt in
    h) usage ;;
    f) file="$OPTARG" ;;
    \?) echo "Invalid: -$OPTARG" >&2; exit 1 ;;
    :) echo "Option -$OPTARG requires argument" >&2; exit 1 ;;
  esac
done
shift $((OPTIND-1))
```

Long options with manual parsing:

```
while [[ $# -gt 0 ]]; do
  case "$1" in
    -h|--help) usage; exit 0 ;;
    -f|--file) file="$2"; shift 2 ;;
    --) shift; break ;;
    -*) echo "Unknown option: $1" >&2; exit 1 ;;
    *) args+=("$1"); shift ;;
  esac
done
```

Logging

Structured logging with timestamps and severity levels improves script debugging and operational monitoring. ANSI color codes enhance terminal readability by visually distinguishing log levels. Redirecting warnings and errors to `stderr` (`>&2`) ensures proper stream separation for pipeline compatibility.

```
readonly RED='\033[0;31m'
readonly GREEN='\033[0;32m'
readonly YELLOW='\033[0;33m'
readonly NC='\033[0m'

log() { echo -e "[$(date -Iseconds)] $*"; }
info() { echo -e "[$(date -Iseconds)] ${GREEN}INFO${NC} $*"; }
warn() { echo -e "[$(date -Iseconds)] ${YELLOW}WARN${NC} $*" >&2; }
error() { echo -e "[$(date -Iseconds)] ${RED}ERROR${NC} $*" >&2; }
```

Regular Expressions

Bash supports extended regular expression matching with the `=~` operator inside `[[]]` conditionals. When a match occurs, captured groups are stored in the `BASH_REMATCH` array, with index 0 containing the full match and subsequent indices holding parenthesized subexpressions.

Bash regex matching with `=~`:

```
if [[ "$str" =~ ^[0-9]+$ ]]; then
    echo "numeric"
fi

# capture groups
if [[ "hello123" =~ ([a-z]+)([0-9]+) ]]; then
    echo "${BASH_REMATCH[0]}" # hello123 (full match)
    echo "${BASH_REMATCH[1]}" # hello
    echo "${BASH_REMATCH[2]}" # 123
fi
```

grep Basic vs Extended Regex

The `grep` command supports basic regular expressions (BRE) by default and extended regular expressions (ERE) with the `-E` flag. In BRE, metacharacters like `?`, `+`, `{}`, and `|` require backslash escaping, while ERE treats them as special by default. Perl-compatible regex (`-P`) offers additional features like lookahead and lookbehind.

```
# basic regex (escape special chars)
echo "987-123-4567" | grep "^([0-9]{3}-[0-9]{3}-[0-9]{4})$"

# extended regex (no escaping)
echo "987-123-4567" | grep -E "^([0-9]{3}-[0-9]{3}-[0-9]{4})$"

# Perl regex
echo "test@email.com" | grep -P "[\w.]+@[\w.]+$"
```

Text Processing

Unix text processing tools form a powerful pipeline ecosystem for data transformation and analysis. These utilities follow the Unix philosophy of doing one thing well and composing via pipes. Mastering `tr`, `sort`, `uniq`, `cut`, `awk`, and `sed` enables efficient manipulation of structured and unstructured text data.

```
# tr - translate characters
echo "hello" | tr 'a-z' 'A-Z' # HELLO
echo "a b c" | tr -s ' ' # squeeze spaces
```

(continues on next page)

(continued from previous page)

```

echo "a:b:c" | tr ':' '\n'      # replace : with newline

# sort
sort file.txt                  # sort lines
sort -n file.txt               # numeric sort
sort -k2 file.txt              # sort by 2nd field
sort -u file.txt               # unique sort
sort -r file.txt               # reverse sort

# uniq (requires sorted input)
sort file.txt | uniq           # remove duplicates
sort file.txt | uniq -c        # count occurrences
sort file.txt | uniq -d        # show only duplicates

# cut
cut -d: -f1 /etc/passwd        # first field
cut -c1-10 file.txt            # first 10 chars

# awk
awk '{print $1}' file.txt       # first column
awk -F: '{print $1}' /etc/passwd # with delimiter
awk 'NR==5' file.txt            # 5th line

# sed
sed 's/old/new/' file.txt      # replace first
sed 's/old/new/g' file.txt     # replace all
sed -i 's/old/new/g' file.txt  # in-place edit
sed -n '5p' file.txt           # print 5th line
sed '5d' file.txt              # delete 5th line

```

Temporary Files

The `mktemp` command creates secure temporary files and directories with unique, unpredictable names. This prevents race conditions and symlink attacks that can occur with predictable temporary filenames. Always combine with `trap` to ensure cleanup on script exit, even when errors occur.

```

tmpfile=$(mktemp)
tmpdir=$(mktemp -d)

trap 'rm -f "$tmpfile"' EXIT

echo "data" > "$tmpfile"

```

Parallel Execution

Parallel execution accelerates batch operations by utilizing multiple CPU cores simultaneously. Background jobs with `&` provide basic parallelism, `xargs -P` offers controlled concurrency for command-line tools, and GNU `parallel` provides advanced features like progress bars, job logging, and remote execution.

```

# background jobs
cmd1 & cmd2 & wait

```

(continues on next page)

(continued from previous page)

```
# xargs parallel
cat urls.txt | xargs -P 4 -I {} curl {}

# GNU parallel
parallel -j 4 gzip ::: *.txt
```

Script Template

A well-structured script template incorporates strict error handling, flexible argument parsing, and consistent coding conventions. Starting from a proven template reduces boilerplate and ensures best practices are followed from the beginning. This pattern serves as a foundation for production-quality scripts.

```
#!/usr/bin/env bash
set -euo pipefail

readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_NAME="$(basename "$0")"

usage() {
    cat <<EOF
Usage: $SCRIPT_NAME [OPTIONS] <args>

Options:
  -h, --help      Show this help
  -v, --verbose   Verbose output
EOF
}

main() {
    local verbose=false

    while [[ $# -gt 0 ]]; do
        case "$1" in
            -h|--help) usage; exit 0 ;;
            -v|--verbose) verbose=true; shift ;;
            --) shift; break ;;
            -*) echo "Unknown option: $1" >&2; exit 1 ;;
            *) break ;;
        esac
    done

    [[ $# -eq 0 ]] && { usage; exit 1; }

    # main logic here
}

main "$@"
```

1.6.2 Operating System

Table of Contents

- *Operating System*
 - *Date and Time*
 - * *Current Date and Time*
 - * *Common Format Specifiers*
 - * *Date Arithmetic*
 - * *Convert Timestamps*
 - *File Searching with find*
 - * *Basic Patterns*
 - * *File Types*
 - * *By Size*
 - * *By Time*
 - * *By Ownership and Permissions*
 - * *Combining Conditions*
 - * *Actions*
 - * *Loop Through Results*
 - *System Information*
 - * *System Information*
 - *Process Management*
 - * *Viewing Processes*
 - * *Process Control*
 - * *Background Jobs*
 - *File Operations*
 - * *File Management*
 - * *Compression*
 - * *Viewing Files*
 - * *File Comparison*
 - *Environment and Shell*
 - * *Environment Variables*
 - * *Command Information*
 - * *History*

From date manipulation to process control, mastering command-line utilities is essential for effective system administration. Here you'll find practical examples for everyday tasks: parsing dates, searching files with `find`, managing processes, and handling compression—all organized for quick reference.

An interactive demo script is available at `src/bash/os.sh` to help you experiment with the concepts covered in this cheatsheet.

```
./src/bash/os.sh           # Run all demos
./src/bash/os.sh date     # Run date/time demo
./src/bash/os.sh find     # Run find command demo
./src/bash/os.sh --help   # Show all available sections
```

Date and Time

The `date` command displays and manipulates system date and time. Format strings control output, while GNU and BSD versions differ in syntax for date arithmetic.

Current Date and Time

```
date           # Full date and time
date +"%Y-%m-%d" # 2024-01-15
date +"%Y%m%d"  # 20240115
date +"%H:%M:%S" # 14:30:45
date +"%Y-%m-%d %H:%M:%S" # 2024-01-15 14:30:45
date -Iseconds  # ISO 8601 format with timezone
```

Common Format Specifiers

```
%Y  # Year (2024)
%m  # Month (01-12)
%d  # Day (01-31)
%H  # Hour 24h (00-23)
%M  # Minute (00-59)
%S  # Second (00-59)
%s  # Unix timestamp
%A  # Weekday name (Monday)
%B  # Month name (January)
%Z  # Timezone (PST)
```

Date Arithmetic

```
# Linux (GNU date)
date +%Y-%m-%d -d "1 day ago"      # Yesterday
date +%Y-%m-%d -d "7 days ago"     # 7 days ago
date +%Y-%m-%d -d "1 month ago"    # 1 month ago
date +%Y-%m-%d -d "next monday"    # Next Monday
date +%Y-%m-%d -d "2024-01-15 + 10 days"

# BSD/macOS
date -j -v-1d +"%Y-%m-%d"          # Yesterday
date -j -v-7d +"%Y-%m-%d"          # 7 days ago
date -j -v-1m +"%Y-%m-%d"          # 1 month ago
date -j -v+10d +"%Y-%m-%d"         # 10 days from now
```

Convert Timestamps

```
# Unix timestamp to date (Linux)
date -d @1705334400

# Unix timestamp to date (BSD/macOS)
date -r 1705334400

# Current timestamp
date +%s
```

File Searching with find

The `find` command recursively searches directories for files matching specified criteria. It supports filtering by name, type, size, time, ownership, and permissions, with actions to execute on matched files.

Basic Patterns

```
find /path -name "*.py"           # By extension
find /path -name "*test*"         # By substring
find /path -iname "*.PY"          # Case insensitive
find /path -path "**/test/*"       # By path pattern
```

File Types

```
find /path -type f                # Regular files
find /path -type d                # Directories
find /path -type l                # Symbolic links
find /path -type f -name "*.py"   # Combine filters

# Type codes: f=file, d=directory, l=symlink,
#             b=block, c=character, p=pipe, s=socket
```

By Size

```
find /path -type f -size +100M    # Larger than 100MB
find /path -type f -size -1M      # Smaller than 1MB
find /path -type f -size 50M      # Exactly 50MB

# Size units: c=bytes, k=KB, M=MB, G=GB
```

By Time

```
# Access time
find /path -type f -atime +7      # Accessed > 7 days ago
find /path -type f -atime -7      # Accessed < 7 days ago
find /path -type f -amin +60      # Accessed > 60 min ago

# Modification time
find /path -type f -mtime +30     # Modified > 30 days ago
```

(continues on next page)

(continued from previous page)

```
find /path -type f -mmin -10      # Modified < 10 min ago

# Change time (metadata)
find /path -type f -ctime +7      # Changed > 7 days ago

# Newer than file
find /path -type f -newer ref.txt
```

By Ownership and Permissions

```
find /path -type f -user root      # Owned by root
find /path -type f -group admin    # Group admin
find /path -type f -perm 644       # Exact permissions
find /path -type f -perm -644      # At least these permissions
find /path -type f -perm /u+x      # User executable
```

Combining Conditions

```
# AND (implicit)
find /path -type f -name "*.log" -size +1M

# OR
find /path \( -name "*.py" -o -name "*.sh" \)

# NOT
find /path -type f ! -name "*.txt"

# Complex
find /path -type f \( -name "*.py" -o -name "*.sh" \) ! -path "*/venv/*"
```

Actions

```
# Delete matched files
find /path -type f -name "*.tmp" -delete

# Execute command for each file
find /path -type f -name "*.sh" -exec chmod +x {} \;

# Execute command with all files
find /path -type f -name "*.txt" -exec grep -l "pattern" {} +

# Delete directories recursively
find /path -type d -name "__pycache__" -exec rm -rf {} +

# Print with null delimiter (for xargs)
find /path -type f -print0 | xargs -0 ls -la
```

Loop Through Results

```
# Using while loop (handles spaces in filenames)
find /path -type f -name "*.txt" -print0 | while IFS= read -r -d '' file; do
    echo "Processing: $file"
done

# Process substitution (preserves variables)
count=0
while IFS= read -r -d '' file; do
    ((count++))
done < <(find /path -type f -print0)
echo "Found: $count files"

# Sort results
find /path -name "*.txt" -print0 | sort -z | xargs -r0 -I{} echo "{}"
```

System Information

Commands for gathering system software and configuration information.

System Information

```
uname -a          # All system info
uname -s          # Kernel name
uname -r          # Kernel release
uname -m          # Machine architecture
hostname          # System hostname
uptime           # System uptime
cat /etc/os-release # OS info (Linux)
sw_vers          # macOS version
```

Process Management

Commands for monitoring and controlling system processes.

Viewing Processes

```
ps aux          # All processes
ps aux | grep nginx # Filter by name
ps -ef         # Full format listing
ps -p PID -o pid,ppid,cmd,%cpu,%mem # Specific process details
pgrep -l nginx # Find PID by name
pstree         # Process tree
top            # Interactive process viewer
htop          # Enhanced top (if installed)
```

Process Control

```
kill PID        # Terminate (SIGTERM)
kill -9 PID     # Force kill (SIGKILL)
```

(continues on next page)

(continued from previous page)

```
kill -HUP PID           # Reload config (SIGHUP)
killall nginx           # Kill by name
pkill -f "python script.py" # Kill by pattern
```

Background Jobs

```
command &           # Run in background
jobs                # List background jobs
fg %1               # Bring job 1 to foreground
bg %1               # Resume job 1 in background
disown %1           # Detach job from shell
nohup command &    # Run immune to hangups
```

File Operations

Commands for file manipulation, compression, and text processing.

File Management

```
cp -r src dest       # Copy recursively
mv old new           # Move/rename
rm -rf dir           # Remove recursively
mkdir -p a/b/c        # Create nested dirs
ln -s target link     # Symbolic link
chmod 755 file        # Change permissions
chown user:group file # Change ownership
```

Compression

```
# tar
tar -cvf archive.tar dir/      # Create tar
tar -xvf archive.tar           # Extract tar
tar -czvf archive.tar.gz dir/  # Create gzipped tar
tar -xzvf archive.tar.gz       # Extract gzipped tar
tar -cjvf archive.tar.bz2 dir/ # Create bz2 tar
tar -tzvf archive.tar.gz       # List contents

# zip
zip -r archive.zip dir/        # Create zip
unzip archive.zip              # Extract zip
unzip -l archive.zip           # List contents

# gzip
gzip file                      # Compress (replaces file)
gunzip file.gz                 # Decompress
zcat file.gz                   # View without extracting
```

Viewing Files

```
cat file           # Print entire file
head -n 20 file    # First 20 lines
tail -n 20 file    # Last 20 lines
tail -f file       # Follow file updates
less file          # Paginated view
wc -l file         # Line count
wc -w file         # Word count
```

File Comparison

```
diff file1 file2   # Line differences
diff -u file1 file2 # Unified format
diff -r dir1 dir2  # Compare directories
cmp file1 file2    # Byte comparison
comm file1 file2   # Common/unique lines (sorted)
md5sum file        # MD5 checksum
sha256sum file     # SHA256 checksum
```

Environment and Shell

Commands for managing shell environment and configuration.

Environment Variables

```
env                # All environment variables
printenv PATH      # Specific variable
export VAR=value   # Set variable
unset VAR          # Remove variable
echo $PATH         # Print variable
```

Command Information

```
which command      # Command location
type command       # Command type
whereis command    # Binary/man/source locations
man command        # Manual page
command --help     # Help text
alias              # List aliases
alias ll='ls -la'  # Create alias
```

History

```
history            # Command history
history | grep pattern # Search history
!n                # Execute command n
!!               # Repeat last command
!$              # Last argument of previous command
ctrl+r          # Reverse search
```


1.6.3 Network

Table of Contents

- *Network*
 - *Network Configuration*
 - * *Interface Information*
 - * *Interface Management*
 - * *Routing*
 - * *ARP Table*
 - *Connection Information*
 - * *Socket Statistics*
 - * *Netstat*
 - * *Process and Ports*
 - *Connectivity Testing*
 - * *Ping*
 - * *Traceroute*
 - * *Port Testing*
 - *DNS Lookup*
 - * *dig*
 - * *Other DNS Tools*
 - *Data Transfer*
 - * *curl*
 - * *wget*
 - * *Remote Copy*
 - *Port Scanning with nmap*
 - * *Basic Scanning*
 - * *Scan Types*
 - * *Host Discovery*
 - * *Service and Version Detection*
 - * *Nmap Scripts (NSE)*
 - * *Output Formats*
 - * *Timing and Performance*
 - *Packet Capture with tcpdump*
 - * *Basic Capture*
 - * *Filtering*

- * *Output Options*
- * *Advanced Filters*
- *Firewall (iptables)*
 - * *View Rules*
 - * *Basic Rules*
 - * *Rule Management*
- *Network Bandwidth*

Network troubleshooting demands a diverse toolkit. Whether you're diagnosing connectivity issues with ping and traceroute, scanning ports with nmap, capturing packets with tcpdump, or configuring firewall rules with iptables, these commands form the foundation of network administration and security auditing.

An interactive demo script is available at [src/bash/net.sh](#) to help you experiment with the concepts covered in this cheatsheet.

```
./src/bash/net.sh          # Run all demos
./src/bash/net.sh dns      # Run DNS lookup demo
./src/bash/net.sh curl     # Run curl examples
./src/bash/net.sh --help   # Show all available sections
```

Network Configuration

Commands for viewing and configuring network interfaces and routing.

Interface Information

```
ip addr                # IP addresses
ip addr show eth0      # Specific interface
ip -4 addr             # IPv4 only
ip -6 addr             # IPv6 only
ip link               # Link layer info
ip link show eth0      # Interface status

ifconfig              # Legacy interface info
ifconfig eth0         # Specific interface
```

Interface Management

```
ip link set eth0 up    # Bring interface up
ip link set eth0 down  # Bring interface down
ip addr add 192.168.1.10/24 dev eth0 # Add IP address
ip addr del 192.168.1.10/24 dev eth0 # Remove IP address
```

Routing

```
ip route              # Routing table
ip route show         # Same as above
```

(continues on next page)

(continued from previous page)

```

ip route get 8.8.8.8           # Route to destination
ip route add default via 192.168.1.1 # Add default gateway
ip route add 10.0.0.0/8 via 192.168.1.1 # Add static route
ip route del 10.0.0.0/8       # Delete route

route -n                      # Legacy routing table
netstat -rn                   # Routing table

```

ARP Table

```

ip neigh           # ARP table
ip neigh show      # Same as above
arp -a             # Legacy ARP table
ip neigh flush all # Clear ARP cache

```

Connection Information

Commands for viewing active connections and listening ports.

Socket Statistics

```

ss -tuln           # TCP/UDP listening ports
ss -tulnp          # Include process info
ss -t              # TCP connections
ss -u              # UDP connections
ss -x              # Unix sockets
ss -s              # Socket summary
ss -o state established # Established connections

# Filter by port
ss -tuln | grep :80
ss -tuln sport = :443

# Filter by state
ss -t state time-wait
ss -t state established

```

Netstat

```

# Linux
netstat -tuln      # Listening ports
netstat -tulnp     # With process info
netstat -an        # All connections
netstat -s         # Protocol statistics
netstat -i         # Interface statistics

# macOS/BSD
netstat -an        # All connections
netstat -p tcp     # TCP connections
netstat -p udp     # UDP connections

```

(continues on next page)

(continued from previous page)

```
netstat -rn          # Routing table
netstat -an | grep LISTEN  # Listening ports
```

Process and Ports

```
lsof -i :80          # Process using port 80
lsof -i :8080 -i :443  # Multiple ports
lsof -i tcp          # All TCP connections
lsof -i @192.168.1.1  # Connections to host
fuser 80/tcp         # PID using port
```

Connectivity Testing

Commands for testing network connectivity and diagnosing issues.

Ping

```
ping host          # Continuous ping
ping -c 4 host     # 4 packets
ping -i 0.2 host   # 200ms interval
ping -s 1000 host  # Packet size 1000 bytes
ping -W 2 host     # 2 second timeout
ping -q -c 10 host # Quiet, summary only
```

Traceroute

```
traceroute host          # Trace route
traceroute -n host       # No DNS resolution
traceroute -T host       # TCP instead of UDP
traceroute -p 443 host   # Specific port
traceroute -m 15 host    # Max 15 hops

tracepath host           # MTU discovery
mtr host                 # Combined ping/traceroute
mtr -r -c 10 host        # Report mode
```

Port Testing

```
# netcat (nc)
nc -zv host 80          # Test TCP port
nc -zv host 20-30       # Port range
nc -uzv host 53         # Test UDP port
nc -w 3 -zv host 443    # 3 second timeout

# telnet
telnet host 80          # Connect to port

# bash built-in
timeout 3 bash -c "</dev/tcp/host/80" && echo "open"
```

DNS Lookup

Commands for querying DNS records and troubleshooting name resolution.

dig

```
dig example.com           # A record
dig example.com A         # Explicit A record
dig example.com AAAA      # IPv6 record
dig example.com MX        # Mail servers
dig example.com NS        # Name servers
dig example.com TXT        # TXT records
dig example.com ANY        # All records

dig +short example.com     # Short answer
dig +trace example.com     # Trace delegation
dig +noall +answer example.com # Answer only

dig @8.8.8.8 example.com  # Use specific DNS
dig -x 8.8.8.8            # Reverse lookup
```

Other DNS Tools

```
nslookup example.com      # Name server lookup
nslookup -type=mx example.com # MX records
nslookup example.com 8.8.8.8 # Use specific DNS

host example.com          # Simple lookup
host -t mx example.com    # MX records
host 8.8.8.8              # Reverse lookup

getent hosts example.com  # NSS lookup
resolvectl query example.com # systemd-resolved
```

Data Transfer

Commands for downloading files and making HTTP requests.

curl

```
curl https://example.com # GET request
curl -o file.html https://url # Save to file
curl -O https://url/file.zip # Save with original name
curl -I https://example.com # Headers only
curl -v https://example.com # Verbose output
curl -s https://example.com # Silent mode
curl -L https://example.com # Follow redirects

# POST requests
curl -X POST https://url
curl -d "data=value" https://url # POST with data
curl -d @file.json https://url # POST from file
```

(continues on next page)

(continued from previous page)

```
curl -H "Content-Type: application/json" -d '{"key":"value"}' https://url
```

Headers and auth

```
curl -H "Authorization: Bearer token" https://url
curl -u user:pass https://url      # Basic auth
curl -b "cookie=value" https://url # Send cookie
curl -c cookies.txt https://url    # Save cookies
```

Other options

```
curl -k https://url      # Ignore SSL errors
curl --connect-timeout 5 https://url
curl -x proxy:port https://url # Use proxy
```

wget

```
wget https://url/file      # Download file
wget -O name.zip https://url # Save with name
wget -c https://url/file    # Resume download
wget -q https://url         # Quiet mode
wget -b https://url         # Background

wget -r https://example.com/ # Recursive download
wget -r -np https://example.com/ # No parent
wget -m https://example.com/   # Mirror site
wget -i urls.txt               # Download from list

wget --limit-rate=1m https://url # Limit bandwidth
wget --user=u --password=p https://url
```

Remote Copy

```
scp file user@host:/path      # Copy to remote
scp user@host:/path/file .    # Copy from remote
scp -r dir user@host:/path     # Recursive copy
scp -P 2222 file user@host:/path # Custom port

rsync -avz src/ user@host:/dest/ # Sync directories
rsync -avz --delete src/ dest/   # Mirror with delete
rsync -avzP src/ dest/          # Progress bar
rsync --dry-run -avz src/ dest/  # Dry run
```

Port Scanning with nmap

Nmap is a powerful network scanner for security auditing and network discovery.

Basic Scanning

```
nmap host      # Default scan (top 1000 ports)
nmap -p 80 host # Single port
nmap -p 80,443,8080 host # Multiple ports
```

(continues on next page)

(continued from previous page)

```

nmap -p 1-1000 host      # Port range
nmap -p- host           # All 65535 ports
nmap -F host            # Fast scan (top 100)

nmap 192.168.1.0/24     # Scan subnet
nmap 192.168.1.1-50     # IP range
nmap -iL hosts.txt      # From file

```

Scan Types

```

nmap -sT host           # TCP connect scan
nmap -sS host           # SYN scan (stealth)
nmap -sU host           # UDP scan
nmap -sA host           # ACK scan
nmap -sN host           # NULL scan
nmap -sF host           # FIN scan
nmap -sX host           # Xmas scan

```

Host Discovery

```

nmap -sn 192.168.1.0/24 # Ping scan (no port scan)
nmap -Pn host           # Skip host discovery
nmap -PS22,80,443 host  # TCP SYN discovery
nmap -PA80,443 host     # TCP ACK discovery
nmap -PU53 host         # UDP discovery
nmap -PR 192.168.1.0/24 # ARP discovery (local)

```

Service and Version Detection

```

nmap -sV host           # Service version detection
nmap -sV --version-intensity 5 host # More aggressive
nmap -A host            # OS + version + scripts
nmap -O host            # OS detection
nmap --osscan-guess host # Aggressive OS guess

```

Nmap Scripts (NSE)

```

nmap --script=default host      # Default scripts
nmap --script=vuln host         # Vulnerability scripts
nmap --script=safe host         # Safe scripts only
nmap --script=http-headers host # Specific script
nmap --script=http-* host       # Wildcard scripts

# Common scripts
nmap --script=ssl-enum-ciphers -p 443 host
nmap --script=http-title host
nmap --script=ssh-hostkey host
nmap --script=dns-brute host
nmap --script=smb-os-discovery host

```

Output Formats

```
nmap -oN scan.txt host      # Normal output
nmap -oX scan.xml host      # XML output
nmap -oG scan.gnmap host    # Grepable output
nmap -oA scan host          # All formats
```

Timing and Performance

```
nmap -T0 host               # Paranoid (slowest)
nmap -T1 host               # Sneaky
nmap -T2 host               # Polite
nmap -T3 host               # Normal (default)
nmap -T4 host               # Aggressive
nmap -T5 host               # Insane (fastest)

nmap --min-rate 1000 host    # Min packets/sec
nmap --max-retries 2 host    # Max retries
```

Packet Capture with tcpdump

Tcpdump captures and analyzes network traffic for debugging and security analysis.

Basic Capture

```
tcpdump                     # Capture all traffic
tcpdump -i eth0             # Specific interface
tcpdump -i any              # All interfaces
tcpdump -c 100              # Capture 100 packets
tcpdump -w capture.pcap     # Write to file
tcpdump -r capture.pcap     # Read from file
```

Filtering

```
# By host
tcpdump host 192.168.1.1
tcpdump src 192.168.1.1
tcpdump dst 192.168.1.1

# By port
tcpdump port 80
tcpdump src port 443
tcpdump dst port 22
tcpdump portrange 8000-9000

# By protocol
tcpdump tcp
tcpdump udp
tcpdump icmp
tcpdump arp
```

(continues on next page)

(continued from previous page)

```
# Combine filters
tcpdump 'host 192.168.1.1 and port 80'
tcpdump 'src 192.168.1.1 and tcp'
tcpdump 'port 80 or port 443'
tcpdump 'not port 22'
```

Output Options

```
tcpdump -n           # No DNS resolution
tcpdump -nn          # No DNS or port names
tcpdump -v           # Verbose
tcpdump -vv          # More verbose
tcpdump -X           # Hex and ASCII
tcpdump -A           # ASCII only
tcpdump -e           # Show MAC addresses
tcpdump -tttt        # Human-readable time
```

Advanced Filters

```
# TCP flags
tcpdump 'tcp[tcpflags] & tcp-syn != 0'
tcpdump 'tcp[tcpflags] & tcp-rst != 0'
tcpdump 'tcp[13] & 2 != 0'          # SYN packets

# HTTP traffic
tcpdump -A 'tcp port 80 and (((ip[2:2] - ((ip[0]&0xf)<<2)) - ((tcp[12]&0xf0)>>2)) != 0)'
```

```
# Packet size
tcpdump 'greater 1000'              # Packets > 1000 bytes
tcpdump 'less 100'                  # Packets < 100 bytes
```

Firewall (iptables)

Commands for managing Linux firewall rules with iptables.

View Rules

```
iptables -L           # List rules
iptables -L -n         # Numeric output
iptables -L -v         # Verbose
iptables -L -n -v --line-numbers # With line numbers
iptables -S           # Rules as commands
iptables -t nat -L     # NAT table
```

Basic Rules

```
# Allow incoming SSH
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

(continues on next page)

(continued from previous page)

```
# Allow incoming HTTP/HTTPS
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow established connections
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Drop all other incoming
iptables -A INPUT -j DROP

# Allow from specific IP
iptables -A INPUT -s 192.168.1.100 -j ACCEPT

# Block specific IP
iptables -A INPUT -s 10.0.0.5 -j DROP
```

Rule Management

```
# Delete rule by number
iptables -D INPUT 3

# Delete rule by specification
iptables -D INPUT -p tcp --dport 80 -j ACCEPT

# Insert rule at position
iptables -I INPUT 1 -p tcp --dport 22 -j ACCEPT

# Flush all rules
iptables -F
iptables -t nat -F

# Save/restore rules
iptables-save > rules.txt
iptables-restore < rules.txt
```

Network Bandwidth

Commands for monitoring and testing network bandwidth.

```
# iftop - interface bandwidth
iftop                                # Default interface
iftop -i eth0                        # Specific interface
iftop -n                             # No DNS resolution

# nethogs - bandwidth by process
nethogs                             # All interfaces
nethogs eth0                        # Specific interface

# iperf3 - bandwidth testing
iperf3 -s                           # Server mode
iperf3 -c server_ip                 # Client mode
```

(continues on next page)

(continued from previous page)

```
iperf3 -c server_ip -t 30      # 30 second test
iperf3 -c server_ip -R        # Reverse mode
iperf3 -c server_ip -u        # UDP test
```

1.6.4 Hardware

Table of Contents

- *Hardware*
 - *CPU Information*
 - * *Basic CPU Info*
 - * *Detailed CPU Information*
 - * *CPU Performance*
 - *Memory Information*
 - * *Memory Usage*
 - * *Memory Hardware*
 - *Disk and Storage*
 - * *Disk Space Usage*
 - * *Block Devices*
 - * *Partition Management*
 - * *Filesystem Operations*
 - * *Disk Health and Performance*
 - *PCI Devices*
 - * *Basic Usage*
 - * *Filtering Devices*
 - * *Detailed Device Information*

Understanding your system’s hardware is crucial for performance tuning and troubleshooting. These commands reveal CPU specifications, memory configuration, disk health, and PCI device details—information essential for capacity planning and diagnosing hardware issues.

An interactive demo script is available at [src/bash/hardware.sh](#) to help you experiment with the concepts covered in this cheatsheet.

```
./src/bash/hardware.sh      # Run all demos
./src/bash/hardware.sh cpu  # Run CPU info demo
./src/bash/hardware.sh disk # Run disk info demo
./src/bash/hardware.sh --help # Show all available sections
```

CPU Information

Commands for viewing processor details, architecture, and performance characteristics.

Basic CPU Info

```
nproc                # Number of processing units
nproc --all          # All available processors
lscpu                # Detailed CPU architecture info
lscpu -e             # Extended CPU info table
lscpu -p             # Parseable output

# macOS
sysctl -n hw.logicalcpu    # Logical CPU count
sysctl -n hw.physicalcpu  # Physical core count
sysctl -a | grep machdep.cpu # Detailed CPU info
```

Detailed CPU Information

```
cat /proc/cpuinfo      # Raw CPU info
cat /proc/cpuinfo | grep "model name" | head -1
cat /proc/cpuinfo | grep "cpu MHz" # Current frequencies
cat /proc/cpuinfo | grep "cache size"

# CPU flags/features
cat /proc/cpuinfo | grep flags | head -1

# Count physical/logical CPUs
grep -c ^processor /proc/cpuinfo    # Logical CPUs
grep "physical id" /proc/cpuinfo | sort -u | wc -l # Physical CPUs
grep "cpu cores" /proc/cpuinfo | head -1          # Cores per CPU
```

CPU Performance

```
# CPU frequency scaling
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor

# CPU usage
mpstat -P ALL 1          # Per-CPU stats (sysstat)
sar -u 1 5              # CPU utilization over time
```

Memory Information

Commands for viewing RAM usage, configuration, and hardware details.

Memory Usage

```
free -h                # Human-readable memory usage
free -m                # Memory in MB
free -g                # Memory in GB
```

(continues on next page)

(continued from previous page)

```

free -s 2                                # Update every 2 seconds

# Detailed memory info
cat /proc/meminfo
cat /proc/meminfo | grep -E "MemTotal|MemFree|MemAvailable|Buffers|Cached"

# macOS
vm_stat                                # Virtual memory stats
top -l 1 | head -n 10                  # Memory summary

```

Memory Hardware

```

# Physical memory details (requires root)
dmidecode -t memory                    # Memory module info
dmidecode -t memory | grep -E "Size|Type|Speed|Manufacturer"

# Memory slots
dmidecode -t memory | grep -A 16 "Memory Device" | grep -E "Size|Locator|Type:"

# Numa topology
numactl --hardware                     # NUMA node info
lsmem                                  # Memory ranges

```

Disk and Storage

Commands for disk space, partitions, and storage device management.

Disk Space Usage

```

df -h                                # Disk space (human-readable)
df -h /                              # Specific mount point
df -hT                               # Include filesystem type
df -i                                # Inode usage

# Directory sizes
du -sh /path                         # Single directory size
du -sh /path/*                       # Size of subdirectories
du -sh /path/* | sort -h             # Sorted by size
du -sh /path/* | sort -rh | head -10 # Top 10 largest
du -d 1 -h /path                    # Depth-limited

# Find large files
find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null
du -ah /path | sort -rh | head -20    # Largest files/dirs

```

Block Devices

```

lsblk                                # List block devices
lsblk -f                             # Include filesystem info
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,FSTYPE

```

(continues on next page)

(continued from previous page)

```
lsblk -d -o NAME,SIZE,MODEL,SERIAL # Disk details
blkid                               # Block device attributes
blkid /dev/sda1                     # Specific device
```

Partition Management

```
fdisk -l                            # List partitions (MBR)
fdisk -l /dev/sda                   # Specific disk
gdisk -l /dev/sda                   # GPT partitions
parted -l                           # All partition tables

# Partition info
cat /proc/partitions
sfdisk -l /dev/sda                  # Scriptable fdisk
```

Filesystem Operations

```
# Mount/unmount
mount /dev/sda1 /mnt                # Mount device
umount /mnt                         # Unmount
mount -o remount,rw /               # Remount with options

# Filesystem info
tune2fs -l /dev/sda1                # ext filesystem details
xfs_info /dev/sda1                  # XFS filesystem info
dumpe2fs /dev/sda1 | head -50       # ext superblock info

# Filesystem check
fsck /dev/sda1                      # Check filesystem
e2fsck -f /dev/sda1                 # Force check ext
```

Disk Health and Performance

```
# SMART monitoring
smartctl -a /dev/sda                # All SMART data
smartctl -H /dev/sda                # Health status
smartctl -t short /dev/sda          # Run short test
smartctl -l selftest /dev/sda       # Test results

# I/O statistics
iostat -x 1                         # Extended I/O stats
iostat -d -x /dev/sda 1             # Specific device
iotop                               # I/O by process

# Disk benchmark
hdparm -Tt /dev/sda                 # Read speed test
dd if=/dev/zero of=test bs=1G count=1 oflag=direct # Write test
```

PCI Devices

The `lspci` command lists all PCI devices including network cards, GPUs, storage controllers, and other hardware components.

Basic Usage

```
lspci                # List all PCI devices
lspci -v             # Verbose output
lspci -vv           # Very verbose
lspci -vvv          # Maximum verbosity

lspci -k             # Show kernel drivers
lspci -nn           # Show numeric IDs
lspci -mm           # Machine-readable output
```

Filtering Devices

```
# By device class
lspci | grep -i vga      # Graphics cards
lspci | grep -i network  # Network devices
lspci | grep -i audio    # Audio devices
lspci | grep -i usb      # USB controllers
lspci | grep -i sata     # SATA controllers
lspci | grep -i nvme     # NVMe controllers

# By vendor
lspci -d 10de:           # NVIDIA devices (vendor 10de)
lspci -d 8086:           # Intel devices
lspci -d 1002:           # AMD devices

# By slot
lspci -s 00:02.0        # Specific slot
lspci -s 00:02.0 -v     # Detailed info for slot
```

Detailed Device Information

```
# Show device tree
lspci -t               # Tree view
lspci -tv              # Tree with details

# Kernel modules
lspci -k | grep -A 3 "VGA" # GPU driver info
lspci -v -s 00:02.0 | grep "Kernel"

# Device capabilities
lspci -vvv -s 00:1f.0 | grep -i capabilities

# Update PCI database
update-pciids          # Download latest IDs
```

1.6.5 GPU

Table of Contents

- *GPU*
 - *Basic GPU Information*
 - *nvidia-smi Basics*
 - *nvidia-smi Monitoring*
 - *nvidia-smi Query Format*
 - * *Common Query Fields*
 - *nvidia-smi Management*
 - *Multi-GPU Systems*
 - *CUDA Environment*

For deep learning, scientific computing, and graphics workloads, nvidia-smi is the essential tool for GPU monitoring and management. From checking memory usage to tuning power limits and managing multi-GPU configurations, these commands help maximize GPU utilization and diagnose performance bottlenecks.

An interactive demo script is available at [src/bash/gpu.sh](#) to help you experiment with the concepts covered in this cheatsheet.

```
./src/bash/gpu.sh           # Run all demos
./src/bash/gpu.sh query     # Run query format demo
./src/bash/gpu.sh --help    # Show all available sections
```

Basic GPU Information

Commands for viewing graphics card details and driver information.

```
lspci | grep -i vga          # List graphics cards
lspci | grep -i nvidia       # NVIDIA devices
lspci -v -s $(lspci | grep -i vga | cut -d' ' -f1) # Detailed GPU info

# OpenGL info
glxinfo | grep -i "render\|vendor\|version"

# Vulkan info
vulkaninfo --summary
```

nvidia-smi Basics

The nvidia-smi tool provides monitoring and management capabilities for NVIDIA GPUs.

```
nvidia-smi                  # GPU status overview
nvidia-smi -L               # List GPUs
nvidia-smi -q               # Full query (all details)
nvidia-smi -q -d MEMORY     # Memory details
nvidia-smi -q -d UTILIZATION # Utilization details
```

(continues on next page)

(continued from previous page)

```
nvidia-smi -q -d TEMPERATURE      # Temperature details
nvidia-smi -q -d POWER             # Power details
nvidia-smi -q -d CLOCK             # Clock speeds
```

nvidia-smi Monitoring

Commands for continuous GPU monitoring and process tracking.

```
# Continuous monitoring
nvidia-smi -l 1                  # Update every 1 second
nvidia-smi dmon -s u             # Device monitoring (utilization)
nvidia-smi dmon -s p             # Power monitoring
nvidia-smi dmon -s t             # Temperature monitoring
nvidia-smi dmon -s m             # Memory monitoring

# Process monitoring
nvidia-smi pmon -s u             # Process utilization
nvidia-smi pmon -s m             # Process memory

# Watch specific metrics
watch -n 1 nvidia-smi --query-gpu=utilization.gpu,utilization.memory,temperature.gpu,
↪ power.draw --format=csv
```

nvidia-smi Query Format

Custom queries allow extracting specific GPU metrics in various formats.

```
# Custom query output
nvidia-smi --query-gpu=name,memory.total,memory.used,memory.free --format=csv
nvidia-smi --query-gpu=gpu_name,gpu_bus_id,vbios_version --format=csv
nvidia-smi --query-gpu=temperature.gpu,fan.speed,power.draw,power.limit --format=csv
nvidia-smi --query-gpu=clocks.gr,clocks.mem,clocks.sm --format=csv

# Query specific GPU
nvidia-smi -i 0 --query-gpu=name,memory.used --format=csv
nvidia-smi -i 0,1 --query-gpu=name,utilization.gpu --format=csv

# Process queries
nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv

# Available query options
nvidia-smi --help-query-gpu      # List all GPU query options
nvidia-smi --help-query-compute-apps # List process query options
```

Common Query Fields

```
# GPU identification
name, gpu_name, gpu_bus_id, gpu_serial, gpu_uuid, vbios_version

# Memory
memory.total, memory.used, memory.free
```

(continues on next page)

(continued from previous page)

```
# Utilization
utilization.gpu, utilization.memory

# Temperature and power
temperature.gpu, fan.speed, power.draw, power.limit

# Clocks
clocks.gr, clocks.mem, clocks.sm, clocks.max.gr, clocks.max.mem

# Performance
pstate, performance_state
```

nvidia-smi Management

Commands for configuring GPU settings and performance tuning.

```
# Set persistence mode (reduces latency)
nvidia-smi -pm 1          # Enable
nvidia-smi -pm 0          # Disable

# Set power limit (watts)
nvidia-smi -pl 250        # Set to 250W

# Set GPU clocks
nvidia-smi -lgc 1200,1800 # Lock graphics clock range
nvidia-smi -rgc           # Reset graphics clocks

# Set compute mode
nvidia-smi -c 0           # Default
nvidia-smi -c 1           # Exclusive thread
nvidia-smi -c 2           # Prohibited
nvidia-smi -c 3           # Exclusive process

# Reset GPU
nvidia-smi -r             # Reset GPU

# Enable/disable ECC
nvidia-smi -e 1           # Enable ECC
nvidia-smi -e 0           # Disable ECC
```

Multi-GPU Systems

Commands for managing multiple GPUs, topology, and interconnects.

```
# Topology
nvidia-smi topo -m        # GPU topology matrix
nvidia-smi topo -p        # P2P capabilities

# NVLink status
nvidia-smi nvlink -s      # NVLink status
nvidia-smi nvlink -c      # NVLink capabilities
```

(continues on next page)

(continued from previous page)

```
# MIG (Multi-Instance GPU) - A100/H100
nvidia-smi mig -lgip           # List GPU instance profiles
nvidia-smi mig -cgi 9,9        # Create GPU instances
nvidia-smi mig -dci           # Destroy compute instances
```

CUDA Environment

Environment variables and commands for CUDA configuration.

```
# Check CUDA version
nvcc --version
cat /usr/local/cuda/version.txt

# CUDA environment variables
export CUDA_VISIBLE_DEVICES=0      # Use only GPU 0
export CUDA_VISIBLE_DEVICES=0,1    # Use GPUs 0 and 1
export CUDA_VISIBLE_DEVICES=""     # Hide all GPUs

# Check CUDA device properties
nvidia-smi -q -d COMPUTE

# cuDNN version
cat /usr/local/cuda/include/cudnn_version.h | grep CUDNN_MAJOR -A 2
```

1.6.6 Systemd

Table of Contents

- *Systemd*
 - *Introduction*
 - *Services Management*
 - *Viewing Logs with journalctl*
 - *Service Unit File*
 - * *Service Types*
 - * *Restart Policies*
 - *Timer Unit File*
 - * *Managing Timers*
 - * *Timer Unit Example*
 - * *Timer Options*
 - * *Calendar Syntax Examples*
 - *User Services*
 - *Analyzing Boot Performance*

- *Core Dumps with coredumpctl*
- *Useful Paths*

Introduction

Systemd is the standard init system and service manager for most modern Linux distributions including Ubuntu, Debian, Fedora, CentOS, and Arch Linux. It replaces the traditional SysV init system with a more powerful and flexible approach to managing system startup, services, logging, and system state.

Systemd uses unit files to define services, timers, mount points, and other system resources. These declarative configuration files specify dependencies, execution parameters, and restart behavior, making service management more predictable and easier to debug than shell scripts.

Services Management

The `systemctl` command is the primary tool for controlling systemd services. It allows administrators to start, stop, restart, enable, and disable services, as well as query their status and manage dependencies. Understanding these commands is essential for Linux system administration.

```
# Start/stop/restart a service
$ systemctl start app.service
$ systemctl stop app.service
$ systemctl restart app.service

# Reload service configuration (without restart)
$ systemctl reload app.service

# Reload systemd after modifying unit files
$ systemctl daemon-reload

# Enable/disable service at boot
$ systemctl enable app.service
$ systemctl disable app.service

# Check service status
$ systemctl status app.service
$ systemctl is-active app.service
$ systemctl is-enabled app.service

# List units
$ systemctl list-units                # all loaded units
$ systemctl list-units --type=service # only services
$ systemctl list-units --state=failed  # failed units
$ systemctl list-unit-files           # all installed unit files

# Show unit dependencies
$ systemctl list-dependencies app.service

# Mask/unmask (completely disable a service)
$ systemctl mask app.service
$ systemctl unmask app.service
```

Viewing Logs with journalctl

Systemd includes a centralized logging system called the journal, which collects logs from all services, the kernel, and system messages. The `journalctl` command provides powerful filtering and querying capabilities that make troubleshooting much easier than parsing traditional log files in `/var/log`.

The journal stores logs in a binary format with rich metadata including timestamps, priority levels, unit names, and process IDs. This enables efficient filtering by service, time range, or severity level.

```
# View logs for a service
$ journalctl -u app.service

# Follow logs in real-time (like tail -f)
$ journalctl -u app.service -f

# Show logs since last boot
$ journalctl -u app.service -b

# Show logs from last hour
$ journalctl -u app.service --since "1 hour ago"

# Show logs between dates
$ journalctl -u app.service --since "2024-01-01" --until "2024-01-02"

# Filter by priority (emerg, alert, crit, err, warning, notice, info, debug)
$ journalctl -u app.service -p err

# Show kernel messages (like dmesg)
$ journalctl -k

# Show disk usage by journal
$ journalctl --disk-usage

# Clean old logs (keep last 1G)
$ journalctl --vacuum-size=1G

# Clean logs older than 2 weeks
$ journalctl --vacuum-time=2weeks
```

Service Unit File

A systemd service unit file is a configuration file that defines how to manage a background service or daemon. Unit files use an INI-style format with three main sections: `[Unit]` for metadata and dependencies, `[Service]` for process execution parameters, and `[Install]` for installation targets.

Place custom unit files in `/etc/systemd/system/` for system services. After creating or modifying a unit file, run `systemctl daemon-reload` to reload the systemd configuration.

```
# /etc/systemd/system/app.service
#
# $ systemctl daemon-reload
# $ systemctl enable app.service
# $ systemctl start app.service

[Unit]
```

(continues on next page)

(continued from previous page)

```

Description=My Application
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=appuser
Group=appgroup
WorkingDirectory=/path/to/app
ExecStart=/usr/bin/python3 app.py
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=10

# Environment variables
Environment=NODE_ENV=production
EnvironmentFile=/etc/app/env

# Security hardening options
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target

```

Service Types

The `Type=` directive tells systemd how the service notifies that it has finished starting. Choosing the correct type ensures systemd accurately tracks the service state and manages dependencies properly.

<code>Type=simple</code>	Default. Process started by <code>ExecStart</code> is the main process.
<code>Type=forking</code>	Process forks and parent exits. Use with <code>PIDFile=</code> .
<code>Type=oneshot</code>	Process exits after completing. Use for scripts.
<code>Type=notify</code>	Like simple, but service sends notification when ready.
<code>Type=idle</code>	Like simple, but waits until other jobs finish.

Restart Policies

The `Restart=` directive controls when systemd automatically restarts a service after it exits. Use `RestartSec=` to add a delay between restart attempts, preventing rapid restart loops that could overwhelm the system.

<code>Restart=no</code>	Don't restart (default)
<code>Restart=on-success</code>	Restart only on clean exit (exit code 0)
<code>Restart=on-failure</code>	Restart on non-zero exit, signal, timeout
<code>Restart=on-abnormal</code>	Restart on signal, timeout, watchdog
<code>Restart=always</code>	Always restart regardless of exit status

Timer Unit File

Systemd timers are a modern replacement for cron jobs, offering better integration with the service manager, logging, and dependency handling. A timer unit activates a corresponding service unit based on time events. Unlike cron, timers can trigger on boot, on service activation, or on calendar schedules.

Each timer requires a matching service file with the same base name (e.g., `backup.timer` triggers `backup.service`).

Managing Timers

```
# List all active timers with next/last run times
$ systemctl list-timers

# List all timers including inactive
$ systemctl list-timers --all

# Show when a specific timer will next execute
$ systemctl list-timers backup.timer

# Example output:
# NEXT                                LEFT                LAST                PASSED              _
→UNIT          ACTIVATES
# Mon 2024-01-08 02:00:00 UTC 5h left      Sun 2024-01-07 02:00:00 UTC 18h ago      _
→backup.timer  backup.service

# Enable and start a timer
$ systemctl enable backup.timer
$ systemctl start backup.timer

# Check timer status
$ systemctl status backup.timer

# Manually trigger the associated service (for testing)
$ systemctl start backup.service

# View timer logs
$ journalctl -u backup.timer
$ journalctl -u backup.service
```

Timer Unit Example

```
# /etc/systemd/system/backup.timer
#
# $ systemctl daemon-reload
# $ systemctl enable backup.timer
# $ systemctl start backup.timer

[Unit]
Description=Daily backup timer

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
```

(continues on next page)

(continued from previous page)

`RandomizedDelaySec=1h`**[Install]**`WantedBy=timers.target``# /etc/systemd/system/backup.service`**[Unit]**`Description=Backup job`**[Service]**`Type=oneshot``ExecStart=/usr/local/bin/backup.sh`

Timer Options

Timers support two types of triggers: monotonic timers that fire relative to a specific event (boot, activation), and realtime timers that fire based on calendar time. Use `Persistent=true` to run missed jobs after system downtime.

<code>OnBootSec=10min</code>	Run 10 minutes after boot
<code>OnUnitActiveSec=1h</code>	Run 1 hour after service last activated
<code>OnCalendar=daily</code>	Run daily at midnight
<code>OnCalendar=weekly</code>	Run weekly on Monday at midnight
<code>OnCalendar=hourly</code>	Run every hour
<code>OnCalendar=*:0/15</code>	Run every 15 minutes
<code>Persistent=true</code>	Run immediately if missed while system was off
<code>RandomizedDelaySec=</code>	Add random delay to prevent thundering herd

Calendar Syntax Examples

Systemd calendar expressions use the format `DayOfWeek Year-Month-Day Hour:Minute:Second`. Use `*` as a wildcard for any value and `/` for intervals. The `systemd-analyze calendar` command validates and normalizes calendar expressions.

<code>*-*-* 00:00:00</code>	Daily at midnight
<code>*-*-* *:00:00</code>	Every hour
<code>*-*-* *:*:00</code>	Every minute
<code>Mon *-*-* 00:00:00</code>	Every Monday at midnight
<code>Mon,Fri *-*-* 17:00</code>	Monday and Friday at 5pm
<code>*-*-*01 00:00:00</code>	First day of every month
<code>*-01-01 00:00:00</code>	January 1st every year
<code>2024-*-* 00:00:00</code>	Every day in 2024

Test and validate calendar expressions
\$ `systemd-analyze calendar "Mon *-*-* 09:00"`

User Services

Systemd supports user-level services that run without root privileges in the user's session. User services are ideal for personal daemons, development servers, or user-specific background tasks like syncing or notifications.

By default, user services only run while the user is logged in. Enable lingering to keep services running after logout, which is essential for servers or long-running background tasks.

```
# Enable lingering (services run without login)
$ sudo loginctl enable-linger $USER

# Create user service directory
$ mkdir -p ~/.config/systemd/user/

# Place unit files in ~/.config/systemd/user/
# Manage with --user flag
$ systemctl --user daemon-reload
$ systemctl --user enable app.service
$ systemctl --user start app.service
$ systemctl --user status app.service

# View user service logs
$ journalctl --user -u app.service -f
```

Enable persistent journal storage to preserve user service logs across reboots:

```
$ sudo vim /etc/systemd/journald.conf

[Journal]
Storage=persistent

$ sudo systemctl restart systemd-journald
```

Analyzing Boot Performance

Systemd provides built-in tools to analyze boot performance and identify slow services that delay system startup. Use these commands to optimize boot time and diagnose startup issues.

```
# Show total boot time breakdown
$ systemd-analyze

# Show time taken by each unit (sorted by duration)
$ systemd-analyze blame

# Show critical chain (services blocking boot)
$ systemd-analyze critical-chain

# Generate visual boot chart as SVG
$ systemd-analyze plot > boot.svg
```

Core Dumps with coredumpctl

Systemd includes `systemd-coredump` which automatically captures and stores core dumps when processes crash. The `coredumpctl` command lets you list, inspect, and debug these core dumps. This is invaluable for debugging crashes in C/C++ applications running as systemd services.

```
# List all core dumps
$ coredumpctl list
```

(continues on next page)

(continued from previous page)

```

# List core dumps for a specific executable
$ coredumpctl list /usr/bin/myapp

# Show info about the most recent core dump
$ coredumpctl info

# Show info for a specific PID
$ coredumpctl info 12345

# Launch debugger (gdb) on most recent core dump
$ coredumpctl debug

# Debug a specific executable's core dump
$ coredumpctl debug /usr/bin/myapp

# Export core dump to a file
$ coredumpctl dump -o core.dump

# Export core dump for specific PID
$ coredumpctl dump 12345 -o core.dump

```

Configure core dump storage in `/etc/systemd/coredump.conf`:

```

[Coredump]
Storage=external      # external, journal, or none
Compress=yes          # compress stored core dumps
MaxUse=1G             # max disk space for core dumps
KeepFree=1G           # min free space to maintain
ProcessSizeMax=2G     # max size of process to dump

```

Useful Paths

<code>/etc/systemd/system/</code>	System unit files (admin-created)
<code>/usr/lib/systemd/system/</code>	System unit files (package-installed)
<code>/run/systemd/system/</code>	Runtime unit files (temporary)
<code>~/.config/systemd/user/</code>	User unit files
<code>/etc/systemd/journald.conf</code>	Journal configuration
<code>/etc/systemd/coredump.conf</code>	Core dump configuration
<code>/etc/systemd/system.conf</code>	System manager configuration
<code>/var/lib/systemd/coredump/</code>	Stored core dumps

1.7 Debugging & Profiling

When software misbehaves or underperforms, the ability to inspect program state and measure execution characteristics becomes invaluable. Debugging and profiling transform opaque failures into understandable problems with actionable solutions.

This section covers essential tools for C/C++ development: GDB for interactive debugging, Valgrind for memory error detection, compiler sanitizers for fast runtime checks, tracing tools for system call analysis, perf for CPU profiling, and NVIDIA Nsight for GPU optimization.

1.7.1 GDB

Table of Contents

- *GDB*
 - *Introduction*
 - *Command Shortcuts*
 - *Getting Started*
 - *Text User Interface*
 - *Running Programs*
 - *Breakpoints*
 - *Watchpoints*
 - *Examining Variables*
 - *Examining Memory*
 - *Stack Traces*
 - *Debugging Crashes*
 - *Reverse Debugging*
 - *Multi-threaded Debugging*
 - *Useful Settings*
 - *Dumping Debug Data to Log File*
 - *CUDA Debugging with cuda-gdb*

Introduction

GDB (GNU Debugger) is the standard debugger for C and C++ programs on Unix-like systems. It allows you to pause program execution, inspect variables and memory, step through code line by line, and understand why programs crash or behave unexpectedly. Mastering GDB is essential for diagnosing segmentation faults, memory corruption, race conditions, and logic errors.

Compile your program with `-g` to include debug symbols, which map machine code back to source lines and variable names. Without debug symbols, GDB can still debug at the assembly level, but source-level debugging is far more productive.

Command Shortcuts

Most GDB commands have single or two-letter shortcuts:

Command	Shortcut	Description
run	r	Run program
continue	c	Continue execution
next	n	Step over (next line)
step	s	Step into function
finish	fin	Run until function returns

(continues on next page)

(continued from previous page)

until	u	Run until line/address
break	b	Set breakpoint
delete	d	Delete breakpoint
info	i	Show info (i b = info breakpoints)
print	p	Print variable
backtrace	bt	Show call stack
frame	f	Select stack frame
list	l	List source code
quit	q	Exit GDB
reverse-next	rn	Reverse step over
reverse-step	rs	Reverse step into
reverse-continue	rc	Reverse continue

Press Enter to repeat the last command (useful for repeated n or s).

Getting Started

Compile with debug symbols and load into GDB:

```
$ gcc -g -Wall -o myprogram main.c
$ gdb ./myprogram
```

Common startup commands:

```
$ gdb ./myprogram           # start GDB with executable
$ gdb ./myprogram core      # debug a core dump
$ gdb -p 1234               # attach to running process
$ gdb --args ./prog arg1 arg2 # pass arguments to program
```

Text User Interface

GDB's TUI mode provides a visual interface showing source code alongside the command prompt, similar to an IDE. This makes it easier to see context while stepping through code.

Key bindings:

- Ctrl-x a - Toggle TUI mode on/off
- Ctrl-x o - Switch focus between windows
- Ctrl-x 1 - Single window layout (source + command)
- Ctrl-x 2 - Two window layout (source + assembly + command)
- Ctrl-l - Refresh display (useful if output corrupts the screen)

```
(gdb) tui enable           # enable TUI mode
(gdb) layout src           # show source window
(gdb) layout asm          # show assembly window
(gdb) layout split        # show source and assembly
(gdb) layout regs         # show registers
```

Running Programs

Control program execution:

```
(gdb) run                # run program from start
(gdb) run arg1 arg2      # run with arguments
(gdb) start              # run and stop at main()
(gdb) continue           # continue after breakpoint (c)
(gdb) next               # step over function calls (n)
(gdb) step               # step into function calls (s)
(gdb) finish             # run until current function returns
(gdb) until 42           # run until line 42
(gdb) kill               # stop the program
```

Breakpoints

Breakpoints pause execution at specific locations. GDB supports line breakpoints, function breakpoints, and conditional breakpoints that only trigger when an expression is true.

```
(gdb) break main          # break at function main
(gdb) break 42            # break at line 42 in current file
(gdb) break file.c:42     # break at line 42 in file.c
(gdb) break func if x > 10 # conditional breakpoint
(gdb) tbreak main        # temporary breakpoint (auto-deleted)

(gdb) info breakpoints    # list all breakpoints
(gdb) delete 1            # delete breakpoint 1
(gdb) delete              # delete all breakpoints
(gdb) disable 1           # disable breakpoint 1
(gdb) enable 1            # enable breakpoint 1
(gdb) clear func          # clear breakpoints at func
```

Watchpoints

Watchpoints pause execution when a variable's value changes, useful for tracking down unexpected modifications to data.

```
(gdb) watch var           # break when var changes
(gdb) watch *0x7fff5678  # watch memory address
(gdb) watch expr          # break when expression changes
(gdb) rwatch var          # break when var is read
(gdb) awatch var          # break on read or write
(gdb) info watchpoints    # list watchpoints
```

Examining Variables

Inspect variable values and types during debugging:

```
(gdb) print var           # print variable value (p)
(gdb) print/x var         # print in hexadecimal
(gdb) print/t var         # print in binary
(gdb) print arr[0]@10     # print 10 elements of array
(gdb) print *ptr          # dereference pointer
(gdb) print sizeof(var)   # print size
```

(continues on next page)

(continued from previous page)

```
(gdb) ptype var           # print type of variable
(gdb) whatis var           # print type (brief)
(gdb) info locals          # print all local variables
(gdb) info args            # print function arguments

(gdb) display var          # print var after each step
(gdb) undisplay 1          # stop displaying
```

Examining Memory

The `x` command examines raw memory. Format: `x/NFU address` where `N` is count, `F` is format (`x`=hex, `d`=decimal, `s`=string, `i`=instruction), and `U` is unit size (`b`=byte, `h`=halfword, `w`=word, `g`=giant/8-byte).

```
(gdb) x/s str              # print as string
(gdb) x/10c str            # print 10 characters
(gdb) x/4xw &var           # print 4 words in hex
(gdb) x/10i $pc             # print 10 instructions at PC
(gdb) x/20xb buf           # print 20 bytes in hex
```

Example with a character array:

```
(gdb) x/s arr
0x7fffffff620: "Hello, World!"
(gdb) x/5c arr
0x7fffffff620: 72 'H' 101 'e' 108 'l' 108 'l' 111 'o'
(gdb) x/5xb arr
0x7fffffff620: 0x48 0x65 0x6c 0x6c 0x6f
```

Stack Traces

Examine the call stack to understand how execution reached the current point:

```
(gdb) backtrace             # show call stack (bt)
(gdb) backtrace full        # show stack with local variables
(gdb) backtrace 5           # show only 5 frames
(gdb) where                 # same as backtrace

(gdb) frame 2               # select frame 2 (f 2)
(gdb) up                    # move up one frame
(gdb) down                  # move down one frame
(gdb) info frame            # detailed frame info
```

Debugging Crashes

When a program crashes with a segmentation fault, GDB helps identify the cause:

```
$ gdb ./myprogram
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x000055555555149 in func () at main.c:10
10      *ptr = 42;
```

(continues on next page)

(continued from previous page)

```
(gdb) bt
#0  0x0000555555555149 in func () at main.c:10
#1  0x0000555555555180 in main () at main.c:20
(gdb) print ptr
$1 = (int *) 0x0
```

Debug a core dump after a crash:

```
$ ulimit -c unlimited      # enable core dumps
$ ./myprogram              # run and crash
$ gdb ./myprogram core     # analyze core dump
(gdb) bt                   # see where it crashed
```

Reverse Debugging

GDB can record execution and step backwards, invaluable for finding the cause of bugs that manifest later than their origin.

```
(gdb) record               # start recording
(gdb) continue             # run program
(gdb) reverse-next         # step backwards (rn)
(gdb) reverse-step        # step backwards into functions (rs)
(gdb) reverse-continue    # run backwards to previous breakpoint
(gdb) record stop          # stop recording
```

Multi-threaded Debugging

Debug programs with multiple threads:

```
(gdb) info threads         # list all threads
(gdb) thread 2             # switch to thread 2
(gdb) thread apply all bt  # backtrace all threads
(gdb) set scheduler-locking on # only run current thread
```

Useful Settings

Customize GDB behavior:

```
(gdb) set print pretty on  # format structs nicely
(gdb) set print array on   # format arrays nicely
(gdb) set pagination off   # don't pause output
(gdb) set logging on       # log output to gdb.txt
(gdb) set history save on  # save command history
```

Define custom commands in ~/.gdbinit:

```
# ~/.gdbinit
set print pretty on
set pagination off

define sf
  where
```

(continues on next page)

(continued from previous page)

```

info args
info locals
end

```

Dumping Debug Data to Log File

Capture GDB output to a file for later analysis or sharing:

```

# Attach to running process and dump backtrace
$ gdb -batch -p 110 -ex "set pagination off" -ex "thread apply all bt" | tee gdb.log

# Debug executable and log all output
$ gdb -batch -x commands.txt ./myprogram | tee gdb.log

# Inside GDB session
(gdb) set logging file output.log
(gdb) set logging on
(gdb) thread apply all bt
(gdb) set logging off

```

The `-batch` flag runs GDB in non-interactive mode and exits after executing commands. The `-ex` flag executes a single command.

For more complex debugging sessions, write commands to a file and run with `-x`. Use `tee` to display output while saving to a file:

```

$ cat > /tmp/cuda_cmds.gdb <<'EOF'
set pagination off
set logging file /tmp/cuda_debug.log
set logging on

printf "=== CUDA KERNELS ===\n"
info cuda kernels
printf "\n=== CUDA BLOCKS (kernel 0) ===\n"
info cuda blocks
printf "\n=== CUDA THREADS (first 32) ===\n"
info cuda threads 0 32
printf "\n=== BACKTRACE (kernel 0, block 0, thread 0) ===\n"
cuda kernel 0 block 0 thread 0
bt
printf "\n=== BACKTRACE (kernel 0, block 1, thread 0) ===\n"
cuda kernel 0 block 1 thread 0
bt
printf "\n=== HOST THREADS ===\n"
info threads
printf "\n=== HOST BACKTRACE (main thread) ===\n"
thread 1
bt
set logging off
detach
quit
EOF
$ cuda-gdb -batch -p <PID> -x /tmp/cuda_cmds.gdb

```


Alternatively, pipe commands directly. This example attaches to a running CUDA process and dumps kernel state, thread backtraces, and host thread information to a log file:

```
$ GDB_CMDS=$(cat <<'EOF'
set pagination off
set logging file /tmp/cuda_debug.log
set logging on
printf "=== CUDA KERNELS ===\n"
info cuda kernels
printf "\n=== HOST THREADS ===\n"
info threads
thread apply all bt
set logging off
detach
quit
EOF
)
$ echo "$GDB_CMDS" | cuda-gdb -batch -p $PID 2>&1
```

CUDA Debugging with cuda-gdb

NVIDIA's cuda-gdb extends GDB for debugging CUDA kernels running on the GPU. It supports breakpoints in device code, inspecting GPU threads and memory, and switching between CPU and GPU contexts.

Compile with debug symbols for both host and device:

```
$ nvcc -g -G -o myprogram main.cu # -g for host, -G for device
$ cuda-gdb ./myprogram
```

Basic CUDA debugging commands:

```
(cuda-gdb) break main # breakpoint in host code
(cuda-gdb) break mykernel # breakpoint in kernel
(cuda-gdb) run

(cuda-gdb) info cuda threads # list all GPU threads
(cuda-gdb) info cuda kernels # list active kernels
(cuda-gdb) info cuda blocks # list thread blocks
(cuda-gdb) info cuda lanes # list lanes in current warp
```

Switch between GPU threads:

```
(cuda-gdb) cuda thread # show current thread
(cuda-gdb) cuda thread 0 0 0 # switch to thread (0,0,0)
(cuda-gdb) cuda block 1 0 0 # switch to block (1,0,0)
(cuda-gdb) cuda kernel 0 # switch to kernel 0

# Thread coordinates: (threadIdx.x, threadIdx.y, threadIdx.z)
# Block coordinates: (blockIdx.x, blockIdx.y, blockIdx.z)
```

Inspect GPU memory and variables:

```
(cuda-gdb) print threadIdx.x # print thread index
(cuda-gdb) print blockIdx.x # print block index
(cuda-gdb) print blockDim.x # print block dimensions
```

(continues on next page)

(continued from previous page)

```
(cuda-gdb) print arr[threadIdx.x]    # print device array element
(cuda-gdb) info cuda devices          # list GPU devices
(cuda-gdb) info cuda sms              # list streaming multiprocessors
```

Conditional breakpoints in kernels:

```
# Break only for specific thread
(cuda-gdb) break mykernel if threadIdx.x == 0 && blockIdx.x == 0

# Break when variable has specific value
(cuda-gdb) break kernel.cu:42 if result > 100
```

Memory checking with cuda-memcheck:

```
$ cuda-memcheck ./myprogram           # detect memory errors
$ cuda-memcheck --tool racecheck ./prog # detect race conditions
$ cuda-memcheck --tool initcheck ./prog # detect uninitialized memory
```

1.7.2 Valgrind

Table of Contents

- *Valgrind*
 - *Introduction*
 - *Memcheck: Memory Errors*
 - *Leak Types*
 - *Suppressing False Positives*
 - *Cachegrind: Cache Profiling*
 - *Callgrind: Call Graph Profiling*
 - *Helgrind: Thread Errors*
 - *Massif: Heap Profiling*
 - *Common Workflows*

Introduction

Valgrind is a dynamic binary instrumentation framework that provides powerful tools for detecting memory errors, profiling cache behavior, and analyzing program execution. It works by running your program in a virtual machine that intercepts every memory access, enabling detection of bugs that would otherwise cause silent corruption, security vulnerabilities, or intermittent crashes.

The most widely used Valgrind tool is Memcheck, which detects memory leaks, use of uninitialized memory, buffer overflows, use-after-free errors, double frees, and mismatched allocation/deallocation (e.g., using `free()` on memory allocated with `new`). These errors are notoriously difficult to debug because they often don't crash immediately—instead, they corrupt memory that may not be accessed until much later.

While Valgrind adds significant overhead (typically 10-50x slower execution), it finds bugs that are nearly impossible to detect through code review or testing alone. The detailed error reports include full stack traces showing exactly where the error occurred and where the problematic memory was allocated.

Memcheck: Memory Errors

Memcheck is Valgrind's default tool and the most commonly used. It tracks every byte of memory, recording whether it has been allocated, initialized, and freed. When your program accesses memory incorrectly, Memcheck reports the error with a stack trace.

```
$ gcc -g -o myprogram main.c
$ valgrind ./myprogram
$ valgrind --leak-check=full ./myprogram
$ valgrind --leak-check=full --show-leak-kinds=all ./myprogram
```

Common options:

```
--leak-check=full           # detailed leak information
--show-leak-kinds=all       # show all leak types
--track-origins=yes         # track uninitialized value origins
--verbose                   # more detailed output
-v                           # verbose
--log-file=valgrind.log     # write output to file
```

Example output for memory leak:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 100 bytes in 1 blocks
==12345==    total heap usage: 10 allocs, 9 frees, 1,024 bytes allocated
==12345==
==12345== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x4C2FB0F: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400537: main (main.c:10)
```

Example output for invalid read:

```
==12345== Invalid read of size 4
==12345==    at 0x400520: main (main.c:8)
==12345== Address 0x5204044 is 0 bytes after a block of size 4 alloc'd
==12345==    at 0x4C2FB0F: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400510: main (main.c:6)
```

Leak Types

Valgrind categorizes memory leaks by how reachable the lost memory is from pointers still in scope. Understanding these categories helps prioritize which leaks to fix first.

definitely lost	- no pointer to block exists, memory is leaked
indirectly lost	- pointer exists only in a "definitely lost" block
possibly lost	- pointer to interior of block (may be intentional)
still reachable	- pointer exists at exit, but memory not freed

“Definitely lost” blocks are the most serious—they represent memory that your program allocated but can no longer free because all pointers to it are gone. “Still reachable” blocks are often acceptable, especially for global data structures that live for the program’s lifetime.

Suppressing False Positives

Some libraries have known memory behaviors that Valgrind reports as errors but are actually intentional. Create suppression files to silence these warnings:

```
$ valgrind --gen-suppressions=all ./myprogram 2>&1 | grep -A 10 "{"
$ valgrind --suppressions=myapp.suppress ./myprogram
```

Cachegrind: Cache Profiling

Cachegrind simulates the CPU cache hierarchy to identify code with poor cache utilization. Cache misses are a major source of performance problems because accessing main memory is 100x slower than accessing L1 cache. Cachegrind counts instruction fetches, data reads/writes, and cache misses at each level.

```
$ valgrind --tool=cachegrind ./myprogram
$ cg_annotate cachegrind.out.<pid>
$ cg_annotate --auto=yes cachegrind.out.<pid> # annotate source
```

Output shows cache misses:

```
I  refs:      1,000,000
I1 misses:    1,000
I1 miss rate: 0.1%
D  refs:      500,000
D1 misses:    10,000
D1 miss rate: 2.0%
LL misses:    5,000
LL miss rate: 0.3%
```

Callgrind: Call Graph Profiling

Callgrind records the call history of a program, counting instructions executed in each function and tracking caller-callee relationships. Unlike sampling profilers, Callgrind counts every instruction, providing exact measurements rather than statistical estimates. The output can be visualized with KCachegrind, which displays interactive call graphs and source annotations.

```
$ valgrind --tool=callgrind ./myprogram
$ callgrind_annotate callgrind.out.<pid>
$ kcachegrind callgrind.out.<pid> # GUI visualization
```

Options:

```
--callgrind-out-file=callgrind.out # custom output file
--dump-instr=yes                  # instruction-level profiling
--collect-jumps=yes               # branch prediction info
```

Helgrind: Thread Errors

Helgrind detects synchronization errors in multithreaded programs, including data races (concurrent access to shared data without proper locking), deadlocks (circular lock dependencies), and misuse of the POSIX threads API. Data races are particularly insidious because they may only manifest under specific timing conditions, making them difficult to reproduce and debug.

```
$ valgrind --tool=helgrind ./myprogram
```


The histogram shows memory usage (Y-axis) over time (X-axis). Each column represents a snapshot, with different characters indicating allocation sources:

```
#    Peak memory usage snapshot
@    Detailed snapshot (includes allocation tree)
:    Normal snapshot
```

Detailed snapshot output shows allocation tree:

```
99.48% (6,400,000B) (heap allocation functions) malloc/new/new[]
->49.74% (3,200,000B) 0x400620: load_data (main.c:25)
| ->49.74% (3,200,000B) 0x400700: main (main.c:50)
|
->49.74% (3,200,000B) 0x400650: process_items (main.c:35)
  ->49.74% (3,200,000B) 0x400720: main (main.c:55)
```

For GUI visualization, use `massif-visualizer`:

```
$ massif-visualizer massif.out.<pid>
```

Common Workflows

Find memory leaks:

```
$ valgrind --leak-check=full --show-leak-kinds=all ./myprogram
```

Track uninitialized values:

```
$ valgrind --track-origins=yes ./myprogram
```

Profile cache performance:

```
$ valgrind --tool=cachegrind ./myprogram
$ cg_annotate --auto=yes cachegrind.out.*
```

Profile heap memory usage:

```
$ valgrind --tool=massif ./myprogram
$ ms_print massif.out.*
```

Generate call graph:

```
$ valgrind --tool=callgrind ./myprogram
$ kcachegrind callgrind.out.* # requires KCachegrind GUI
```

1.7.3 Sanitizers

Table of Contents

- *Sanitizers*
 - *Introduction*
 - *AddressSanitizer (ASan)*

- *LeakSanitizer (LSan)*
- *ThreadSanitizer (TSan)*
- *UndefinedBehaviorSanitizer (UBSan)*
- *MemorySanitizer (MSan)*
- *Combining Sanitizers*
- *CMake Integration*

Introduction

Sanitizers are compile-time instrumentation tools built into GCC and Clang that detect bugs at runtime with significantly lower overhead than Valgrind—typically 2-5x slowdown compared to Valgrind’s 10-50x. They work by inserting runtime checks around memory accesses, thread operations, or arithmetic operations during compilation, catching errors as they occur.

Each sanitizer targets specific bug classes: AddressSanitizer catches memory errors like buffer overflows and use-after-free, ThreadSanitizer detects data races in multithreaded code, UndefinedBehaviorSanitizer catches undefined behavior like integer overflow and null pointer dereference, and MemorySanitizer finds reads of uninitialized memory.

Because sanitizers are integrated into the compiler, they understand your code’s structure and can provide detailed error reports with source locations. They’re designed to be used during development and testing, catching bugs before they reach production. Many organizations run sanitizer-enabled builds in their continuous integration pipelines.

AddressSanitizer (ASan)

AddressSanitizer is the most widely used sanitizer, detecting memory errors that cause security vulnerabilities and crashes. It catches heap buffer overflows, stack buffer overflows, use-after-free, use-after-return, use-after-scope, double-free, and memory leaks. ASan uses shadow memory to track the state of every byte of application memory, enabling fast checks on every memory access.

```
$ gcc -fsanitize=address -g -o myprogram main.c
$ ./myprogram

$ clang -fsanitize=address -g -o myprogram main.c
$ ./myprogram
```

Example output for heap buffer overflow:

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000014
READ of size 4 at 0x602000000014 thread T0
    #0 0x4005a7 in main main.c:8
    #1 0x7f1234 in __libc_start_main

0x602000000014 is located 0 bytes to the right of 20-byte region
allocated by thread T0 here:
    #0 0x7f5678 in malloc
    #1 0x400567 in main main.c:5
```

Environment variables:

```
ASAN_OPTIONS=detect_leaks=1      # enable leak detection (default on Linux)
ASAN_OPTIONS=abort_on_error=1    # abort instead of exit
```

(continues on next page)

(continued from previous page)

```
ASAN_OPTIONS=symbolize=1          # symbolize stack traces
ASAN_OPTIONS=halt_on_error=0      # continue after first error
```

LeakSanitizer (LSan)

LeakSanitizer detects memory leaks by scanning memory at program exit to find allocated blocks that are no longer reachable. It's included with ASan by default on Linux, providing leak detection with minimal additional overhead. LSan can also be used standalone when you only need leak detection without the full memory error checking of ASan.

```
$ gcc -fsanitize=leak -g -o myprogram main.c
$ ./myprogram

# Or with ASan (leak detection enabled by default)
$ gcc -fsanitize=address -g -o myprogram main.c
$ ASAN_OPTIONS=detect_leaks=1 ./myprogram
```

Example output:

```
==12345==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 100 bytes in 1 object(s) allocated from:
    #0 0x7f1234 in malloc
    #1 0x400537 in main main.c:10

SUMMARY: LeakSanitizer: 100 byte(s) leaked in 1 allocation(s).
```

ThreadSanitizer (TSan)

ThreadSanitizer detects data races, which occur when two threads access the same memory location concurrently, at least one access is a write, and there's no synchronization between them. Data races cause undefined behavior and lead to bugs that are extremely difficult to reproduce because they depend on thread scheduling. TSan also detects deadlocks and improper use of synchronization primitives.

```
$ gcc -fsanitize=thread -g -o myprogram main.c -pthread
$ ./myprogram

$ clang -fsanitize=thread -g -o myprogram main.c -pthread
$ ./myprogram
```

Example output for data race:

```
WARNING: ThreadSanitizer: data race (pid=12345)
  Write of size 4 at 0x7f1234 by thread T1:
    #0 increment main.c:10

  Previous read of size 4 at 0x7f1234 by thread T2:
    #0 increment main.c:10

  Location is global 'counter' of size 4 at 0x7f1234
```

Environment variables:


```
TSAN_OPTIONS=halt_on_error=1      # stop on first error
TSAN_OPTIONS=second_deadlock_stack=1 # show both stacks for deadlock
```

UndefinedBehaviorSanitizer (UBSan)

UndefinedBehaviorSanitizer detects operations that the C/C++ standards declare as undefined behavior. When undefined behavior occurs, the compiler is free to do anything—including appearing to work correctly until a different compiler version or optimization level causes failures. UBSan catches these issues at runtime, including signed integer overflow, null pointer dereference, misaligned pointer access, out-of-bounds array indexing, and invalid shift operations.

```
$ gcc -fsanitize=undefined -g -o myprogram main.c
$ ./myprogram

$ clang -fsanitize=undefined -g -o myprogram main.c
$ ./myprogram
```

Specific checks:

```
-fsanitize=signed-integer-overflow  # signed overflow
-fsanitize=unsigned-integer-overflow # unsigned overflow (not UB but often buggy)
-fsanitize=null                    # null pointer dereference
-fsanitize=alignment                # misaligned pointer
-fsanitize=bounds                   # array bounds
-fsanitize=shift                    # invalid shift
-fsanitize=vla-bound                # negative VLA size
```

Example output:

```
main.c:10:15: runtime error: signed integer overflow:
2147483647 + 1 cannot be represented in type 'int'
```

UBSan can be combined with ASan:

```
$ gcc -fsanitize=address,undefined -g -o myprogram main.c
```

MemorySanitizer (MSan)

MemorySanitizer detects reads of uninitialized memory, which cause unpredictable behavior because the value depends on whatever happened to be in memory. These bugs are difficult to find because the program may appear to work correctly when uninitialized memory happens to contain benign values. MSan tracks the initialization state of every bit of memory and reports when uninitialized values affect program behavior. Only available in Clang.

```
$ clang -fsanitize=memory -g -o myprogram main.c
$ ./myprogram

# Track origins of uninitialized values (slower but more helpful)
$ clang -fsanitize=memory -fsanitize=memory-track-origins -g -o myprogram main.c
```

Example output:

```
==12345==WARNING: MemorySanitizer: use-of-uninitialized-value
#0 0x400567 in main main.c:8
```

(continues on next page)

(continued from previous page)

Uninitialized value was created by a heap allocation

```
#0 0x7f1234 in malloc
#1 0x400537 in main main.c:5
```

Combining Sanitizers

Some sanitizers can be combined to catch multiple bug classes in a single build, while others are mutually exclusive due to conflicting instrumentation:

```
# ASan + UBSan (common combination)
$ gcc -fsanitize=address,undefined -g -o myprogram main.c

# ASan + LSan (LSan included by default with ASan on Linux)
$ gcc -fsanitize=address -g -o myprogram main.c
```

Cannot combine:

- ASan with TSan (both instrument memory accesses differently)
- ASan with MSan (conflicting shadow memory schemes)
- TSan with MSan

CMake Integration

Add sanitizers to CMake builds for easy toggling during development:

```
# Option to enable sanitizers
option(ENABLE_ASAN "Enable AddressSanitizer" OFF)
option(ENABLE_TSAN "Enable ThreadSanitizer" OFF)
option(ENABLE_UBSAN "Enable UndefinedBehaviorSanitizer" OFF)

if(ENABLE_ASAN)
  add_compile_options(-fsanitize=address -fno-omit-frame-pointer)
  add_link_options(-fsanitize=address)
endif()

if(ENABLE_TSAN)
  add_compile_options(-fsanitize=thread)
  add_link_options(-fsanitize=thread)
endif()

if(ENABLE_UBSAN)
  add_compile_options(-fsanitize=undefined)
  add_link_options(-fsanitize=undefined)
endif()
```

Build with:

```
$ cmake -DENABLE_ASAN=ON ..
$ make
```

1.7.4 Tracing

Table of Contents

- *Tracing*
 - *Introduction*
 - *strace: System Call Tracing*
 - * *Common Workflows*
 - *ltrace: Library Call Tracing*
 - *ftrace: Kernel Function Tracing*
 - *perf trace*
 - *Comparison*

Introduction

Tracing tools intercept and log system calls, library calls, or kernel functions as a program executes. Unlike debuggers that pause execution at breakpoints, tracers observe programs with minimal interference, making them ideal for understanding program behavior, diagnosing I/O issues, debugging permission problems, and reverse engineering unfamiliar code.

strace traces system calls—the interface between user programs and the Linux kernel—showing every file operation, network connection, and process management call. ltrace traces calls to shared library functions like libc, revealing how programs use standard library APIs. ftrace traces kernel functions, useful for driver development and understanding kernel behavior.

These tools work on any executable without recompilation or debug symbols, making them invaluable for debugging production systems and third-party software.

strace: System Call Tracing

strace intercepts and records every system call made by a program, showing the call name, arguments, and return value. This reveals how programs interact with the operating system: which files they open, what network connections they make, how they spawn child processes, and where they spend time waiting.

Basic usage:

```
$ strace ./myprogram           # trace program
$ strace -p 1234               # attach to running process
$ strace -f ./myprogram        # follow child processes
$ strace -o trace.log ./myprogram # output to file
```

Filter by system call:

```
$ strace -e open ./myprogram    # only open() calls
$ strace -e trace=open,read,write ./prog # multiple syscalls
$ strace -e trace=file ./myprogram # file-related syscalls
$ strace -e trace=network ./myprogram # network syscalls
$ strace -e trace=process ./myprogram # process syscalls
$ strace -e trace=memory ./myprogram # memory syscalls
```

Useful options:

```
-c          # count time, calls, and errors per syscall
-T          # show time spent in each syscall
-t          # timestamp each line
-tt         # timestamp with microseconds
-r          # relative timestamp (time since last syscall)
-s 1000     # print 1000 chars of strings (default 32)
-y          # print paths for file descriptors
-yy         # print socket details
```

Example output:

```
open("config.txt", O_RDONLY)      = 3
read(3, "key=value\n", 4096)     = 10
close(3)                          = 0
write(1, "Hello, World!\n", 14)  = 14
```

Count syscalls:

```
$ strace -c ./myprogram
% time      seconds  usecs/call   calls   errors syscall
-----
 45.00      0.000450      45        10        read
 30.00      0.000300      30        10        write
 25.00      0.000250      25        10        5 open
-----
100.00      0.001000                30        5 total
```

Common Workflows

Debug file not found errors:

```
$ strace -e open,openat ./myprogram 2>&1 | grep -i "no such file"
```

See what files a program opens:

```
$ strace -e trace=file -y ./myprogram
```

Debug network issues:

```
$ strace -e trace=network -s 1000 ./myprogram
```

Find why a program hangs:

```
$ strace -p $(pgrep myprogram)
# Shows which syscall is blocking
```

ltrace: Library Call Tracing

`ltrace` intercepts calls to shared library functions, showing how programs use `libc`, `libm`, and other libraries. This is useful for understanding program behavior at a higher level than system calls—seeing string operations, memory allocations, and mathematical functions rather than raw kernel interfaces.

```
$ ltrace ./myprogram          # trace library calls
$ ltrace -p 1234              # attach to process
$ ltrace -e malloc+free ./prog # specific functions
$ ltrace -c ./myprogram       # count calls
```

Example output:

```
malloc(100)           = 0x55555556
strcpy(0x55555556, "hello") = 0x55555556
strlen("hello")       = 5
printf("Length: %d\n", 5) = 10
free(0x55555556)      = <void>
```

Options:

```
-e expr          # filter functions (e.g., -e malloc+free)
-l libname       # trace only calls to specific library
-s 1000         # string length limit
-n 2            # indent nested calls
-c              # count time and calls
-S             # trace system calls too (like strace)
```

fttrace: Kernel Function Tracing

fttrace is a kernel tracing framework built into Linux, accessed through the debugfs filesystem. It traces kernel functions with very low overhead, making it suitable for production systems. fttrace is essential for kernel and driver development, helping understand how the kernel processes system calls and handles hardware interrupts.

Setup:

```
# Mount debugfs if not mounted
$ sudo mount -t debugfs none /sys/kernel/debug

# fttrace files are in /sys/kernel/debug/tracing/
$ cd /sys/kernel/debug/tracing
```

Basic function tracing:

```
# List available tracers
$ cat available_tracers
function function_graph nop

# Enable function tracer
$ echo function > current_tracer

# Filter specific functions
$ echo 'tcp_*' > set_fttrace_filter

# Start tracing
$ echo 1 > tracing_on

# Run your program
$ ./myprogram
```

(continues on next page)

(continued from previous page)

```
# Stop and view trace
$ echo 0 > tracing_on
$ cat trace
```

Function graph tracer shows the call hierarchy with timing:

```
$ echo function_graph > current_tracer
$ echo 1 > tracing_on
$ ./myprogram
$ echo 0 > tracing_on
$ cat trace
```

Example output:

```
0)          | sys_read() {
0)          |   vfs_read() {
0)  0.500 us |     rw_verify_area();
0)          |     __vfs_read() {
0)  1.200 us |         ext4_file_read_iter();
0)  1.500 us |     }
0)  2.500 us | }
0)  3.000 us | }
```

trace-cmd (ftrace frontend):

```
$ sudo trace-cmd record -p function -l 'tcp_*' ./myprogram
$ trace-cmd report
```

perf trace

perf also provides system call tracing with lower overhead than strace, making it more suitable for tracing performance-sensitive applications or production systems where strace's overhead would be problematic:

```
$ perf trace ./myprogram          # trace syscalls
$ perf trace -p 1234             # attach to process
$ perf trace -e open ./myprogram # specific syscall
$ perf trace -s ./myprogram      # summary statistics
```

Comparison

Tool	Traces	Overhead	Use Case
strace	System calls	High	Debug I/O, files, network
ltrace	Library calls	Medium	Debug libc usage
ftrace	Kernel functions	Low	Kernel/driver debugging
perf trace	System calls	Low	Production tracing

1.7.5 Perf

Table of Contents

- *Perf*
 - *Introduction*
 - *Basic Profiling*
 - *Perf Stat*
 - *Perf Top*
 - *Useful Events*
 - *Call Graphs*
 - *Flame Graphs*
 - *Annotate Source*
 - *System-Wide Analysis*
 - *Tracing*
 - *Comparing Runs*
 - *Common Workflows*

Introduction

Linux perf is a powerful performance analysis tool that uses hardware performance counters and kernel tracepoints to profile applications and the system. Unlike sampling profilers that add overhead, perf leverages CPU hardware to count events like cycles, instructions, cache misses, and branch mispredictions with minimal impact on program execution.

Perf helps identify CPU hotspots, memory access patterns, and system bottlenecks. It's essential for optimizing performance-critical code, understanding where time is spent, and validating optimization efforts. The tool works on any executable without recompilation, though debug symbols (-g) improve output readability.

Basic Profiling

Record and analyze CPU samples:

```
$ perf record ./myprogram      # record CPU samples
$ perf report                 # interactive report
$ perf report --stdio         # text report

$ perf record -g ./myprogram   # record with call graphs
$ perf report -g               # show call graph in report
```

Common record options:

```
$ perf record -F 99 ./prog     # sample at 99 Hz
$ perf record -p 1234          # attach to running process
$ perf record -a sleep 10      # system-wide for 10 seconds
$ perf record -o out.data ./prog # custom output file
```

Perf Stat

Get summary statistics without recording samples:

```
$ perf stat ./myprogram
Performance counter stats for './myprogram':
    1,234.56 msec task-clock
        123 context-switches
    1,000,000 cycles
    800,000 instructions # 0.80 insn per cycle
    50,000 cache-misses
    10,000 branch-misses

$ perf stat -e cycles,instructions ./prog # specific events
$ perf stat -r 5 ./prog                 # run 5 times, show stats
$ perf stat -d ./prog                   # detailed stats
```

Perf Top

Real-time view of system or process hotspots:

```
$ perf top # system-wide live view
$ perf top -p 1234 # specific process
$ perf top -F 99 # sample at 99 Hz
$ perf top -g # show call graphs
$ perf top -ns comm,dso # sort by process and library
$ perf top -e cache-misses # profile cache misses
```

Useful Events

Profile specific hardware and software events:

```
# CPU events
$ perf stat -e cycles,instructions,branches,branch-misses ./prog

# Cache events
$ perf stat -e cache-references,cache-misses ./prog
$ perf stat -e L1-dcache-loads,L1-dcache-load-misses ./prog
$ perf stat -e LLC-loads,LLC-load-misses ./prog

# Memory events
$ perf stat -e page-faults,minor-faults,major-faults ./prog

# System calls
$ perf stat -e 'syscalls:sys_enter_*' ./prog
```

List available events:

```
$ perf list # all events
$ perf list hw # hardware events
$ perf list sw # software events
$ perf list cache # cache events
$ perf list tracepoint # kernel tracepoints
```


Call Graphs

Understand where time is spent in the call hierarchy:

```
# Record with frame pointers (compile with -fno-omit-frame-pointer)
$ perf record -g ./myprogram
$ perf report -g

# Record with DWARF unwinding (works without frame pointers)
$ perf record --call-graph dwarf ./myprogram

# Record with LBR (Last Branch Record, Intel CPUs)
$ perf record --call-graph lbr ./myprogram
```

Flame Graphs

Flame graphs visualize profiling data as interactive SVGs. The x-axis shows stack depth, and width represents time spent. Download FlameGraph tools from <https://github.com/brendangregg/FlameGraph>.

```
$ perf record -g ./myprogram
$ perf script | stackcollapse-perf.pl | flamegraph.pl > flame.svg

# CPU flame graph
$ perf record -F 99 -g ./prog
$ perf script | stackcollapse-perf.pl | flamegraph.pl > cpu.svg

# Off-CPU flame graph (shows where program waits)
$ perf record -e sched:sched_switch -g ./prog
```

Annotate Source

See which source lines consume the most cycles:

```
$ perf record ./myprogram
$ perf annotate                # interactive annotation
$ perf annotate func_name     # annotate specific function
$ perf annotate --stdio       # text output
```

Requires debug symbols (-g) and ideally compiled with -fno-omit-frame-pointer.

System-Wide Analysis

Profile the entire system to find bottlenecks:

```
$ perf top -a                # live system-wide view
$ perf record -a -g sleep 30 # record system for 30 seconds
$ perf report

# Find which processes use most CPU
$ perf top -ns comm

# Trace context switches
$ perf record -e context-switches -a sleep 10
```

Tracing

Trace specific events and system calls:

```
# Trace system calls
$ perf trace ./myprogram
$ perf trace -p 1234          # trace running process

# Trace specific syscalls
$ perf trace -e open,read,write ./prog

# Count syscalls
$ perf stat -e 'syscalls:sys_enter_*' ./prog
```

Comparing Runs

Compare performance between two runs:

```
$ perf record -o before.data ./prog_v1
$ perf record -o after.data ./prog_v2
$ perf diff before.data after.data
```

Common Workflows

Find CPU hotspots:

```
$ perf record -g ./myprogram
$ perf report
# Look for functions with highest "Overhead" percentage
```

Diagnose cache performance:

```
$ perf stat -e cache-references,cache-misses,L1-dcache-load-misses ./prog
# High cache-miss ratio indicates poor memory access patterns
```

Profile a running server:

```
$ perf record -p $(pgrep myserver) -g sleep 30
$ perf report
```

Check if CPU-bound or I/O-bound:

```
$ perf stat ./myprogram
# Low instructions-per-cycle + high context-switches = I/O bound
# High instructions-per-cycle = CPU bound
```

1.7.6 Nsight Systems

Table of Contents

- *Nsight Systems*
 - *Introduction*

- *Basic Profiling*
- *Viewing Results*
- *NVTX Annotations*
- *Common Workflows*
- *Remote Profiling*
- *What to Look For*

Introduction

NVIDIA Nsight Systems provides system-wide performance analysis for GPU applications, showing CPU activity, GPU kernels, memory transfers, and API calls on a unified timeline. Understanding GPU performance requires visibility into both the CPU-side code that launches kernels and the GPU-side execution that processes data—Nsight Systems captures both simultaneously.

The timeline view reveals how CPU and GPU activities overlap and interact, exposing idle time, serialization bottlenecks, and opportunities for better concurrency. You can see exactly when kernels launch, how long memory transfers take, and where the CPU waits for GPU completion. This holistic view is essential for optimizing the overall application, not just individual kernels.

Nsight Systems works with any CUDA application without requiring source code modifications, though adding NVTX annotations improves the clarity of timeline visualizations. It supports both command-line profiling for automated workflows and GUI visualization for interactive analysis.

Basic Profiling

Command-line profiling captures a trace that can be analyzed later:

```
$ nsys profile ./myprogram
$ nsys profile -o report ./myprogram      # custom output name
$ nsys profile --stats=true ./myprogram  # print summary stats

# Generate multiple output formats
$ nsys profile -o report --export=sqlite,text ./myprogram
```

Common options:

```
-o, --output=NAME           # output file name (without extension)
-t, --trace=TRACE           # what to trace: cuda,nvtx,osrt,cublas,cudnn
--stats=true                # print summary statistics
--force-overwrite=true      # overwrite existing report
-w, --show-output=true      # show application output
--sample=cpu                # CPU sampling
--cudabacktrace=true        # CUDA API backtraces
```

Trace specific APIs:

```
$ nsys profile -t cuda,nvtx ./myprogram  # CUDA + NVTX markers
$ nsys profile -t cuda,osrt ./myprogram  # CUDA + OS runtime
$ nsys profile -t cuda,cublas ./myprogram # CUDA + cuBLAS
```

Viewing Results

Open the report in the GUI for interactive timeline exploration:

```
$ nsys-ui report.nsys-rep    # open in GUI
$ nsys stats report.nsys-rep # command-line statistics
```

Export to different formats for custom analysis:

```
$ nsys export -t sqlite report.nsys-rep    # SQLite database
$ nsys export -t text report.nsys-rep      # text summary
```

The GUI timeline shows:

- CPU thread activity and call stacks
- CUDA API calls (cudaMalloc, cudaMemcpy, kernel launches)
- GPU kernel executions with duration
- Memory transfers between host and device
- NVTX ranges and markers

NVTX Annotations

NVTX (NVIDIA Tools Extension) lets you add custom markers and ranges to your code, making the timeline easier to understand. Ranges show up as colored bars in the timeline, helping you correlate application phases with GPU activity.

```
#include <nvtx3/nvToolsExt.h>

void myFunction() {
    nvtxRangePush("myFunction");
    // ... work ...
    nvtxRangePop();
}

// Or with colors
nvtxEventAttributes_t attr = {0};
attr.version = NVTX_VERSION;
attr.colorType = NVTX_COLOR_ARGB;
attr.color = 0xFF00FF00; // green
attr.messageType = NVTX_MESSAGE_TYPE_ASCII;
attr.message.ascii = "Important Section";
nvtxRangePushEx(&attr);
```

Compile with:

```
$ nvcc -o myprogram main.cu -lnvToolsExt
```

Common Workflows

Profile and get summary statistics:

```
$ nsys profile --stats=true -o system ./myprogram
# Shows kernel times, memory transfer times, API call counts
```

Identify CPU-GPU synchronization issues:

```
$ nsys profile -t cuda ./myprogram
$ nsys-ui system.nsys-rep
# Look for gaps between kernel launches (CPU idle or sync points)
```

Profile a running application:

```
$ nsys profile --attach-pid=1234 -o attached
# Or launch with delayed start
$ nsys profile --delay=5 ./myprogram # start profiling after 5 seconds
```

Profile specific duration:

```
$ nsys profile --duration=10 ./myprogram # profile for 10 seconds
```

Remote Profiling

Profile applications running on remote machines or clusters where you can't run the GUI directly:

```
# On remote machine
$ nsys profile -o /tmp/report ./myprogram

# Copy report to local machine
$ scp remote:/tmp/report.nsys-rep .
$ nsys-ui report.nsys-rep
```

Nsight Systems GUI also supports connecting to remote machines via SSH for interactive profiling sessions.

What to Look For

Common performance issues visible in the timeline:

Issue	Timeline Pattern
CPU-GPU serialization	Gaps between kernel launches
Excessive synchronization	Many cudaDeviceSynchronize calls
Memory transfer overhead	Large cudaMemcpy blocks
Kernel launch overhead	Many small kernels with gaps
Underutilized GPU	Long CPU sections, short GPU bursts

1.7.7 Binary Tools**Table of Contents**

- *Binary Tools*
 - *Introduction*
 - *Cheatsheet*
 - *nm: Symbol Table*
 - *readelf: ELF Structure*

- *objdump: Disassembly*
- *file: File Type*
- *ldd: Shared Library Dependencies*
- *strings: Embedded Strings*
- *size: Section Sizes*
- *strip: Remove Symbols*
- *c++filt: Demangle C++ Symbols*
- *Practical Examples*

Introduction

Binary inspection tools examine compiled executables, object files, and shared libraries without running them. They reveal symbol tables, section layouts, dependencies, and embedded strings—essential for debugging linker errors, understanding library dependencies, and reverse engineering binaries.

On Linux, binaries use the ELF (Executable and Linkable Format) and tools like `readelf`, `objdump`, and `ldd` are the standard way to inspect them. On macOS, binaries use the Mach-O format and the equivalent tool is `otool`. Some tools like `nm`, `strings`, `size`, `file`, `strip`, and `c++filt` work on both platforms. When debugging “undefined reference” linker errors, checking shared library dependencies, understanding why a binary is large, or verifying that symbols are exported correctly, these tools are indispensable.

Source

`src/debug/binary-inspect`

All examples below use the `binary-inspect_test` executable built from `src/debug/binary-inspect/`. The companion script `inspect.sh` exercises every tool covered in this note against the compiled binary—run it standalone or through `ctest` to verify the commands work on your system.

```
# Build and run
$ cmake -B build -S . && cmake --build build --target binary-inspect_test
$ ./src/debug/binary-inspect/inspect.sh ./build/src/debug/binary-inspect/binary-inspect_
  ↪ test
```

Cheatsheet

```
# --- file: identify binary type & arch ---
file ./binary
file /usr/lib/x86_64-linux-gnu/libc.so.6

# --- nm: list symbols ---
nm ./binary           # all symbols (mangled)
nm -C ./binary        # demangle C++ names
nm -u ./binary        # undefined symbols only
nm -g ./binary        # global (external) symbols only
nm -n ./binary        # sort by address
nm -S ./binary        # show symbol sizes
nm -DC /usr/lib/libm.so.6 # dynamic symbols in shared lib

# --- readelf: inspect ELF structure ---
```

(continues on next page)

(continued from previous page)

```

readelf -h ./binary          # ELF header (arch, entry point)
readelf -S ./binary          # section headers (.text .data .bss ...)
readelf -s ./binary          # symbol table
readelf -l ./binary          # program headers (LOAD segments)
readelf -d ./binary          # dynamic section (NEEDED libs)
readelf -r ./binary          # relocations
readelf -n ./binary          # notes (build-id, ABI)
readelf -a ./binary          # all of the above
readelf --debug-dump=info ./binary # DWARF debug info
readelf -p .rodata ./binary  # dump .rodata as strings

# --- objdump: disassemble ---
objdump -d -C ./binary       # disassemble with demangling
objdump -d -S -C ./binary    # interleave source (needs -g)
objdump -s -j .rodata ./binary # hex dump of .rodata section
objdump -t -C ./binary       # symbol table
objdump -R ./binary          # dynamic relocations

# --- ldd: shared library dependencies ---
ldd ./binary                 # list required .so files
# WARNING: ldd runs the dynamic linker - don't use on untrusted binaries
# Use readelf -d instead for untrusted files

# --- strings: extract printable strings ---
strings ./binary              # default min length 4
strings -n 10 ./binary        # min length 10
strings -t x ./binary         # show hex offset of each string
strings ./binary | grep -i version

# --- size: section sizes ---
size ./binary                 # text, data, bss totals
size -A ./binary              # all sections with sizes

# --- strip: remove symbols ---
strip ./binary                # strip in place
strip -o stripped ./binary    # strip to new file
strip --strip-debug ./binary  # remove debug info only, keep symbols

# --- c++filt: demangle ---
echo "_ZN4demo6Widget5valueEv" | c++filt
nm ./binary | c++filt | grep demo

# --- ar: static library inspection ---
ar -t libfoo.a                # list object files in archive
ar -x libfoo.a                # extract all object files
nm -C libfoo.a                # list symbols across all members

# --- otool: macOS equivalent of readelf/objdump/ldd ---
otool -h ./binary             # Mach-O header (magic, cputype, filetype)
otool -l ./binary             # load commands (segments, sections)
otool -L ./binary             # shared library dependencies (like ldd)
otool -tV ./binary            # disassemble __TEXT,__text

```

(continues on next page)

(continued from previous page)

```
otool -s __TEXT __cstring ./binary # dump a specific section
otool -p _add -tV ./binary # disassemble starting at symbol _add

# --- install_name_tool: macOS rpath/dylib management ---
install_name_tool -change old.dylib new.dylib ./binary
install_name_tool -add_rpath /usr/local/lib ./binary
```

nm: Symbol Table

nm lists symbols in object files and executables. Each line shows an address, a symbol type letter, and the symbol name. The type letter tells you where the symbol lives: T for code in `.text`, D for initialized data in `.data`, B for zero-initialized data in `.bss`, R for read-only data in `.rodata`, and U for undefined symbols that must be resolved by the linker. Uppercase means the symbol is global (externally visible), lowercase means it is local to the translation unit. This is the first tool to reach for when debugging linker errors like “undefined reference” or “multiple definition”.

Common symbol types:

```
T/t  Code (.text) - global/local
D/d  Initialized data (.data) - global/local
B/b  Uninitialized data (.bss) - global/local
R/r  Read-only data (.rodata) - global/local
U    Undefined (needs linking)
W/w  Weak symbol
```

List all symbols:

```
$ nm build/src/debug/binary-inspect/binary-inspect_test
```

C++ symbols are mangled. Use `c++filt` or `-C` to demangle:

```
$ nm -C build/src/debug/binary-inspect/binary-inspect_test
```

Filter by symbol type:

```
# Only undefined symbols (external dependencies)
$ nm -u build/src/debug/binary-inspect/binary-inspect_test

# Only defined symbols, sorted by address
$ nm -n build/src/debug/binary-inspect/binary-inspect_test

# Only global (external) symbols
$ nm -g build/src/debug/binary-inspect/binary-inspect_test
```

Inspect an object file to see what it exports:

```
$ nm -C build/src/debug/binary-inspect/CMakeFiles/binary-inspect_test.dir/helper.cc.o
# Expected output:
# 0000000000000000 D global_initialized
# 0000000000000004 C global_uninitialized
# 0000000000000000 T add(int, int)
```


readelf: ELF Structure

`readelf` displays detailed information about ELF (Executable and Linkable Format) files—the standard binary format on Linux. Unlike `objdump`, it does not rely on the BFD library and can parse corrupted or partially linked files. It is the most comprehensive tool for inspecting ELF metadata: the file header reveals architecture and entry point, section headers show how the binary is organized (`.text`, `.data`, `.bss`, `.rodata`, etc.), program headers describe how the kernel loads segments into memory, and the dynamic section lists shared library dependencies. It also reads DWARF debug information, GNU notes (build-id), and relocation entries. This is a Linux-only tool; on macOS, use `otool -l` for similar load command inspection.

ELF header (architecture, entry point, type):

```
$ readelf -h build/src/debug/binary-inspect/binary-inspect_test
# Shows: Class (32/64-bit), OS/ABI, Type (EXEC/DYN), Entry point
```

Section headers (all sections in the binary):

```
$ readelf -S build/src/debug/binary-inspect/binary-inspect_test
# Shows: .text, .data, .bss, .rodata, .symtab, .strtab, etc.
```

Symbol table:

```
$ readelf -s build/src/debug/binary-inspect/binary-inspect_test

# With demangling (readelf doesn't have -C, pipe through c++filt)
$ readelf -s build/src/debug/binary-inspect/binary-inspect_test | c++filt
```

Program headers (segments loaded at runtime):

```
$ readelf -l build/src/debug/binary-inspect/binary-inspect_test
# Shows LOAD, DYNAMIC, INTERP segments
```

Dynamic section (shared library dependencies):

```
$ readelf -d build/src/debug/binary-inspect/binary-inspect_test
# Shows NEEDED entries (libstdc++, libgtest, libc, etc.)
```

DWARF debug info (if compiled with `-g`):

```
$ readelf --debug-dump=info build/src/debug/binary-inspect/binary-inspect_test | head -50
```

objdump: Disassembly

`objdump` disassembles machine code back to assembly and displays section contents. It is the go-to tool when you need to see what the compiler actually generated—whether to verify an optimization kicked in, understand a crash at a specific instruction address, or compare codegen between compiler flags. With `-S` and debug info (`-g`), it interleaves source lines with assembly, making it easy to correlate C/C++ code with the generated instructions. It can also dump raw section contents as hex (`-s -j .rodata`), which is useful for inspecting string literals, vtables, or constant data embedded in the binary. On macOS, `otool -tV` provides similar disassembly for Mach-O files, and the LLVM `objdump` (shipped with Xcode) supports both formats.

Disassemble all code:

```
$ objdump -d -C build/src/debug/binary-inspect/binary-inspect_test
```

Disassemble a specific function:

```
$ objdump -d -C build/src/debug/binary-inspect/binary-inspect_test | \
sed -n '/<add(int, int)>:./,/^$/p'
```

Interleave source with assembly (requires `-g`):

```
$ objdump -d -S -C build/src/debug/binary-inspect/binary-inspect_test
```

Display section contents as hex:

```
# Show .rodata (read-only data like string literals)
$ objdump -s -j .rodata build/src/debug/binary-inspect/binary-inspect_test
```

file: File Type

`file` identifies file type, architecture, and linking mode by reading magic bytes and metadata from the file header. It instantly tells you whether a binary is 32-bit or 64-bit, statically or dynamically linked, stripped or not stripped, and what architecture it targets (x86-64, ARM64, etc.). This is often the first command to run when you receive an unfamiliar binary or need to confirm that a cross-compilation produced the correct target format. It works on any file type, not just executables—object files, shared libraries, core dumps, and even scripts are all recognized:

```
$ file build/src/debug/binary-inspect/binary-inspect_test
# ELF 64-bit LSB pie executable, x86-64, version 1 (GNU/Linux),
# dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
# with debug_info, not stripped
```

Key information: 32/64-bit, static/dynamic linking, stripped/not stripped.

ldd: Shared Library Dependencies

`ldd` lists shared libraries required at runtime and where the dynamic linker resolves them on the filesystem. When a program fails to start with “error while loading shared libraries” or “cannot open shared object file”, `ldd` shows exactly which `.so` file is missing and where the linker searched. It also reveals version mismatches—if a binary was linked against `libfoo.so.2` but only `libfoo.so.1` is installed, `ldd` will show “not found” for that entry. On macOS, use `otool -L` instead to list dylib dependencies:

```
$ ldd build/src/debug/binary-inspect/binary-inspect_test
# linux-vdso.so.1 => (0x00007ffd...)
# libgtest.so => /usr/lib/... (0x00007f...)
# libstdc++.so.6 => /usr/lib/... (0x00007f...)
# libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f...)
# /lib64/ld-linux-x86-64.so.2 (0x00007f...)
```

Warning

`ldd` executes the binary’s dynamic linker, so never use it on untrusted binaries. Use `readelf -d` or `objdump -p` instead for untrusted files.

strings: Embedded Strings

`strings` scans a binary file and extracts sequences of printable characters (by default, 4 or more consecutive printable bytes). This is surprisingly useful in practice: you can find hardcoded file paths, configuration values, error messages,

version strings, URLs, SQL queries, and even accidentally embedded credentials. It works on any file—not just executables—so you can also use it on core dumps, firmware images, or unknown binary blobs to get a quick sense of what they contain:

```
$ strings build/src/debug/binary-inspect/binary-inspect_test | grep "binary-inspect"
# binary-inspect v1.0

# Minimum string length of 10 characters
$ strings -n 10 build/src/debug/binary-inspect/binary-inspect_test

# Show file offset of each string
$ strings -t x build/src/debug/binary-inspect/binary-inspect_test
```

Useful for finding hardcoded paths, version strings, error messages, and configuration values embedded in binaries.

size: Section Sizes

`size` shows the size of each major section in an object file or executable, giving a quick overview of how binary space is distributed between code (`.text`), initialized data (`.data`), and zero-initialized data (`.bss`). This is the fastest way to answer “why is my binary so large?”—if `.text` is huge, you have a lot of code (possibly from heavy template instantiation or inlining); if `.data` is large, you have big initialized globals or static arrays; if `.bss` is large, you have large uninitialized buffers. The `-A` flag shows every section individually, which reveals additional contributors like debug info, exception handling tables, and string tables:

```
$ size build/src/debug/binary-inspect/binary-inspect_test
#   text    data    bss    dec    hex filename
#   ...      ...      ...      ...      ...  binary-inspect_test

# Berkeley format (default) vs System V format
$ size -A build/src/debug/binary-inspect/binary-inspect_test
# Shows every section with size and address
```

Compare object files to see code size impact:

```
$ size build/src/debug/binary-inspect/CMakeFiles/binary-inspect_test.dir/helper.cc.o
$ size build/src/debug/binary-inspect/CMakeFiles/binary-inspect_test.dir/main.cc.o
```

strip: Remove Symbols

`strip` removes symbol table entries and debug information from a binary, significantly reducing its size. Production deployments typically strip binaries to save disk space and memory, and to make reverse engineering harder. A debug build with `-g` can easily be 5–10x larger than its stripped counterpart. You can strip everything (`strip binary`), strip only debug info while keeping the symbol table (`strip --strip-debug`), or output to a separate file (`strip -o`). A common workflow is to keep an unstripped copy for debugging and deploy the stripped version—if a crash occurs, you can use the unstripped binary with GDB to get full symbol and line information from the core dump:

```
# Check size before
$ ls -la build/src/debug/binary-inspect/binary-inspect_test

# Strip (use a copy to preserve the original)
$ cp build/src/debug/binary-inspect/binary-inspect_test /tmp/stripped_test
$ strip /tmp/stripped_test

# Check size after
```

(continues on next page)

(continued from previous page)

```
$ ls -la /tmp/stripped_test
# Verify symbols are gone
$ nm /tmp/stripped_test
# nm: /tmp/stripped_test: no symbols
```

c++filt: Demangle C++ Symbols

c++filt translates mangled C++ symbol names back to human-readable form. C++ compilers encode function signatures into symbol names to support overloading, namespaces, and templates—so `_ZN4demo6Widget5valueEv` becomes `demo::Widget::value()`. Most tools like `nm` and `objdump` support `-C` for built-in demangling, but `c++filt` is useful as a standalone filter when piping output from tools that don't have a demangle flag (like `readelf`) or when you encounter a mangled name in a log file or crash report:

```
$ echo "_ZN4demo6Widget5valueEv" | c++filt
# demo::Widget::value()

$ nm build/src/debug/binary-inspect/binary-inspect_test | c++filt | grep demo
# ... T demo::Widget::Widget(int)
# ... T demo::Widget::value() const
```

Practical Examples

Debugging “undefined reference” linker errors:

```
# Find which object file defines a symbol
$ nm -C build/src/debug/binary-inspect/CMakeFiles/binary-inspect_test.dir/*.o | grep "T_
↪add"

# Find which library provides a symbol
$ nm -DC /usr/lib/x86_64-linux-gnu/libm.so.6 | grep " T sin"
```

Checking if a binary is stripped:

```
$ file build/src/debug/binary-inspect/binary-inspect_test | grep -c "not stripped"
```

Finding why a binary is large:

```
$ size -A build/src/debug/binary-inspect/binary-inspect_test | sort -k2 -n -r | head
```

1.8 Rust Programming

Rust is a systems programming language that guarantees memory safety without garbage collection. For C++ developers, Rust offers familiar concepts like RAII, zero-cost abstractions, and low-level control, while adding compile-time safety guarantees that prevent data races, null pointer dereferences, and use-after-free bugs.

This section maps C++ concepts to their Rust equivalents, highlighting similarities and key differences. Each chapter includes side-by-side code comparisons with working examples.

1.8.1 Rust Basics

Table of Contents

- *Rust Basics*
 - *Built-in Types*
 - * *No Implicit Conversions*
 - *Variables and Mutability*
 - * *Shadowing*
 - *Underscore _ Placeholder*
 - * *Partial Type Inference*
 - * *Ignoring Values in Patterns*
 - * *Suppressing Unused Warnings*
 - *Printing and Formatting*
 - *Control Flow*
 - * *if as an Expression*
 - * *loop with Break Values*
 - * *for and Ranges*
 - * *Expression Blocks*
 - *Enums and Pattern Matching*
 - * *Enums with Data*
 - * *match Expression*
 - * *if let and matches!*
 - * *Destructuring in match*
 - *Ownership*
 - * *Copy vs Move Types*
 - *References and Borrowing*
 - * *Borrowing Rules*
 - *Lifetimes*
 - * *Explicit Lifetime Annotations*
 - *Slices*
 - *The ? Operator*
 - * *Result<T, E> and io::Result<T>*
 - *Struct Mutability*
 - * *Nested Struct Mutability*
 - * *Interior Mutability*

- *Pointers and References*
 - * *Dereferencing and Auto-deref*
 - * *Lifetimes*
- *See Also*

This chapter covers fundamental Rust concepts that differ most from C++: built-in types, immutability by default, control flow as expressions, pattern matching, ownership, and borrowing. Understanding these concepts is essential for writing idiomatic Rust code.

Built-in Types

Source

[src/rust/types](#)

Rust's primitive types are explicit about size and signedness, unlike C/C++ where `int` size is platform-dependent. Rust has no implicit numeric conversions — all casts must be explicit with `as`.

C/C++	Rust	Size
<code>int8_t</code> / signed char	<code>i8</code>	1 byte
<code>uint8_t</code> / unsigned char	<code>u8</code>	1 byte
<code>int16_t</code> / short	<code>i16</code>	2 bytes
<code>uint16_t</code> / unsigned short	<code>u16</code>	2 bytes
<code>int32_t</code> / int	<code>i32</code>	4 bytes
<code>uint32_t</code> / unsigned	<code>u32</code>	4 bytes
<code>int64_t</code> / long long	<code>i64</code>	8 bytes
<code>uint64_t</code> / unsigned long long	<code>u64</code>	8 bytes
<code>float</code>	<code>f32</code>	4 bytes
<code>double</code>	<code>f64</code>	8 bytes
<code>bool</code>	<code>bool</code>	1 byte
<code>char</code> (1 byte)	<code>char</code> (4 bytes, Unicode scalar)	4 bytes
<code>size_t</code>	<code>usize</code>	pointer-sized
<code>ptrdiff_t</code> / <code>ssize_t</code>	<code>isize</code>	pointer-sized

Rust permits `_` as a visual separator in numeric literals (like C++14 digit separators `'`), and supports type suffixes to specify the type inline:

C++:

```
#include <cstdint>

int32_t a = -42;
uint64_t b = 1'000'000;    // C++14 digit separator
uint8_t c = 0xFF;
double pi = 3.14159;
bool flag = true;
```

Rust:

```
let a: i32 = -42;
let b: u64 = 1_000_000;    // _ as digit separator
let c = 0xff_u8;          // type suffix
```

(continues on next page)

(continued from previous page)

```
let pi: f64 = 3.14159;
let flag: bool = true;
let ch: char = ' ';           // 4-byte Unicode scalar
```

No Implicit Conversions

C++ silently converts between numeric types, which can cause subtle bugs. Rust requires explicit casts with `as` or safe conversions with `From/Into`.

C++ (implicit conversion — compiles, may lose data):

```
int x = 3.14;           // silently truncates to 3
unsigned u = -1;        // wraps to UINT_MAX
char c = 1000;          // implementation-defined
```

Rust (explicit conversion required):

```
let x = 3.14_f64 as i32;           // explicit truncation
// let u: u32 = -1_i32;           // error: mismatched types
let u: u32 = (-1_i32) as u32;      // explicit wrap

// Prefer From/Into for safe widening
let wide: u32 = 42_u8.into();      // safe, infallible

// TryFrom for fallible narrowing
let narrow: Result<u8, _> = 300_u16.try_into(); // Err
```

Variables and Mutability

Source

src/rust/variables

In C++, variables are mutable by default and you use `const` to make them immutable. Rust takes the opposite approach: variables are immutable by default, and you must explicitly use the `mut` keyword to allow mutation.

C++:

```
int x = 5;           // mutable by default
x = 10;              // OK
const int y = 5;     // immutable
// y = 10;           // error
```

Rust:

```
let x = 5;           // immutable by default
// x = 10;           // error: cannot assign twice to immutable variable
let mut y = 5;       // mutable
y = 10;              // OK
```

Shadowing

Rust allows re-declaring a variable with the same name, which shadows the previous binding. This is different from mutation and allows changing the type.

C++ does not support shadowing in the same scope. Re-declaring a variable with the same name is a compilation error. You must use a different variable name or explicitly cast/convert the value.

C++ (not allowed):

```
int x = 5;
int x = x + 1;           // error: redefinition of 'x'
// const char* x = "hello"; // error: redefinition with different type

// Workaround: use different names
int x1 = 5;
int x2 = x1 + 1;
const char* x3 = "hello";
```

Rust (shadowing allowed):

```
let x = 5;
let x = x + 1;           // shadows previous x, x is now 6
let x = "hello";         // shadows again, different type - OK in Rust
```

Shadowing is useful in Rust for transforming a value while keeping the same name, especially when parsing or converting types:

Underscore _ Placeholder

Source

`src/rust/underscore`

Rust uses `_` as a wildcard or placeholder in several contexts: type inference, pattern matching, and ignoring unused values. C++ achieves similar goals with `auto`, `std::ignore`, and `[[maybe_unused]]`, but Rust's `_` is more versatile — especially for partial type inference, which C++ does not support.

Partial Type Inference

Rust allows inferring individual type parameters using `_`, while C++ `auto` is all-or-nothing — you either write the full type or let the compiler infer everything.

C++ (no partial inference):

```
#include <vector>
#include <map>

// auto infers the entire type
auto v = std::vector{1, 2, 3};           // deduces vector<int>
auto m = std::map<std::string, int>{};    // must write full type or use auto

// Cannot say: "I know it's a vector, infer the element type"
// std::vector<auto> v = {1, 2, 3};        // not valid C++
```

Rust (partial inference with ```_```):

```
// Infer the element type, but specify the container
let v: Vec<_> = vec![1, 2, 3];           // compiler infers Vec<i32>

use std::collections::HashMap;
let m: HashMap<_, _> = vec![]
```

(continues on next page)

(continued from previous page)

```

    ("key".to_string(), 1),
].into_iter().collect();           // infers HashMap<String, i32>

// Especially useful with collect() where the compiler
// needs to know the target collection type
let squares: Vec<_> = (0..5).map(|x| x * x).collect();

```

Ignoring Values in Patterns

Rust's `_` can discard values in destructuring and match expressions. C++ uses `std::ignore` with `std::tie` or unnamed variables in structured bindings.

C++:

```

#include <tuple>

auto [x, _] = std::make_pair(1, 2);           // _ is just a variable name
// Note: _ is not special in C++, it's a regular identifier

int val;
std::tie(val, std::ignore) = std::make_pair(1, 2); // truly discards

```

Rust:

```

// Destructuring - _ truly discards the value
let (x, _) = (1, 2);

// Match expressions
let value = Some(42);
match value {
    Some(_) => println!("has a value"),    // don't care what's inside
    None => println!("empty"),
}

// Ignoring parts of a struct
struct Point { x: i32, y: i32, z: i32 }
let p = Point { x: 1, y: 2, z: 3 };
let Point { x, .. } = p;                  // ignore y and z

```

Suppressing Unused Warnings

C++:

```

[[maybe_unused]] int result = do_something(); // suppress warning (C++17)

```

Rust:

```

let _result = do_something(); // prefix with _ to suppress warning

```

 **Note**

Rust's `_` is **not** like any in TypeScript or Object in Java. It does not mean “any type.” The compiler still determines a single concrete type at compile time — `_` simply means “infer this type for me from context.”

Printing and Formatting

Rust uses macros (`println!`, `format!`, `eprintln!`) for formatted output instead of `printf` or `std::cout`. The `{}` placeholder uses the `Display` trait, while `{:?}` uses the `Debug` trait.

C++:

```
#include <iostream>
#include <vector>

int x = 42;
std::cout << "value: " << x << std::endl;

// No built-in debug print for containers
std::vector<int> v = {1, 2, 3};
// Must write custom loop or operator<<
```

Rust:

```
let x = 42;
println!("value: {}", x);           // Display trait
println!("value: {x}");             // inline variable (Rust 1.58+)

let v = vec![1, 2, 3];
println!("{:?}", v);                // Debug trait: [1, 2, 3]
println!("{:#?}", v);               // Pretty-print Debug

// eprintln! writes to stderr
eprintln!("error: {}", "something went wrong");

// format! returns a String instead of printing
let s = format!("x = {}, v = {:?}", x, v);
```

Note

Most standard library types implement `Debug`. For custom types, derive it with `#[derive(Debug)]`. The `Display` trait must be implemented manually for custom formatting.

Control Flow

Source

`src/rust/control_flow`

Rust's control flow constructs are expressions that return values, unlike C++ where `if`, `for`, and `while` are statements. This enables a more functional style and eliminates the need for the ternary operator.

if as an Expression

In C++, `if` is a statement. To assign based on a condition, you use the ternary operator `?:`. In Rust, `if` is an expression that returns a value directly.

C++:

```
int x = 42;
const char* msg = (x == 42) ? "found it" : "nope"; // ternary
// or
std::string msg2;
if (x == 42) {
    msg2 = "found it";
} else {
    msg2 = "nope";
}
```

Rust:

```
let x = 42;
let msg = if x == 42 { "found it" } else { "nope" };
println!("{}", msg);

// No ternary operator in Rust - if/else IS the ternary
```

loop with Break Values

Rust's loop creates an infinite loop. Unlike C++ `while(true)`, Rust's loop can return a value via `break`.

C++:

```
int counter = 0;
int result;
while (true) {
    counter++;
    if (counter == 10) {
        result = counter * 2;
        break;
    }
}
```

Rust:

```
let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 {
        break counter * 2; // loop returns 20
    }
};
```

for and Ranges

Rust uses ranges (`0..n` exclusive, `0..=n` inclusive) instead of C-style `for(int i=0; i<n; i++)`.

C++:

```
for (int i = 0; i < 5; i++) {
    std::cout << i << " ";
}
// range-based for (C++11)
std::vector<int> v = {1, 2, 3};
for (const auto& x : v) {
    std::cout << x << " ";
}
```

Rust:

```
for i in 0..5 {                // 0, 1, 2, 3, 4
    print!("{}", i);
}
for i in 0..=5 {               // 0, 1, 2, 3, 4, 5 (inclusive)
    print!("{}", i);
}
let v = vec![1, 2, 3];
for x in &v {
    print!("{}", x);
}
```

Expression Blocks

In Rust, a block `{}` is an expression. The last expression (without a semicolon) becomes the block's value. This replaces many uses of temporary variables.

C++:

```
// Must declare variable, then assign in separate steps
int val;
{
    int a = 10;
    int b = 32;
    val = a + b;
}
```

Rust:

```
let val = {
    let a = 10;
    let b = 32;
    a + b // no semicolon - this is the return value
};
// val == 42
```

Functions also use this: the last expression is the return value (no `return` keyword needed):

```
fn is_answer(x: u32) -> bool {
    x == 42 // no semicolon, no return keyword
}
```

Enums and Pattern Matching

Source

src/rust/pattern_matching, src/rust/enums

Rust enums are discriminated unions (tagged unions) — each variant can carry different data. Combined with `match`, they replace C++ class hierarchies, `std::variant`, and `switch` statements with compile-time exhaustiveness checking.

Enums with Data

C++ enums are just named integers. Rust enums can carry data in each variant, like `std::variant` but with exhaustive pattern matching and no `std::get` exceptions.

C++ (enum + variant):

```
#include <variant>
#include <string>

enum class ShapeType { Circle, Rectangle };

// std::variant for tagged union
struct Circle { double radius; };
struct Rectangle { double w, h; };
using Shape = std::variant<Circle, Rectangle>;

double area(const Shape& s) {
    return std::visit([](auto&& arg) -> double {
        using T = std::decay_t<decltype(arg)>;
        if constexpr (std::is_same_v<T, Circle>)
            return 3.14159 * arg.radius * arg.radius;
        else
            return arg.w * arg.h;
    }, s);
}
```

Rust (enum with data):

```
enum Shape {
    Circle(f64),
    Rectangle(f64, f64),
    Triangle { base: f64, height: f64 }, // named fields
}

fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle(r) => std::f64::consts::PI * r * r,
        Shape::Rectangle(w, h) => w * h,
        Shape::Triangle { base, height } => 0.5 * base * height,
```

(continues on next page)

(continued from previous page)

```
}
}
```

match Expression

match is Rust's equivalent of switch, but it must be exhaustive (cover all cases) and can destructure, bind values, and use guards.

C++ (switch):

```
int x = 42;
switch (x) {
    case 0: std::cout << "zero"; break;
    case 42: std::cout << "answer"; break;
    default: std::cout << "other"; break;
}
// Forgetting break causes fallthrough - a common bug
```

Rust (match):

```
let x = 42;
match x {
    0 => println!("zero"),
    42 => println!("answer"),
    _ => println!("other"),      // _ is the wildcard
}
// No fallthrough. Must be exhaustive.

// match with ranges and guards
match x {
    0..=41 => println!("too small"),
    42 => println!("perfect"),
    n if n > 100 => println!("{}", n), // guard
    _ => println!("other"),
}

// match returns a value
let msg = match x {
    42 => "the answer",
    _ => "not the answer",
};
```

if let and matches!

For matching a single pattern, if let is more concise than a full match. The matches! macro returns a bool.

C++:

```
#include <optional>

std::optional<int> val = 42;
if (val.has_value()) {
```

(continues on next page)

(continued from previous page)

```
std::cout << *val;
}
```

Rust:

```
let val = Some(42);

// if let - match one pattern
if let Some(x) = val {
    println!("got {}", x);
}

// matches! macro - returns bool
let is_some = matches!(val, Some(_));
let is_42 = matches!(val, Some(42));
```

Destructuring in match

match can destructure structs, tuples, and nested enums:

Rust:

```
struct Point { x: i32, y: i32 }

let p = Point { x: 0, y: 7 };
match p {
    Point { x: 0, y } => println!("on y-axis at {}", y),
    Point { x, y: 0 } => println!("on x-axis at {}", x),
    Point { x, y } => println!("({}, {})", x, y),
}

// Slice patterns
let v = [1, 2, 3];
match v {
    [1, rest @ ..] => println!("starts with 1, rest: {:?}", rest),
    [.., 3] => println!("ends with 3"),
    _ => println!("other"),
}
```

Ownership**Source**

[src/rust/ownership](#)

C++ has copy semantics by default. Move semantics were added in C++11 via `std::move`, but using a moved-from object is undefined behavior that the compiler won't catch.

Rust uses move semantics by default for types that manage resources. When you assign a `String` to another variable, ownership transfers and the original becomes invalid. The compiler enforces this, preventing use-after-move bugs.

C++:

```
#include <string>
#include <utility>
```

(continues on next page)

(continued from previous page)

```
int main() {
    std::string s1 = "hello";
    std::string s2 = s1;           // copy (deep clone)
    std::string s3 = std::move(s1); // move, s1 is now in valid but unspecified state
    // Using s1 here is undefined behavior but compiles
}
```

Rust:

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1.clone(); // explicit clone (deep copy)
    let s3 = s1;         // move, s1 is no longer valid
    // println!("{}", s1); // error: borrow of moved value
}
```

Copy vs Move Types

Types that implement the Copy trait (like integers, floats, bools) are copied implicitly. Types that manage heap resources (like String, Vec) are moved.

In C++, all types are copyable by default (unless explicitly deleted). There's no built-in distinction between “copy types” and “move types” - the programmer must remember which types are expensive to copy.

C++:

```
int x = 5;
int y = x;           // copy (cheap, primitive)

std::string s1 = "hello";
std::string s2 = s1; // copy (expensive, heap allocation)
// Both s1 and s2 are valid - no compiler help
```

Rust:

```
// Copy types - implicit copy
let x = 5;
let y = x; // copy, both x and y are valid
println!("{}", x, y); // OK

// Move types - ownership transfer
let s1 = String::from("hello");
let s2 = s1; // move, s1 is invalid
// println!("{}", s1); // error: use of moved value
```

References and Borrowing

Source

src/rust/borrowing

C++ references are aliases that the programmer must ensure don't outlive their data. Rust's borrow checker enforces at compile time that:

1. You can have either one mutable reference OR any number of immutable references

- References must always be valid (no dangling references)

C++:

```
void modify(int& x) { x += 1; }
void read(const int& x) { std::cout << x; }

int main() {
    int val = 5;
    modify(val);    // mutable reference
    read(val);      // const reference
    // No compile-time check for dangling references
}
```

Rust:

```
fn modify(x: &mut i32) { *x += 1; }
fn read(x: &i32) { println!("{}", x); }

fn main() {
    let mut val = 5;
    modify(&mut val); // mutable borrow
    read(&val);       // immutable borrow
    // Compiler guarantees no dangling references
}
```

Borrowing Rules

C++ has no equivalent compile-time enforcement. You can have multiple pointers or references to the same data, with any combination of const/non-const, and the compiler won't prevent data races or aliasing bugs.

C++ (compiles but dangerous):

```
std::string s = "hello";
std::string& r1 = s;    // mutable reference
const std::string& r2 = s; // const reference
r1 += " world";        // modifies s while r2 exists
// No compiler error, but can cause subtle bugs in multithreaded code
```

Rust (enforced at compile time):

```
let mut s = String::from("hello");

// Multiple immutable borrows - OK
let r1 = &s;
let r2 = &s;
println!("{}", r1, r2);

// Mutable borrow after immutable borrows end - OK
let r3 = &mut s;
r3.push_str(" world");

// Cannot have mutable and immutable borrows simultaneously
// let r4 = &s;
// let r5 = &mut s; // error: cannot borrow as mutable
```

Lifetimes

Lifetimes are Rust's way of ensuring references don't outlive the data they point to. Most of the time, lifetimes are inferred. When the compiler can't infer them, you must annotate explicitly.

C++ (dangling reference - compiles but UB):

```
int& get_ref() {  
    int x = 5;  
    return x; // dangling reference - undefined behavior  
}
```

Rust (compile error):

```
fn get_ref() -> &i32 {  
    let x = 5;  
    &x // error: cannot return reference to local variable  
}
```

Explicit Lifetime Annotations

When a function takes multiple references and returns a reference, you may need to specify how the lifetimes relate.

C++ has no equivalent syntax. The programmer must document and manually ensure that returned references remain valid. The compiler provides no help.

C++ (no lifetime tracking):

```
// Programmer must ensure returned reference outlives usage  
// No way to express "return value lives as long as inputs"  
const std::string& longest(const std::string& x, const std::string& y) {  
    return x.length() > y.length() ? x : y;  
}  
  
// Dangerous: easy to return reference to temporary  
const std::string& dangerous(const std::string& x) {  
    std::string temp = x + "!";  
    return temp; // dangling reference - compiles but UB  
}
```

Rust (explicit lifetime annotations):

```
// 'a is a lifetime parameter - return value lives as long as inputs  
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}  
  
fn main() {  
    let s1 = String::from("long string");  
    let s2 = String::from("short");  
    let result = longest(&s1, &s2);  
    println!("Longest: {}", result);  
}
```

Slices

See [Collections](#) for details on slices (`&[T]`), `Vec<T>`, and fixed-size arrays (`[T; N]`).

The ? Operator

Rust's `?` operator unwraps a `Result` on success or returns the error early. It replaces verbose match blocks for error propagation.

C++ (manual error checking):

```
#include <fstream>
#include <vector>
#include <optional>

std::optional<std::vector<char>> read_file(const std::string& path) {
    std::ifstream file(path, std::ios::binary);
    if (!file.is_open()) {
        return std::nullopt; // manual error check
    }
    std::vector<char> data((std::istreambuf_iterator<char>(file)),
                          std::istreambuf_iterator<char>());
    return data;
}
```

Rust (with `??` operator):

```
use std::fs;
use std::io;

fn read_file(path: &str) -> io::Result<Vec<u8>> {
    let data = fs::read(path)?; // returns Err early if file read fails
    Ok(data)
}
```

The `?` operator is equivalent to:

```
let data = match fs::read(path) {
    Ok(bytes) => bytes,           // unwrap the value
    Err(e) => return Err(e),     // propagate the error
};
```

Note

The `?` operator can only be used in functions that return `Result` (or `Option`). It does not work in `main()` unless `main` returns `Result`.

`Result<T, E>` and `io::Result<T>`

`Result` is a generic enum with two type parameters:

```
enum Result<T, E> {
    Ok(T), // success - holds a value of type T
```

(continues on next page)

(continued from previous page)

```
Err(E),    // failure - holds an error of type E
}
```

The standard library provides a type alias `io::Result<T>` for I/O operations, which fixes the error type to `io::Error`:

```
// defined in std::io
type Result<T> = Result<T, io::Error>;
```

So `io::Result<>` expands to `Result<>, io::Error>`:

- `Ok()` — success, no return value (like `void` in C++)
- `Err(io::Error)` — failure, holds an I/O error

```
use std::fs;
use std::io;

fn load(path: &str) -> io::Result<> {
    let data = fs::read(path)?; // could fail → io::Error
    println!("read {} bytes", data.len());
    Ok() // success, nothing to return
}
```

Struct Mutability

In Rust, mutability is a property of the **binding**, not individual fields. You cannot make some fields mutable and others immutable — the entire struct is either mutable or immutable based on how the variable is declared.

C++ (per-field mutability):

```
struct Foo {
    int x;           // mutable
    const int y;     // immutable per-field
    mutable int z;   // always mutable, even on const instance
};

const Foo foo{1, 2, 3};
// foo.x = 10;      // error: foo is const
foo.z = 42;         // OK: mutable member
```

Rust (whole-struct mutability):

```
struct Foo {
    x: i32,
    y: i32,
    z: i32,
}

let foo = Foo { x: 1, y: 2, z: 3 }; // immutable - ALL fields
// foo.x = 10;                     // error: cannot mutate

let mut foo = Foo { x: 1, y: 2, z: 3 }; // mutable - ALL fields
foo.x = 10;                         // OK
foo.y = 20;                         // OK
```

Nested Struct Mutability

When a struct contains another struct, the nested struct inherits the mutability of the parent binding. If the parent is `mut`, all nested fields are mutable too.

```
struct Bar {
    val: i32,
}

struct Foo {
    bar: Bar,
}

let mut foo = Foo { bar: Bar { val: 1 } };
foo.bar.val = 10;           // OK - parent is mut, so nested fields are mut

let foo2 = Foo { bar: Bar { val: 1 } };
// foo2.bar.val = 10;       // error: entire tree is immutable
```

Interior Mutability

When you need to mutate data behind an immutable reference, Rust provides **interior mutability** types that move the borrow check to runtime:

```
use std::cell::RefCell;
use std::rc::Rc;

struct Foo {
    val: Rc<RefCell<i32>>, // shared ownership + interior mutability
}

let foo = Foo { val: Rc::new(RefCell::new(1)) };
*foo.val.borrow_mut() = 42; // mutate through immutable binding
```

For thread-safe interior mutability, use `Arc<Mutex<T>>`:

```
use std::sync::{Arc, Mutex};

let data = Arc::new(Mutex::new(0));
let data2 = Arc::clone(&data);

// Mutate from any thread holding a clone
*data2.lock().unwrap() = 42;
```

Summary table:

Pattern	Ownership	Thread-safe	Check
<code>&mut T</code>	single owner	N/A	compile-time
<code>RefCell<T></code>	single owner	No	runtime
<code>Rc<RefCell<T>></code>	multiple owners	No	runtime
<code>Arc<Mutex<T>></code>	multiple owners	Yes	runtime

Pointers and References

Source

`src/rust/borrowing`, `src/rust/lifetimes`

C++ has pointers (`T*`) and references (`T&`, `T&&`). Rust splits these into safe references (`&T`, `&mut T`) and unsafe raw pointers (`*const T`, `*mut T`). The key difference: Rust references are always valid — the compiler guarantees they never dangle.

C++	Rust	Notes
<code>const T&</code>	<code>&T</code>	Shared (immutable) reference
<code>T&</code>	<code>&mut T</code>	Exclusive (mutable) reference
<code>const T*</code>	<code>*const T</code>	Raw pointer, requires <code>unsafe</code>
<code>T*</code>	<code>*mut T</code>	Raw mutable pointer, requires <code>unsafe</code>
<code>T&&</code> (rvalue ref / forwarding ref)	(no equivalent)	Rust moves by default; <code>T&&</code> in a template context is a forwarding (universal) reference — Rust doesn't need this since ownership transfer is the default

C++:

```
void increment(int& x) { x += 1; }           // mutable ref
void print(const int& x) { cout << x; }     // immutable ref

struct Config {
    const std::string& name; // reference member - no lifetime check!
    int value;

    void print() const {
        std::cout << name << "=" << value << "\n";
    }
};

int main() {
    int val = 5;
    increment(val);
    print(val);

    // BUG: reference member can easily dangle
    Config* c;
    {
        std::string s = "timeout";
        c = new Config{s, 30};
    } // s destroyed - c->name is now dangling!
    c->print(); // undefined behavior
}
```

Rust:

```
fn increment(x: &mut i32) { *x += 1; } // exclusive reference
fn print_val(x: &i32) { println!("{}", x); } // shared reference

// Struct holding a reference MUST declare a lifetime parameter.
```

(continues on next page)

(continued from previous page)

```
// This tells the compiler: Config cannot outlive the data it borrows.
struct Config<'a> {
    name: &'a str,
    value: i32,
}

impl<'a> Config<'a> {
    fn new(name: &'a str, value: i32) -> Self {
        Config { name, value }
    }

    // Accessing members - return lifetime tied to struct's lifetime
    fn name(&self) -> &str { self.name }

    fn display(&self) {
        println!("{}", self.name, self.value);
    }
}

fn main() {
    let mut val = 5;
    increment(&mut val);
    print_val(&val);

    // Compiler ensures Config cannot outlive the borrowed string
    let name = String::from("timeout");
    let cfg = Config::new(&name, 30);
    cfg.display(); // timeout=30

    // This would NOT compile - Rust prevents the dangling reference:
    // let cfg;
    // {
    //     let name = String::from("timeout");
    //     cfg = Config::new(&name, 30);
    // } // name dropped here
    // cfg.display(); // error: `name` does not live long enough
}
```

Borrowing rules (enforced at compile time):

- You can have **many** &T OR **one** &mut T — never both at the same time
- References must always be valid (no dangling)

Dereferencing and Auto-deref

For primitives, you must explicitly dereference with *. But for struct member access, Rust's . operator auto-dereferences — no -> operator like C++:

C++:

```
struct Point { int x, y; };

Point p{1, 2};
```

(continues on next page)

(continued from previous page)

```
Point& r = p;
Point* ptr = &p;

r.x;      // dot for references
ptr->x;    // arrow for pointers
(*ptr).x; // or explicit deref + dot
```

Rust:

```
struct Point { x: i32, y: i32 }

let p = Point { x: 1, y: 2 };
let r = &p;
let b = Box::new(Point { x: 3, y: 4 });

r.x;      // auto-deref: same as (*r).x
b.x;      // auto-deref through Box too
// No -> operator in Rust - dot handles everything

// Explicit * only needed for primitives
let mut val = 5;
let m = &mut val;
*m += 1;   // must deref to assign to i32
```

Lifetimes

When a struct holds a reference or a function returns one, Rust needs to know how long it's valid. In C++, this is entirely the programmer's responsibility (and a common source of bugs). Rust makes it explicit with lifetime annotations 'a.

C++ (dangling reference — compiles, crashes at runtime):

```
struct Parser {
    const std::string& input; // no lifetime tracking

    // Can easily outlive the string it references
    std::string_view next_token() const {
        return std::string_view(input).substr(0, input.find(' '));
    }
};
```

Rust (lifetime annotation — compiler enforces validity):

```
struct Parser<'a> {
    input: &'a str, // 'a = "input must live at least as long as Parser"
}

impl<'a> Parser<'a> {
    fn new(input: &'a str) -> Self {
        Parser { input }
    }

    // Return type borrows from self, which borrows from 'a
    fn next_token(&self) -> &str {
```

(continues on next page)

(continued from previous page)

```

        self.input.split_whitespace().next().unwrap_or("")
    }
}

let text = String::from("hello world");
let parser = Parser::new(&text);
println!("{}", parser.next_token()); // "hello"

// Won't compile - parser cannot outlive text:
// let parser;
// {
//     let text = String::from("hello world");
//     parser = Parser::new(&text);
// }
// parser.next_token(); // error: `text` does not live long enough

```

Multiple lifetimes — when a struct borrows from different sources:

```

// 'a and 'b can be different lifetimes
struct Pair<'a, 'b> {
    key: &'a str,
    value: &'b str,
}

impl<'a, 'b> Pair<'a, 'b> {
    fn key(&self) -> &'a str { self.key }
    fn value(&self) -> &'b str { self.value }
}

```

When you don't need lifetime annotations (lifetime elision rules):

```

// Compiler infers: input and output have the same lifetime
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

// Methods on &self - return lifetime tied to self automatically
impl<'a> Parser<'a> {
    fn peek(&self) -> &str { // no annotation needed
        self.input
    }
}

```

'static is a special lifetime meaning “valid for the entire program”:

```

// String literals are always &static str
let s: &'static str = "I live forever";

// Struct can hold 'static references without lifetime parameter issues
let cfg = Config::new("timeout", 30); // &static str, always valid

```

See Also

- *Ownership* - Ownership, borrowing rules, and borrow checker
- *RAII* - Resource management and Drop trait
- *Smart Pointers* - Box, Rc, Arc, RefCell

1.8.2 Ownership

Table of Contents

- *Ownership*
 - *Ownership Rules*
 - *Move vs Copy*
 - *Borrowing vs C++ References*
 - *Borrowing Rules*
 - *Dangling References*
 - *Lifetime Annotations*
 - *Lifetime Elision*
 - *Structs with References*
 - *Static Lifetime*
 - *Interior Mutability*
 - *Common Ownership Patterns*

Rust's ownership system is its most distinctive feature, providing memory safety without garbage collection. For C++ developers, ownership combines familiar concepts (RAII, move semantics, references) with compile-time enforcement that prevents dangling pointers, data races, and use-after-free bugs.

Ownership Rules

Source

`src/rust/ownership`

Rust's three ownership rules:

1. Each value has exactly one owner
2. When the owner goes out of scope, the value is dropped
3. Ownership can be transferred (moved) or borrowed

In C++, there's no compiler enforcement of ownership. You can create pointers to local variables, and the compiler won't stop you from using them after the pointed-to object is destroyed. This leads to dangling pointer bugs that are notoriously difficult to debug because they may appear to work in some runs but crash in others.

C++ (no enforced ownership):

```
std::string* ptr;
{
    std::string s = "hello";
```

(continues on next page)

(continued from previous page)

```
ptr = &s;
} // s destroyed
std::cout << *ptr; // Undefined behavior: dangling pointer
```

Rust's ownership model makes the single-owner rule explicit. When you assign a heap-allocated value to another variable, ownership transfers (moves) to the new variable. The original variable becomes invalid, and the compiler will reject any attempt to use it. This eliminates use-after-move bugs at compile time rather than discovering them through crashes or sanitizers at runtime.

Rust (ownership enforced):

```
let s1 = String::from("hello"); // s1 owns the String
let s2 = s1;                     // ownership moved to s2
// println!("{}", s1);          // Error: s1 no longer valid
println!("{}", s2);              // OK: s2 is the owner
```

Move vs Copy

Source

src/rust/ownership

C++ defaults to copying on assignment, which can be expensive for large objects. To move, you must explicitly use `std::move()`. This opt-in approach means programmers often forget to move when they should, leading to unnecessary copies. After a move, the source object is in a “valid but unspecified state” - you can still accidentally use it, leading to subtle bugs.

C++:

```
std::string s1 = "hello";
std::string s2 = s1;           // Copy (expensive)
std::string s3 = std::move(s1); // Move (s1 now empty)
```

Rust inverts this default: assignment moves by default for types that manage heap memory (like `String`, `Vec`, `Box`). If you want a copy, you must explicitly call `.clone()`. This makes the cost of copying visible in the code. For simple stack-only types like integers and floats, Rust implements the `Copy` trait, allowing implicit copying since it's cheap. After a move in Rust, the source variable is completely invalid - not just empty - so the compiler catches any accidental use.

Rust:

```
let s1 = String::from("hello");
let s2 = s1;           // Move (s1 invalidated)
let s3 = s2.clone();   // Explicit copy

// Copy types (primitives, tuples of Copy types)
let x = 5;
let y = x;             // Copy (both valid)
println!("{}", x, y);  // OK
```

Borrowing vs C++ References

Source

src/rust/borrowing

Rust borrowing (`&T`, `&mut T`) looks like C++ references but with crucial differences enforced at compile time:

Aspect	C++ References	Rust Borrowing
Validity	Not checked (can dangle)	Compiler-enforced (never dangle)
Multiple readers	Allowed (no enforcement)	Allowed (&T)
Single writer	Not enforced	Enforced (&mut T exclusive)
Aliasing + mutation	Allowed (causes bugs)	Forbidden at compile time
Null	Possible (undefined behavior)	Not possible

One of the most insidious bugs in C++ occurs when you hold a reference to an element inside a container, then modify the container in a way that invalidates that reference. The classic example is holding a reference to a vector element, then pushing to the vector. If the vector reallocates, your reference now points to freed memory. The C++ compiler cannot detect this because it doesn't track the relationship between the reference and the container.

C++ (compiles but has data race potential):

```
void process(std::vector<int>& vec, int& elem) {
    vec.push_back(42); // May invalidate elem!
    std::cout << elem; // Undefined behavior if reallocated
}

int main() {
    std::vector<int> v = {1, 2, 3};
    process(v, v[0]); // Dangerous: elem may dangle after push_back
}
```

Rust's borrow checker understands that when you borrow an element from a vector, you're borrowing from the vector itself. It won't let you mutate the vector while that borrow exists. This prevents iterator invalidation, a entire class of bugs that plagues C++ codebases. The error message clearly explains why the code is rejected: you cannot have a mutable borrow of the vector while an immutable borrow of its contents exists.

Rust (compile error prevents the bug):

```
fn process(vec: &mut Vec<i32>, elem: &i32) {
    vec.push(42); // Error: cannot borrow `vec` as mutable
    println!("{}", elem); // because `elem` is borrowed from `vec`
}

fn main() {
    let mut v = vec![1, 2, 3];
    let elem = &v[0];
    // process(&mut v, elem); // Won't compile
}
```

Borrowing Rules

Source

[src/rust/borrowing](#)

The borrow checker enforces two fundamental rules at compile time that prevent data races and aliasing bugs:

1. You can have either one mutable reference OR any number of immutable references
2. References must always be valid (no dangling)

These rules implement a reader-writer lock pattern at compile time. Multiple readers can access data simultaneously (shared/immutable borrows), but a writer needs exclusive access (mutable borrow). Unlike runtime locks, violations

are caught during compilation with zero runtime overhead. The borrow checker also tracks when borrows end, so you can have a mutable borrow after immutable borrows go out of scope.

```
let mut s = String::from("hello");

// Multiple immutable borrows: OK
let r1 = &s;
let r2 = &s;
println!("{}", r1, r2);

// Mutable borrow after immutable borrows end: OK
let r3 = &mut s;
r3.push_str(" world");

// Simultaneous mutable and immutable: ERROR
// let r4 = &s;
// let r5 = &mut s; // Error: cannot borrow as mutable
```

Dangling References

Source

src/rust/lifetimes

Returning a reference to a local variable is a classic C++ bug. The function creates a local string on the stack, returns a reference to it, and then the stack frame is destroyed. The caller receives a reference to garbage memory. Most compilers will warn about this, but it's not a hard error, and more complex cases (like returning a reference through multiple function calls) often escape detection.

C++ (compiles but undefined behavior):

```
std::string& get_string() {
    std::string s = "hello";
    return s; // Dangling reference!
}
```

Rust's borrow checker performs lifetime analysis to prove that all references are valid. It understands that a reference to a local variable cannot outlive the function call. The error message is clear: the local variable `s` does not live long enough to be returned as a reference. The solution is to return owned data, transferring ownership to the caller, or to borrow from data the caller provides.

Rust (compile error):

```
// Won't compile: cannot return reference to local variable
fn get_string() -> &String {
    let s = String::from("hello");
    &s // Error: `s` does not live long enough
}

// Solution: return owned data
fn get_string() -> String {
    String::from("hello") // Ownership transferred to caller
}
```

Lifetime Annotations

Source

[src/rust/lifetimes](#)

When a function takes multiple references and returns a reference, the compiler needs to know which input the output is derived from. This determines how long the returned reference is valid. C++ has no way to express this relationship, so the compiler cannot verify that the returned reference won't dangle.

C++ (no lifetime tracking):

```
// C++ doesn't track which input the return value depends on
const std::string& longer(const std::string& a, const std::string& b) {
    return a.size() > b.size() ? a : b;
}

int main() {
    const std::string& result = longer("short", "longer string");
    // Both temporaries destroyed, result is dangling!
}
```

Rust's lifetime annotations ('a) are a way to tell the compiler about the relationship between input and output references. The syntax `fn longer<'a>(a: &'a str, b: &'a str) -> &'a str` means “the returned reference will be valid as long as both input references are valid.” The compiler uses this information to ensure callers don't use the result after either input has been dropped. Lifetimes don't change how long values live; they're annotations that help the compiler verify your code is safe.

Rust (explicit lifetime annotation):

```
// 'a means: returned reference lives as long as both inputs
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() { a } else { b }
}

fn main() {
    let s1 = String::from("short");
    let s2 = String::from("longer string");
    let result = longer(&s1, &s2); // OK: s1 and s2 outlive result
    println!("{}", result);
}
```

Lifetime Elision

Source

[src/rust/lifetimes](#)

Writing lifetime annotations everywhere would be tedious, so Rust has elision rules that let you omit them in common patterns. The compiler applies these rules automatically, inserting the lifetimes for you. If the rules don't apply (like when you have multiple input references and return a reference), you must write explicit annotations. Understanding elision helps you know when annotations are needed and makes reading Rust code easier.

1. Each input reference gets its own lifetime
2. If there's exactly one input lifetime, it's assigned to all outputs
3. If there's `&self` or `&mut self`, its lifetime is assigned to outputs

```
// These are equivalent:
fn first_word(s: &str) -> &str { ... }
fn first_word<'a>(s: &'a str) -> &'a str { ... }

// These are equivalent:
fn get_name(&self) -> &str { ... }
fn get_name<'a>(&'a self) -> &'a str { ... }
```

Structs with References

Source

src/rust/lifetimes

Storing references in structs is dangerous in C++ because there's no guarantee the referenced data outlives the struct. A common bug is constructing a struct with a reference to a temporary, which is destroyed immediately, leaving the struct with a dangling reference. The C++ compiler may not warn about this, especially when the temporary's lifetime is extended in some cases but not others.

C++ (no compile-time guarantee):

```
struct Excerpt {
    const std::string& text; // Reference member
    Excerpt(const std::string& t) : text(t) {}
};

int main() {
    Excerpt e("temp"); // Dangling: temporary destroyed
    std::cout << e.text; // Undefined behavior
}
```

In Rust, a struct containing a reference must declare a lifetime parameter. This lifetime represents “the struct cannot outlive the data it references.” The compiler enforces this constraint at every use site. You cannot create an `Excerpt` that outlives the string it borrows from. This makes reference-holding structs safe to use, enabling patterns like zero-copy parsing where you return structs that borrow from the input data.

Rust (lifetime enforced):

```
struct Excerpt<'a> {
    text: &'a str, // Reference must live at least as long as struct
}

fn main() {
    let novel = String::from("Call me Ishmael...");
    let excerpt = Excerpt { text: &novel };
    println!("{}", excerpt.text);
} // novel outlives excerpt, so this is safe
```

Static Lifetime

Source

src/rust/lifetimes

The `'static` lifetime is a special lifetime that means “valid for the entire program duration.” String literals in both C++ and Rust have static storage duration - they're embedded in the binary and exist from program start to end. In Rust, string literals have type `&'static str`. The `'static` bound is also used in trait bounds to indicate a type contains no non-static references, which is required for spawning threads (since the thread might outlive the current scope).

```
// String literals are &'static str
fn get_greeting() -> &'static str {
    "Hello, world!" // OK: string literal has 'static lifetime
}

// Static variables also have 'static lifetime
static GREETING: &str = "Hello";
```

Interior Mutability

Source

src/rust/smart_pointers

Sometimes you need to mutate data through a shared (immutable) reference. In C++, the mutable keyword allows modifying members even in const methods - commonly used for caches, lazy initialization, or reference counting. This bypasses const correctness and can lead to data races if not carefully managed.

C++ (mutable keyword):

```
class Counter {
    mutable int count = 0; // Can modify even in const methods
public:
    void increment() const { ++count; }
    int get() const { return count; }
};
```

Rust provides interior mutability through types like `RefCell<T>` and `Cell<T>`. These types move borrow checking from compile time to runtime. `RefCell` tracks borrows dynamically and will panic if you violate the borrowing rules (e.g., two mutable borrows at once). This is useful for patterns like the observer pattern, graph structures with cycles, or mock objects in tests. Use interior mutability sparingly - compile-time checking is preferable when possible.

Rust (RefCell for interior mutability):

```
use std::cell::RefCell;

struct Counter {
    count: RefCell<i32>,
}

impl Counter {
    fn new() -> Self {
        Counter { count: RefCell::new(0) }
    }

    fn increment(&self) { // &self, not &mut self
        *self.count.borrow_mut() += 1;
    }

    fn get(&self) -> i32 {
        *self.count.borrow()
    }
}
```


Common Ownership Patterns

Source

`src/rust/ownership`

Understanding when to take ownership versus borrow is key to writing idiomatic Rust. Take ownership when the function needs to store the value, transfer it elsewhere, or the caller won't need it anymore. Borrow when you only need to read or temporarily modify the value. These patterns become second nature with practice.

Taking ownership (consuming):

When a function takes ownership, the caller gives up the value permanently. This is appropriate when the function will store the value in a data structure, return it as part of a larger value, or when the caller is done with it. The value is dropped when the function ends (unless returned or stored).

```
fn consume(s: String) {
    println!("{}", s);
} // s dropped here

let s = String::from("hello");
consume(s);
// println!("{}", s); // Error: s was moved
```

Borrowing (non-consuming):

Borrowing with `&T` lets a function read data without taking ownership. The caller retains ownership and can continue using the value after the function returns. This is the most common pattern for functions that only need to inspect data.

```
fn borrow(s: &String) {
    println!("{}", s);
}

let s = String::from("hello");
borrow(&s);
println!("{}", s); // OK: s still valid
```

Mutable borrowing:

Mutable borrowing with `&mut T` allows temporary exclusive access to modify data. The caller retains ownership but cannot access the value while it's mutably borrowed. This is used when a function needs to modify data in place without taking ownership.

```
fn modify(s: &mut String) {
    s.push_str(" world");
}

let mut s = String::from("hello");
modify(&mut s);
println!("{}", s); // "hello world"
```

Returning ownership:

Functions can create values and transfer ownership to the caller. This is how constructors work in Rust (conventionally named `new`). The caller becomes the owner and is responsible for the value's lifetime.

```
fn create() -> String {
    String::from("created") // Ownership transferred to caller
```

(continues on next page)

(continued from previous page)

```

}

let s = create(); // s owns the String

```

1.8.3 RAII

Table of Contents

- *RAII*
 - *Constructors*
 - * *Default Trait*
 - *Destructors and Drop*
 - * *Drop Order*
 - *Explicit Drop*
 - * *Preventing Drop*
 - *Rule of Three/Five vs Rust*
 - *this vs self*
 - *See Also*

RAII (Resource Acquisition Is Initialization) is fundamental to both C++ and Rust. Resources are acquired during initialization and released when the object goes out of scope. Rust's ownership system makes RAII even more powerful by guaranteeing exactly when destructors run.

Constructors

Source

src/rust/structs

C++ has dedicated constructor syntax. Rust doesn't have constructors as a language feature; instead, you define associated functions (commonly named `new`) that return `Self`.

C++:

```

class Point {
    double x_, y_;
public:
    // Default constructor
    Point() : x_(0), y_(0) {}

    // Parameterized constructor
    Point(double x, double y) : x_(x), y_(y) {}

    // Copy constructor (implicit or explicit)
    Point(const Point& other) = default;

    // Move constructor
    Point(Point&& other) noexcept = default;

```

(continues on next page)

(continued from previous page)

```
};

int main() {
    Point p1;           // default constructor
    Point p2(3.0, 4.0); // parameterized constructor
    Point p3 = p2;      // copy constructor
}
```

Rust:

```
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // Associated function (not a method - no self)
    fn new(x: f64, y: f64) -> Self {
        Point { x, y }
    }

    // Default-like constructor
    fn origin() -> Self {
        Point { x: 0.0, y: 0.0 }
    }
}

fn main() {
    let p1 = Point::origin(); // "default" constructor
    let p2 = Point::new(3.0, 4.0); // parameterized constructor
    let p3 = p2;                // move (Point doesn't impl Copy)
}
```

Default Trait

Rust's Default trait is similar to C++ default constructors:

```
#[derive(Default)]
struct Config {
    debug: bool, // defaults to false
    timeout: u32, // defaults to 0
    name: String, // defaults to ""
}

fn main() {
    let config = Config::default();
    // Or with struct update syntax
    let custom = Config {
        debug: true,
        ..Default::default()
    };
}
```

Destructors and Drop

Source

src/rust/raii

C++ uses destructors (~ClassName). Rust uses the Drop trait with a drop method. Both are called automatically when the object goes out of scope.

C++:

```
#include <iostream>

class Resource {
    int* data;
public:
    Resource(int value) : data(new int(value)) {
        std::cout << "Acquired: " << *data << "\n";
    }

    ~Resource() {
        std::cout << "Released: " << *data << "\n";
        delete data;
    }

    // Rule of five: need copy/move constructors and assignments
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
};

int main() {
    Resource r(42);
    // destructor called at end of scope
}
```

Rust:

```
struct Resource {
    data: i32,
}

impl Resource {
    fn new(value: i32) -> Self {
        println!("Acquired: {}", value);
        Resource { data: value }
    }
}

impl Drop for Resource {
    fn drop(&mut self) {
        println!("Released: {}", self.data);
    }
}

fn main() {
    let r = Resource::new(42);
}
```

(continues on next page)

(continued from previous page)

```

    // Drop::drop called at end of scope
}

```

Drop Order

Variables are dropped in reverse order of declaration:

```

struct Droppable(i32);

impl Drop for Droppable {
    fn drop(&mut self) {
        println!("Dropping {}", self.0);
    }
}

fn main() {
    let a = Droppable(1);
    let b = Droppable(2);
    let c = Droppable(3);
}
// Output: Dropping 3, Dropping 2, Dropping 1

```

Explicit Drop

Use `std::mem::drop()` to drop a value before its scope ends:

C++ (no direct equivalent, use `scope` or `unique_ptr`):

```

{
    Resource r(42);
    // use r
} // r destroyed here
// continue without r

```

Rust:

```

fn main() {
    let r = Resource::new(42);
    // use r
    drop(r); // explicitly drop
    // r is no longer valid
    println!("Resource already released");
}

```

Preventing Drop

`std::mem::forget()` prevents the destructor from running. Useful for FFI when transferring ownership to C code:

```

use std::mem;

fn main() {
    let v = vec![1, 2, 3];

```

(continues on next page)

(continued from previous page)

```

mem::forget(v); // destructor won't run, memory leaked
// Use carefully - typically only for FFI
}

```

Rule of Three/Five vs Rust

C++ requires implementing copy/move constructors and assignment operators (Rule of Three/Five). Rust handles this through traits:

C++ Rule of Five:

```

class Buffer {
    char* data;
    size_t size;
public:
    Buffer(size_t n) : data(new char[n]), size(n) {}
    ~Buffer() { delete[] data; }

    // Copy constructor
    Buffer(const Buffer& other) : data(new char[other.size]), size(other.size) {
        std::copy(other.data, other.data + size, data);
    }

    // Copy assignment
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new char[size];
            std::copy(other.data, other.data + size, data);
        }
        return *this;
    }

    // Move constructor
    Buffer(Buffer&& other) noexcept : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }

    // Move assignment
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            size = other.size;
            other.data = nullptr;
            other.size = 0;
        }
        return *this;
    }
};

```

Rust (much simpler):

```

struct Buffer {
    data: Vec<u8>,
}

impl Buffer {
    fn new(size: usize) -> Self {
        Buffer { data: vec![0; size] }
    }
}

// Move is automatic, no implementation needed
// Clone can be derived if needed
impl Clone for Buffer {
    fn clone(&self) -> Self {
        Buffer { data: self.data.clone() }
    }
}

// Drop is automatic for Vec, no implementation needed

```

this vs self**Source**

`src/rust/self_this`

C++ has implicit `this` pointer. Rust requires explicit `self` parameter with three forms that integrate with ownership:

C++:

```

class Counter {
    int value = 0;
public:
    void increment() { this->value++; } // this is implicit
    int get() const { return value; }
};

```

Rust:

```

struct Counter {
    value: i32,
}

impl Counter {
    fn increment(&mut self) { // mutable borrow
        self.value += 1;
    }

    fn get(&self) -> i32 { // immutable borrow
        self.value
    }

    fn into_value(self) -> i32 { // takes ownership, consumes self
        self.value
    }
}

```

(continues on next page)

(continued from previous page)

```
}
}
```

See Also

- *Rust Basics* - Ownership and borrowing fundamentals
- *Smart Pointers* - Smart pointers for shared ownership

1.8.4 Strings

Table of Contents

- *Strings*
 - *String vs &str*
 - *Creating Strings*
 - *String Operations*
 - *Common Methods*
 - *String vs &str in Function Parameters*
 - *UTF-8 Handling*
 - *Converting Between Types*
 - *See Also*

Source

src/rust/strings

Rust has two main string types: `String` (owned, heap-allocated, growable) and `&str` (borrowed string slice). This is similar to C++'s `std::string` and `std::string_view`, but Rust strings are always valid UTF-8.

String vs &str

C++:

```
#include <string>
#include <string_view>

void print(std::string_view s) { // borrowed, no copy
    std::cout << s << "\n";
}

int main() {
    std::string owned = "hello"; // owned string
    owned += " world";           // mutable
    print(owned);                // implicit conversion
    print("literal");            // works with literals
}
```

Rust:


```
fn print(s: &str) { // borrowed string slice
    println!("{}", s);
}

fn main() {
    let owned = String::from("hello"); // owned String
    let owned = owned + " world";      // concatenation
    print(&owned);                     // borrow as &str
    print("literal");                  // &str literal
}
```

Creating Strings

```
// From literal
let s1 = String::from("hello");
let s2 = "hello".to_string();
let s3: String = "hello".into();

// Empty string
let s4 = String::new();

// With capacity
let s5 = String::with_capacity(100);

// From format
let name = "world";
let s6 = format!("Hello, {}!", name);
```

String Operations

Concatenation:

```
let s1 = String::from("Hello");
let s2 = String::from(" World");

// Using + (takes ownership of s1)
let s3 = s1 + &s2; // s1 moved, s2 borrowed

// Using format! (doesn't take ownership)
let s1 = String::from("Hello");
let s4 = format!("{}", s1, s2); // s1 and s2 still valid

// Using push_str
let mut s5 = String::from("Hello");
s5.push_str(" World");
```

Slicing:

```
let s = String::from("hello world");

let hello = &s[0..5]; // "hello"
let world = &s[6..];  // "world"
```

(continues on next page)

(continued from previous page)

```
let full = &s[..];      // "hello world"

// Warning: indices are byte positions, not char positions
// Slicing in middle of UTF-8 char will panic
```

Iteration:

```
let s = "hello";

// By characters
for c in s.chars() {
    println!("{}", c);
}

// By bytes
for b in s.bytes() {
    println!("{}", b);
}
```

Common Methods

```
let s = String::from(" Hello World ");

// Trimming
let trimmed = s.trim();           // "Hello World"
let left = s.trim_start();        // "Hello World "
let right = s.trim_end();         // " Hello World"

// Case conversion
let upper = s.to_uppercase();     // " HELLO WORLD "
let lower = s.to_lowercase();     // " hello world "

// Searching
let contains = s.contains("World"); // true
let starts = s.starts_with(" Hello"); // true
let pos = s.find("World");         // Some(8)

// Replacing
let replaced = s.replace("World", "Rust"); // " Hello Rust "

// Splitting
let parts: Vec<&str> = "a,b,c".split(',').collect(); // ["a", "b", "c"]
```

String vs &str in Function Parameters

Prefer &str for function parameters when you don't need ownership:

```
// Good: accepts both String and &str
fn greet(name: &str) {
    println!("Hello, {}!", name);
}
```

(continues on next page)

(continued from previous page)

```
fn main() {
    let owned = String::from("Alice");
    greet(&owned);    // borrow String as &str
    greet("Bob");    // &str literal directly
}

// Less flexible: only accepts String
fn greet_owned(name: String) {
    println!("Hello, {}!", name);
}
```

UTF-8 Handling

Rust strings are always valid UTF-8. This differs from C++ where strings are byte sequences:

```
let s = "Hello, !";

// Length in bytes vs characters
println!("Bytes: {}", s.len());    // 14 bytes
println!("Chars: {}", s.chars().count()); // 10 characters

// Accessing by index doesn't work directly
// let c = s[0]; // error: cannot index String

// Use chars() or bytes()
let first_char = s.chars().next();    // Some('H')
let first_byte = s.bytes().next();    // Some(72)
```

Converting Between Types

```
// &str to String
let s: &str = "hello";
let owned: String = s.to_string();
let owned: String = s.to_owned();
let owned: String = String::from(s);

// String to &str
let owned = String::from("hello");
let slice: &str = &owned;
let slice: &str = owned.as_str();

// Numbers to String
let n = 42;
let s = n.to_string();
let s = format!("{}", n);

// String to numbers
let s = "42";
let n: i32 = s.parse().unwrap();
let n: i32 = s.parse().expect("not a number");
```

See Also

- *Rust Basics* - Ownership and borrowing
- *Error Handling* - Error handling with `parse()`

1.8.5 Collections

Table of Contents

- *Collections*
 - *Vec (Vector)*
 - * *Vec Operations*
 - * *Vec<T> vs Fixed-size Arrays [T; N]*
 - * *Slices &[T]*
 - *HashMap*
 - * *HashMap Entry API*
 - *BTreeMap (Ordered Map)*
 - *HashSet*
 - * *Set Operations*
 - *VecDeque (Double-ended Queue)*
 - *Iterator Invalidation*
 - *See Also*

Source

[src/rust/collections](#)

Rust's standard library provides collections similar to C++ STL containers, but with a key difference: Rust collections integrate deeply with the ownership and borrowing system. This integration prevents iterator invalidation bugs at compile time - a common source of undefined behavior in C++. When you iterate over a Rust collection, the borrow checker ensures you cannot accidentally modify the collection in ways that would invalidate the iteration. Additionally, Rust collections are designed with modern CPU architectures in mind, favoring cache-friendly data structures like `Vec` and `HashMap` over pointer-heavy structures like linked lists.

The table below maps common C++ STL containers to their Rust equivalents. Note that Rust's `HashMap` and `HashSet` use a different hashing algorithm (SipHash by default) than most C++ implementations, prioritizing security against hash flooding attacks over raw speed.

C++	Rust
<code>std::vector</code>	<code>Vec<T></code>
<code>std::map</code>	<code>BTreeMap<K, V></code>
<code>std::unordered_map</code>	<code>HashMap<K, V></code>
<code>std::set</code>	<code>BTreeSet<T></code>
<code>std::unordered_set</code>	<code>HashSet<T></code>
<code>std::deque</code>	<code>VecDeque<T></code>
<code>std::list</code>	<code>LinkedList<T></code>

Vec (Vector)

`Vec<T>` is Rust's growable array type, equivalent to C++'s `std::vector`. It stores elements contiguously in heap memory and provides $O(1)$ amortized push/pop at the end. The `vec!` macro provides convenient initialization syntax similar to C++'s initializer lists.

The following example demonstrates basic vector operations including creation, modification, and iteration. Note that Rust requires explicit mutability with `let mut` to modify the vector:

C++:

```
#include <iostream>
#include <vector>

int main() {
    // Create and initialize a vector
    std::vector<int> v = {1, 2, 3};

    // Add and remove elements from the back
    v.push_back(4);
    v.push_back(5);
    v.pop_back(); // removes 5

    // Iterate and print elements
    std::cout << "Elements: ";
    for (int x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    // Access elements - unchecked vs bounds-checked
    int first = v[0];           // no bounds check, UB if out of range
    int safe = v.at(0);         // throws std::out_of_range if invalid

    std::cout << "First element: " << first << "\n";
    std::cout << "Size: " << v.size() << "\n";

    return 0;
}
```

Rust:

```
fn main() {
    // Create and initialize a vector using vec! macro
    let mut v = vec![1, 2, 3];

    // Add and remove elements from the back
    v.push(4);
    v.push(5);
    v.pop(); // removes 5, returns Some(5)

    // Iterate and print elements (borrowing with &v)
    print!("Elements: ");
    for x in &v {
        print!("{}", x);
    }
}
```

(continues on next page)

(continued from previous page)

```

}
println!();

// Access elements - panics vs returns Option
let first = v[0];           // panics if out of bounds
let safe = v.get(0);        // returns Option<T>, None if invalid

println!("First element: {}", first);
println!("Safe access: {:?}", safe);
println!("Size: {}", v.len());
}

```

Vec Operations

This example shows common vector operations including element access, modification, searching, sorting, and capacity management. Rust's iterator-based search methods return `Option` types, making it explicit when an element might not be found:

C++:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // Access elements
    int first = v.front();
    int last = v.back();
    // Slice via iterators (no direct slice syntax)

    // Modification
    v.push_back(6);
    v.pop_back();
    v.insert(v.begin(), 0);    // insert at front
    v.erase(v.begin());       // remove from front

    // Search
    auto it = std::find(v.begin(), v.end(), 3);
    bool found = (it != v.end());
    size_t pos = std::distance(v.begin(), it);

    // Sorting
    std::sort(v.begin(), v.end());
    std::sort(v.begin(), v.end(), std::greater<int>()); // reverse

    // Capacity
    size_t len = v.size();
    size_t cap = v.capacity();
    v.reserve(100);
    v.shrink_to_fit();
}

```

(continues on next page)

(continued from previous page)

```
std::cout << "Length: " << len << ", Capacity after reserve: " << v.capacity() << "\n";
return 0;
}
```

Rust:

```
fn main() {
    let mut v = vec![1, 2, 3, 4, 5];

    // Access elements
    let first = &v[0];
    let last = v.last();           // Option<&T>
    let slice = &v[1..3];          // [2, 3] - native slice syntax
    println!("First: {}, Last: {:?}, Slice: {:?}", first, last, slice);

    // Modification
    v.push(6);
    v.pop();                       // returns Option<T>
    v.insert(0, 0);                 // insert at index
    v.remove(0);                   // remove at index, returns element

    // Search - returns Option types for safety
    let pos = v.iter().position(|&x| x == 3); // Option<usize>
    let contains = v.contains(&3);           // bool
    println!("Position of 3: {:?}, Contains 3: {}", pos, contains);

    // Sorting
    v.sort();
    v.sort_by(|a, b| b.cmp(a));             // reverse order

    // Capacity management
    let len = v.len();
    let cap = v.capacity();
    v.reserve(100);
    v.shrink_to_fit();

    println!("Length: {}, Capacity: {}", len, v.capacity());
}
```

Vec<T> vs Fixed-size Arrays [T; N]

Rust has two main array types: `Vec<T>` (heap-allocated, growable) and `[T; N]` (stack-allocated, fixed size known at compile time). C++ has the same distinction with `std::vector<T>` vs `std::array<T, N>`.

C++:

```
#include <array>
#include <vector>

int main() {
```

(continues on next page)

(continued from previous page)

```

std::vector<uint8_t> rom;           // heap, size determined at runtime
rom.resize(0x4000);

std::array<uint8_t, 256> oam;      // stack, size fixed at compile time

return 0;
}

```

Rust:

```

fn main() {
    let rom: Vec<u8> = vec![0; 0x4000]; // heap, size determined at runtime

    let oam: [u8; 256] = [0; 256];     // stack, size fixed at compile time
}

```

Use `Vec<T>` when the size is unknown at compile time (e.g., loading ROM data from a file). Use `[T; N]` when the size is fixed by the hardware specification (e.g., PPU sprite memory is always 256 bytes).

Slices &[T]

A slice `&[T]` is a borrowed view into a contiguous sequence of elements. It is the idiomatic way to pass arrays or vectors to functions without transferring ownership. In C++, the closest equivalent is `std::span<T>` (C++20) or passing `const T*` with a length.

C++:

```

#include <iostream>
#include <span>
#include <vector>

// C++20: std::span borrows a contiguous range
void print_data(std::span<const uint8_t> data) {
    for (auto byte : data) {
        std::cout << (int)byte << " ";
    }
    std::cout << "\n";
}

// Before C++20: pass pointer + length
void print_data_legacy(const uint8_t* data, size_t len) {
    for (size_t i = 0; i < len; ++i) {
        std::cout << (int)data[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    std::vector<uint8_t> rom = {0x4E, 0x45, 0x53, 0x1A};
    print_data(rom);           // pass vector as span
    print_data({rom.data(), 2}); // pass first 2 bytes
    return 0;
}

```


Rust:

```
// &[u8] borrows a contiguous sequence - works with Vec, arrays, or other slices
fn print_data(data: &[u8]) {
    for byte in data {
        print!("{}", byte);
    }
    println();
}

fn main() {
    let rom: Vec<u8> = vec![0x4E, 0x45, 0x53, 0x1A];
    print_data(&rom);           // Vec<u8> auto-borrows as &[u8]
    print_data(&rom[..2]);       // pass first 2 bytes as a slice

    let arr: [u8; 4] = [0x4E, 0x45, 0x53, 0x1A];
    print_data(&arr);           // fixed-size array also works
}
```

Note

Prefer `&[T]` over `&Vec<T>` in function parameters. A `&[T]` accepts both `Vec<T>` and fixed-size arrays `[T; N]`, making the function more flexible. Returning `&[T]` from a method that owns a `Vec<T>` is also idiomatic:

```
struct Cartridge {
    prg_rom: Vec<u8>,
}

impl Cartridge {
    // return &[u8] instead of &Vec<u8>
    pub fn prg_rom(&self) -> &[u8] {
        &self.prg_rom
    }
}
```

HashMap

`HashMap<K, V>` is Rust's hash table implementation, equivalent to C++'s `std::unordered_map`. Keys must implement the `Hash` and `Eq` traits. The entry API provides an elegant way to handle insert-or-update patterns that would require multiple lookups in C++.

This example demonstrates basic hash map operations including insertion, lookup, and iteration. Note that Rust's `get` method returns an `Option`, making it impossible to accidentally use a non-existent key:

C++:

```
#include <iostream>
#include <string>
#include <unordered_map>

int main() {
    std::unordered_map<std::string, int> map;
```

(continues on next page)

(continued from previous page)

```

// Insert elements
map["one"] = 1;
map["two"] = 2;
map.insert({"three", 3});

// Lookup - must check if key exists
if (auto it = map.find("one"); it != map.end()) {
    std::cout << "Found: " << it->second << "\n";
}

// Dangerous: creates entry if missing!
int val = map["four"]; // inserts {"four", 0}

// Iterate over key-value pairs
std::cout << "All entries:\n";
for (const auto& [key, value] : map) {
    std::cout << "  " << key << ": " << value << "\n";
}

std::cout << "Size: " << map.size() << "\n";

return 0;
}

```

Rust:

```

use std::collections::HashMap;

fn main() {
    let mut map = HashMap::new();

    // Insert elements
    map.insert("one", 1);
    map.insert("two", 2);
    map.insert("three", 3);

    // Lookup - returns Option, no accidental insertion
    if let Some(value) = map.get("one") {
        println!("Found: {}", value);
    }

    // Safe: get returns None for missing keys
    let missing = map.get("four");
    println!("Missing key: {:?}", missing); // None

    // Iterate over key-value pairs
    println!("All entries:");
    for (key, value) in &map {
        println!("  {}: {}", key, value);
    }

    println!("Size: {}", map.len());
}

```

(continues on next page)

(continued from previous page)

}

HashMap Entry API

The entry API is one of Rust's most powerful collection features. It allows you to inspect and modify map entries in a single lookup, avoiding the double-lookup pattern common in C++. This is particularly useful for counting, caching, and accumulation patterns:

C++:

```
#include <iostream>
#include <string>
#include <unordered_map>

int main() {
    std::unordered_map<std::string, int> word_count;
    std::string words[] = {"apple", "banana", "apple", "cherry", "banana", "apple"};

    // Count words - requires lookup then insert/update
    for (const auto& word : words) {
        auto it = word_count.find(word);
        if (it != word_count.end()) {
            it->second++;
        } else {
            word_count[word] = 1;
        }
    }

    // Or use operator[] which default-constructs if missing
    std::unordered_map<std::string, int> word_count2;
    for (const auto& word : words) {
        word_count2[word]++; // relies on int default being 0
    }

    for (const auto& [word, count] : word_count) {
        std::cout << word << ": " << count << "\n";
    }

    return 0;
}
```

Rust:

```
use std::collections::HashMap;

fn main() {
    let mut word_count = HashMap::new();
    let words = ["apple", "banana", "apple", "cherry", "banana", "apple"];

    // Count words using entry API - single lookup!
    for word in &words {
        // or_insert returns &mut V, allowing in-place modification
    }
}
```

(continues on next page)

(continued from previous page)

```

        *word_count.entry(*word).or_insert(0) += 1;
    }

    // Entry API variants:
    let mut map: HashMap<&str, Vec<i32>> = HashMap::new();

    // or_insert: insert value if missing
    map.entry("numbers").or_insert(vec![]).push(1);

    // or_insert_with: insert computed value if missing (lazy)
    map.entry("computed").or_insert_with(|| {
        println!("Computing default...");
        vec![42]
    });

    // or_default: insert Default::default() if missing
    map.entry("default").or_default().push(100);

    // Print word counts
    println!("Word counts:");
    for (word, count) in &word_count {
        println!(" {}: {}", word, count);
    }
}

```

BTreeMap (Ordered Map)

BTreeMap<K, V> is Rust's ordered map, equivalent to C++'s `std::map`. It keeps keys in sorted order and provides $O(\log n)$ operations. Unlike HashMap, keys only need to implement Ord (not Hash). BTreeMap also supports efficient range queries, which are not available with hash-based maps:

C++:

```

#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<int, std::string> map;

    // Insert in arbitrary order
    map[3] = "three";
    map[1] = "one";
    map[2] = "two";
    map[5] = "five";
    map[4] = "four";

    // Iteration is in sorted key order
    std::cout << "All entries (sorted):\n";
    for (const auto& [key, value] : map) {
        std::cout << "  " << key << ": " << value << "\n";
    }
}

```

(continues on next page)

(continued from previous page)

```
// Range query using lower_bound/upper_bound
std::cout << "Range [2, 4):\n";
for (auto it = map.lower_bound(2); it != map.lower_bound(4); ++it) {
    std::cout << "  " << it->first << ": " << it->second << "\n";
}

return 0;
}
```

Rust:

```
use std::collections::BTreeMap;

fn main() {
    let mut map = BTreeMap::new();

    // Insert in arbitrary order
    map.insert(3, "three");
    map.insert(1, "one");
    map.insert(2, "two");
    map.insert(5, "five");
    map.insert(4, "four");

    // Iteration is in sorted key order
    println!("All entries (sorted):");
    for (k, v) in &map {
        println!(" {}: {}", k, v);
    }

    // Range queries - more ergonomic than C++
    println!("Range [2, 4):");
    for (k, v) in map.range(2..4) {
        println!(" {}: {}", k, v);
    }

    // Other range variants
    println!("Keys >= 3:");
    for (k, v) in map.range(3..) {
        println!(" {}: {}", k, v);
    }
}
```

HashSet

HashSet<T> is Rust's hash set implementation, equivalent to C++'s `std::unordered_set`. It stores unique values and provides O(1) average-case lookup, insertion, and removal. Rust's HashSet provides convenient methods for set-theoretic operations like union, intersection, and difference:

C++:

```
#include <iostream>
#include <unordered_set>
```

(continues on next page)

(continued from previous page)

```

#include <algorithm>
#include <iterator>

int main() {
    std::unordered_set<int> set = {1, 2, 3};

    // Insert and check membership
    set.insert(4);
    set.insert(2); // duplicate, ignored
    bool has_two = set.count(2) > 0;

    std::cout << "Contains 2: " << (has_two ? "yes" : "no") << "\n";
    std::cout << "Size: " << set.size() << "\n";

    // Set operations require manual implementation or std::set_*
    std::unordered_set<int> other = {3, 4, 5};

    // No built-in union/intersection for unordered_set
    // Must convert to sorted containers or implement manually

    return 0;
}

```

Rust:

```

use std::collections::HashSet;

fn main() {
    let mut set: HashSet<i32> = [1, 2, 3].into();

    // Insert and check membership
    set.insert(4);
    set.insert(2); // duplicate, returns false
    let has_two = set.contains(&2);

    println!("Contains 2: {}", has_two);
    println!("Size: {}", set.len());

    // Iterate (order is arbitrary)
    print!("Elements: ");
    for x in &set {
        print!("{}", x);
    }
    println!();
}

```

Set Operations

Rust provides built-in methods for common set operations. These methods return iterators, allowing lazy evaluation and chaining. The following example demonstrates union, intersection, difference, and symmetric difference:

C++:

```

#include <algorithm>
#include <iostream>
#include <set> // using ordered set for set operations
#include <iterator>

int main() {
    std::set<int> a = {1, 2, 3};
    std::set<int> b = {2, 3, 4};

    // Union
    std::set<int> union_set;
    std::set_union(a.begin(), a.end(), b.begin(), b.end(),
                  std::inserter(union_set, union_set.begin()));

    // Intersection
    std::set<int> intersection;
    std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
                          std::inserter(intersection, intersection.begin()));

    // Difference (a - b)
    std::set<int> difference;
    std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
                       std::inserter(difference, difference.begin()));

    // Symmetric difference
    std::set<int> sym_diff;
    std::set_symmetric_difference(a.begin(), a.end(), b.begin(), b.end(),
                                 std::inserter(sym_diff, sym_diff.begin()));

    auto print_set = [](const std::string& name, const std::set<int>& s) {
        std::cout << name << ": {";
        for (auto it = s.begin(); it != s.end(); ++it) {
            if (it != s.begin()) std::cout << ", ";
            std::cout << *it;
        }
        std::cout << "}\n";
    };

    print_set("Union", union_set);
    print_set("Intersection", intersection);
    print_set("Difference", difference);
    print_set("Symmetric Diff", sym_diff);

    return 0;
}

```

Rust:

```

use std::collections::HashSet;

fn main() {
    let a: HashSet<i32> = [1, 2, 3].into();
    let b: HashSet<i32> = [2, 3, 4].into();
}

```

(continues on next page)

(continued from previous page)

```

// Set operations return iterators - collect into HashSet
let union_set: HashSet<_> = a.union(&b).cloned().collect();
let intersection: HashSet<_> = a.intersection(&b).cloned().collect();
let difference: HashSet<_> = a.difference(&b).cloned().collect();
let sym_diff: HashSet<_> = a.symmetric_difference(&b).cloned().collect();

println!("Union: {:?}", union_set);           // {1, 2, 3, 4}
println!("Intersection: {:?}", intersection);  // {2, 3}
println!("Difference: {:?}", difference);      // {1}
println!("Symmetric Diff: {:?}", sym_diff);   // {1, 4}

// Predicates for set relationships
let subset: HashSet<i32> = [2, 3].into();
println!("Is {{2,3}} subset of a? {}", subset.is_subset(&a));
println!("Are a and b disjoint? {}", a.is_disjoint(&b));
}

```

VecDeque (Double-ended Queue)

VecDeque<T> is Rust's double-ended queue, equivalent to C++'s `std::deque`. It provides $O(1)$ push and pop operations at both ends, making it ideal for queue and sliding window implementations. Unlike Vec, it uses a ring buffer internally:

C++:

```

#include <deque>
#include <iostream>

int main() {
    std::deque<int> dq = {2, 3, 4};

    // Push to both ends
    dq.push_front(1);
    dq.push_back(5);

    // Pop from both ends
    dq.pop_front();
    dq.pop_back();

    // Access elements
    std::cout << "Front: " << dq.front() << "\n";
    std::cout << "Back: " << dq.back() << "\n";

    // Random access
    std::cout << "Element at index 1: " << dq[1] << "\n";

    // Iterate
    std::cout << "Elements: ";
    for (int x : dq) {
        std::cout << x << " ";
    }
}

```

(continues on next page)

(continued from previous page)

```
std::cout << "\n";

return 0;
}
```

Rust:

```
use std::collections::VecDeque;

fn main() {
    let mut dq = VecDeque::from([2, 3, 4]);

    // Push to both ends
    dq.push_front(1);
    dq.push_back(5);

    // Pop from both ends - returns Option<T>
    let front = dq.pop_front(); // Some(1)
    let back = dq.pop_back();   // Some(5)
    println!("Popped front: {:?}", back: {:?}", front, back);

    // Access elements
    println!("Front: {:?}", dq.front()); // Option<&T>
    println!("Back: {:?}", dq.back());

    // Random access
    println!("Element at index 1: {:?}", dq.get(1));

    // Iterate
    print!("Elements: ");
    for x in &dq {
        print!("{}", x);
    }
    println();
}
```

Iterator Invalidation

One of Rust's most significant safety improvements over C++ is compile-time prevention of iterator invalidation. In C++, modifying a collection while iterating over it leads to undefined behavior that may silently corrupt data or crash. Rust's borrow checker makes this a compile-time error:

C++ (compiles but has undefined behavior):

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // DANGEROUS: modifying vector while iterating
    for (auto it = v.begin(); it != v.end(); ++it) {
        if (*it == 3) {
```

(continues on next page)

(continued from previous page)

```

    v.erase(it); // iterator invalidated! UB!
    // Even "fixing" with: it = v.erase(it); --it;
    // is error-prone
}
}

// This might crash, print garbage, or appear to work
for (int x : v) {
    std::cout << x << " ";
}

return 0;
}

```

Rust (compile error prevents the bug):

```

fn main() {
    let mut v = vec![1, 2, 3, 4, 5];

    // This won't compile - borrow checker prevents it
    // for x in &v {
    //     if *x == 3 {
    //         v.remove(2); // error: cannot borrow `v` as mutable
    //     }               // because it's borrowed as immutable
    // }

    // Correct approach 1: use retain (most idiomatic)
    v.retain(|&x| x != 3);
    println!("After retain: {:?}", v);

    // Correct approach 2: collect indices first, remove in reverse
    let mut v2 = vec![1, 2, 3, 4, 5];
    let indices: Vec<_> = v2.iter()
        .enumerate()
        .filter(|(_, &x)| x == 3)
        .map(|(i, _)| i)
        .collect();
    for i in indices.into_iter().rev() {
        v2.remove(i);
    }
    println!("After index removal: {:?}", v2);

    // Correct approach 3: drain_filter (nightly) or filter + collect
    let v3 = vec![1, 2, 3, 4, 5];
    let v3: Vec<_> = v3.into_iter().filter(|&x| x != 3).collect();
    println!("After filter: {:?}", v3);
}

```

See Also

- *Iterators* - Iterator adapters and patterns
- *Rust Basics* - Ownership with collections

1.8.6 Iterators

Table of Contents

- *Iterators*
 - *Basic Iteration*
 - *Three Iterator Types*
 - *Iterator Adapters*
 - * *map - Transform Elements*
 - * *filter - Keep Matching Elements*
 - * *filter_map - Filter and Transform*
 - * *take and skip*
 - * *enumerate - Add Indices*
 - * *zip - Combine Iterators*
 - * *chain - Concatenate Iterators*
 - * *flatten - Flatten Nested Iterators*
 - *Consuming Adapters*
 - * *collect - Gather into Collection*
 - * *fold and reduce - Accumulate Values*
 - * *sum and product*
 - * *find and position*
 - * *any and all*
 - * *count, min, and max*
 - *C++ Comparison*
 - * *C++ STL Algorithms vs Rust Iterators*
 - * *C++20 Ranges*
 - *Creating Custom Iterators*
 - *See Also*

Source

`src/rust/iterators`

Rust iterators are lazy and chainable, similar to C++20 ranges. Operations like `map` and `filter` don't execute until you consume the iterator (e.g., with `collect`). This lazy evaluation allows the compiler to fuse multiple operations into a single pass over the data, often generating code as efficient as hand-written loops. Unlike C++ STL algorithms which operate on iterator pairs, Rust iterators are single objects that carry their own state, making them easier to compose

and pass around. The iterator trait system also enables powerful abstractions - any type that implements `Iterator` automatically gains access to dozens of adapter methods.

Basic Iteration

Rust provides multiple ways to iterate over collections, each with different ownership semantics. The `for` loop syntax automatically calls `into_iter()` on the collection, but you can control borrowing behavior with `&` and `&mut`. This example shows the basic iteration patterns in both languages:

C++:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // Range-based for (copies by default, use & for reference)
    std::cout << "By value: ";
    for (int x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    // By reference (no copy)
    std::cout << "By reference: ";
    for (const int& x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    // Iterator-based (traditional style)
    std::cout << "Iterator-based: ";
    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n";

    return 0;
}
```

Rust:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // Borrowing iteration - v remains valid after loop
    print!("By reference: ");
    for x in &v {
        print!("{}", x); // x is &i32
    }
    println!();

    // v is still valid here
}
```

(continues on next page)

(continued from previous page)

```
println!("v still has {} elements", v.len());

// Consuming iteration - v is moved into the loop
print!("By value (consuming): ");
for x in v {
    print!("{}", x); // x is i32
}
println!();

// v is no longer valid here - would be compile error to use it
// println!("{}", v.len()); // error: borrow of moved value
}
```

Three Iterator Types

Rust collections provide three iterator methods that control ownership: `iter()` borrows elements immutably, `iter_mut()` borrows mutably, and `into_iter()` takes ownership. Understanding these is crucial for writing efficient Rust code. C++ achieves similar patterns through `const`/`non-const` iterators and move semantics:

C++:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};

    // Const iteration (like Rust's iter())
    std::cout << "Const iteration: ";
    for (auto it = v.cbegin(); it != v.cend(); ++it) {
        std::cout << *it << " ";
        // *it = 10; // error: cannot modify through const iterator
    }
    std::cout << "\n";

    // Mutable iteration (like Rust's iter_mut())
    std::cout << "Mutable iteration: ";
    for (auto it = v.begin(); it != v.end(); ++it) {
        *it += 10; // can modify
        std::cout << *it << " ";
    }
    std::cout << "\n";

    // Move iteration (like Rust's into_iter())
    std::vector<std::string> strings = {"hello", "world"};
    std::vector<std::string> moved;
    for (auto& s : strings) {
        moved.push_back(std::move(s)); // explicit move required
    }
    // strings still exists but elements are in moved-from state

    return 0;
}
```

Rust:

```
fn main() {
    let mut v = vec![1, 2, 3];

    // iter() - borrows immutably, yields &T
    print!("iter() yields &T: ");
    for x in v.iter() {
        print!("{}", x); // x is &i32
        // *x = 10; // error: cannot assign to immutable reference
    }
    println!();

    // iter_mut() - borrows mutably, yields &mut T
    print!("iter_mut() yields &mut T: ");
    for x in v.iter_mut() {
        *x += 10; // can modify through mutable reference
        print!("{}", x);
    }
    println!();

    // into_iter() - consumes collection, yields T
    print!("into_iter() yields T: ");
    for x in v.into_iter() {
        print!("{}", x); // x is i32, owns the value
    }
    println!();

    // v is no longer valid - ownership was transferred
    // println!("{}", v); // error: borrow of moved value
}
```

Iterator Adapters

Iterator adapters transform iterators without consuming them. They're lazy - no work happens until you call a consuming method like `collect()`. This enables efficient chaining of multiple operations. Each adapter below shows both C++ and Rust approaches.

map - Transform Elements

The map adapter applies a function to each element, producing a new iterator of transformed values. This is equivalent to C++'s `std::transform`:

C++:

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    std::vector<int> doubled;

    // std::transform requires output iterator
```

(continues on next page)

(continued from previous page)

```

std::transform(v.begin(), v.end(), std::back_inserter(doubled),
               [](int x) { return x * 2; });

std::cout << "Doubled: ";
for (int x : doubled) {
    std::cout << x << " "; // 2 4 6
}
std::cout << "\n";

return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3];

    // map is lazy - nothing happens until collect()
    let doubled: Vec<i32> = v.iter().map(|x| x * 2).collect();
    println!("Doubled: {:?}", doubled); // [2, 4, 6]

    // Can chain multiple maps
    let result: Vec<i32> = v.iter()
        .map(|x| x * 2)
        .map(|x| x + 1)
        .collect();
    println!("Doubled + 1: {:?}", result); // [3, 5, 7]
}

```

filter - Keep Matching Elements

The filter adapter keeps only elements that satisfy a predicate. This is equivalent to C++'s `std::copy_if`:

C++:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    std::vector<int> evens;

    // copy_if is the filtering algorithm
    std::copy_if(v.begin(), v.end(), std::back_inserter(evens),
                [](int x) { return x % 2 == 0; });

    std::cout << "Evens: ";
    for (int x : evens) {
        std::cout << x << " "; // 2 4
    }
    std::cout << "\n";
}

```

(continues on next page)

(continued from previous page)

```

return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // filter takes a predicate returning bool
    // Note: &&x because filter yields &T, and we're borrowing that
    let evens: Vec<i32> = v.iter()
        .filter(|&&x| x % 2 == 0)
        .cloned() // convert &i32 to i32
        .collect();
    println!("Evens: {:?}", evens); // [2, 4]

    // Alternative: use copied() instead of cloned() for Copy types
    let evens: Vec<i32> = v.iter()
        .filter(|&&x| x % 2 == 0)
        .copied()
        .collect();
    println!("Evens (copied): {:?}", evens);
}

```

filter_map - Filter and Transform

The `filter_map` adapter combines filtering and mapping in one step. It takes a function returning `Option<T>` - Some values are kept, None values are filtered out. This is particularly useful for parsing or fallible transformations:

C++:

```

#include <charconv>
#include <iostream>
#include <optional>
#include <string>
#include <vector>

std::optional<int> try_parse(const std::string& s) {
    int value;
    auto result = std::from_chars(s.data(), s.data() + s.size(), value);
    if (result.ec == std::errc()) {
        return value;
    }
    return std::nullopt;
}

int main() {
    std::vector<std::string> strings = {"1", "two", "3", "four", "5"};
    std::vector<int> numbers;

    for (const auto& s : strings) {

```

(continues on next page)

(continued from previous page)

```

    if (auto n = try_parse(s)) {
        numbers.push_back(*n);
    }
}

std::cout << "Parsed numbers: ";
for (int x : numbers) {
    std::cout << x << " "; // 1 3 5
}
std::cout << "\n";

return 0;
}

```

Rust:

```

fn main() {
    let strings = vec!["1", "two", "3", "four", "5"];

    // filter_map: return Some to keep, None to filter out
    let numbers: Vec<i32> = strings
        .iter()
        .filter_map(|s| s.parse().ok()) // parse returns Result, .ok() converts to
    ↪ Option
        .collect();
    println!("Parsed numbers: {:?}", numbers); // [1, 3, 5]

    // Equivalent using filter + map (less elegant)
    let numbers2: Vec<i32> = strings
        .iter()
        .filter(|s| s.parse::<i32>().is_ok())
        .map(|s| s.parse().unwrap())
        .collect();
    println!("Parsed (filter+map): {:?}", numbers2);
}

```

take and skip

The `take` adapter limits iteration to the first `N` elements, while `skip` skips the first `N` elements. These are useful for pagination and windowing:

C++:

```

#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // Take first 3 (manual loop or use ranges in C++20)
    std::cout << "First 3: ";
    for (size_t i = 0; i < 3 && i < v.size(); ++i) {

```

(continues on next page)

(continued from previous page)

```

    std::cout << v[i] << " ";
}
std::cout << "\n";

// Skip first 2
std::cout << "Skip 2: ";
for (size_t i = 2; i < v.size(); ++i) {
    std::cout << v[i] << " ";
}
std::cout << "\n";

return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // take: limit to first N elements
    let first_three: Vec<_> = v.iter().take(3).collect();
    println!("First 3: {:?}", first_three); // [1, 2, 3]

    // skip: skip first N elements
    let skip_two: Vec<_> = v.iter().skip(2).collect();
    println!("Skip 2: {:?}", skip_two); // [3, 4, 5]

    // Combine for pagination
    let page_size = 2;
    let page = 1; // 0-indexed
    let page_items: Vec<_> = v.iter()
        .skip(page * page_size)
        .take(page_size)
        .collect();
    println!("Page 1 (size 2): {:?}", page_items); // [3, 4]

    // take_while and skip_while use predicates
    let until_four: Vec<_> = v.iter().take_while(|&x| x < 4).collect();
    println!("Take while < 4: {:?}", until_four); // [1, 2, 3]
}

```

enumerate - Add Indices

The `enumerate` adapter pairs each element with its index. This is cleaner than maintaining a separate counter variable:

C++:

```

#include <iostream>
#include <vector>
#include <string>

int main() {

```

(continues on next page)

(continued from previous page)

```
std::vector<std::string> v = {"apple", "banana", "cherry"};

// Manual index tracking
size_t i = 0;
for (const auto& x : v) {
    std::cout << i << ": " << x << "\n";
    ++i;
}

// Or use traditional for loop
for (size_t i = 0; i < v.size(); ++i) {
    std::cout << i << ": " << v[i] << "\n";
}

return 0;
}
```

Rust:

```
fn main() {
    let v = vec!["apple", "banana", "cherry"];

    // enumerate yields (index, element) tuples
    for (i, x) in v.iter().enumerate() {
        println!("{i}: {x}");
    }

    // Can destructure in closures too
    let indexed: Vec<_> = v.iter()
        .enumerate()
        .map(|(i, s)| format!("{i}: {s}"))
        .collect();
    println!("Indexed: {:?}", indexed);
}
```

zip - Combine Iterators

The zip adapter combines two iterators into one that yields pairs. Iteration stops when either iterator is exhausted:

C++:

```
#include <iostream>
#include <vector>
#include <string>

int main() {
    std::vector<int> numbers = {1, 2, 3};
    std::vector<std::string> words = {"one", "two", "three"};

    // Manual parallel iteration
    for (size_t i = 0; i < numbers.size() && i < words.size(); ++i) {
        std::cout << numbers[i] << " -> " << words[i] << "\n";
    }
}
```

(continues on next page)

(continued from previous page)

```

}

return 0;
}

```

Rust:

```

fn main() {
    let numbers = vec![1, 2, 3];
    let words = vec!["one", "two", "three"];

    // zip combines two iterators into pairs
    for (n, w) in numbers.iter().zip(words.iter()) {
        println!("{}", n, w);
    }

    // Collect into vector of tuples
    let pairs: Vec<_> = numbers.iter().zip(words.iter()).collect();
    println!("Pairs: {:?}", pairs);

    // Unequal lengths: stops at shorter
    let short = vec![1, 2];
    let long = vec!["a", "b", "c", "d"];
    let zipped: Vec<_> = short.iter().zip(long.iter()).collect();
    println!("Unequal zip: {:?}", zipped); // [(1, "a"), (2, "b")]
}

```

chain - Concatenate Iterators

The chain adapter concatenates two iterators, yielding all elements from the first followed by all elements from the second:

C++:

```

#include <iostream>
#include <vector>

int main() {
    std::vector<int> a = {1, 2};
    std::vector<int> b = {3, 4};

    // Manual concatenation
    std::vector<int> combined;
    combined.insert(combined.end(), a.begin(), a.end());
    combined.insert(combined.end(), b.begin(), b.end());

    std::cout << "Combined: ";
    for (int x : combined) {
        std::cout << x << " ";
    }
    std::cout << "\n";
}

```

(continues on next page)

(continued from previous page)

```

return 0;
}

```

Rust:

```

fn main() {
    let a = vec![1, 2];
    let b = vec![3, 4];

    // chain concatenates iterators lazily
    let combined: Vec<_> = a.iter().chain(b.iter()).collect();
    println!("Combined: {:?}", combined); // [1, 2, 3, 4]

    // Can chain multiple iterators
    let c = vec![5, 6];
    let all: Vec<_> = a.iter()
        .chain(b.iter())
        .chain(c.iter())
        .collect();
    println!("All: {:?}", all); // [1, 2, 3, 4, 5, 6]
}

```

flatten - Flatten Nested Iterators

The `flatten` adapter flattens nested iterators (or collections) into a single iterator. This is useful for working with nested data structures:

C++:

```

#include <iostream>
#include <vector>

int main() {
    std::vector<std::vector<int>> nested = {{1, 2}, {3, 4}, {5}};

    // Manual flattening
    std::vector<int> flat;
    for (const auto& inner : nested) {
        for (int x : inner) {
            flat.push_back(x);
        }
    }

    std::cout << "Flattened: ";
    for (int x : flat) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    return 0;
}

```

Rust:

```
fn main() {
    let nested = vec![vec![1, 2], vec![3, 4], vec![5]];

    // flatten collapses one level of nesting
    let flat: Vec<_> = nested.iter().flatten().collect();
    println!("Flattened: {:?}", flat); // [1, 2, 3, 4, 5]

    // flat_map combines map + flatten (very common pattern)
    let words = vec!["hello", "world"];
    let chars: Vec<_> = words.iter()
        .flat_map(|s| s.chars())
        .collect();
    println!("All chars: {:?}", chars); // ['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']
}
```

Consuming Adapters

Consuming adapters (also called “terminal operations”) consume the iterator and produce a final result. Unlike adapters like `map` and `filter`, these methods trigger actual iteration.

collect - Gather into Collection

The `collect` method consumes an iterator and gathers elements into a collection. The target type is inferred from context or specified with turbofish syntax:

C++:

```
#include <iostream>
#include <set>
#include <string>
#include <vector>

int main() {
    // Collecting into vector (manual)
    std::vector<int> v;
    for (int i = 1; i <= 5; ++i) {
        v.push_back(i);
    }

    // Collecting into set
    std::set<int> s(v.begin(), v.end());

    // Collecting chars into string
    std::vector<char> chars = {'h', 'e', 'l', 'l', 'o'};
    std::string str(chars.begin(), chars.end());

    std::cout << "String: " << str << "\n";

    return 0;
}
```

Rust:

```

use std::collections::HashSet;

fn main() {
    // Collect range into Vec
    let v: Vec<i32> = (1..=5).collect();
    println!("Vec: {:?}", v);

    // Collect into HashSet (removes duplicates)
    let set: HashSet<i32> = vec![1, 2, 2, 3, 3, 3].into_iter().collect();
    println!("Set: {:?}", set);

    // Collect chars into String
    let s: String = ['h', 'e', 'l', 'l', 'o'].iter().collect();
    println!("String: {}", s);

    // Turbofish syntax when type can't be inferred
    let v = (1..=5).collect::<Vec<_>>();
    println!("Turbofish: {:?}", v);
}

```

fold and reduce - Accumulate Values

The fold method accumulates values with an initial accumulator, while reduce uses the first element as the initial value. These are equivalent to C++'s `std::accumulate` and `std::reduce`:

C++:

```

#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // accumulate with initial value (like fold)
    int sum = std::accumulate(v.begin(), v.end(), 0,
                             [](int acc, int x) { return acc + x; });
    std::cout << "Sum: " << sum << "\n"; // 15

    // reduce without initial value (C++17)
    // Note: std::reduce may reorder operations for parallelism
    int product = std::reduce(v.begin(), v.end(), 1, std::multiplies<int>());
    std::cout << "Product: " << product << "\n"; // 120

    return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // fold: requires initial value, always succeeds
}

```

(continues on next page)

(continued from previous page)

```

let sum = v.iter().fold(0, |acc, x| acc + x);
println!("Sum (fold): {}", sum); // 15

// reduce: uses first element as initial, returns Option
let product = v.iter().copied().reduce(|acc, x| acc * x);
println!("Product (reduce): {:?}", product); // Some(120)

// reduce on empty iterator returns None
let empty: Vec<i32> = vec![];
let result = empty.iter().copied().reduce(|acc, x| acc + x);
println!("Empty reduce: {:?}", result); // None

// Building a string with fold
let words = vec!["hello", "world"];
let sentence = words.iter().fold(String::new(), |mut acc, &word| {
    if !acc.is_empty() {
        acc.push(' ');
    }
    acc.push_str(word);
    acc
});
println!("Sentence: {}", sentence); // "hello world"
}

```

sum and product

The sum and product methods are specialized folds for numeric types:

C++:

```

#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    int sum = std::accumulate(v.begin(), v.end(), 0);
    int product = std::accumulate(v.begin(), v.end(), 1, std::multiplies<int>());

    std::cout << "Sum: " << sum << "\n"; // 15
    std::cout << "Product: " << product << "\n"; // 120

    return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // sum and product require type annotation

```

(continues on next page)

(continued from previous page)

```

let sum: i32 = v.iter().sum();
let product: i32 = v.iter().product();

println!("Sum: {}", sum);          // 15
println!("Product: {}", product); // 120

// Works with floating point too
let floats = vec![1.5, 2.5, 3.0];
let sum: f64 = floats.iter().sum();
println!("Float sum: {}", sum); // 7.0
}

```

find and position

The `find` method returns the first element matching a predicate, while `position` returns its index:

C++:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // find returns iterator
    auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 3; });
    if (it != v.end()) {
        std::cout << "Found: " << *it << "\n"; // 4
    }

    // position requires distance calculation
    auto pos = std::distance(v.begin(), it);
    std::cout << "Position: " << pos << "\n"; // 3

    return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // find returns Option<T>
    let found = v.iter().find(|&x| x > 3);
    println!("Found: {:?}", found); // Some(&4)

    // position returns Option<usize>
    let pos = v.iter().position(|&x| x > 3);
    println!("Position: {:?}", pos); // Some(3)

    // Not found returns None
}

```

(continues on next page)

(continued from previous page)

```

let not_found = v.iter().find(|&x| x > 10);
println!("Not found: {:?}", not_found); // None

// find_map combines find and map
let strings = vec!["1", "two", "3"];
let first_num: Option<i32> = strings.iter().find_map(|s| s.parse().ok());
println!("First parseable: {:?}", first_num); // Some(1)
}

```

any and all

The `any` method returns true if any element matches, while `all` returns true if all elements match. These short-circuit on the first decisive result:

C++:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    bool has_even = std::any_of(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
    bool all_positive = std::all_of(v.begin(), v.end(), [](int x) { return x > 0; });
    bool none_negative = std::none_of(v.begin(), v.end(), [](int x) { return x < 0; });

    std::cout << "Has even: " << (has_even ? "yes" : "no") << "\n";
    std::cout << "All positive: " << (all_positive ? "yes" : "no") << "\n";
    std::cout << "None negative: " << (none_negative ? "yes" : "no") << "\n";

    return 0;
}

```

Rust:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // any: true if any element matches
    let has_even = v.iter().any(|&x| x % 2 == 0);
    println!("Has even: {}", has_even); // true

    // all: true if all elements match
    let all_positive = v.iter().all(|&x| x > 0);
    println!("All positive: {}", all_positive); // true

    // Short-circuit behavior
    let v2 = vec![1, 2, 3, 4, 5];
    let found_early = v2.iter().any(|&x| {
        println!("Checking {}", x);
        x == 2
    });
}

```

(continues on next page)

(continued from previous page)

```
});
// Only prints "Checking 1" and "Checking 2"
println!("Found: {}", found_early);
}
```

count, min, and max

These methods provide basic statistics about iterator elements:

C++:

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

    size_t count = v.size();
    auto [min_it, max_it] = std::minmax_element(v.begin(), v.end());

    std::cout << "Count: " << count << "\n";
    std::cout << "Min: " << *min_it << "\n";
    std::cout << "Max: " << *max_it << "\n";

    return 0;
}
```

Rust:

```
fn main() {
    let v = vec![3, 1, 4, 1, 5, 9, 2, 6];

    // count consumes the iterator
    let count = v.iter().count();
    println!("Count: {}", count); // 8

    // min and max return Option (None for empty iterators)
    let min = v.iter().min();
    let max = v.iter().max();
    println!("Min: {:?}", min); // Some(&1)
    println!("Max: {:?}", max); // Some(&9)

    // min_by and max_by for custom comparison
    let words = vec!["apple", "pie", "extraordinary"];
    let longest = words.iter().max_by_key(|s| s.len());
    println!("Longest: {:?}", longest); // Some(&"extraordinary")
}
```

C++ Comparison

This section provides a comprehensive comparison between C++ STL algorithms and Rust iterator methods. C++ traditionally uses algorithm functions that take iterator pairs, while Rust uses method chaining on iterator objects.

C++ STL Algorithms vs Rust Iterators

The following example shows a complete transformation pipeline in both languages. Notice how Rust's method chaining is more concise and the lazy evaluation allows the compiler to optimize the entire chain:

C++ with algorithms:

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Transform: double each element
    std::vector<int> doubled;
    std::transform(v.begin(), v.end(), std::back_inserter(doubled),
        [](int x) { return x * 2; });

    // Filter: keep only evens (from original)
    std::vector<int> evens;
    std::copy_if(v.begin(), v.end(), std::back_inserter(evens),
        [](int x) { return x % 2 == 0; });

    // Sum
    int sum = std::accumulate(v.begin(), v.end(), 0);

    // Chained operations require intermediate vectors
    std::vector<int> temp;
    std::copy_if(v.begin(), v.end(), std::back_inserter(temp),
        [](int x) { return x % 2 == 0; });
    std::vector<int> result;
    std::transform(temp.begin(), temp.end(), std::back_inserter(result),
        [](int x) { return x * 2; });

    std::cout << "Doubled: ";
    for (int x : doubled) std::cout << x << " ";
    std::cout << "\nEvens: ";
    for (int x : evens) std::cout << x << " ";
    std::cout << "\nSum: " << sum;
    std::cout << "\nFiltered+Doubled: ";
    for (int x : result) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Rust equivalent:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // Transform: double each element
    let doubled: Vec<_> = v.iter().map(|x| x * 2).collect();

    // Filter: keep only evens
    let evens: Vec<_> = v.iter().filter(|&&x| x % 2 == 0).collect();

    // Sum
    let sum: i32 = v.iter().sum();

    // Chained operations - no intermediate allocations!
    let result: Vec<_> = v.iter()
        .filter(|&&x| x % 2 == 0)
        .map(|x| x * 2)
        .collect();

    println!("Doubled: {:?}", doubled);
    println!("Evens: {:?}", evens);
    println!("Sum: {}", sum);
    println!("Filtered+Doubled: {:?}", result);
}
```

C++20 Ranges

C++20 introduced ranges, which provide a more Rust-like experience with lazy evaluation and method chaining via the pipe operator:

C++20 Ranges:

```
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Pipe syntax similar to Rust's method chaining
    auto result = v
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x * 2; });

    // Lazy evaluation - nothing computed until iteration
    std::cout << "Filtered and doubled evens: ";
    for (int x : result) {
        std::cout << x << " "; // 4 8 12 16 20
    }
    std::cout << "\n";

    // Take first N elements
    auto first_three = v | std::views::take(3);
    std::cout << "First three: ";
```

(continues on next page)

(continued from previous page)

```

for (int x : first_three) {
    std::cout << x << " ";
}
std::cout << "\n";

return 0;
}

```

Rust equivalent:

```

fn main() {
    let v = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // Method chaining (Rust's native style)
    let result: Vec<_> = v.iter()
        .filter(|&&x| x % 2 == 0)
        .map(|x| x * 2)
        .collect();

    println!("Filtered and doubled evens: {:?}", result); // [4, 8, 12, 16, 20]

    // Take first N elements
    let first_three: Vec<_> = v.iter().take(3).collect();
    println!("First three: {:?}", first_three);
}

```

Creating Custom Iterators

You can create custom iterators by implementing the `Iterator` trait. This requires defining the `Item` associated type and the `next` method. Once implemented, your type automatically gains access to all iterator adapter methods.

The following example creates a counter iterator that yields numbers from 1 to a maximum value:

C++:

```

#include <iostream>
#include <iterator>

class Counter {
public:
    using iterator_category = std::input_iterator_tag;
    using value_type = int;
    using difference_type = std::ptrdiff_t;
    using pointer = int*;
    using reference = int&;

private:
    int count_;
    int max_;

public:
    Counter(int max) : count_(0), max_(max) {}
}

```

(continues on next page)

(continued from previous page)

```

// For end sentinel
static Counter end() {
    Counter c(0);
    c.count_ = c.max_ + 1;
    return c;
}

int operator*() const { return count_; }

Counter& operator++() {
    ++count_;
    return *this;
}

bool operator!=(const Counter& other) const {
    return count_ <= max_;
}
};

int main() {
    // Manual iteration
    int sum = 0;
    for (Counter c(5); c != Counter::end(); ++c) {
        if (*c > 0) { // skip 0
            sum += *c;
        }
    }
    std::cout << "Sum 1-5: " << sum << "\n"; // 15

    return 0;
}

```

Rust:

```

struct Counter {
    count: u32,
    max: u32,
}

impl Counter {
    fn new(max: u32) -> Self {
        Counter { count: 0, max }
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.count < self.max {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

        } else {
            None
        }
    }
}

fn main() {
    // All iterator methods are automatically available
    let sum: u32 = Counter::new(5).sum();
    println!("Sum 1-5: {}", sum); // 15

    // Can use any adapter
    let evens: Vec<_> = Counter::new(10)
        .filter(|&x| x % 2 == 0)
        .collect();
    println!("Evens 1-10: {:?}", evens); // [2, 4, 6, 8, 10]

    // Chaining with other iterators
    let doubled: Vec<_> = Counter::new(3)
        .map(|x| x * 2)
        .collect();
    println!("Doubled 1-3: {:?}", doubled); // [2, 4, 6]
}

```

See Also

- *Collections* - Collections that implement Iterator
- *Closures* - Closures used with iterator adapters

1.8.7 Traits and Generics

Table of Contents

- *Traits and Generics*
 - *Traits vs C++ Concepts/Interfaces*
 - * *Default Implementations*
 - *Generic Functions*
 - * *Multiple Trait Bounds*
 - *Generic Structs*
 - *Associated Types*
 - *Common Standard Traits*
 - * *Deriving Traits*
 - *Trait Objects (Dynamic Dispatch)*
 - * *Static vs Dynamic Dispatch*

- *Implementing External Traits*
- *See Also*

Source

src/rust/generics

Rust traits are the foundation of Rust's approach to polymorphism and code reuse. They define shared behavior that types can implement, similar to interfaces in other languages. Unlike C++ templates which use duck typing (any type that has the required operations will work), Rust generics require explicit trait bounds that are checked at compile time. This means you get clear error messages at the call site rather than deep inside template instantiation. Traits also enable Rust's powerful derive system, operator overloading, and the orphan rules that prevent conflicting implementations.

Traits vs C++ Concepts/Interfaces

Traits serve a similar purpose to C++ abstract classes and C++20 concepts, but with key differences. Unlike abstract classes, traits don't require inheritance and can be implemented for any type, including primitives and types from other crates (with restrictions). Unlike C++ concepts which are constraints on templates, Rust traits are first-class types that can be used for both static and dynamic dispatch.

The following example shows how to define a trait and implement it for a custom type. Note that Rust separates data (struct) from behavior (impl), unlike C++ where methods are typically defined inside the class:

C++ (abstract class):

```
#include <iostream>

// Abstract base class defines interface
class Printable {
public:
    virtual void print() const = 0;
    virtual ~Printable() = default;
};

// Concrete class inherits and implements
class Point : public Printable {
    int x_, y_;
public:
    Point(int x, int y) : x_(x), y_(y) {}

    void print() const override {
        std::cout << "(" << x_ << ", " << y_ << ")";
    }

    // Additional methods specific to Point
    int x() const { return x_; }
    int y() const { return y_; }
};

// Function accepting any Printable
void display(const Printable& p) {
    p.print();
    std::cout << "\n";
}
```

(continues on next page)

(continued from previous page)

```
int main() {
    Point p(3, 4);
    display(p); // (3, 4)
    return 0;
}
```

Rust:

```
// Trait defines shared behavior
trait Printable {
    fn print(&self);
}

// Struct defines data
struct Point {
    x: i32,
    y: i32,
}

// impl block implements trait for type
impl Printable for Point {
    fn print(&self) {
        println!("{}", self.x, self.y);
    }
}

// Additional methods in separate impl block
impl Point {
    fn new(x: i32, y: i32) -> Self {
        Point { x, y }
    }
}

// Function accepting any Printable (static dispatch)
fn display(p: &impl Printable) {
    p.print();
}

fn main() {
    let p = Point::new(3, 4);
    display(&p); // (3, 4)
}
```

Default Implementations

Traits can provide default implementations for methods. Types implementing the trait can use the default or override it. This is similar to non-pure virtual methods in C++ abstract classes:

C++:

```
#include <iostream>
```

(continues on next page)

(continued from previous page)

```

#include <string>

class Greet {
public:
    virtual std::string name() const = 0;

    // Default implementation using pure virtual method
    virtual void greet() const {
        std::cout << "Hello, " << name() << "!\n";
    }

    virtual ~Greet() = default;
};

class Person : public Greet {
    std::string name_;
public:
    Person(const std::string& name) : name_(name) {}

    std::string name() const override { return name_; }
    // greet() uses default implementation
};

int main() {
    Person p("Alice");
    p.greet(); // Hello, Alice!
    return 0;
}

```

Rust:

```

trait Greet {
    // Required method - must be implemented
    fn name(&self) -> &str;

    // Default implementation - can be overridden
    fn greet(&self) {
        println!("Hello, {}", self.name());
    }

    // Another default that calls other trait methods
    fn formal_greet(&self) {
        println!("Good day, {}. How do you do?", self.name());
    }
}

struct Person {
    name: String,
}

impl Greet for Person {
    fn name(&self) -> &str {

```

(continues on next page)

(continued from previous page)

```

        &self.name
    }
    // greet() and formal_greet() use default implementations
}

struct Robot {
    id: u32,
}

impl Greet for Robot {
    fn name(&self) -> &str {
        "Robot"
    }

    // Override default implementation
    fn greet(&self) {
        println!("BEEP BOOP. Unit {} operational.", self.id);
    }
}

fn main() {
    let person = Person { name: String::from("Alice") };
    person.greet();           // Hello, Alice!
    person.formal_greet();    // Good day, Alice. How do you do?

    let robot = Robot { id: 42 };
    robot.greet();            // BEEP BOOP. Unit 42 operational.
    robot.formal_greet();    // Good day, Robot. How do you do?
}

```

Generic Functions

Generic functions work with any type that satisfies the specified trait bounds. Unlike C++ templates where constraints are implicit (duck typing), Rust requires explicit bounds that are checked at the function definition site:

C++ template:

```

#include <iostream>

// Unconstrained template - any type with operator> works
template<typename T>
T max_value(T a, T b) {
    return (a > b) ? a : b;
}

// With concepts (C++20) - explicit constraint
template<typename T>
concept Comparable = requires(T a, T b) {
    { a > b } -> std::convertible_to<bool>;
};

template<Comparable T>

```

(continues on next page)

(continued from previous page)

```
T max_concept(T a, T b) {
    return (a > b) ? a : b;
}
```

Rust:

```
// Trait bound required
fn max<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}

// Alternative: where clause
fn max_where<T>(a: T, b: T) -> T
where
    T: PartialOrd,
{
    if a > b { a } else { b }
}

// impl Trait syntax (simpler for single use)
fn max_impl(a: impl PartialOrd, b: impl PartialOrd) -> impl PartialOrd {
    // Note: a and b must be same type in practice
    if a > b { a } else { b }
}
```

Multiple Trait Bounds

When a generic type needs to satisfy multiple constraints, you can combine trait bounds with +. For complex bounds, the where clause provides better readability:

C++:

```
#include <concepts>
#include <iostream>

// Multiple constraints with concepts (C++20)
template<typename T>
concept DebugPrintable = requires(T t) {
    { std::cout << t }; // can be printed
};

template<typename T>
concept Cloneable = std::copyable<T>;

// Combining concepts
template<typename T>
requires DebugPrintable<T> && Cloneable<T>
void process(T value) {
    T copy = value;
    std::cout << "Processing: " << copy << "\n";
}
```

(continues on next page)

(continued from previous page)

```
int main() {
    process(42);
    process(std::string("hello"));
    return 0;
}
```

Rust:

```
use std::fmt::{Debug, Display};

// Multiple bounds with + syntax
fn print_both<T: Debug + Display>(value: T) {
    println!("Debug: {:?}", value);
    println!("Display: {}", value);
}

// Where clause for complex bounds (more readable)
fn complex_function<T, U>(t: T, u: U) -> String
where
    T: Debug + Clone + Default,
    U: Display + PartialOrd + Into<String>,
{
    let t_clone = t.clone();
    println!("Debug of t: {:?}", t_clone);
    u.into()
}

fn main() {
    // i32 implements both Debug and Display
    print_both(42);

    // String also implements both
    print_both(String::from("hello"));
}
```

Generic Structs

Generic structs allow you to define data structures that work with any type. You can add trait bounds to restrict which types can be used, and you can provide specialized implementations for specific types:

C++:

```
#include <iostream>

template<typename T>
struct Pair {
    T first;
    T second;

    Pair(T f, T s) : first(f), second(s) {}

    void print() const {
        std::cout << "(" << first << ", " << second << ")\n";
    }
}
```

(continues on next page)

(continued from previous page)

```

    }
};

// Specialization for int
template<>
struct Pair<int> {
    int first;
    int second;

    Pair(int f, int s) : first(f), second(s) {}

    int sum() const { return first + second; }

    void print() const {
        std::cout << "(" << first << ", " << second << ") sum=" << sum() << "\n";
    }
};

int main() {
    Pair<double> pd(1.5, 2.5);
    pd.print(); // (1.5, 2.5)

    Pair<int> pi(3, 4);
    pi.print(); // (3, 4) sum=7
    std::cout << "Sum: " << pi.sum() << "\n";

    return 0;
}

```

Rust:

```

use std::fmt::Display;

// Generic struct
struct Pair<T> {
    first: T,
    second: T,
}

// Methods for all Pair<T>
impl<T> Pair<T> {
    fn new(first: T, second: T) -> Self {
        Pair { first, second }
    }
}

// Methods only when T: Display
impl<T: Display> Pair<T> {
    fn print(&self) {
        println!("{}", self.first, self.second);
    }
}

```

(continues on next page)

(continued from previous page)

```
// Methods only for Pair<i32> (specialization)
impl Pair<i32> {
    fn sum(&self) -> i32 {
        self.first + self.second
    }
}

// Methods only when T supports addition
impl<T: std::ops::Add<Output = T> + Copy> Pair<T> {
    fn add(&self) -> T {
        self.first + self.second
    }
}

fn main() {
    let pd = Pair::new(1.5, 2.5);
    pd.print(); // (1.5, 2.5)
    println!("Add: {}", pd.add()); // 4.0

    let pi = Pair::new(3, 4);
    pi.print(); // (3, 4)
    println!("Sum: {}", pi.sum()); // 7 (only available for i32)
}
```

Associated Types

Associated types are type placeholders within traits that implementors must specify. They're similar to C++ template type aliases but are part of the trait definition. Associated types are preferred over generic traits when there should be only one implementation per type:

C++:

```
#include <iostream>
#include <vector>

// Template with associated type via typedef
template<typename Derived>
class Container {
public:
    // Associated type defined by derived class
    using Item = typename Derived::ItemType;

    virtual Item* get(size_t index) = 0;
    virtual size_t len() const = 0;
    virtual ~Container() = default;
};

class IntVec : public Container<IntVec> {
    std::vector<int> data_;
public:
    using ItemType = int;
}
```

(continues on next page)

(continued from previous page)

```

IntVec(std::initializer_list<int> init) : data_(init) {}

int* get(size_t index) override {
    return index < data_.size() ? &data_[index] : nullptr;
}

size_t len() const override { return data_.size(); }
};

int main() {
    IntVec v = {1, 2, 3, 4, 5};
    std::cout << "Length: " << v.len() << "\n";
    if (auto* item = v.get(2)) {
        std::cout << "Item at 2: " << *item << "\n";
    }
    return 0;
}

```

Rust:

```

// Trait with associated type
trait Container {
    type Item; // associated type - implementor specifies this

    fn get(&self, index: usize) -> Option<&Self::Item>;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool {
        self.len() == 0
    }
}

// Wrapper around Vec<i32>
struct IntVec(Vec<i32>);

impl Container for IntVec {
    type Item = i32; // specify the associated type

    fn get(&self, index: usize) -> Option<&i32> {
        self.0.get(index)
    }

    fn len(&self) -> usize {
        self.0.len()
    }
}

// Generic wrapper - associated type depends on T
struct GenericVec<T>(Vec<T>);

impl<T> Container for GenericVec<T> {
    type Item = T;
}

```

(continues on next page)

(continued from previous page)

```

fn get(&self, index: usize) -> Option<T> {
    self.0.get(index)
}

fn len(&self) -> usize {
    self.0.len()
}

fn main() {
    let v = IntVec(vec![1, 2, 3, 4, 5]);
    println!("Length: {}", v.len());
    if let Some(item) = v.get(2) {
        println!("Item at 2: {}", item);
    }

    let gv = GenericVec(vec!["a", "b", "c"]);
    println!("Generic length: {}", gv.len());
}

```

Common Standard Traits

Rust Trait	Purpose	C++ Equivalent
Clone	Explicit deep copy	Copy constructor
Copy	Implicit bitwise copy	Trivially copyable
Drop	Destructor	Destructor
Default	Default value	Default constructor
Debug	Debug formatting	operator<< (debug)
Display	User-facing formatting	operator<<
PartialEq	Equality comparison	operator==
Eq	Total equality	operator== (reflexive)
PartialOrd	Partial ordering	operator<
Ord	Total ordering	operator<=> (C++20)
Hash	Hashing	std::hash
From/Into	Type conversion	Conversion constructor
Iterator	Iteration	begin()/end()

Deriving Traits

```

#[derive(Debug, Clone, PartialEq, Eq, Hash, Default)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p1 = Point { x: 1, y: 2 };
    let p2 = p1.clone();
    println!("{:?}", p1);           // Debug

```

(continues on next page)

(continued from previous page)

```
println!("{}", p1 == p2);    // PartialEq
}
```

Trait Objects (Dynamic Dispatch)

C++ (virtual functions):

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

void print_area(const Shape& shape) {
    std::cout << shape.area();
}
```

Rust (dyn Trait):

```
trait Shape {
    fn area(&self) -> f64;
}

// Dynamic dispatch with trait object
fn print_area(shape: &dyn Shape) {
    println!("{}", shape.area());
}

// Or with Box for owned trait objects
fn print_area_boxed(shape: Box<dyn Shape>) {
    println!("{}", shape.area());
}
```

Static vs Dynamic Dispatch

```
trait Draw {
    fn draw(&self);
}

// Static dispatch (monomorphization, like C++ templates)
fn draw_static<T: Draw>(item: &T) {
    item.draw();
}

// Dynamic dispatch (vtable, like C++ virtual)
fn draw_dynamic(item: &dyn Draw) {
    item.draw();
}
```

Implementing External Traits

Rust's orphan rule: you can implement a trait for a type only if either the trait or the type is defined in your crate.

```
use std::fmt::Display;

struct Wrapper(Vec<String>);

// Can implement Display for our Wrapper type
impl Display for Wrapper {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}
```

See Also

- *Error Handling* - Result and Option traits
- *Iterators* - Iterator trait

1.8.8 Polymorphism

Table of Contents

- *Polymorphism*
 - *Trait Objects (Dynamic Dispatch)*
 - *Static vs Dynamic Dispatch*
 - *Enum-based Dispatch*
 - *Returning Trait Objects*
 - *Trait Object References (&dyn Trait)*
 - *Rc<RefCell<dyn Trait>> — Shared Mutable Trait Objects*
 - * *Rc<RefCell<Box<dyn Trait>>> — Shared Mutable Trait Objects with Box*
 - *Object Safety*
 - *See Also*

Source

[src/rust/polymorphism](#)

Rust achieves polymorphism without inheritance. Where C++ uses virtual functions and class hierarchies, Rust provides two main approaches: trait objects (dynamic dispatch) and enums (closed-set dispatch). Both avoid the fragile base class problem.

Trait Objects (Dynamic Dispatch)

Trait objects (&dyn Trait or Box<dyn Trait>) are Rust's equivalent of C++ virtual function calls. They use a vtable for runtime dispatch.

C++ (virtual):

```

#include <iostream>
#include <memory>
#include <vector>

class Shape {
public:
    virtual double area() const = 0;
    virtual const char* name() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius_;
public:
    Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159265 * radius_ * radius_; }
    const char* name() const override { return "Circle"; }
};

class Rectangle : public Shape {
    double w_, h_;
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
    const char* name() const override { return "Rectangle"; }
};

void print_area(const Shape& s) {
    std::cout << s.name() << ": " << s.area() << "\n";
}

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(3.0));
    shapes.push_back(std::make_unique<Rectangle>(4.0, 5.0));
    for (auto& s : shapes) print_area(*s);
}

```

Rust:

```

trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &str;
}

struct Circle { radius: f64 }
struct Rectangle { width: f64, height: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.radius * self.radius }
    fn name(&self) -> &str { "Circle" }
}

```

(continues on next page)

(continued from previous page)

```

impl Shape for Rectangle {
    fn area(&self) -> f64 { self.width * self.height }
    fn name(&self) -> &str { "Rectangle" }
}

fn print_area(shape: &dyn Shape) {
    println!("{}", shape.name(), shape.area());
}

fn main() {
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { radius: 3.0 }),
        Box::new(Rectangle { width: 4.0, height: 5.0 }),
    ];
    for s in &shapes {
        print_area(s.as_ref());
    }
}

```

Key differences from C++:

- No inheritance hierarchy — any type can implement any trait
- No virtual destructor needed — `Box<dyn Trait>` handles cleanup via `Drop`
- `dyn` keyword makes dynamic dispatch explicit (C++ hides it behind `virtual`)

Static vs Dynamic Dispatch

Rust lets you choose between static dispatch (monomorphization, like C++ templates) and dynamic dispatch (vtable, like C++ virtual) per call site.

```

// Static dispatch - compiler generates specialized code per type
// Equivalent to C++ templates: zero overhead, but larger binary
fn print_area_static<T: Shape>(shape: &T) {
    println!("{}", shape.name(), shape.area());
}

// Dynamic dispatch - single function, vtable lookup at runtime
// Equivalent to C++ virtual: smaller binary, slight runtime cost
fn print_area_dynamic(shape: &dyn Shape) {
    println!("{}", shape.name(), shape.area());
}

```

Aspect	Static (impl Trait / generics)	Dynamic (dyn Trait)
C++ equivalent	Templates	Virtual functions
Dispatch	Compile-time	Runtime (vtable)
Performance	Zero-cost, inlinable	Indirect call overhead
Binary size	Larger (monomorphized copies)	Smaller (single function)
Heterogeneous collections	No	Yes

Enum-based Dispatch

When the set of variants is known at compile time, enums provide a closed-set alternative to trait objects. This avoids heap allocation and vtable overhead.

C++ (variant):

```
#include <iostream>
#include <variant>
#include <string>

struct Dog { std::string name; };
struct Cat { std::string name; };

using Animal = std::variant<Dog, Cat>;

const char* speak(const Animal& a) {
    return std::visit([](auto& v) -> const char* {
        if constexpr (std::is_same_v<std::decay_t<decltype(v)>, Dog>) return "Woof!";
        else return "Meow!";
    }, a);
}
```

Rust:

```
enum Animal {
    Dog(String),
    Cat(String),
}

impl Animal {
    fn speak(&self) -> &str {
        match self {
            Animal::Dog(_) => "Woof!",
            Animal::Cat(_) => "Meow!",
        }
    }
}
```

Advantages over trait objects:

- Stack-allocated, no Box needed
- Exhaustive match — compiler warns if you miss a variant
- Better cache locality

Returning Trait Objects

Functions can return different concrete types via `Box<dyn Trait>`, similar to returning `std::unique_ptr<Base>` in C++.

```
fn make_shape(kind: &str) -> Box<dyn Shape> {
    match kind {
        "circle" => Box::new(Circle { radius: 5.0 }),
        _ => Box::new(Rectangle { width: 4.0, height: 3.0 }),
    }
}
```

(continues on next page)

(continued from previous page)

```
}
}
```

Trait Object References (&dyn Trait)

A &dyn Trait is a **fat pointer** — two machine words (16 bytes on 64-bit): one pointer to the data, one pointer to the vtable. No heap allocation is involved; it simply borrows an existing value.

This is the lightest way to do dynamic dispatch:

```
fn total_area(shapes: &[&dyn Shape]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

let c = Circle { radius: 1.0 };
let r = Rectangle { width: 2.0, height: 3.0 };
let refs: Vec<&dyn Shape> = vec! [&c, &r]; // no Box, no heap
println!("{}", total_area(&refs));
```

In C++, the equivalent is passing `const Shape&` — but C++ references are thin pointers (the vtable pointer lives inside the object). Rust's fat pointer keeps the vtable external, which is why `dyn` is needed to opt in.

Rc<RefCell<dyn Trait>> — Shared Mutable Trait Objects

When you need **shared ownership** and **interior mutability** with trait objects, combine `Rc` (reference counting) with `RefCell` (runtime borrow checking):

```
use std::cell::RefCell;
use std::rc::Rc;

trait Counter {
    fn increment(&mut self);
    fn count(&self) -> u32;
}

struct ClickCounter { clicks: u32 }

impl Counter for ClickCounter {
    fn increment(&mut self) { self.clicks += 1; }
    fn count(&self) -> u32 { self.clicks }
}

let counters: Vec<Rc<RefCell<dyn Counter>>> = vec![
    Rc::new(RefCell::new(ClickCounter { clicks: 0 })),
];

let shared = Rc::clone(&counters[0]); // second owner
shared.borrow_mut().increment();      // mutate through RefCell
counters[0].borrow_mut().increment();
assert_eq!(counters[0].borrow().count(), 2);
```

C++ equivalent: `std::shared_ptr<Shape>` — but C++ doesn't distinguish shared ownership from mutability. Rust forces you to be explicit:

Rust	C++ equivalent	Use case
<code>Box<dyn Trait></code>	<code>unique_ptr<Base></code>	Single owner, heap-allocated
<code>&dyn Trait</code>	<code>const Base&</code>	Borrowed reference, no allocation
<code>Rc<dyn Trait></code>	<code>shared_ptr<const Base></code>	Shared ownership, immutable
<code>Rc<RefCell<dyn Trait>></code>	<code>shared_ptr<Base></code>	Shared ownership, mutable at runtime

Note

For multithreaded code, replace `Rc` with `Arc` and `RefCell` with `Mutex` or `RwLock`: `Arc<Mutex<dyn Trait + Send>>`.

Rc<RefCell<Box<dyn Trait>>> — Shared Mutable Trait Objects with Box

You may see a `Box` inside the `RefCell`. This pattern appears in real codebases (e.g. [this example](#)):

```
type SharedLogger = Rc<RefCell<Box<dyn Logger>>>;
```

Why the extra `Box`? Each layer serves a distinct purpose:

```
Rc<          -- shared ownership (multiple owners hold a clone)
  RefCell<   -- interior mutability (borrow checked at runtime)
    Box<     -- heap-allocate the trait object (dyn Trait is unsized)
      dyn Logger
    >
  >
>
```

On modern Rust (2021+), `Rc<RefCell<dyn Trait>>` works directly because `Rc` can hold unsized types via `CoerceUnsized`. The `Box` layer is still common because:

- Factory functions naturally return `Box<dyn Trait>` which slots right in
- Older codebases (pre-2021 edition) required it

```
use std::cell::RefCell;
use std::rc::Rc;

trait Logger {
    fn log(&mut self, msg: &str);
    fn entries(&self) -> &[String];
}

struct ConsoleLogger { logs: Vec<String> }

impl Logger for ConsoleLogger {
    fn log(&mut self, msg: &str) { self.logs.push(msg.to_string()); }
    fn entries(&self) -> &[String] { &self.logs }
}

type SharedLogger = Rc<RefCell<Box<dyn Logger>>>;

fn create_logger() -> SharedLogger {
```

(continues on next page)

(continued from previous page)

```

let logger: Box<dyn Logger> = Box::new(ConsoleLogger { logs: vec![] });
Rc::new(RefCell::new(logger))
}

let logger = create_logger();
let writer = Rc::clone(&logger); // same type, refcount = 2
let reader = Rc::clone(&logger); // same type, refcount = 3

// All three are Rc<RefCell<Box<dyn Logger>>>.
// Rc::clone does NOT unwrap a layer - it bumps the reference count.
// RefCell::borrow_mut() provides &mut access checked at runtime.
writer.borrow_mut().log("hello");
reader.borrow_mut().log("world");
assert_eq!(logger.borrow().entries().len(), 2);

```

How `borrow_mut()` provides mutability — the `Box<dyn Logger>` itself is not mutable. `RefCell` is what enables mutation through a shared reference:

```

writer // Rc<RefCell<Box<dyn Logger>>>
    .borrow_mut() // RefCell -> RefMut<Box<dyn Logger>>
    .log("hello") // auto-deref: &mut Box<dyn Logger> -> &mut dyn_
↳ Logger

```

`Rc::clone` does **not** unwrap a layer. All clones have the same type (`Rc<RefCell<Box<dyn Logger>>>`). It only increments the reference count.

Object Safety

Not all traits can be used as trait objects. A trait is object-safe if:

- No methods return `Self`
- No methods have generic type parameters
- All methods have a receiver (`&self`, `&mut self`, `self`, etc.)

```

// Object-safe - can use as `dyn Drawable`
trait Drawable {
    fn draw(&self);
}

// NOT object-safe - returns Self
trait Clonable {
    fn clone(&self) -> Self;
}

// NOT object-safe - generic method
trait Converter {
    fn convert<T>(&self) -> T;
}

```

The `Clone` trait in `std` is not object-safe, which is why you cannot write `Box<dyn Clone>`. Use workarounds like a helper trait when needed.

See Also

- *Traits and Generics* - Trait definitions, bounds, and deriving
- *Smart Pointers* - Box, Rc, Arc for trait object storage
- *Casting* - Type conversions and Any trait

1.8.9 Casting

Table of Contents

- *Casting*
 - *Casting Comparison*
 - *Primitive Casting with as*
 - * *as Casting Rules*
 - *From and Into Traits*
 - * *Common From Implementations*
 - *TryFrom and TryInto*
 - * *Custom TryFrom*
 - *String Conversions*
 - *Deref Coercion*
 - *AsRef and AsMut*
 - *Unsafe Transmute*
 - *See Also*

Source

`src/rust/casting`

Rust provides several mechanisms for type conversion, each with different safety guarantees. Unlike C++, Rust has no implicit numeric conversions - every conversion must be explicit. This prevents subtle bugs from unintended narrowing or widening conversions. Rust's type conversion system is built around traits (`From`, `Into`, `TryFrom`, `TryInto`) that provide a consistent interface for both infallible and fallible conversions. The `as` keyword is reserved for primitive type casts where the conversion rules are well-defined by the language.

Casting Comparison

The following table maps C++ casting mechanisms to their Rust equivalents. Note that Rust deliberately omits some C++ casts - `dynamic_cast` isn't needed because Rust uses trait objects with explicit `dyn` syntax, and `reinterpret_cast` requires unsafe code in Rust:

C++	Rust
<code>static_cast<T></code>	<code>as T</code> (primitives)
Conversion constructor	<code>From<T></code> / <code>Into<T></code>
<code>explicit</code> conversion	<code>TryFrom<T></code>
<code>dynamic_cast<T></code>	No direct equivalent
<code>reinterpret_cast<T></code>	<code>transmute</code> (unsafe)

Primitive Casting with as

The `as` keyword performs primitive type conversions similar to C++'s `static_cast`. However, unlike C++, Rust never performs implicit numeric conversions - you must always use `as` explicitly. This example shows basic numeric conversions:

C++:

```
#include <iostream>

int main() {
    int i = 42;

    // Explicit casts
    double d = static_cast<double>(i);
    char c = static_cast<char>(i);

    // Implicit conversions (allowed in C++, can be surprising)
    double d2 = i;           // implicit widening
    short s = i;             // implicit narrowing (may warn)
    unsigned int u = -1;     // implicit sign conversion (dangerous!)

    std::cout << "d: " << d << ", c: " << static_cast<int>(c) << "\n";
    std::cout << "d2: " << d2 << ", s: " << s << ", u: " << u << "\n";

    return 0;
}
```

Rust:

```
fn main() {
    let i: i32 = 42;

    // Explicit casts required
    let d: f64 = i as f64;
    let c: u8 = i as u8;

    // No implicit conversions - these won't compile:
    // let d2: f64 = i;           // error: expected f64, found i32
    // let s: i16 = i;           // error: expected i16, found i32
    // let u: u32 = -1_i32;      // error: expected u32, found i32

    // Must be explicit about everything
    let d2: f64 = i as f64;
    let s: i16 = i as i16;
    let u: u32 = (-1_i32) as u32; // wraps to u32::MAX

    println!("d: {}, c: {}", d, c);
    println!("d2: {}, s: {}, u: {}", d2, s, u);
}
```

as Casting Rules

The `as` keyword follows specific rules for different conversion types. Numeric conversions may truncate or wrap, float-to-int truncates toward zero, and pointer casts are allowed between compatible types. This example demonstrates the various `as` casting behaviors:

C++:

```
#include <cstdint>
#include <iostream>

int main() {
    // Narrowing: truncates high bits
    int32_t x = 1000;
    uint8_t y = static_cast<uint8_t>(x); // 232 (1000 % 256)

    // Float to int: truncates toward zero
    double f = 3.9;
    int i = static_cast<int>(f); // 3

    double neg = -3.9;
    int neg_i = static_cast<int>(neg); // -3

    // Pointer to integer
    int value = 42;
    int* ptr = &value;
    uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);

    std::cout << "y: " << static_cast<int>(y) << "\n";
    std::cout << "i: " << i << ", neg_i: " << neg_i << "\n";
    std::cout << "addr: " << addr << "\n";

    return 0;
}
```

Rust:

```
fn main() {
    // Narrowing: truncates high bits (wrapping behavior)
    let x: i32 = 1000;
    let y: u8 = x as u8; // 232 (1000 % 256)
    println!("1000 as u8 = {}", y);

    // Float to int: truncates toward zero
    let f: f64 = 3.9;
    let i: i32 = f as i32; // 3
    println!("3.9 as i32 = {}", i);

    let neg: f64 = -3.9;
    let neg_i: i32 = neg as i32; // -3
    println!("-3.9 as i32 = {}", neg_i);

    // Pointer casts
    let value: i32 = 42;
```

(continues on next page)

(continued from previous page)

```

let ptr: *const i32 = &value;
let addr: usize = ptr as usize;
println!("Address: {:#x}", addr);

// Reference to raw pointer
let r: &i32 = &value;
let raw: *const i32 = r as *const i32;
println!("Raw pointer: {:?}", raw);
}

```

From and Into Traits

From and Into are traits for infallible type conversions. They're the idiomatic way to convert between types in Rust, similar to C++ conversion constructors but more explicit and composable. Implementing `From<T>` for a type automatically provides the `Into<T>` implementation for free. These traits are used extensively in the standard library and are the preferred way to handle type conversions:

C++ (conversion constructor):

```

#include <iostream>

class Fahrenheit; // forward declaration

class Celsius {
public:
    double value;

    explicit Celsius(double v) : value(v) {}

    // Conversion constructor from Fahrenheit
    explicit Celsius(const Fahrenheit& f);

    void print() const {
        std::cout << value << "°C";
    }
};

class Fahrenheit {
public:
    double value;

    explicit Fahrenheit(double v) : value(v) {}

    // Conversion constructor from Celsius
    explicit Fahrenheit(const Celsius& c)
        : value(c.value * 9.0 / 5.0 + 32.0) {}

    void print() const {
        std::cout << value << "°F";
    }
};

```

(continues on next page)

(continued from previous page)

```
// Define after Fahrenheit is complete
Celsius::Celsius(const Fahrenheit& f)
    : value((f.value - 32.0) * 5.0 / 9.0) {}

int main() {
    Fahrenheit body_temp(98.6);
    Celsius c(body_temp); // explicit conversion

    std::cout << "Body temperature: ";
    body_temp.print();
    std::cout << " = ";
    c.print();
    std::cout << "\n";

    return 0;
}
```

Rust:

```
#[derive(Debug)]
struct Celsius(f64);

#[derive(Debug)]
struct Fahrenheit(f64);

// Implement From<Fahrenheit> for Celsius
impl From<Fahrenheit> for Celsius {
    fn from(f: Fahrenheit) -> Self {
        Celsius((f.0 - 32.0) * 5.0 / 9.0)
    }
}

// Implement From<Celsius> for Fahrenheit
impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Self {
        Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
    }
}

fn main() {
    let body_temp = Fahrenheit(98.6);

    // Using From::from explicitly
    let c1 = Celsius::from(Fahrenheit(98.6));
    println!("Body temperature: {:?} = {:?}", body_temp, c1);

    // Using Into (automatically available when From is implemented)
    let f2 = Fahrenheit(212.0);
    let c2: Celsius = f2.into();
    println!("Boiling point: 212°F = {:?}", c2);

    // Into is useful in generic contexts
}
```

(continues on next page)

(continued from previous page)

```

fn print_celsius(temp: impl Into<Celsius>) {
    let c = temp.into();
    println!("Temperature: {:.1}°C", c.0);
}

print_celsius(Fahrenheit(32.0)); // accepts Fahrenheit
print_celsius(Celsius(0.0));    // also accepts Celsius (From<T> for T)
}

```

Common From Implementations

The standard library provides many From implementations for common conversions. These make it easy to convert between related types without explicit casting:

C++:

```

#include <filesystem>
#include <iostream>
#include <string>
#include <vector>

int main() {
    // String from literal
    std::string s1 = "hello"; // implicit conversion
    std::string s2("hello");  // explicit construction

    // Vector from initializer list
    std::vector<int> v = {1, 2, 3};

    // Path from string
    std::filesystem::path p = "/tmp/file.txt";

    std::cout << "String: " << s1 << "\n";
    std::cout << "Vector size: " << v.size() << "\n";
    std::cout << "Path: " << p << "\n";

    return 0;
}

```

Rust:

```

use std::path::PathBuf;

fn main() {
    // String from &str
    let s1: String = String::from("hello");
    let s2: String = "hello".into(); // equivalent
    let s3: String = "hello".to_string(); // also common
    println!("Strings: {}, {}, {}", s1, s2, s3);

    // Vec from array
    let v1: Vec<i32> = Vec::from([1, 2, 3]);
}

```

(continues on next page)

(continued from previous page)

```

let v2: Vec<i32> = [1, 2, 3].into();
println!("Vectors: {:?}, {:?}", v1, v2);

// PathBuf from string
let p1: PathBuf = PathBuf::from("/tmp/file.txt");
let p2: PathBuf = "/tmp/file.txt".into();
println!("Paths: {:?}, {:?}", p1, p2);

// Box from value
let boxed: Box<i32> = Box::from(42);
let boxed2: Box<i32> = 42.into();
println!("Boxed: {}, {}", boxed, boxed2);

// String from number (via ToString trait, not From)
let num_str = 42.to_string();
println!("Number as string: {}", num_str);
}

```

TryFrom and TryInto

For conversions that might fail, Rust provides TryFrom and TryInto traits. These return Result<T, E> instead of the converted value directly, forcing callers to handle potential errors. This is similar to C++ functions that return std::optional or throw exceptions:

C++:

```

#include <stdint>
#include <iostream>
#include <optional>
#include <stdexcept>

// Return optional for fallible conversion
std::optional<uint8_t> try_from_int(int value) {
    if (value < 0 || value > 255) {
        return std::nullopt;
    }
    return static_cast<uint8_t>(value);
}

// Or throw exception
uint8_t from_int_throwing(int value) {
    if (value < 0 || value > 255) {
        throw std::out_of_range("Value out of u8 range");
    }
    return static_cast<uint8_t>(value);
}

int main() {
    // Using optional
    if (auto result = try_from_int(100)) {
        std::cout << "Converted: " << static_cast<int>(*result) << "\n";
    }
}

```

(continues on next page)

(continued from previous page)

```

if (auto result = try_from_int(1000)) {
    std::cout << "This won't print\n";
} else {
    std::cout << "Conversion failed for 1000\n";
}

// Using exception
try {
    auto val = from_int_throwing(1000);
} catch (const std::out_of_range& e) {
    std::cout << "Exception: " << e.what() << "\n";
}

return 0;
}

```

Rust:

```

use std::convert::TryFrom;

fn main() {
    let big: i32 = 1000;

    // TryFrom returns Result - must handle error
    let result: Result<u8, _> = u8::try_from(big);
    match result {
        Ok(val) => println!("Converted: {}", val),
        Err(e) => println!("Conversion failed: {}", e),
    }

    // Using ? operator in functions that return Result
    let fits: i32 = 100;
    let small: u8 = u8::try_from(fits).expect("Value should fit");
    println!("Converted 100 to u8: {}", small);

    // TryInto is also available
    let big2: i32 = 1000;
    let result2: Result<u8, _> = big2.try_into();
    println!("TryInto result: {:?}", result2);
}

```

Custom TryFrom

You can implement TryFrom for your own types to provide fallible conversions with custom error types. This is useful for validation during construction:

C++:

```

#include <iostream>
#include <optional>
#include <stdexcept>

```

(continues on next page)

(continued from previous page)

```

#include <string>

class PositiveInt {
    int value_;

public:
    // Factory function returning optional
    static std::optional<PositiveInt> try_from(int value) {
        if (value > 0) {
            return PositiveInt(value);
        }
        return std::nullopt;
    }

    // Or throwing constructor
    explicit PositiveInt(int value) : value_(value) {
        if (value <= 0) {
            throw std::invalid_argument("value must be positive");
        }
    }

    int get() const { return value_; }
};

int main() {
    // Using optional factory
    if (auto pos = PositiveInt::try_from(42)) {
        std::cout << "Created: " << pos->get() << "\n";
    }

    if (auto neg = PositiveInt::try_from(-1)) {
        std::cout << "This won't print\n";
    } else {
        std::cout << "Failed to create from -1\n";
    }

    return 0;
}

```

Rust:

```

use std::convert::TryFrom;
use std::fmt;

#[derive(Debug)]
struct PositiveInt(i32);

#[derive(Debug)]
struct NotPositiveError(i32);

impl fmt::Display for NotPositiveError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {

```

(continues on next page)

(continued from previous page)

```

        write!(f, "{} is not positive", self.0)
    }
}

impl TryFrom<i32> for PositiveInt {
    type Error = NotPositiveError;

    fn try_from(value: i32) -> Result<Self, Self::Error> {
        if value > 0 {
            Ok(PositiveInt(value))
        } else {
            Err(NotPositiveError(value))
        }
    }
}

fn main() {
    // Successful conversion
    let pos = PositiveInt::try_from(42);
    println!("From 42: {:?}", pos); // Ok(PositiveInt(42))

    // Failed conversion
    let neg = PositiveInt::try_from(-1);
    println!("From -1: {:?}", neg); // Err(NotPositiveError(-1))

    // Pattern matching on result
    match PositiveInt::try_from(100) {
        Ok(p) => println!("Created positive int: {:?}", p),
        Err(e) => println!("Error: {}", e),
    }
}

```

String Conversions

String parsing and formatting are common conversion operations. Rust uses the `FromStr` trait (called via `.parse()`) for parsing and `ToString/Display` for formatting:

C++:

```

#include <iostream>
#include <sstream>
#include <string>

int main() {
    // Number to string
    int n = 42;
    std::string s1 = std::to_string(n);
    std::cout << "to_string: " << s1 << "\n";

    // Using stringstream for formatting
    std::ostringstream oss;
    oss << "Value: " << n;
}

```

(continues on next page)

(continued from previous page)

```

std::string s2 = oss.str();
std::cout << "stringstream: " << s2 << "\n";

// String to number
std::string num_str = "42";
int parsed = std::stoi(num_str);
std::cout << "Parsed: " << parsed << "\n";

// Error handling
try {
    int bad = std::stoi("not a number");
} catch (const std::invalid_argument& e) {
    std::cout << "Parse error: " << e.what() << "\n";
}

return 0;
}

```

Rust:

```

use std::str::FromStr;

fn main() {
    // Number to String
    let n = 42;
    let s1: String = n.to_string();
    let s2: String = format!("Value: {}", n);
    println!("to_string: {}", s1);
    println!("format!: {}", s2);

    // String to number using parse()
    let num_str = "42";
    let parsed: i32 = num_str.parse().unwrap();
    println!("Parsed: {}", parsed);

    // With turbofish syntax
    let parsed2 = "42".parse::<i32>().unwrap();
    println!("Turbofish: {}", parsed2);

    // Using FromStr directly
    let parsed3 = i32::from_str("42").unwrap();
    println!("FromStr: {}", parsed3);

    // Error handling - parse returns Result
    let bad: Result<i32, _> = "not a number".parse();
    match bad {
        Ok(n) => println!("Parsed: {}", n),
        Err(e) => println!("Parse error: {}", e),
    }
}

```

Deref Coercion

Rust automatically applies deref coercion when a type implements the Deref trait. This allows `&String` to be used where `&str` is expected, and `&Box<T>` where `&T` is expected. This is similar to C++'s implicit conversions but more controlled:

C++:

```
#include <iostream>
#include <memory>
#include <string>

void print_cstr(const char* s) {
    std::cout << s << "\n";
}

void print_int_ref(const int& n) {
    std::cout << n << "\n";
}

int main() {
    std::string s = "hello";

    // std::string implicitly converts to const char*
    print_cstr(s.c_str()); // explicit .c_str() needed

    // unique_ptr dereferences to T
    auto ptr = std::make_unique<int>(42);
    print_int_ref(*ptr); // explicit * needed

    return 0;
}
```

Rust:

```
fn print_str(s: &str) {
    println!("{}", s);
}

fn print_int_ref(n: &i32) {
    println!("{}", n);
}

fn main() {
    let s = String::from("hello");

    // String automatically derefs to &str - no explicit conversion!
    print_str(&s);

    // Box<T> automatically derefs to &T
    let boxed = Box::new(42);
    print_int_ref(&boxed); // auto-deref, no * needed

    // Works through multiple levels
    let boxed_string = Box::new(String::from("nested"));
}
```

(continues on next page)

(continued from previous page)

```

print_str(&boxed_string); // Box<String> -> String -> &str

// Vec<T> derefs to &[T]
fn print_slice(s: &[i32]) {
    println!("{:?}", s);
}
let v = vec![1, 2, 3];
print_slice(&v); // Vec<i32> -> &[i32]
}

```

AsRef and AsMut

AsRef and AsMut traits provide cheap reference-to-reference conversions. They're commonly used in generic functions to accept multiple types that can be viewed as a reference to some target type:

C++:

```

#include <filesystem>
#include <iostream>
#include <string>
#include <string_view>

// Overloads for different string types
void print_path(const std::filesystem::path& p) {
    std::cout << p << "\n";
}

void print_path(const std::string& s) {
    print_path(std::filesystem::path(s));
}

void print_path(const char* s) {
    print_path(std::filesystem::path(s));
}

int main() {
    print_path("/tmp/file.txt");
    print_path(std::string("/home/user"));
    print_path(std::filesystem::path("/var/log"));

    return 0;
}

```

Rust:

```

use std::path::Path;

// Single generic function accepts anything that can be viewed as &Path
fn print_path<P: AsRef<Path>>(path: P) {
    println!("{}", path.as_ref().display());
}

fn main() {

```

(continues on next page)

(continued from previous page)

```

// All of these work with the same function!
print_path("hello.txt");           // &str
print_path(String::from("hi.txt")); // String
print_path(Path::new("x.txt"));     // &Path
print_path(std::path::PathBuf::from("y")); // PathBuf

// AsRef is also useful for byte slices
fn process_bytes<T: AsRef<[u8]>>>(data: T) {
    let bytes = data.as_ref();
    println!("Got {} bytes", bytes.len());
}

process_bytes("hello");           // &str -> &[u8]
process_bytes(vec![1, 2, 3]);     // Vec<u8> -> &[u8]
process_bytes(&[4, 5, 6][..]);    // &[u8]
}

```

Unsafe Transmute

`std::mem::transmute` reinterprets the bits of one type as another, similar to C++'s `reinterpret_cast`. This is unsafe and should only be used when you're certain the bit patterns are valid for both types:

C++:

```

#include <stdint>
#include <cstring>
#include <iostream>

int main() {
    // reinterpret_cast for pointer types
    uint32_t x = 0x12345678;
    uint8_t* bytes = reinterpret_cast<uint8_t*>(&x);

    std::cout << "Bytes: ";
    for (int i = 0; i < 4; ++i) {
        std::cout << std::hex << static_cast<int>(bytes[i]) << " ";
    }
    std::cout << "\n";

    // memcpy for type punning (safer than reinterpret_cast)
    float f = 3.14f;
    uint32_t bits;
    std::memcpy(&bits, &f, sizeof(f));
    std::cout << "Float bits: 0x" << std::hex << bits << "\n";

    return 0;
}

```

Rust:

```

fn main() {
    // UNSAFE: transmute reinterprets bits
    unsafe {

```

(continues on next page)

(continued from previous page)

```

let x: u32 = 0x12345678;
let bytes: [u8; 4] = std::mem::transmute(x);
println!("Bytes: {:02x?}", bytes);

// Float to bits
let f: f32 = 3.14;
let bits: u32 = std::mem::transmute(f);
println!("Float bits: {:#x}", bits);
}

// Safe alternatives exist for common cases:
let x: u32 = 0x12345678;
let bytes = x.to_ne_bytes(); // native endian
println!("Safe bytes: {:02x?}", bytes);

let f: f32 = 3.14;
let bits = f.to_bits(); // safe float-to-bits
println!("Safe float bits: {:#x}", bits);
}

```

See Also

- *Traits and Generics* - From/Into as traits
- *Error Handling* - TryFrom returns Result

1.8.10 Const Functions**Table of Contents**

- *Const Functions*
 - *Basic const fn*
 - *Const Context*
 - *Const Limitations*
 - *Const Generics*
 - * *Const Expressions in Types*
 - *Const Trait Methods*
 - *Compile-Time Assertions*
 - *See Also*

Source

src/rust/const_fn

Rust's `const fn` is similar to C++ `constexpr` - functions that can be evaluated at compile time. Rust's `const` evaluation is more restrictive but guarantees deterministic behavior.

Basic const fn

C++:

```
constexpr int square(int x) {
    return x * x;
}

constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

int main() {
    constexpr int a = square(5);      // compile-time
    constexpr int b = factorial(5);   // compile-time
    int runtime_val = 10;
    int c = square(runtime_val);      // runtime
}
```

Rust:

```
const fn square(x: i32) -> i32 {
    x * x
}

const fn factorial(n: u32) -> u32 {
    if n <= 1 { 1 } else { n * factorial(n - 1) }
}

fn main() {
    const A: i32 = square(5);          // compile-time
    const B: u32 = factorial(5);       // compile-time
    let runtime_val = 10;
    let c = square(runtime_val);      // runtime
}
```

Const Context

const fn can be called in const contexts:

```
const fn compute() -> usize { 42 }

// Array size
const SIZE: usize = compute();
let arr: [i32; SIZE] = [0; SIZE];

// Static variables
static VALUE: i32 = square(10);

// Const generics
struct Buffer<const N: usize> {
    data: [u8; N],
}
```

(continues on next page)

(continued from previous page)

```
const fn buffer_size() -> usize { 1024 }
let buf: Buffer<{ buffer_size() }> = Buffer { data: [0; 1024] };
```

Const Limitations

Rust's const fn has more restrictions than C++ constexpr:

```
// Allowed in const fn:
const fn allowed() -> i32 {
    let x = 5;
    let y = if x > 0 { 10 } else { 20 };
    let mut sum = 0;
    let mut i = 0;
    while i < 5 {
        sum += i;
        i += 1;
    }
    sum
}

// NOT allowed (as of Rust 1.70):
// - Heap allocation (Box, Vec, String)
// - Trait objects
// - Most iterator methods
// - Floating point operations (stabilizing)
// - Mutable references to non-local data
```

Const Generics

C++ (non-type template parameters):

```
template<size_t N>
struct Buffer {
    char data[N];

    constexpr size_t size() const { return N; }
};

int main() {
    Buffer<1024> buf;
}
```

Rust:

```
struct Buffer<const N: usize> {
    data: [u8; N],
}

impl<const N: usize> Buffer<N> {
    const fn new() -> Self {
        Buffer { data: [0; N] }
    }
}
```

(continues on next page)

(continued from previous page)

```

    const fn size(&self) -> usize {
        N
    }
}

fn main() {
    let buf = Buffer::<1024>::new();
    const SIZE: usize = buf.size();
}

```

Const Expressions in Types

```

// Array with computed size
const fn array_len<T>() -> usize {
    std::mem::size_of::<T>() * 8
}

struct BitSet<T> {
    bits: [u8; std::mem::size_of::<T>()],
}

// Generic over const expression
fn process<const N: usize>(arr: [i32; N]) -> i32 {
    let mut sum = 0;
    let mut i = 0;
    while i < N {
        sum += arr[i];
        i += 1;
    }
    sum
}

```

Const Trait Methods

```

struct Point {
    x: i32,
    y: i32,
}

impl Point {
    // const method
    const fn new(x: i32, y: i32) -> Self {
        Point { x, y }
    }

    const fn origin() -> Self {
        Point { x: 0, y: 0 }
    }

    const fn manhattan_distance(&self) -> i32 {

```

(continues on next page)

(continued from previous page)

```

        // abs() is const fn
        self.x.abs() + self.y.abs()
    }
}

const ORIGIN: Point = Point::origin();
const P: Point = Point::new(3, 4);
const DIST: i32 = P.manhattan_distance();

```

Compile-Time Assertions

C++:

```

static_assert(sizeof(int) == 4, "int must be 4 bytes");

template<typename T>
constexpr void check_size() {
    static_assert(sizeof(T) <= 16, "Type too large");
}

```

Rust:

```

// Using const evaluation
const _: () = assert!(std::mem::size_of::<i32>() == 4);

// In const fn
const fn check_size<T>() {
    assert!(std::mem::size_of::<T>() <= 16, "Type too large");
}

// Compile-time check
const _: () = check_size::<i32>();

```

See Also

- *Traits and Generics* - Const trait bounds

1.8.11 Closures

Table of Contents

- *Closures*
 - *Basic Syntax*
 - *Capture Modes*
 - * *Move Closures*
 - *Fn Traits*
 - *Closures as Parameters*

- *Returning Closures*
- *Closure Type Inference*
- *Closures with Iterators*
- *See Also*

Source

src/rust/closures

Rust closures are anonymous functions that can capture variables from their enclosing scope. They're similar to C++ lambdas but with a key difference: Rust automatically infers how to capture each variable (by reference, mutable reference, or by value) based on how the closure uses it. This eliminates the need for explicit capture lists like `[&]` or `[=]` in C++. Closures in Rust also implement one or more of the `Fn`, `FnMut`, or `FnOnce` traits, which determines how they can be called.

Basic Syntax

Rust closures use vertical bars `|args|` instead of parentheses for parameters, and the body can be a single expression or a block. Type annotations are optional - Rust infers types from how the closure is used. Unlike C++ where you must explicitly specify capture mode, Rust analyzes the closure body and captures each variable in the least restrictive way possible:

C++:

```
#include <functional>

int main() {
    // Lambda with explicit capture
    int x = 10;
    auto add = [x](int y) { return x + y; };
    auto result = add(5); // 15

    // Mutable capture
    int count = 0;
    auto increment = [&count]() { count++; };
}
```

Rust:

```
fn main() {
    // Closure with automatic capture
    let x = 10;
    let add = |y| x + y;
    let result = add(5); // 15

    // Mutable capture
    let mut count = 0;
    let mut increment = || count += 1;
    increment();
}
```

Capture Modes

C++ requires explicit capture specifications ([=], [&], [x], [&x]), which can be error-prone - capturing by reference when you meant by value can cause dangling references. Rust infers the capture mode automatically: if the closure only reads a variable, it captures by immutable reference; if it modifies the variable, it captures by mutable reference; if it moves the variable (e.g., returns it or passes it to a function taking ownership), it captures by value:

C++ explicit captures:

```
int a = 1, b = 2;

auto by_value = [a, b]() { return a + b; };
auto by_ref = [&a, &b]() { return a + b; };
auto all_value = [=]() { return a + b; };
auto all_ref = [&]() { return a + b; };
auto mixed = [a, &b]() { return a + b; };
```

Rust automatic inference:

```
let a = 1;
let mut b = 2;

// Rust infers capture mode based on usage
let by_ref = || a + b;           // borrows a and b
let by_mut_ref = || { b += 1; }; // mutably borrows b

// Force move with `move` keyword
let by_value = move || a + b;    // moves/copies a and b
```

Move Closures

The move keyword forces a closure to take ownership of all captured variables, even if the closure body would only require borrowing. This is essential when the closure needs to outlive the current scope, such as when spawning threads or returning closures from functions. For Copy types like integers, move creates a copy; for non-Copy types like String, it transfers ownership:

```
fn main() {
    let s = String::from("hello");

    // Without move: borrows s
    let borrow = || println!("{}", s);

    // With move: takes ownership of s
    let own = move || println!("{}", s);
    // s is no longer valid here

    // Essential for threads
    let data = vec![1, 2, 3];
    std::thread::spawn(move || {
        println!("{:?}", data);
    });
}
```

Fn Traits

Rust closures implement one or more of three traits based on how they use captured variables. `FnOnce` is the most general - all closures implement it. `FnMut` is for closures that don't consume captured values but may mutate them. `Fn` is for closures that only read captured values. When accepting closures as parameters, use the most general trait that works (`FnOnce > FnMut > Fn`) to maximize flexibility:

Trait	Receiver	Can be called
<code>FnOnce</code>	<code>self</code>	Once (consumes captured values)
<code>FnMut</code>	<code>&mut self</code>	Multiple times (may mutate)
<code>Fn</code>	<code>&self</code>	Multiple times (no mutation)

```
// FnOnce - consumes captured value
let s = String::from("hello");
let consume = || drop(s); // can only call once

// FnMut - mutates captured value
let mut count = 0;
let mut increment = || count += 1; // can call multiple times

// Fn - only reads captured values
let x = 10;
let read = || x + 1; // can call multiple times
```

Closures as Parameters

Functions can accept closures as parameters using generics with trait bounds or trait objects. The generic approach (monomorphization) generates specialized code for each closure type, giving the best performance. The trait object approach (`dyn Fn`) uses dynamic dispatch, which has a small runtime cost but allows storing different closure types in the same collection:

C++:

```
#include <functional>

void apply(std::function<int(int)> f, int x) {
    std::cout << f(x);
}

// Or with templates
template<typename F>
void apply_template(F f, int x) {
    std::cout << f(x);
}
```

Rust:

```
// Generic with trait bound (monomorphized, like C++ template)
fn apply<F>(f: F, x: i32) -> i32
where
    F: Fn(i32) -> i32,
{
    f(x)
```

(continues on next page)

(continued from previous page)

```

}

// impl Trait syntax (simpler)
fn apply_impl(f: impl Fn(i32) -> i32, x: i32) -> i32 {
    f(x)
}

// Dynamic dispatch (like std::function)
fn apply_dyn(f: &dyn Fn(i32) -> i32, x: i32) -> i32 {
    f(x)
}

fn main() {
    let double = |x| x * 2;
    println!("{}", apply(double, 5)); // 10
}

```

Returning Closures

Returning closures from functions requires either `impl Trait` syntax (when returning a single closure type) or `Box<dyn Fn>` (when the closure type might vary). The `move` keyword is usually necessary because the closure needs to own its captured variables - otherwise they would be dropped when the function returns:

C++:

```

std::function<int(int)> make_adder(int x) {
    return [x](int y) { return x + y; };
}

```

Rust:

```

// Return with impl Trait
fn make_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y
}

// Return boxed closure (for dynamic dispatch)
fn make_adder_boxed(x: i32) -> Box<dyn Fn(i32) -> i32> {
    Box::new(move |y| x + y)
}

fn main() {
    let add5 = make_adder(5);
    println!("{}", add5(10)); // 15
}

```

Closure Type Inference

Rust infers closure parameter and return types from how the closure is used. This means you usually don't need type annotations, but it also means each closure has a unique anonymous type - you can't have two closures with the same signature stored in the same variable without using trait objects. Once a closure's types are inferred from its first use, they're fixed:

```
// Types inferred from usage
let add = |a, b| a + b;
let result: i32 = add(1, 2); // infers i32

// Explicit types when needed
let parse = |s: &str| -> i32 { s.parse().unwrap() };

// Once inferred, types are fixed
let identity = |x| x;
let s = identity("hello"); // infers &str
// let n = identity(42);    // error: expected &str
```

Closures with Iterators

Closures are most commonly used with iterator adapters like `map`, `filter`, and `fold`. These methods take closures that transform or filter elements. The closure syntax is concise enough that complex transformations remain readable. Iterator adapters are lazy - they don't execute until you consume the iterator with `collect`, `for_each`, or similar methods:

```
let numbers = vec![1, 2, 3, 4, 5];

// map
let doubled: Vec<_> = numbers.iter().map(|x| x * 2).collect();

// filter
let evens: Vec<_> = numbers.iter().filter(|&&x| x % 2 == 0).collect();

// fold
let sum = numbers.iter().fold(0, |acc, x| acc + x);

// for_each
numbers.iter().for_each(|x| println!("{}", x));

// find
let first_even = numbers.iter().find(|&&x| x % 2 == 0);
```

See Also

- *Iterators* - Closures with iterator adapters
- *Traits and Generics* - Fn traits in detail

1.8.12 Smart Pointers

Table of Contents

- *Smart Pointers*
 - *Smart Pointer Comparison*
 - *Box (Unique Ownership)*
 - * *Use Cases for Box*

- *Rc (Reference Counting)*
 - * *Rc with RefCell (Interior Mutability)*
- *Arc (Atomic Reference Counting)*
 - * *Arc with Mutex*
- *RefCell (Interior Mutability)*
 - * *Cell (Copy Types)*
- *Weak References*
- *Cow (Clone on Write)*
- *See Also*

Source

`src/rust/smart_pointers`

Rust smart pointers provide different ownership and borrowing patterns beyond what basic references offer. Unlike C++ where smart pointers are library types that can be misused (e.g., creating cycles with `shared_ptr`), Rust's ownership system prevents most pointer-related bugs at compile time. Smart pointers in Rust implement the `Deref` trait, allowing them to be used like regular references, and the `Drop` trait for automatic cleanup.

Smart Pointer Comparison

The following table maps C++ smart pointers to their Rust equivalents. Note that Rust splits C++'s `shared_ptr` into two types: `Rc` for single-threaded code (faster, no atomic operations) and `Arc` for multi-threaded code (thread-safe with atomic reference counting):

C++	Rust
<code>std::unique_ptr<T></code>	<code>Box<T></code>
<code>std::shared_ptr<T></code>	<code>Rc<T></code> / <code>Arc<T></code>
<code>std::weak_ptr<T></code>	<code>Weak<T></code>
N/A	<code>RefCell<T></code>
<code>std::mutex<T></code>	<code>Mutex<T></code>

Box (Unique Ownership)

`Box<T>` provides heap allocation with single ownership, similar to `std::unique_ptr<T>`. When a `Box` goes out of scope, it automatically frees the heap memory. Unlike C++ where you might accidentally copy a `unique_ptr` (which is a compile error but easy to attempt), Rust's move semantics make ownership transfer explicit and safe. `Box` is commonly used for recursive data structures, large data that shouldn't be copied, and trait objects.

C++:

```
#include <memory>

int main() {
    auto ptr = std::make_unique<int>(42);
    std::cout << *ptr;

    // Transfer ownership
    auto ptr2 = std::move(ptr);
}
```

(continues on next page)

(continued from previous page)

```
// ptr is now null
}
```

Rust:

```
fn main() {
    let boxed = Box::new(42);
    println!("{}", *boxed);

    // Transfer ownership
    let boxed2 = boxed;
    // boxed is no longer valid - compile error if used
}
```

Use Cases for Box

Box is essential in several scenarios where stack allocation isn't possible or desirable. Recursive types like linked lists and trees require Box because the compiler needs to know the size of types at compile time - without indirection, a recursive type would have infinite size. Large data benefits from Box to avoid expensive stack copies. Trait objects require Box (or another pointer type) because different implementations may have different sizes:

```
// 1. Recursive types (unknown size at compile time)
enum List {
    Cons(i32, Box<List>),
    Nil,
}

// 2. Large data on heap
let large = Box::new([0u8; 1_000_000]);

// 3. Trait objects
let drawable: Box<dyn Draw> = Box::new(Circle { radius: 5.0 });
```

Rc (Reference Counting)

Rc<T> (Reference Counted) enables shared ownership where multiple parts of your code need to read the same data. It's similar to `std::shared_ptr<T>` but is explicitly single-threaded - attempting to send an Rc to another thread is a compile error. This restriction allows Rc to use non-atomic reference counting, making it faster than Arc when thread safety isn't needed. The reference count is incremented when you clone an Rc and decremented when an Rc is dropped.

C++:

```
#include <memory>

int main() {
    auto shared = std::make_shared<int>(42);
    auto shared2 = shared; // ref count = 2

    std::cout << shared.use_count(); // 2
}
```

Rust:

```
use std::rc::Rc;

fn main() {
    let shared = Rc::new(42);
    let shared2 = Rc::clone(&shared); // ref count = 2

    println!("{}", Rc::strong_count(&shared)); // 2
}
```

Rc with RefCell (Interior Mutability)

`Rc<T>` only provides shared (immutable) access to its contents - you cannot get a mutable reference through an `Rc`. When you need shared ownership with mutation, combine `Rc` with `RefCell`. `RefCell` moves Rust's borrow checking from compile time to runtime: it tracks borrows dynamically and will panic if you violate the borrowing rules (e.g., two simultaneous mutable borrows). This pattern is common for graph structures, observer patterns, and caches:

```
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let shared = Rc::new(RefCell::new(42));

    // Multiple owners
    let shared2 = Rc::clone(&shared);

    // Mutate through RefCell
    *shared.borrow_mut() += 1;

    println!("{}", shared2.borrow()); // 43
}
```

Arc (Atomic Reference Counting)

`Arc<T>` (Atomically Reference Counted) is the thread-safe version of `Rc<T>`. It uses atomic operations for reference counting, making it safe to share across threads. This is equivalent to `std::shared_ptr<T>` in C++, which also uses atomic reference counting. The atomic operations add some overhead compared to `Rc`, so prefer `Rc` when you don't need thread safety. `Arc` is commonly used with `Mutex` or `RwLock` to provide thread-safe shared mutable state.

C++:

```
#include <memory>
#include <thread>

int main() {
    auto shared = std::make_shared<int>(42);

    std::thread t([shared]() {
        std::cout << *shared;
    });
    t.join();
}
```

Rust:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let shared = Arc::new(42);
    let shared2 = Arc::clone(&shared);

    let handle = thread::spawn(move || {
        println!("{}", *shared2);
    });

    handle.join().unwrap();
}
```

Arc with Mutex

For thread-safe mutable access, combine `Arc` with `Mutex`. The `Mutex` ensures only one thread can access the data at a time, while `Arc` allows multiple threads to hold references to the mutex. This pattern is the Rust equivalent of sharing a `std::shared_ptr<std::mutex<T>>` in C++, but Rust's type system ensures you can't forget to lock the mutex - the data is only accessible through the lock guard:

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("{}", *counter.lock().unwrap()); // 10
}
```

RefCell (Interior Mutability)

`RefCell<T>` provides interior mutability - the ability to mutate data even when there are immutable references to it. This is checked at runtime rather than compile time: `RefCell` tracks active borrows and panics if you violate the borrowing rules. Use `RefCell` when you know your code follows borrowing rules but the compiler can't verify it, such as in graph structures or mock objects for testing. The runtime checks have a small performance cost, so prefer compile-time borrowing when possible:

```

use std::cell::RefCell;

fn main() {
    let cell = RefCell::new(42);

    // Immutable borrow
    let r1 = cell.borrow();
    println!("{}", *r1);
    drop(r1); // must drop before mutable borrow

    // Mutable borrow
    *cell.borrow_mut() += 1;

    // Runtime panic if rules violated:
    // let r1 = cell.borrow();
    // let r2 = cell.borrow_mut(); // panic!
}

```

Cell (Copy Types)

For types that implement Copy (like integers and floats), Cell<T> provides a simpler alternative to RefCell. Instead of borrowing, Cell copies values in and out. This avoids the runtime borrow tracking overhead and can never panic, but only works with Copy types since it needs to copy the entire value:

```

use std::cell::Cell;

fn main() {
    let cell = Cell::new(42);

    cell.set(43);
    let value = cell.get(); // copies value out
}

```

Weak References

Weak<T> prevents reference cycles that would cause memory leaks. A Weak reference doesn't contribute to the reference count, so it won't keep the data alive. Before using a Weak, you must upgrade it to an Rc or Arc, which returns None if the data has been dropped. This is essential for parent-child relationships where children need to reference their parent without creating a cycle:

C++:

```

#include <memory>

struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev; // weak to break cycle
};

```

Rust:

```

use std::rc::{Rc, Weak};
use std::cell::RefCell;

```

(continues on next page)

(continued from previous page)

```

struct Node {
    next: Option<Rc<RefCell<Node>>>,
    prev: Option<Weak<RefCell<Node>>>, // weak to break cycle
}

fn main() {
    let node = Rc::new(RefCell::new(Node {
        next: None,
        prev: None,
    }));

    // Create weak reference
    let weak: Weak<_> = Rc::downgrade(&node);

    // Upgrade to Rc (returns Option)
    if let Some(strong) = weak.upgrade() {
        println!("Node still exists");
    }
}

```

Cow (Clone on Write)

Cow<T> (Clone on Write) is a smart pointer that can hold either borrowed or owned data. It delays cloning until mutation is actually needed, which can significantly improve performance when you often don't need to modify the data. This is useful for functions that might need to modify their input but usually don't, or for caching scenarios where you want to avoid unnecessary allocations:

```

use std::borrow::Cow;

fn process(input: &str) -> Cow<str> {
    if input.contains("bad") {
        // Only allocate if we need to modify
        Cow::Owned(input.replace("bad", "good"))
    } else {
        // No allocation, just borrow
        Cow::Borrowed(input)
    }
}

fn main() {
    let s1 = process("hello"); // Borrowed
    let s2 = process("bad word"); // Owned
}

```

See Also

- *Ownership* - Ownership fundamentals
- *Threads* - Arc and Mutex with threads

1.8.13 Error Handling

Table of Contents

- *Error Handling*
 - *Error Handling Comparison*
 - *Option Type*
 - * *Option Methods*
 - *Result Type*
 - * *Result Methods*
 - *The ? Operator*
 - *Custom Error Types*
 - *Pattern Matching Errors*
 - *Panic vs Result*
 - *See Also*

Source

[src/rust/error_handling](#)

Rust uses `Result<T, E>` and `Option<T>` for error handling instead of exceptions. This approach makes error handling explicit - you can see from a function's signature whether it can fail and what errors it might return. Unlike exceptions which can be thrown from anywhere and caught anywhere (or not caught at all), Rust errors are values that must be explicitly handled at each call site. This eliminates the “invisible control flow” problem of exceptions and makes code easier to reason about.

Error Handling Comparison

The following table maps C++ error handling mechanisms to their Rust equivalents. Note that Rust's `Result` is similar to C++23's `std::expected`, but `Result` has been in Rust since 1.0 and is deeply integrated into the language with the `?` operator:

C++	Rust
<code>throw exception</code>	<code>Err(e)</code>
<code>try { } catch { }</code>	<code>match / ?</code>
<code>std::optional<T></code>	<code>Option<T></code>
<code>std::expected<T, E></code>	<code>Result<T, E></code>
<code>nullptr check</code>	<code>Option<T></code>

Option Type

`Option<T>` represents a value that may or may not exist - it's either `Some(value)` or `None`. This replaces null pointers and eliminates null pointer dereference bugs at compile time. Unlike C++ where you might forget to check if an `optional` has a value before accessing it, Rust forces you to handle both cases through pattern matching or methods like `unwrap_or`. The compiler won't let you access the inner value without acknowledging that it might not exist:

C++:

```
#include <optional>

std::optional<int> find_index(const std::vector<int>& v, int target) {
    for (size_t i = 0; i < v.size(); i++) {
        if (v[i] == target) return i;
    }
    return std::nullopt;
}

int main() {
    std::vector<int> v = {1, 2, 3};
    if (auto idx = find_index(v, 2)) {
        std::cout << "Found at " << *idx;
    }
}
```

Rust:

```
fn find_index(v: &[i32], target: i32) -> Option<usize> {
    for (i, &x) in v.iter().enumerate() {
        if x == target {
            return Some(i);
        }
    }
    None
}

fn main() {
    let v = vec![1, 2, 3];
    if let Some(idx) = find_index(&v, 2) {
        println!("Found at {}", idx);
    }
}
```

Option Methods

Option provides many methods for working with optional values without verbose pattern matching. These methods form a fluent API for transforming, combining, and extracting optional values. The `unwrap` family of methods extracts the value but panics if it's `None` - use these only when you're certain the value exists or in prototyping code:

```
let x: Option<i32> = Some(42);
let y: Option<i32> = None;

// Unwrapping
let val = x.unwrap();           // panics if None
let val = x.expect("no value"); // panics with message
let val = x.unwrap_or(0);       // default if None
let val = x.unwrap_or_default(); // Default::default() if None
let val = x.unwrap_or_else(|| compute_default());

// Checking
let is_some = x.is_some();
let is_none = y.is_none();
```

(continues on next page)

(continued from previous page)

```
// Transforming
let doubled = x.map(|v| v * 2);           // Some(84)
let filtered = x.filter(|&v| v > 50);     // None
let chained = x.and_then(|v| Some(v + 1)); // Some(43)

// Converting to Result
let result: Result<i32, &str> = x.ok_or("no value");
```

Result Type

`Result<T, E>` represents an operation that can succeed with a value of type `T` or fail with an error of type `E`. Unlike exceptions which separate the error path from the normal code path, `Result` keeps them together - you handle the error right where it occurs. This makes error handling more predictable and prevents errors from silently propagating up the call stack. The type system ensures you can't accidentally ignore an error:

C++:

```
#include <expected> // C++23
#include <string>

std::expected<int, std::string> parse_int(const std::string& s) {
    try {
        return std::stoi(s);
    } catch (...) {
        return std::unexpected("parse error");
    }
}
```

Rust:

```
fn parse_int(s: &str) -> Result<i32, std::num::ParseIntError> {
    s.parse()
}

fn main() {
    match parse_int("42") {
        Ok(n) => println!("Parsed: {}", n),
        Err(e) => println!("Error: {}", e),
    }
}
```

Result Methods

`Result` provides methods similar to `Option` for working with fallible operations. The `map` and `map_err` methods let you transform the success or error value independently. The `and_then` method (also called `flatmap` in other languages) chains operations that might fail:

```
let x: Result<i32, &str> = Ok(42);
let y: Result<i32, &str> = Err("error");

// Unwrapping
```

(continues on next page)

(continued from previous page)

```

let val = x.unwrap();           // panics if Err
let val = x.expect("failed");   // panics with message
let val = x.unwrap_or(0);       // default if Err
let val = x.unwrap_or_else(|e| handle_error(e));

// Checking
let is_ok = x.is_ok();
let is_err = y.is_err();

// Transforming
let doubled = x.map(|v| v * 2); // Ok(84)
let mapped_err = y.map_err(|e| format!("Error: {}", e));

// Converting to Option
let opt: Option<i32> = x.ok();   // Some(42)
let opt: Option<&str> = y.err(); // Some("error")

```

The ? Operator

The `?` operator is Rust's way of propagating errors concisely. When applied to a `Result`, it either unwraps the `Ok` value and continues, or returns early with the `Err` value. This eliminates the boilerplate of matching on every `Result` and manually returning errors. The `?` operator also performs automatic error type conversion using the `From` trait, making it easy to combine different error types:

C++ (manual propagation):

```

std::expected<int, Error> read_config() {
    auto file = open_file("config.txt");
    if (!file) return std::unexpected(file.error());

    auto content = read_content(*file);
    if (!content) return std::unexpected(content.error());

    auto value = parse_int(*content);
    if (!value) return std::unexpected(value.error());

    return *value;
}

```

Rust (with ?):

```

fn read_config() -> Result<i32, Error> {
    let file = open_file("config.txt")?;
    let content = read_content(&file)?;
    let value = parse_int(&content)?;
    Ok(value)
}

// ? works with Option too
fn first_char(s: &str) -> Option<char> {
    let c = s.chars().next()?;
    Some(c)
}

```

Custom Error Types

For libraries and larger applications, you'll want to define custom error types that can represent all the ways your code can fail. A common pattern is an enum with variants for each error kind. Implementing `From` for other error types enables automatic conversion with the `?` operator, so you can use `?` on standard library errors and have them automatically converted to your custom type:

```
use std::fmt;

#[derive(Debug)]
enum AppError {
    IoError(std::io::Error),
    ParseError(std::num::ParseIntError),
    Custom(String),
}

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            AppError::IoError(e) => write!(f, "IO error: {}", e),
            AppError::ParseError(e) => write!(f, "Parse error: {}", e),
            AppError::Custom(s) => write!(f, "{}", s),
        }
    }
}

impl std::error::Error for AppError {}

// Implement From for automatic conversion with ?
impl From<std::io::Error> for AppError {
    fn from(e: std::io::Error) -> Self {
        AppError::IoError(e)
    }
}
```

Pattern Matching Errors

Pattern matching gives you fine-grained control over error handling. You can match on specific error variants, use guards to add conditions, and handle multiple cases differently. The `if let` syntax is convenient when you only care about one case, and `let else` (Rust 1.65+) provides early return for the error case:

```
fn process(input: &str) -> Result<i32, String> {
    match input.parse::<i32>() {
        Ok(n) if n > 0 => Ok(n * 2),
        Ok(_) => Err("must be positive".into()),
        Err(e) => Err(format!("parse error: {}", e)),
    }
}

// if let for single case
if let Ok(n) = "42".parse::<i32>() {
    println!("{}", n);
}
```

(continues on next page)

(continued from previous page)

```
// let else for early return
fn process2(input: &str) -> i32 {
    let Ok(n) = input.parse::<i32>() else {
        return 0;
    };
    n * 2
}
```

Panic vs Result

Rust distinguishes between recoverable errors (`Result`) and unrecoverable errors (`panic!`). Use `panic!` for bugs and invariant violations - situations that indicate a programming error rather than an expected failure. Use `Result` for operations that can legitimately fail, like file I/O or parsing user input. The `assert!` macro panics in debug builds but can be compiled out in release builds for performance-critical code:

```
// panic! - unrecoverable errors (bugs, invariant violations)
fn get_index(v: &[i32], i: usize) -> i32 {
    if i >= v.len() {
        panic!("index out of bounds");
    }
    v[i]
}

// Result - recoverable errors (expected failures)
fn read_file(path: &str) -> Result<String, std::io::Error> {
    std::fs::read_to_string(path)
}

// assert! - debug checks (removed in release)
fn divide(a: i32, b: i32) -> i32 {
    debug_assert!(b != 0, "division by zero");
    a / b
}
```

See Also

- *Traits and Generics* - Error trait
- *Iterators* - Iterator methods returning Option/Result

1.8.14 Threads

Table of Contents

- *Threads*
 - *Basic Threading*
 - *Moving Data to Threads*
 - *Shared State with Arc and Mutex*
 - * *RwLock (Read-Write Lock)*

- *Channels (Message Passing)*
 - * *Multiple Producers*
- *Send and Sync Traits*
- *Scoped Threads*
- *Thread Pool Pattern*
- *Atomic Types*
- *See Also*

Source

src/rust/threads

Rust's ownership system extends to concurrency, preventing data races at compile time. The Send and Sync marker traits tell the compiler which types can safely cross thread boundaries. Unlike C++ where data races are undefined behavior that might silently corrupt data, Rust makes concurrent access errors into compile-time errors. This means if your Rust code compiles, it's free from data races - a guarantee no other mainstream systems language provides.

Basic Threading

Spawning threads in Rust is similar to C++, but with an important difference: the closure passed to `thread::spawn` must have a 'static lifetime, meaning it can't borrow data from the parent thread (unless using scoped threads). This prevents dangling references when the parent thread exits before the child. The `join` method returns a `Result` because the thread might have panicked:

C++:

```
#include <thread>
#include <iostream>

int main() {
    std::thread t([]() {
        std::cout << "Hello from thread\n";
    });
    t.join();
}
```

Rust:

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        println!("Hello from thread");
    });
    handle.join().unwrap();
}
```

Moving Data to Threads

When a thread needs to own data, use the `move` keyword to transfer ownership into the closure. This is safer than C++ where you might accidentally capture by reference and create a data race. For Copy types, `move` creates a copy; for owned types like `Vec` or `String`, it transfers ownership completely - the original variable becomes invalid:

C++:

```
#include <thread>
#include <vector>

int main() {
    std::vector<int> data = {1, 2, 3};

    std::thread t([data = std::move(data)]() {
        for (int x : data) {
            std::cout << x << " ";
        }
    });
    t.join();
}
```

Rust:

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        for x in &data {
            print!("{}", x);
        }
    });
    handle.join().unwrap();
}
```

Shared State with Arc and Mutex

When multiple threads need to access the same data, use `Arc` (atomic reference counting) for shared ownership and `Mutex` for mutual exclusion. Unlike C++ where you can forget to lock a mutex before accessing shared data, Rust's `Mutex` wraps the data it protects - you literally cannot access the data without going through the lock. The lock guard automatically releases the lock when dropped:

C++:

```
#include <thread>
#include <mutex>
#include <memory>
#include <vector>

int main() {
    auto counter = std::make_shared<int>(0);
    std::mutex mtx;
    std::vector<std::thread> threads;
```

(continues on next page)

(continued from previous page)

```

for (int i = 0; i < 10; i++) {
    threads.emplace_back([&mtx, counter]() {
        std::lock_guard<std::mutex> lock(mtx);
        (*counter)++;
    });
}

for (auto& t : threads) t.join();
std::cout << *counter; // 10
}

```

Rust:

```

use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("{}", *counter.lock().unwrap()); // 10
}

```

RwLock (Read-Write Lock)

RwLock allows multiple readers or a single writer, providing better performance than Mutex when reads are more common than writes. Multiple threads can hold read locks simultaneously, but a write lock requires exclusive access. This is useful for data that's read frequently but updated rarely, like configuration or caches:

```

use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));

    // Multiple readers
    let data1 = Arc::clone(&data);
    let r1 = thread::spawn(move || {

```

(continues on next page)

(continued from previous page)

```

    let read = data1.read().unwrap();
    println!("Reader 1: {:?}", *read);
});

// Single writer
let data2 = Arc::clone(&data);
let w = thread::spawn(move || {
    let mut write = data2.write().unwrap();
    write.push(4);
});

r1.join().unwrap();
w.join().unwrap();
}

```

Channels (Message Passing)

Channels provide a way for threads to communicate by sending messages rather than sharing memory. Rust's standard library provides `mpsc` (multi-producer, single-consumer) channels. The sender can be cloned to allow multiple producers, but there's only one receiver. Channels transfer ownership of sent values, preventing data races by design. This is Rust's preferred concurrency model - "share memory by communicating" rather than "communicate by sharing memory":

C++:

```

// No standard channel, typically use condition variables
// or third-party libraries

```

Rust:

```

use std::sync::mpsc; // multi-producer, single-consumer
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        tx.send("hello").unwrap();
    });

    let msg = rx.recv().unwrap();
    println!("{}", msg);
}

```

Multiple Producers

Clone the sender to create multiple producers. Each clone can be moved to a different thread. The receiver iterates over messages until all senders are dropped. This pattern is useful for worker pools where multiple threads produce results that a single thread collects:

```

use std::sync::mpsc;
use std::thread;

```

(continues on next page)

(continued from previous page)

```
fn main() {
    let (tx, rx) = mpsc::channel();

    for i in 0..3 {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(i).unwrap();
        });
    }
    drop(tx); // drop original sender

    for msg in rx {
        println!("{}", msg);
    }
}
```

Send and Sync Traits

Rust uses marker traits to ensure thread safety at compile time. `Send` means a type can be transferred to another thread (ownership can cross thread boundaries). `Sync` means a type can be shared between threads (multiple threads can have references to it). Most types are automatically `Send` and `Sync`, but types like `Rc` (non-atomic reference counting) and `RefCell` (non-thread-safe interior mutability) are not:

```
use std::rc::Rc;
use std::sync::Arc;

// Rc is NOT Send or Sync (not thread-safe)
let rc = Rc::new(42);
// thread::spawn(move || { println!("{}", rc); }); // compile error!

// Arc IS Send and Sync
let arc = Arc::new(42);
thread::spawn(move || { println!("{}", arc); }); // OK

// Raw pointers are NOT Send or Sync
let ptr: *const i32 = &42;
// thread::spawn(move || { unsafe { println!("{}", *ptr); } }); // error!
```

Scoped Threads

Scoped threads (`thread::scope`) can borrow from the parent stack because they're guaranteed to complete before the scope exits. This eliminates the need for `Arc` when you just want to share read-only data with spawned threads. All threads spawned within the scope are automatically joined when the scope ends, making it impossible to forget to join:

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| {
```

(continues on next page)

(continued from previous page)

```

        // Can borrow data without move
        println!("{:?}", data);
    });

    s.spawn(|| {
        println!("{:?}", data);
    });
});
// All spawned threads joined here

println!("data still valid: {:?}", data);
}

```

Thread Pool Pattern

A thread pool maintains a set of worker threads that process jobs from a queue. This avoids the overhead of creating new threads for each task. The following example shows a basic implementation using channels - workers receive jobs through a channel and execute them. Production code would typically use a crate like `rayon` or `threadpool`:

```

use std::sync::{mpsc, Arc, Mutex};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

struct ThreadPool {
    workers: Vec<thread::JoinHandle<()>>,
    sender: mpsc::Sender<Job>,
}

impl ThreadPool {
    fn new(size: usize) -> Self {
        let (sender, receiver) = mpsc::channel();
        let receiver = Arc::new(Mutex::new(receiver));
        let mut workers = Vec::with_capacity(size);

        for _ in 0..size {
            let receiver = Arc::clone(&receiver);
            workers.push(thread::spawn(move || loop {
                let job = receiver.lock().unwrap().recv();
                match job {
                    Ok(job) => job(),
                    Err(_) => break,
                }
            }));
        }

        ThreadPool { workers, sender }
    }

    fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,

```

(continues on next page)

(continued from previous page)

```

{
    self.sender.send(Box::new(f)).unwrap();
}
}

```

Atomic Types

Atomic types provide lock-free thread-safe operations on primitive values. They're faster than mutexes for simple operations like counters because they use CPU atomic instructions rather than OS-level locks. The `Ordering` parameter specifies memory ordering guarantees - `SeqCst` (sequentially consistent) is the safest but slowest; `Relaxed` is fastest but provides minimal guarantees:

```

use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    let counter = AtomicUsize::new(0);

    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                counter.fetch_add(1, Ordering::SeqCst);
            });
        }
    });

    println!("{}", counter.load(Ordering::SeqCst)); // 10
}

```

See Also

- *Smart Pointers* - Arc and Mutex details
- *Closures* - Move closures for threads

1.8.15 Macros

Table of Contents

- *Macros*
 - *Built-in Attributes*
 - *Derive Macros*
 - *Declarative Macros (macro_rules!)*
 - *Procedural Macros*
 - *Comparison Summary*
 - *See Also*

Rust has two kinds of macros: declarative macros (`macro_rules!`) for pattern-based code generation, and procedural

macros for AST-level transformations at compile time. Both are more powerful and safer than C/C++ preprocessor macros.

Built-in Attributes

Source

src/rust/proc_macros

Attributes are metadata applied to items using `#[...]`. They control compilation, generate code, or suppress warnings.

C++:

```
// Conditional compilation
#ifdef _WIN32
void platform() { std::cout << "windows"; }
#else
void platform() { std::cout << "other"; }
#endif

// Suppress warnings
#pragma warning(disable: 4101) // MSVC
[[maybe_unused]] int x = 5;    // C++17

// Inline hint
inline int add(int a, int b) { return a + b; }
```

Rust:

```
// Conditional compilation
#[cfg(target_os = "windows")]
fn platform() -> &'static str { "windows" }

#[cfg(not(target_os = "windows"))]
fn platform() -> &'static str { "other" }

// Suppress warnings
#[allow(dead_code)]
fn unused_function() -> i32 { 42 }

// Inline hint
#[inline]
fn add(a: i32, b: i32) -> i32 { a + b }
```

Common built-in attributes:

```
#[derive(Debug, Clone, PartialEq)] // auto-generate trait impls
#[cfg(test)]                       // only compile in test builds
#[test]                            // mark as test function
#[allow(unused_variables)]          // suppress warning
#[inline]                          // inline hint
#[must_use]                        // warn if return value is ignored
```

Derive Macros

`#[derive]` auto-generates trait implementations. This has no C++ equivalent — C++ requires manually writing operators, copy constructors, etc.

C++ (manual implementation):

```
#include <iostream>

struct Point {
    double x, y;

    // Must manually implement operator==
    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }

    // Must manually implement operator<< for printing
    friend std::ostream& operator<<(std::ostream& os, const Point& p) {
        return os << "Point { x: " << p.x << ", y: " << p.y << " }";
    }

    // Copy constructor/assignment generated by default in C++
};
```

Rust (auto-generated with derive):

```
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

fn main() {
    let p = Point { x: 1.0, y: 2.0 };
    let p2 = p.clone();           // Clone
    println!("{:?}", p);          // Debug
    assert_eq!(p, p2);            // PartialEq
}
```

Common derivable traits:

```
#[derive(
    Debug,           // {:?} formatting
    Clone,           // .clone() method
    Copy,            // implicit copy (like C++ trivial types)
    PartialEq,       // == and !=
    Eq,              // stricter equality (no NaN issues)
    Hash,            // for use in HashMap keys
    Default,         // Default::default() with zero/empty values
)]
```

Declarative Macros (macro_rules!)

Source

src/rust/proc_macros

Declarative macros use pattern matching to generate code. Unlike C++ #define, they are hygienic (no name collisions) and type-aware.

C++ (preprocessor macros):

```
// Simple macro - text substitution, no type safety
#define SAY_HELLO() std::cout << "Hello!" << std::endl

// Macro with argument - prone to double evaluation bugs
#define MAX(a, b) ((a) > (b) ? (a) : (b))

// Variadic macro
#define LOG(fmt, ...) printf(fmt, __VA_ARGS__)
```

Rust (declarative macros):

```
// Simple macro
macro_rules! say_hello {
    () => {
        println!("Hello!");
    };
    ($name:expr) => {
        println!("Hello, {}!", $name);
    };
}

say_hello!();           // Hello!
say_hello!("Rust");     // Hello, Rust!
```

Repetition patterns — no C++ equivalent:

```
// $(...),* matches zero or more comma-separated items
macro_rules! vec_of_strings {
    ($($x:expr),* $(,)? ) => {
        vec![$($x.to_string()),*]
    };
}

let v = vec_of_strings!["a", "b", "c"];
// expands to: vec!["a".to_string(), "b".to_string(), "c".to_string()]
```

Generating functions with macros:

```
macro_rules! make_adder {
    ($name:ident, $t:ty) => {
        fn $name(a: $t, b: $t) -> $t {
            a + b
        }
    };
}
```

(continues on next page)

(continued from previous page)

```
make_adder!(add_i32, i32);
make_adder!(add_f64, f64);

assert_eq!(add_i32(1, 2), 3);
assert_eq!(add_f64(1.0, 2.0), 4.0);
```

Macro fragment types:

```
// $x:expr - expression (1 + 2, foo(), "hello")
// $x:ident - identifier (variable/function name)
// $x:ty - type (i32, String, Vec<u8>)
// $x:pat - pattern (Some(x), (a, b))
// $x:stmt - statement (let x = 1;)
// $x:block - block ({ ... })
// $x:tt - single token tree (any token)
```

Procedural Macros

Procedural macros operate on the token stream (AST) at compile time. They are more powerful than `macro_rules!` but must be defined in a **separate crate** with `proc-macro = true`.

There are three types:

```
// 1. Attribute macros - #[something]
//   Transforms the annotated item
#[proc_macro_attribute]
pub fn my_attr(args: TokenStream, item: TokenStream) -> TokenStream { ... }
// Usage: #[my_attr]

// 2. Derive macros - #[derive(Something)]
//   Adds code alongside the annotated item
#[proc_macro_derive(MyTrait)]
pub fn my_derive(input: TokenStream) -> TokenStream { ... }
// Usage: #[derive(MyTrait)]

// 3. Function-like macros - something!(...)
//   Called like a function
#[proc_macro]
pub fn sql(input: TokenStream) -> TokenStream { ... }
// Usage: sql!(SELECT * FROM users)
```

Crate setup for procedural macros:

```
# Cargo.toml for a proc macro crate
[lib]
proc-macro = true

[dependencies]
syn = "2"           # parse Rust syntax
quote = "1"         # generate Rust code
proc-macro2 = "1"   # TokenStream utilities
```

Real-world example — `#[tokio::main]` expands `async main`:

```
// What you write:
#[tokio::main]
async fn main() {
    do_stuff().await;
}

// What it expands to:
fn main() {
    tokio::runtime::Runtime::new()
        .unwrap()
        .block_on(async {
            do_stuff().await;
        });
}
```

C++ has no equivalent to procedural macros. The C preprocessor does text substitution, while Rust proc macros parse and transform actual syntax trees at compile time.

Comparison Summary

Feature	Rust	C++
Simple macros	<code>macro_rules!</code> (hygienic, type-aware)	<code>#define</code> (text substitution)
Conditional compilation	<code>#[cfg(...)]</code>	<code>#ifdef</code> / <code>#if</code>
Auto-generate code	<code>#[derive(Trait)]</code>	No equivalent
AST transformation	Procedural macros	No equivalent
Suppress warnings	<code>#[allow(...)]</code>	<code>#pragma</code> / <code>[[maybe_unused]]</code>
Inline hint	<code>#[inline]</code>	<code>inline</code> keyword

See Also

- [Rust Basics](#) - Variables, mutability, underscore placeholder
- [Traits and Generics](#) - Traits that can be derived

1.8.16 Modules

Table of Contents

- *Modules*
 - *Module Comparison*
 - *Defining Modules*
 - *File-Based Modules*
 - *Visibility (pub)*
 - * *Struct Field Visibility*
 - *The use Keyword*
 - * *use Paths*

- *Crate Structure*
- *Prelude Pattern*
- *External Crates*
- *Conditional Compilation*
- *See Also*

Source

`src/rust/modules`

Rust's module system organizes code into hierarchical namespaces. Unlike C++ headers, Rust modules are part of the language with explicit visibility rules.

Module Comparison

C++	Rust
<code>#include</code>	<code>mod / use</code>
<code>namespace</code>	<code>mod</code>
Header files (.h)	Module files (.rs)
<code>public:</code>	<code>pub</code>
<code>private:</code>	(default, no keyword)
<code>using namespace</code>	<code>use</code>

Defining Modules

C++:

```
// math.h
namespace math {
    int add(int a, int b);

    namespace utils {
        int square(int x);
    }
}

// math.cpp
namespace math {
    int add(int a, int b) { return a + b; }

    namespace utils {
        int square(int x) { return x * x; }
    }
}
```

Rust (inline modules):

```
mod math {
    pub fn add(a: i32, b: i32) -> i32 {
        a + b
    }
}
```

(continues on next page)

(continued from previous page)

```

    }

    pub mod utils {
        pub fn square(x: i32) -> i32 {
            x * x
        }
    }
}

fn main() {
    let sum = math::add(1, 2);
    let sq = math::utils::square(3);
}

```

File-Based Modules

Rust maps modules to files:

```

src/
├── main.rs
├── math.rs          # mod math
└── math/
    └── utils.rs     # mod math::utils

```

src/main.rs:

```

mod math; // loads math.rs or math/mod.rs

fn main() {
    let sum = math::add(1, 2);
    let sq = math::utils::square(3);
}

```

src/math.rs:

```

pub mod utils; // loads math/utils.rs

pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

```

src/math/utils.rs:

```

pub fn square(x: i32) -> i32 {
    x * x
}

```

Visibility (pub)

```

mod outer {
    pub fn public_fn() {} // visible outside module
    fn private_fn() {}   // only visible in this module
}

```

(continues on next page)

(continued from previous page)

```

pub mod inner {
    pub fn inner_public() {}
    fn inner_private() {}

    // pub(super) - visible to parent module
    pub(super) fn parent_only() {}

    // pub(crate) - visible within crate
    pub(crate) fn crate_only() {}
}

fn main() {
    outer::public_fn();           // OK
    // outer::private_fn();       // error: private
    outer::inner::inner_public(); // OK
    // outer::inner::parent_only(); // error: not visible here
}

```

Struct Field Visibility

```

mod shapes {
    pub struct Rectangle {
        pub width: u32,    // public field
        height: u32,       // private field
    }

    impl Rectangle {
        pub fn new(width: u32, height: u32) -> Self {
            Rectangle { width, height }
        }

        pub fn area(&self) -> u32 {
            self.width * self.height
        }
    }
}

fn main() {
    let rect = shapes::Rectangle::new(10, 20);
    println!("{}", rect.width); // OK
    // println!("{}", rect.height); // error: private
}

```

The use Keyword

C++:

```

using namespace std;
using std::vector;
namespace fs = std::filesystem;

```

Rust:

```
// Import single item
use std::collections::HashMap;

// Import multiple items
use std::collections::{HashMap, HashSet};

// Import all public items (glob)
use std::collections::*;

// Rename import
use std::collections::HashMap as Map;

// Re-export
pub use std::collections::HashMap;
```

use Paths

```
mod foo {
    pub mod bar {
        pub fn baz() {}
    }
}

// Absolute path from crate root
use crate::foo::bar::baz;

// Relative path
use self::foo::bar::baz;

// Parent module
use super::something;

// External crate
use std::io::Read;
```

Crate Structure

```
my_crate/
├── Cargo.toml
├── src/
│   ├── lib.rs      # crate root for library
│   ├── main.rs     # crate root for binary
│   ├── module_a.rs
│   └── module_b/
│       ├── mod.rs  # module_b root
│       └── sub.rs  # module_b::sub
```

src/lib.rs:

```
pub mod module_a;
pub mod module_b;
```

(continues on next page)

(continued from previous page)

```
// Re-export for convenient access
pub use module_a::important_fn;
```

Prelude Pattern

```
// src/lib.rs
pub mod prelude {
    pub use crate::module_a::TypeA;
    pub use crate::module_b::TypeB;
    pub use crate::traits::*;
}

// Users can import common items easily:
// use my_crate::prelude::*;
```

External Crates

Cargo.toml:

```
[dependencies]
serde = "1.0"
tokio = { version = "1", features = ["full"] }
```

src/main.rs:

```
use serde::{Serialize, Deserialize};
use tokio::fs::File;

#[derive(Serialize, Deserialize)]
struct Config {
    name: String,
}
```

Conditional Compilation

```
// Platform-specific modules
#[cfg(target_os = "linux")]
mod linux;

#[cfg(target_os = "windows")]
mod windows;

// Feature-gated modules
#[cfg(feature = "advanced")]
pub mod advanced;

// Test module
#[cfg(test)]
mod tests {
    use super::*;
```

(continues on next page)

(continued from previous page)

```
#[test]
fn test_something() {
    assert!(true);
}
```

See Also

- *Traits and Generics* - Trait visibility
- *Error Handling* - Error types across modules

1.8.17 Rust FFI - Calling C++ from Rust

Table of Contents

- *Rust FFI - Calling C++ from Rust*
 - *Basic FFI Setup*
 - *Calling Simple Functions*
 - *Passing Arrays and Pointers*
 - *String Handling*
 - *Passing Structs*
 - *Type Mapping Reference*
 - *Bindgen for Automatic Bindings*
 - *See Also*

Rust can call C and C++ code through its Foreign Function Interface (FFI). Since C++ has no stable ABI, C++ functions must be exposed with `extern "C"` linkage to be callable from Rust. This chapter covers the fundamentals of binding C++ libraries, handling different data types across the FFI boundary, and creating safe Rust wrappers.

The key challenge in FFI is that Rust's safety guarantees don't extend across the boundary - all FFI calls are inherently unsafe. The common pattern is to write thin unsafe FFI declarations, then wrap them in safe Rust functions that enforce proper usage.

Basic FFI Setup

Source

`src/rust/ffi`

Setting up Rust to call C++ code requires three components working together: the C++ source code with C-compatible function signatures, a build script that compiles and links the C++ code into your Rust project, and Rust declarations that tell the compiler about the foreign functions. The `cc` crate handles the compilation complexity, automatically detecting the system's C++ compiler and configuring the build correctly.

Project structure:


```
ffi/
├─ Cargo.toml      # build-dependencies = { cc = "1.0" }
├─ build.rs        # Compiles cpp-lib.cc
├─ cpp-lib.cc      # C++ library with extern "C"
└─ main.rs         # Rust FFI declarations and wrappers
```

Cargo.toml:

The `cc` crate is specified as a build dependency since it's only needed during compilation, not at runtime. This keeps your final binary free of unnecessary dependencies.

```
[package]
name = "ffi"
version = "0.1.0"
edition = "2021"

[build-dependencies]
cc = "1.0"
```

build.rs:

The build script runs before your Rust code compiles. It invokes the `cc` crate to compile the C++ source file into a static library, which Cargo then automatically links into your final executable. The `.cpp(true)` flag tells `cc` to use the C++ compiler instead of the C compiler.

```
fn main() {
    cc::Build::new()
        .cpp(true)           // Compile as C++
        .file("cpp-lib.cc")
        .compile("cpp_lib"); // Output library name
}
```

Calling Simple Functions

The simplest FFI case involves functions with primitive types like integers and floats. These types have identical memory representations in both Rust and C++, so no conversion is needed. The `extern "C"` block in Rust declares the function signature, and the `unsafe` block is required because the compiler cannot verify the C++ implementation is correct.

C++ (cpp-lib.cc):

```
extern "C" {
    int32_t cpp_add(int32_t a, int32_t b) {
        return a + b;
    }
}
```

Rust:

```
extern "C" {
    fn cpp_add(a: i32, b: i32) -> i32;
}

// Safe wrapper
pub fn add(a: i32, b: i32) -> i32 {
```

(continues on next page)

(continued from previous page)

```

    unsafe { cpp_add(a, b) }
}

fn main() {
    println!("Result: {}", add(10, 20)); // 30
}

```

C++ equivalent (calling C from C++):

```

// In C++, you'd use extern "C" to call C functions
extern "C" int c_function(int x);

int main() {
    int result = c_function(42);
}

```

Passing Arrays and Pointers

When passing arrays across the FFI boundary, C and C++ represent them as raw pointers with a separate length parameter. Rust's slice type (`&[T]` or `&mut [T]`) combines the pointer and length into a single fat pointer, but this representation isn't compatible with C. The safe wrapper pattern extracts the raw pointer and length from the slice, passes them to the C++ function, and ensures the slice remains valid for the duration of the call.

C++:

```

extern "C" {
    void cpp_fill_array(int32_t* arr, size_t len, int32_t value) {
        for (size_t i = 0; i < len; ++i) {
            arr[i] = value;
        }
    }
}

```

Rust:

```

extern "C" {
    fn cpp_fill_array(arr: *mut i32, len: usize, value: i32);
}

// Safe wrapper using slices
pub fn fill_array(arr: &mut [i32], value: i32) {
    unsafe {
        cpp_fill_array(arr.as_mut_ptr(), arr.len(), value)
    }
}

fn main() {
    let mut arr = [0i32; 5];
    fill_array(&mut arr, 42);
    println!("{:?}", arr); // [42, 42, 42, 42, 42]
}

```

C++ equivalent:

```
#include <span> // C++20

void fill_array(std::span<int32_t> arr, int32_t value) {
    for (auto& x : arr) x = value;
}
```

String Handling

Strings are one of the trickiest types to pass across FFI boundaries because Rust and C use fundamentally different string representations. Rust's `String` is a UTF-8 encoded, length-prefixed, heap-allocated buffer without a null terminator. C strings are null-terminated byte arrays with no length field. The `CString` type creates an owned, null-terminated string suitable for passing to C, while `CStr` provides a borrowed view of a C string for reading data returned from C functions.

C++:

```
extern "C" {
    // Returns heap-allocated string - caller must free
    char* cpp_create_greeting(const char* name) {
        const char* prefix = "Hello, ";
        const char* suffix = "!";
        size_t len = strlen(prefix) + strlen(name) + strlen(suffix) + 1;
        char* result = new char[len];
        strcpy(result, prefix);
        strcat(result, name);
        strcat(result, suffix);
        return result;
    }

    void cpp_free_string(char* s) {
        delete[] s;
    }
}
```

Rust:

```
use std::ffi::{CStr, CString};
use std::os::raw::c_char;

extern "C" {
    fn cpp_create_greeting(name: *const c_char) -> *mut c_char;
    fn cpp_free_string(s: *mut c_char);
}

pub fn create_greeting(name: &str) -> String {
    // Convert Rust &str to C string
    let c_name = CString::new(name).expect("CString::new failed");

    unsafe {
        let ptr = cpp_create_greeting(c_name.as_ptr());
        // Convert C string back to Rust String
        let result = CStr::from_ptr(ptr).to_string_lossy().into_owned();
        // Free the C++ allocated memory
    }
}
```

(continues on next page)

(continued from previous page)

```

        cpp_free_string(ptr);
        result
    }
}

```

Key types:

- CString - Owned, null-terminated string for passing to C. Allocates memory and appends a null byte.
- CStr - Borrowed reference to a null-terminated string from C. Zero-cost wrapper around a `*const c_char`.
- c_char - Platform-specific C char type (usually `i8` on most platforms).

Passing Structs

Structs can be shared between Rust and C++ when they have compatible memory layouts. By default, Rust is free to reorder struct fields and add padding for optimization. The `#[repr(C)]` attribute forces Rust to use the same field ordering and alignment rules as C, making the struct binary-compatible. This is essential for any struct that crosses the FFI boundary, whether passed by value or by pointer.

C++:

```

extern "C" {
    struct Point {
        double x;
        double y;
    };

    double cpp_distance(const Point* p1, const Point* p2) {
        double dx = p2->x - p1->x;
        double dy = p2->y - p1->y;
        return sqrt(dx * dx + dy * dy);
    }

    Point cpp_midpoint(const Point* p1, const Point* p2) {
        return Point{(p1->x + p2->x) / 2.0, (p1->y + p2->y) / 2.0};
    }
}

```

Rust:

```

#[repr(C)] // Use C-compatible memory layout
#[derive(Debug, Clone, Copy)]
pub struct Point {
    pub x: f64,
    pub y: f64,
}

extern "C" {
    fn cpp_distance(p1: *const Point, p2: *const Point) -> f64;
    fn cpp_midpoint(p1: *const Point, p2: *const Point) -> Point;
}

pub fn distance(p1: &Point, p2: &Point) -> f64 {
    unsafe { cpp_distance(p1, p2) }
}

```

(continues on next page)

(continued from previous page)

```

}

pub fn midpoint(p1: &Point, p2: &Point) -> Point {
    unsafe { cpp_midpoint(p1, p2) }
}

fn main() {
    let p1 = Point { x: 0.0, y: 0.0 };
    let p2 = Point { x: 3.0, y: 4.0 };
    println!("Distance: {}", distance(&p1, &p2)); // 5.0
    println!("Midpoint: {:?}", midpoint(&p1, &p2)); // Point { x: 1.5, y: 2.0 }
}

```

Type Mapping Reference

Common type mappings between Rust and C/C++:

Rust	C/C++	Notes
i8, i16, i32, i64	int8_t, int16_t, int32_t, int64_t	Fixed-size integers
u8, u16, u32, u64	uint8_t, uint16_t, uint32_t, uint64_t	Fixed-size unsigned
f32, f64	float, double	Floating point
bool	bool	Boolean (1 byte)
usize	size_t	Pointer-sized unsigned
isize	ptrdiff_t	Pointer-sized signed
const T	const T	Immutable pointer
mut T	T	Mutable pointer
c_char	char	Platform-specific char
c_void	void	Opaque type

Bindgen for Automatic Bindings

For large C/C++ libraries with hundreds of functions and types, manually writing FFI declarations is tedious and error-prone. The `bindgen` tool solves this by parsing C/C++ header files and automatically generating the corresponding Rust `extern` blocks, struct definitions, and type aliases. This ensures your Rust declarations always match the actual C++ signatures, eliminating a common source of subtle bugs.

Cargo.toml:

```

[build-dependencies]
bindgen = "0.69"
cc = "1.0"

```

build.rs with bindgen:

The build script first compiles the C++ library, then runs `bindgen` to parse the header file and generate Rust bindings. The generated code is written to the `OUT_DIR`, a Cargo-managed directory for build artifacts, and included in your Rust code at compile time.

```

use std::env;
use std::path::PathBuf;

```

(continues on next page)

(continued from previous page)

```
fn main() {
    // Compile C++ library
    cc::Build::new()
        .cpp(true)
        .file("cpp-lib.cc")
        .compile("cpp_lib");

    // Generate bindings from header
    let bindings = bindgen::Builder::default()
        .header("cpp_lib.h")
        .parse_callbacks(Box::new(bindgen::CargoCallbacks::new()))
        .generate()
        .expect("Unable to generate bindings");

    let out_path = PathBuf::from(env::var("OUT_DIR").unwrap());
    bindings
        .write_to_file(out_path.join("bindings.rs"))
        .expect("Couldn't write bindings");
}
```

Using generated bindings:

```
// Include the generated bindings
include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
```

See Also

- [The Rustonomicon - FFI](#)
- [bindgen User Guide](#)
- [cc crate documentation](#)

1.9 Blog

Deep dives into systems programming, GPU communication, and high-performance computing topics beyond the typical cheatsheet format. These posts explore real-world implementations, covering everything from RDMA networking and GPUDirect to distributed training infrastructure. Each article walks through the underlying architecture and provides practical insights for building low-level, performance-critical systems.

1.9.1 Building NVSHMEM from Scratch: GPU-Initiated Networking

Table of Contents

- *Building NVSHMEM from Scratch: GPU-Initiated Networking*
 - *Abstract*
 - *Introduction*
 - *Fabric: RDMA Transport with libfabric and AWS EFA*

- * *libfabric Object Hierarchy*
- * *Querying EFA Devices with fi_info*
- *Topology: GPU-NIC Affinity and NUMA-Aware Placement*
 - * *Detecting Topology with hwloc*
- *Bootstrap: Out-of-Band Connection Setup*
- *Communication: RDMA Verbs and Data Transfer Patterns*
 - * *Two-Sided Communication (SEND/RECV)*
 - * *One-Sided Communication (RDMA WRITE)*
 - * *Why NVSHMEM Uses One-Sided Semantics*
- *GPU-CPU Queue: Low-Latency Signaling for Proxy Threads*
- *Symmetric Memory*
- *GPUDirect RDMA*
 - * *DMA-BUF: Exporting GPU Memory to the NIC*
- *CUDA IPC*
 - * *IPC Handle Exchange*
 - * *Integration with Symmetric Memory*
- *Simple NVSHMEM Implementation*
 - * *API Surface*
 - * *Example*
- *Conclusion*
- *References*

Abstract

GPU-to-GPU communication is critical for large language model (LLM) training and inference, as modern models cannot fit on a single GPU or even a single node. This necessitates partitioning model parameters across GPUs and using collective communication to aggregate results. NCCL is a widely-used collective library that achieves high throughput over RDMA fabrics such as InfiniBand and AWS Elastic Fabric Adapter (EFA). Recently, GPU-Initiated Networking (GIN)³ has gained attention for its ability to fuse CUDA kernels with GPUDirect communication, reducing kernel launch overhead and improving overlap. DeepEP⁶ exemplifies this approach—a high-performance Mixture-of-Experts (MoE) layer dispatch/combine implementation that significantly reduces All-to-All collective latency using NVSHMEM with InfiniBand GPUDirect Async (IBGDA). However, not all RDMA providers support IBGDA (at least as of late 2025). Instead, they rely on a “Proxy Thread” technique to achieve GIN. InfiniBand Reliable Connection (IBRC) uses this approach, as do similar implementations like UCCL⁴ and MSCCL++⁵. In this article, we present a systematic breakdown of how proxy thread solutions achieve GPU-initiated behavior over AWS EFA, and describe the design and implementation of a minimal NVSHMEM-like library.

³ Liu, S. et al., “GPU-Initiated Networking,” *arXiv preprint arXiv:2511.15076*, 2025.

⁶ DeepSeek-AI, “DeepEP: Expert Parallelism with GPU-Initiated Communication.” Available: <https://github.com/deepseek-ai/DeepEP>

⁴ UCCL Project, “UCCL: User-space Collective Communication Library.” Available: <https://github.com/uccl-project/uccl>

⁵ Microsoft, “MSCCL++: Multi-Scale Collective Communication Library.” Available: <https://github.com/microsoft/mscclpp>

Note

All source code and benchmarks are available at [Libefaxx](https://libfabric.org).

Introduction

The rapid scaling of LLMs has made efficient GPU-to-GPU communication a first-class concern in distributed systems. Libraries such as NCCL² abstract collective operations over RDMA fabrics, but emerging workloads—particularly MoE architectures—demand finer-grained, kernel-level control over communication. NVSHMEM provides a GPU-initiated programming model where CUDA kernels directly issue put/get operations, enabling overlap of computation and communication without returning control to the host.

This article targets practitioners who wish to understand the internals of NVSHMEM/OpenSHMEM by building one from scratch. We decompose the problem into the following components and describe each in detail.

First, we discuss the **RDMA transport layer**, where we use libfabric¹ to initialize fabric objects, create endpoints, and manage completion queues over AWS EFA. We then examine **hardware topology** discovery using hwloc, which enables NUMA-aware placement by mapping GPU-NIC affinity through the PCIe hierarchy — a critical step for minimizing cross-switch latency on multi-NIC instances. Next, we describe the **bootstrap** phase, in which peers exchange endpoint addresses and memory region keys through an out-of-band channel (e.g., MPI or TCP) before any RDMA data transfer can occur.

With the transport established, we cover two **communication patterns** supported by libfabric: two-sided SEND/RECV, which requires active participation from both peers, and one-sided RDMA WRITE, which allows a sender to write directly into remote registered memory without receiver involvement — the natural fit for NVSHMEM’s put/get semantics. We then present the **GPU-CPU queue**, a multi-producer single-consumer (MPSC) structure that enables GPU kernels to enqueue RDMA requests to a **proxy thread** — a CPU-side thread that dequeues and issues fabric operations on behalf of the GPU, bridging the gap when hardware does not support IBGDA.

Finally, we describe three remaining components that complete the design: **symmetric memory**, which provides a globally addressable memory space across all GPUs; **GPUDirect RDMA** via DMA-BUF, which enables zero-copy registration of GPU memory for direct NIC access; and **CUDA IPC**, which provides low-latency intra-node GPU-to-GPU data transfer through shared memory handles.

Fabric: RDMA Transport with libfabric and AWS EFA

To perform RDMA operations, applications typically use low-level libraries such as [libverbs](#) (for InfiniBand/RoCE) or [libfabric](#) (a higher-level, provider-agnostic fabric interface). Since this article targets AWS Elastic Fabric Adapter (EFA), we use [libfabric](#) — the recommended interface for EFA. The AWS EFA provider in libfabric handles the [Scalable Reliable Datagram \(SRD\)](#) protocol internally, so applications do not need to manage reliability or ordering at the transport layer.

libfabric Object Hierarchy

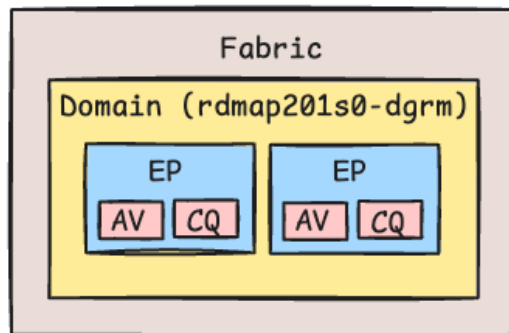
The diagram below illustrates the core libfabric object hierarchy used to set up RDMA communication over EFA:

- **Fabric** — represents the physical network (e.g., an EFA device).
- **Domain** — maps to a specific network interface, analogous to binding to an IP address. Each domain provides access to resources such as memory registration and address resolution.
- **Endpoint** — a communication channel, analogous to a socket. Each endpoint is associated with:
 - An **Address Vector (AV)** — a table that maps peer addresses for connectionless (datagram) communication.
 - A **Completion Queue (CQ)** — used to poll for RDMA operation completions (e.g., send/recv/write done).

² Punniyamurthy, K. et al., “Optimizing Distributed ML Communication with Collective Operations,” *arXiv preprint arXiv:2305.06942*, 2023.

¹ Le, Q., “Libfabric EFA Series,” 2024. Available: <https://le.qun.ch/en/blog/2024/12/25/libfabric-efa-0-intro/>

The full initialization sequence can be found in `efa.h`.



```
struct fi_info = fi_getinfo(efa)
fi_fabric(fabric)
fi_domain(domain) # similar to ip addr
fi_cq_open(av)
fi_av_open(av)
fi_endpoint(ep)   # similar to port
fi_ep_bind(av)
fi_ep_bind(cq)
fi_enable(ep)
```

Querying EFA Devices with `fi_info`

On AWS EC2 instances with EFA enabled (e.g., `p4d.24xlarge`, `p5.48xlarge`), available fabric providers can be queried using the `fi_info` utility:

```
$ fi_info -p efa

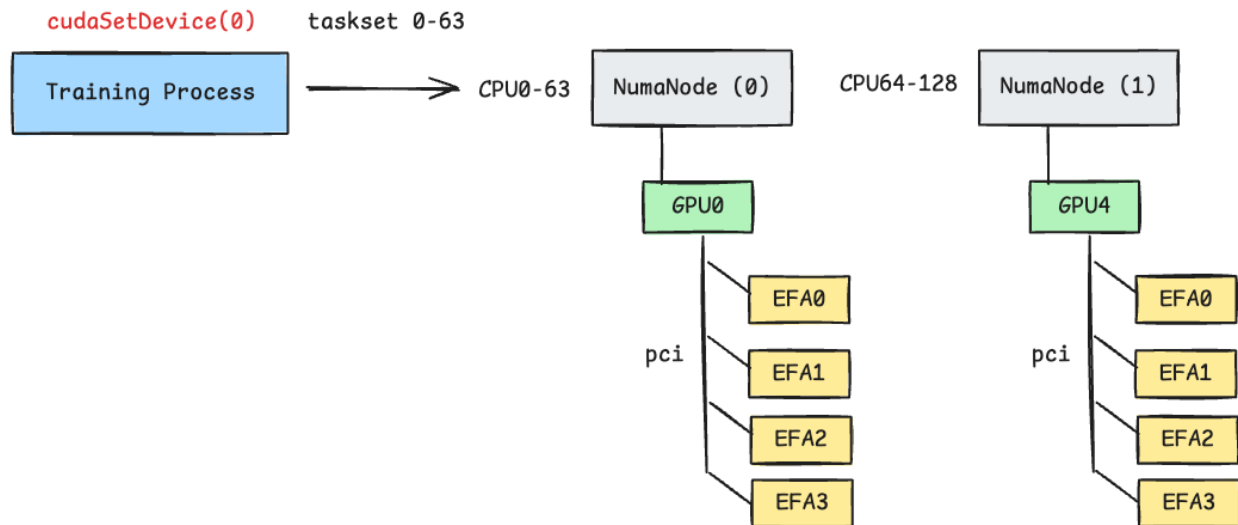
provider: efa
fabric: efa
domain: rdmap201s0-dgrm
version: 203.10
type: FI_EP_DGRAM
protocol: FI_PROTO_EFA
```

The output shows that the EFA provider exposes a datagram endpoint (`FI_EP_DGRAM`) using the `FI_PROTO_EFA` protocol. The domain field identifies the specific EFA device. On multi-NIC instances (e.g., `p5.48xlarge` with 32 EFA interfaces), `fi_info` lists multiple domains — one per NIC — which is important for topology-aware placement discussed in the next section.

Topology: GPU-NIC Affinity and NUMA-Aware Placement

Hardware topology awareness is essential for achieving optimal RDMA performance in multi-GPU systems. On instances like AWS `p5.48xlarge`, each GPU is physically closer to certain EFA NICs and CPU cores through the PCIe topology. Routing RDMA traffic through a topology-local NIC avoids costly cross-NUMA or cross-PCIe-switch transfers.

The diagram below illustrates this principle. If a process is bound to GPU 0, routing RDMA traffic through the EFA device on the same PCIe switch minimizes latency. Using a distant NIC (e.g., one closer to GPU 4) forces data to traverse additional PCIe hops, increasing transfer time.



Detecting Topology with hwloc

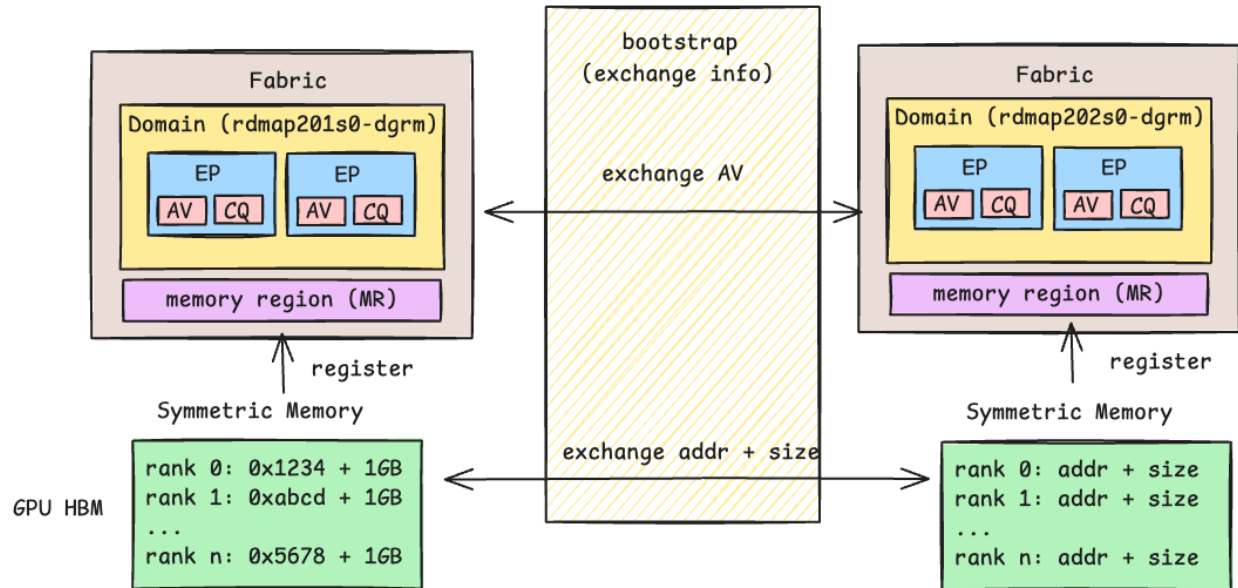
One approach to discovering hardware topology is to parse `/sys/bus/pci/devices` directly, but this is error-prone and difficult to maintain. A more robust approach is to use `hwloc` — a portable library for querying the hierarchical topology of CPUs, caches, NUMA nodes, and PCI devices. The programming pattern resembles a depth-first search (DFS) pre-order traversal over a tree data structure. Basic usage examples are available in the [hwloc cheat sheet](#). For a real-world example of detecting GPU-NIC affinity on AWS p5.48xlarge and using `taskset` to pin processes to topology-local CPU cores, see [affinity.h](#).

Bootstrap: Out-of-Band Connection Setup

Unlike traditional networking stacks where protocols like ARP handle address discovery automatically, RDMA requires an explicit out-of-band (OOB) exchange to set up connections. Before any RDMA data transfer can occur, peers must exchange endpoint addresses and memory region keys through a separate control channel — a process known as **bootstrapping**.

Common bootstrap methods in the RDMA ecosystem include:

- **MPI** — NCCL and NVSHMEM can use MPI collectives (e.g., `MPI_Allgather`) to distribute connection identifiers such as `nccl_id` across all ranks.
- **TCPStore** — PyTorch’s distributed runtime uses `TCPStore` as a key-value store to exchange connection information (e.g., rank addresses, NCCL IDs) between processes.



Once the RDMA connection is established and memory regions are registered, the OOB channel is no longer needed for data transfer. In our implementation, the symmetric memory layer uses `MPI_Allgather` to exchange remote RDMA addresses and memory region sizes. Further details are available in [fabric.h](#).

Communication: RDMA Verbs and Data Transfer Patterns

libfabric supports two primary communication patterns, each suited to different use cases.

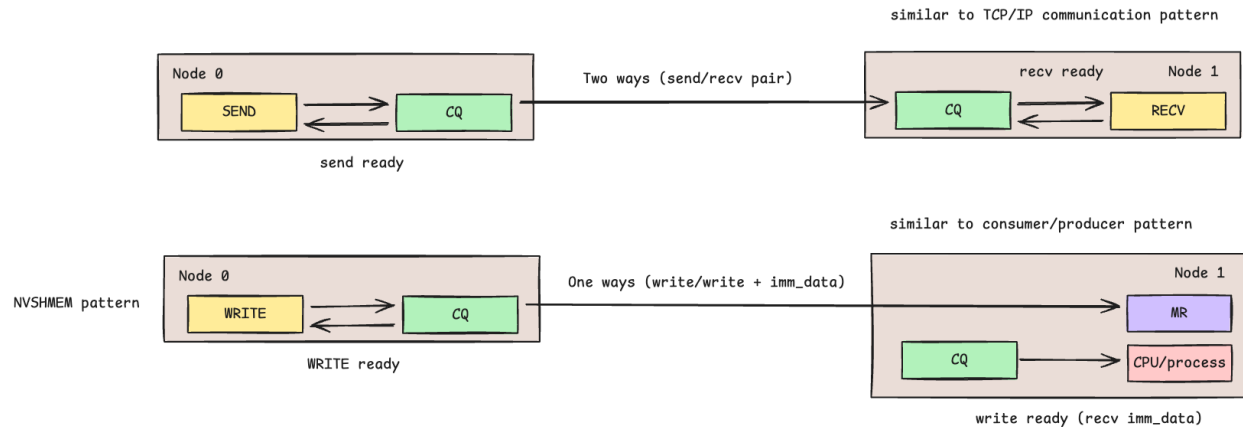
Two-Sided Communication (SEND/RECV)

This pattern resembles traditional TCP/IP socket communication. Both the sender and receiver must actively participate — the sender calls `fi_sendmsg` and the receiver calls `fi_recvmsg`. Each side's completion queue (CQ) signals when its respective operation completes. This is useful when the receiver needs to know exactly when data arrives and control where it lands.

One-Sided Communication (RDMA WRITE)

This pattern resembles a producer-consumer model with shared memory. The writer uses `fi_writemsg` to write directly into the remote node's registered memory region (MR) — the remote CPU is not involved in the data path. Only the writer's CQ signals completion; the remote side has no automatic notification that data arrived.

To notify the remote side, RDMA provides **write with immediate data** (`FI_REMOTE_CQ_DATA`). The writer attaches a small immediate value to the write operation. When the write completes, the remote CQ receives a completion event containing this immediate data, signaling that new data is available. This is commonly used as an “end-of-write” tag.



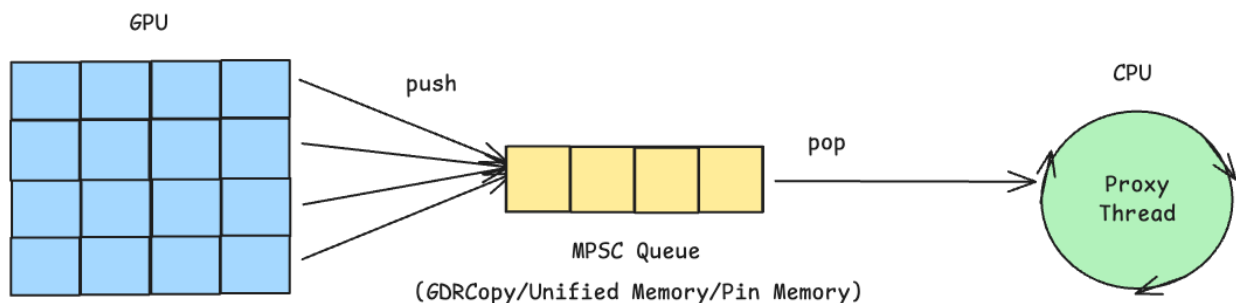
Implementation examples for both patterns are available in Libefaxx: [Send/Recv benchmark](#) and [Write benchmark](#).

Why NVSHMEM Uses One-Sided Semantics

The name “NVSHMEM” (and OpenSHMEM) might suggest it is merely a shared memory IPC library. However, NVSHMEM also supports inter-node communication over RDMA. The “SHMEM” terminology reflects the programming model: like shared memory IPC, the communication is one-sided — a producer writes to a remote address without the consumer explicitly receiving. RDMA one-sided write maps naturally to this model: the caller specifies a remote virtual address and offset, and the NIC performs a DMA transfer directly into remote memory. This is why one-sided RDMA is the foundation for NVSHMEM’s `nvshmem_put` / `nvshmem_get` APIs.

GPU-CPU Queue: Low-Latency Signaling for Proxy Threads

The proxy thread architecture relies on a GPU-CPU queue to coordinate between GPU kernels and the CPU thread that issues RDMA operations. Since GPU kernels run thousands of threads, this is typically implemented as a multi-producer, single-consumer (MPSC) queue — many GPU threads enqueue requests, while a single CPU proxy thread dequeues and processes them.



Several memory strategies can implement this queue, each with different trade-offs:

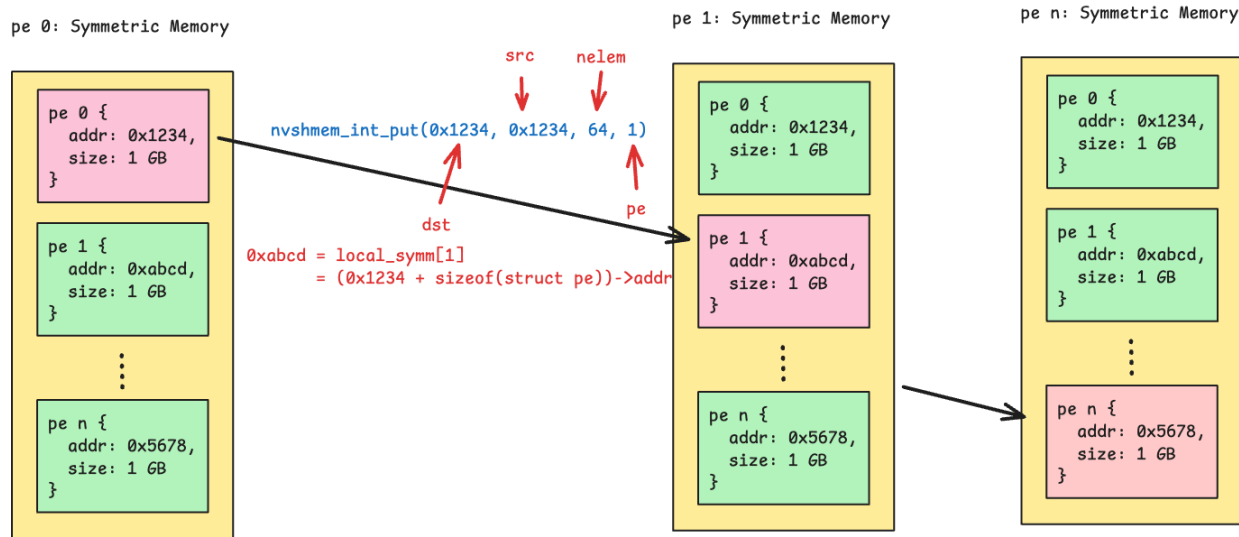
- **GDRCopy** — Maps GPU memory to CPU address space via PCIe BAR (Base Address Register). The CPU can directly read/write GPU memory without invoking CUDA APIs.
- **CUDA Unified Memory** — Automatically migrates pages between GPU and CPU on access. Simpler to program but incurs page fault overhead on first access.
- **Pinned (Page-Locked) Host Memory** — CPU memory allocated with `cudaHostAlloc` that GPUs can access directly via PCIe.

For benchmarks comparing these approaches, see the [Command Queue Implementation Comparison](#) in Libefaxx.

Symmetric Memory

In NVSHMEM's [Memory Model](#), symmetric memory refers to allocations that have the same name, type, and size on all PEs (Processing Elements). This uniformity is required because each PE must maintain RDMA metadata — remote virtual addresses and memory region sizes — for every other PE, so that one-sided operations can directly access remote memory without coordination.

The diagram below illustrates a straightforward implementation of the symmetric memory data structure. Each PE's local symmetric object holds both a block of its own allocated memory and the remote virtual addresses of all other PEs. When a shmem-like API such as `nvshmem_int_put` is called, the library looks up the target PE's remote address and issues an RDMA write directly into that region.



For implementation details, see [memory.h](#) in Libefaxx.

GPUDirect RDMA

Standard RDMA requires data to reside in registered host memory. When the source data lives on a GPU, a naive approach copies GPU memory to a pinned host buffer, registers it, and then issues the RDMA operation — adding a full device-to-host transfer to every communication. GPUDirect RDMA eliminates this copy by allowing the NIC to read from and write to GPU memory directly over PCIe, without staging through host memory.

DMA-BUF: Exporting GPU Memory to the NIC

The Linux kernel's [DMA-BUF](#) framework provides a standard mechanism for sharing memory buffers between devices. For GPUDirect RDMA, the workflow is:

1. Allocate GPU memory with `cudaMalloc`.
2. Export a DMA-BUF file descriptor using `cuMemGetHandleForAddressRange` with `CU_MEM_RANGE_HANDLE_TYPE_DMA_BUF_FD`.
3. Register the memory with libfabric using `fi_mr_regattr` with `FI_HMEM_CUDA` and the DMA-BUF descriptor in `fi_mr_dmabuf`.

Once registered, the NIC can perform RDMA read/write operations directly against GPU memory through GPU virtual address. The DMA-BUF file descriptor is closed after registration, and the memory region is deregistered when no longer needed.

For the full implementation, see [buffer.h](#) in Libefaxx.

CUDA IPC

RDMA handles inter-node communication, but GPUs within the same node are connected by NVLink — a high-bandwidth interconnect that provides up to 3600 Gbps on H100 GPUs, far exceeding what any network fabric can offer. CUDA IPC (Inter-Process Communication) allows processes to share GPU memory across process boundaries, enabling direct GPU-to-GPU transfers over NVLink without involving the CPU or NIC.

IPC Handle Exchange

The setup follows a pattern similar to RDMA bootstrapping:

1. Each rank calls `cudaIpcGetMemHandle` to export a handle for its GPU buffer.
2. Handles are exchanged across local ranks via `MPI_Allgather` (or any OOB channel).
3. Each rank calls `cudaIpcOpenMemHandle` on peer handles to obtain a device pointer that maps into the remote GPU's memory.

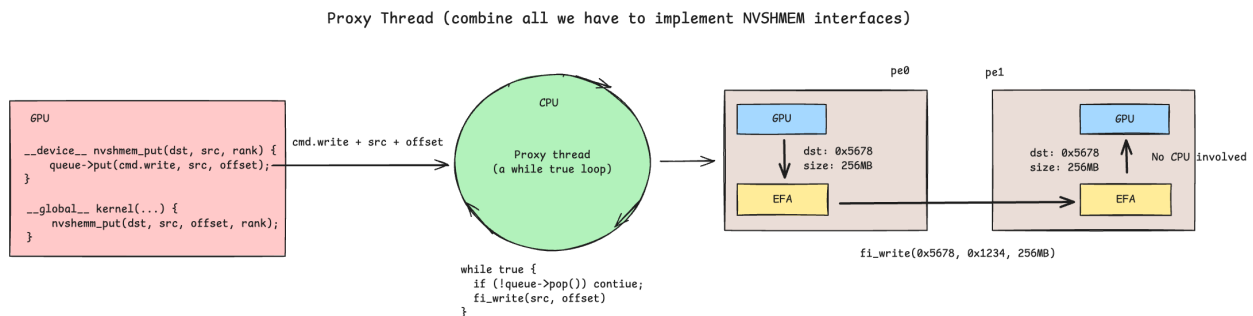
Once opened, a GPU kernel can write directly to a peer GPU's buffer using the mapped pointer — the transfer occurs over NVLink with no CPU involvement. A `__threadfence_system()` ensures writes are visible to the remote GPU. This access pattern — writing to a remote address without receiver participation — is identical to RDMA one-sided semantics, which makes CUDA IPC a natural fit for the symmetric memory abstraction.

Integration with Symmetric Memory

Because CUDA IPC and RDMA one-sided write share the same put/get programming model — both operate on a remote virtual address without receiver participation — the symmetric memory layer can store IPC pointers and RDMA remote addresses side by side in a single data structure. At runtime, the library checks whether the target PE resides on the same node. If so, it writes directly through the IPC pointer over NVLink; otherwise, it enqueues the request to the proxy thread for RDMA delivery. This routing is transparent to the caller, and the performance difference is substantial: NVLink IPC transfers can reach ~2971 Gbps (78% of H100 NVLink peak), as shown in the [NVLink IPC benchmark](#).

For implementation details, see [symmetric.h](#) in Libefaxx.

Simple NVSHMEM Implementation



With all the building blocks in place — RDMA transport, topology discovery, bootstrap, proxy thread, symmetric memory, GPUDirect RDMA, and CUDA IPC — we can assemble a minimal NVSHMEM-like API. The goal is to provide familiar `shmem_*` functions that CUDA kernels call directly, while the library transparently handles intra-node vs. inter-node routing underneath.

API Surface

The host-side API mirrors NVSHMEM's structure:

- `shmem_init` / `shmem_finalize` — bootstrap MPI, discover topology, create EFA endpoints, and establish connections.
- `shmem_malloc` / `shmem_free` — allocate symmetric memory with DMA-BUF registration, exchange RDMA keys via `MPI_Sendrecv`, and exchange CUDA IPC handles among local ranks.
- `shmem_my_pe` / `shmem_n_pes` — query the current PE index and total PE count.
- `shmem_barrier_all` — global barrier across all PEs.

On the device side, `shmem_int_p` (and other typed variants) perform a one-sided put from within a CUDA kernel. The implementation checks the target PE's IPC pointer: if non-null (same node), it writes directly over NVLink; otherwise, it writes to local symmetric memory, issues a `__threadfence_system`, and pushes a request to the proxy thread's MPSC queue for RDMA delivery. The blocking variant calls `shmem_quiet` to wait until the proxy thread confirms completion.

Example

The following example demonstrates a simple ring shift — each PE writes its rank to the next PE's symmetric buffer, equivalent to the [NVSHMEM example](#):

```
#include <shmem/shmem.cuh>

template <typename Ctx>
__global__ void simple_shift(Ctx ctx, int* target, int mype, int npes) {
    int peer = (mype + 1) % npes;
    shmem_int_p(ctx, target, mype, peer);
}

int main() {
    shmem_init();
    int mype = shmem_my_pe();
    int npes = shmem_n_pes();
    int* target = static_cast<int*>(shmem_malloc(sizeof(int)));
    auto ctx = shmem_ctx(target);

    simple_shift<<<1, 1>>>>(ctx, target, mype, npes);
    // ... run proxy thread for inter-node RDMA if needed ...

    shmem_barrier_all();
    shmem_free(target);
    shmem_finalize();
}
```

For the full API implementation, see [shmem.cuh](#). For the complete working example including proxy thread setup, see the [shmem experiment](#) in Libefaxx.

Conclusion

The goal of this article was to learn how NVSHMEM-like libraries work by building one from scratch. Recent Mixture-of-Experts (MoE) implementations such as DeepEP have shown that high-performance GPU-initiated communication can significantly improve dispatch/combine latency beyond what naive All-to-All collectives achieve. To better understand the technologies behind these projects, I conducted a series of small experiments covering RDMA transport with

libfabric, NUMA-aware topology discovery, out-of-band bootstrapping, one-sided communication, MPSC queues for proxy threads, symmetric memory, GPUDirect RDMA via DMA-BUF, and CUDA IPC over NVLink.

I am grateful to the open-source projects referenced throughout this article — NVSHMEM, UCCL, MSCCL++, and DeepEP — whose designs and source code were invaluable in deepening my understanding of RDMA and CUDA communication. This article represents my current understanding, and I welcome any corrections or suggestions. For additional experiments and benchmark results, please refer to the [Libefaxx experiments](#).

References